

Curso Profesional de JasperReports 6.20.0 Community

Módulo 6 — Exportación

PRÁCTICAS

AUTOR: JAIME GALLO

JasperReports Library 6.20.0 Community · Jaspersoft Studio 6.20.0 Community Edition · Java 8 · Maven · SQLite · CSV · XML · JSON · Proyecto EditorialReports

Módulo 6 — Práctica de exportación

Proyecto acumulativo: **EditorialReports**. Cada checkpoint parte físicamente del anterior.

Parte A reproduce el trabajo manual/IDE que conduce al checkpoint. Partes B/C incrustan código real del repositorio. Parte D representa estructura, salidas y evidencia E2E.

Punto 6.1 — Exportación a PDF

Objetivos de aprendizaje

- Comprender el papel del exportador PDF y su ubicación en la biblioteca.
- Diferenciar los métodos simples de JasperExportManager de los exportadores avanzados.
- Configurar las propiedades del exportador mediante SimplePdfExporterConfiguration.
- Establecer metadatos del documento PDF (título, autor, palabras clave).
- Aplicar protección con contraseña y permisos al PDF generado.
- Documentar la exportación a PDF del proyecto EditorialReports.

Parte A — Práctica visual/IDE verificada

Paso 1: Abrir el generador heredado de M5

Acciones:

1. En Project Explorer, expandir M6/6.1/EditorialReportsJava/src.
2. Abrir `GeneradorInformeVentas.java`.
3. Confirmar que sigue compilando `subinforme_ventas_detalle.jrxml` e `informe_ventas.jrxml` y que llena un único JasperPrint documento.

Verificación visual: el editor muestra la lógica heredada y documento se crea antes de cualquier exportación.

Qué hace: completa la operación «Abrir el generador heredado de M5» en el checkpoint 6.1.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.1.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.1.

Analogía: es como configurar la prensa PDF antes de lanzar la tirada definitiva.

Paso 2: Sustituir la exportación PDF simple por el exportador avanzado

Acciones:

1. Añadir imports para `JRPdfExporter`, `SimpleExporterInput`, `SimpleOutputStreamExporterOutput` y `SimplePdfExporterConfiguration`.
2. Eliminar la llamada simple de exportación del informe de ventas si aún existiera.
3. Mantener los generadores heredados sin cambios.

Verificación visual: Problems no muestra imports sin resolver y la clase referencia `JRPdfExporter`.

Qué hace: completa la operación «Sustituir la exportación PDF simple por el exportador avanzado» en el checkpoint 6.1.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.1.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.1.

Analogía: es como configurar la prensa PDF antes de lanzar la tirada definitiva.

Paso 3: Declarar las dos rutas PDF

Acciones:

1. Mantener `String rutaPdf = "output/informe_ventas.pdf";`.
2. Añadir `String rutaPdfProtegido = "output/informe_ventas_protegido.pdf";`.
3. Conservar `new File("output").mkdirs();` antes de exportar.

Verificación visual: Source contiene las dos rutas exactamente con esos nombres.

Qué hace: completa la operación «Declarar las dos rutas PDF» en el checkpoint 6.1.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.1.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.1.

Analogía: es como configurar la prensa PDF antes de lanzar la tirada definitiva.

Paso 4: Crear el método exportarPdf**Acciones:**

1. Añadir `private static void exportarPdf(JasperPrint documento, String ruta) throws Exception`.
2. Crear dentro un `JRPdfExporter` y un `SimplePdfExporterConfiguration`.
3. No abrir una nueva conexión ni volver a ejecutar `fillReport`.

Verificación visual: el método recibe el `JasperPrint` ya llenado y una ruta de salida.

Qué hace: completa la operación «Crear el método exportarPdf» en el checkpoint 6.1.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.1.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.1.

Analogía: es como configurar la prensa PDF antes de lanzar la tirada definitiva.

Paso 5: Configurar metadatos y compresión**Acciones:**

1. Usar `setMetadataTitle`, `setMetadataAuthor`, `setMetadataSubject`, `setMetadataKeywords` y `setMetadataCreator`.
2. Añadir `setDisplayMetadataTitle(Boolean.TRUE)`.
3. Añadir `setCompressed(Boolean.TRUE)`.
4. No utilizar `setTitle`, `setAuthor` ni `setCharacterEncoding` sobre `SimplePdfExporterConfiguration`.

Verificación visual: Source muestra exactamente la API que compila con JasperReports 6.20.0.

Qué hace: completa la operación «Configurar metadatos y compresión» en el checkpoint 6.1.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.1.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.1.

Analogía: es como configurar la prensa PDF antes de lanzar la tirada definitiva.

Paso 6: Asignar configuración, entrada y salida

Acciones:

1. Llamar a `exportador.setConfiguration(configuracion)`.
2. Usar `new SimpleExporterInput(documento)` como entrada.
3. Usar `new SimpleOutputStreamExporterOutput(ruta)` como salida.
4. Finalizar con `exportador.exportReport()`.

Verificación visual: el método `exportarPdf` contiene las cuatro operaciones en ese orden lógico.

Qué hace: completa la operación «Asignar configuración, entrada y salida» en el checkpoint 6.1.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.1.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.1.

Analogía: es como configurar la prensa PDF antes de lanzar la tirada definitiva.

Paso 7: Crear el método `exportarPdfProtegido`

Acciones:

1. Añadir un segundo método que reciba `JasperPrint documento` y `String ruta`.
2. Crear un `JRPdfExporter` y una `SimplePdfExporterConfiguration` independientes.
3. Fijar el título Informe de Ventas Protegido - EditorialReports.

Verificación visual: existen dos métodos PDF separados, uno normal y otro de seguridad.

Qué hace: completa la operación «Crear el método `exportarPdfProtegido`» en el checkpoint 6.1.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.1.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.1.

Analogía: es como configurar la prensa PDF antes de lanzar la tirada definitiva.

Paso 8: Configurar cifrado, contraseñas y permisos

Acciones:

1. Activar `setEncrypted(Boolean.TRUE)`.
2. Configurar `setUserPassword("editorial2026")`.
3. Configurar `setOwnerPassword("editorial-admin")`.
4. Aplicar `setAllowedPermissionsHint("PRINTING|COPY|SCREENREADERS")`.

Verificación visual: la configuración protegida contiene las cuatro propiedades y no usa las APIs obsoletas del origen.

Qué hace: completa la operación «Configurar cifrado, contraseñas y permisos» en el checkpoint 6.1.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.1.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.1.

Analogía: es como configurar la prensa PDF antes de lanzar la tirada definitiva.

Paso 9: Invocar ambas exportaciones sobre el mismo JasperPrint

Acciones:

1. Después de `fillReport`, llamar a `exportarPdf(documento, rutaPdf)`.
2. A continuación llamar a `exportarPdfProtegido(documento, rutaPdfProtegido)`.
3. Mantener la misma conexión y el mismo mapa de parámetros heredado.

Verificación visual: las dos llamadas están dentro del mismo `try` (`Connection conexion...`).

Qué hace: completa la operación «Invocar ambas exportaciones sobre el mismo JasperPrint» en el checkpoint 6.1.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.1.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.1.

Analogía: es como configurar la prensa PDF antes de lanzar la tirada definitiva.

Paso 10: Compilar el proyecto Java

Acciones:

1. Guardar la clase.
2. Ejecutar `mvn clean package` desde `EditorialReportsJava` o Build Project en el IDE.
3. Revisar Problems/Console y corregir cualquier `cannot find symbol` antes de continuar.

Verificación visual: la compilación termina sin errores.

Qué hace: completa la operación «Compilar el proyecto Java» en el checkpoint 6.1.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.1.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.1.

Analogía: es como configurar la prensa PDF antes de lanzar la tirada definitiva.

Paso 11: Ejecutar y comprobar el PDF normal

Acciones:

1. Ejecutar `GeneradorInformeVentas` como Java Application desde `EditorialReports`.
2. Abrir `output/informe_ventas.pdf`.
3. Comprobar que conserva las 6 páginas del cierre M5.
4. Revisar en Propiedades los metadatos de título, autor y creador.

Verificación visual: el PDF normal se abre sin contraseña y conserva el contenido completo.

Qué hace: completa la operación «Ejecutar y comprobar el PDF normal» en el checkpoint 6.1.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.1.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.1.

Analogía: es como configurar la prensa PDF antes de lanzar la tirada definitiva.

Paso 12: Comprobar el PDF protegido

Acciones:

1. Abrir `output/informe_ventas_protegido.pdf`.
2. Introducir la contraseña `editorial2026`.
3. Comprobar que el documento se abre y mantiene el mismo contenido del informe.

Verificación visual: el lector solicita la contraseña documentada y el archivo se abre con ella.

Qué hace: completa la operación «Comprobar el PDF protegido» en el checkpoint 6.1.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.1.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.1.

Analogía: es como configurar la prensa PDF antes de lanzar la tirada definitiva.

Paso 13: Documentar la exportación PDF

Acciones:

1. Crear/abrir `EditorialReports/EXPORTACION_PDF.md`.
2. Registrar API avanzada, metadatos, compresión, cifrado, contraseñas y permisos.
3. Registrar las dos rutas de salida y la corrección de las APIs que aparecían en la fuente original.

Verificación visual: Project Explorer muestra `EXPORTACION_PDF.md` y su contenido coincide con el código.

Qué hace: completa la operación «Documentar la exportación PDF» en el checkpoint 6.1.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.1.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.1.

Analogía: es como configurar la prensa PDF antes de lanzar la tirada definitiva.

Parte B — JRXML/JRTX completo explicado línea por línea

En M6 el diseño no cambia: estos tres archivos deben permanecer byte a byte iguales a M5/5.6.

Informe maestro heredado y ejecutable

```
<?xml version="1.0" encoding="UTF-8"?>
<jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/
jasperreport.xsd"
  name="informe_ventas"
  language="java"
  pageWidth="595"
  pageHeight="842"
  columnWidth="555"
```

XML

```

        leftMargin="20"
        rightMargin="20"
        topMargin="20"
        bottomMargin="20"
        uid="3dc22bd7-3b93-4da9-8b60-6b3c45674c91">
<property name="com.jaspersoft.studio.data.defaultdataadapter" value="SQLiteEditorial"/>
<template><![CDATA["resources/styles/EditorialStyles.jrtx"]]></template>
<style name="Sans_Normal" isDefault="true" fontName="DejaVu Sans" fontSize="10"/>
<style name="TituloPrincipal" style="Sans_Normal" fontSize="18" isBold="true" forecolor="#173F6B"/>
<style name="Cabecera" style="Sans_Normal" fontSize="9" isBold="true" forecolor="#173F6B"/>
<style name="Dato" style="Sans_Normal" fontSize="9"/>
<style name="UnidadesCondicional" style="Dato" isBold="true">
    <conditionalStyle>
        <conditionExpression><![CDATA[$F{unidades_vendidas} != null && $P{umbralUnidades} != null &&
$F{unidades_vendidas}.intValue() >= $P{umbralUnidades}.intValue()]]></conditionExpression>
        <style forecolor="#1B5E20"/>
    </conditionalStyle>
    <conditionalStyle>
        <conditionExpression><![CDATA[$F{unidades_vendidas} != null && $P{umbralUnidades} != null &&
$F{unidades_vendidas}.intValue() >= 3 && $F{unidades_vendidas}.intValue() < $P{umbralUnidades}.intValue()]]></conditionExpression>
        <style forecolor="#1D5D88"/>
    </conditionalStyle>
    <conditionalStyle>
        <conditionExpression><![CDATA[$F{unidades_vendidas} == null || $F{unidades_vendidas}.intValue() < 3]]></conditionExpression>
        <style forecolor="#9D3429"/>
    </conditionalStyle>
</style>
<style name="M5TableHeader" style="Dato" mode="Opaque" backcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/>
<style name="M5TableDetail" style="Dato"/>
<style name="M5CrossHeader" style="Dato" mode="Opaque" backcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/>
<style name="M5CrossDetail" style="Dato" mode="Opaque" backcolor="#FFFFFF"/>
<style name="M5CrossTotal" style="Dato" mode="Opaque" backcolor="#D6EAF8" forecolor="#173F6B" isBold="true"/>
<subDataset name="DatasetTopVentas">
    <parameter name="tituloLibro" class="java.lang.String"/>
    <queryString language="sql">
        <![CDATA[
            SELECT fecha_venta, cantidad, precio_unitario
            FROM ventas
            WHERE titulo_libro = $P{tituloLibro}
            ORDER BY cantidad DESC, fecha_venta
            LIMIT 3
        ]]>
    </queryString>
    <field name="fecha_venta" class="java.lang.String"/>
    <field name="cantidad" class="java.lang.Integer"/>
    <field name="precio_unitario" class="java.lang.Double"/>
</subDataset>
<subDataset name="DatasetVentasPorCategoria">
    <queryString language="sql">
        <![CDATA[
            SELECT l.categoria AS categoria_grafico,
                   COALESCE(SUM(v.cantidad * v.precio_unitario), 0.0) AS importe_categoria
            FROM libros l
            LEFT JOIN ventas v ON l.titulo = v.titulo_libro
            GROUP BY l.categoria
            ORDER BY l.categoria
        ]]>
    </queryString>
    <field name="categoria_grafico" class="java.lang.String"/>
    <field name="importe_categoria" class="java.lang.Double"/>
</subDataset>
<subDataset name="DatasetCrosstabVentas">
    <queryString language="sql">
        <![CDATA[
            SELECT l.categoria AS categoria_cross,
                   SUBSTR(v.fecha_venta, 1, 4) AS anio_cross,
                   (v.cantidad * v.precio_unitario) AS importe_cross,
                   1 AS ventas_cross
            FROM ventas v
            JOIN libros l ON l.titulo = v.titulo_libro
            ORDER BY l.categoria, anio_cross, v.fecha_venta
        ]]>
    </queryString>
    <field name="categoria_cross" class="java.lang.String"/>
    <field name="anio_cross" class="java.lang.String"/>
    <field name="importe_cross" class="java.lang.Double"/>
    <field name="ventas_cross" class="java.lang.Integer"/>
</subDataset>
<parameter name="usuario" class="java.lang.String" isForPrompting="true"/>
<parameter name="fechaInforme" class="java.util.Date" isForPrompting="true">
    <defaultValueExpression><![CDATA[new java.util.Date()]]></defaultValueExpression>
</parameter>
<parameter name="departamento" class="java.lang.String" isForPrompting="true">
    <defaultValueExpression><![CDATA["General"]]></defaultValueExpression>
</parameter>
<parameter name="periodo" class="java.lang.String" isForPrompting="true">
    <defaultValueExpression><![CDATA["Mensual"]]></defaultValueExpression>
</parameter>
<parameter name="tipoIva" class="java.lang.Double" isForPrompting="true">
    <defaultValueExpression><![CDATA[Double.valueOf(0.21d)]]></defaultValueExpression>
</parameter>
<parameter name="mostrarDetalle" class="java.lang.Boolean" isForPrompting="true">

```

```

        <defaultValueExpression><![CDATA[Boolean.TRUE]]></defaultValueExpression>
    </parameter>
    <parameter name="categoria" class="java.lang.String" isForPrompting="true"/>
    <parameter name="precioMinimo" class="java.lang.Double" isForPrompting="true"/>
    <parameter name="precioMaximo" class="java.lang.Double" isForPrompting="true"/>
    <parameter name="umbralUnidades" class="java.lang.Integer" isForPrompting="true">
        <defaultValueExpression><![CDATA[Integer.valueOf(5)]]></defaultValueExpression>
    </parameter>
    <parameter name="textoBusqueda" class="java.lang.String" isForPrompting="true"/>
    <parameter name="categoriasLista" class="java.util.Collection" isForPrompting="false">
        <defaultValueExpression><![CDATA[java.util.Arrays.asList("Novela", "Realismo mágico", "Cuento", "Poesía")]]></
defaultValueExpression>
    </parameter>
    <queryString language="sql">
        <![CDATA[
            SELECT l.titulo,
                   l.categoria,
                   SUM(v.cantidad) AS unidades_vendidas,
                   SUM(v.cantidad * v.precio_unitario) AS importe_total,
                   AVG(v.precio_unitario) AS precio_medio,
                   MIN(v.fecha_venta) AS primera_venta,
                   MAX(v.fecha_venta) AS ultima_venta
            FROM libros l
            LEFT JOIN ventas v ON l.titulo = v.titulo_libro
            WHERE ($P{categoria} IS NULL OR l.categoria = $P{categoria})
              AND ($P{precioMinimo} IS NULL OR l.precio >= $P{precioMinimo})
              AND ($P{precioMaximo} IS NULL OR l.precio <= $P{precioMaximo})
              AND ($P{textoBusqueda} IS NULL OR $P{textoBusqueda} = '' OR l.titulo LIKE '%' || $P{textoBusqueda} || '%')
              AND $X{IN, l.categoria, categoriasLista}
            GROUP BY l.titulo, l.categoria
            ORDER BY l.categoria, COALESCE(importe_total, 0) DESC, l.titulo
        ]]>
    </queryString>
    <field name="titulo" class="java.lang.String"/>
    <field name="categoria" class="java.lang.String"/>
    <field name="unidades_vendidas" class="java.lang.Integer"/>
    <field name="importe_total" class="java.lang.Double"/>
    <field name="precio_medio" class="java.lang.Double"/>
    <field name="primera_venta" class="java.lang.String"/>
    <field name="ultima_venta" class="java.lang.String"/>
    <variable name="TotalUnidades" class="java.lang.Integer" calculation="Sum" resetType="Report">
        <variableExpression><![CDATA[$F{unidades_vendidas}]]></variableExpression>
    </variable>
    <variable name="TotalImporte" class="java.lang.Double" calculation="Sum" resetType="Report">
        <variableExpression><![CDATA[$F{importe_total}]]></variableExpression>
    </variable>
    <variable name="TotalPagina" class="java.lang.Double" calculation="Sum" resetType="Page">
        <variableExpression><![CDATA[$F{importe_total}]]></variableExpression>
    </variable>
    <variable name="PrecioMedio" class="java.lang.Double" calculation="Average" resetType="Report">
        <variableExpression><![CDATA[$F{precio_medio}]]></variableExpression>
    </variable>
    <variable name="PrecioMaximo" class="java.lang.Double" calculation="Highest" resetType="Report">
        <variableExpression><![CDATA[$F{precio_medio}]]></variableExpression>
    </variable>
    <variable name="NumeroLibros" class="java.lang.Integer" calculation="Count" resetType="Report">
        <variableExpression><![CDATA[$F{titulo}]]></variableExpression>
    </variable>
    <variable name="ImporteConIva" class="java.lang.Double" calculation="Sum" resetType="Report">
        <variableExpression><![CDATA[$F{importe_total} == null || $P{tipoIva} == null ? null :
Double.valueOf($F{importe_total}.doubleValue() * (1.0d + $P{tipoIva}.doubleValue()))]]></variableExpression>
    </variable>
    <variable name="GrupoUnidades" class="java.lang.Integer" calculation="Sum" resetType="Group" resetGroup="CategoriaGroup">
        <variableExpression><![CDATA[$F{unidades_vendidas}]]></variableExpression>
    </variable>
    <variable name="GrupoImporte" class="java.lang.Double" calculation="Sum" resetType="Group" resetGroup="CategoriaGroup">
        <variableExpression><![CDATA[$F{importe_total}]]></variableExpression>
    </variable>
    <variable name="GrupoLibros" class="java.lang.Integer" calculation="Count" resetType="Group" resetGroup="CategoriaGroup">
        <variableExpression><![CDATA[$F{titulo}]]></variableExpression>
    </variable>
    <group name="CategoriaGroup" isStartNewPage="false" isReprintHeaderOnEachPage="true" minHeightToStartNewPage="80">
        <groupExpression><![CDATA[$F{categoria}]]></groupExpression>
        <groupHeader>
            <band height="28">
                <textField><reportElement x="0" y="2" width="555" height="22" style="M5GrupoCabecera"/><textFieldExpression><![
CDATA["Categoría: " + $F{categoria}]]></textFieldExpression></textField>
            </band>
        </groupHeader>
        <groupFooter>
            <band height="34">
                <textField><reportElement x="0" y="3" width="185" height="18" style="Dato"/><textFieldExpression><![CDATA["Libros del
grupo: " + $V{GrupoLibros}]]></textFieldExpression></textField>
                <textField><reportElement x="185" y="3" width="180" height="18" style="Dato"/><textFieldExpression><![CDATA["Unidades: "
+ ($V{GrupoUnidades} == null ? 0 : $V{GrupoUnidades}]]></textFieldExpression></textField>
                <textField><reportElement x="365" y="3" width="190" height="18" style="Dato"/><textFieldExpression><![CDATA["Importe: "
+ ($V{GrupoImporte} == null ? 0 : $V{GrupoImporte}]]></textFieldExpression></textField>
            </band>
        </groupFooter>
    </group>
</background><band height="0"/></background>

```

```

<title>
  <band height="124">
    <staticText>
      <reportElement x="0" y="4" width="555" height="28" uid="40000000-0000-4000-8000-000000000001"
style="M5TituloPrincipal"/>
      <textElement textAlignment="Center" verticalAlignment="Middle"/>
      <text><![CDATA[Informe de Ventas - Agregación por Título]]></text>
    </staticText>
    <staticText><reportElement x="0" y="38" width="110" height="18" uid="40000000-0000-4000-8000-000000000002"/><text><![
CDATA[Generado por:]]></text></staticText>
    <textField isBlankWhenNull="true"><reportElement x="110" y="38" width="160" height="18"
uid="40000000-0000-4000-8000-000000000003"/><textFieldExpression><![CDATA[{$usuario}]]></textFieldExpression></textField>
    <staticText><reportElement x="300" y="38" width="80" height="18" uid="40000000-0000-4000-8000-000000000004"/><text><![
CDATA[Fecha:]]></text></staticText>
    <textField pattern="dd/MM/yyyy"><reportElement x="380" y="38" width="175" height="18"
uid="40000000-0000-4000-8000-000000000005"/><textFieldExpression><![CDATA[{$fechaInforme}]]></textFieldExpression></textField>
    <staticText><reportElement x="0" y="62" width="100" height="18" uid="40000000-0000-4000-8000-000000000006"/><text><![
CDATA[Departamento:]]></text></staticText>
    <textField isBlankWhenNull="true"><reportElement x="100" y="62" width="170" height="18"
uid="40000000-0000-4000-8000-000000000007"/><textFieldExpression><![CDATA[{$departamento}]]></textFieldExpression></textField>
    <staticText><reportElement x="300" y="62" width="70" height="18" uid="40000000-0000-4000-8000-000000000008"/><text><![
CDATA[Periodo:]]></text></staticText>
    <textField isBlankWhenNull="true"><reportElement x="370" y="62" width="185" height="18"
uid="40000000-0000-4000-8000-000000000009"/><textFieldExpression><![CDATA[{$periodo}]]></textFieldExpression></textField>
    <staticText><reportElement x="0" y="86" width="100" height="18" uid="40000000-0000-4000-8000-000000000010"/><text><![
CDATA[Búsqueda:]]></text></staticText>
    <textField isBlankWhenNull="true"><reportElement x="100" y="86" width="170" height="18"
uid="40000000-0000-4000-8000-000000000011"/><textFieldExpression><![CDATA[{$textoBusqueda} == null ||
{$textoBusqueda}.trim().isEmpty() ? "(todas)" : {$textoBusqueda}]]></textFieldExpression></textField>
    <staticText><reportElement x="300" y="86" width="90" height="18" uid="40000000-0000-4000-8000-000000000012"/><text><![
CDATA[Categorías:]]></text></staticText>
    <textField textAdjust="StretchHeight"><reportElement x="390" y="86" width="165" height="34"
uid="40000000-0000-4000-8000-000000000013"/><textFieldExpression><![CDATA[String.valueOf({$categoriasLista}]]></
textFieldExpression></textField>
  </band>
</title>
<columnHeader>
  <band height="62">
    <staticText><reportElement x="0" y="2" width="215" height="18" uid="41000000-0000-4000-8000-000000000001" style="Cabecera"/>
<text><![CDATA[Título]]></text></staticText>
    <staticText><reportElement x="215" y="2" width="55" height="18" uid="41000000-0000-4000-8000-000000000002"
style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[Unid.]]></text></staticText>
    <staticText><reportElement x="280" y="2" width="90" height="18" uid="41000000-0000-4000-8000-000000000003"
style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[Importe]]></text></staticText>
    <staticText><reportElement x="380" y="2" width="65" height="18" uid="41000000-0000-4000-8000-000000000004"
style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[Precio med.]]></text></staticText>
    <staticText><reportElement x="455" y="2" width="100" height="18" uid="41000000-0000-4000-8000-000000000005"
style="Cabecera"/><text><![CDATA[Categoría]]></text></staticText>
    <staticText><reportElement x="0" y="24" width="130" height="18" uid="41000000-0000-4000-8000-000000000006"
style="Cabecera"/><text><![CDATA[Primera venta]]></text></staticText>
    <staticText><reportElement x="130" y="24" width="130" height="18" uid="41000000-0000-4000-8000-000000000007"
style="Cabecera"/><text><![CDATA[Última venta]]></text></staticText>
    <staticText><reportElement x="260" y="24" width="160" height="18" uid="41000000-0000-4000-8000-000000000008"
style="Cabecera"/><text><![CDATA[Periodo de ventas]]></text></staticText>
    <staticText>
      <reportElement x="420" y="24" width="135" height="18" uid="41000000-0000-4000-8000-000000000009" style="Cabecera">
        <printWhenExpression><![CDATA[Boolean.TRUE.equals({$mostrarDetalle}]]></printWhenExpression>
      </reportElement>
      <textElement textAlignment="Right"/>
      <text><![CDATA[Importe con IVA]]></text>
    </staticText>
  </band>
</columnHeader>
<detail>
  <band height="82" splitType="Stretch">
    <textField textAdjust="StretchHeight"><reportElement x="0" y="0" width="215" height="20"
uid="42000000-0000-4000-8000-000000000001" style="Dato"/><textFieldExpression><![CDATA[{$titulo}]]></textFieldExpression></textField>
    <textField isBlankWhenNull="true"><reportElement x="215" y="0" width="55" height="20"
uid="42000000-0000-4000-8000-000000000002" style="UnidadesCondiciona]]><textElement textAlignment="Right"/><textFieldExpression><![
CDATA[{$F{unidades_vendidas}]]></textFieldExpression></textField>
    <textField pattern="#,##0.00 €" isBlankWhenNull="true"><reportElement x="280" y="0" width="90" height="20"
uid="42000000-0000-4000-8000-000000000003" style="Dato"/><textElement textAlignment="Right"/><textFieldExpression><![
CDATA[{$F{importe_total}]]></textFieldExpression></textField>
    <textField isBlankWhenNull="true"><reportElement x="380" y="0" width="65" height="20"
uid="42000000-0000-4000-8000-000000000004" style="Dato"/><textElement textAlignment="Right"/><textFieldExpression><![
CDATA[{$F{precio_medio} == null ? "Sin datos" : new java.text.DecimalFormat("#0.00 '€').format({$F{precio_medio}]]></textFieldExpression></textField>
    <textField isBlankWhenNull="true"><reportElement x="455" y="0" width="100" height="20"
uid="42000000-0000-4000-8000-000000000005" style="Dato"/><textFieldExpression><![CDATA[{$F{categoria}]]></textFieldExpression></
textField>
    <textField isBlankWhenNull="true"><reportElement x="0" y="24" width="130" height="18"
uid="42000000-0000-4000-8000-000000000006" style="Dato"/><textFieldExpression><![CDATA[{$F{primera_venta}]]></textFieldExpression></
textField>
    <textField isBlankWhenNull="true"><reportElement x="130" y="24" width="130" height="18"
uid="42000000-0000-4000-8000-000000000007" style="Dato"/><textFieldExpression><![CDATA[{$F{ultima_venta}]]></textFieldExpression></
textField>
    <textField><reportElement x="260" y="24" width="160" height="18" uid="42000000-0000-4000-8000-000000000008" style="Dato"/>
<textElement textAlignment="Center"/><textFieldExpression><![CDATA[{$F{primera_venta} == null ? "Sin ventas" : $F{primera_venta} + " → "
+ $F{ultima_venta}]]></textFieldExpression></textField>
    <textField pattern="#,##0.00 €" isBlankWhenNull="true">
      <reportElement x="420" y="24" width="135" height="18" uid="42000000-0000-4000-8000-000000000009" style="Dato">
        <printWhenExpression><![CDATA[Boolean.TRUE.equals({$mostrarDetalle}]]></printWhenExpression>

```

```

        </reportElement>
        <textElement textAlignment="Right"/>
        <textFieldExpression><![CDATA[${importe_total} == null || ${tipoIva} == null ? null :
Double.valueOf(${importe_total}).doubleValue() * (1.0d + ${tipoIva}.doubleValue())]]></textFieldExpression>
    </textField>
    <textField><reportElement x="0" y="48" width="105" height="18" uuid="42000000-0000-4000-8000-000000000010" style="Dato"/
><textFieldExpression><![CDATA[${unidades_vendidas} == null ? "Sin ventas" : (${unidades_vendidas}.intValue() >= 6 ? "Premium" :
(${unidades_vendidas}.intValue() >= 3 ? "Estándar" : "Económico"))]]></textFieldExpression></textField>
    <textField><reportElement x="105" y="48" width="185" height="18" uuid="42000000-0000-4000-8000-000000000011" style="Dato"/
><textFieldExpression><![CDATA[${titulo} == null ? "" : ${titulo}.trim().toUpperCase(java.util.Locale.ROOT)]]></textFieldExpression></
textField>
    <textField><reportElement x="290" y="48" width="80" height="18" uuid="42000000-0000-4000-8000-000000000012" style="Dato"/
><textElement textAlignment="Right"/><textFieldExpression><![CDATA[${precio_medio} == null ? "-" : String.format(java.util.Locale.ROOT,
"%2f", Double.valueOf(Math.round(${precio_medio}.doubleValue() * 100.0d) / 100.0d)]]></textFieldExpression></textField>
    <textField><reportElement x="370" y="48" width="90" height="18" uuid="42000000-0000-4000-8000-000000000013" style="Dato"/
><textElement textAlignment="Center"/><textFieldExpression><![CDATA[${primera_venta} == null || ${ultima_venta} == null ? "-" :
java.lang.Long.toString(java.time.temporal.ChronoUnit.DAYS.between(java.time.LocalDate.parse(${primera_venta}),
java.time.LocalDate.parse(${ultima_venta}))) + " días"]></textFieldExpression></textField>
    <textField><reportElement x="460" y="48" width="95" height="18" uuid="42000000-0000-4000-8000-000000000014" style="Dato"/
><textElement textAlignment="Right"/><textFieldExpression><![CDATA[${unidades_vendidas} == null ? "0.0%" :
String.format(java.util.Locale.ROOT, "%1f%%", Double.valueOf(${unidades_vendidas}.doubleValue() / Math.max(1.0d, ${umbralUnidades} ==
null ? 1.0d : ${umbralUnidades}.doubleValue()) * 100.0d)]]></textFieldExpression></textField>
    </band>
    <band height="14">
        <printWhenExpression><![CDATA[${unidades_vendidas} != null && ${umbralUnidades} != null &&
${unidades_vendidas}.intValue() >= ${umbralUnidades}.intValue()]]></printWhenExpression>
        <textField><reportElement x="0" y="0" width="555" height="12" uuid="42000000-0000-4000-8000-000000000015"/><textElement
textAlignment="Center"><font fontName="DejaVu Sans" size="8" isBold="true"/><textElement><textFieldExpression><![CDATA["Fila destacada:
" + ${titulo} + " supera el umbral de " + ${umbralUnidades} + " unidades"]></textFieldExpression></textField>
    </band>
    <band height="88" splitType="Stretch">
        <printWhenExpression><![CDATA[${unidades_vendidas} != null]]></printWhenExpression>
        <staticText>
            <reportElement x="0" y="2" width="555" height="16" style="Cabecera"/>
            <text><![CDATA[Detalle de ventas]]></text>
        </staticText>
        <subreport>
            <reportElement x="0" y="22" width="555" height="60" isRemoveLineWhenBlank="true"/>
            <subreportParameter name="tituloLibro">
                <subreportParameterExpression><![CDATA[${titulo}]]></subreportParameterExpression>
            </subreportParameter>
            <connectionExpression><![CDATA[${REPORT_CONNECTION}]]></connectionExpression>
            <subreportExpression><![CDATA["reports/subinforme_ventas_detalle.jasper"]></subreportExpression>
        </subreport>
    </band>
    <band height="104" splitType="Stretch">
        <printWhenExpression><![CDATA[${unidades_vendidas} != null]]></printWhenExpression>
        <staticText>
            <reportElement x="0" y="2" width="555" height="16" style="Cabecera"/>
            <text><![CDATA[Top 3 ventas por cantidad]]></text>
        </staticText>
        <componentElement>
            <reportElement x="0" y="22" width="555" height="76"/>
            <c:table xmlns:c="http://jasperreports.sourceforge.net/jasperreports/components"
xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports/components http://
jasperreports.sourceforge.net/xsd/components.xsd">
                <datasetRun subDataset="DatasetTopVentas">
                    <datasetParameter name="tituloLibro">
                        <datasetParameterExpression><![CDATA[${titulo}]]></datasetParameterExpression>
                    </datasetParameter>
                    <connectionExpression><![CDATA[${REPORT_CONNECTION}]]></connectionExpression>
                </datasetRun>
                <c:column width="255">
                    <c:columnHeader style="M5TablaCabecera" height="20"><staticText><reportElement x="0" y="0" width="255"
height="20"/><text><![CDATA[Fecha]]></text></staticText></c:columnHeader>
                    <c:detailCell style="M5TablaDetalle" height="18"><textField><reportElement x="0" y="0" width="255" height="18"/
><textFieldExpression><![CDATA[${fecha_venta}]]></textFieldExpression></textField></c:detailCell>
                </c:column>
                <c:column width="100">
                    <c:columnHeader style="M5TablaCabecera" height="20"><staticText><reportElement x="0" y="0" width="100"
height="20"/><textElement textAlignment="Right"/><text><![CDATA[Cantidad]]></text></staticText></c:columnHeader>
                    <c:detailCell style="M5TablaDetalle" height="18"><textField><reportElement x="0" y="0" width="100" height="18"/
><textElement textAlignment="Right"/><textFieldExpression><![CDATA[${cantidad}]]></textFieldExpression></textField></c:detailCell>
                </c:column>
                <c:column width="200">
                    <c:columnHeader style="M5TablaCabecera" height="20"><staticText><reportElement x="0" y="0" width="200"
height="20"/><textElement textAlignment="Right"/><text><![CDATA[Precio unitario]]></text></staticText></c:columnHeader>
                    <c:detailCell style="M5TablaDetalle" height="18"><textField pattern="#,##0.00 €"><reportElement x="0" y="0"
width="200" height="18"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[${precio_unitario}]]></
textFieldExpression></textField></c:detailCell>
                </c:column>
            </c:table>
        </componentElement>
    </band>
</detail>
<pageFooter>
    <band height="62">
        <staticText><reportElement x="0" y="4" width="120" height="15" uuid="43000000-0000-4000-8000-000000000001"/><text><![
CDATA[Total de títulos:]]></text></staticText>
        <textField evaluationTime="Report"><reportElement x="120" y="4" width="60" height="15"
uuid="43000000-0000-4000-8000-000000000002"/><textFieldExpression><![CDATA[${REPORT_COUNT}]]></textFieldExpression></textField>
        <textField><reportElement x="190" y="28" width="180" height="15" uuid="43000000-0000-4000-8000-000000000003"/><textElement

```

```

textAlignment="Right"/><textFieldExpression><![CDATA["Página " + ${PAGE_NUMBER} + " de"]]></textFieldExpression></textField>
    <textField evaluationTime="Report"><reportElement x="375" y="28" width="35" height="15"
uuid="43000000-0000-4000-8000-000000000004"/><textFieldExpression><![CDATA[${PAGE_NUMBER}]]></textField>
    <staticText><reportElement x="300" y="4" width="120" height="15" uuid="43000000-0000-4000-8000-000000000005"/><text><![
CDATA[Subtotal página:]]></text></staticText>
    <textField pattern="##0.00 €"><reportElement x="420" y="4" width="135" height="15"
uuid="43000000-0000-4000-8000-000000000006"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[${TotalPagina}]]></
textFieldExpression></textField>
    </band>
</pageFooter>
<summary>
    <band height="700">
        <staticText><reportElement x="0" y="5" width="205" height="18" uuid="44000000-0000-4000-8000-000000000001"/><text><![
CDATA[Total de unidades vendidas:]]></text></staticText>
        <textField><reportElement x="205" y="5" width="80" height="18" uuid="44000000-0000-4000-8000-000000000002"/><textElement
textAlignment="Right"/><textFieldExpression><![CDATA[${TotalUnidades}]]></textFieldExpression></textField>
        <staticText><reportElement x="300" y="5" width="120" height="18" uuid="44000000-0000-4000-8000-000000000003"/><text><![
CDATA[Importe total:]]></text></staticText>
        <textField pattern="##0.00 €"><reportElement x="420" y="5" width="135" height="18"
uuid="44000000-0000-4000-8000-000000000004"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[${TotalImporte}]]></
textFieldExpression></textField>
        <staticText><reportElement x="0" y="30" width="205" height="18" uuid="44000000-0000-4000-8000-000000000005"/><text><![
CDATA[Precio medio agregado:]]></text></staticText>
        <textField pattern="##0.00 €"><reportElement x="205" y="30" width="80" height="18"
uuid="44000000-0000-4000-8000-000000000006"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[${PrecioMedio}]]></
textFieldExpression></textField>
        <staticText><reportElement x="0" y="30" width="120" height="18" uuid="44000000-0000-4000-8000-000000000007"/><text><![
CDATA[Precio máximo:]]></text></staticText>
        <textField pattern="##0.00 €"><reportElement x="420" y="30" width="135" height="18"
uuid="44000000-0000-4000-8000-000000000008"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[${PrecioMaximo}]]></
textFieldExpression></textField>
        <staticText><reportElement x="0" y="55" width="205" height="18" uuid="44000000-0000-4000-8000-000000000009"/><text><![
CDATA[Número de libros:]]></text></staticText>
        <textField><reportElement x="205" y="55" width="80" height="18" uuid="44000000-0000-4000-8000-000000000010"/><textElement
textAlignment="Right"/><textFieldExpression><![CDATA[${NumeroLibros}]]></textFieldExpression></textField>
        <staticText><reportElement x="300" y="55" width="120" height="18" uuid="44000000-0000-4000-8000-000000000011"/><text><![
CDATA[Importe con IVA:]]></text></staticText>
        <textField pattern="##0.00 €"><reportElement x="420" y="55" width="135" height="18"
uuid="44000000-0000-4000-8000-000000000012"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[${ImporteConIva}]]></
textFieldExpression></textField>
        <textField><reportElement x="0" y="80" width="555" height="18" uuid="44000000-0000-4000-8000-000000000013"/><textElement
textAlignment="Center"/><textFieldExpression><![CDATA[String.format(java.util.Locale.ROOT, "Resumen: %d títulos · %d unidades · %.2f €",
${NumeroLibros}, ${TotalUnidades}, ${TotalImporte}]]></textFieldExpression></textField>
        <textField><reportElement x="0" y="103" width="350" height="18" uuid="44000000-0000-4000-8000-000000000014"/><textElement
textAlignment="Center"/><font fontName="DejaVu Sans" size="10" isBold="true"/><textFieldExpression><![
CDATA[${TotalUnidades} != null && ${umbralUnidades} != null && ${TotalUnidades}.intValue() >= ${umbralUnidades}.intValue() ?
"Objetivo de ventas alcanzado" : "Objetivo de ventas pendiente"]></textFieldExpression></textField>
        <textField><reportElement x="360" y="103" width="195" height="18" uuid="44000000-0000-4000-8000-000000000015"/>
><textFieldExpression><![CDATA["Resultados encontrados: " + ${REPORT_COUNT}]]></textFieldExpression></textField>
        <staticText><reportElement x="0" y="140" width="555" height="20" style="Cabecera"/><text><![CDATA[Ventas por categoría –
importe]]></text></staticText>
    <barChart>
        <chart>
            <reportElement x="0" y="165" width="555" height="250"/>
            <chartTitle><titleExpression><![CDATA["Ventas por categoría"]]></titleExpression></chartTitle>
            <chartSubtitle/>
            <chartLegend position="Bottom"/>
        </chart>
        <categoryDataset>
            <dataset>
                <datasetRun subDataset="DatasetVentasPorCategoría">
                    <connectionExpression><![CDATA[${REPORT_CONNECTION}]]></connectionExpression>
                </datasetRun>
            </dataset>
            <categorySeries>
                <seriesExpression><![CDATA["Importe"]]></seriesExpression>
                <categoryExpression><![CDATA[${F{categoria_grafico}}]></categoryExpression>
                <valueExpression><![CDATA[${F{importe_categoria}}]></valueExpression>
            </categorySeries>
        </categoryDataset>
    </barChart>
    <staticText><reportElement x="0" y="430" width="555" height="20" style="Cabecera"/><text><![CDATA[Ventas por categoría y
año]]></text></staticText>
    <crossTab>
        <reportElement x="0" y="455" width="555" height="225"/>
        <crossTabDataset>
            <dataset>
                <datasetRun subDataset="DatasetCrossTabVentas">
                    <connectionExpression><![CDATA[${REPORT_CONNECTION}]]></connectionExpression>
                </datasetRun>
            </dataset>
        </crossTabDataset>
        <rowGroup name="CategoríaCross" width="150" totalPosition="End">
            <bucket class="java.lang.String"><bucketExpression><![CDATA[${F{categoria_cross}}]></bucketExpression></bucket>
            <crossTabRowHeader><cellContents style="MSCrossTabCabecera"><textField><reportElement x="0" y="0" width="150"
height="34"/><textElement verticalAlignment="Middle"/><textFieldExpression><![CDATA[${F{CategoríaCross}}]></textFieldExpression></

```

```

textField></cellContents></crosstabRowHeader>
    <crosstabTotalRowHeader><cellContents style="M5CrosstabTotal"><staticText><reportElement x="0" y="0" width="150"
height="34"/><textElement verticalAlignment="Middle"/><text><![CDATA[TOTAL]]></text></staticText></cellContents></
crosstabTotalRowHeader>
    </rowGroup>
    <columnGroup name="AnioCross" height="28" totalPosition="End">
        <bucket class="java.lang.String"><bucketExpression><![CDATA[{F{anio_cross}}]></bucketExpression></bucket>
        <crosstabColumnHeader><cellContents style="M5CrosstabCabecera"><textField><reportElement x="0" y="0" width="100"
height="28"/><textElement textAlignment="Center" verticalAlignment="Middle"/><textFieldExpression><![CDATA[{V{AnioCross}}]></
textFieldExpression></textField></cellContents></crosstabColumnHeader>
        <crosstabTotalColumnHeader><cellContents style="M5CrosstabTotal"><staticText><reportElement x="0" y="0" width="100"
height="28"/><textElement textAlignment="Center" verticalAlignment="Middle"/><text><![CDATA[TOTAL]]></text></staticText></
cellContents></crosstabTotalColumnHeader>
    </columnGroup>
    <measure name="ImporteCross" class="java.lang.Double" calculation="Sum"><measureExpression><![
CDATA[{F{importe_cross}}]></measureExpression></measure>
    <measure name="VentasCross" class="java.lang.Integer" calculation="Sum"><measureExpression><![
CDATA[{F{ventas_cross}}]></measureExpression></measure>
    <crosstabCell width="100" height="34"><cellContents style="M5CrosstabDetalle"><textField pattern="#.##0.00
€"><reportElement x="0" y="0" width="100" height="18"/><textElement textAlignment="Right"/><textFieldExpression><![
CDATA[{V{ImporteCross}}]></textFieldExpression></textField><textField><reportElement x="0" y="18" width="100" height="14"/><textElement
textAlignment="Right"/><textFieldExpression><![CDATA[{V{VentasCross} + " ventas"}]></textFieldExpression></textField></cellContents></
crosstabCell>
    <crosstabCell width="100" height="34" rowTotalGroup="CategoriaCross"><cellContents style="M5CrosstabTotal"><textField
pattern="#.##0.00 €"><reportElement x="0" y="0" width="100" height="18"/><textElement textAlignment="Right"/><textFieldExpression><![
CDATA[{V{ImporteCross}}]></textFieldExpression></textField><textField><reportElement x="0" y="18" width="100" height="14"/><textElement
textAlignment="Right"/><textFieldExpression><![CDATA[{V{VentasCross} + " ventas"}]></textFieldExpression></textField></cellContents></
crosstabCell>
    <crosstabCell width="100" height="34" columnTotalGroup="AnioCross"><cellContents style="M5CrosstabTotal"><textField
pattern="#.##0.00 €"><reportElement x="0" y="0" width="100" height="18"/><textElement textAlignment="Right"/><textFieldExpression><![
CDATA[{V{ImporteCross}}]></textFieldExpression></textField><textField><reportElement x="0" y="18" width="100" height="14"/><textElement
textAlignment="Right"/><textFieldExpression><![CDATA[{V{VentasCross} + " ventas"}]></textFieldExpression></textField></cellContents></
crosstabCell>
    <crosstabCell width="100" height="34" rowTotalGroup="CategoriaCross" columnTotalGroup="AnioCross"><cellContents
style="M5CrosstabTotal"><textField pattern="#.##0.00 €"><reportElement x="0" y="0" width="100" height="18"/><textElement
textAlignment="Right"/><textFieldExpression><![CDATA[{V{ImporteCross}}]></textFieldExpression></textField><textField><reportElement
x="0" y="18" width="100" height="14"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[{V{VentasCross} + " ventas"}]></
textFieldExpression></textField></cellContents></crosstabCell>
    </crosstab>
</band>
</summary>
</jasperReport>

```

Explicación línea por línea

Línea 1	<?xml version="1.0" encoding="UTF-8"?> → Declara XML 1.0 y codificación UTF-8 para que nombres, textos y símbolos del informe se interpreten correctamente.
Línea 2	<jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports" → Abre el documento raíz jasperReport del informe y fija el namespace principal de JasperReports.
Línea 3	xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" → Declara el namespace XML Schema Instance usado por xsi:schemaLocation para validar el documento.
Línea 4	xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/jasperreport.xsd" → Relaciona el namespace de JasperReports con su XSD para que Studio y el compilador validen la estructura.
Línea 5	name="informe_ventas" → Asigna al documento JasperReports el nombre interno informe_ventas.
Línea 6	language="java" → Configura language=java para evaluar expresiones con el lenguaje Java.
Línea 7	pageWidth="595" → Fija el ancho físico de página en 595 puntos.
Línea 8	pageHeight="842" → Fija la altura física de página en 842 puntos.
Línea 9	columnWidth="555" → Fija el ancho útil de la columna de contenido en 555 puntos.
Línea 10	leftMargin="20" → Fija el margen izquierdo del informe en 20 puntos.
Línea 11	rightMargin="20" → Fija el margen derecho del informe en 20 puntos.
Línea 12	topMargin="20" → Fija el margen superior del informe en 20 puntos.
Línea 13	bottomMargin="20" → Fija el margen inferior del informe en 20 puntos y completa la apertura del elemento raíz.
Línea 14	uuid="3d2c2bd7-3b93-4da9-8b60-6b3c45674c91"> → Asigna el UUID de diseño 3d2c2bd7-3b93-4da9-8b60-6b3c45674c91 para identificar de forma estable el informe en Studio.

Línea 15	<code><property name="com.jaspersoft.studio.data.defaultdataadapter" value="SQLiteEditorial"/></code> → Indica a Jaspersoft Studio que use el Data Adapter SQLiteEditorial como conexión de diseño por defecto.
Línea 16	<code><template><![CDATA["resources/styles/EditorialStyles.jrtx"]]></template></code> → Importa la plantilla externa cuya expresión CDATA devuelve <code>resources/styles/EditorialStyles.jrtx</code> .
Línea 17	<code><style name="Sans_Normal" isDefault="true" fontName="DejaVu Sans" fontSize="10"/></code> → Declara el estilo Sans_Normal; es el estilo por defecto, fuente DejaVu Sans, tamaño 10.
Línea 18	<code><style name="TituloPrincipal" style="Sans_Normal" fontSize="18" isBold="true" forecolor="#173F6B"/></code> → Declara el estilo TituloPrincipal; hereda de Sans_Normal, tamaño 18.
Línea 19	<code><style name="Cabecera" style="Sans_Normal" fontSize="9" isBold="true" forecolor="#173F6B"/></code> → Declara el estilo Cabecera; hereda de Sans_Normal, tamaño 9.
Línea 20	<code><style name="Dato" style="Sans_Normal" fontSize="9"/></code> → Declara el estilo Dato; hereda de Sans_Normal, tamaño 9.
Línea 21	<code><style name="UnidadesCondicional" style="Dato" isBold="true"></code> → Declara el estilo UnidadesCondicional; hereda de Dato.
Línea 22	<code><conditionalStyle></code> → Abre una variante condicional del estilo; sólo se aplicará cuando su <code>conditionExpression</code> sea verdadera.
Línea 23	<code><conditionExpression><![CDATA[\${unidades_vendidas} != null && \${umbralUnidades} != null && \${unidades_vendidas}.intValue() >= \${umbralUnidades}.intValue()]]></conditionExpression></code> → Define la condición booleana que activa el estilo condicional usando field <code>unidades_vendidas</code> , field <code>unidades_vendidas</code> , parámetro <code>umbralUnidades</code> , parámetro <code>umbralUnidades</code> .
Línea 24	<code><style forecolor="#1B5E20"/></code> → Declara el estilo None.
Línea 25	<code></conditionalStyle></code> → Cierra <code>conditionalStyle</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 26	<code><conditionalStyle></code> → Abre una variante condicional del estilo; sólo se aplicará cuando su <code>conditionExpression</code> sea verdadera.
Línea 27	<code><conditionExpression><![CDATA[\${unidades_vendidas} != null && \${umbralUnidades} != null && \${unidades_vendidas}.intValue() >= 3 && \${unidades_vendidas}.intValue() < \${umbralUnidades}.intValue()]]></conditionExpression></code> → Define la condición booleana que activa el estilo condicional usando field <code>unidades_vendidas</code> , field <code>unidades_vendidas</code> , field <code>unidades_vendidas</code> , parámetro <code>umbralUnidades</code> , parámetro <code>umbralUnidades</code> .
Línea 28	<code><style forecolor="#1D5D88"/></code> → Declara el estilo None.
Línea 29	<code></conditionalStyle></code> → Cierra <code>conditionalStyle</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 30	<code><conditionalStyle></code> → Abre una variante condicional del estilo; sólo se aplicará cuando su <code>conditionExpression</code> sea verdadera.
Línea 31	<code><conditionExpression><![CDATA[\${unidades_vendidas} == null \${unidades_vendidas}.intValue() < 3]]></conditionExpression></code> → Define la condición booleana que activa el estilo condicional usando field <code>unidades_vendidas</code> , field <code>unidades_vendidas</code> .
Línea 32	<code><style forecolor="#9D3429"/></code> → Declara el estilo None.
Línea 33	<code></conditionalStyle></code> → Cierra <code>conditionalStyle</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 34	<code></style></code> → Cierra <code>style</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 35	<code><style name="M5TableHeader" style="Dato" mode="Opaque" backcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/></code> → Declara el estilo M5TableHeader; hereda de Dato, fondo #EAF2F8.
Línea 36	<code><style name="M5TableDetail" style="Dato"/></code> → Declara el estilo M5TableDetail; hereda de Dato.
Línea 37	<code><style name="M5CrossHeader" style="Dato" mode="Opaque" backcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/></code> → Declara el estilo M5CrossHeader; hereda de Dato, fondo #EAF2F8.
Línea 38	<code><style name="M5CrossDetail" style="Dato" mode="Opaque" backcolor="#FFFFFF"/></code> → Declara el estilo M5CrossDetail; hereda de Dato, fondo #FFFFFF.

Línea 39	<code><style name="M5CrossTotal" style="Dato" mode="Opaque" backcolor="#D6EAF8" forecolor="#173F6B" isBold="true"/></code> → Declara el estilo M5CrossTotal; hereda de Dato, fondo #D6EAF8.
Línea 40	<code><subDataset name="DatasetTopVentas"></code> → Declara el subdataset DatasetTopVentas, con consulta y fields propios independientes del dataset principal.
Línea 41	<code><parameter name="tituloLibro" class="java.lang.String"/></code> → Declara el parámetro tituloLibro con tipo java.lang.String y lo expone al diálogo de parámetros de Studio.
Línea 42	<code><queryString language="sql"></code> → Abre la consulta SQL que JasperReports ejecutará para el dataset actual.
Línea 43	<code><![CDATA[</code> → Abre CDATA para escribir SQL o una expresión Java sin que sus caracteres especiales se interpreten como XML.
Línea 44	<code>SELECT fecha_venta, cantidad, precio_unitario</code> → Cláusula SQL SELECT: selecciona y calcula las columnas que devolverá la consulta.
Línea 45	<code>FROM ventas</code> → Cláusula SQL FROM: define la tabla base de la consulta.
Línea 46	<code>WHERE titulo_libro = \${tituloLibro}</code> → Cláusula SQL WHERE: aplica el filtro de filas.
Línea 47	<code>ORDER BY cantidad DESC, fecha_venta</code> → Cláusula SQL ORDER BY: ordena el resultado que recibirá JasperReports.
Línea 48	<code>LIMIT 3</code> → Cláusula SQL LIMIT: limita el número de filas devueltas.
Línea 49	<code>]]></code> → Cierra el bloque CDATA y devuelve el control al parser XML.
Línea 50	<code></queryString></code> → Cierra la consulta SQL del dataset actual.
Línea 51	<code><field name="fecha_venta" class="java.lang.String"/></code> → Declara el field fecha_venta con tipo Java java.lang.String para mapear una columna del dataset.
Línea 52	<code><field name="cantidad" class="java.lang.Integer"/></code> → Declara el field cantidad con tipo Java java.lang.Integer para mapear una columna del dataset.
Línea 53	<code><field name="precio_unitario" class="java.lang.Double"/></code> → Declara el field precio_unitario con tipo Java java.lang.Double para mapear una columna del dataset.
Línea 54	<code></subDataset></code> → Finaliza el subdataset auxiliar y vuelve al nivel del informe.
Línea 55	<code><subDataset name="DatasetVentasPorCategoria"></code> → Declara el subdataset DatasetVentasPorCategoria, con consulta y fields propios independientes del dataset principal.
Línea 56	<code><queryString language="sql"></code> → Abre la consulta SQL que JasperReports ejecutará para el dataset actual.
Línea 57	<code><![CDATA[</code> → Abre CDATA para escribir SQL o una expresión Java sin que sus caracteres especiales se interpreten como XML.
Línea 58	<code>SELECT l.categoria AS categoria_grafico,</code> → Cláusula SQL SELECT: selecciona y calcula las columnas que devolverá la consulta.
Línea 59	<code>COALESCE(SUM(v.cantidad * v.precio_unitario), 0.0) AS importe_categoria</code> → Calcula o selecciona un valor SQL y lo expone con el alias importe_categoria, que después coincide con un field del subdataset.
Línea 60	<code>FROM libros l</code> → Cláusula SQL FROM: define la tabla base de la consulta.
Línea 61	<code>LEFT JOIN ventas v ON l.titulo = v.titulo_libro</code> → Cláusula SQL LEFT JOIN: une datos conservando las filas del lado izquierdo aunque no tengan ventas.
Línea 62	<code>GROUP BY l.categoria</code> → Cláusula SQL GROUP BY: agrupa las filas antes de evaluar las funciones agregadas.
Línea 63	<code>ORDER BY l.categoria</code> → Cláusula SQL ORDER BY: ordena el resultado que recibirá JasperReports.
Línea 64	<code>]]></code> → Cierra el bloque CDATA y devuelve el control al parser XML.
Línea 65	<code></queryString></code> → Cierra la consulta SQL del dataset actual.
Línea 66	<code><field name="categoria_grafico" class="java.lang.String"/></code> → Declara el field categoria_grafico con tipo Java java.lang.String para mapear una columna del dataset.

Línea 67	<code><field name="importe_categoria" class="java.lang.Double"/></code> → Declara el field <code>importe_categoria</code> con tipo Java <code>java.lang.Double</code> para mapear una columna del dataset.
Línea 68	<code></subDataset></code> → Finaliza el subdataset auxiliar y vuelve al nivel del informe.
Línea 69	<code><subDataset name="DatasetCrosstabVentas"></code> → Declara el subdataset <code>DatasetCrosstabVentas</code> , con consulta y fields propios independientes del dataset principal.
Línea 70	<code><queryString language="sql"></code> → Abre la consulta SQL que JasperReports ejecutará para el dataset actual.
Línea 71	<code><![CDATA[</code> → Abre CDATA para escribir SQL o una expresión Java sin que sus caracteres especiales se interpreten como XML.
Línea 72	<code>SELECT l.categoria AS categoria_cross,</code> → Cláusula SQL <code>SELECT</code> : selecciona y calcula las columnas que devolverá la consulta.
Línea 73	<code>SUBSTR(v.fecha_venta, 1, 4) AS anio_cross,</code> → Calcula o selecciona un valor SQL y lo expone con el alias <code>anio_cross</code> , que después coincide con un field del subdataset.
Línea 74	<code>(v.cantidad * v.precio_unitario) AS importe_cross,</code> → Calcula o selecciona un valor SQL y lo expone con el alias <code>importe_cross</code> , que después coincide con un field del subdataset.
Línea 75	<code>1 AS ventas_cross</code> → Calcula o selecciona un valor SQL y lo expone con el alias <code>ventas_cross</code> , que después coincide con un field del subdataset.
Línea 76	<code>FROM ventas v</code> → Cláusula SQL <code>FROM</code> : define la tabla base de la consulta.
Línea 77	<code>JOIN libros l ON l.titulo = v.titulo_libro</code> → Cláusula SQL <code>JOIN</code> : une las filas que cumplen la relación indicada.
Línea 78	<code>ORDER BY l.categoria, anio_cross, v.fecha_venta</code> → Cláusula SQL <code>ORDER BY</code> : ordena el resultado que recibirá JasperReports.
Línea 79	<code>]]></code> → Cierra el bloque CDATA y devuelve el control al parser XML.
Línea 80	<code></queryString></code> → Cierra la consulta SQL del dataset actual.
Línea 81	<code><field name="categoria_cross" class="java.lang.String"/></code> → Declara el field <code>categoria_cross</code> con tipo Java <code>java.lang.String</code> para mapear una columna del dataset.
Línea 82	<code><field name="anio_cross" class="java.lang.String"/></code> → Declara el field <code>anio_cross</code> con tipo Java <code>java.lang.String</code> para mapear una columna del dataset.
Línea 83	<code><field name="importe_cross" class="java.lang.Double"/></code> → Declara el field <code>importe_cross</code> con tipo Java <code>java.lang.Double</code> para mapear una columna del dataset.
Línea 84	<code><field name="ventas_cross" class="java.lang.Integer"/></code> → Declara el field <code>ventas_cross</code> con tipo Java <code>java.lang.Integer</code> para mapear una columna del dataset.
Línea 85	<code></subDataset></code> → Finaliza el subdataset auxiliar y vuelve al nivel del informe.
Línea 86	<code><parameter name="usuario" class="java.lang.String" isForPrompting="true"/></code> → Declara el parámetro <code>usuario</code> con tipo <code>java.lang.String</code> y lo expone al diálogo de parámetros de Studio.
Línea 87	<code><parameter name="fechaInforme" class="java.util.Date" isForPrompting="true"></code> → Declara el parámetro <code>fechaInforme</code> con tipo <code>java.util.Date</code> y lo expone al diálogo de parámetros de Studio.
Línea 88	<code><defaultValueExpression><![CDATA[new java.util.Date()]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 89	<code></parameter></code> → Cierra <code>parameter</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 90	<code><parameter name="departamento" class="java.lang.String" isForPrompting="true"></code> → Declara el parámetro <code>departamento</code> con tipo <code>java.lang.String</code> y lo expone al diálogo de parámetros de Studio.
Línea 91	<code><defaultValueExpression><![CDATA["General"]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 92	<code></parameter></code> → Cierra <code>parameter</code> y vuelve al elemento padre de la jerarquía JRXML.

Línea 93	<code><parameter name="periodo" class="java.lang.String" isForPrompting="true"></code> → Declara el parámetro periodo con tipo <code>java.lang.String</code> y lo expone al diálogo de parámetros de Studio.
Línea 94	<code><defaultValueExpression><![CDATA["Mensual"]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 95	<code></parameter></code> → Cierra parameter y vuelve al elemento padre de la jerarquía JRXML.
Línea 96	<code><parameter name="tipoIva" class="java.lang.Double" isForPrompting="true"></code> → Declara el parámetro tipoIva con tipo <code>java.lang.Double</code> y lo expone al diálogo de parámetros de Studio.
Línea 97	<code><defaultValueExpression><![CDATA[Double.valueOf(0.21d)]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 98	<code></parameter></code> → Cierra parameter y vuelve al elemento padre de la jerarquía JRXML.
Línea 99	<code><parameter name="mostrarDetalle" class="java.lang.Boolean" isForPrompting="true"></code> → Declara el parámetro mostrarDetalle con tipo <code>java.lang.Boolean</code> y lo expone al diálogo de parámetros de Studio.
Línea 100	<code><defaultValueExpression><![CDATA[Boolean.TRUE]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 101	<code></parameter></code> → Cierra parameter y vuelve al elemento padre de la jerarquía JRXML.
Línea 102	<code><parameter name="categoria" class="java.lang.String" isForPrompting="true"/></code> → Declara el parámetro categoria con tipo <code>java.lang.String</code> y lo expone al diálogo de parámetros de Studio.
Línea 103	<code><parameter name="precioMinimo" class="java.lang.Double" isForPrompting="true"/></code> → Declara el parámetro precioMinimo con tipo <code>java.lang.Double</code> y lo expone al diálogo de parámetros de Studio.
Línea 104	<code><parameter name="precioMaximo" class="java.lang.Double" isForPrompting="true"/></code> → Declara el parámetro precioMaximo con tipo <code>java.lang.Double</code> y lo expone al diálogo de parámetros de Studio.
Línea 105	<code><parameter name="umbralUnidades" class="java.lang.Integer" isForPrompting="true"></code> → Declara el parámetro umbralUnidades con tipo <code>java.lang.Integer</code> y lo expone al diálogo de parámetros de Studio.
Línea 106	<code><defaultValueExpression><![CDATA[Integer.valueOf(5)]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 107	<code></parameter></code> → Cierra parameter y vuelve al elemento padre de la jerarquía JRXML.
Línea 108	<code><parameter name="textoBusqueda" class="java.lang.String" isForPrompting="true"/></code> → Declara el parámetro textoBusqueda con tipo <code>java.lang.String</code> y lo expone al diálogo de parámetros de Studio.
Línea 109	<code><parameter name="categoriasLista" class="java.util.Collection" isForPrompting="false"></code> → Declara el parámetro categoriasLista con tipo <code>java.util.Collection</code> como parámetro interno no solicitado al usuario.
Línea 110	<code><defaultValueExpression><![CDATA[java.util.Arrays.asList("Novela", "Realismo mágico", "Cuento", "Poesía")]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 111	<code></parameter></code> → Cierra parameter y vuelve al elemento padre de la jerarquía JRXML.
Línea 112	<code><queryString language="sql"></code> → Abre la consulta SQL que JasperReports ejecutará para el dataset actual.
Línea 113	<code><![CDATA[</code> → Abre CDATA para escribir SQL o una expresión Java sin que sus caracteres especiales se interpreten como XML.
Línea 114	<code>SELECT l.titulo,</code> → Cláusula SQL SELECT: selecciona y calcula las columnas que devolverá la consulta.
Línea 115	<code>l.categoria,</code> → Aporta el valor literal <code>l.categoria</code> , al elemento XML/POM actualmente abierto.
Línea 116	<code>SUM(v.cantidad) AS unidades_vendidas,</code> → Calcula o selecciona un valor SQL y lo expone con el alias <code>unidades_vendidas</code> , que después coincide con un field del subdataset.
Línea 117	<code>SUM(v.cantidad * v.precio_unitario) AS importe_total,</code> → Calcula o selecciona un valor SQL y lo expone con el alias <code>importe_total</code> , que después coincide con un field del subdataset.

Línea 118	AVG(v.precio_unitario) AS precio_medio, → Calcula o selecciona un valor SQL y lo expone con el alias precio_medio, que después coincide con un field del subdataset.
Línea 119	MIN(v.fecha_venta) AS primera_venta, → Calcula o selecciona un valor SQL y lo expone con el alias primera_venta, que después coincide con un field del subdataset.
Línea 120	MAX(v.fecha_venta) AS ultima_venta → Calcula o selecciona un valor SQL y lo expone con el alias ultima_venta, que después coincide con un field del subdataset.
Línea 121	FROM libros l → Cláusula SQL FROM: define la tabla base de la consulta.
Línea 122	LEFT JOIN ventas v ON l.titulo = v.titulo_libro → Cláusula SQL LEFT JOIN: une datos conservando las filas del lado izquierdo aunque no tengan ventas.
Línea 123	WHERE (\$P{categoria} IS NULL OR l.categoria = \$P{categoria}) → Cláusula SQL WHERE: aplica el filtro de filas.
Línea 124	AND (\$P{precioMinimo} IS NULL OR l.precio >= \$P{precioMinimo}) → Añade otra condición lógica al filtro SQL, combinándola con la condición anterior.
Línea 125	AND (\$P{precioMaximo} IS NULL OR l.precio <= \$P{precioMaximo}) → Añade otra condición lógica al filtro SQL, combinándola con la condición anterior.
Línea 126	AND (\$P{textoBusqueda} IS NULL OR \$P{textoBusqueda} = '' OR l.titulo LIKE '%' \$P{textoBusqueda} '%') → Añade otra condición lógica al filtro SQL, combinándola con la condición anterior.
Línea 127	AND \$X{IN, l.categoria, categoriasLista} → Añade otra condición lógica al filtro SQL, combinándola con la condición anterior.
Línea 128	GROUP BY l.titulo, l.categoria → Cláusula SQL GROUP BY: agrupa las filas antes de evaluar las funciones agregadas.
Línea 129	ORDER BY l.categoria, COALESCE(importe_total, 0) DESC, l.titulo → Cláusula SQL ORDER BY: ordena el resultado que recibirá JasperReports.
Línea 130	]]> → Cierra el bloque CDATA y devuelve el control al parser XML.
Línea 131	</queryString> → Cierra la consulta SQL del dataset actual.
Línea 132	<field name="titulo" class="java.lang.String"/> → Declara el field titulo con tipo Java java.lang.String para mapear una columna del dataset.
Línea 133	<field name="categoria" class="java.lang.String"/> → Declara el field categoria con tipo Java java.lang.String para mapear una columna del dataset.
Línea 134	<field name="unidades_vendidas" class="java.lang.Integer"/> → Declara el field unidades_vendidas con tipo Java java.lang.Integer para mapear una columna del dataset.
Línea 135	<field name="importe_total" class="java.lang.Double"/> → Declara el field importe_total con tipo Java java.lang.Double para mapear una columna del dataset.
Línea 136	<field name="precio_medio" class="java.lang.Double"/> → Declara el field precio_medio con tipo Java java.lang.Double para mapear una columna del dataset.
Línea 137	<field name="primera_venta" class="java.lang.String"/> → Declara el field primera_venta con tipo Java java.lang.String para mapear una columna del dataset.
Línea 138	<field name="ultima_venta" class="java.lang.String"/> → Declara el field ultima_venta con tipo Java java.lang.String para mapear una columna del dataset.
Línea 139	<variable name="TotalUnidades" class="java.lang.Integer" calculation="Sum" resetType="Report"> → Declara la variable TotalUnidades con cálculo Sum y reinicio Report.
Línea 140	<variableExpression><![CDATA[\$F{unidades_vendidas}]]></variableExpression> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field unidades_vendidas.
Línea 141	</variable> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.

Línea 142	<code><variable name="TotalImporte" class="java.lang.Double" calculation="Sum" resetType="Report"></code> → Declara la variable TotalImporte con cálculo Sum y reinicio Report.
Línea 143	<code><variableExpression><![CDATA[{\$importe_total}]]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field importe_total.
Línea 144	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 145	<code><variable name="TotalPagina" class="java.lang.Double" calculation="Sum" resetType="Page"></code> → Declara la variable TotalPagina con cálculo Sum y reinicio Page.
Línea 146	<code><variableExpression><![CDATA[{\$importe_total}]]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field importe_total.
Línea 147	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 148	<code><variable name="PrecioMedio" class="java.lang.Double" calculation="Average" resetType="Report"></code> → Declara la variable PrecioMedio con cálculo Average y reinicio Report.
Línea 149	<code><variableExpression><![CDATA[{\$precio_medio}]]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field precio_medio.
Línea 150	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 151	<code><variable name="PrecioMaximo" class="java.lang.Double" calculation="Highest" resetType="Report"></code> → Declara la variable PrecioMaximo con cálculo Highest y reinicio Report.
Línea 152	<code><variableExpression><![CDATA[{\$precio_medio}]]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field precio_medio.
Línea 153	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 154	<code><variable name="NumeroLibros" class="java.lang.Integer" calculation="Count" resetType="Report"></code> → Declara la variable NumeroLibros con cálculo Count y reinicio Report.
Línea 155	<code><variableExpression><![CDATA[{\$titulo}]]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field titulo.
Línea 156	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 157	<code><variable name="ImporteConIva" class="java.lang.Double" calculation="Sum" resetType="Report"></code> → Declara la variable ImporteConIva con cálculo Sum y reinicio Report.
Línea 158	<code><variableExpression><![CDATA[{\$importe_total} == null {\$tipoIva} == null ? null : Double.valueOf(\$importe_total).doubleValue() * (1.0d + {\$tipoIva}.doubleValue())]]></v...></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field importe_total, field importe_total, parámetro tipoIva, parámetro tipoIva.
Línea 159	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 160	<code><variable name="GrupoUnidades" class="java.lang.Integer" calculation="Sum" resetType="Group" resetGroup="CategoriaGroup"></code> → Declara la variable GrupoUnidades con cálculo Sum y reinicio Group asociado a CategoriaGroup.
Línea 161	<code><variableExpression><![CDATA[{\$unidades_vendidas}]]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field unidades_vendidas.
Línea 162	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 163	<code><variable name="GrupoImporte" class="java.lang.Double" calculation="Sum" resetType="Group" resetGroup="CategoriaGroup"></code> → Declara la variable GrupoImporte con cálculo Sum y reinicio Group asociado a CategoriaGroup.
Línea 164	<code><variableExpression><![CDATA[{\$importe_total}]]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field importe_total.
Línea 165	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.

Línea 166	<code><variable name="GrupoLibros" class="java.lang.Integer" calculation="Count" resetType="Group" resetGroup="CategoriaGroup"></code> → Declara la variable GrupoLibros con cálculo Count y reinicio Group asociado a CategoriaGroup.
Línea 167	<code><variableExpression><![CDATA[$\\$F\{titulo\}$]]></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field titulo.
Línea 168	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 169	<code><group name="CategoriaGroup" isStartNewPage="false" isReprintHeaderOnEachPage="true" minHeightToStartNewPage="80"></code> → Declara el grupo CategoriaGroup y sus propiedades de paginación/reimpresión.
Línea 170	<code><groupExpression><![CDATA[$\\$F\{categoria\}$]]></code> → Define la clave que decide cuándo cambia el grupo mediante field categoria.
Línea 171	<code><groupHeader></code> → Abre la cabecera del grupo, que se emite cuando comienza cada nuevo valor de agrupación.
Línea 172	<code><band height="28"></code> → Define una banda de 28 puntos, reservando ese espacio para sus elementos.
Línea 173	<code><textField><reportElement x="0" y="2" width="555" height="22" style="M5GrupoCabecera"/><textFieldExpression><![CDATA["Categoría: " + $\\$F\{categoria\}$]]></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=2, ancho=555, alto=22 y aplica el estilo M5GrupoCabecera; evalúa la expresión usando field categoria.
Línea 174	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 175	<code></groupHeader></code> → Cierra groupHeader y vuelve al elemento padre de la jerarquía JRXML.
Línea 176	<code><groupFooter></code> → Abre el pie del grupo, donde se muestran los acumulados justo antes de cambiar de grupo.
Línea 177	<code><band height="34"></code> → Define una banda de 34 puntos, reservando ese espacio para sus elementos.
Línea 178	<code><textField><reportElement x="0" y="3" width="185" height="18" style="Dato"/><textFieldExpression><![CDATA["Libros del grupo: " + $\\$V\{GrupoLibros\}$]]></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=3, ancho=185, alto=18 y aplica el estilo Dato; evalúa la expresión usando variable GrupoLibros.
Línea 179	<code><textField><reportElement x="185" y="3" width="180" height="18" style="Dato"/><textFieldExpression><![CDATA["Unidades: " + ($\\$V\{GrupoUnidades\}$ == null ? 0 : $\\$V\{GrupoUnidades\}$)]></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=185, y=3, ancho=180, alto=18 y aplica el estilo Dato; evalúa la expresión usando variable GrupoUnidades, variable GrupoUnidades.
Línea 180	<code><textField><reportElement x="365" y="3" width="190" height="18" style="Dato"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA["Importe: " + new java.text.DecimalFormat("###,###.##").format($\\$F\{Importe\}$)]></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=365, y=3, ancho=190, alto=18 y aplica el estilo Dato; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable GrupoImporte, variable GrupoImporte.
Línea 181	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 182	<code></groupFooter></code> → Cierra groupFooter y vuelve al elemento padre de la jerarquía JRXML.
Línea 183	<code></group></code> → Finaliza la definición del grupo y sus bandas asociadas.
Línea 184	<code><background><band height="0"/></background></code> → Composición de la línea: encadena además <background>, <band> dentro de la misma jerarquía.
Línea 185	<code><title></code> → Abre la banda Title, emitida una sola vez al inicio del informe.
Línea 186	<code><band height="124"></code> → Define una banda de 124 puntos, reservando ese espacio para sus elementos.
Línea 187	<code><staticText></code> → Abre un elemento de texto literal; su contenido no depende de fields, parámetros ni variables.
Línea 188	<code><reportElement x="0" y="4" width="555" height="28" uuid="40000000-0000-4000-8000-000000000001" style="M5TituloPrincipal"/></code> → Posiciona el elemento en x=0, y=4, con ancho 555 y alto 28, aplicando el estilo M5TituloPrincipal.

Línea 189	<code><textElement textAlignment="Center" verticalAlignment="Middle" /></code> → Configura el formato interno del texto: alineación horizontal Center, alineación vertical Middle.
Línea 190	<code><text><![CDATA[Informe de Ventas - Agregación por Título]]></text></code> → Define el texto literal visible: Informe de Ventas - Agregación por Título.
Línea 191	<code></staticText></code> → Cierra <code>staticText</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 192	<code><staticText><reportElement x="0" y="38" width="110" height="18" uuid="40000000-0000-4000-8000-000000000002" /><text><![CDATA[Generado por:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=38, ancho=110, alto=18; muestra el literal 'Generado por:'.
Línea 193	<code><textField isBlankWhenNull="true"><reportElement x="110" y="38" width="160" height="18" uuid="40000000-0000-4000-8000-000000000003" /><textFieldExpression><![CDATA[{P{usuario}}]>...</textFieldExpression></code> → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en x=110, y=38, ancho=160, alto=18; evalúa la expresión usando parámetro usuario.
Línea 194	<code><staticText><reportElement x="300" y="38" width="80" height="18" uuid="40000000-0000-4000-8000-000000000004" /><text><![CDATA[Fecha:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=38, ancho=80, alto=18; muestra el literal 'Fecha:'.
Línea 195	<code><textField pattern="dd/MM/yyyy"><reportElement x="380" y="38" width="175" height="18" uuid="40000000-0000-4000-8000-000000000005" /><textFieldExpression><![CDATA[{P{fechaInforme}}]>...</textFieldExpression></code> → Composición de la línea: crea un <code>textField</code> dinámico con patrón dd/MM/yyyy; lo posiciona en x=380, y=38, ancho=175, alto=18; evalúa la expresión usando parámetro fechaInforme.
Línea 196	<code><staticText><reportElement x="0" y="62" width="100" height="18" uuid="40000000-0000-4000-8000-000000000006" /><text><![CDATA[Departamento:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=62, ancho=100, alto=18; muestra el literal 'Departamento:'.
Línea 197	<code><textField isBlankWhenNull="true"><reportElement x="100" y="62" width="170" height="18" uuid="40000000-0000-4000-8000-000000000007" /><textFieldExpression><![CDATA[{P{departament...}}]>...</textFieldExpression></code> → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en x=100, y=62, ancho=170, alto=18; evalúa la expresión usando parámetro departamento.
Línea 198	<code><staticText><reportElement x="300" y="62" width="70" height="18" uuid="40000000-0000-4000-8000-000000000008" /><text><![CDATA[Periodo:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=62, ancho=70, alto=18; muestra el literal 'Periodo:'.
Línea 199	<code><textField isBlankWhenNull="true"><reportElement x="370" y="62" width="185" height="18" uuid="40000000-0000-4000-8000-000000000009" /><textFieldExpression><![CDATA[{P{periodo}}]>...</textFieldExpression></code> → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en x=370, y=62, ancho=185, alto=18; evalúa la expresión usando parámetro periodo.
Línea 200	<code><staticText><reportElement x="0" y="86" width="100" height="18" uuid="40000000-0000-4000-8000-000000000010" /><text><![CDATA[Búsqueda:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=86, ancho=100, alto=18; muestra el literal 'Búsqueda:'.
Línea 201	<code><textField isBlankWhenNull="true"><reportElement x="100" y="86" width="170" height="18" uuid="40000000-0000-4000-8000-000000000011" /><textFieldExpression><![CDATA[{P{textoBusque...}}]>...</textFieldExpression></code> → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en x=100, y=86, ancho=170, alto=18; evalúa la expresión usando parámetro textoBusqueda, parámetro textoBusqueda, parámetro textoBusqueda.
Línea 202	<code><staticText><reportElement x="300" y="86" width="90" height="18" uuid="40000000-0000-4000-8000-000000000012" /><text><![CDATA[Categorías:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=86, ancho=90, alto=18; muestra el literal 'Categorías:'.
Línea 203	<code><textField textAdjust="StretchHeight"><reportElement x="390" y="86" width="165" height="34" uuid="40000000-0000-4000-8000-000000000013" /><textFieldExpression><![CDATA[String.val...>...</textFieldExpression></code> → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en x=390, y=86, ancho=165, alto=34; evalúa la expresión usando parámetro categoriasLista.

Línea 204	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 205	<code></title></code> → Cierra title y vuelve al elemento padre de la jerarquía JRXML.
Línea 206	<code><columnHeader></code> → Abre Column Header, repetida al comienzo de cada columna/página según la paginación.
Línea 207	<code><band height="62"></code> → Define una banda de 62 puntos, reservando ese espacio para sus elementos.
Línea 208	<code><staticText><reportElement x="0" y="2" width="215" height="18" uuid="41000000-0000-4000-8000-000000000001" style="Cabecera"/><text><![CDATA[Título]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=2, ancho=215, alto=18 y aplica el estilo Cabecera; muestra el literal 'Título'.
Línea 209	<code><staticText><reportElement x="215" y="2" width="55" height="18" uuid="41000000-0000-4000-8000-000000000002" style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[...]</code> → Composición de la línea: crea un texto literal; lo posiciona en x=215, y=2, ancho=55, alto=18 y aplica el estilo Cabecera; configura la alineación del texto (horizontal Right); muestra el literal 'Unid.'.
Línea 210	<code><staticText><reportElement x="280" y="2" width="90" height="18" uuid="41000000-0000-4000-8000-000000000003" style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[...]</code> → Composición de la línea: crea un texto literal; lo posiciona en x=280, y=2, ancho=90, alto=18 y aplica el estilo Cabecera; configura la alineación del texto (horizontal Right); muestra el literal 'Importe'.
Línea 211	<code><staticText><reportElement x="380" y="2" width="65" height="18" uuid="41000000-0000-4000-8000-000000000004" style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[...]</code> → Composición de la línea: crea un texto literal; lo posiciona en x=380, y=2, ancho=65, alto=18 y aplica el estilo Cabecera; configura la alineación del texto (horizontal Right); muestra el literal 'Precio med.'.
Línea 212	<code><staticText><reportElement x="455" y="2" width="100" height="18" uuid="41000000-0000-4000-8000-000000000005" style="Cabecera"/><text><![CDATA[Categoría]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=455, y=2, ancho=100, alto=18 y aplica el estilo Cabecera; muestra el literal 'Categoría'.
Línea 213	<code><staticText><reportElement x="0" y="24" width="130" height="18" uuid="41000000-0000-4000-8000-000000000006" style="Cabecera"/><text><![CDATA[Primera venta]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=24, ancho=130, alto=18 y aplica el estilo Cabecera; muestra el literal 'Primera venta'.
Línea 214	<code><staticText><reportElement x="130" y="24" width="130" height="18" uuid="41000000-0000-4000-8000-000000000007" style="Cabecera"/><text><![CDATA[Última venta]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=130, y=24, ancho=130, alto=18 y aplica el estilo Cabecera; muestra el literal 'Última venta'.
Línea 215	<code><staticText><reportElement x="260" y="24" width="160" height="18" uuid="41000000-0000-4000-8000-000000000008" style="Cabecera"/><text><![CDATA[Periodo de ventas]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=260, y=24, ancho=160, alto=18 y aplica el estilo Cabecera; muestra el literal 'Periodo de ventas'.
Línea 216	<code><staticText></code> → Abre un elemento de texto literal; su contenido no depende de fields, parámetros ni variables.
Línea 217	<code><reportElement x="420" y="24" width="135" height="18" uuid="41000000-0000-4000-8000-000000000009" style="Cabecera"></code> → Posiciona el elemento en x=420, y=24, con ancho 135 y alto 18, aplicando el estilo Cabecera.
Línea 218	<code><printWhenExpression><![CDATA[Boolean.TRUE.equals(\$P{mostrarDetalle})]]></printWhenExpression></code> → Evalúa una condición booleana para decidir si la banda o elemento se imprime usando parámetro mostrarDetalle.
Línea 219	<code></reportElement></code> → Cierra reportElement y vuelve al elemento padre de la jerarquía JRXML.
Línea 220	<code><textElement textAlignment="Right"/></code> → Configura el formato interno del texto: alineación horizontal Right.
Línea 221	<code><text><![CDATA[Importe con IVA]]></text></code> → Define el texto literal visible: Importe con IVA.

Línea 222	<code></staticText></code> → Cierra <code>staticText</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 223	<code></band></code> → Cierra <code>band</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 224	<code></columnHeader></code> → Cierra <code>columnHeader</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 225	<code><detail></code> → Abre <code>Detail</code> , la sección que se repite para cada registro del dataset principal.
Línea 226	<code><band height="82" splitType="Stretch"></code> → Define una banda de 82 puntos con <code>splitType Stretch</code> , reservando ese espacio para sus elementos.
Línea 227	<code><textField textAdjust="StretchHeight"><reportElement x="0" y="0" width="215" height="20" uuid="42000000-0000-4000-8000-000000000001" style="Dato"/><textFieldExpression><![CDATA[... → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=0, y=0</code>, ancho=215, alto=20 y aplica el estilo <code>Dato</code>; evalúa la expresión usando <code>field titulo</code>.</code>
Línea 228	<code><textField isBlankWhenNull="true"><reportElement x="215" y="0" width="55" height="20" uuid="42000000-0000-4000-8000-000000000002" style="UnidadesCondicional"/><textElement textA... → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=215, y=0</code>, ancho=55, alto=20 y aplica el estilo <code>UnidadesCondicional</code>; configura la alineación del texto (<code>horizontal Right</code>); evalúa la expresión usando <code>field unidades_vendidas</code>.</code>
Línea 229	<code><textField pattern="#.##0.00 €" isBlankWhenNull="true"><reportElement x="280" y="0" width="90" height="20" uuid="42000000-0000-4000-8000-000000000003" style="Dato"/><textElement... → Composición de la línea: crea un <code>textField</code> dinámico con patrón <code>#.##0.00 €</code>; lo posiciona en <code>x=280, y=0</code>, ancho=90, alto=20 y aplica el estilo <code>Dato</code>; configura la alineación del texto (<code>horizontal Right</code>); evalúa la expresión usando <code>field importe_total</code>.</code>
Línea 230	<code><textField isBlankWhenNull="true"><reportElement x="380" y="0" width="65" height="20" uuid="42000000-0000-4000-8000-000000000004" style="Dato"/><textElement textAlignment="Right"... → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=380, y=0</code>, ancho=65, alto=20 y aplica el estilo <code>Dato</code>; configura la alineación del texto (<code>horizontal Right</code>); evalúa la expresión usando <code>field precio_medio</code>, <code>field precio_medio</code>.</code>
Línea 231	<code><textField isBlankWhenNull="true"><reportElement x="455" y="0" width="100" height="20" uuid="42000000-0000-4000-8000-000000000005" style="Dato"/><textFieldExpression><![CDATA[\$F... → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=455, y=0</code>, ancho=100, alto=20 y aplica el estilo <code>Dato</code>; evalúa la expresión usando <code>field categoria</code>.</code>
Línea 232	<code><textField isBlankWhenNull="true"><reportElement x="0" y="24" width="130" height="18" uuid="42000000-0000-4000-8000-000000000006" style="Dato"/><textFieldExpression><![CDATA[\$F{... → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=0, y=24</code>, ancho=130, alto=18 y aplica el estilo <code>Dato</code>; evalúa la expresión usando <code>field primera_venta</code>.</code>
Línea 233	<code><textField isBlankWhenNull="true"><reportElement x="130" y="24" width="130" height="18" uuid="42000000-0000-4000-8000-000000000007" style="Dato"/><textFieldExpression><![CDATA[\$... → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=130, y=24</code>, ancho=130, alto=18 y aplica el estilo <code>Dato</code>; evalúa la expresión usando <code>field ultima_venta</code>.</code>
Línea 234	<code><textField><reportElement x="260" y="24" width="160" height="18" uuid="42000000-0000-4000-8000-000000000008" style="Dato"/><textElement textAlignment="Center"/><textFieldExpress... → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=260, y=24</code>, ancho=160, alto=18 y aplica el estilo <code>Dato</code>; configura la alineación del texto (<code>horizontal Center</code>); evalúa la expresión usando <code>field primera_venta</code>, <code>field primera_venta</code>, <code>field ultima_venta</code>.</code>
Línea 235	<code><textField pattern="#.##0.00 €" isBlankWhenNull="true"></code> → Abre un <code>textField</code> dinámico con formato <code>#.##0.00 €</code> , cuyo valor se obtiene de su <code>textFieldExpression</code> .
Línea 236	<code><reportElement x="420" y="24" width="135" height="18" uuid="42000000-0000-4000-8000-000000000009" style="Dato"></code> → Posiciona el elemento en <code>x=420, y=24</code> , con ancho 135 y alto 18, aplicando el estilo <code>Dato</code> .
Línea 237	<code><printWhenExpression><![CDATA[Boolean.TRUE.equals(\$P{mostrarDetalle})]]></printWhenExpression></code> → Evalúa una condición booleana para decidir si la banda o elemento se imprime usando parámetro <code>mostrarDetalle</code> .
Línea 238	<code></reportElement></code> → Cierra <code>reportElement</code> y vuelve al elemento padre de la jerarquía JRXML.

Línea 239	<code><textElement textAlignment="Right"/></code> → Configura el formato interno del texto: alineación horizontal Right.
Línea 240	<code><textFieldExpression><![CDATA[\${F{importe_total}} == null \${P{tipoIva}} == null ? null : Double.valueOf(\${F{importe_total}}.doubleValue() * (1.0d + \${P{tipoIva}}.doubleValue()))]]></...></code> → Calcula el valor mostrado por el textField mediante una expresión Java que usa field importe_total, field importe_total, parámetro tipoIva, parámetro tipoIva.
Línea 241	<code></textField></code> → Cierra textField y vuelve al elemento padre de la jerarquía JRXML.
Línea 242	<code><textField><reportElement x="0" y="48" width="105" height="18" uuid="42000000-0000-4000-8000-000000000010" style="Dato"/><textFieldExpression><![CDATA[\${F{unidades_vendidas}} == n...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=48, ancho=105, alto=18 y aplica el estilo Dato; evalúa la expresión usando field unidades_vendidas, field unidades_vendidas, field unidades_vendidas.
Línea 243	<code><textField><reportElement x="105" y="48" width="185" height="18" uuid="42000000-0000-4000-8000-000000000011" style="Dato"/><textFieldExpression><![CDATA[\${F{titulo}} == null ? "" ...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=105, y=48, ancho=185, alto=18 y aplica el estilo Dato; evalúa la expresión usando field titulo, field titulo.
Línea 244	<code><textField><reportElement x="290" y="48" width="80" height="18" uuid="42000000-0000-4000-8000-000000000012" style="Dato"/><textElement textAlignment="Right"/><textFieldExpressio...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=290, y=48, ancho=80, alto=18 y aplica el estilo Dato; configura la alineación del texto (horizontal Right); evalúa la expresión usando field precio_medio, field precio_medio.
Línea 245	<code><textField><reportElement x="370" y="48" width="90" height="18" uuid="42000000-0000-4000-8000-000000000013" style="Dato"/><textElement textAlignment="Center"/><textFieldExpressio...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=370, y=48, ancho=90, alto=18 y aplica el estilo Dato; configura la alineación del texto (horizontal Center); evalúa la expresión usando field primera_venta, field ultima_venta, field primera_venta, field ultima_venta.
Línea 246	<code><textField><reportElement x="460" y="48" width="95" height="18" uuid="42000000-0000-4000-8000-000000000014" style="Dato"/><textElement textAlignment="Right"/><textFieldExpressio...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=460, y=48, ancho=95, alto=18 y aplica el estilo Dato; configura la alineación del texto (horizontal Right); evalúa la expresión usando field unidades_vendidas, field unidades_vendidas, parámetro umbralUnidades, parámetro umbralUnidades.
Línea 247	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 248	<code><band height="14"></code> → Define una banda de 14 puntos, reservando ese espacio para sus elementos.
Línea 249	<code><printWhenExpression><![CDATA[\${F{unidades_vendidas}} != null && \${P{umbralUnidades}} != null && \${F{unidades_vendidas}}.intValue() >= \${P{umbralUnidades}}.intValue()]]></printWhenExpre...></code> → Evalúa una condición booleana para decidir si la banda o elemento se imprime usando field unidades_vendidas, field unidades_vendidas, parámetro umbralUnidades, parámetro umbralUnidades.
Línea 250	<code><textField><reportElement x="0" y="0" width="555" height="12" uuid="42000000-0000-4000-8000-000000000015"/><textElement textAlignment="Center"></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=0, ancho=555, alto=12; configura la alineación del texto (horizontal Center); configura la fuente (familia DejaVu Sans, tamaño 8, negrita); evalúa la expresión usando field titulo, parámetro umbralUnidades.
Línea 251	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 252	<code><band height="88" splitType="Stretch"></code> → Define una banda de 88 puntos con splitType Stretch, reservando ese espacio para sus elementos.
Línea 253	<code><printWhenExpression><![CDATA[\${F{unidades_vendidas}} != null]]></printWhenExpression></code> → Evalúa una condición booleana para decidir si la banda o elemento se imprime usando field unidades_vendidas.
Línea 254	<code><staticText></code> → Abre un elemento de texto literal; su contenido no depende de fields, parámetros ni variables.
Línea 255	<code><reportElement x="0" y="2" width="555" height="16" style="Cabecera"/></code> → Posiciona el elemento en x=0, y=2, con ancho 555 y alto 16, aplicando el estilo Cabecera.

Línea 256	<code><text><![CDATA[Detalle de ventas]]></text></code> → Define el texto literal visible: Detalle de ventas.
Línea 257	<code></staticText></code> → Cierra <code>staticText</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 258	<code><subreport></code> → Abre el componente subreport que ejecuta un informe hijo dentro de la banda del maestro.
Línea 259	<code><reportElement x="0" y="22" width="555" height="60" isRemoveLineWhenBlank="true"/></code> → Posiciona el elemento en x=0, y=22, con ancho 555 y alto 60, y elimina su línea cuando queda vacío.
Línea 260	<code><subreportParameter name="tituloLibro"></code> → Declara el parámetro del subreporte <code>tituloLibro</code> que recibirá un valor del informe maestro.
Línea 261	<code><subreportParameterExpression><![CDATA[\${titulo}]]></subreportParameterExpression></code> → Calcula el valor enviado al parámetro del subreporte a partir de field <code>titulo</code> .
Línea 262	<code></subreportParameter></code> → Cierra <code>subreportParameter</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 263	<code><connectionExpression><![CDATA[\${REPORT_CONNECTION}]]></connectionExpression></code> → Entrega al subreporte/subdataset la misma <code>REPORT_CONNECTION</code> usada por el informe maestro, evitando abrir otra conexión.
Línea 264	<code><subreportExpression><![CDATA["reports/subinforme_ventas_detalle.jasper"]]></subreportExpression></code> → Devuelve la ruta del archivo <code>subinforme_ventas_detalle.jasper</code> que JasperReports cargará como informe hijo.
Línea 265	<code></subreport></code> → Finaliza el componente de subreporte.
Línea 266	<code></band></code> → Cierra <code>band</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 267	<code><band height="104" splitType="Stretch"></code> → Define una banda de 104 puntos con <code>splitType Stretch</code> , reservando ese espacio para sus elementos.
Línea 268	<code><printWhenExpression><![CDATA[\${unidades_vendidas} != null]]></printWhenExpression></code> → Evalúa una condición booleana para decidir si la banda o elemento se imprime usando field <code>unidades_vendidas</code> .
Línea 269	<code><staticText></code> → Abre un elemento de texto literal; su contenido no depende de fields, parámetros ni variables.
Línea 270	<code><reportElement x="0" y="2" width="555" height="16" style="Cabecera"/></code> → Posiciona el elemento en x=0, y=2, con ancho 555 y alto 16, aplicando el estilo <code>Cabecera</code> .
Línea 271	<code><text><![CDATA[Top 3 ventas por cantidad]]></text></code> → Define el texto literal visible: Top 3 ventas por cantidad.
Línea 272	<code></staticText></code> → Cierra <code>staticText</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 273	<code><componentElement></code> → Abre un contenedor de componentes extendidos; en este checkpoint contiene la tabla <code>c:table</code> .
Línea 274	<code><reportElement x="0" y="22" width="555" height="76"/></code> → Posiciona el elemento en x=0, y=22, con ancho 555 y alto 76.
Línea 275	<code><c:table xmlns:c="http://jasperreports.sourceforge.net/jasperreports/components"></code> → Abre la tabla del namespace de componentes JasperReports; sus columnas usan un <code>datasetRun</code> independiente.
Línea 276	<code>xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports/components http://jasperreports.sourceforge.net/xsd/components.xsd"></code> → Relaciona el namespace de JasperReports con su XSD para que Studio y el compilador validen la estructura.
Línea 277	<code><datasetRun subDataset="DatasetTopVentas"></code> → Asocia el componente con el subdataset <code>DatasetTopVentas</code> para ejecutar su consulta.
Línea 278	<code><datasetParameter name="tituloLibro"></code> → Declara el parámetro <code>tituloLibro</code> que se enviará al subdataset de la tabla.
Línea 279	<code><datasetParameterExpression><![CDATA[\${titulo}]]></datasetParameterExpression></code> → Calcula el valor enviado al parámetro del subdataset desde field <code>titulo</code> .
Línea 280	<code></datasetParameter></code> → Cierra <code>datasetParameter</code> y vuelve al elemento padre de la jerarquía JRXML.

Línea 281	<code><connectionExpression><![CDATA[\$P{REPORT_CONNECTION}]]></connectionExpression></code> → Entrega al subreporte/subdataset la misma REPORT_CONNECTION usada por el informe maestro, evitando abrir otra conexión.
Línea 282	<code></datasetRun></code> → Cierra datasetRun y vuelve al elemento padre de la jerarquía JRXML.
Línea 283	<code><c:column width="255"></code> → Declara una columna de tabla de 255 puntos de ancho.
Línea 284	<code><c:columnHeader style="M5TablaCabecera" height="20"><staticText><reportElement x="0" y="0" width="255" height="20"/><text><![CDATA[Fecha]]></text></staticText></c:columnHeader></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=0, ancho=255, alto=20 y aplica el estilo M5TablaCabecera; muestra el literal 'Fecha'; define una celda de cabecera de la tabla.
Línea 285	<code><c:detailCell style="M5TablaDetalle" height="18"><textField><reportElement x="0" y="0" width="255" height="18"/><textFieldExpression><![CDATA[\${fecha_venta}]]></textFieldExpres...</code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=0, ancho=255, alto=18 y aplica el estilo M5TablaDetalle; evalúa la expresión usando field fecha_venta; define una celda repetida de detalle de la tabla.
Línea 286	<code></c:column></code> → Cierra c:column y vuelve al elemento padre de la jerarquía JRXML.
Línea 287	<code><c:column width="100"></code> → Declara una columna de tabla de 100 puntos de ancho.
Línea 288	<code><c:columnHeader style="M5TablaCabecera" height="20"><staticText><reportElement x="0" y="0" width="100" height="20"/><textElement textAlignment="Right"/><text><![CDATA[Cantidad]]...</code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=0, ancho=100, alto=20 y aplica el estilo M5TablaCabecera; configura la alineación del texto (horizontal Right); muestra el literal 'Cantidad'; define una celda de cabecera de la tabla.
Línea 289	<code><c:detailCell style="M5TablaDetalle" height="18"><textField><reportElement x="0" y="0" width="100" height="18"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[...</code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=0, ancho=100, alto=18 y aplica el estilo M5TablaDetalle; configura la alineación del texto (horizontal Right); evalúa la expresión usando field cantidad; define una celda repetida de detalle de la tabla.
Línea 290	<code></c:column></code> → Cierra c:column y vuelve al elemento padre de la jerarquía JRXML.
Línea 291	<code><c:column width="200"></code> → Declara una columna de tabla de 200 puntos de ancho.
Línea 292	<code><c:columnHeader style="M5TablaCabecera" height="20"><staticText><reportElement x="0" y="0" width="200" height="20"/><textElement textAlignment="Right"/><text><![CDATA[Precio uni...</code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=0, ancho=200, alto=20 y aplica el estilo M5TablaCabecera; configura la alineación del texto (horizontal Right); muestra el literal 'Precio unitario'; define una celda de cabecera de la tabla.
Línea 293	<code><c:detailCell style="M5TablaDetalle" height="18"><textField pattern="#.##0.00 €"><reportElement x="0" y="0" width="200" height="18"/><textElement textAlignment="Right"/><textFie...</code> → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=0, y=0, ancho=200, alto=18 y aplica el estilo M5TablaDetalle; configura la alineación del texto (horizontal Right); evalúa la expresión usando field precio_unitario; define una celda repetida de detalle de la tabla.
Línea 294	<code></c:column></code> → Cierra c:column y vuelve al elemento padre de la jerarquía JRXML.
Línea 295	<code></c:table></code> → Finaliza la tabla integrada.
Línea 296	<code></componentElement></code> → Cierra componentElement y vuelve al elemento padre de la jerarquía JRXML.
Línea 297	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 298	<code></detail></code> → Finaliza la sección Detail del informe.
Línea 299	<code><pageFooter></code> → Abre Page Footer, emitido al pie de cada página.
Línea 300	<code><band height="62"></code> → Define una banda de 62 puntos, reservando ese espacio para sus elementos.
Línea 301	<code><staticText><reportElement x="0" y="4" width="120" height="15" uuid="43000000-0000-4000-8000-000000000001"/><text><![CDATA[Total de títulos:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=4, ancho=120, alto=15; muestra el literal 'Total de títulos:'.

Línea 302	<pre><textField evaluationTime="Report"><reportElement x="120" y="4" width="60" height="15" uuid="43000000-0000-4000-8000-000000000002"/><textFieldExpression><![CDATA[{\$V{REPORT_COUNT... → Composición de la línea: crea un textField dinámico; lo posiciona en x=120, y=4, ancho=60, alto=15; evalúa la expresión usando variable REPORT_COUNT.</pre>
Línea 303	<pre><textField><reportElement x="190" y="28" width="180" height="15" uuid="43000000-0000-4000-8000-000000000003"/><textElement textAlignment="Right"/ ><textFieldExpression><![CDATA["... → Composición de la línea: crea un textField dinámico; lo posiciona en x=190, y=28, ancho=180, alto=15; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable PAGE_NUMBER.</pre>
Línea 304	<pre><textField evaluationTime="Report"><reportElement x="375" y="28" width="35" height="15" uuid="43000000-0000-4000-8000-000000000004"/><textFieldExpression><![CDATA[{\$V{PAGE_NUMBER... → Composición de la línea: crea un textField dinámico; lo posiciona en x=375, y=28, ancho=35, alto=15; evalúa la expresión usando variable PAGE_NUMBER.</pre>
Línea 305	<pre><staticText><reportElement x="300" y="4" width="120" height="15" uuid="43000000-0000-4000-8000-000000000005"/><text><![CDATA[Subtotal página:]]></text></ staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=4, ancho=120, alto=15; muestra el literal 'Subtotal página:'.</pre>
Línea 306	<pre><textField pattern="#.##0.00 €"><reportElement x="420" y="4" width="135" height="15" uuid="43000000-0000-4000-8000-000000000006"/><textElement textAlignment="Right"/ ><textFieldE... → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=420, y=4, ancho=135, alto=15; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable TotalPagina.</pre>
Línea 307	<pre></band></pre> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 308	<pre></pageFooter></pre> → Cierra pageFooter y vuelve al elemento padre de la jerarquía JRXML.
Línea 309	<pre><summary></pre> → Abre Summary, emitido una sola vez después del último registro.
Línea 310	<pre><band height="700"></pre> → Define una banda de 700 puntos, reservando ese espacio para sus elementos.
Línea 311	<pre><staticText><reportElement x="0" y="5" width="205" height="18" uuid="44000000-0000-4000-8000-000000000001"/><text><![CDATA[Total de unidades vendidas:]]></ text></staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=5, ancho=205, alto=18; muestra el literal 'Total de unidades vendidas:'.</pre>
Línea 312	<pre><textField><reportElement x="205" y="5" width="80" height="18" uuid="44000000-0000-4000-8000-000000000002"/><textElement textAlignment="Right"/ ><textFieldExpression><![CDATA[{\$V{... → Composición de la línea: crea un textField dinámico; lo posiciona en x=205, y=5, ancho=80, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable TotalUnidades.</pre>
Línea 313	<pre><staticText><reportElement x="300" y="5" width="120" height="18" uuid="44000000-0000-4000-8000-000000000003"/><text><![CDATA[Importe total:]]></text></ staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=5, ancho=120, alto=18; muestra el literal 'Importe total:'.</pre>
Línea 314	<pre><textField pattern="#.##0.00 €"><reportElement x="420" y="5" width="135" height="18" uuid="44000000-0000-4000-8000-000000000004"/><textElement textAlignment="Right"/ ><textFieldE... → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=420, y=5, ancho=135, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable TotalImporte.</pre>
Línea 315	<pre><staticText><reportElement x="0" y="30" width="205" height="18" uuid="44000000-0000-4000-8000-000000000005"/><text><![CDATA[Precio medio agregado:]]></ text></staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=30, ancho=205, alto=18; muestra el literal 'Precio medio agregado:'.</pre>
Línea 316	<pre><textField pattern="#.##0.00 €"><reportElement x="205" y="30" width="80" height="18" uuid="44000000-0000-4000-8000-000000000006"/><textElement textAlignment="Right"/ ><textFieldE... → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=205, y=30, ancho=80, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable PrecioMedio.</pre>

Línea 317	<code><staticText><reportElement x="300" y="30" width="120" height="18" uuid="44000000-0000-4000-8000-000000000007"/><text><![CDATA[Precio máximo:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=30, ancho=120, alto=18; muestra el literal 'Precio máximo:'.
Línea 318	<code><textField pattern="#.##0.00 €"><reportElement x="420" y="30" width="135" height="18" uuid="44000000-0000-4000-8000-000000000008"/><textElement textAlignment="Right"/><textField...></code> → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=420, y=30, ancho=135, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable PrecioMaximo.
Línea 319	<code><staticText><reportElement x="0" y="55" width="205" height="18" uuid="44000000-0000-4000-8000-000000000009"/><text><![CDATA[Número de libros:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=55, ancho=205, alto=18; muestra el literal 'Número de libros:'.
Línea 320	<code><textField><reportElement x="205" y="55" width="80" height="18" uuid="44000000-0000-4000-8000-000000000010"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[\$V...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=205, y=55, ancho=80, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable NumeroLibros.
Línea 321	<code><staticText><reportElement x="300" y="55" width="120" height="18" uuid="44000000-0000-4000-8000-000000000011"/><text><![CDATA[Importe con IVA:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=55, ancho=120, alto=18; muestra el literal 'Importe con IVA:'.
Línea 322	<code><textField pattern="#.##0.00 €"><reportElement x="420" y="55" width="135" height="18" uuid="44000000-0000-4000-8000-000000000012"/><textElement textAlignment="Right"/><textField...></code> → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=420, y=55, ancho=135, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable ImporteConIva.
Línea 323	<code><textField><reportElement x="0" y="80" width="555" height="18" uuid="44000000-0000-4000-8000-000000000013"/><textElement textAlignment="Center"/><textFieldExpression><![CDATA[St...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=80, ancho=555, alto=18; configura la alineación del texto (horizontal Center); evalúa la expresión usando variable NumeroLibros, variable TotalUnidades, variable TotalImporte.
Línea 324	<code><textField><reportElement x="0" y="103" width="350" height="18" uuid="44000000-0000-4000-8000-000000000014"/><textElement textAlignment="Center"></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=103, ancho=350, alto=18; configura la alineación del texto (horizontal Center); configura la fuente (familia DejaVu Sans, tamaño 10, negrita); evalúa la expresión usando parámetro umbralUnidades, parámetro umbralUnidades, variable TotalUnidades, variable TotalUnidades.
Línea 325	<code><textField><reportElement x="360" y="103" width="195" height="18" uuid="44000000-0000-4000-8000-000000000015"/><textFieldExpression><![CDATA["Resultados encontrados: " + \$V{REPO...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=360, y=103, ancho=195, alto=18; evalúa la expresión usando variable REPORT_COUNT.
Línea 326	<code><staticText><reportElement x="0" y="140" width="555" height="20" style="Cabecera"/><text><![CDATA[Ventas por categoría – importe]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=140, ancho=555, alto=20 y aplica el estilo Cabecera; muestra el literal 'Ventas por categoría — importe'.
Línea 327	<code><barChart></code> → Abre el gráfico de barras nativo de JasperReports que se integrará en el informe maestro.
Línea 328	<code><chart></code> → Abre la configuración común del gráfico: geometría, título, subtítulo y leyenda.
Línea 329	<code><reportElement x="0" y="165" width="555" height="250"/></code> → Posiciona el elemento en x=0, y=165, con ancho 555 y alto 250.
Línea 330	<code><chartTitle><titleExpression><![CDATA["Ventas por categoría"]]></titleExpression></chartTitle></code> → Composición de la línea: encadena además <chartTitle>, <titleExpression> dentro de la misma jerarquía.
Línea 331	<code><chartSubtitle/></code> → Declara el subtítulo del gráfico; en este checkpoint queda vacío.

Línea 332	<code><chartLegend position="Bottom"/></code> → Configura la leyenda del gráfico en la posición Bottom.
Línea 333	<code></chart></code> → Cierra chart y vuelve al elemento padre de la jerarquía JRXML.
Línea 334	<code><categoryDataset></code> → Abre el dataset categórico que alimenta al gráfico con serie, categoría y valor.
Línea 335	<code><dataset></code> → Abre el contenedor de ejecución de datos del componente actual.
Línea 336	<code><datasetRun subDataset="DatasetVentasPorCategoria"></code> → Asocia el componente con el subdataset DatasetVentasPorCategoria para ejecutar su consulta.
Línea 337	<code><connectionExpression><![CDATA[\${REPORT_CONNECTION}]]></connectionExpression></code> → Entrega al subreporte/subdataset la misma REPORT_CONNECTION usada por el informe maestro, evitando abrir otra conexión.
Línea 338	<code></datasetRun></code> → Cierra datasetRun y vuelve al elemento padre de la jerarquía JRXML.
Línea 339	<code></dataset></code> → Cierra dataset y vuelve al elemento padre de la jerarquía JRXML.
Línea 340	<code><categorySeries></code> → Abre una serie del dataset categórico; cada fila del subdataset aportará categoría y valor.
Línea 341	<code><seriesExpression><![CDATA["Importe"]]></seriesExpression></code> → Define el nombre lógico de la serie que aparecerá en la leyenda.
Línea 342	<code><categoryExpression><![CDATA[\${categoria_grafico}]]></categoryExpression></code> → Define la categoría del eje X a partir de field categoria_grafico.
Línea 343	<code><valueExpression><![CDATA[\${importe_categoria}]]></valueExpression></code> → Define el valor numérico representado por cada barra a partir de field importe_categoria.
Línea 344	<code></categorySeries></code> → Cierra categorySeries y vuelve al elemento padre de la jerarquía JRXML.
Línea 345	<code></categoryDataset></code> → Cierra categoryDataset y vuelve al elemento padre de la jerarquía JRXML.
Línea 346	<code><barPlot></code> → Abre el plot específico del gráfico de barras, donde se configuran etiquetas y ejes.
Línea 347	<code><plot/></code> → Declara el bloque base del plot; mantiene la configuración visual por defecto del checkpoint.
Línea 348	<code><itemLabel/></code> → Habilita el bloque de configuración de etiquetas de los ítems/barras.
Línea 349	<code><categoryAxisFormat><axisFormat/></categoryAxisFormat></code> → Composición de la línea: encadena además <code><categoryAxisFormat></code> , <code><axisFormat></code> dentro de la misma jerarquía.
Línea 350	<code><valueAxisFormat><axisFormat/></valueAxisFormat></code> → Composición de la línea: encadena además <code><valueAxisFormat></code> , <code><axisFormat></code> dentro de la misma jerarquía.
Línea 351	<code></barPlot></code> → Cierra barPlot y vuelve al elemento padre de la jerarquía JRXML.
Línea 352	<code></barChart></code> → Finaliza el gráfico de barras.
Línea 353	<code><staticText><reportElement x="0" y="430" width="555" height="20" style="Cabecera"/><text><![CDATA[Ventas por categoría y año]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=430, ancho=555, alto=20 y aplica el estilo Cabecera; muestra el literal 'Ventas por categoría y año'.
Línea 354	<code><crosstab></code> → Abre la tabla cruzada nativa que genera dinámicamente la matriz de filas, columnas, medidas y totales.
Línea 355	<code><reportElement x="0" y="455" width="555" height="225"/></code> → Posiciona el elemento en x=0, y=455, con ancho 555 y alto 225.
Línea 356	<code><crosstabDataset></code> → Abre la fuente de datos específica del crosstab.
Línea 357	<code><dataset></code> → Abre el contenedor de ejecución de datos del componente actual.
Línea 358	<code><datasetRun subDataset="DatasetCrosstabVentas"></code> → Asocia el componente con el subdataset DatasetCrosstabVentas para ejecutar su consulta.

Línea 359	<code><connectionExpression><![CDATA[\$P{REPORT_CONNECTION}]]></connectionExpression></code> → Entrega al subreporte/subdataset la misma REPORT_CONNECTION usada por el informe maestro, evitando abrir otra conexión.
Línea 360	<code></datasetRun></code> → Cierra datasetRun y vuelve al elemento padre de la jerarquía JRXML.
Línea 361	<code></dataset></code> → Cierra dataset y vuelve al elemento padre de la jerarquía JRXML.
Línea 362	<code></crosstabDataset></code> → Cierra crosstabDataset y vuelve al elemento padre de la jerarquía JRXML.
Línea 363	<code><rowGroup name="CategoriaCross" width="150" totalPosition="End"></code> → Declara el grupo de filas CategoriaCross, ancho 150 y total en End.
Línea 364	<code><bucket class="java.lang.String"><bucketExpression><![CDATA[\$F{categoria_cross}]]></bucketExpression></bucket></code> → Composición de la línea: define la clave de agrupación del bucket desde field categoria_cross; encadena además <bucket> dentro de la misma jerarquía.
Línea 365	<code><crosstabRowHeader><cellContents style="M5CrosstabCabecera"><textField><reportElement x="0" y="0" width="150" height="34"><textElement verticalAlignment="Middle"><textFieldExp...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=0, ancho=150, alto=34 y aplica el estilo M5CrosstabCabecera; configura la alineación del texto (vertical Middle); evalúa la expresión usando variable CategoriaCross; abre el contenido de celda con estilo M5CrosstabCabecera; define la cabecera de fila del crosstab.
Línea 366	<code><crosstabTotalRowHeader><cellContents style="M5CrosstabTotal"><staticText><reportElement x="0" y="0" width="150" height="34"><textElement verticalAlignment="Middle"><text><![CDATA[...></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=0, ancho=150, alto=34 y aplica el estilo M5CrosstabTotal; configura la alineación del texto (vertical Middle); muestra el literal 'TOTAL'; abre el contenido de celda con estilo M5CrosstabTotal; define la cabecera de total de fila.
Línea 367	<code></rowGroup></code> → Cierra rowGroup y vuelve al elemento padre de la jerarquía JRXML.
Línea 368	<code><columnGroup name="AnioCross" height="28" totalPosition="End"></code> → Declara el grupo de columnas AnioCross, altura 28 y total en End.
Línea 369	<code><bucket class="java.lang.String"><bucketExpression><![CDATA[\$F{anio_cross}]]></bucketExpression></bucket></code> → Composición de la línea: define la clave de agrupación del bucket desde field anio_cross; encadena además <bucket> dentro de la misma jerarquía.
Línea 370	<code><crosstabColumnHeader><cellContents style="M5CrosstabCabecera"><textField><reportElement x="0" y="0" width="100" height="28"><textElement textAlignment="Center" verticalAlignme...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=0, ancho=100, alto=28 y aplica el estilo M5CrosstabCabecera; configura la alineación del texto (horizontal Center, vertical Middle); evalúa la expresión usando variable AnioCross; abre el contenido de celda con estilo M5CrosstabCabecera; define la cabecera de columna del crosstab.
Línea 371	<code><crosstabTotalColumnHeader><cellContents style="M5CrosstabTotal"><staticText><reportElement x="0" y="0" width="100" height="28"><textElement textAlignment="Center" verticalAlig...></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=0, ancho=100, alto=28 y aplica el estilo M5CrosstabTotal; configura la alineación del texto (horizontal Center, vertical Middle); muestra el literal 'TOTAL'; abre el contenido de celda con estilo M5CrosstabTotal; define la cabecera de total de columna.
Línea 372	<code></columnGroup></code> → Cierra columnGroup y vuelve al elemento padre de la jerarquía JRXML.
Línea 373	<code><measure name="ImporteCross" class="java.lang.Double" calculation="Sum"><measureExpression><![CDATA[\$F{importe_cross}]]></measureExpression></measure></code> → Composición de la línea: aporta a la medida el valor de field importe_cross; encadena además <measure> dentro de la misma jerarquía.
Línea 374	<code><measure name="VentasCross" class="java.lang.Integer" calculation="Sum"><measureExpression><![CDATA[\$F{ventas_cross}]]></measureExpression></measure></code> → Composición de la línea: aporta a la medida el valor de field ventas_cross; encadena además <measure> dentro de la misma jerarquía.
Línea 375	<code><crosstabCell width="100" height="34"><cellContents style="M5CrosstabDetalle"><textField pattern="#.##0.00 €"><reportElement x="0" y="0" width="100" height="18"><textElement te...></code> → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=0, y=0, ancho=100, alto=34 y aplica el estilo M5CrosstabDetalle; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable ImporteCross, variable VentasCross; abre el contenido de celda con estilo M5CrosstabDetalle; define la celda de detalle del crosstab.

Línea 376	<code><crosstabCell width="100" height="34" rowTotalGroup="CategoriaCross"><cellContents style="M5CrosstabTotal"><textField pattern="#,##0.00 €"><reportElement x="0" y="0" width="100" . . .</code> → Composición de la línea: crea un textField dinámico con patrón #,##0.00 €; lo posiciona en x=0, y=0, ancho=100, alto=34 y aplica el estilo M5CrosstabTotal; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable ImporteCross, variable VentasCross; abre el contenido de celda con estilo M5CrosstabTotal; define una celda de total de fila para CategoriaCross.
Línea 377	<code><crosstabCell width="100" height="34" columnTotalGroup="AnioCross"><cellContents style="M5CrosstabTotal"><textField pattern="#,##0.00 €"><reportElement x="0" y="0" width="100" h. . .</code> → Composición de la línea: crea un textField dinámico con patrón #,##0.00 €; lo posiciona en x=0, y=0, ancho=100, alto=34 y aplica el estilo M5CrosstabTotal; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable ImporteCross, variable VentasCross; abre el contenido de celda con estilo M5CrosstabTotal; define una celda de total de columna para AnioCross.
Línea 378	<code><crosstabCell width="100" height="34" rowTotalGroup="CategoriaCross" columnTotalGroup="AnioCross"><cellContents style="M5CrosstabTotal"><textField pattern="#,##0.00 €"><reportEl. . .</code> → Composición de la línea: crea un textField dinámico con patrón #,##0.00 €; lo posiciona en x=0, y=0, ancho=100, alto=34 y aplica el estilo M5CrosstabTotal; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable ImporteCross, variable VentasCross; abre el contenido de celda con estilo M5CrosstabTotal; define la celda de total general.
Línea 379	<code></crosstab></code> → Finaliza la tabla cruzada.
Línea 380	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 381	<code></summary></code> → Finaliza la sección Summary.
Línea 382	<code></jasperReport></code> → Finaliza la definición completa del informe JasperReports.

Subinforme heredado y ejecutable

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/
jasperreport.xsd"
  name="subinforme_ventas_detalle"
  language="java"
  pageWidth="555"
  pageHeight="842"
  columnWidth="555"
  leftMargin="0"
  rightMargin="0"
  topMargin="0"
  bottomMargin="0">
  <property name="com.jaspersoft.studio.data.defaultdataadapter" value="SQLiteEditorial"/>
  <style name="SubBase" isDefault="true" fontName="DejaVu Sans" fontSize="8"/>
  <style name="SubHeader" style="SubBase" mode="Opaque" backcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/>
  <parameter name="tituloLibro" class="java.lang.String"/>
  <queryString language="sql">
    <![CDATA[
      SELECT fecha_venta, cantidad, precio_unitario
      FROM ventas
      WHERE titulo_libro = $P{tituloLibro}
      ORDER BY fecha_venta
    ]]>
  </queryString>
  <field name="fecha_venta" class="java.lang.String"/>
  <field name="cantidad" class="java.lang.Integer"/>
  <field name="precio_unitario" class="java.lang.Double"/>
  <columnHeader>
    <band height="18">
      <staticText><reportElement x="0" y="0" width="245" height="18" style="SubHeader"/><text><![CDATA[Fecha]]></text></staticText>
      <staticText><reportElement x="245" y="0" width="100" height="18" style="SubHeader"/><textElement textAlignment="Right"/>
    </band>
  </columnHeader>
  <detail>
    <band height="18">
      <textField><reportElement x="0" y="0" width="245" height="18"/><textFieldExpression><![CDATA[$F{fecha_venta}]]></textFieldExpression></textField>
      <textField><reportElement x="245" y="0" width="100" height="18"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[$F{cantidad}]]></textFieldExpression></textField>
      <textField><reportElement x="345" y="0" width="210" height="18"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[$F{precio_unitario}]]></textFieldExpression></textField>
    </band>
  </detail>
</jasperReport>
```

```

</band>
</detail>
</jasperReport>

```

Explicación línea por línea

Línea 1	<code><?xml version="1.0" encoding="UTF-8"?></code> → Declara XML 1.0 y codificación UTF-8 para que nombres, textos y símbolos del informe se interpreten correctamente.
Línea 2	<code><jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports"</code> → Abre el documento raíz <code>jasperReport</code> del informe y fija el namespace principal de JasperReports.
Línea 3	<code>xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"</code> → Declara el namespace XML Schema Instance usado por <code>xsi:schemaLocation</code> para validar el documento.
Línea 4	<code>xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/jasperreport.xsd"</code> → Relaciona el namespace de JasperReports con su XSD para que Studio y el compilador validen la estructura.
Línea 5	<code>name="subinforme_ventas_detalle"</code> → Asigna al documento JasperReports el nombre interno <code>subinforme_ventas_detalle</code> .
Línea 6	<code>language="java"</code> → Configura <code>language=java</code> para evaluar expresiones con el lenguaje Java.
Línea 7	<code>pageWidth="555"</code> → Fija el ancho físico de página en 555 puntos.
Línea 8	<code>pageHeight="842"</code> → Fija la altura física de página en 842 puntos.
Línea 9	<code>columnWidth="555"</code> → Fija el ancho útil de la columna de contenido en 555 puntos.
Línea 10	<code>leftMargin="0"</code> → Fija el margen izquierdo del informe en 0 puntos.
Línea 11	<code>rightMargin="0"</code> → Fija el margen derecho del informe en 0 puntos.
Línea 12	<code>topMargin="0"</code> → Fija el margen superior del informe en 0 puntos.
Línea 13	<code>bottomMargin="0"></code> → Fija el margen inferior del informe en 0 puntos y completa la apertura del elemento raíz.
Línea 14	<code><property name="com.jaspersoft.studio.data.defaultdataadapter" value="SQLiteEditorial"/></code> → Indica a Jaspersoft Studio que use el Data Adapter <code>SQLiteEditorial</code> como conexión de diseño por defecto.
Línea 15	<code><style name="SubBase" isDefault="true" fontName="DejaVu Sans" fontSize="8"/></code> → Declara el estilo <code>SubBase</code> ; es el estilo por defecto, fuente <code>DejaVu Sans</code> , tamaño 8.
Línea 16	<code><style name="SubHeader" style="SubBase" mode="Opaque" backcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/></code> → Declara el estilo <code>SubHeader</code> ; hereda de <code>SubBase</code> , fondo <code>#EAF2F8</code> .
Línea 17	<code><parameter name="tituloLibro" class="java.lang.String"/></code> → Declara el parámetro <code>tituloLibro</code> con tipo <code>java.lang.String</code> y lo expone al diálogo de parámetros de Studio.
Línea 18	<code><queryString language="sql"></code> → Abre la consulta SQL que JasperReports ejecutará para el dataset actual.
Línea 19	<code><![CDATA[</code> → Abre CDATA para escribir SQL o una expresión Java sin que sus caracteres especiales se interpreten como XML.
Línea 20	<code>SELECT fecha_venta, cantidad, precio_unitario</code> → Cláusula SQL <code>SELECT</code> : selecciona y calcula las columnas que devolverá la consulta.
Línea 21	<code>FROM ventas</code> → Cláusula SQL <code>FROM</code> : define la tabla base de la consulta.
Línea 22	<code>WHERE titulo_libro = \${tituloLibro}</code> → Cláusula SQL <code>WHERE</code> : aplica el filtro de filas.
Línea 23	<code>ORDER BY fecha_venta</code> → Cláusula SQL <code>ORDER BY</code> : ordena el resultado que recibirá JasperReports.
Línea 24	<code>]]></code> → Cierra el bloque CDATA y devuelve el control al parser XML.
Línea 25	<code></queryString></code> → Cierra la consulta SQL del dataset actual.

Línea 26	<code><field name="fecha_venta" class="java.lang.String"/></code> → Declara el field fecha_venta con tipo Java <code>java.lang.String</code> para mapear una columna del dataset.
Línea 27	<code><field name="cantidad" class="java.lang.Integer"/></code> → Declara el field cantidad con tipo Java <code>java.lang.Integer</code> para mapear una columna del dataset.
Línea 28	<code><field name="precio_unitario" class="java.lang.Double"/></code> → Declara el field precio_unitario con tipo Java <code>java.lang.Double</code> para mapear una columna del dataset.
Línea 29	<code><columnHeader></code> → Abre Column Header, repetida al comienzo de cada columna/página según la paginación.
Línea 30	<code><band height="18"></code> → Define una banda de 18 puntos, reservando ese espacio para sus elementos.
Línea 31	<code><staticText><reportElement x="0" y="0" width="245" height="18" style="SubHeader"/><text><![CDATA[Fecha]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=0, ancho=245, alto=18 y aplica el estilo SubHeader; muestra el literal 'Fecha'.
Línea 32	<code><staticText><reportElement x="245" y="0" width="100" height="18" style="SubHeader"/><textElement textAlignment="Right"><text><![CDATA[Cantidad]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=245, y=0, ancho=100, alto=18 y aplica el estilo SubHeader; configura la alineación del texto (horizontal Right); muestra el literal 'Cantidad'.
Línea 33	<code><staticText><reportElement x="345" y="0" width="210" height="18" style="SubHeader"/><textElement textAlignment="Right"><text><![CDATA[Precio unitario]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=345, y=0, ancho=210, alto=18 y aplica el estilo SubHeader; configura la alineación del texto (horizontal Right); muestra el literal 'Precio unitario'.
Línea 34	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 35	<code></columnHeader></code> → Cierra columnHeader y vuelve al elemento padre de la jerarquía JRXML.
Línea 36	<code><detail></code> → Abre Detail, la sección que se repite para cada registro del dataset principal.
Línea 37	<code><band height="18"></code> → Define una banda de 18 puntos, reservando ese espacio para sus elementos.
Línea 38	<code><textField><reportElement x="0" y="0" width="245" height="18"/><textFieldExpression><![CDATA[\${fecha_venta}]]></textFieldExpression></textField></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=0, ancho=245, alto=18; evalúa la expresión usando field fecha_venta.
Línea 39	<code><textField><reportElement x="245" y="0" width="100" height="18"/><textElement textAlignment="Right"><textFieldExpression><![CDATA[\${cantidad}]]></textFieldExpression></textElement></textField></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=245, y=0, ancho=100, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando field cantidad.
Línea 40	<code><textField pattern="#.##0.00 €"><reportElement x="345" y="0" width="210" height="18"/><textElement textAlignment="Right"><textFieldExpression><![CDATA[\${precio_unitario}]]></textElement></textField></code> → Composición de la línea: crea un textField dinámico con patrón <code>#.##0.00 €</code> ; lo posiciona en x=345, y=0, ancho=210, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando field precio_unitario.
Línea 41	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 42	<code></detail></code> → Finaliza la sección Detail del informe.
Línea 43	<code></jasperReport></code> → Finaliza la definición completa del informe JasperReports.

Plantilla JRTX heredada y ejecutable

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<jasperTemplate xmlns="http://jasperreports.sourceforge.net/jasperreports/template"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports/template http://jasperreports.sourceforge.net/
xsd/jasptemplate.xsd">
  <style name="MSTituloPrincipal" fontName="DejaVu Sans" fontSize="18" isBold="true" forecolor="#173F6B"/>
  <style name="MSGrupoCabecera" fontName="DejaVu Sans" fontSize="10" isBold="true" forecolor="#173F6B" mode="Opaque"
backcolor="#D6EAF8"/>
  <style name="MSTablaCabecera" fontName="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque"
backcolor="#EAF2F8"/>
  <style name="MSTablaDetalle" fontName="DejaVu Sans" fontSize="9"/>
  <style name="MSCrosstabCabecera" fontName="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque">
```

```

backcolor="#EAF2F8"/>
<style name="M5CrosstabDetalle" fontName="DejaVu Sans" fontSize="9" mode="Opaque" backcolor="#FFFFFF"/>
<style name="M5CrosstabTotal" fontName="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque"
backcolor="#D6EAF8"/>
</jasperTemplate>

```

Explicación línea por línea

Línea 1	<?xml version="1.0" encoding="UTF-8"?> → Declara XML 1.0 y codificación UTF-8 para que nombres, textos y símbolos del informe se interpreten correctamente.
Línea 2	<jasperTemplate xmlns="http://jasperreports.sourceforge.net/jasperreports/template" → Abre el documento raíz jasperTemplate de la plantilla JRTX que contiene estilos reutilizables.
Línea 3	xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" → Declara el namespace XML Schema Instance usado por xsi:schemaLocation para validar el documento.
Línea 4	xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports/template http://jasperreports.sourceforge.net/xsd/jasptemplate.xsd" → Relaciona el namespace de JasperReports con su XSD para que Studio y el compilador validen la estructura.
Línea 5	<style name="M5TituloPrincipal" fontName="DejaVu Sans" fontSize="18" isBold="true" forecolor="#173F6B"/> → Declara el estilo M5TituloPrincipal; fuente DejaVu Sans, tamaño 18.
Línea 6	<style name="M5GrupoCabecera" fontName="DejaVu Sans" fontSize="10" isBold="true" forecolor="#173F6B" mode="Opaque" backcolor="#D6EAF8"/> → Declara el estilo M5GrupoCabecera; fuente DejaVu Sans, tamaño 10, fondo #D6EAF8.
Línea 7	<style name="M5TablaCabecera" fontName="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque" backcolor="#EAF2F8"/> → Declara el estilo M5TablaCabecera; fuente DejaVu Sans, tamaño 9, fondo #EAF2F8.
Línea 8	<style name="M5TablaDetalle" fontName="DejaVu Sans" fontSize="9"/> → Declara el estilo M5TablaDetalle; fuente DejaVu Sans, tamaño 9.
Línea 9	<style name="M5CrosstabCabecera" fontName="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque" backcolor="#EAF2F8"/> → Declara el estilo M5CrosstabCabecera; fuente DejaVu Sans, tamaño 9, fondo #EAF2F8.
Línea 10	<style name="M5CrosstabDetalle" fontName="DejaVu Sans" fontSize="9" mode="Opaque" backcolor="#FFFFFF"/> → Declara el estilo M5CrosstabDetalle; fuente DejaVu Sans, tamaño 9, fondo #FFFFFF.
Línea 11	<style name="M5CrosstabTotal" fontName="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque" backcolor="#D6EAF8"/> → Declara el estilo M5CrosstabTotal; fuente DejaVu Sans, tamaño 9, fondo #D6EAF8.
Línea 12	</jasperTemplate> → Finaliza la plantilla externa JRTX.

Parte C — Código y configuración ejecutable explicados línea por línea

GeneradorInformeVentas.java

JAVA

```

import java.io.File;
import java.sql.Connection;
import java.sql.DriverManager;
import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;
import net.sf.jasperreports.engine.JasperCompileManager;
import net.sf.jasperreports.engine.JasperFillManager;
import net.sf.jasperreports.engine.JasperPrint;
import net.sf.jasperreports.engine.export.JRPdfExporter;
import net.sf.jasperreports.export.SimpleExporterInput;
import net.sf.jasperreports.export.SimpleOutputStreamExporterOutput;
import net.sf.jasperreports.export.SimplePdfExporterConfiguration;

public class GeneradorInformeVentas {
    public static void main(String[] args) {
        try {
            String rutaJrxml = "reports/informe_ventas.jrxml";
            String rutaJasper = "reports/informe_ventas.jasper";
            String rutaPdf = "output/informe_ventas.pdf";

```

```

String rutaPdfProtegido = "output/informe_ventas_protegido.pdf";
String urlBD = "jdbc:sqlite:../EditorialReportsJava/data/editorial.db";
new File("output").mkdirs();

String rutaSubJrxml = "reports/subinforme_ventas_detalle.jrxml";
String rutaSubJasper = "reports/subinforme_ventas_detalle.jasper";
JasperCompileManager.compileReportToFile(rutaSubJrxml, rutaSubJasper);
JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);

Map<String, Object> parametros = new HashMap<String, Object>();
parametros.put("usuario", "Ana Martínez");
parametros.put("departamento", "Comercial");
parametros.put("periodo", "Septiembre 2026");
parametros.put("tipoIva", Double.valueOf(0.21d));
parametros.put("mostrarDetalle", Boolean.TRUE);
parametros.put("categoria", null);
parametros.put("precioMinimo", null);
parametros.put("precioMaximo", null);
parametros.put("umbralUnidades", Integer.valueOf(5));
parametros.put("textoBusqueda", null);
parametros.put("categoriasLista", Arrays.asList("Novela", "Realismo mágico", "Cuento", "Poesía"));

try (Connection conexion = DriverManager.getConnection(urlBD)) {
    JasperPrint documento = JasperFillManager.fillReport(rutaJasper, parametros, conexion);
    exportarPdf(documento, rutaPdf);
    exportarPdfProtegido(documento, rutaPdfProtegido);
    System.out.println("Informe PDF generado en: " + new File(rutaPdf).getAbsolutePath());
    System.out.println("Informe PDF protegido generado en: " + new File(rutaPdfProtegido).getAbsolutePath());
    System.out.println("Paginas del documento: " + documento.getPages().size());
    System.out.println("Parametro usuario: " + parametros.get("usuario"));
    System.out.println("M6 checkpoint generado correctamente");
}
} catch (Exception e) {
    e.printStackTrace();
    System.exit(1);
}
}

private static void exportarPdf(JasperPrint documento, String ruta) throws Exception {
    JRpdfExporter exportador = new JRpdfExporter();
    SimplePdfExporterConfiguration configuracion = new SimplePdfExporterConfiguration();
    configuracion.setMetadataTitle("Informe de Ventas - EditorialReports");
    configuracion.setMetadataAuthor("Departamento Comercial");
    configuracion.setMetadataSubject("Resumen de ventas del catálogo");
    configuracion.setMetadataKeywords("ventas, catálogo, libros, editorial");
    configuracion.setMetadataCreator("JasperReports 6.20.0");
    configuracion.setDisplayMetadataTitle(Boolean.TRUE);
    configuracion.setCompressed(Boolean.TRUE);
    exportador.setConfiguration(configuracion);
    exportador.setExporterInput(new SimpleExporterInput(documento));
    exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta));
    exportador.exportReport();
}

private static void exportarPdfProtegido(JasperPrint documento, String ruta) throws Exception {
    JRpdfExporter exportador = new JRpdfExporter();
    SimplePdfExporterConfiguration configuracion = new SimplePdfExporterConfiguration();
    configuracion.setMetadataTitle("Informe de Ventas Protegido - EditorialReports");
    configuracion.setEncrypted(Boolean.TRUE);
    configuracion.setUserPassword("editorial2026");
    configuracion.setOwnerPassword("editorial-admin");
    configuracion.setAllowedPermissionsHint("PRINTING|COPY|SCREENREADERS");
    exportador.setConfiguration(configuracion);
    exportador.setExporterInput(new SimpleExporterInput(documento));
    exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta));
    exportador.exportReport();
}
}

```

Explicación línea por línea

Línea 1	import java.io.File; → Importa java.io.File para gestionar rutas y crear la carpeta de salida.
Línea 2	import java.sql.Connection; → Importa java.sql.Connection para representar la conexión JDBC abierta contra SQLite.
Línea 3	import java.sql.DriverManager; → Importa java.sql.DriverManager para abrir la conexión JDBC a partir de la URL SQLite.
Línea 4	import java.util.Arrays; → Importa java.util.Arrays para construir la colección de categorías usada por el parámetro de lista.

Línea 5	<code>import java.util.HashMap;</code> → Importa <code>java.util.HashMap</code> para crear la implementación mutable del mapa de parámetros.
Línea 6	<code>import java.util.Map;</code> → Importa <code>java.util.Map</code> para tipar el mapa de parámetros que recibe JasperReports.
Línea 7	<code>import net.sf.jasperreports.engine.JasperCompileManager;</code> → Importa <code>net.sf.jasperreports.engine.JasperCompileManager</code> para compilar los JRXML a artefactos <code>.jasper</code> .
Línea 8	<code>import net.sf.jasperreports.engine.JasperFillManager;</code> → Importa <code>net.sf.jasperreports.engine.JasperFillManager</code> para llenar el informe compilado con parámetros y conexión.
Línea 9	<code>import net.sf.jasperreports.engine.JasperPrint;</code> → Importa <code>net.sf.jasperreports.engine.JasperPrint</code> para representar en memoria el documento ya paginado por JasperReports.
Línea 10	<code>import net.sf.jasperreports.engine.export.JRPdfExporter;</code> → Importa <code>net.sf.jasperreports.engine.export.JRPdfExporter</code> para usar la clase <code>JRPdfExporter</code> en el generador.
Línea 11	<code>import net.sf.jasperreports.export.SimpleExporterInput;</code> → Importa <code>net.sf.jasperreports.export.SimpleExporterInput</code> para usar la clase <code>SimpleExporterInput</code> en el generador.
Línea 12	<code>import net.sf.jasperreports.export.SimpleOutputStreamExporterOutput;</code> → Importa <code>net.sf.jasperreports.export.SimpleOutputStreamExporterOutput</code> para usar la clase <code>SimpleOutputStreamExporterOutput</code> en el generador.
Línea 13	<code>import net.sf.jasperreports.export.SimplePdfExporterConfiguration;</code> → Importa <code>net.sf.jasperreports.export.SimplePdfExporterConfiguration</code> para usar la clase <code>SimplePdfExporterConfiguration</code> en el generador.
Línea 14	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 15	<code>public class GeneradorInformeVentas {</code> → Declara la clase ejecutable <code>GeneradorInformeVentas</code> que encapsula el generador del informe.
Línea 16	<code>public static void main(String[] args) {</code> → Declara <code>main</code> como punto de entrada de la aplicación Java; recibe los argumentos de línea de comandos aunque este ejemplo no los utiliza.
Línea 17	<code>try {</code> → Abre el bloque principal protegido: cualquier error de compilación, conexión, llenado o exportación será capturado por el <code>catch</code> final.
Línea 18	<code>String rutaJrxml = "reports/informe_ventas.jrxml";</code> → Declara <code>rutaJrxml</code> con ruta del JRXML maestro que se compilará; el valor configurado es <code>"reports/informe_ventas.jrxml"</code> .
Línea 19	<code>String rutaJasper = "reports/informe_ventas.jasper";</code> → Declara <code>rutaJasper</code> con ruta del <code>.jasper</code> maestro que producirá la compilación; el valor configurado es <code>"reports/informe_ventas.jasper"</code> .
Línea 20	<code>String rutaPdf = "output/informe_ventas.pdf";</code> → Declara <code>rutaPdf</code> con ruta del PDF final exportado; el valor configurado es <code>"output/informe_ventas.pdf"</code> .
Línea 21	<code>String rutaPdfProtegido = "output/informe_ventas_protegido.pdf";</code> → Declara <code>rutaPdfProtegido</code> con valor de configuración usado por el generador; el valor configurado es <code>"output/informe_ventas_protegido.pdf"</code> .
Línea 22	<code>String urlBD = "jdbc:sqlite:../EditorialReportsJava/data/editorial.db";</code> → Declara <code>urlBD</code> con URL JDBC de la base SQLite; el valor configurado es <code>"jdbc:sqlite:../EditorialReportsJava/data/editorial.db"</code> .
Línea 23	<code>new File("output").mkdirs();</code> → Crea la carpeta <code>output</code> si todavía no existe para evitar que la exportación falle por una ruta inexistente.
Línea 24	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 25	<code>String rutaSubJrxml = "reports/subinforme_ventas_detalle.jrxml";</code> → Declara <code>rutaSubJrxml</code> con ruta del JRXML del subinforme de detalle; el valor configurado es <code>"reports/subinforme_ventas_detalle.jrxml"</code> .

Línea 26	<code>String rutaSubJasper = "reports/subinforme_ventas_detalle.jasper";</code> → Declara rutaSubJasper con ruta del .jasper del subinforme compilado; el valor configurado es "reports/subinforme_ventas_detalle.jasper".
Línea 27	<code>JasperCompileManager.compileReportToFile(rutaSubJrxml, rutaSubJasper);</code> → Compila el JRXML indicado en rutaSubJrxml y escribe el artefacto compilado en rutaSubJasper.
Línea 28	<code>JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);</code> → Compila el JRXML indicado en rutaJrxml y escribe el artefacto compilado en rutaJasper.
Línea 29	`` → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 30	`` → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 31	<code>Map<String, Object> parametros = new HashMap<String, Object>();</code> → Crea el mapa tipado de parámetros que se entregará a <code>JasperFillManager.fillReport</code> .
Línea 32	<code>parametros.put("usuario", "Ana Martínez");</code> → Asigna al parámetro JasperReports usuario el valor Java "Ana Martínez" antes del llenado.
Línea 33	<code>parametros.put("departamento", "Comercial");</code> → Asigna al parámetro JasperReports departamento el valor Java "Comercial" antes del llenado.
Línea 34	<code>parametros.put("periodo", "Septiembre 2026");</code> → Asigna al parámetro JasperReports periodo el valor Java "Septiembre 2026" antes del llenado.
Línea 35	<code>parametros.put("tipoIva", Double.valueOf(0.21d));</code> → Asigna al parámetro JasperReports tipoIva el valor Java Double.valueOf(0.21d) antes del llenado.
Línea 36	<code>parametros.put("mostrarDetalle", Boolean.TRUE);</code> → Asigna al parámetro JasperReports mostrarDetalle el valor Java Boolean.TRUE antes del llenado.
Línea 37	<code>parametros.put("categoria", null);</code> → Asigna al parámetro JasperReports categoria el valor Java null antes del llenado.
Línea 38	<code>parametros.put("precioMinimo", null);</code> → Asigna al parámetro JasperReports precioMinimo el valor Java null antes del llenado.
Línea 39	<code>parametros.put("precioMaximo", null);</code> → Asigna al parámetro JasperReports precioMaximo el valor Java null antes del llenado.
Línea 40	<code>parametros.put("umbralUnidades", Integer.valueOf(5));</code> → Asigna al parámetro JasperReports umbralUnidades el valor Java Integer.valueOf(5) antes del llenado.
Línea 41	<code>parametros.put("textoBusqueda", null);</code> → Asigna al parámetro JasperReports textoBusqueda el valor Java null antes del llenado.
Línea 42	<code>parametros.put("categoriasLista", Arrays.asList("Novela", "Realismo mágico", "Cuento", "Poesía"));</code> → Asigna al parámetro JasperReports categoriasLista el valor Java Arrays.asList("Novela", "Realismo mágico", "Cuento", "Poesía") antes del llenado.
Línea 43	`` → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 44	<code>try (Connection conexion = DriverManager.getConnection(urlBD)) {</code> → Abre la conexión SQLite mediante DriverManager dentro de un try-with-resources, por lo que conexion se cierra automáticamente al terminar el bloque.
Línea 45	<code>JasperPrint documento = JasperFillManager.fillReport(rutaJasper, parametros, conexion);</code> → Inicia el llenado del informe y guarda en documento el JasperPrint paginado que devolverá JasperReports.
Línea 46	<code>exportarPdf(documento, rutaPdf);</code> → Invoca la exportación PDF normal usando el JasperPrint ya llenado y la ruta principal de salida.
Línea 47	<code>exportarPdfProtegido(documento, rutaPdfProtegido);</code> → Genera una segunda salida PDF cifrada para validar contraseñas y permisos sin alterar el PDF normal.
Línea 48	<code>System.out.println("Informe PDF generado en: " + new File(rutaPdf).getAbsolutePath());</code> → Escribe en la consola la evidencia "Informe PDF generado en: " + new File(rutaPdf).getAbsolutePath(), que queda registrada por el workflow E2E.

Línea 49	<code>System.out.println("Informe PDF protegido generado en: " + new File(rutaPdfProtegido).getAbsolutePath());</code> → Escribe en la consola la evidencia "Informe PDF protegido generado en: " + <code>new File(rutaPdfProtegido).getAbsolutePath()</code> , que queda registrada por el workflow E2E.
Línea 50	<code>System.out.println("Paginas del documento: " + documento.getPages().size());</code> → Escribe en la consola la evidencia "Paginas del documento: " + <code>documento.getPages().size()</code> , que queda registrada por el workflow E2E.
Línea 51	<code>System.out.println("Parametro usuario: " + parametros.get("usuario"));</code> → Escribe en la consola la evidencia "Parametro usuario: " + <code>parametros.get("usuario")</code> , que queda registrada por el workflow E2E.
Línea 52	<code>System.out.println("M6 checkpoint generado correctamente");</code> → Escribe en la consola la evidencia "M6 checkpoint generado correctamente", que queda registrada por el workflow E2E.
Línea 53	<code>}</code> → Cierra el bloque try-with-resources de la conexión JDBC.
Línea 54	<code>} catch (Exception e) {</code> → Cierra el bloque protegido y abre el manejador que captura cualquier excepción del proceso completo.
Línea 55	<code>e.printStackTrace();</code> → Imprime la traza completa de la excepción para que el fallo sea diagnosticable en local y en GitHub Actions.
Línea 56	<code>System.exit(1);</code> → Finaliza el proceso con código 1 para que CI marque la ejecución como fallida y no oculte el error.
Línea 57	<code>}</code> → Cierra el bloque catch o el bloque principal de control asociado a main.
Línea 58	<code>}</code> → Cierra el método main.
Línea 59	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 60	<code>private static void exportarPdf(JasperPrint documento, String ruta) throws Exception {</code> → Declara el helper privado <code>exportarPdf</code> , que encapsula una exportación concreta y propaga cualquier error al catch principal.
Línea 61	<code>JRPdfExporter exportador = new JRPdfExporter();</code> → Crea el exportador PDF avanzado que admite configuración documental y de seguridad.
Línea 62	<code>SimplePdfExporterConfiguration configuracion = new SimplePdfExporterConfiguration();</code> → Crea el objeto <code>SimplePdfExporterConfiguration</code> que recibirá las opciones específicas de esta exportación.
Línea 63	<code>configuracion.setMetadataTitle("Informe de Ventas - EditorialReports");</code> → Fija el título de los metadatos PDF con la API específica de <code>SimplePdfExporterConfiguration</code> .
Línea 64	<code>configuracion.setMetadataAuthor("Departamento Comercial");</code> → Fija el autor en los metadatos del PDF.
Línea 65	<code>configuracion.setMetadataSubject("Resumen de ventas del catálogo");</code> → Fija el asunto documental del PDF.
Línea 66	<code>configuracion.setMetadataKeywords("ventas, catálogo, libros, editorial");</code> → Fija las palabras clave que quedarán registradas en las propiedades del PDF.
Línea 67	<code>configuracion.setMetadataCreator("JasperReports 6.20.0");</code> → Registra JasperReports 6.20.0 como creador del PDF.
Línea 68	<code>configuracion.setDisplayMetadataTitle(Boolean.TRUE);</code> → Solicita a los lectores PDF que utilicen el título de metadatos cuando soporten esa preferencia.
Línea 69	<code>configuracion.setCompressed(Boolean.TRUE);</code> → Activa la compresión del PDF mediante la configuración del exportador.
Línea 70	<code>exportador.setConfiguration(configuracion);</code> → Asocia al exportador la configuración específica preparada en las líneas anteriores.
Línea 71	<code>exportador.setExporterInput(new SimpleExporterInput(documento));</code> → Entrega al exportador el <code>JasperPrint documento</code> ya llenado; no se vuelve a consultar la base de datos.

Línea 72	<code>exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta));</code> → Dirige la salida binaria del exportador al archivo indicado por el parámetro <code>ruta</code> .
Línea 73	<code>exportador.exportReport();</code> → Ejecuta la exportación con la entrada, salida y configuración ya asignadas; aquí se materializa el archivo.
Línea 74	<code>}</code> → Cierra el método <code>main</code> .
Línea 75	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 76	<code>private static void exportarPdfProtegido(JasperPrint documento, String ruta) throws Exception {</code> → Declara el helper privado <code>exportarPdfProtegido</code> , que encapsula una exportación concreta y propaga cualquier error al <code>catch</code> principal.
Línea 77	<code>JRPdfExporter exportador = new JRPdfExporter();</code> → Crea el exportador PDF avanzado que admite configuración documental y de seguridad.
Línea 78	<code>SimplePdfExporterConfiguration configuracion = new SimplePdfExporterConfiguration();</code> → Crea el objeto <code>SimplePdfExporterConfiguration</code> que recibirá las opciones específicas de esta exportación.
Línea 79	<code>configuracion.setMetadataTitle("Informe de Ventas Protegido - EditorialReports");</code> → Fija el título de los metadatos PDF con la API específica de <code>SimplePdfExporterConfiguration</code> .
Línea 80	<code>configuracion.setEncrypted(Boolean.TRUE);</code> → Activa el cifrado de la salida PDF protegida.
Línea 81	<code>configuracion.setUserPassword("editorial2026");</code> → Configura la contraseña de apertura que el E2E verifica con <code>pdfinfo -upw</code> .
Línea 82	<code>configuracion.setOwnerPassword("editorial-admin");</code> → Configura la contraseña de propietario del PDF protegido.
Línea 83	<code>configuracion.setAllowedPermissionsHint("PRINTING COPY SCREENREADERS");</code> → Declara los permisos PDF autorizados mediante la cadena de hints admitida por JasperReports.
Línea 84	<code>exportador.setConfiguration(configuracion);</code> → Asocia al exportador la configuración específica preparada en las líneas anteriores.
Línea 85	<code>exportador.setExporterInput(new SimpleExporterInput(documento));</code> → Entrega al exportador el <code>JasperPrint</code> <code>documento</code> ya llenado; no se vuelve a consultar la base de datos.
Línea 86	<code>exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta));</code> → Dirige la salida binaria del exportador al archivo indicado por el parámetro <code>ruta</code> .
Línea 87	<code>exportador.exportReport();</code> → Ejecuta la exportación con la entrada, salida y configuración ya asignadas; aquí se materializa el archivo.
Línea 88	<code>}</code> → Cierra el método <code>main</code> .
Línea 89	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 90	<code>}</code> → Cierra la clase <code>GeneradorInformeVentas</code> .

Parte D — Simulación del resultado y de la estructura del proyecto

D.1 — Flujo de exportación / diseño

TEXT

```
JasperPrint de informe_ventas (6 páginas)
├── JRPdfExporter normal -> informe_ventas.pdf
│   ├── metadatos
│   └── compresión
└── JRPdfExporter protegido -> informe_ventas_protegido.pdf
    ├── user password: editorial2026
    ├── owner password
    └── PRINTING | COPY | SCREENREADERS
```

Qué representa: la transformación funcional que debe existir al terminar 6.1.

Cómo verificarlo: comparar el flujo con la Parte C y ejecutar el generador; el JRXML/JRTX debe seguir siendo el heredado de M5/5.6.

D.2 — Estructura lógica en código y recursos

TEXT

```
JRXML/JRTX/Outline = idénticos a M5/5.6
Java
├─ exportarPdf
└─ exportarPdfProtegido
```

Qué representa: las clases, métodos y recursos que sustituyen en M6 al trabajo visual sobre bandas y componentes.

Cómo verificarlo: abrir Java/POM/CSS/properties según corresponda y contrastar nombres y tipos con la Parte C ejecutable.

D.3 — Archivos de salida

TEXT

```
output/
├─ informe_ventas.pdf
└─ informe_ventas_protegido.pdf
```

Qué representa: los artefactos acumulativos esperados en output.

Cómo verificarlo: ejecutar `GeneradorInformeVentas`, abrir cada formato con una herramienta compatible y contrastar los contratos automatizados del E2E.

D.4 — Árbol acumulativo del checkpoint

TEXT

```
M6/6.1/
├─ EditorialReports/EXPORTACION_PDF.md
├─ EditorialReports/reports/ (heredado)
├─ EditorialReports/resources/ (heredado)
└─ EditorialReportsJava/src/GeneradorInformeVentas.java
```

Evidencia E2E: run **36249131955**, commit `12a0eba90859a92b12578d59ae592ad17dac5fb6`, artifact runtime **10908358174**.

Invariantes: 14 libros, 9 ventas, 31 unidades, 633,40 € y 6 páginas en `informe_ventas`.

Errores comunes del ejercicio completo

Error	Causa	Solución
cannot find symbol: JRpdfExporter	Falta el import	Importar <code>net.sf.jasperreports.engine.export.JRpdfExporter</code>
El PDF no contiene metadatos	No se asignó configuración	Aplicar <code>SimplePdfExporterConfiguration</code> con <code>setConfiguration</code>
setTitle / setAuthor no compilan	API incorrecta	Usar <code>setMetadataTitle</code> / <code>setMetadataAuthor</code>
El PDF protegido no solicita contraseña	No se activó cifrado	Usar <code>setEncrypted(Boolean.TRUE)</code> y <code>setUserPassword</code>
El archivo no se genera	Falta output o <code>exportReport()</code>	Crear la carpeta y ejecutar el exportador

Reto resuelto paso a paso

Enunciado original: proteger el PDF con contraseña `editorial12026` y permitir impresión/copia.

1. Se crea `informe_ventas_protegido.pdf` como salida separada.
2. Se activa `setEncrypted(Boolean.TRUE)`.
3. Se configura `setUserPassword("editorial12026")`.
4. Se configura una contraseña de propietario independiente.
5. Se aplican `PRINTING|COPY|SCREENREADERS` con `setAllowedPermissionsHint`.
6. El mismo `JasperPrint` de seis páginas alimenta ambos PDFs.
7. El E2E abre el PDF protegido con `pdftinfo -upw editorial12026`.

Resultado del reto: la contraseña documentada abre un PDF real y protegido.

Analogía final con el contexto de la editorial

El PDF normal es la tirada estándar y el PDF protegido es la misma tirada bajo control de acceso. Los metadatos son la ficha técnica del documento.

Resultado esperado

- `informe_ventas.pdf` con metadatos y compresión.
- `informe_ventas_protegido.pdf` cifrado.
- `EXPORTACION_PDF.md` coherente con la API real.
- JRXML/JRTX idénticos a M5/5.6.
- Seis páginas y datos heredados intactos.

Conclusión y enlace al siguiente punto

6.1 separa llenado y exportación y deja preparada la arquitectura para reutilizar el mismo `JasperPrint` en los formatos siguientes.

Punto 6.2 — Exportación a Excel

Objetivos de aprendizaje

- Comprender las diferencias entre los formatos XLS y XLSX.
- Utilizar el exportador JRXLsxExporter para generar archivos Excel modernos.
- Configurar las propiedades del exportador mediante SimpleXlsxExporterConfiguration.
- Ajustar el nombre de la hoja, el ancho de columnas y las celdas combinadas.
- Aplicar formato a las celdas exportadas.
- Documentar la exportación a Excel del proyecto EditorialReports.

Parte A — Práctica visual/IDE verificada

Paso 1: Abrir el checkpoint 6.2 y comprobar la herencia 6.1

Acciones:

1. Abrir `M6/6.2/EditorialReportsJava/src/GeneradorInformeVentas.java`.
2. Confirmar las dos exportaciones PDF de 6.1.
3. Abrir `pom.xml` en paralelo.

Verificación visual: el Java conserva PDF normal/protegido y el POM contiene JasperReports 6.20.0.

Qué hace: completa la operación «Abrir el checkpoint 6.2 y comprobar la herencia 6.1» en el checkpoint 6.2.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.2.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.2.

Analogía: es como preparar dos libros contables con hojas identificadas y editables.

Paso 2: Añadir Apache POI al POM

Acciones:

1. Añadir dependencia `org.apache.poi:poi:5.1.0`.
2. Añadir dependencia `org.apache.poi:poi-ooxml:5.1.0`.
3. Guardar el POM y actualizar el proyecto Maven.

Verificación visual: Maven resuelve POI y POI-OOXML sin dependencias faltantes.

Qué hace: completa la operación «Añadir Apache POI al POM» en el checkpoint 6.2.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.2.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.2.

Analogía: es como preparar dos libros contables con hojas identificadas y editables.

Paso 3: Añadir imports XLSX y JRCsvDataSource

Acciones:

1. Importar `JRXLsxExporter`.

2. Importar `SimpleXlsxReportConfiguration` y `SimpleXlsxExporterConfiguration`.
3. Importar `JRCsvDataSource` para el reto de catálogo.

Verificación visual: Problems no muestra imports sin resolver.

Qué hace: completa la operación «Añadir imports XLSX y JRCsvDataSource» en el checkpoint 6.2.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.2.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.2.

Analogía: es como preparar dos libros contables con hojas identificadas y editables.

Paso 4: Declarar rutas de ventas y catálogo

Acciones:

1. Añadir `rutaXlsx = "output/informe_ventas.xlsx"`.
2. Añadir rutas JRXML/JASPER para `informe_catalogo_csv`.
3. Añadir `rutaXlsxCatalogo = "output/informe_catalogo.xlsx"`.

Verificación visual: Source contiene las cuatro rutas y conserva las rutas PDF.

Qué hace: completa la operación «Declarar rutas de ventas y catálogo» en el checkpoint 6.2.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.2.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.2.

Analogía: es como preparar dos libros contables con hojas identificadas y editables.

Paso 5: Compilar también el informe de catálogo

Acciones:

1. Después de compilar el subinforme y `informe_ventas`, compilar `rutaCatalogoJrxml` a `rutaCatalogoJasper`.
2. No modificar `informe_catalogo_csv.jrxml`.

Verificación visual: la ejecución crea `reports/informe_catalogo_csv.jasper`.

Qué hace: completa la operación «Compilar también el informe de catálogo» en el checkpoint 6.2.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.2.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.2.

Analogía: es como preparar dos libros contables con hojas identificadas y editables.

Paso 6: Crear exportarXlsx con nombre de hoja

Acciones:

1. Declarar `exportarXlsx(JasperPrint documento, String ruta, String nombreHoja)`.

2. Crear `JRXlsxExporter`.
3. Crear `SimpleXlsxReportConfiguration` y asignar `new String[]{nombreHoja}` a `setSheetNames`.

Verificación visual: el método no tiene el nombre de hoja Ventas codificado internamente.

Qué hace: completa la operación «Crear exportarXlsx con nombre de hoja» en el checkpoint 6.2.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.2.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.2.

Analogía: es como preparar dos libros contables con hojas identificadas y editables.

Paso 7: Configurar la hoja XLSX

Acciones:

1. Aplicar `setShowGridLines(Boolean.FALSE)`.
2. Aplicar `setCellLocked(Boolean.FALSE)` y `setCellHidden(Boolean.FALSE)`.
3. Aplicar `setDetectCellType(Boolean.TRUE)` y `setOnePagePerSheet(Boolean.FALSE)`.

Verificación visual: todas las opciones de hoja pertenecen a `SimpleXlsxReportConfiguration`.

Qué hace: completa la operación «Configurar la hoja XLSX» en el checkpoint 6.2.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.2.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.2.

Analogía: es como preparar dos libros contables con hojas identificadas y editables.

Paso 8: Configurar el libro XLSX y exportar

Acciones:

1. Crear `SimpleXlsxExporterConfiguration` libro.
2. Aplicar `libro.setCreateCustomPalette(Boolean.TRUE)`.
3. Asignar ambas configuraciones al exportador, después input y output, y llamar a `exportReport()`.

Verificación visual: la paleta está en `SimpleXlsxExporterConfiguration`, separada de la configuración de hoja.

Qué hace: completa la operación «Configurar el libro XLSX y exportar» en el checkpoint 6.2.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.2.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.2.

Analogía: es como preparar dos libros contables con hojas identificadas y editables.

Paso 9: Exportar el informe de ventas

Acciones:

1. Dentro del mismo bloque de conexión, llamar a `exportarXlsx(documento, rutaXlsx, "Ventas")`.
2. No volver a llenar `informe_ventas`.

Verificación visual: la primera salida XLSX reutiliza el JasperPrint documento.

Qué hace: completa la operación «Exportar el informe de ventas» en el checkpoint 6.2.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.2.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.2.

Analogía: es como preparar dos libros contables con hojas identificadas y editables.

Paso 10: Resolver el reto del catálogo con su fuente CSV real**Acciones:**

1. Crear `JRCsvDataSource` sobre `data/catalogo.csv` con UTF-8.
2. Configurar delimitador coma y primera fila como cabecera.
3. Llenar `rutaCatalogoJasper` con ese datasource y un mapa vacío.
4. Exportar ese JasperPrint a `rutaXlsxCatalogo` con hoja Catálogo.
5. Cerrar el datasource en `finally`.

Verificación visual: Source reproduce el mismo origen CSV que `GeneradorCatalogoCSV`; no intenta llenar el catálogo con JDBC.

Qué hace: completa la operación «Resolver el reto del catálogo con su fuente CSV real» en el checkpoint 6.2.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.2.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.2.

Analogía: es como preparar dos libros contables con hojas identificadas y editables.

Paso 11: Compilar y ejecutar**Acciones:**

1. Guardar Java y POM.
2. Ejecutar `mvn clean package`.
3. Ejecutar `GeneradorInformeVentas`.

Verificación visual: Console termina con el mensaje de checkpoint correcto y sin excepciones.

Qué hace: completa la operación «Compilar y ejecutar» en el checkpoint 6.2.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.2.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.2.

Analogía: es como preparar dos libros contables con hojas identificadas y editables.

Paso 12: Validar los dos libros Excel

Acciones:

1. Abrir `output/informe_ventas.xlsx` y comprobar la hoja Ventas.
2. Abrir `output/informe_catalogo.xlsx` y comprobar la hoja Catálogo.
3. Verificar que ambos archivos contienen datos y se abren sin reparación.

Verificación visual: las dos hojas tienen los nombres exigidos por la práctica y el reto.

Qué hace: completa la operación «Validar los dos libros Excel» en el checkpoint 6.2.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.2.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.2.

Analogía: es como preparar dos libros contables con hojas identificadas y editables.

Paso 13: Documentar XLSX

Acciones:

1. Crear/abrir `EXPORTACION_EXCEL.md`.
2. Documentar la separación `ReportConfiguration/ExporterConfiguration`.
3. Registrar POI 5.1.0 y las dos salidas Ventas/Catálogo.

Verificación visual: la documentación coincide con el POM y el Java ejecutable.

Qué hace: completa la operación «Documentar XLSX» en el checkpoint 6.2.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.2.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.2.

Analogía: es como preparar dos libros contables con hojas identificadas y editables.

Parte B — JRXML/JRTX completo explicado línea por línea

En M6 el diseño no cambia: estos tres archivos deben permanecer byte a byte iguales a M5/5.6.

Informe maestro heredado y ejecutable

```
<?xml version="1.0" encoding="UTF-8"?>
<jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/
jasperreport.xsd"
  name="informe_ventas"
  language="java"
  pageWidth="595"
  pageHeight="842"
  columnWidth="555"
  leftMargin="20"
  rightMargin="20"
  topMargin="20"
  bottomMargin="20"
  uuid="3d2c2bd7-3b93-4da9-8b60-6b3c45674c91">
  <property name="com.jaspersoft.studio.data.defaultdataadapter" value="SQLiteEditorial"/>
  <template><![CDATA[resources/styles/EditorialStyles.jrtx]]></template>
```

XML

```

<style name="Sans_Normal" isDefault="true" fontName="DejaVu Sans" fontSize="10"/>
<style name="TituloPrincipal" style="Sans_Normal" fontSize="18" isBold="true" forecolor="#173F6B"/>
<style name="Cabecera" style="Sans_Normal" fontSize="9" isBold="true" forecolor="#173F6B"/>
<style name="Dato" style="Sans_Normal" fontSize="9"/>
<style name="UnidadesCondicional" style="Dato" isBold="true">
  <conditionalStyle>
    <conditionExpression><![CDATA[{$f{unidades_vendidas} != null && $P{umbralUnidades} != null &&
$F{unidades_vendidas}.intValue() >= $P{umbralUnidades}.intValue()}]></conditionExpression>
    <style forecolor="#1B5E20"/>
  </conditionalStyle>
  <conditionalStyle>
    <conditionExpression><![CDATA[{$f{unidades_vendidas} != null && $P{umbralUnidades} != null &&
$F{unidades_vendidas}.intValue() >= 3 && $F{unidades_vendidas}.intValue() < $P{umbralUnidades}.intValue()}]></conditionExpression>
    <style forecolor="#1D5D88"/>
  </conditionalStyle>
  <conditionalStyle>
    <conditionExpression><![CDATA[{$f{unidades_vendidas} == null || $F{unidades_vendidas}.intValue() < 3}]></conditionExpression>
    <style forecolor="#9D3429"/>
  </conditionalStyle>
</style>
<style name="M5TableHeader" style="Dato" mode="Opaque" backcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/>
<style name="M5TableDetail" style="Dato"/>
<style name="M5CrossHeader" style="Dato" mode="Opaque" backcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/>
<style name="M5CrossDetail" style="Dato" mode="Opaque" backcolor="#FFFFFF"/>
<style name="M5CrossTotal" style="Dato" mode="Opaque" backcolor="#D6EAF8" forecolor="#173F6B" isBold="true"/>
<subDataset name="DatasetTopVentas">
  <parameter name="tituloLibro" class="java.lang.String"/>
  <queryString language="sql">
    <![CDATA[
      SELECT fecha_venta, cantidad, precio_unitario
      FROM ventas
      WHERE titulo_libro = $P{tituloLibro}
      ORDER BY cantidad DESC, fecha_venta
      LIMIT 3
    ]]>
  </queryString>
  <field name="fecha_venta" class="java.lang.String"/>
  <field name="cantidad" class="java.lang.Integer"/>
  <field name="precio_unitario" class="java.lang.Double"/>
</subDataset>
<subDataset name="DatasetVentasPorCategoria">
  <queryString language="sql">
    <![CDATA[
      SELECT l.categoria AS categoria_grafico,
      COALESCE(SUM(v.cantidad * v.precio_unitario), 0.0) AS importe_categoria
      FROM libros l
      LEFT JOIN ventas v ON l.titulo = v.titulo_libro
      GROUP BY l.categoria
      ORDER BY l.categoria
    ]]>
  </queryString>
  <field name="categoria_grafico" class="java.lang.String"/>
  <field name="importe_categoria" class="java.lang.Double"/>
</subDataset>
<subDataset name="DatasetCrosstabVentas">
  <queryString language="sql">
    <![CDATA[
      SELECT l.categoria AS categoria_cross,
      SUBSTR(v.fecha_venta, 1, 4) AS anio_cross,
      (v.cantidad * v.precio_unitario) AS importe_cross,
      1 AS ventas_cross
      FROM ventas v
      JOIN libros l ON l.titulo = v.titulo_libro
      ORDER BY l.categoria, anio_cross, v.fecha_venta
    ]]>
  </queryString>
  <field name="categoria_cross" class="java.lang.String"/>
  <field name="anio_cross" class="java.lang.String"/>
  <field name="importe_cross" class="java.lang.Double"/>
  <field name="ventas_cross" class="java.lang.Integer"/>
</subDataset>
<parameter name="usuario" class="java.lang.String" isForPrompting="true"/>
<parameter name="fechaInforme" class="java.util.Date" isForPrompting="true">
  <defaultValueExpression><![CDATA[new java.util.Date()]]></defaultValueExpression>
</parameter>
<parameter name="departamento" class="java.lang.String" isForPrompting="true">
  <defaultValueExpression><![CDATA["General"]]></defaultValueExpression>
</parameter>
<parameter name="periodo" class="java.lang.String" isForPrompting="true">
  <defaultValueExpression><![CDATA["Mensual"]]></defaultValueExpression>
</parameter>
<parameter name="tipoIva" class="java.lang.Double" isForPrompting="true">
  <defaultValueExpression><![CDATA[Double.valueOf(0.21d)]]></defaultValueExpression>
</parameter>
<parameter name="mostrarDetalle" class="java.lang.Boolean" isForPrompting="true">
  <defaultValueExpression><![CDATA[Boolean.TRUE]]></defaultValueExpression>
</parameter>
<parameter name="categoria" class="java.lang.String" isForPrompting="true"/>
<parameter name="precioMinimo" class="java.lang.Double" isForPrompting="true"/>
<parameter name="precioMaximo" class="java.lang.Double" isForPrompting="true"/>
<parameter name="umbralUnidades" class="java.lang.Integer" isForPrompting="true">
  <defaultValueExpression><![CDATA[Integer.valueOf(5)]]></defaultValueExpression>

```

Página 48 de 225

```

</staticText>
<staticText><reportElement x="0" y="38" width="110" height="18" uuid="40000000-0000-4000-8000-000000000002"/><text><![
[CDATA[Generado por:]]></text></staticText>
<textField isBlankWhenNull="true"><reportElement x="110" y="38" width="160" height="18"
uuid="40000000-0000-4000-8000-000000000003"/><textFieldExpression><![CDATA[{P{usuario}}]></textFieldExpression></textField>
<staticText><reportElement x="300" y="38" width="80" height="18" uuid="40000000-0000-4000-8000-000000000004"/><text><![
[CDATA[Fecha:]]></text></staticText>
<textField pattern="dd/MM/yyyy"><reportElement x="380" y="38" width="175" height="18"
uuid="40000000-0000-4000-8000-000000000005"/><textFieldExpression><![CDATA[{P{fechaInforme}}]></textFieldExpression></textField>
<staticText><reportElement x="0" y="62" width="100" height="18" uuid="40000000-0000-4000-8000-000000000006"/><text><![
[CDATA[Departamento:]]></text></staticText>
<textField isBlankWhenNull="true"><reportElement x="100" y="62" width="170" height="18"
uuid="40000000-0000-4000-8000-000000000007"/><textFieldExpression><![CDATA[{P{departamento}}]></textFieldExpression></textField>
<staticText><reportElement x="300" y="62" width="70" height="18" uuid="40000000-0000-4000-8000-000000000008"/><text><![
[CDATA[Periodo:]]></text></staticText>
<textField isBlankWhenNull="true"><reportElement x="370" y="62" width="185" height="18"
uuid="40000000-0000-4000-8000-000000000009"/><textFieldExpression><![CDATA[{P{periodo}}]></textFieldExpression></textField>
<staticText><reportElement x="0" y="86" width="100" height="18" uuid="40000000-0000-4000-8000-000000000010"/><text><![
[CDATA[Búsqueda:]]></text></staticText>
<textField isBlankWhenNull="true"><reportElement x="100" y="86" width="170" height="18"
uuid="40000000-0000-4000-8000-000000000011"/><textFieldExpression><![CDATA[{P{textoBusqueda}} == null ||
{P{textoBusqueda}}.trim().isEmpty() ? "(todas)" : {P{textoBusqueda}}]></textFieldExpression></textField>
<staticText><reportElement x="300" y="86" width="90" height="18" uuid="40000000-0000-4000-8000-000000000012"/><text><![
[CDATA[Categorías:]]></text></staticText>
<textField textAdjust="StretchHeight"><reportElement x="390" y="86" width="165" height="34"
uuid="40000000-0000-4000-8000-000000000013"/><textFieldExpression><![CDATA[String.valueOf({P{categoriasLista}})]></
textFieldExpression></textField>
</band>
</title>
<columnHeader>
<band height="62">
<staticText><reportElement x="0" y="2" width="215" height="18" uuid="41000000-0000-4000-8000-000000000001" style="Cabecera"/
><text><![CDATA[{Titulo}]]></text></staticText>
<staticText><reportElement x="215" y="2" width="55" height="18" uuid="41000000-0000-4000-8000-000000000002"
style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[{Unid.}]]></text></staticText>
<staticText><reportElement x="280" y="2" width="90" height="18" uuid="41000000-0000-4000-8000-000000000003"
style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[{Importe}]]></text></staticText>
<staticText><reportElement x="380" y="2" width="65" height="18" uuid="41000000-0000-4000-8000-000000000004"
style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[{Precio med.}]]></text></staticText>
<staticText><reportElement x="455" y="2" width="100" height="18" uuid="41000000-0000-4000-8000-000000000005"
style="Cabecera"/><text><![CDATA[{Categoría}]]></text></staticText>
<staticText><reportElement x="0" y="24" width="130" height="18" uuid="41000000-0000-4000-8000-000000000006"
style="Cabecera"/><text><![CDATA[{Primera venta}]]></text></staticText>
<staticText><reportElement x="130" y="24" width="130" height="18" uuid="41000000-0000-4000-8000-000000000007"
style="Cabecera"/><text><![CDATA[{Última venta}]]></text></staticText>
<staticText><reportElement x="260" y="24" width="160" height="18" uuid="41000000-0000-4000-8000-000000000008"
style="Cabecera"/><text><![CDATA[{Periodo de ventas}]]></text></staticText>
<staticText>
<reportElement x="420" y="24" width="135" height="18" uuid="41000000-0000-4000-8000-000000000009" style="Cabecera">
<printWhenExpression><![CDATA[{Boolean.TRUE.equals({P{mostrarDetalle}})}]]></printWhenExpression>
</reportElement>
<textElement textAlignment="Right"/>
<text><![CDATA[{Importe con IVA}]]></text>
</staticText>
</band>
</columnHeader>
<detail>
<band height="82" splitType="Stretch">
<textField textAdjust="StretchHeight"><reportElement x="0" y="0" width="215" height="20"
uuid="42000000-0000-4000-8000-000000000001" style="Dato"/><textFieldExpression><![CDATA[{F{titulo}}]]></textFieldExpression></textField>
<textField isBlankWhenNull="true"><reportElement x="215" y="0" width="55" height="20"
uuid="42000000-0000-4000-8000-000000000002" style="UnidadesCondicionales"/><textElement textAlignment="Right"/><textFieldExpression><![
[CDATA[{F{unidades_vendidas}}]]></textFieldExpression></textField>
<textField pattern="#,##0.00 €" isBlankWhenNull="true"><reportElement x="280" y="0" width="90" height="20"
uuid="42000000-0000-4000-8000-000000000003" style="Dato"/><textElement textAlignment="Right"/><textFieldExpression><![
[CDATA[{F{importe_total}}]]></textFieldExpression></textField>
<textField isBlankWhenNull="true"><reportElement x="380" y="0" width="65" height="20"
uuid="42000000-0000-4000-8000-000000000004" style="Dato"/><textElement textAlignment="Right"/><textFieldExpression><![
[CDATA[{F{precio_medio}} == null ? "Sin datos" : new java.text.DecimalFormat("#0.00 '€').format({F{precio_medio}})]></
textFieldExpression></textField>
<textField isBlankWhenNull="true"><reportElement x="455" y="0" width="100" height="20"
uuid="42000000-0000-4000-8000-000000000005" style="Dato"/><textFieldExpression><![CDATA[{F{categoria}}]]></textFieldExpression></
textField>
<textField isBlankWhenNull="true"><reportElement x="0" y="24" width="130" height="18"
uuid="42000000-0000-4000-8000-000000000006" style="Dato"/><textFieldExpression><![CDATA[{F{primera_venta}}]]></textFieldExpression></
textField>
<textField isBlankWhenNull="true"><reportElement x="130" y="24" width="130" height="18"
uuid="42000000-0000-4000-8000-000000000007" style="Dato"/><textFieldExpression><![CDATA[{F{ultima_venta}}]]></textFieldExpression></
textField>
<textField><reportElement x="260" y="24" width="160" height="18" uuid="42000000-0000-4000-8000-000000000008" style="Dato"/
><textElement textAlignment="Center"/><textFieldExpression><![CDATA[{F{primera_venta}} == null ? "Sin ventas" : {F{primera_venta}} + " → "
+ {F{ultima_venta}}]]></textFieldExpression></textField>
<textField pattern="#,##0.00 €" isBlankWhenNull="true">
<reportElement x="420" y="24" width="135" height="18" uuid="42000000-0000-4000-8000-000000000009" style="Dato">
<printWhenExpression><![CDATA[{Boolean.TRUE.equals({P{mostrarDetalle}})}]]></printWhenExpression>
</reportElement>
<textElement textAlignment="Right"/>
<textFieldExpression><![CDATA[{F{importe_total}} == null || {P{tipoIva}} == null ? null :
Double.valueOf({F{importe_total}}.doubleValue()) * (1.0d + {P{tipoIva}}.doubleValue())]]></textFieldExpression>
</textField>
<textField><reportElement x="0" y="48" width="105" height="18" uuid="42000000-0000-4000-8000-000000000010" style="Dato"/
><textFieldExpression><![CDATA[{F{unidades_vendidas}} == null ? "Sin ventas" : ({F{unidades_vendidas}}.intValue()) >= 6 ? "Premium" :

```

```

($F{unidades_vendidas}.intValue() >= 3 ? "Estándar" : "Económico"))]]></textFieldExpression></textField>
<textField><reportElement x="105" y="48" width="185" height="18" uuid="42000000-0000-4000-8000-000000000011" style="Dato"/
><textFieldExpression><![CDATA[$F{titulo} == null ? "" : $F{titulo}.trim().toUpperCase(java.util.Locale.ROOT)]]></textFieldExpression></
textField>
<textField><reportElement x="290" y="48" width="80" height="18" uuid="42000000-0000-4000-8000-000000000012" style="Dato"/
><textFieldExpression><![CDATA[$F{precio_medio} == null ? "-" : String.format(java.util.Locale.ROOT,
"%2f", Double.valueOf(Math.round($F{precio_medio}.doubleValue() * 100.0d) / 100.0d)]]></textFieldExpression></textField>
<textField><reportElement x="370" y="48" width="90" height="18" uuid="42000000-0000-4000-8000-000000000013" style="Dato"/
><textFieldExpression><![CDATA[$F{primera_venta} == null || $F{ultima_venta} == null ? "-" :
java.lang.Long.toString(java.time.temporal.ChronoUnit.DAYS.between(java.time.LocalDate.parse($F{primera_venta}),
java.time.LocalDate.parse($F{ultima_venta}))) + " días"]></textFieldExpression></textField>
<textField><reportElement x="460" y="48" width="95" height="18" uuid="42000000-0000-4000-8000-000000000014" style="Dato"/
><textFieldExpression><![CDATA[$F{unidades_vendidas} == null ? "0.0%" :
String.format(java.util.Locale.ROOT, "%1f%%", Double.valueOf($F{unidades_vendidas}.doubleValue() / Math.max(1.0d, $P{umbralUnidades} ==
null ? 1.0d : $P{umbralUnidades}.doubleValue()) * 100.0d)]]></textFieldExpression></textField>
</band>
<band height="14">
<printWhenExpression><![CDATA[$F{unidades_vendidas} != null && $P{umbralUnidades} != null &&
$F{unidades_vendidas}.intValue() >= $P{umbralUnidades}.intValue()]]></printWhenExpression>
<textField><reportElement x="0" y="0" width="555" height="12" uuid="42000000-0000-4000-8000-000000000015"/><textFieldExpression><![CDATA["Fila destacada:
" + $F{titulo} + " supera el umbral de " + $P{umbralUnidades} + " unidades"]></textFieldExpression></textField>
</band>
<band height="88" splitType="Stretch">
<printWhenExpression><![CDATA[$F{unidades_vendidas} != null]]></printWhenExpression>
<staticText>
<reportElement x="0" y="2" width="555" height="16" style="Cabecera"/>
<text><![CDATA[Detalle de ventas]]></text>
</staticText>
<subreport>
<reportElement x="0" y="22" width="555" height="60" isRemoveLineWhenBlank="true"/>
<subreportParameter name="tituloLibro">
<subreportParameterExpression><![CDATA[$F{titulo}]]></subreportParameterExpression>
</subreportParameter>
<connectionExpression><![CDATA[$P{REPORT_CONNECTION}]]></connectionExpression>
<subreportExpression><![CDATA["reports/subinforme_ventas_detalle.jasper"]></subreportExpression>
</subreport>
</band>
<band height="104" splitType="Stretch">
<printWhenExpression><![CDATA[$F{unidades_vendidas} != null]]></printWhenExpression>
<staticText>
<reportElement x="0" y="2" width="555" height="16" style="Cabecera"/>
<text><![CDATA[Top 3 ventas por cantidad]]></text>
</staticText>
<componentElement>
<reportElement x="0" y="22" width="555" height="76"/>
<c:table xmlns:c="http://jasperreports.sourceforge.net/jasperreports/components"
xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports/components http://
jasperreports.sourceforge.net/xsd/components.xsd">
<datasetRun subDataset="DatasetTopVentas">
<datasetParameter name="tituloLibro">
<datasetParameterExpression><![CDATA[$F{titulo}]]></datasetParameterExpression>
</datasetParameter>
<connectionExpression><![CDATA[$P{REPORT_CONNECTION}]]></connectionExpression>
</datasetRun>
<c:column width="255">
<c:columnHeader style="M5TablaCabecera" height="20"><staticText><reportElement x="0" y="0" width="255"
height="20"/><text><![CDATA[Fecha]]></text></staticText></c:columnHeader>
<c:detailCell style="M5TablaDetalle" height="18"><textField><reportElement x="0" y="0" width="255" height="18"/
><textFieldExpression><![CDATA[$F{fecha_venta}]]></textFieldExpression></textField></c:detailCell>
</c:column>
<c:column width="100">
<c:columnHeader style="M5TablaCabecera" height="20"><staticText><reportElement x="0" y="0" width="100"
height="20"/><text><![CDATA[Cantidad]]></text></staticText></c:columnHeader>
<c:detailCell style="M5TablaDetalle" height="18"><textField><reportElement x="0" y="0" width="100" height="18"/
><textFieldExpression><![CDATA[$F{cantidad}]]></textFieldExpression></textField></c:detailCell>
</c:column>
<c:column width="200">
<c:columnHeader style="M5TablaCabecera" height="20"><staticText><reportElement x="0" y="0" width="200"
height="20"/><text><![CDATA[Precio unitario]]></text></staticText></c:columnHeader>
<c:detailCell style="M5TablaDetalle" height="18"><textField pattern="#.##0.00 €"><reportElement x="0" y="0"
width="200" height="18"/><text><![CDATA[$F{precio_unitario}]]></text></textField></c:detailCell>
</c:column>
</c:table>
</componentElement>
</band>
</detail>
<pageFooter>
<band height="62">
<staticText><reportElement x="0" y="4" width="120" height="15" uuid="43000000-0000-4000-8000-000000000001"/><text><![
CDATA[Total de títulos:]]></text></staticText>
<textField evaluationTime="Report"><reportElement x="120" y="4" width="60" height="15"
uuid="43000000-0000-4000-8000-000000000002"/><textFieldExpression><![CDATA[$P{REPORT_COUNT}]]></textFieldExpression></textField>
<textField><reportElement x="190" y="28" width="180" height="15" uuid="43000000-0000-4000-8000-000000000003"/><text><![CDATA["Página " + $V{PAGE_NUMBER} + " de"]></text></staticText>
<textField evaluationTime="Report"><reportElement x="375" y="28" width="35" height="15"
uuid="43000000-0000-4000-8000-000000000004"/><textFieldExpression><![CDATA[$V{PAGE_NUMBER}]]></textFieldExpression></textField>
<staticText><reportElement x="300" y="4" width="120" height="15" uuid="43000000-0000-4000-8000-000000000005"/><text><![
CDATA[Subtotal página:]]></text></staticText>
<textField pattern="#.##0.00 €"><reportElement x="420" y="4" width="135" height="15"
uuid="43000000-0000-4000-8000-000000000006"/><text><![CDATA[$V{TotalPagina}]]></text></staticText>
</band>
</pageFooter>

```

```

textFieldExpression></textField>
</band>
</pageFooter>
<summary>
<band height="700">
<staticText><reportElement x="0" y="5" width="205" height="18" uuid="44000000-0000-4000-8000-000000000001"/><text><![
[CDATA[Total de unidades vendidas:]]></text></staticText>
<textField><reportElement x="205" y="5" width="80" height="18" uuid="44000000-0000-4000-8000-000000000002"/><textElement
textAlignment="Right"/><textFieldExpression><![CDATA[${V{TotalUnidades}}]></textFieldExpression></textField>
<staticText><reportElement x="300" y="5" width="120" height="18" uuid="44000000-0000-4000-8000-000000000003"/><text><![
[CDATA[Importe total:]]></text></staticText>
<textField pattern="#.##0.00 €"><reportElement x="420" y="5" width="135" height="18"
uuid="44000000-0000-4000-8000-000000000004"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[${V{TotalImporte}}]></
textFieldExpression></textField>
<staticText><reportElement x="0" y="30" width="205" height="18" uuid="44000000-0000-4000-8000-000000000005"/><text><![
[CDATA[Precio medio agregado:]]></text></staticText>
<textField pattern="#.##0.00 €"><reportElement x="205" y="30" width="80" height="18"
uuid="44000000-0000-4000-8000-000000000006"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[${V{PrecioMedio}}]></
textFieldExpression></textField>
<staticText><reportElement x="300" y="30" width="120" height="18" uuid="44000000-0000-4000-8000-000000000007"/><text><![
[CDATA[Precio máximo:]]></text></staticText>
<textField pattern="#.##0.00 €"><reportElement x="420" y="30" width="135" height="18"
uuid="44000000-0000-4000-8000-000000000008"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[${V{PrecioMaximo}}]></
textFieldExpression></textField>
<staticText><reportElement x="0" y="55" width="205" height="18" uuid="44000000-0000-4000-8000-000000000009"/><text><![
[CDATA[Número de libros:]]></text></staticText>
<textField><reportElement x="205" y="55" width="80" height="18" uuid="44000000-0000-4000-8000-000000000010"/><textElement
textAlignment="Right"/><textFieldExpression><![CDATA[${V{NumeroLibros}}]></textFieldExpression></textField>
<staticText><reportElement x="300" y="55" width="120" height="18" uuid="44000000-0000-4000-8000-000000000011"/><text><![
[CDATA[Importe con IVA:]]></text></staticText>
<textField pattern="#.##0.00 €"><reportElement x="420" y="55" width="135" height="18"
uuid="44000000-0000-4000-8000-000000000012"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[${V{ImporteConIva}}]></
textFieldExpression></textField>
<textField><reportElement x="0" y="80" width="555" height="18" uuid="44000000-0000-4000-8000-000000000013"/><textElement
textAlignment="Center"/><textFieldExpression><![CDATA[String.format(java.util.Locale.ROOT, "Resumen: %d títulos · %d unidades · %.2f €",
${V{NumeroLibros}}, ${V{TotalUnidades}}, ${V{TotalImporte}})]></textFieldExpression></textField>
<textField><reportElement x="0" y="103" width="350" height="18" uuid="44000000-0000-4000-8000-000000000014"/><textElement
textAlignment="Center"><font fontName="DejaVu Sans" size="10" isBold="true"/></textElement><textFieldExpression><![
[CDATA[${V{TotalUnidades}} != null && ${P{umbralUnidades}} != null && ${V{TotalUnidades}.intValue() >= ${P{umbralUnidades}.intValue() ?
"Objetivo de ventas alcanzado" : "Objetivo de ventas pendiente"}]]></textFieldExpression></textField>
<textField><reportElement x="360" y="103" width="195" height="18" uuid="44000000-0000-4000-8000-000000000015"/>
><textFieldExpression><![CDATA["Resultados encontrados: " + ${V{REPORT_COUNT}}]]></textFieldExpression></textField>
<staticText><reportElement x="0" y="140" width="555" height="20" style="Cabecera"/><text><![CDATA[Ventas por categoría –
importe]]></text></staticText>
<barChart>
<chart>
<reportElement x="0" y="165" width="555" height="250"/>
<chartTitle><titleExpression><![CDATA["Ventas por categoría"]]></titleExpression></chartTitle>
<chartSubtitle/>
<chartLegend position="Bottom"/>
</chart>
<categoryDataset>
<dataset>
<datasetRun subDataset="DatasetVentasPorCategoria">
<connectionExpression><![CDATA[${P{REPORT_CONNECTION}}]></connectionExpression>
</datasetRun>
</dataset>
<categorySeries>
<seriesExpression><![CDATA["Importe"]]></seriesExpression>
<categoryExpression><![CDATA[${F{categoria_grafico}}]></categoryExpression>
<valueExpression><![CDATA[${F{importe_categoria}}]></valueExpression>
</categorySeries>
</categoryDataset>
<barPlot>
<plot/>
<itemLabel/>
<categoryAxisFormat><axisFormat/></categoryAxisFormat>
<valueAxisFormat><axisFormat/></valueAxisFormat>
</barPlot>
</barChart>
<staticText><reportElement x="0" y="430" width="555" height="20" style="Cabecera"/><text><![CDATA[Ventas por categoría y
año]]></text></staticText>
<crosstab>
<reportElement x="0" y="455" width="555" height="225"/>
<crosstabDataset>
<dataset>
<datasetRun subDataset="DatasetCrosstabVentas">
<connectionExpression><![CDATA[${P{REPORT_CONNECTION}}]></connectionExpression>
</datasetRun>
</dataset>
</crosstabDataset>
<rowGroup name="CategoriaCross" width="150" totalPosition="End">
<bucket class="java.lang.String"><bucketExpression><![CDATA[${F{categoria_cross}}]></bucketExpression></bucket>
<crosstabRowHeader><cellContents style="M5CrosstabCabecera"><textField><reportElement x="0" y="0" width="150"
height="34"/><textElement verticalAlignment="Middle"/><textFieldExpression><![CDATA[${V{CategoriaCross}}]></textFieldExpression></
textField></cellContents></crosstabRowHeader>
<crosstabTotalRowHeader><cellContents style="M5CrosstabTotal"><staticText><reportElement x="0" y="0" width="150"
height="34"/><textElement verticalAlignment="Middle"/><text><![CDATA[TOTAL]]></text></staticText></cellContents></
crosstabTotalRowHeader>
</rowGroup>
<columnGroup name="AnioCross" height="28" totalPosition="End">
<bucket class="java.lang.String"><bucketExpression><![CDATA[${F{anio_cross}}]></bucketExpression></bucket>

```

```

        <crossstabColumnHeader><cellContents style="M5CrosstabCabecera"><textField><reportElement x="0" y="0" width="100"
height="28"/><textFieldExpression><![CDATA[$V{AnioCross}]]></textFieldExpression></cellContents></crossstabColumnHeader>
        <crossstabTotalColumnHeader><cellContents style="M5CrosstabTotal"><staticText><reportElement x="0" y="0" width="100"
height="28"/><textFieldExpression><![CDATA[TOTAL]]></textFieldExpression></staticText></cellContents></crossstabTotalColumnHeader>
    </columnGroup>
    <measure name="ImporteCross" class="java.lang.Double" calculation="Sum"><measureExpression><![CDATA[$F{importe_cross}]]></measureExpression></measure>
    <measure name="VentasCross" class="java.lang.Integer" calculation="Sum"><measureExpression><![CDATA[$F{ventas_cross}]]></measureExpression></measure>
    <crossstabCell width="100" height="34"><cellContents style="M5CrosstabDetalle"><textField pattern="#,##0.00
€"><reportElement x="0" y="0" width="100" height="18"/><textFieldExpression><![CDATA[$V{ImporteCross}]]></textFieldExpression></textField><textField><reportElement x="0" y="18" width="100" height="14"/><textFieldExpression><![CDATA[$V{VentasCross} + " ventas"]]></textFieldExpression></textField></cellContents></crossstabCell>
    <crossstabCell width="100" height="34" rowTotalGroup="CategoriaCross"><cellContents style="M5CrosstabTotal"><textField pattern="#,##0.00 €"><reportElement x="0" y="0" width="100" height="18"/><textFieldExpression><![CDATA[$V{ImporteCross}]]></textFieldExpression></textField><textField><reportElement x="0" y="18" width="100" height="14"/><textFieldExpression><![CDATA[$V{VentasCross} + " ventas"]]></textFieldExpression></textField></cellContents></crossstabCell>
    <crossstabCell width="100" height="34" columnTotalGroup="AnioCross"><cellContents style="M5CrosstabTotal"><textField pattern="#,##0.00 €"><reportElement x="0" y="0" width="100" height="18"/><textFieldExpression><![CDATA[$V{ImporteCross}]]></textFieldExpression></textField><textField><reportElement x="0" y="18" width="100" height="14"/><textFieldExpression><![CDATA[$V{VentasCross} + " ventas"]]></textFieldExpression></textField></cellContents></crossstabCell>
    <crossstabCell width="100" height="34" rowTotalGroup="CategoriaCross" columnTotalGroup="AnioCross"><cellContents style="M5CrosstabTotal"><textField pattern="#,##0.00 €"><reportElement x="0" y="0" width="100" height="18"/><textFieldExpression><![CDATA[$V{ImporteCross}]]></textFieldExpression></textField><textField><reportElement x="0" y="18" width="100" height="14"/><textFieldExpression><![CDATA[$V{VentasCross} + " ventas"]]></textFieldExpression></textField></cellContents></crossstabCell>
</crossstab>
</band>
</summary>
</jasperReport>

```

Explicación línea por línea

Línea 1	<?xml version="1.0" encoding="UTF-8"?> → Declara XML 1.0 y codificación UTF-8 para que nombres, textos y símbolos del informe se interpreten correctamente.
Línea 2	<jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports" → Abre el documento raíz jasperReport del informe y fija el namespace principal de JasperReports.
Línea 3	xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" → Declara el namespace XML Schema Instance usado por xsi:schemaLocation para validar el documento.
Línea 4	xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/jasperreport.xsd" → Relaciona el namespace de JasperReports con su XSD para que Studio y el compilador validen la estructura.
Línea 5	name="informe_ventas" → Asigna al documento JasperReports el nombre interno informe_ventas.
Línea 6	language="java" → Configura language=java para evaluar expresiones con el lenguaje Java.
Línea 7	pageWidth="595" → Fija el ancho físico de página en 595 puntos.
Línea 8	pageHeight="842" → Fija la altura física de página en 842 puntos.
Línea 9	columnWidth="555" → Fija el ancho útil de la columna de contenido en 555 puntos.
Línea 10	leftMargin="20" → Fija el margen izquierdo del informe en 20 puntos.
Línea 11	rightMargin="20" → Fija el margen derecho del informe en 20 puntos.
Línea 12	topMargin="20" → Fija el margen superior del informe en 20 puntos.
Línea 13	bottomMargin="20" → Fija el margen inferior del informe en 20 puntos y completa la apertura del elemento raíz.
Línea 14	uuid="3d2c2bd7-3b93-4da9-8b60-6b3c45674c91"> → Asigna el UUID de diseño 3d2c2bd7-3b93-4da9-8b60-6b3c45674c91 para identificar de forma estable el informe en Studio.
Línea 15	<property name="com.jaspersoft.studio.data.defaultdataadapter" value="SQLiteEditorial"/> → Indica a Jaspersoft Studio que use el Data Adapter SQLiteEditorial como conexión de diseño por defecto.

Línea 16	<code><template><![CDATA["resources/styles/EditorialStyles.jrtx"]]></template></code> → Importa la plantilla externa cuya expresión CDATA devuelve <code>resources/styles/EditorialStyles.jrtx</code> .
Línea 17	<code><style name="Sans_Normal" isDefault="true" fontName="DejaVu Sans" fontSize="10"/></code> → Declara el estilo <code>Sans_Normal</code> ; es el estilo por defecto, fuente <code>DejaVu Sans</code> , tamaño 10.
Línea 18	<code><style name="TituloPrincipal" style="Sans_Normal" fontSize="18" isBold="true" forecolor="#173F6B"/></code> → Declara el estilo <code>TituloPrincipal</code> ; hereda de <code>Sans_Normal</code> , tamaño 18.
Línea 19	<code><style name="Cabecera" style="Sans_Normal" fontSize="9" isBold="true" forecolor="#173F6B"/></code> → Declara el estilo <code>Cabecera</code> ; hereda de <code>Sans_Normal</code> , tamaño 9.
Línea 20	<code><style name="Dato" style="Sans_Normal" fontSize="9"/></code> → Declara el estilo <code>Dato</code> ; hereda de <code>Sans_Normal</code> , tamaño 9.
Línea 21	<code><style name="UnidadesCondicional" style="Dato" isBold="true"></code> → Declara el estilo <code>UnidadesCondicional</code> ; hereda de <code>Dato</code> .
Línea 22	<code><conditionalStyle></code> → Abre una variante condicional del estilo; sólo se aplicará cuando su <code>conditionExpression</code> sea verdadera.
Línea 23	<code><conditionExpression><![CDATA[\$F{unidades_vendidas} != null && \$P{umbralUnidades} != null && \$F{unidades_vendidas}.intValue() >= \$P{umbralUnidades}.intValue()]]></conditionExpres...></code> → Define la condición booleana que activa el estilo condicional usando field <code>unidades_vendidas</code> , field <code>unidades_vendidas</code> , parámetro <code>umbralUnidades</code> , parámetro <code>umbralUnidades</code> .
Línea 24	<code><style forecolor="#1B5E20"/></code> → Declara el estilo <code>None</code> .
Línea 25	<code></conditionalStyle></code> → Cierra <code>conditionalStyle</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 26	<code><conditionalStyle></code> → Abre una variante condicional del estilo; sólo se aplicará cuando su <code>conditionExpression</code> sea verdadera.
Línea 27	<code><conditionExpression><![CDATA[\$F{unidades_vendidas} != null && \$P{umbralUnidades} != null && \$F{unidades_vendidas}.intValue() >= 3 && \$F{unidades_vendidas}.intValue() < \$P{umbralUnidades}.intValue()]]></conditionExpression></code> → Define la condición booleana que activa el estilo condicional usando field <code>unidades_vendidas</code> , field <code>unidades_vendidas</code> , parámetro <code>umbralUnidades</code> , parámetro <code>umbralUnidades</code> .
Línea 28	<code><style forecolor="#1D5D88"/></code> → Declara el estilo <code>None</code> .
Línea 29	<code></conditionalStyle></code> → Cierra <code>conditionalStyle</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 30	<code><conditionalStyle></code> → Abre una variante condicional del estilo; sólo se aplicará cuando su <code>conditionExpression</code> sea verdadera.
Línea 31	<code><conditionExpression><![CDATA[\$F{unidades_vendidas} == null \$F{unidades_vendidas}.intValue() < 3]]></conditionExpression></code> → Define la condición booleana que activa el estilo condicional usando field <code>unidades_vendidas</code> , field <code>unidades_vendidas</code> .
Línea 32	<code><style forecolor="#9D3429"/></code> → Declara el estilo <code>None</code> .
Línea 33	<code></conditionalStyle></code> → Cierra <code>conditionalStyle</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 34	<code></style></code> → Cierra <code>style</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 35	<code><style name="M5TableHeader" style="Dato" mode="Opaque" backcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/></code> → Declara el estilo <code>M5TableHeader</code> ; hereda de <code>Dato</code> , fondo <code>#EAF2F8</code> .
Línea 36	<code><style name="M5TableDetail" style="Dato"/></code> → Declara el estilo <code>M5TableDetail</code> ; hereda de <code>Dato</code> .
Línea 37	<code><style name="M5CrossHeader" style="Dato" mode="Opaque" backcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/></code> → Declara el estilo <code>M5CrossHeader</code> ; hereda de <code>Dato</code> , fondo <code>#EAF2F8</code> .
Línea 38	<code><style name="M5CrossDetail" style="Dato" mode="Opaque" backcolor="FFFFFF"/></code> → Declara el estilo <code>M5CrossDetail</code> ; hereda de <code>Dato</code> , fondo <code>FFFFFF</code> .
Línea 39	<code><style name="M5CrossTotal" style="Dato" mode="Opaque" backcolor="#D6EAF8" forecolor="#173F6B" isBold="true"/></code> → Declara el estilo <code>M5CrossTotal</code> ; hereda de <code>Dato</code> , fondo <code>#D6EAF8</code> .

Línea 40	<code><subDataset name="DatasetTopVentas"></code> → Declara el subdataset DatasetTopVentas, con consulta y fields propios independientes del dataset principal.
Línea 41	<code><parameter name="tituloLibro" class="java.lang.String"/></code> → Declara el parámetro tituloLibro con tipo java.lang.String y lo expone al diálogo de parámetros de Studio.
Línea 42	<code><queryString language="sql"></code> → Abre la consulta SQL que JasperReports ejecutará para el dataset actual.
Línea 43	<code><![CDATA[</code> → Abre CDATA para escribir SQL o una expresión Java sin que sus caracteres especiales se interpreten como XML.
Línea 44	<code>SELECT fecha_venta, cantidad, precio_unitario</code> → Cláusula SQL SELECT: selecciona y calcula las columnas que devolverá la consulta.
Línea 45	<code>FROM ventas</code> → Cláusula SQL FROM: define la tabla base de la consulta.
Línea 46	<code>WHERE titulo_libro = \$P{tituloLibro}</code> → Cláusula SQL WHERE: aplica el filtro de filas.
Línea 47	<code>ORDER BY cantidad DESC, fecha_venta</code> → Cláusula SQL ORDER BY: ordena el resultado que recibirá JasperReports.
Línea 48	<code>LIMIT 3</code> → Cláusula SQL LIMIT: limita el número de filas devueltas.
Línea 49	<code>]]></code> → Cierra el bloque CDATA y devuelve el control al parser XML.
Línea 50	<code></queryString></code> → Cierra la consulta SQL del dataset actual.
Línea 51	<code><field name="fecha_venta" class="java.lang.String"/></code> → Declara el field fecha_venta con tipo Java java.lang.String para mapear una columna del dataset.
Línea 52	<code><field name="cantidad" class="java.lang.Integer"/></code> → Declara el field cantidad con tipo Java java.lang.Integer para mapear una columna del dataset.
Línea 53	<code><field name="precio_unitario" class="java.lang.Double"/></code> → Declara el field precio_unitario con tipo Java java.lang.Double para mapear una columna del dataset.
Línea 54	<code></subDataset></code> → Finaliza el subdataset auxiliar y vuelve al nivel del informe.
Línea 55	<code><subDataset name="DatasetVentasPorCategoria"></code> → Declara el subdataset DatasetVentasPorCategoria, con consulta y fields propios independientes del dataset principal.
Línea 56	<code><queryString language="sql"></code> → Abre la consulta SQL que JasperReports ejecutará para el dataset actual.
Línea 57	<code><![CDATA[</code> → Abre CDATA para escribir SQL o una expresión Java sin que sus caracteres especiales se interpreten como XML.
Línea 58	<code>SELECT l.categoria AS categoria_grafico,</code> → Cláusula SQL SELECT: selecciona y calcula las columnas que devolverá la consulta.
Línea 59	<code>COALESCE(SUM(v.cantidad * v.precio_unitario), 0.0) AS importe_categoria</code> → Calcula o selecciona un valor SQL y lo expone con el alias importe_categoria, que después coincide con un field del subdataset.
Línea 60	<code>FROM libros l</code> → Cláusula SQL FROM: define la tabla base de la consulta.
Línea 61	<code>LEFT JOIN ventas v ON l.titulo = v.titulo_libro</code> → Cláusula SQL LEFT JOIN: une datos conservando las filas del lado izquierdo aunque no tengan ventas.
Línea 62	<code>GROUP BY l.categoria</code> → Cláusula SQL GROUP BY: agrupa las filas antes de evaluar las funciones agregadas.
Línea 63	<code>ORDER BY l.categoria</code> → Cláusula SQL ORDER BY: ordena el resultado que recibirá JasperReports.
Línea 64	<code>]]></code> → Cierra el bloque CDATA y devuelve el control al parser XML.
Línea 65	<code></queryString></code> → Cierra la consulta SQL del dataset actual.
Línea 66	<code><field name="categoria_grafico" class="java.lang.String"/></code> → Declara el field categoria_grafico con tipo Java java.lang.String para mapear una columna del dataset.
Línea 67	<code><field name="importe_categoria" class="java.lang.Double"/></code> → Declara el field importe_categoria con tipo Java java.lang.Double para mapear una columna del dataset.

Línea 68	<code></subDataset></code> → Finaliza el subdataset auxiliar y vuelve al nivel del informe.
Línea 69	<code><subDataset name="DatasetCrosstabVentas"></code> → Declara el subdataset DatasetCrosstabVentas, con consulta y fields propios independientes del dataset principal.
Línea 70	<code><queryString language="sql"></code> → Abre la consulta SQL que JasperReports ejecutará para el dataset actual.
Línea 71	<code><![CDATA[</code> → Abre CDATA para escribir SQL o una expresión Java sin que sus caracteres especiales se interpreten como XML.
Línea 72	<code>SELECT l.categoria AS categoria_cross,</code> → Cláusula SQL SELECT: selecciona y calcula las columnas que devolverá la consulta.
Línea 73	<code>SUBSTR(v.fecha_venta, 1, 4) AS anio_cross,</code> → Calcula o selecciona un valor SQL y lo expone con el alias anio_cross, que después coincide con un field del subdataset.
Línea 74	<code>(v.cantidad * v.precio_unitario) AS importe_cross,</code> → Calcula o selecciona un valor SQL y lo expone con el alias importe_cross, que después coincide con un field del subdataset.
Línea 75	<code>1 AS ventas_cross</code> → Calcula o selecciona un valor SQL y lo expone con el alias ventas_cross, que después coincide con un field del subdataset.
Línea 76	<code>FROM ventas v</code> → Cláusula SQL FROM: define la tabla base de la consulta.
Línea 77	<code>JOIN libros l ON l.titulo = v.titulo_libro</code> → Cláusula SQL JOIN: une las filas que cumplen la relación indicada.
Línea 78	<code>ORDER BY l.categoria, anio_cross, v.fecha_venta</code> → Cláusula SQL ORDER BY: ordena el resultado que recibirá JasperReports.
Línea 79	<code>]]></code> → Cierra el bloque CDATA y devuelve el control al parser XML.
Línea 80	<code></queryString></code> → Cierra la consulta SQL del dataset actual.
Línea 81	<code><field name="categoria_cross" class="java.lang.String"/></code> → Declara el field categoria_cross con tipo Java java.lang.String para mapear una columna del dataset.
Línea 82	<code><field name="anio_cross" class="java.lang.String"/></code> → Declara el field anio_cross con tipo Java java.lang.String para mapear una columna del dataset.
Línea 83	<code><field name="importe_cross" class="java.lang.Double"/></code> → Declara el field importe_cross con tipo Java java.lang.Double para mapear una columna del dataset.
Línea 84	<code><field name="ventas_cross" class="java.lang.Integer"/></code> → Declara el field ventas_cross con tipo Java java.lang.Integer para mapear una columna del dataset.
Línea 85	<code></subDataset></code> → Finaliza el subdataset auxiliar y vuelve al nivel del informe.
Línea 86	<code><parameter name="usuario" class="java.lang.String" isForPrompting="true"/></code> → Declara el parámetro usuario con tipo java.lang.String y lo expone al diálogo de parámetros de Studio.
Línea 87	<code><parameter name="fechaInforme" class="java.util.Date" isForPrompting="true"></code> → Declara el parámetro fechaInforme con tipo java.util.Date y lo expone al diálogo de parámetros de Studio.
Línea 88	<code><defaultValueExpression><![CDATA[new java.util.Date()]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 89	<code></parameter></code> → Cierra parameter y vuelve al elemento padre de la jerarquía JRXML.
Línea 90	<code><parameter name="departamento" class="java.lang.String" isForPrompting="true"></code> → Declara el parámetro departamento con tipo java.lang.String y lo expone al diálogo de parámetros de Studio.
Línea 91	<code><defaultValueExpression><![CDATA["General"]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 92	<code></parameter></code> → Cierra parameter y vuelve al elemento padre de la jerarquía JRXML.
Línea 93	<code><parameter name="periodo" class="java.lang.String" isForPrompting="true"></code> → Declara el parámetro periodo con tipo java.lang.String y lo expone al diálogo de parámetros de Studio.

Línea 94	<code><defaultValueExpression><![CDATA["Mensual"]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 95	<code></parameter></code> → Cierra parameter y vuelve al elemento padre de la jerarquía JRXML.
Línea 96	<code><parameter name="tipoIva" class="java.lang.Double" isForPrompting="true"></code> → Declara el parámetro tipoIva con tipo <code>java.lang.Double</code> y lo expone al diálogo de parámetros de Studio.
Línea 97	<code><defaultValueExpression><![CDATA[Double.valueOf(0.21d)]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 98	<code></parameter></code> → Cierra parameter y vuelve al elemento padre de la jerarquía JRXML.
Línea 99	<code><parameter name="mostrarDetalle" class="java.lang.Boolean" isForPrompting="true"></code> → Declara el parámetro mostrarDetalle con tipo <code>java.lang.Boolean</code> y lo expone al diálogo de parámetros de Studio.
Línea 100	<code><defaultValueExpression><![CDATA[Boolean.TRUE]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 101	<code></parameter></code> → Cierra parameter y vuelve al elemento padre de la jerarquía JRXML.
Línea 102	<code><parameter name="categoria" class="java.lang.String" isForPrompting="true"/></code> → Declara el parámetro categoria con tipo <code>java.lang.String</code> y lo expone al diálogo de parámetros de Studio.
Línea 103	<code><parameter name="precioMinimo" class="java.lang.Double" isForPrompting="true"/></code> → Declara el parámetro precioMinimo con tipo <code>java.lang.Double</code> y lo expone al diálogo de parámetros de Studio.
Línea 104	<code><parameter name="precioMaximo" class="java.lang.Double" isForPrompting="true"/></code> → Declara el parámetro precioMaximo con tipo <code>java.lang.Double</code> y lo expone al diálogo de parámetros de Studio.
Línea 105	<code><parameter name="umbralUnidades" class="java.lang.Integer" isForPrompting="true"></code> → Declara el parámetro umbralUnidades con tipo <code>java.lang.Integer</code> y lo expone al diálogo de parámetros de Studio.
Línea 106	<code><defaultValueExpression><![CDATA[Integer.valueOf(5)]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 107	<code></parameter></code> → Cierra parameter y vuelve al elemento padre de la jerarquía JRXML.
Línea 108	<code><parameter name="textoBusqueda" class="java.lang.String" isForPrompting="true"/></code> → Declara el parámetro textoBusqueda con tipo <code>java.lang.String</code> y lo expone al diálogo de parámetros de Studio.
Línea 109	<code><parameter name="categoriasLista" class="java.util.Collection" isForPrompting="false"></code> → Declara el parámetro categoriasLista con tipo <code>java.util.Collection</code> como parámetro interno no solicitado al usuario.
Línea 110	<code><defaultValueExpression><![CDATA[java.util.Arrays.asList("Novela", "Realismo mágico", "Cuento", "Poesía")]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 111	<code></parameter></code> → Cierra parameter y vuelve al elemento padre de la jerarquía JRXML.
Línea 112	<code><queryString language="sql"></code> → Abre la consulta SQL que JasperReports ejecutará para el dataset actual.
Línea 113	<code><![CDATA[</code> → Abre CDATA para escribir SQL o una expresión Java sin que sus caracteres especiales se interpreten como XML.
Línea 114	<code>SELECT l.titulo,</code> → Cláusula SQL SELECT: selecciona y calcula las columnas que devolverá la consulta.
Línea 115	<code>l.categoria,</code> → Aporta el valor literal <code>l.categoria</code> , al elemento XML/POM actualmente abierto.
Línea 116	<code>SUM(v.cantidad) AS unidades_vendidas,</code> → Calcula o selecciona un valor SQL y lo expone con el alias <code>unidades_vendidas</code> , que después coincide con un field del subdataset.
Línea 117	<code>SUM(v.cantidad * v.precio_unitario) AS importe_total,</code> → Calcula o selecciona un valor SQL y lo expone con el alias <code>importe_total</code> , que después coincide con un field del subdataset.
Línea 118	<code>AVG(v.precio_unitario) AS precio_medio,</code> → Calcula o selecciona un valor SQL y lo expone con el alias <code>precio_medio</code> , que después coincide con un field del subdataset.

Línea 119	<code>MIN(v.fecha_venta) AS primera_venta</code> , → Calcula o selecciona un valor SQL y lo expone con el alias <code>primera_venta</code> , que después coincide con un field del subdataset.
Línea 120	<code>MAX(v.fecha_venta) AS ultima_venta</code> → Calcula o selecciona un valor SQL y lo expone con el alias <code>ultima_venta</code> , que después coincide con un field del subdataset.
Línea 121	<code>FROM libros l</code> → Cláusula SQL <code>FROM</code> : define la tabla base de la consulta.
Línea 122	<code>LEFT JOIN ventas v ON l.titulo = v.titulo_libro</code> → Cláusula SQL <code>LEFT JOIN</code> : une datos conservando las filas del lado izquierdo aunque no tengan ventas.
Línea 123	<code>WHERE (\$P{categoria} IS NULL OR l.categoria = \$P{categoria})</code> → Cláusula SQL <code>WHERE</code> : aplica el filtro de filas.
Línea 124	<code>AND (\$P{precioMinimo} IS NULL OR l.precio >= \$P{precioMinimo})</code> → Añade otra condición lógica al filtro SQL, combinándola con la condición anterior.
Línea 125	<code>AND (\$P{precioMaximo} IS NULL OR l.precio <= \$P{precioMaximo})</code> → Añade otra condición lógica al filtro SQL, combinándola con la condición anterior.
Línea 126	<code>AND (\$P{textoBusqueda} IS NULL OR \$P{textoBusqueda} = '' OR l.titulo LIKE '%' \$P{textoBusqueda} '%')</code> → Añade otra condición lógica al filtro SQL, combinándola con la condición anterior.
Línea 127	<code>AND \$X{IN, l.categoria, categoriasLista}</code> → Añade otra condición lógica al filtro SQL, combinándola con la condición anterior.
Línea 128	<code>GROUP BY l.titulo, l.categoria</code> → Cláusula SQL <code>GROUP BY</code> : agrupa las filas antes de evaluar las funciones agregadas.
Línea 129	<code>ORDER BY l.categoria, COALESCE(importe_total, 0) DESC, l.titulo</code> → Cláusula SQL <code>ORDER BY</code> : ordena el resultado que recibirá JasperReports.
Línea 130	<code>]]></code> → Cierra el bloque <code>CDATA</code> y devuelve el control al parser XML.
Línea 131	<code></queryString></code> → Cierra la consulta SQL del dataset actual.
Línea 132	<code><field name="titulo" class="java.lang.String"/></code> → Declara el field <code>titulo</code> con tipo Java <code>java.lang.String</code> para mapear una columna del dataset.
Línea 133	<code><field name="categoria" class="java.lang.String"/></code> → Declara el field <code>categoria</code> con tipo Java <code>java.lang.String</code> para mapear una columna del dataset.
Línea 134	<code><field name="unidades_vendidas" class="java.lang.Integer"/></code> → Declara el field <code>unidades_vendidas</code> con tipo Java <code>java.lang.Integer</code> para mapear una columna del dataset.
Línea 135	<code><field name="importe_total" class="java.lang.Double"/></code> → Declara el field <code>importe_total</code> con tipo Java <code>java.lang.Double</code> para mapear una columna del dataset.
Línea 136	<code><field name="precio_medio" class="java.lang.Double"/></code> → Declara el field <code>precio_medio</code> con tipo Java <code>java.lang.Double</code> para mapear una columna del dataset.
Línea 137	<code><field name="primera_venta" class="java.lang.String"/></code> → Declara el field <code>primera_venta</code> con tipo Java <code>java.lang.String</code> para mapear una columna del dataset.
Línea 138	<code><field name="ultima_venta" class="java.lang.String"/></code> → Declara el field <code>ultima_venta</code> con tipo Java <code>java.lang.String</code> para mapear una columna del dataset.
Línea 139	<code><variable name="TotalUnidades" class="java.lang.Integer" calculation="Sum" resetType="Report"></code> → Declara la variable <code>TotalUnidades</code> con cálculo <code>Sum</code> y reinicio <code>Report</code> .
Línea 140	<code><variableExpression><![CDATA[\$F{unidades_vendidas}]]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field <code>unidades_vendidas</code> .
Línea 141	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 142	<code><variable name="TotalImporte" class="java.lang.Double" calculation="Sum" resetType="Report"></code> → Declara la variable <code>TotalImporte</code> con cálculo <code>Sum</code> y reinicio <code>Report</code> .

Línea 143	<code><variableExpression><![CDATA[{\$importe_total}]]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field importe_total.
Línea 144	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 145	<code><variable name="TotalPagina" class="java.lang.Double" calculation="Sum" resetType="Page"></code> → Declara la variable TotalPagina con cálculo Sum y reinicio Page.
Línea 146	<code><variableExpression><![CDATA[{\$importe_total}]]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field importe_total.
Línea 147	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 148	<code><variable name="PrecioMedio" class="java.lang.Double" calculation="Average" resetType="Report"></code> → Declara la variable PrecioMedio con cálculo Average y reinicio Report.
Línea 149	<code><variableExpression><![CDATA[{\$precio_medio}]]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field precio_medio.
Línea 150	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 151	<code><variable name="PrecioMaximo" class="java.lang.Double" calculation="Highest" resetType="Report"></code> → Declara la variable PrecioMaximo con cálculo Highest y reinicio Report.
Línea 152	<code><variableExpression><![CDATA[{\$precio_medio}]]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field precio_medio.
Línea 153	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 154	<code><variable name="NumeroLibros" class="java.lang.Integer" calculation="Count" resetType="Report"></code> → Declara la variable NumeroLibros con cálculo Count y reinicio Report.
Línea 155	<code><variableExpression><![CDATA[{\$titulo}]]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field titulo.
Línea 156	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 157	<code><variable name="ImporteConIva" class="java.lang.Double" calculation="Sum" resetType="Report"></code> → Declara la variable ImporteConIva con cálculo Sum y reinicio Report.
Línea 158	<code><variableExpression><![CDATA[{\$importe_total} == null \$P{tipoIva} == null ? null : Double.valueOf(\$F{importe_total}.doubleValue() * (1.0d + \$P{tipoIva}.doubleValue()))]]></v...></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field importe_total, field importe_total, parámetro tipoIva, parámetro tipoIva.
Línea 159	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 160	<code><variable name="GrupoUnidades" class="java.lang.Integer" calculation="Sum" resetType="Group" resetGroup="CategoriaGroup"></code> → Declara la variable GrupoUnidades con cálculo Sum y reinicio Group asociado a CategoriaGroup.
Línea 161	<code><variableExpression><![CDATA[{\$unidades_vendidas}]]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field unidades_vendidas.
Línea 162	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 163	<code><variable name="GrupoImporte" class="java.lang.Double" calculation="Sum" resetType="Group" resetGroup="CategoriaGroup"></code> → Declara la variable GrupoImporte con cálculo Sum y reinicio Group asociado a CategoriaGroup.
Línea 164	<code><variableExpression><![CDATA[{\$importe_total}]]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field importe_total.
Línea 165	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 166	<code><variable name="GrupoLibros" class="java.lang.Integer" calculation="Count" resetType="Group" resetGroup="CategoriaGroup"></code> → Declara la variable GrupoLibros con cálculo Count y reinicio Group asociado a CategoriaGroup.

Línea 167	<code><variableExpression><![CDATA[{\$F{titulo}}]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field titulo.
Línea 168	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 169	<code><group name="CategoriaGroup" isStartNewPage="false" isReprintHeaderOnEachPage="true" minHeightToStartNewPage="80"></code> → Declara el grupo CategoriaGroup y sus propiedades de paginación/reimpresión.
Línea 170	<code><groupExpression><![CDATA[{\$F{categoria}}]></groupExpression></code> → Define la clave que decide cuándo cambia el grupo mediante field categoria.
Línea 171	<code><groupHeader></code> → Abre la cabecera del grupo, que se emite cuando comienza cada nuevo valor de agrupación.
Línea 172	<code><band height="28"></code> → Define una banda de 28 puntos, reservando ese espacio para sus elementos.
Línea 173	<code><textField><reportElement x="0" y="2" width="555" height="22" style="M5GrupoCabecera"/><textFieldExpression><![CDATA["Categoría: " + {\$F{categoria}}]></textFieldExpression></textField></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=2, ancho=555, alto=22 y aplica el estilo M5GrupoCabecera; evalúa la expresión usando field categoria.
Línea 174	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 175	<code></groupHeader></code> → Cierra groupHeader y vuelve al elemento padre de la jerarquía JRXML.
Línea 176	<code><groupFooter></code> → Abre el pie del grupo, donde se muestran los acumulados justo antes de cambiar de grupo.
Línea 177	<code><band height="34"></code> → Define una banda de 34 puntos, reservando ese espacio para sus elementos.
Línea 178	<code><textField><reportElement x="0" y="3" width="185" height="18" style="Dato"/><textFieldExpression><![CDATA["Libros del grupo: " + {\$V{GrupoLibros}}]></textFieldExpression></textField></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=3, ancho=185, alto=18 y aplica el estilo Dato; evalúa la expresión usando variable GrupoLibros.
Línea 179	<code><textField><reportElement x="185" y="3" width="180" height="18" style="Dato"/><textFieldExpression><![CDATA["Unidades: " + (\$V{GrupoUnidades} == null ? 0 : {\$V{GrupoUnidades}})]></textFieldExpression></textField></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=185, y=3, ancho=180, alto=18 y aplica el estilo Dato; evalúa la expresión usando variable GrupoUnidades, variable GrupoUnidades.
Línea 180	<code><textField><reportElement x="365" y="3" width="190" height="18" style="Dato"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA["Importe: " + new java.text.DecimalFormat("#,###.##").format(\$F{GrupoImporte})]></textFieldExpression></textField></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=365, y=3, ancho=190, alto=18 y aplica el estilo Dato; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable GrupoImporte, variable GrupoImporte.
Línea 181	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 182	<code></groupFooter></code> → Cierra groupFooter y vuelve al elemento padre de la jerarquía JRXML.
Línea 183	<code></group></code> → Finaliza la definición del grupo y sus bandas asociadas.
Línea 184	<code><background><band height="0"/></background></code> → Composición de la línea: encadena además <background>, <band> dentro de la misma jerarquía.
Línea 185	<code><title></code> → Abre la banda Title, emitida una sola vez al inicio del informe.
Línea 186	<code><band height="124"></code> → Define una banda de 124 puntos, reservando ese espacio para sus elementos.
Línea 187	<code><staticText></code> → Abre un elemento de texto literal; su contenido no depende de fields, parámetros ni variables.
Línea 188	<code><reportElement x="0" y="4" width="555" height="28" uuid="40000000-0000-4000-8000-000000000001" style="M5TituloPrincipal"/></code> → Posiciona el elemento en x=0, y=4, con ancho 555 y alto 28, aplicando el estilo M5TituloPrincipal.
Línea 189	<code><textElement textAlignment="Center" verticalAlignment="Middle"/></code> → Configura el formato interno del texto: alineación horizontal Center, alineación vertical Middle.

Línea 190	<code><text><![CDATA[Informe de Ventas - Agregación por Título]]></text></code> → Define el texto literal visible: Informe de Ventas - Agregación por Título.
Línea 191	<code></staticText></code> → Cierra <code>staticText</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 192	<code><staticText><reportElement x="0" y="38" width="110" height="18" uuid="40000000-0000-4000-8000-000000000002"/><text><![CDATA[Generado por:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=38, ancho=110, alto=18; muestra el literal 'Generado por:'.
Línea 193	<code><textField isBlankWhenNull="true"><reportElement x="110" y="38" width="160" height="18" uuid="40000000-0000-4000-8000-000000000003"/><textFieldExpression><![CDATA[\${P{usuario}}]>...</textFieldExpression></code> → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en x=110, y=38, ancho=160, alto=18; evalúa la expresión usando parámetro usuario.
Línea 194	<code><staticText><reportElement x="300" y="38" width="80" height="18" uuid="40000000-0000-4000-8000-000000000004"/><text><![CDATA[Fecha:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=38, ancho=80, alto=18; muestra el literal 'Fecha:'.
Línea 195	<code><textField pattern="dd/MM/yyyy"><reportElement x="380" y="38" width="175" height="18" uuid="40000000-0000-4000-8000-000000000005"/><textFieldExpression><![CDATA[\${P{fechaInforme}}]>...</textFieldExpression></code> → Composición de la línea: crea un <code>textField</code> dinámico con patrón dd/MM/yyyy; lo posiciona en x=380, y=38, ancho=175, alto=18; evalúa la expresión usando parámetro fechaInforme.
Línea 196	<code><staticText><reportElement x="0" y="62" width="100" height="18" uuid="40000000-0000-4000-8000-000000000006"/><text><![CDATA[Departamento:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=62, ancho=100, alto=18; muestra el literal 'Departamento:'.
Línea 197	<code><textField isBlankWhenNull="true"><reportElement x="100" y="62" width="170" height="18" uuid="40000000-0000-4000-8000-000000000007"/><textFieldExpression><![CDATA[\${P{departament...}}]>...</textFieldExpression></code> → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en x=100, y=62, ancho=170, alto=18; evalúa la expresión usando parámetro departamento.
Línea 198	<code><staticText><reportElement x="300" y="62" width="70" height="18" uuid="40000000-0000-4000-8000-000000000008"/><text><![CDATA[Periodo:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=62, ancho=70, alto=18; muestra el literal 'Periodo:'.
Línea 199	<code><textField isBlankWhenNull="true"><reportElement x="370" y="62" width="185" height="18" uuid="40000000-0000-4000-8000-000000000009"/><textFieldExpression><![CDATA[\${P{periodo}}]>...</textFieldExpression></code> → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en x=370, y=62, ancho=185, alto=18; evalúa la expresión usando parámetro periodo.
Línea 200	<code><staticText><reportElement x="0" y="86" width="100" height="18" uuid="40000000-0000-4000-8000-000000000010"/><text><![CDATA[Búsqueda:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=86, ancho=100, alto=18; muestra el literal 'Búsqueda:'.
Línea 201	<code><textField isBlankWhenNull="true"><reportElement x="100" y="86" width="170" height="18" uuid="40000000-0000-4000-8000-000000000011"/><textFieldExpression><![CDATA[\${P{textoBusque...}}]>...</textFieldExpression></code> → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en x=100, y=86, ancho=170, alto=18; evalúa la expresión usando parámetro textoBusqueda, parámetro textoBusqueda, parámetro textoBusqueda.
Línea 202	<code><staticText><reportElement x="300" y="86" width="90" height="18" uuid="40000000-0000-4000-8000-000000000012"/><text><![CDATA[Categorías:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=86, ancho=90, alto=18; muestra el literal 'Categorías:'.
Línea 203	<code><textField textAdjust="StretchHeight"><reportElement x="390" y="86" width="165" height="34" uuid="40000000-0000-4000-8000-000000000013"/><textFieldExpression><![CDATA[String.val...</textFieldExpression></code> → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en x=390, y=86, ancho=165, alto=34; evalúa la expresión usando parámetro categoriasLista.
Línea 204	<code></band></code> → Cierra <code>band</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 205	<code></title></code> → Cierra <code>title</code> y vuelve al elemento padre de la jerarquía JRXML.

Línea 206	<code><columnHeader></code> → Abre Column Header, repetida al comienzo de cada columna/página según la paginación.
Línea 207	<code><band height="62"></code> → Define una banda de 62 puntos, reservando ese espacio para sus elementos.
Línea 208	<code><staticText><reportElement x="0" y="2" width="215" height="18" uuid="41000000-0000-4000-8000-000000000001" style="Cabecera"/><text><![CDATA[Título]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=2, ancho=215, alto=18 y aplica el estilo Cabecera; muestra el literal 'Título'.
Línea 209	<code><staticText><reportElement x="215" y="2" width="55" height="18" uuid="41000000-0000-4000-8000-000000000002" style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[...]</code> → Composición de la línea: crea un texto literal; lo posiciona en x=215, y=2, ancho=55, alto=18 y aplica el estilo Cabecera; configura la alineación del texto (horizontal Right); muestra el literal 'Unid.'.
Línea 210	<code><staticText><reportElement x="280" y="2" width="90" height="18" uuid="41000000-0000-4000-8000-000000000003" style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[...]</code> → Composición de la línea: crea un texto literal; lo posiciona en x=280, y=2, ancho=90, alto=18 y aplica el estilo Cabecera; configura la alineación del texto (horizontal Right); muestra el literal 'Importe'.
Línea 211	<code><staticText><reportElement x="380" y="2" width="65" height="18" uuid="41000000-0000-4000-8000-000000000004" style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[...]</code> → Composición de la línea: crea un texto literal; lo posiciona en x=380, y=2, ancho=65, alto=18 y aplica el estilo Cabecera; configura la alineación del texto (horizontal Right); muestra el literal 'Precio med.'.
Línea 212	<code><staticText><reportElement x="455" y="2" width="100" height="18" uuid="41000000-0000-4000-8000-000000000005" style="Cabecera"/><text><![CDATA[Categoría]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=455, y=2, ancho=100, alto=18 y aplica el estilo Cabecera; muestra el literal 'Categoría'.
Línea 213	<code><staticText><reportElement x="0" y="24" width="130" height="18" uuid="41000000-0000-4000-8000-000000000006" style="Cabecera"/><text><![CDATA[Primera venta]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=24, ancho=130, alto=18 y aplica el estilo Cabecera; muestra el literal 'Primera venta'.
Línea 214	<code><staticText><reportElement x="130" y="24" width="130" height="18" uuid="41000000-0000-4000-8000-000000000007" style="Cabecera"/><text><![CDATA[Última venta]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=130, y=24, ancho=130, alto=18 y aplica el estilo Cabecera; muestra el literal 'Última venta'.
Línea 215	<code><staticText><reportElement x="260" y="24" width="160" height="18" uuid="41000000-0000-4000-8000-000000000008" style="Cabecera"/><text><![CDATA[Periodo de ventas]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=260, y=24, ancho=160, alto=18 y aplica el estilo Cabecera; muestra el literal 'Periodo de ventas'.
Línea 216	<code><staticText></code> → Abre un elemento de texto literal; su contenido no depende de fields, parámetros ni variables.
Línea 217	<code><reportElement x="420" y="24" width="135" height="18" uuid="41000000-0000-4000-8000-000000000009" style="Cabecera"></code> → Posiciona el elemento en x=420, y=24, con ancho 135 y alto 18, aplicando el estilo Cabecera.
Línea 218	<code><printWhenExpression><![CDATA[Boolean.TRUE.equals(\$P{mostrarDetalle})]]></printWhenExpression></code> → Evalúa una condición booleana para decidir si la banda o elemento se imprime usando parámetro mostrarDetalle.
Línea 219	<code></reportElement></code> → Cierra reportElement y vuelve al elemento padre de la jerarquía JRXML.
Línea 220	<code><textElement textAlignment="Right"/></code> → Configura el formato interno del texto: alineación horizontal Right.
Línea 221	<code><text><![CDATA[Importe con IVA]]></text></code> → Define el texto literal visible: Importe con IVA.
Línea 222	<code></staticText></code> → Cierra staticText y vuelve al elemento padre de la jerarquía JRXML.
Línea 223	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.

Línea 224	<code></columnHeader></code> → Cierra <code>columnHeader</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 225	<code><detail></code> → Abre <code>Detail</code> , la sección que se repite para cada registro del dataset principal.
Línea 226	<code><band height="82" splitType="Stretch"></code> → Define una banda de 82 puntos con <code>splitType Stretch</code> , reservando ese espacio para sus elementos.
Línea 227	<code><textField textAdjust="StretchHeight"><reportElement x="0" y="0" width="215" height="20" uuid="42000000-0000-4000-8000-000000000001" style="Dato"/><textFieldExpression><![CDATA[... → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=0, y=0</code>, ancho=215, alto=20 y aplica el estilo <code>Dato</code>; evalúa la expresión usando <code>field titulo</code>.</code>
Línea 228	<code><textField isBlankWhenNull="true"><reportElement x="215" y="0" width="55" height="20" uuid="42000000-0000-4000-8000-000000000002" style="UnidadesCondicional"/><textElement textA... → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=215, y=0</code>, ancho=55, alto=20 y aplica el estilo <code>UnidadesCondicional</code>; configura la alineación del texto (<code>horizontal Right</code>); evalúa la expresión usando <code>field unidades_vendidas</code>.</code>
Línea 229	<code><textField pattern="#.##0.00 €" isBlankWhenNull="true"><reportElement x="280" y="0" width="90" height="20" uuid="42000000-0000-4000-8000-000000000003" style="Dato"/><textElement... → Composición de la línea: crea un <code>textField</code> dinámico con patrón <code>#.##0.00 €</code>; lo posiciona en <code>x=280, y=0</code>, ancho=90, alto=20 y aplica el estilo <code>Dato</code>; configura la alineación del texto (<code>horizontal Right</code>); evalúa la expresión usando <code>field importe_total</code>.</code>
Línea 230	<code><textField isBlankWhenNull="true"><reportElement x="380" y="0" width="65" height="20" uuid="42000000-0000-4000-8000-000000000004" style="Dato"/><textElement textAlignment="Right"... → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=380, y=0</code>, ancho=65, alto=20 y aplica el estilo <code>Dato</code>; configura la alineación del texto (<code>horizontal Right</code>); evalúa la expresión usando <code>field precio_medio</code>, <code>field precio_medio</code>.</code>
Línea 231	<code><textField isBlankWhenNull="true"><reportElement x="455" y="0" width="100" height="20" uuid="42000000-0000-4000-8000-000000000005" style="Dato"/><textFieldExpression><![CDATA[\$F... → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=455, y=0</code>, ancho=100, alto=20 y aplica el estilo <code>Dato</code>; evalúa la expresión usando <code>field categoria</code>.</code>
Línea 232	<code><textField isBlankWhenNull="true"><reportElement x="0" y="24" width="130" height="18" uuid="42000000-0000-4000-8000-000000000006" style="Dato"/><textFieldExpression><![CDATA[\$F{... → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=0, y=24</code>, ancho=130, alto=18 y aplica el estilo <code>Dato</code>; evalúa la expresión usando <code>field primera_venta</code>.</code>
Línea 233	<code><textField isBlankWhenNull="true"><reportElement x="130" y="24" width="130" height="18" uuid="42000000-0000-4000-8000-000000000007" style="Dato"/><textFieldExpression><![CDATA[\$... → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=130, y=24</code>, ancho=130, alto=18 y aplica el estilo <code>Dato</code>; evalúa la expresión usando <code>field ultima_venta</code>.</code>
Línea 234	<code><textField><reportElement x="260" y="24" width="160" height="18" uuid="42000000-0000-4000-8000-000000000008" style="Dato"/><textElement textAlignment="Center"/><textFieldExpress... → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=260, y=24</code>, ancho=160, alto=18 y aplica el estilo <code>Dato</code>; configura la alineación del texto (<code>horizontal Center</code>); evalúa la expresión usando <code>field primera_venta</code>, <code>field primera_venta</code>, <code>field ultima_venta</code>.</code>
Línea 235	<code><textField pattern="#.##0.00 €" isBlankWhenNull="true"></code> → Abre un <code>textField</code> dinámico con formato <code>#.##0.00 €</code> , cuyo valor se obtiene de su <code>textFieldExpression</code> .
Línea 236	<code><reportElement x="420" y="24" width="135" height="18" uuid="42000000-0000-4000-8000-000000000009" style="Dato"></code> → Posiciona el elemento en <code>x=420, y=24</code> , con ancho 135 y alto 18, aplicando el estilo <code>Dato</code> .
Línea 237	<code><printWhenExpression><![CDATA[Boolean.TRUE.equals(\$P{mostrarDetalle})]]></printWhenExpression></code> → Evalúa una condición booleana para decidir si la banda o elemento se imprime usando parámetro <code>mostrarDetalle</code> .
Línea 238	<code></reportElement></code> → Cierra <code>reportElement</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 239	<code><textElement textAlignment="Right"/></code> → Configura el formato interno del texto: alineación horizontal <code>Right</code> .

Línea 240	<code><textFieldExpression><![CDATA[\$F{importe_total} == null \$P{tipoIva} == null ? null : Double.valueOf(\$F{importe_total}.doubleValue()) * (1.0d + \$P{tipoIva}.doubleValue())]]></...></code> → Calcula el valor mostrado por el textField mediante una expresión Java que usa field importe_total, field importe_total, parámetro tipoIva, parámetro tipoIva.
Línea 241	<code></textField></code> → Cierra textField y vuelve al elemento padre de la jerarquía JRXML.
Línea 242	<code><textField><reportElement x="0" y="48" width="105" height="18" uuid="42000000-0000-4000-8000-000000000010" style="Dato"/><textFieldExpression><![CDATA[\$F{unidades_vendidas} == n...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=48, ancho=105, alto=18 y aplica el estilo Dato; evalúa la expresión usando field unidades_vendidas, field unidades_vendidas, field unidades_vendidas.
Línea 243	<code><textField><reportElement x="105" y="48" width="185" height="18" uuid="42000000-0000-4000-8000-000000000011" style="Dato"/><textFieldExpression><![CDATA[\$F{titulo} == null ? "" ...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=105, y=48, ancho=185, alto=18 y aplica el estilo Dato; evalúa la expresión usando field titulo, field titulo.
Línea 244	<code><textField><reportElement x="290" y="48" width="80" height="18" uuid="42000000-0000-4000-8000-000000000012" style="Dato"/><textElement textAlignment="Right"/><textFieldExpressio...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=290, y=48, ancho=80, alto=18 y aplica el estilo Dato; configura la alineación del texto (horizontal Right); evalúa la expresión usando field precio_medio, field precio_medio.
Línea 245	<code><textField><reportElement x="370" y="48" width="90" height="18" uuid="42000000-0000-4000-8000-000000000013" style="Dato"/><textElement textAlignment="Center"/><textFieldExpressi...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=370, y=48, ancho=90, alto=18 y aplica el estilo Dato; configura la alineación del texto (horizontal Center); evalúa la expresión usando field primera_venta, field ultima_venta, field primera_venta, field ultima_venta.
Línea 246	<code><textField><reportElement x="460" y="48" width="95" height="18" uuid="42000000-0000-4000-8000-000000000014" style="Dato"/><textElement textAlignment="Right"/><textFieldExpressio...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=460, y=48, ancho=95, alto=18 y aplica el estilo Dato; configura la alineación del texto (horizontal Right); evalúa la expresión usando field unidades_vendidas, field unidades_vendidas, parámetro umbralUnidades, parámetro umbralUnidades.
Línea 247	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 248	<code><band height="14"></code> → Define una banda de 14 puntos, reservando ese espacio para sus elementos.
Línea 249	<code><printWhenExpression><![CDATA[\$F{unidades_vendidas} != null && \$P{umbralUnidades} != null && \$F{unidades_vendidas}.intValue() >= \$P{umbralUnidades}.intValue()]]></printWhenExpresio...></code> → Evalúa una condición booleana para decidir si la banda o elemento se imprime usando field unidades_vendidas, field unidades_vendidas, parámetro umbralUnidades, parámetro umbralUnidades.
Línea 250	<code><textField><reportElement x="0" y="0" width="555" height="12" uuid="42000000-0000-4000-8000-000000000015"/><textElement textAlignment="Center"></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=0, ancho=555, alto=12; configura la alineación del texto (horizontal Center); configura la fuente (familia DejaVu Sans, tamaño 8, negrita); evalúa la expresión usando field titulo, parámetro umbralUnidades.
Línea 251	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 252	<code><band height="88" splitType="Stretch"></code> → Define una banda de 88 puntos con splitType Stretch, reservando ese espacio para sus elementos.
Línea 253	<code><printWhenExpression><![CDATA[\$F{unidades_vendidas} != null]]></printWhenExpression></code> → Evalúa una condición booleana para decidir si la banda o elemento se imprime usando field unidades_vendidas.
Línea 254	<code><staticText></code> → Abre un elemento de texto literal; su contenido no depende de fields, parámetros ni variables.
Línea 255	<code><reportElement x="0" y="2" width="555" height="16" style="Cabecera"/></code> → Posiciona el elemento en x=0, y=2, con ancho 555 y alto 16, aplicando el estilo Cabecera.
Línea 256	<code><text><![CDATA[Detalle de ventas]]></text></code> → Define el texto literal visible: Detalle de ventas.

Línea 257	<code></staticText></code> → Cierra <code>staticText</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 258	<code><subreport></code> → Abre el componente subreport que ejecuta un informe hijo dentro de la banda del maestro.
Línea 259	<code><reportElement x="0" y="22" width="555" height="60" isRemoveLineWhenBlank="true"/></code> → Posiciona el elemento en x=0, y=22, con ancho 555 y alto 60, y elimina su línea cuando queda vacío.
Línea 260	<code><subreportParameter name="tituloLibro"></code> → Declara el parámetro del subreporte <code>tituloLibro</code> que recibirá un valor del informe maestro.
Línea 261	<code><subreportParameterExpression><![CDATA[{\$F{titulo}}]></subreportParameterExpression></code> → Calcula el valor enviado al parámetro del subreporte a partir de field <code>titulo</code> .
Línea 262	<code></subreportParameter></code> → Cierra <code>subreportParameter</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 263	<code><connectionExpression><![CDATA[{\$P{REPORT_CONNECTION}}]></connectionExpression></code> → Entrega al subreporte/subdataset la misma <code>REPORT_CONNECTION</code> usada por el informe maestro, evitando abrir otra conexión.
Línea 264	<code><subreportExpression><![CDATA["reports/subinforme_ventas_detalle.jasper"]]></subreportExpression></code> → Devuelve la ruta del archivo <code>subinforme_ventas_detalle.jasper</code> que JasperReports cargará como informe hijo.
Línea 265	<code></subreport></code> → Finaliza el componente de subreporte.
Línea 266	<code></band></code> → Cierra <code>band</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 267	<code><band height="104" splitType="Stretch"></code> → Define una banda de 104 puntos con <code>splitType Stretch</code> , reservando ese espacio para sus elementos.
Línea 268	<code><printWhenExpression><![CDATA[{\$F{unidades_vendidas} != null}]></printWhenExpression></code> → Evalúa una condición booleana para decidir si la banda o elemento se imprime usando field <code>unidades_vendidas</code> .
Línea 269	<code><staticText></code> → Abre un elemento de texto literal; su contenido no depende de fields, parámetros ni variables.
Línea 270	<code><reportElement x="0" y="2" width="555" height="16" style="Cabecera"/></code> → Posiciona el elemento en x=0, y=2, con ancho 555 y alto 16, aplicando el estilo <code>Cabecera</code> .
Línea 271	<code><text><![CDATA[Top 3 ventas por cantidad]]></text></code> → Define el texto literal visible: <code>Top 3 ventas por cantidad</code> .
Línea 272	<code></staticText></code> → Cierra <code>staticText</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 273	<code><componentElement></code> → Abre un contenedor de componentes extendidos; en este checkpoint contiene la tabla <code>c:table</code> .
Línea 274	<code><reportElement x="0" y="22" width="555" height="76"/></code> → Posiciona el elemento en x=0, y=22, con ancho 555 y alto 76.
Línea 275	<code><c:table xmlns:c="http://jasperreports.sourceforge.net/jasperreports/components"></code> → Abre la tabla del namespace de componentes JasperReports; sus columnas usan un <code>datasetRun</code> independiente.
Línea 276	<code>xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports/components http://jasperreports.sourceforge.net/xsd/components.xsd"></code> → Relaciona el namespace de JasperReports con su XSD para que Studio y el compilador validen la estructura.
Línea 277	<code><datasetRun subDataset="DatasetTopVentas"></code> → Asocia el componente con el subdataset <code>DatasetTopVentas</code> para ejecutar su consulta.
Línea 278	<code><datasetParameter name="tituloLibro"></code> → Declara el parámetro <code>tituloLibro</code> que se enviará al subdataset de la tabla.
Línea 279	<code><datasetParameterExpression><![CDATA[{\$F{titulo}}]></datasetParameterExpression></code> → Calcula el valor enviado al parámetro del subdataset desde field <code>titulo</code> .
Línea 280	<code></datasetParameter></code> → Cierra <code>datasetParameter</code> y vuelve al elemento padre de la jerarquía JRXML.

Línea 281	<code><connectionExpression><![CDATA[\$P{REPORT_CONNECTION}]]></connectionExpression></code> → Entrega al subreporte/subdataset la misma REPORT_CONNECTION usada por el informe maestro, evitando abrir otra conexión.
Línea 282	<code></datasetRun></code> → Cierra datasetRun y vuelve al elemento padre de la jerarquía JRXML.
Línea 283	<code><c:column width="255"></code> → Declara una columna de tabla de 255 puntos de ancho.
Línea 284	<code><c:columnHeader style="M5TablaCabecera" height="20"><staticText><reportElement x="0" y="0" width="255" height="20"/><text><![CDATA[Fecha]]></text></staticText></c:columnHeader></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=0, ancho=255, alto=20 y aplica el estilo M5TablaCabecera; muestra el literal 'Fecha'; define una celda de cabecera de la tabla.
Línea 285	<code><c:detailCell style="M5TablaDetalle" height="18"><textField><reportElement x="0" y="0" width="255" height="18"/><textFieldExpression><![CDATA[\${fecha_venta}]]></textFieldExpres...</code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=0, ancho=255, alto=18 y aplica el estilo M5TablaDetalle; evalúa la expresión usando field fecha_venta; define una celda repetida de detalle de la tabla.
Línea 286	<code></c:column></code> → Cierra c:column y vuelve al elemento padre de la jerarquía JRXML.
Línea 287	<code><c:column width="100"></code> → Declara una columna de tabla de 100 puntos de ancho.
Línea 288	<code><c:columnHeader style="M5TablaCabecera" height="20"><staticText><reportElement x="0" y="0" width="100" height="20"/><textElement textAlignment="Right"/><text><![CDATA[Cantidad]]...></text></staticText></c:columnHeader></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=0, ancho=100, alto=20 y aplica el estilo M5TablaCabecera; configura la alineación del texto (horizontal Right); muestra el literal 'Cantidad'; define una celda de cabecera de la tabla.
Línea 289	<code><c:detailCell style="M5TablaDetalle" height="18"><textField><reportElement x="0" y="0" width="100" height="18"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[...]]></textFieldExpres...</code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=0, ancho=100, alto=18 y aplica el estilo M5TablaDetalle; configura la alineación del texto (horizontal Right); evalúa la expresión usando field cantidad; define una celda repetida de detalle de la tabla.
Línea 290	<code></c:column></code> → Cierra c:column y vuelve al elemento padre de la jerarquía JRXML.
Línea 291	<code><c:column width="200"></code> → Declara una columna de tabla de 200 puntos de ancho.
Línea 292	<code><c:columnHeader style="M5TablaCabecera" height="20"><staticText><reportElement x="0" y="0" width="200" height="20"/><textElement textAlignment="Right"/><text><![CDATA[Precio uni...]]></text></staticText></c:columnHeader></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=0, ancho=200, alto=20 y aplica el estilo M5TablaCabecera; configura la alineación del texto (horizontal Right); muestra el literal 'Precio unitario'; define una celda de cabecera de la tabla.
Línea 293	<code><c:detailCell style="M5TablaDetalle" height="18"><textField pattern="#.##0.00 €"><reportElement x="0" y="0" width="200" height="18"/><textElement textAlignment="Right"/><textFie...</code> → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=0, y=0, ancho=200, alto=18 y aplica el estilo M5TablaDetalle; configura la alineación del texto (horizontal Right); evalúa la expresión usando field precio_unitario; define una celda repetida de detalle de la tabla.
Línea 294	<code></c:column></code> → Cierra c:column y vuelve al elemento padre de la jerarquía JRXML.
Línea 295	<code></c:table></code> → Finaliza la tabla integrada.
Línea 296	<code></componentElement></code> → Cierra componentElement y vuelve al elemento padre de la jerarquía JRXML.
Línea 297	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 298	<code></detail></code> → Finaliza la sección Detail del informe.
Línea 299	<code><pageFooter></code> → Abre Page Footer, emitido al pie de cada página.
Línea 300	<code><band height="62"></code> → Define una banda de 62 puntos, reservando ese espacio para sus elementos.
Línea 301	<code><staticText><reportElement x="0" y="4" width="120" height="15" uuid="43000000-0000-4000-8000-000000000001"/><text><![CDATA[Total de títulos:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=4, ancho=120, alto=15; muestra el literal 'Total de títulos:'.

Línea 302	<pre><textField evaluationTime="Report"><reportElement x="120" y="4" width="60" height="15" uuid="43000000-0000-4000-8000-000000000002"/><textFieldExpression><![CDATA[{\$V{REPORT_COUNT... → Composición de la línea: crea un textField dinámico; lo posiciona en x=120, y=4, ancho=60, alto=15; evalúa la expresión usando variable REPORT_COUNT.</pre>
Línea 303	<pre><textField><reportElement x="190" y="28" width="180" height="15" uuid="43000000-0000-4000-8000-000000000003"/><textElement textAlignment="Right"/ ><textFieldExpression><![CDATA["... → Composición de la línea: crea un textField dinámico; lo posiciona en x=190, y=28, ancho=180, alto=15; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable PAGE_NUMBER.</pre>
Línea 304	<pre><textField evaluationTime="Report"><reportElement x="375" y="28" width="35" height="15" uuid="43000000-0000-4000-8000-000000000004"/><textFieldExpression><![CDATA[{\$V{PAGE_NUMBER... → Composición de la línea: crea un textField dinámico; lo posiciona en x=375, y=28, ancho=35, alto=15; evalúa la expresión usando variable PAGE_NUMBER.</pre>
Línea 305	<pre><staticText><reportElement x="300" y="4" width="120" height="15" uuid="43000000-0000-4000-8000-000000000005"/><text><![CDATA[Subtotal página:]]></text></ staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=4, ancho=120, alto=15; muestra el literal 'Subtotal página:'.</pre>
Línea 306	<pre><textField pattern="#.##0.00 €"><reportElement x="420" y="4" width="135" height="15" uuid="43000000-0000-4000-8000-000000000006"/><textElement textAlignment="Right"/ ><textFieldE... → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=420, y=4, ancho=135, alto=15; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable TotalPagina.</pre>
Línea 307	<pre></band></pre> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 308	<pre></pageFooter></pre> → Cierra pageFooter y vuelve al elemento padre de la jerarquía JRXML.
Línea 309	<pre><summary></pre> → Abre Summary, emitido una sola vez después del último registro.
Línea 310	<pre><band height="700"></pre> → Define una banda de 700 puntos, reservando ese espacio para sus elementos.
Línea 311	<pre><staticText><reportElement x="0" y="5" width="205" height="18" uuid="44000000-0000-4000-8000-000000000001"/><text><![CDATA[Total de unidades vendidas:]]></ text></staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=5, ancho=205, alto=18; muestra el literal 'Total de unidades vendidas:'.</pre>
Línea 312	<pre><textField><reportElement x="205" y="5" width="80" height="18" uuid="44000000-0000-4000-8000-000000000002"/><textElement textAlignment="Right"/ ><textFieldExpression><![CDATA[{\$V{... → Composición de la línea: crea un textField dinámico; lo posiciona en x=205, y=5, ancho=80, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable TotalUnidades.</pre>
Línea 313	<pre><staticText><reportElement x="300" y="5" width="120" height="18" uuid="44000000-0000-4000-8000-000000000003"/><text><![CDATA[Importe total:]]></text></ staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=5, ancho=120, alto=18; muestra el literal 'Importe total:'.</pre>
Línea 314	<pre><textField pattern="#.##0.00 €"><reportElement x="420" y="5" width="135" height="18" uuid="44000000-0000-4000-8000-000000000004"/><textElement textAlignment="Right"/ ><textFieldE... → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=420, y=5, ancho=135, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable TotalImporte.</pre>
Línea 315	<pre><staticText><reportElement x="0" y="30" width="205" height="18" uuid="44000000-0000-4000-8000-000000000005"/><text><![CDATA[Precio medio agregado:]]></ text></staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=30, ancho=205, alto=18; muestra el literal 'Precio medio agregado:'.</pre>
Línea 316	<pre><textField pattern="#.##0.00 €"><reportElement x="205" y="30" width="80" height="18" uuid="44000000-0000-4000-8000-000000000006"/><textElement textAlignment="Right"/ ><textFieldE... → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=205, y=30, ancho=80, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable PrecioMedio.</pre>

Línea 317	<code><staticText><reportElement x="300" y="30" width="120" height="18" uuid="44000000-0000-4000-8000-000000000007"/><text><![CDATA[Precio máximo:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=30, ancho=120, alto=18; muestra el literal 'Precio máximo:'.
Línea 318	<code><textField pattern="#.##0.00 €"><reportElement x="420" y="30" width="135" height="18" uuid="44000000-0000-4000-8000-000000000008"/><textElement textAlignment="Right"/><textField...></code> → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=420, y=30, ancho=135, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable PrecioMaximo.
Línea 319	<code><staticText><reportElement x="0" y="55" width="205" height="18" uuid="44000000-0000-4000-8000-000000000009"/><text><![CDATA[Número de libros:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=55, ancho=205, alto=18; muestra el literal 'Número de libros:'.
Línea 320	<code><textField><reportElement x="205" y="55" width="80" height="18" uuid="44000000-0000-4000-8000-000000000010"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[\$V...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=205, y=55, ancho=80, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable NumeroLibros.
Línea 321	<code><staticText><reportElement x="300" y="55" width="120" height="18" uuid="44000000-0000-4000-8000-000000000011"/><text><![CDATA[Importe con IVA:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=55, ancho=120, alto=18; muestra el literal 'Importe con IVA:'.
Línea 322	<code><textField pattern="#.##0.00 €"><reportElement x="420" y="55" width="135" height="18" uuid="44000000-0000-4000-8000-000000000012"/><textElement textAlignment="Right"/><textField...></code> → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=420, y=55, ancho=135, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable ImporteConIva.
Línea 323	<code><textField><reportElement x="0" y="80" width="555" height="18" uuid="44000000-0000-4000-8000-000000000013"/><textElement textAlignment="Center"/><textFieldExpression><![CDATA[St...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=80, ancho=555, alto=18; configura la alineación del texto (horizontal Center); evalúa la expresión usando variable NumeroLibros, variable TotalUnidades, variable TotalImporte.
Línea 324	<code><textField><reportElement x="0" y="103" width="350" height="18" uuid="44000000-0000-4000-8000-000000000014"/><textElement textAlignment="Center"></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=103, ancho=350, alto=18; configura la alineación del texto (horizontal Center); configura la fuente (familia DejaVu Sans, tamaño 10, negrita); evalúa la expresión usando parámetro umbralUnidades, parámetro umbralUnidades, variable TotalUnidades, variable TotalUnidades.
Línea 325	<code><textField><reportElement x="360" y="103" width="195" height="18" uuid="44000000-0000-4000-8000-000000000015"/><textFieldExpression><![CDATA["Resultados encontrados: " + \$V{REPO...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=360, y=103, ancho=195, alto=18; evalúa la expresión usando variable REPORT_COUNT.
Línea 326	<code><staticText><reportElement x="0" y="140" width="555" height="20" style="Cabecera"/><text><![CDATA[Ventas por categoría – importe]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=140, ancho=555, alto=20 y aplica el estilo Cabecera; muestra el literal 'Ventas por categoría — importe'.
Línea 327	<code><barChart></code> → Abre el gráfico de barras nativo de JasperReports que se integrará en el informe maestro.
Línea 328	<code><chart></code> → Abre la configuración común del gráfico: geometría, título, subtítulo y leyenda.
Línea 329	<code><reportElement x="0" y="165" width="555" height="250"/></code> → Posiciona el elemento en x=0, y=165, con ancho 555 y alto 250.
Línea 330	<code><chartTitle><titleExpression><![CDATA["Ventas por categoría"]]></titleExpression></chartTitle></code> → Composición de la línea: encadena además <chartTitle>, <titleExpression> dentro de la misma jerarquía.
Línea 331	<code><chartSubtitle/></code> → Declara el subtítulo del gráfico; en este checkpoint queda vacío.

Línea 332	<code><chartLegend position="Bottom"/></code> → Configura la leyenda del gráfico en la posición Bottom.
Línea 333	<code></chart></code> → Cierra chart y vuelve al elemento padre de la jerarquía JRXML.
Línea 334	<code><categoryDataset></code> → Abre el dataset categórico que alimenta al gráfico con serie, categoría y valor.
Línea 335	<code><dataset></code> → Abre el contenedor de ejecución de datos del componente actual.
Línea 336	<code><datasetRun subDataset="DatasetVentasPorCategoría"></code> → Asocia el componente con el subdataset DatasetVentasPorCategoría para ejecutar su consulta.
Línea 337	<code><connectionExpression><![CDATA[\$P{REPORT_CONNECTION}]]></connectionExpression></code> → Entrega al subreporte/subdataset la misma REPORT_CONNECTION usada por el informe maestro, evitando abrir otra conexión.
Línea 338	<code></datasetRun></code> → Cierra datasetRun y vuelve al elemento padre de la jerarquía JRXML.
Línea 339	<code></dataset></code> → Cierra dataset y vuelve al elemento padre de la jerarquía JRXML.
Línea 340	<code><categorySeries></code> → Abre una serie del dataset categórico; cada fila del subdataset aportará categoría y valor.
Línea 341	<code><seriesExpression><![CDATA["Importe"]]></seriesExpression></code> → Define el nombre lógico de la serie que aparecerá en la leyenda.
Línea 342	<code><categoryExpression><![CDATA[\$F{categoría_grafico}]]></categoryExpression></code> → Define la categoría del eje X a partir de field categoría_grafico.
Línea 343	<code><valueExpression><![CDATA[\$F{importe_categoria}]]></valueExpression></code> → Define el valor numérico representado por cada barra a partir de field importe_categoria.
Línea 344	<code></categorySeries></code> → Cierra categorySeries y vuelve al elemento padre de la jerarquía JRXML.
Línea 345	<code></categoryDataset></code> → Cierra categoryDataset y vuelve al elemento padre de la jerarquía JRXML.
Línea 346	<code><barPlot></code> → Abre el plot específico del gráfico de barras, donde se configuran etiquetas y ejes.
Línea 347	<code><plot/></code> → Declara el bloque base del plot; mantiene la configuración visual por defecto del checkpoint.
Línea 348	<code><itemLabel/></code> → Habilita el bloque de configuración de etiquetas de los ítems/barras.
Línea 349	<code><categoryAxisFormat><axisFormat/></categoryAxisFormat></code> → Composición de la línea: encadena además <code><categoryAxisFormat></code> , <code><axisFormat></code> dentro de la misma jerarquía.
Línea 350	<code><valueAxisFormat><axisFormat/></valueAxisFormat></code> → Composición de la línea: encadena además <code><valueAxisFormat></code> , <code><axisFormat></code> dentro de la misma jerarquía.
Línea 351	<code></barPlot></code> → Cierra barPlot y vuelve al elemento padre de la jerarquía JRXML.
Línea 352	<code></barChart></code> → Finaliza el gráfico de barras.
Línea 353	<code><staticText><reportElement x="0" y="430" width="555" height="20" style="Cabecera"/><text><![CDATA[Ventas por categoría y año]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=430, ancho=555, alto=20 y aplica el estilo Cabecera; muestra el literal 'Ventas por categoría y año'.
Línea 354	<code><crosstab></code> → Abre la tabla cruzada nativa que genera dinámicamente la matriz de filas, columnas, medidas y totales.
Línea 355	<code><reportElement x="0" y="455" width="555" height="225"/></code> → Posiciona el elemento en x=0, y=455, con ancho 555 y alto 225.
Línea 356	<code><crosstabDataset></code> → Abre la fuente de datos específica del crosstab.
Línea 357	<code><dataset></code> → Abre el contenedor de ejecución de datos del componente actual.
Línea 358	<code><datasetRun subDataset="DatasetCrosstabVentas"></code> → Asocia el componente con el subdataset DatasetCrosstabVentas para ejecutar su consulta.

Línea 359	<code><connectionExpression><![CDATA[\$P{REPORT_CONNECTION}]]></connectionExpression></code> → Entrega al subreporte/subdataset la misma REPORT_CONNECTION usada por el informe maestro, evitando abrir otra conexión.
Línea 360	<code></datasetRun></code> → Cierra datasetRun y vuelve al elemento padre de la jerarquía JRXML.
Línea 361	<code></dataset></code> → Cierra dataset y vuelve al elemento padre de la jerarquía JRXML.
Línea 362	<code></crosstabDataset></code> → Cierra crosstabDataset y vuelve al elemento padre de la jerarquía JRXML.
Línea 363	<code><rowGroup name="CategoriaCross" width="150" totalPosition="End"></code> → Declara el grupo de filas CategoriaCross, ancho 150 y total en End.
Línea 364	<code><bucket class="java.lang.String"><bucketExpression><![CDATA[\$F{categoria_cross}]]></bucketExpression></bucket></code> → Composición de la línea: define la clave de agrupación del bucket desde field categoria_cross; encadena además <bucket> dentro de la misma jerarquía.
Línea 365	<code><crosstabRowHeader><cellContents style="M5CrosstabCabecera"><textField><reportElement x="0" y="0" width="150" height="34"><textElement verticalAlignment="Middle"><textFieldExp...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=0, ancho=150, alto=34 y aplica el estilo M5CrosstabCabecera; configura la alineación del texto (vertical Middle); evalúa la expresión usando variable CategoriaCross; abre el contenido de celda con estilo M5CrosstabCabecera; define la cabecera de fila del crosstab.
Línea 366	<code><crosstabTotalRowHeader><cellContents style="M5CrosstabTotal"><staticText><reportElement x="0" y="0" width="150" height="34"><textElement verticalAlignment="Middle"><text><![CDATA[...></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=0, ancho=150, alto=34 y aplica el estilo M5CrosstabTotal; configura la alineación del texto (vertical Middle); muestra el literal 'TOTAL'; abre el contenido de celda con estilo M5CrosstabTotal; define la cabecera de total de fila.
Línea 367	<code></rowGroup></code> → Cierra rowGroup y vuelve al elemento padre de la jerarquía JRXML.
Línea 368	<code><columnGroup name="AnioCross" height="28" totalPosition="End"></code> → Declara el grupo de columnas AnioCross, altura 28 y total en End.
Línea 369	<code><bucket class="java.lang.String"><bucketExpression><![CDATA[\$F{anio_cross}]]></bucketExpression></bucket></code> → Composición de la línea: define la clave de agrupación del bucket desde field anio_cross; encadena además <bucket> dentro de la misma jerarquía.
Línea 370	<code><crosstabColumnHeader><cellContents style="M5CrosstabCabecera"><textField><reportElement x="0" y="0" width="100" height="28"><textElement textAlignment="Center" verticalAlignme...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=0, ancho=100, alto=28 y aplica el estilo M5CrosstabCabecera; configura la alineación del texto (horizontal Center, vertical Middle); evalúa la expresión usando variable AnioCross; abre el contenido de celda con estilo M5CrosstabCabecera; define la cabecera de columna del crosstab.
Línea 371	<code><crosstabTotalColumnHeader><cellContents style="M5CrosstabTotal"><staticText><reportElement x="0" y="0" width="100" height="28"><textElement textAlignment="Center" verticalAlign...></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=0, ancho=100, alto=28 y aplica el estilo M5CrosstabTotal; configura la alineación del texto (horizontal Center, vertical Middle); muestra el literal 'TOTAL'; abre el contenido de celda con estilo M5CrosstabTotal; define la cabecera de total de columna.
Línea 372	<code></columnGroup></code> → Cierra columnGroup y vuelve al elemento padre de la jerarquía JRXML.
Línea 373	<code><measure name="ImporteCross" class="java.lang.Double" calculation="Sum"><measureExpression><![CDATA[\$F{importe_cross}]]></measureExpression></measure></code> → Composición de la línea: aporta a la medida el valor de field importe_cross; encadena además <measure> dentro de la misma jerarquía.
Línea 374	<code><measure name="VentasCross" class="java.lang.Integer" calculation="Sum"><measureExpression><![CDATA[\$F{ventas_cross}]]></measureExpression></measure></code> → Composición de la línea: aporta a la medida el valor de field ventas_cross; encadena además <measure> dentro de la misma jerarquía.
Línea 375	<code><crosstabCell width="100" height="34"><cellContents style="M5CrosstabDetalle"><textField pattern="#.##0.00 €"><reportElement x="0" y="0" width="100" height="18"><textElement te...></code> → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=0, y=0, ancho=100, alto=34 y aplica el estilo M5CrosstabDetalle; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable ImporteCross, variable VentasCross; abre el contenido de celda con estilo M5CrosstabDetalle; define la celda de detalle del crosstab.

Línea 376	<code><crosstabCell width="100" height="34" rowTotalGroup="CategoriaCross"><cellContents style="M5CrosstabTotal"><textField pattern="#,##0.00 €"><reportElement x="0" y="0" width="100" . . . → Composición de la línea: crea un textField dinámico con patrón #,##0.00 €; lo posiciona en x=0, y=0, ancho=100, alto=34 y aplica el estilo M5CrosstabTotal; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable ImporteCross, variable VentasCross; abre el contenido de celda con estilo M5CrosstabTotal; define una celda de total de fila para CategoriaCross.</code>
Línea 377	<code><crosstabCell width="100" height="34" columnTotalGroup="AnioCross"><cellContents style="M5CrosstabTotal"><textField pattern="#,##0.00 €"><reportElement x="0" y="0" width="100" h. . . → Composición de la línea: crea un textField dinámico con patrón #,##0.00 €; lo posiciona en x=0, y=0, ancho=100, alto=34 y aplica el estilo M5CrosstabTotal; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable ImporteCross, variable VentasCross; abre el contenido de celda con estilo M5CrosstabTotal; define una celda de total de columna para AnioCross.</code>
Línea 378	<code><crosstabCell width="100" height="34" rowTotalGroup="CategoriaCross" columnTotalGroup="AnioCross"><cellContents style="M5CrosstabTotal"><textField pattern="#,##0.00 €"><reportEl. . . → Composición de la línea: crea un textField dinámico con patrón #,##0.00 €; lo posiciona en x=0, y=0, ancho=100, alto=34 y aplica el estilo M5CrosstabTotal; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable ImporteCross, variable VentasCross; abre el contenido de celda con estilo M5CrosstabTotal; define la celda de total general.</code>
Línea 379	<code></crosstab> → Finaliza la tabla cruzada.</code>
Línea 380	<code></band> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.</code>
Línea 381	<code></summary> → Finaliza la sección Summary.</code>
Línea 382	<code></jasperReport> → Finaliza la definición completa del informe JasperReports.</code>

Subinforme heredado y ejecutable

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/
jasperreport.xsd"
  name="subinforme_ventas_detalle"
  language="java"
  pageWidth="555"
  pageHeight="842"
  columnWidth="555"
  leftMargin="0"
  rightMargin="0"
  topMargin="0"
  bottomMargin="0">
  <property name="com.jaspersoft.studio.data.defaultdataadapter" value="SQLiteEditorial"/>
  <style name="SubBase" isDefault="true" fontName="DejaVu Sans" fontSize="8"/>
  <style name="SubHeader" style="SubBase" mode="Opaque" backcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/>
  <parameter name="tituloLibro" class="java.lang.String"/>
  <queryString language="sql">
    <![CDATA[
      SELECT fecha_venta, cantidad, precio_unitario
      FROM ventas
      WHERE titulo_libro = $P{tituloLibro}
      ORDER BY fecha_venta
    ]]>
  </queryString>
  <field name="fecha_venta" class="java.lang.String"/>
  <field name="cantidad" class="java.lang.Integer"/>
  <field name="precio_unitario" class="java.lang.Double"/>
  <columnHeader>
    <band height="18">
      <staticText><reportElement x="0" y="0" width="245" height="18" style="SubHeader"/><text><![CDATA[Fecha]]></text></staticText>
      <staticText><reportElement x="245" y="0" width="100" height="18" style="SubHeader"/><textElement textAlignment="Right"/>
    </band>
  </columnHeader>
  <detail>
    <band height="18">
      <textField><reportElement x="0" y="0" width="245" height="18"/><textFieldExpression><![CDATA[$F{fecha_venta}]]></textFieldExpression></textField>
      <textField><reportElement x="245" y="0" width="100" height="18"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[$F{cantidad}]]></textFieldExpression></textField>
      <textField><reportElement x="345" y="0" width="210" height="18"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[$F{precio_unitario}]]></textFieldExpression></textField>
    </band>
  </detail>
</jasperReport>
```

```

</band>
</detail>
</jasperReport>

```

Explicación línea por línea

Línea 1	<code><?xml version="1.0" encoding="UTF-8"?></code> → Declara XML 1.0 y codificación UTF-8 para que nombres, textos y símbolos del informe se interpreten correctamente.
Línea 2	<code><jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports"</code> → Abre el documento raíz <code>jasperReport</code> del informe y fija el namespace principal de JasperReports.
Línea 3	<code>xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"</code> → Declara el namespace XML Schema Instance usado por <code>xsi:schemaLocation</code> para validar el documento.
Línea 4	<code>xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/jasperreport.xsd"</code> → Relaciona el namespace de JasperReports con su XSD para que Studio y el compilador validen la estructura.
Línea 5	<code>name="subinforme_ventas_detalle"</code> → Asigna al documento JasperReports el nombre interno <code>subinforme_ventas_detalle</code> .
Línea 6	<code>language="java"</code> → Configura <code>language=java</code> para evaluar expresiones con el lenguaje Java.
Línea 7	<code>pageWidth="555"</code> → Fija el ancho físico de página en 555 puntos.
Línea 8	<code>pageHeight="842"</code> → Fija la altura física de página en 842 puntos.
Línea 9	<code>columnWidth="555"</code> → Fija el ancho útil de la columna de contenido en 555 puntos.
Línea 10	<code>leftMargin="0"</code> → Fija el margen izquierdo del informe en 0 puntos.
Línea 11	<code>rightMargin="0"</code> → Fija el margen derecho del informe en 0 puntos.
Línea 12	<code>topMargin="0"</code> → Fija el margen superior del informe en 0 puntos.
Línea 13	<code>bottomMargin="0"></code> → Fija el margen inferior del informe en 0 puntos y completa la apertura del elemento raíz.
Línea 14	<code><property name="com.jaspersoft.studio.data.defaultdataadapter" value="SQLiteEditorial"/></code> → Indica a Jaspersoft Studio que use el Data Adapter <code>SQLiteEditorial</code> como conexión de diseño por defecto.
Línea 15	<code><style name="SubBase" isDefault="true" fontName="DejaVu Sans" fontSize="8"/></code> → Declara el estilo <code>SubBase</code> ; es el estilo por defecto, fuente <code>DejaVu Sans</code> , tamaño 8.
Línea 16	<code><style name="SubHeader" style="SubBase" mode="Opaque" backcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/></code> → Declara el estilo <code>SubHeader</code> ; hereda de <code>SubBase</code> , fondo <code>#EAF2F8</code> .
Línea 17	<code><parameter name="tituloLibro" class="java.lang.String"/></code> → Declara el parámetro <code>tituloLibro</code> con tipo <code>java.lang.String</code> y lo expone al diálogo de parámetros de Studio.
Línea 18	<code><queryString language="sql"></code> → Abre la consulta SQL que JasperReports ejecutará para el dataset actual.
Línea 19	<code><![CDATA[</code> → Abre CDATA para escribir SQL o una expresión Java sin que sus caracteres especiales se interpreten como XML.
Línea 20	<code>SELECT fecha_venta, cantidad, precio_unitario</code> → Cláusula SQL <code>SELECT</code> : selecciona y calcula las columnas que devolverá la consulta.
Línea 21	<code>FROM ventas</code> → Cláusula SQL <code>FROM</code> : define la tabla base de la consulta.
Línea 22	<code>WHERE titulo_libro = \${tituloLibro}</code> → Cláusula SQL <code>WHERE</code> : aplica el filtro de filas.
Línea 23	<code>ORDER BY fecha_venta</code> → Cláusula SQL <code>ORDER BY</code> : ordena el resultado que recibirá JasperReports.
Línea 24	<code>]]></code> → Cierra el bloque CDATA y devuelve el control al parser XML.
Línea 25	<code></queryString></code> → Cierra la consulta SQL del dataset actual.

Línea 26	<code><field name="fecha_venta" class="java.lang.String"/></code> → Declara el field fecha_venta con tipo Java <code>java.lang.String</code> para mapear una columna del dataset.
Línea 27	<code><field name="cantidad" class="java.lang.Integer"/></code> → Declara el field cantidad con tipo Java <code>java.lang.Integer</code> para mapear una columna del dataset.
Línea 28	<code><field name="precio_unitario" class="java.lang.Double"/></code> → Declara el field precio_unitario con tipo Java <code>java.lang.Double</code> para mapear una columna del dataset.
Línea 29	<code><columnHeader></code> → Abre Column Header, repetida al comienzo de cada columna/página según la paginación.
Línea 30	<code><band height="18"></code> → Define una banda de 18 puntos, reservando ese espacio para sus elementos.
Línea 31	<code><staticText><reportElement x="0" y="0" width="245" height="18" style="SubHeader"/><text><![CDATA[Fecha]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=0, ancho=245, alto=18 y aplica el estilo SubHeader; muestra el literal 'Fecha'.
Línea 32	<code><staticText><reportElement x="245" y="0" width="100" height="18" style="SubHeader"/><textElement textAlignment="Right"><text><![CDATA[Cantidad]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=245, y=0, ancho=100, alto=18 y aplica el estilo SubHeader; configura la alineación del texto (horizontal Right); muestra el literal 'Cantidad'.
Línea 33	<code><staticText><reportElement x="345" y="0" width="210" height="18" style="SubHeader"/><textElement textAlignment="Right"><text><![CDATA[Precio unitario]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=345, y=0, ancho=210, alto=18 y aplica el estilo SubHeader; configura la alineación del texto (horizontal Right); muestra el literal 'Precio unitario'.
Línea 34	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 35	<code></columnHeader></code> → Cierra columnHeader y vuelve al elemento padre de la jerarquía JRXML.
Línea 36	<code><detail></code> → Abre Detail, la sección que se repite para cada registro del dataset principal.
Línea 37	<code><band height="18"></code> → Define una banda de 18 puntos, reservando ese espacio para sus elementos.
Línea 38	<code><textField><reportElement x="0" y="0" width="245" height="18"/><textFieldExpression><![CDATA[\${fecha_venta}]]></textFieldExpression></textField></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=0, ancho=245, alto=18; evalúa la expresión usando field fecha_venta.
Línea 39	<code><textField><reportElement x="245" y="0" width="100" height="18"/><textElement textAlignment="Right"><textFieldExpression><![CDATA[\${cantidad}]]></textFieldExpression></textElement></textField></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=245, y=0, ancho=100, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando field cantidad.
Línea 40	<code><textField pattern="#.##0.00 €"><reportElement x="345" y="0" width="210" height="18"/><textElement textAlignment="Right"><textFieldExpression><![CDATA[\${precio_unitario}]]></textElement></textField></code> → Composición de la línea: crea un textField dinámico con patrón <code>#.##0.00 €</code> ; lo posiciona en x=345, y=0, ancho=210, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando field precio_unitario.
Línea 41	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 42	<code></detail></code> → Finaliza la sección Detail del informe.
Línea 43	<code></jasperReport></code> → Finaliza la definición completa del informe JasperReports.

Plantilla JRTX heredada y ejecutable

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<jasperTemplate xmlns="http://jasperreports.sourceforge.net/jasperreports/template"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports/template http://jasperreports.sourceforge.net/
xsd/jasptemplate.xsd">
  <style name="MSTituloPrincipal" fontName="DejaVu Sans" fontSize="18" isBold="true" forecolor="#173F6B"/>
  <style name="MSGrupoCabecera" fontName="DejaVu Sans" fontSize="10" isBold="true" forecolor="#173F6B" mode="Opaque"
backcolor="#D6EAF8"/>
  <style name="MSTablaCabecera" fontName="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque"
backcolor="#EAF2F8"/>
  <style name="MSTablaDetalle" fontName="DejaVu Sans" fontSize="9"/>
  <style name="MSCrosstabCabecera" fontName="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque">
```

```

backcolor="#EAF2F8" />
  <style name="M5CrosstabDetalle" fontName="DejaVu Sans" fontSize="9" mode="Opaque" backcolor="#FFFFFF" />
  <style name="M5CrosstabTotal" fontName="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque"
backcolor="#D6EAF8" />
</jasperTemplate>

```

Explicación línea por línea

Línea 1	<?xml version="1.0" encoding="UTF-8"?> → Declara XML 1.0 y codificación UTF-8 para que nombres, textos y símbolos del informe se interpreten correctamente.
Línea 2	<jasperTemplate xmlns="http://jasperreports.sourceforge.net/jasperreports/template" → Abre el documento raíz jasperTemplate de la plantilla JRTX que contiene estilos reutilizables.
Línea 3	xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" → Declara el namespace XML Schema Instance usado por xsi:schemaLocation para validar el documento.
Línea 4	xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports/template http://jasperreports.sourceforge.net/xsd/jasptemplate.xsd" → Relaciona el namespace de JasperReports con su XSD para que Studio y el compilador validen la estructura.
Línea 5	<style name="M5TituloPrincipal" fontName="DejaVu Sans" fontSize="18" isBold="true" forecolor="#173F6B" /> → Declara el estilo M5TituloPrincipal; fuente DejaVu Sans, tamaño 18.
Línea 6	<style name="M5GrupoCabecera" fontName="DejaVu Sans" fontSize="10" isBold="true" forecolor="#173F6B" mode="Opaque" backcolor="#D6EAF8" /> → Declara el estilo M5GrupoCabecera; fuente DejaVu Sans, tamaño 10, fondo #D6EAF8.
Línea 7	<style name="M5TablaCabecera" fontName="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque" backcolor="#EAF2F8" /> → Declara el estilo M5TablaCabecera; fuente DejaVu Sans, tamaño 9, fondo #EAF2F8.
Línea 8	<style name="M5TablaDetalle" fontName="DejaVu Sans" fontSize="9" /> → Declara el estilo M5TablaDetalle; fuente DejaVu Sans, tamaño 9.
Línea 9	<style name="M5CrosstabCabecera" fontName="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque" backcolor="#EAF2F8" /> → Declara el estilo M5CrosstabCabecera; fuente DejaVu Sans, tamaño 9, fondo #EAF2F8.
Línea 10	<style name="M5CrosstabDetalle" fontName="DejaVu Sans" fontSize="9" mode="Opaque" backcolor="#FFFFFF" /> → Declara el estilo M5CrosstabDetalle; fuente DejaVu Sans, tamaño 9, fondo #FFFFFF.
Línea 11	<style name="M5CrosstabTotal" fontName="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque" backcolor="#D6EAF8" /> → Declara el estilo M5CrosstabTotal; fuente DejaVu Sans, tamaño 9, fondo #D6EAF8.
Línea 12	</jasperTemplate> → Finaliza la plantilla externa JRTX.

Parte C — Código y configuración ejecutable explicados línea por línea

GeneradorInformeVentas.java

JAVA

```

import java.io.File;
import java.sql.Connection;
import java.sql.DriverManager;
import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;
import net.sf.jasperreports.engine.JasperCompileManager;
import net.sf.jasperreports.engine.JasperFillManager;
import net.sf.jasperreports.engine.JasperPrint;
import net.sf.jasperreports.engine.export.JRPdfExporter;
import net.sf.jasperreports.export.SimpleExporterInput;
import net.sf.jasperreports.export.SimpleOutputStreamExporterOutput;
import net.sf.jasperreports.export.SimplePdfExporterConfiguration;
import net.sf.jasperreports.engine.data.JRCsvDataSource;
import net.sf.jasperreports.engine.export.ooxml.JRXlsxExporter;
import net.sf.jasperreports.export.SimpleXlsxReportConfiguration;
import net.sf.jasperreports.export.SimpleXlsxExporterConfiguration;

public class GeneradorInformeVentas {
    public static void main(String[] args) {

```

```

try {
    String rutaJrxml = "reports/informe_ventas.jrxml";
    String rutaJasper = "reports/informe_ventas.jasper";
    String rutaPdf = "output/informe_ventas.pdf";
    String rutaPdfProtegido = "output/informe_ventas_protegido.pdf";
    String rutaXlsx = "output/informe_ventas.xlsx";
    String rutaCatalogoJrxml = "reports/informe_catalogo_csv.jrxml";
    String rutaCatalogoJasper = "reports/informe_catalogo_csv.jasper";
    String rutaXlsxCatalogo = "output/informe_catalogo.xlsx";
    String urlBD = "jdbc:sqlite:../EditorialReportsJava/data/editorial.db";
    new File("output").mkdirs();

    String rutaSubJrxml = "reports/subinforme_ventas_detalle.jrxml";
    String rutaSubJasper = "reports/subinforme_ventas_detalle.jasper";
    JasperCompileManager.compileReportToFile(rutaSubJrxml, rutaSubJasper);
    JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);
    JasperCompileManager.compileReportToFile(rutaCatalogoJrxml, rutaCatalogoJasper);

    Map<String, Object> parametros = new HashMap<String, Object>();
    parametros.put("usuario", "Ana Martínez");
    parametros.put("departamento", "Comercial");
    parametros.put("periodo", "Septiembre 2026");
    parametros.put("tipoIva", Double.valueOf(0.21d));
    parametros.put("mostrarDetalle", Boolean.TRUE);
    parametros.put("categoria", null);
    parametros.put("precioMinimo", null);
    parametros.put("precioMaximo", null);
    parametros.put("umbralUnidades", Integer.valueOf(5));
    parametros.put("textoBusqueda", null);
    parametros.put("categoriasLista", Arrays.asList("Novela", "Realismo mágico", "Cuento", "Poesía"));

    try (Connection conexion = DriverManager.getConnection(urlBD)) {
        JasperPrint documento = JasperFillManager.fillReport(rutaJasper, parametros, conexion);
        exportarPdf(documento, rutaPdf);
        exportarPdfProtegido(documento, rutaPdfProtegido);
        exportarXlsx(documento, rutaXlsx, "Ventas");
        JRCsvDataSource catalogoDataSource = new JRCsvDataSource(new File("data/catalogo.csv"), "UTF-8");
        try {
            catalogoDataSource.setFieldDelimiter(',');
            catalogoDataSource.setUseFirstRowAsHeader(true);
            JasperPrint documentoCatalogo = JasperFillManager.fillReport(
                rutaCatalogoJasper, new HashMap<String, Object>(), catalogoDataSource);
            exportarXlsx(documentoCatalogo, rutaXlsxCatalogo, "Catálogo");
        } finally {
            catalogoDataSource.close();
        }
        System.out.println("Informe PDF generado en: " + new File(rutaPdf).getAbsolutePath());
        System.out.println("Informe PDF protegido generado en: " + new File(rutaPdfProtegido).getAbsolutePath());
        System.out.println("Informe Excel generado en: " + new File(rutaXlsx).getAbsolutePath());
        System.out.println("Informe catálogo Excel generado en: " + new File(rutaXlsxCatalogo).getAbsolutePath());
        System.out.println("Paginas del documento: " + documento.getPages().size());
        System.out.println("Parametro usuario: " + parametros.get("usuario"));
        System.out.println("M6 checkpoint generado correctamente");
    }
} catch (Exception e) {
    e.printStackTrace();
    System.exit(1);
}

private static void exportarPdf(JasperPrint documento, String ruta) throws Exception {
    JRPdfExporter exportador = new JRPdfExporter();
    SimplePdfExporterConfiguration configuracion = new SimplePdfExporterConfiguration();
    configuracion.setMetadataTitle("Informe de Ventas - EditorialReports");
    configuracion.setMetadataAuthor("Departamento Comercial");
    configuracion.setMetadataSubject("Resumen de ventas del catálogo");
    configuracion.setMetadataKeywords("ventas, catálogo, libros, editorial");
    configuracion.setMetadataCreator("JasperReports 6.20.0");
    configuracion.setDisplayMetadataTitle(Boolean.TRUE);
    configuracion.setCompressed(Boolean.TRUE);
    exportador.setConfiguration(configuracion);
    exportador.setExporterInput(new SimpleExporterInput(documento));
    exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta));
    exportador.exportReport();
}

private static void exportarPdfProtegido(JasperPrint documento, String ruta) throws Exception {
    JRPdfExporter exportador = new JRPdfExporter();
    SimplePdfExporterConfiguration configuracion = new SimplePdfExporterConfiguration();
    configuracion.setMetadataTitle("Informe de Ventas Protegido - EditorialReports");
    configuracion.setEncrypted(Boolean.TRUE);
    configuracion.setUserPassword("editorial2026");
    configuracion.setOwnerPassword("editorial-admin");
    configuracion.setAllowedPermissionsHint("PRINTING|COPY|SCREENREADERS");
    exportador.setConfiguration(configuracion);
    exportador.setExporterInput(new SimpleExporterInput(documento));
    exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta));
    exportador.exportReport();
}

private static void exportarXlsx(JasperPrint documento, String ruta, String nombreHoja) throws Exception {

```

```

JRXLsxExporter exportador = new JRXLsxExporter();
SimpleXlsxReportConfiguration informe = new SimpleXlsxReportConfiguration();
informe.setSheetNames(new String[]{nombreHoja});
informe.setShowGridLines(Boolean.FALSE);
informe.setCellLocked(Boolean.FALSE);
informe.setCellHidden(Boolean.FALSE);
informe.setDetectCellType(Boolean.TRUE);
informe.setOnePagePerSheet(Boolean.FALSE);
SimpleXlsxExporterConfiguration libro = new SimpleXlsxExporterConfiguration();
libro.setCreateCustomPalette(Boolean.TRUE);
exportador.setConfiguration(informe);
exportador.setConfiguration(libro);
exportador.setExporterInput(new SimpleExporterInput(documento));
exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta));
exportador.exportReport();
}
}

```

Explicación línea por línea

Línea 1	<code>import java.io.File;</code> → Importa <code>java.io.File</code> para gestionar rutas y crear la carpeta de salida.
Línea 2	<code>import java.sql.Connection;</code> → Importa <code>java.sql.Connection</code> para representar la conexión JDBC abierta contra SQLite.
Línea 3	<code>import java.sql.DriverManager;</code> → Importa <code>java.sql.DriverManager</code> para abrir la conexión JDBC a partir de la URL SQLite.
Línea 4	<code>import java.util.Arrays;</code> → Importa <code>java.util.Arrays</code> para construir la colección de categorías usada por el parámetro de lista.
Línea 5	<code>import java.util.HashMap;</code> → Importa <code>java.util.HashMap</code> para crear la implementación mutable del mapa de parámetros.
Línea 6	<code>import java.util.Map;</code> → Importa <code>java.util.Map</code> para tipar el mapa de parámetros que recibe JasperReports.
Línea 7	<code>import net.sf.jasperreports.engine.JasperCompileManager;</code> → Importa <code>net.sf.jasperreports.engine.JasperCompileManager</code> para compilar los JRXML a artefactos .jasper.
Línea 8	<code>import net.sf.jasperreports.engine.JasperFillManager;</code> → Importa <code>net.sf.jasperreports.engine.JasperFillManager</code> para llenar el informe compilado con parámetros y conexión.
Línea 9	<code>import net.sf.jasperreports.engine.JasperPrint;</code> → Importa <code>net.sf.jasperreports.engine.JasperPrint</code> para representar en memoria el documento ya paginado por JasperReports.
Línea 10	<code>import net.sf.jasperreports.engine.export.JRPdfExporter;</code> → Importa <code>net.sf.jasperreports.engine.export.JRPdfExporter</code> para usar la clase <code>JRPdfExporter</code> en el generador.
Línea 11	<code>import net.sf.jasperreports.export.SimpleExporterInput;</code> → Importa <code>net.sf.jasperreports.export.SimpleExporterInput</code> para usar la clase <code>SimpleExporterInput</code> en el generador.
Línea 12	<code>import net.sf.jasperreports.export.SimpleOutputStreamExporterOutput;</code> → Importa <code>net.sf.jasperreports.export.SimpleOutputStreamExporterOutput</code> para usar la clase <code>SimpleOutputStreamExporterOutput</code> en el generador.
Línea 13	<code>import net.sf.jasperreports.export.SimplePdfExporterConfiguration;</code> → Importa <code>net.sf.jasperreports.export.SimplePdfExporterConfiguration</code> para usar la clase <code>SimplePdfExporterConfiguration</code> en el generador.
Línea 14	<code>import net.sf.jasperreports.engine.data.JRCsvDataSource;</code> → Importa <code>net.sf.jasperreports.engine.data.JRCsvDataSource</code> para usar la clase <code>JRCsvDataSource</code> en el generador.
Línea 15	<code>import net.sf.jasperreports.engine.export.ooxml.JRXLsxExporter;</code> → Importa <code>net.sf.jasperreports.engine.export.ooxml.JRXLsxExporter</code> para usar la clase <code>JRXLsxExporter</code> en el generador.

Línea 16	<code>import net.sf.jasperreports.export.SimpleXlsxReportConfiguration;</code> → Importa <code>net.sf.jasperreports.export.SimpleXlsxReportConfiguration</code> para usar la clase <code>SimpleXlsxReportConfiguration</code> en el generador.
Línea 17	<code>import net.sf.jasperreports.export.SimpleXlsxExporterConfiguration;</code> → Importa <code>net.sf.jasperreports.export.SimpleXlsxExporterConfiguration</code> para usar la clase <code>SimpleXlsxExporterConfiguration</code> en el generador.
Línea 18	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 19	<code>public class GeneradorInformeVentas {</code> → Declara la clase ejecutable <code>GeneradorInformeVentas</code> que encapsula el generador del informe.
Línea 20	<code>public static void main(String[] args) {</code> → Declara <code>main</code> como punto de entrada de la aplicación Java; recibe los argumentos de línea de comandos aunque este ejemplo no los utiliza.
Línea 21	<code>try {</code> → Abre el bloque principal protegido: cualquier error de compilación, conexión, llenado o exportación será capturado por el <code>catch</code> final.
Línea 22	<code>String rutaJrxml = "reports/informe_ventas.jrxml";</code> → Declara <code>rutaJrxml</code> con ruta del JRXML maestro que se compilará; el valor configurado es <code>"reports/informe_ventas.jrxml"</code> .
Línea 23	<code>String rutaJasper = "reports/informe_ventas.jasper";</code> → Declara <code>rutaJasper</code> con ruta del <code>.jasper</code> maestro que producirá la compilación; el valor configurado es <code>"reports/informe_ventas.jasper"</code> .
Línea 24	<code>String rutaPdf = "output/informe_ventas.pdf";</code> → Declara <code>rutaPdf</code> con ruta del PDF final exportado; el valor configurado es <code>"output/informe_ventas.pdf"</code> .
Línea 25	<code>String rutaPdfProtegido = "output/informe_ventas_protegido.pdf";</code> → Declara <code>rutaPdfProtegido</code> con valor de configuración usado por el generador; el valor configurado es <code>"output/informe_ventas_protegido.pdf"</code> .
Línea 26	<code>String rutaXlsx = "output/informe_ventas.xlsx";</code> → Declara <code>rutaXlsx</code> con valor de configuración usado por el generador; el valor configurado es <code>"output/informe_ventas.xlsx"</code> .
Línea 27	<code>String rutaCatalogoJrxml = "reports/informe_catalogo_csv.jrxml";</code> → Declara <code>rutaCatalogoJrxml</code> con valor de configuración usado por el generador; el valor configurado es <code>"reports/informe_catalogo_csv.jrxml"</code> .
Línea 28	<code>String rutaCatalogoJasper = "reports/informe_catalogo_csv.jasper";</code> → Declara <code>rutaCatalogoJasper</code> con valor de configuración usado por el generador; el valor configurado es <code>"reports/informe_catalogo_csv.jasper"</code> .
Línea 29	<code>String rutaXlsxCatalogo = "output/informe_catalogo.xlsx";</code> → Declara <code>rutaXlsxCatalogo</code> con valor de configuración usado por el generador; el valor configurado es <code>"output/informe_catalogo.xlsx"</code> .
Línea 30	<code>String urlBD = "jdbc:sqlite:../EditorialReportsJava/data/editorial.db";</code> → Declara <code>urlBD</code> con URL JDBC de la base SQLite; el valor configurado es <code>"jdbc:sqlite:../EditorialReportsJava/data/editorial.db"</code> .
Línea 31	<code>new File("output").mkdirs();</code> → Crea la carpeta <code>output</code> si todavía no existe para evitar que la exportación falle por una ruta inexistente.
Línea 32	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 33	<code>String rutaSubJrxml = "reports/subinforme_ventas_detalle.jrxml";</code> → Declara <code>rutaSubJrxml</code> con ruta del JRXML del subinforme de detalle; el valor configurado es <code>"reports/subinforme_ventas_detalle.jrxml"</code> .
Línea 34	<code>String rutaSubJasper = "reports/subinforme_ventas_detalle.jasper";</code> → Declara <code>rutaSubJasper</code> con ruta del <code>.jasper</code> del subinforme compilado; el valor configurado es <code>"reports/subinforme_ventas_detalle.jasper"</code> .
Línea 35	<code>JasperCompileManager.compileReportToFile(rutaSubJrxml, rutaSubJasper);</code> → Compila el JRXML indicado en <code>rutaSubJrxml</code> y escribe el artefacto compilado en <code>rutaSubJasper</code> .
Línea 36	<code>JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);</code> → Compila el JRXML indicado en <code>rutaJrxml</code> y escribe el artefacto compilado en <code>rutaJasper</code> .

Línea 37	<code>JasperCompileManager.compileReportToFile(rutaCatalogoJrxml, rutaCatalogoJasper);</code> → Compila el JRXML indicado en <code>rutaCatalogoJrxml</code> y escribe el artefacto compilado en <code>rutaCatalogoJasper</code> .
Línea 38	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 39	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 40	<code>Map<String, Object> parametros = new HashMap<String, Object>();</code> → Crea el mapa tipado de parámetros que se entregará a <code>JasperFillManager.fillReport</code> .
Línea 41	<code>parametros.put("usuario", "Ana Martínez");</code> → Asigna al parámetro JasperReports <code>usuario</code> el valor Java <code>"Ana Martínez"</code> antes del llenado.
Línea 42	<code>parametros.put("departamento", "Comercial");</code> → Asigna al parámetro JasperReports <code>departamento</code> el valor Java <code>"Comercial"</code> antes del llenado.
Línea 43	<code>parametros.put("periodo", "Septiembre 2026");</code> → Asigna al parámetro JasperReports <code>periodo</code> el valor Java <code>"Septiembre 2026"</code> antes del llenado.
Línea 44	<code>parametros.put("tipoIva", Double.valueOf(0.21d));</code> → Asigna al parámetro JasperReports <code>tipoIva</code> el valor Java <code>Double.valueOf(0.21d)</code> antes del llenado.
Línea 45	<code>parametros.put("mostrarDetalle", Boolean.TRUE);</code> → Asigna al parámetro JasperReports <code>mostrarDetalle</code> el valor Java <code>Boolean.TRUE</code> antes del llenado.
Línea 46	<code>parametros.put("categoria", null);</code> → Asigna al parámetro JasperReports <code>categoria</code> el valor Java <code>null</code> antes del llenado.
Línea 47	<code>parametros.put("precioMinimo", null);</code> → Asigna al parámetro JasperReports <code>precioMinimo</code> el valor Java <code>null</code> antes del llenado.
Línea 48	<code>parametros.put("precioMaximo", null);</code> → Asigna al parámetro JasperReports <code>precioMaximo</code> el valor Java <code>null</code> antes del llenado.
Línea 49	<code>parametros.put("umbralUnidades", Integer.valueOf(5));</code> → Asigna al parámetro JasperReports <code>umbralUnidades</code> el valor Java <code>Integer.valueOf(5)</code> antes del llenado.
Línea 50	<code>parametros.put("textoBusqueda", null);</code> → Asigna al parámetro JasperReports <code>textoBusqueda</code> el valor Java <code>null</code> antes del llenado.
Línea 51	<code>parametros.put("categoriasLista", Arrays.asList("Novela", "Realismo mágico", "Cuento", "Poesía"));</code> → Asigna al parámetro JasperReports <code>categoriasLista</code> el valor Java <code>Arrays.asList("Novela", "Realismo mágico", "Cuento", "Poesía")</code> antes del llenado.
Línea 52	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 53	<code>try (Connection conexion = DriverManager.getConnection(urlBD)) {</code> → Abre la conexión SQLite mediante <code>DriverManager</code> dentro de un <code>try-with-resources</code> , por lo que <code>conexion</code> se cierra automáticamente al terminar el bloque.
Línea 54	<code>JasperPrint documento = JasperFillManager.fillReport(rutaJasper, parametros, conexion);</code> → Inicia el llenado del informe y guarda en <code>documento</code> el <code>JasperPrint</code> paginado que devolverá JasperReports.
Línea 55	<code>exportarPdf(documento, rutaPdf);</code> → Invoca la exportación PDF normal usando el <code>JasperPrint</code> ya llenado y la ruta principal de salida.
Línea 56	<code>exportarPdfProtegido(documento, rutaPdfProtegido);</code> → Genera una segunda salida PDF cifrada para validar contraseñas y permisos sin alterar el PDF normal.
Línea 57	<code>exportarXlsx(documento, rutaXlsx, "Ventas");</code> → Exporta el <code>JasperPrint</code> de ventas a <code>informe_ventas.xlsx</code> usando el nombre de hoja <code>Ventas</code> .
Línea 58	<code>JRCsvDataSource catalogoDataSource = new JRCsvDataSource(new File("data/catalogo.csv"), "UTF-8");</code> → Abre <code>data/catalogo.csv</code> como <code>datasource</code> JasperReports en UTF-8 para llenar el informe de catálogo sin JDBC.
Línea 59	<code>try {</code> → Abre el bloque principal protegido: cualquier error de compilación, conexión, llenado o exportación será capturado por el <code>catch</code> final.

Línea 60	<code>catalogoDataSource.setFieldDelimiter(' ');</code> → Configura punto y coma como delimitador de campos CSV.
Línea 61	<code>catalogoDataSource.setUseFirstRowAsHeader(true);</code> → Indica que la primera fila del CSV contiene los nombres de los fields del informe de catálogo.
Línea 62	<code>JasperPrint documentoCatalogo = JasperFillManager.fillReport(</code> → Inicia el llenado del informe de catálogo y guarda el resultado paginado en <code>documentoCatalogo</code> .
Línea 63	<code>rutaCatalogoJasper, new HashMap<String, Object>(), catalogoDataSource);</code> → Completa <code>fillReport</code> pasando el catálogo compilado, un mapa de parámetros vacío y el <code>JRCsvDataSource</code> .
Línea 64	<code>exportarXlsx(documentoCatalogo, rutaXlsxCatalogo, "Catálogo");</code> → Exporta el <code>JasperPrint</code> del catálogo a <code>informe_catalogo.xlsx</code> usando la hoja <code>Catálogo</code> .
Línea 65	<code>}</code> <code>finally {</code> → Abre el bloque <code>finally</code> que se ejecutará siempre para liberar el <code>datasource</code> CSV aunque falle el llenado o la exportación.
Línea 66	<code>catalogoDataSource.close();</code> → Cierra explícitamente el <code>JRCsvDataSource</code> para liberar el lector del archivo de catálogo.
Línea 67	<code>}</code> → Cierra el bloque <code>try-with-resources</code> de la conexión JDBC.
Línea 68	<code>System.out.println("Informe PDF generado en: " + new File(rutaPdf).getAbsolutePath());</code> → Escribe en la consola la evidencia "Informe PDF generado en: " + <code>new File(rutaPdf).getAbsolutePath()</code> , que queda registrada por el workflow E2E.
Línea 69	<code>System.out.println("Informe PDF protegido generado en: " + new File(rutaPdfProtegido).getAbsolutePath());</code> → Escribe en la consola la evidencia "Informe PDF protegido generado en: " + <code>new File(rutaPdfProtegido).getAbsolutePath()</code> , que queda registrada por el workflow E2E.
Línea 70	<code>System.out.println("Informe Excel generado en: " + new File(rutaXlsx).getAbsolutePath());</code> → Escribe en la consola la evidencia "Informe Excel generado en: " + <code>new File(rutaXlsx).getAbsolutePath()</code> , que queda registrada por el workflow E2E.
Línea 71	<code>System.out.println("Informe catálogo Excel generado en: " + new File(rutaXlsxCatalogo).getAbsolutePath());</code> → Escribe en la consola la evidencia "Informe catálogo Excel generado en: " + <code>new File(rutaXlsxCatalogo).getAbsolutePath()</code> , que queda registrada por el workflow E2E.
Línea 72	<code>System.out.println("Paginas del documento: " + documento.getPages().size());</code> → Escribe en la consola la evidencia "Paginas del documento: " + <code>documento.getPages().size()</code> , que queda registrada por el workflow E2E.
Línea 73	<code>System.out.println("Parametro usuario: " + parametros.get("usuario"));</code> → Escribe en la consola la evidencia "Parametro usuario: " + <code>parametros.get("usuario")</code> , que queda registrada por el workflow E2E.
Línea 74	<code>System.out.println("M6 checkpoint generado correctamente");</code> → Escribe en la consola la evidencia "M6 checkpoint generado correctamente", que queda registrada por el workflow E2E.
Línea 75	<code>}</code> → Cierra el bloque <code>try-with-resources</code> de la conexión JDBC.
Línea 76	<code>}</code> <code>catch (Exception e) {</code> → Cierra el bloque protegido y abre el manejador que captura cualquier excepción del proceso completo.
Línea 77	<code>e.printStackTrace();</code> → Imprime la traza completa de la excepción para que el fallo sea diagnosticable en local y en GitHub Actions.
Línea 78	<code>System.exit(1);</code> → Finaliza el proceso con código 1 para que CI marque la ejecución como fallida y no oculte el error.
Línea 79	<code>}</code> → Cierra el bloque <code>catch</code> o el bloque principal de control asociado a <code>main</code> .
Línea 80	<code>}</code> → Cierra el método <code>main</code> .
Línea 81	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.

Línea 82	<code>private static void exportarPdf(JasperPrint documento, String ruta) throws Exception {</code> → Declara el helper privado <code>exportarPdf</code> , que encapsula una exportación concreta y propaga cualquier error al <code>catch</code> principal.
Línea 83	<code>JRPdfExporter exportador = new JRPdfExporter();</code> → Crea el exportador PDF avanzado que admite configuración documental y de seguridad.
Línea 84	<code>SimplePdfExporterConfiguration configuracion = new SimplePdfExporterConfiguration();</code> → Crea el objeto <code>SimplePdfExporterConfiguration</code> que recibirá las opciones específicas de esta exportación.
Línea 85	<code>configuracion.setMetadataTitle("Informe de Ventas - EditorialReports");</code> → Fija el título de los metadatos PDF con la API específica de <code>SimplePdfExporterConfiguration</code> .
Línea 86	<code>configuracion.setMetadataAuthor("Departamento Comercial");</code> → Fija el autor en los metadatos del PDF.
Línea 87	<code>configuracion.setMetadataSubject("Resumen de ventas del catálogo");</code> → Fija el asunto documental del PDF.
Línea 88	<code>configuracion.setMetadataKeywords("ventas, catálogo, libros, editorial");</code> → Fija las palabras clave que quedarán registradas en las propiedades del PDF.
Línea 89	<code>configuracion.setMetadataCreator("JasperReports 6.20.0");</code> → Registra JasperReports 6.20.0 como creador del PDF.
Línea 90	<code>configuracion.setDisplayMetadataTitle(Boolean.TRUE);</code> → Solicita a los lectores PDF que utilicen el título de metadatos cuando soporten esa preferencia.
Línea 91	<code>configuracion.setCompressed(Boolean.TRUE);</code> → Activa la compresión del PDF mediante la configuración del exportador.
Línea 92	<code>exportador.setConfiguration(configuracion);</code> → Asocia al exportador la configuración específica preparada en las líneas anteriores.
Línea 93	<code>exportador.setExporterInput(new SimpleExporterInput(documento));</code> → Entrega al exportador el <code>JasperPrint</code> documento ya llenado; no se vuelve a consultar la base de datos.
Línea 94	<code>exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta));</code> → Dirige la salida binaria del exportador al archivo indicado por el parámetro <code>ruta</code> .
Línea 95	<code>exportador.exportReport();</code> → Ejecuta la exportación con la entrada, salida y configuración ya asignadas; aquí se materializa el archivo.
Línea 96	<code>}</code> → Cierra el método <code>main</code> .
Línea 97	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 98	<code>private static void exportarPdfProtegido(JasperPrint documento, String ruta) throws Exception {</code> → Declara el helper privado <code>exportarPdfProtegido</code> , que encapsula una exportación concreta y propaga cualquier error al <code>catch</code> principal.
Línea 99	<code>JRPdfExporter exportador = new JRPdfExporter();</code> → Crea el exportador PDF avanzado que admite configuración documental y de seguridad.
Línea 100	<code>SimplePdfExporterConfiguration configuracion = new SimplePdfExporterConfiguration();</code> → Crea el objeto <code>SimplePdfExporterConfiguration</code> que recibirá las opciones específicas de esta exportación.
Línea 101	<code>configuracion.setMetadataTitle("Informe de Ventas Protegido - EditorialReports");</code> → Fija el título de los metadatos PDF con la API específica de <code>SimplePdfExporterConfiguration</code> .
Línea 102	<code>configuracion.setEncrypted(Boolean.TRUE);</code> → Activa el cifrado de la salida PDF protegida.
Línea 103	<code>configuracion.setUserPassword("editorial2026");</code> → Configura la contraseña de apertura que el E2E verifica con <code>pdftinfo -upw</code> .
Línea 104	<code>configuracion.setOwnerPassword("editorial-admin");</code> → Configura la contraseña de propietario del PDF protegido.
Línea 105	<code>configuracion.setAllowedPermissionsHint("PRINTING COPY SCREENREADERS");</code> → Declara los permisos PDF autorizados mediante la cadena de hints admitida por JasperReports.

Línea 106	<code>exportador.setConfiguration(configuracion);</code> → Asocia al exportador la configuración específica preparada en las líneas anteriores.
Línea 107	<code>exportador.setExporterInput(new SimpleExporterInput(documento));</code> → Entrega al exportador el JasperPrint documento ya llenado; no se vuelve a consultar la base de datos.
Línea 108	<code>exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta));</code> → Dirige la salida binaria del exportador al archivo indicado por el parámetro ruta.
Línea 109	<code>exportador.exportReport();</code> → Ejecuta la exportación con la entrada, salida y configuración ya asignadas; aquí se materializa el archivo.
Línea 110	<code>}</code> → Cierra el método main.
Línea 111	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 112	<code>private static void exportarXlsx(JasperPrint documento, String ruta, String nombreHoja) throws Exception {</code> → Declara el helper privado exportarXlsx, que encapsula una exportación concreta y propaga cualquier error al catch principal.
Línea 113	<code>JRXLsxExporter exportador = new JRXLsxExporter();</code> → Crea el exportador OOXML que escribirá el libro XLSX.
Línea 114	<code>SimpleXlsxReportConfiguration informe = new SimpleXlsxReportConfiguration();</code> → Crea la configuración de cómo el JasperPrint se distribuye en hojas y celdas XLSX.
Línea 115	<code>informe.setSheetNames(new String[]{nombreHoja});</code> → Asigna el nombre Ventas a la hoja; el E2E lo comprueba dentro de xl/workbook.xml.
Línea 116	<code>informe.setShowGridLines(Boolean.FALSE);</code> → Desactiva la cuadrícula predeterminada de la hoja Excel.
Línea 117	<code>informe.setCellLocked(Boolean.FALSE);</code> → Configura las celdas exportadas sin bloqueo adicional.
Línea 118	<code>informe.setCellHidden(Boolean.FALSE);</code> → Evita marcar como ocultas las celdas exportadas.
Línea 119	<code>informe.setDetectCellType(Boolean.TRUE);</code> → Pide al exportador detectar tipos numéricos/fecha en lugar de convertir indiscriminadamente a texto.
Línea 120	<code>informe.setOnePagePerSheet(Boolean.FALSE);</code> → Mantiene el informe en una misma hoja lógica en lugar de crear una hoja por página.
Línea 121	<code>SimpleXlsxExporterConfiguration libro = new SimpleXlsxExporterConfiguration();</code> → Crea la configuración propia del libro/exportador XLSX.
Línea 122	<code>libro.setCreateCustomPalette(Boolean.TRUE);</code> → Activa la paleta personalizada del exportador XLSX para reproducir mejor los colores.
Línea 123	<code>exportador.setConfiguration(informe);</code> → Aplica la configuración de informe XLSX: hoja, cuadrícula, bloqueo, tipos y paginación.
Línea 124	<code>exportador.setConfiguration(libro);</code> → Aplica además la configuración global del libro XLSX, incluida la paleta personalizada.
Línea 125	<code>exportador.setExporterInput(new SimpleExporterInput(documento));</code> → Entrega al exportador el JasperPrint documento ya llenado; no se vuelve a consultar la base de datos.
Línea 126	<code>exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta));</code> → Dirige la salida binaria del exportador al archivo indicado por el parámetro ruta.
Línea 127	<code>exportador.exportReport();</code> → Ejecuta la exportación con la entrada, salida y configuración ya asignadas; aquí se materializa el archivo.
Línea 128	<code>}</code> → Cierra el método main.
Línea 129	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 130	<code>}</code> → Cierra la clase GeneradorInformeVentas.

pom.xml

XML

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>es.jaimegallo.editorialreports</groupId>
  <artifactId>editorial-reports-m3</artifactId>
  <version>1.0-SNAPSHOT</version>
  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.source>8</maven.compiler.source>
    <maven.compiler.target>8</maven.compiler.target>
  </properties>
  <repositories>
    <repository><id>jaspersoft-third-party</id><url>https://jaspersoft.jfrog.io/jaspersoft/third-party-ce-artifacts</url></repository>
    <repository><id>jr-ce-releases</id><url>https://jaspersoft.jfrog.io/jaspersoft/jr-ce-releases</url></repository>
  </repositories>
  <dependencies>
    <dependency><groupId>net.sf.jasperreports</groupId><artifactId>jasperreports</artifactId><version>6.20.0</version></dependency>
    <dependency><groupId>net.sf.jasperreports</groupId><artifactId>jasperreports-fonts</artifactId><version>6.20.0</version></
dependency>
    <dependency><groupId>xalan</groupId><artifactId>xalan</artifactId><version>2.7.2</version></dependency>
    <dependency><groupId>org.xerial</groupId><artifactId>sqlite-jdbc</artifactId><version>3.44.0.0</version></dependency>
    <dependency><groupId>org.apache.poi</groupId><artifactId>poi</artifactId><version>5.1.0</version></dependency>
    <dependency><groupId>org.apache.poi</groupId><artifactId>poi-ooxml</artifactId><version>5.1.0</version></dependency>
    <dependency><groupId>org.slf4j</groupId><artifactId>slf4j-simple</artifactId><version>1.7.36</version></dependency>
  </dependencies>
  <build>
    <sourceDirectory>src</sourceDirectory>
    <plugins>
      <plugin><groupId>org.apache.maven.plugins</groupId><artifactId>maven-compiler-plugin</artifactId><version>3.11.0</version></
plugin>
      <plugin><groupId>org.apache.maven.plugins</groupId><artifactId>maven-dependency-plugin</artifactId><version>3.6.1</version></
plugin>
    </plugins>
  </build>
</project>
```

Explicación línea por línea

Línea 1	<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" → Abre el documento Maven project y declara los namespaces del modelo POM.
Línea 2	xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd" → Relaciona el namespace de JasperReports con su XSD para que Studio y el compilador validen la estructura.
Línea 3	<modelVersion>4.0.0</modelVersion> → Declara la versión 4.0.0 del modelo de proyecto Maven.
Línea 4	<groupId>es.jaimegallo.editorialreports</groupId> → Identifica el grupo Maven del proyecto EditorialReports.
Línea 5	<artifactId>editorial-reports-m3</artifactId> → Define el identificador del artefacto Maven que se compila y empaqueta.
Línea 6	<version>1.0-SNAPSHOT</version> → Fija la versión del artefacto Maven.
Línea 7	<properties> → Abre el bloque de propiedades Maven usado para codificación y nivel del compilador Java.
Línea 8	<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding> → Fija UTF-8 como codificación fuente del proyecto Maven.
Línea 9	<maven.compiler.source>8</maven.compiler.source> → Fija Java 8 como nivel de lenguaje de compilación.
Línea 10	<maven.compiler.target>8</maven.compiler.target> → Fija Java 8 como bytecode objetivo.
Línea 11	</properties> → Cierra properties y vuelve al elemento padre de la jerarquía JRXML.
Línea 12	<repositories> → Abre la lista de repositorios adicionales desde los que Maven puede resolver dependencias.
Línea 13	<repository><id>jaspersoft-third-party</id><url>https://jaspersoft.jfrog.io/jaspersoft/third-party-ce-artifacts</url></repository> → Composición de la línea: encadena además <repository>, <id>, <url> dentro de la misma jerarquía.

Línea 14	<code><repository><id>jr-ce-releases</id><url>https://jaspersoft.jfrog.io/jaspersoft/jr-ce-releases/</url></repository></code> → Composición de la línea: encadena además <code><repository></code> , <code><id></code> , <code><url></code> dentro de la misma jerarquía.
Línea 15	<code></repositories></code> → Cierra <code>repositories</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 16	<code><dependencies></code> → Abre la colección de dependencias runtime/compilación, incluida JasperReports y POI.
Línea 17	<code><dependency><groupId>net.sf.jasperreports</groupId><artifactId>jasperreports</artifactId><version>6.20.0</version></dependency></code> → Composición de la línea: encadena además <code><dependency></code> , <code><groupId></code> , <code><artifactId></code> , <code><version></code> dentro de la misma jerarquía.
Línea 18	<code><dependency><groupId>net.sf.jasperreports</groupId><artifactId>jasperreports-fonts</artifactId><version>6.20.0</version></dependency></code> → Composición de la línea: encadena además <code><dependency></code> , <code><groupId></code> , <code><artifactId></code> , <code><version></code> dentro de la misma jerarquía.
Línea 19	<code><dependency><groupId>xalan</groupId><artifactId>xalan</artifactId><version>2.7.2</version></dependency></code> → Composición de la línea: encadena además <code><dependency></code> , <code><groupId></code> , <code><artifactId></code> , <code><version></code> dentro de la misma jerarquía.
Línea 20	<code><dependency><groupId>org.xerial</groupId><artifactId>sqlite-jdbc</artifactId><version>3.44.0.0</version></dependency></code> → Composición de la línea: encadena además <code><dependency></code> , <code><groupId></code> , <code><artifactId></code> , <code><version></code> dentro de la misma jerarquía.
Línea 21	<code><dependency><groupId>org.apache.poi</groupId><artifactId>poi</artifactId><version>5.1.0</version></dependency></code> → Composición de la línea: encadena además <code><dependency></code> , <code><groupId></code> , <code><artifactId></code> , <code><version></code> dentro de la misma jerarquía.
Línea 22	<code><dependency><groupId>org.apache.poi</groupId><artifactId>poi-ooxml</artifactId><version>5.1.0</version></dependency></code> → Composición de la línea: encadena además <code><dependency></code> , <code><groupId></code> , <code><artifactId></code> , <code><version></code> dentro de la misma jerarquía.
Línea 23	<code><dependency><groupId>org.slf4j</groupId><artifactId>slf4j-simple</artifactId><version>1.7.36</version></dependency></code> → Composición de la línea: encadena además <code><dependency></code> , <code><groupId></code> , <code><artifactId></code> , <code><version></code> dentro de la misma jerarquía.
Línea 24	<code></dependencies></code> → Cierra <code>dependencies</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 25	<code><build></code> → Abre la configuración de construcción Maven.
Línea 26	<code><sourceDirectory>src</sourceDirectory></code> → Indica que las clases Java fuente del proyecto están directamente en la carpeta <code>src</code> .
Línea 27	<code><plugins></code> → Abre la lista de plugins Maven usados durante la construcción.
Línea 28	<code><plugin><groupId>org.apache.maven.plugins</groupId><artifactId>maven-compiler-plugin</artifactId><version>3.11.0</version></plugin></code> → Composición de la línea: encadena además <code><plugin></code> , <code><groupId></code> , <code><artifactId></code> , <code><version></code> dentro de la misma jerarquía.
Línea 29	<code><plugin><groupId>org.apache.maven.plugins</groupId><artifactId>maven-dependency-plugin</artifactId><version>3.6.1</version></plugin></code> → Composición de la línea: encadena además <code><plugin></code> , <code><groupId></code> , <code><artifactId></code> , <code><version></code> dentro de la misma jerarquía.
Línea 30	<code></plugins></code> → Cierra <code>plugins</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 31	<code></build></code> → Cierra <code>build</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 32	<code></project></code> → Cierra <code>project</code> y vuelve al elemento padre de la jerarquía JRXML.

Parte D — Simulación del resultado y de la estructura del proyecto

D.1 — Flujo de exportación / diseño

TEXT

JasperPrint ventas
└─ XLSX hoja Ventas

```
JasperPrint catálogo desde JRCsvDataSource
└─ XLSX hoja Catálogo
```

Qué representa: la transformación funcional que debe existir al terminar 6.2.

Cómo verificarlo: comparar el flujo con la Parte C y ejecutar el generador; el JRXML/JRTX debe seguir siendo el heredado de M5/5.6.

D.2 — Estructura lógica en código y recursos

```
Java
├─ JRXlsxExporter
├─ SimpleXlsxReportConfiguration
├─ SimpleXlsxExporterConfiguration
└─ JRCsvDataSource para el reto de catálogo

pom.xml
└─ POI 5.1.0 + POI-OOXML 5.1.0
```

TEXT

Qué representa: las clases, métodos y recursos que sustituyen en M6 al trabajo visual sobre bandas y componentes.

Cómo verificarlo: abrir Java/POM/CSS/properties según corresponda y contrastar nombres y tipos con la Parte C ejecutable.

D.3 — Archivos de salida

```
output/
├─ informe_ventas.pdf
├─ informe_ventas_protegido.pdf
├─ informe_ventas.xlsx [Ventas]
└─ informe_catalogo.xlsx [Catálogo]
```

TEXT

Qué representa: los artefactos acumulativos esperados en output.

Cómo verificarlo: ejecutar `GeneradorInformeVentas`, abrir cada formato con una herramienta compatible y contrastar los contratos automatizados del E2E.

D.4 — Árbol acumulativo del checkpoint

```
M6/6.2/
├─ EditorialReports/EXPORTACION_EXCEL.md
├─ EditorialReportsJava/pom.xml
└─ EditorialReportsJava/src/GeneradorInformeVentas.java
```

TEXT

Evidencia E2E: run **36249131955**, commit 12a0eba90859a92b12578d59ae592ad17dac5fb6, artifact runtime **10907873672**.

Invariantes: 14 libros, 9 ventas, 31 unidades, 633,40 € y 6 páginas en `informe_ventas`.

Errores comunes del ejercicio completo

Error	Causa	Solución
NoClassDefFoundError de POI	POI no está en Maven	Añadir poi y poi-ooxml 5.1.0
setSheetNames no compila	Se usa la clase de configuración equivocada	Usar SimpleXlsxReportConfiguration
La hoja se llama Sheet1	No se aplicó el nombre	Pasar nombreHoja a setSheetNames
El catálogo queda vacío	Se intenta llenar con JDBC	Usar JRCsvDataSource sobre data/catalogo.csv
El XLSX está corrupto	Salida incompleta	Revisar exportReport() y dependencias

Reto resuelto paso a paso

Enunciado original: generar además un Excel del catálogo con hoja Catálogo.

1. Se compila `informe_catalogo_csv.jrxml`.
2. Se crea `JRCsvDataSource` sobre `data/catalogo.csv` en UTF-8.
3. Se configura coma como delimitador y primera fila como cabecera.

4. Se llena el informe de catálogo con su datasource real.
5. `exportarXlsx` recibe el nombre de hoja como parámetro.
6. Ventas se exporta con hoja Ventas.
7. Catálogo se exporta con hoja Catálogo.
8. El E2E abre ambos OOXML y verifica los nombres en `xl/workbook.xml`.

Resultado del reto: los dos XLSX se generan en la misma ejecución y contienen las hojas correctas.

Analogía final con el contexto de la editorial

Son dos libros contables producidos por la misma cadena: uno resume ventas y otro publica el catálogo.

Resultado esperado

- PDF normal/protegido heredados.
- `informe_ventas.xlsx` con hoja Ventas.
- `informe_catalogo.xlsx` con hoja Catálogo.
- POI 5.1.0 resuelto por Maven.
- `EXPORTACION_EXCEL.md` trazado al código.

Conclusión y enlace al siguiente punto

6.2 añade XLSX con configuración correcta de hoja/libro y demuestra el uso de un segundo origen de datos real para el reto de catálogo.

Punto 6.3 — Exportación a HTML

Objetivos de aprendizaje

- Comprender el papel del exportador HTML y sus limitaciones respecto a PDF.
- Configurar el exportador JRHtmlExporter con SimpleHtmlExporterConfiguration.
- Exportar las imágenes del informe a un directorio y referenciarlas desde el HTML.
- Añadir cabecera, pie y separador de páginas al archivo HTML.
- Integrar el HTML generado con una hoja de estilos CSS externa.
- Documentar la exportación a HTML del proyecto EditorialReports.

Parte A — Práctica visual/IDE verificada

Paso 1: Abrir 6.3 y verificar PDF/XLSX heredados

Acciones:

1. Abrir `GeneradorInformeVentas.java` del checkpoint 6.3.
2. Confirmar las exportaciones PDF normal/protegido y los dos XLSX.
3. Comprobar que el POM conserva las dependencias POI.

Verificación visual: el código acumulado de 6.2 permanece intacto.

Qué hace: completa la operación «Abrir 6.3 y verificar PDF/XLSX heredados» en el checkpoint 6.3.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.3.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.3.

Analogía: es como publicar el catálogo en la intranet junto con sus recursos y un acceso a la versión imprimible.

Paso 2: Añadir imports HTML y de recursos

Acciones:

1. Importar `HtmlExporter` y `FileHtmlResourceHandler`.
2. Importar `SimpleHtmlExporterConfiguration` y `SimpleHtmlExporterOutput`.
3. Importar `Files`, `Paths` y `StandardCopyOption`.

Verificación visual: Problems no muestra clases HTML o NIO sin resolver.

Qué hace: completa la operación «Añadir imports HTML y de recursos» en el checkpoint 6.3.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.3.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.3.

Analogía: es como publicar el catálogo en la intranet junto con sus recursos y un acceso a la versión imprimible.

Paso 3: Declarar la ruta HTML

Acciones:

1. Añadir `String rutaHtml = "output/informe_ventas.html";` junto a las rutas de salida.

2. Mantener las rutas PDF/XLSX anteriores.

Verificación visual: Source contiene la nueva ruta HTML y las salidas acumuladas.

Qué hace: completa la operación «Declarar la ruta HTML» en el checkpoint 6.3.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.3.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.3.

Analogía: es como publicar el catálogo en la intranet junto con sus recursos y un acceso a la versión imprimible.

Paso 4: Crear la hoja CSS fuente

Acciones:

1. Crear `EditorialReports/resources/styles/editorial.css`.
2. Añadir reglas para `body`, `.jrPage`, `.salto-pagina` y `.enlace-pdf`.
3. Usar DejaVu Sans como primera familia del `font-family`.

Verificación visual: Project Explorer muestra el CSS y la regla `.enlace-pdf`.

Qué hace: completa la operación «Crear la hoja CSS fuente» en el checkpoint 6.3.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.3.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.3.

Analogía: es como publicar el catálogo en la intranet junto con sus recursos y un acceso a la versión imprimible.

Paso 5: Preparar carpetas y copiar CSS a output

Acciones:

1. Crear `output/images` con `mkdirs()`.
2. Crear `output/styles` con `mkdirs()`.
3. Copiar `resources/styles/editorial.css` a `output/styles/editorial.css` con `Files.copy(..., REPLACE_EXISTING)`.

Verificación visual: al ejecutar, la hoja CSS aparece junto al HTML publicado.

Qué hace: completa la operación «Preparar carpetas y copiar CSS a output» en el checkpoint 6.3.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.3.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.3.

Analogía: es como publicar el catálogo en la intranet junto con sus recursos y un acceso a la versión imprimible.

Paso 6: Crear el método exportarHtml

Acciones:

1. Declarar `exportarHtml(JasperPrint documento, String ruta)`.

2. Crear `HtmlExporter` y `SimpleHtmlExporterConfiguration`.
3. No utilizar la clase antigua `JRHtmlExporter`.

Verificación visual: Source contiene `HtmlExporter`, que es la clase usada por el checkpoint compilado.

Qué hace: completa la operación «Crear el método exportarHtml» en el checkpoint 6.3.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.3.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.3.

Analogía: es como publicar el catálogo en la intranet junto con sus recursos y un acceso a la versión imprimible.

Paso 7: Configurar cabecera, pie y separación de páginas

Acciones:

1. En `setHtmlHeader`, incluir `<meta charset='UTF-8'>`, título y enlace `styles/editorial.css`.
2. Añadir en la misma cabecera el enlace Descargar PDF con `href='informe_ventas.pdf'`.
3. Configurar `setHtmlFooter` (`"</body></html>"`).
4. Configurar `setBetweenPagesHtml` (`"<hr class='salto-pagina' />"`).

Verificación visual: la cabecera HTML contiene el CSS y el reto del enlace al PDF.

Qué hace: completa la operación «Configurar cabecera, pie y separación de páginas» en el checkpoint 6.3.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.3.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.3.

Analogía: es como publicar el catálogo en la intranet junto con sus recursos y un acceso a la versión imprimible.

Paso 8: Configurar la salida y los recursos de imagen

Acciones:

1. Crear `SimpleHtmlExporterOutput` (`ruta, "UTF-8"`).
2. Asignar `new FileHtmlResourceHandler(new File("output/images"), "images/{0}"))` mediante `setImageHandler`.
3. Asignar configuración, input y output al exportador.

Verificación visual: la ruta física de imágenes y la URI relativa están definidas en el output, no en la configuración HTML.

Qué hace: completa la operación «Configurar la salida y los recursos de imagen» en el checkpoint 6.3.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.3.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.3.

Analogía: es como publicar el catálogo en la intranet junto con sus recursos y un acceso a la versión imprimible.

Paso 9: Ejecutar la exportación HTML

Acciones:

1. Finalizar el método con `exportador.exportReport()`.
2. Invocar `exportarHtml(documento, rutaHtml)` después de copiar el CSS.
3. Mantener el mismo `JasperPrint` documento.

Verificación visual: la exportación HTML no vuelve a ejecutar `fillReport`.

Qué hace: completa la operación «Ejecutar la exportación HTML» en el checkpoint 6.3.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.3.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.3.

Analogía: es como publicar el catálogo en la intranet junto con sus recursos y un acceso a la versión imprimible.

Paso 10: Compilar y ejecutar**Acciones:**

1. Guardar Java y CSS.
2. Ejecutar `mvn clean package`.
3. Ejecutar `GeneradorInformeVentas` desde `EditorialReports`.

Verificación visual: Console informa las salidas PDF, XLSX y HTML sin excepción.

Qué hace: completa la operación «Compilar y ejecutar» en el checkpoint 6.3.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.3.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.3.

Analogía: es como publicar el catálogo en la intranet junto con sus recursos y un acceso a la versión imprimible.

Paso 11: Abrir el HTML en navegador**Acciones:**

1. Abrir `output/informe_ventas.html`.
2. Comprobar título, contenido del informe y separación entre páginas.
3. Comprobar que el estilo externo se carga.

Verificación visual: el navegador presenta el informe y no muestra recursos rotos.

Qué hace: completa la operación «Abrir el HTML en navegador» en el checkpoint 6.3.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.3.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.3.

Analogía: es como publicar el catálogo en la intranet junto con sus recursos y un acceso a la versión imprimible.

Paso 12: Validar el reto del enlace al PDF

Acciones:

1. Comprobar que aparece Descargar PDF al inicio del HTML.
2. Hacer clic en el enlace.
3. Verificar que abre output/informe_ventas.pdf.

Verificación visual: el enlace relativo resuelve el PDF generado en la misma carpeta output.

Qué hace: completa la operación «Validar el reto del enlace al PDF» en el checkpoint 6.3.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.3.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.3.

Analogía: es como publicar el catálogo en la intranet junto con sus recursos y un acceso a la versión imprimible.

Paso 13: Documentar HTML y recursos

Acciones:

1. Crear/abrir EXPORTACION_HTML.md.
2. Registrar HtmlExporter, cabecera/pie, FileHtmlResourceHandler, CSS y enlace al PDF.
3. Registrar las carpetas output/images y output/styles.

Verificación visual: la documentación coincide con Java, CSS y árbol de salida.

Qué hace: completa la operación «Documentar HTML y recursos» en el checkpoint 6.3.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.3.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.3.

Analogía: es como publicar el catálogo en la intranet junto con sus recursos y un acceso a la versión imprimible.

Parte B — JRXML/JRTX completo explicado línea por línea

En M6 el diseño no cambia: estos tres archivos deben permanecer byte a byte iguales a M5/5.6.

Informe maestro heredado y ejecutable

```
<?xml version="1.0" encoding="UTF-8"?>
<jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/
jasperreport.xsd"
  name="informe_ventas"
  language="java"
  pageWidth="595"
  pageHeight="842"
  columnWidth="555"
  leftMargin="20"
  rightMargin="20"
  topMargin="20"
  bottomMargin="20"
  uuid="3d2c2bd7-3b93-4da9-8b60-6b3c45674c91">
  <property name="com.jaspersoft.studio.data.defaultdataadapter" value="SQLiteEditorial"/>
  <template><![CDATA["resources/styles/EditorialStyles.jrtx"]]></template>
  <style name="Sans_Normal" isDefault="true" fontName="DejaVu Sans" fontSize="10"/>
  <style name="TituloPrincipal" style="Sans_Normal" fontSize="18" isBold="true" forecolor="#173F68"/>
```

XML

```

<style name="Cabecera" style="Sans_Normal" fontSize="9" isBold="true" forecolor="#173F6B"/>
<style name="Dato" style="Sans_Normal" fontSize="9"/>
<style name="UnidadesCondicional" style="Dato" isBold="true">
  <conditionalStyle>
    <conditionExpression><![CDATA[{$F{unidades_vendidas} != null && $P{umbralUnidades} != null &&
$F{unidades_vendidas}.intValue() >= $P{umbralUnidades}.intValue()]}></conditionExpression>
    <style forecolor="#1B5E20"/>
  </conditionalStyle>
  <conditionalStyle>
    <conditionExpression><![CDATA[{$F{unidades_vendidas} != null && $P{umbralUnidades} != null &&
$F{unidades_vendidas}.intValue() >= 3 && $F{unidades_vendidas}.intValue() < $P{umbralUnidades}.intValue()]}></conditionExpression>
    <style forecolor="#1D5D88"/>
  </conditionalStyle>
  <conditionalStyle>
    <conditionExpression><![CDATA[{$F{unidades_vendidas} == null || $F{unidades_vendidas}.intValue() < 3]}></conditionExpression>
    <style forecolor="#9D3429"/>
  </conditionalStyle>
</style>
<style name="M5TableHeader" style="Dato" mode="Opaque" backcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/>
<style name="M5TableDetail" style="Dato"/>
<style name="M5CrossHeader" style="Dato" mode="Opaque" backcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/>
<style name="M5CrossDetail" style="Dato" mode="Opaque" backcolor="#FFFFFF"/>
<style name="M5CrossTotal" style="Dato" mode="Opaque" backcolor="#D6EAF8" forecolor="#173F6B" isBold="true"/>
<subDataset name="DatasetTopVentas">
  <parameter name="tituloLibro" class="java.lang.String"/>
  <queryString language="sql">
    <![CDATA[
      SELECT fecha_venta, cantidad, precio_unitario
      FROM ventas
      WHERE titulo_libro = $P{tituloLibro}
      ORDER BY cantidad DESC, fecha_venta
      LIMIT 3
    ]]>
  </queryString>
  <field name="fecha_venta" class="java.lang.String"/>
  <field name="cantidad" class="java.lang.Integer"/>
  <field name="precio_unitario" class="java.lang.Double"/>
</subDataset>
<subDataset name="DatasetVentasPorCategoria">
  <queryString language="sql">
    <![CDATA[
      SELECT l.categoria AS categoria_grafico,
      COALESCE(SUM(v.cantidad * v.precio_unitario), 0.0) AS importe_categoria
      FROM libros l
      LEFT JOIN ventas v ON l.titulo = v.titulo_libro
      GROUP BY l.categoria
      ORDER BY l.categoria
    ]]>
  </queryString>
  <field name="categoria_grafico" class="java.lang.String"/>
  <field name="importe_categoria" class="java.lang.Double"/>
</subDataset>
<subDataset name="DatasetCrosstabVentas">
  <queryString language="sql">
    <![CDATA[
      SELECT l.categoria AS categoria_cross,
      SUBSTR(v.fecha_venta, 1, 4) AS anio_cross,
      (v.cantidad * v.precio_unitario) AS importe_cross,
      1 AS ventas_cross
      FROM ventas v
      JOIN libros l ON l.titulo = v.titulo_libro
      ORDER BY l.categoria, anio_cross, v.fecha_venta
    ]]>
  </queryString>
  <field name="categoria_cross" class="java.lang.String"/>
  <field name="anio_cross" class="java.lang.String"/>
  <field name="importe_cross" class="java.lang.Double"/>
  <field name="ventas_cross" class="java.lang.Integer"/>
</subDataset>
<parameter name="usuario" class="java.lang.String" isForPrompting="true"/>
<parameter name="fechaInforme" class="java.util.Date" isForPrompting="true">
  <defaultValueExpression><![CDATA[new java.util.Date()]]></defaultValueExpression>
</parameter>
<parameter name="departamento" class="java.lang.String" isForPrompting="true">
  <defaultValueExpression><![CDATA["General"]]></defaultValueExpression>
</parameter>
<parameter name="periodo" class="java.lang.String" isForPrompting="true">
  <defaultValueExpression><![CDATA["Mensual"]]></defaultValueExpression>
</parameter>
<parameter name="tipoIva" class="java.lang.Double" isForPrompting="true">
  <defaultValueExpression><![CDATA[Double.valueOf(0.21d)]]></defaultValueExpression>
</parameter>
<parameter name="mostrarDetalle" class="java.lang.Boolean" isForPrompting="true">
  <defaultValueExpression><![CDATA[Boolean.TRUE]]></defaultValueExpression>
</parameter>
<parameter name="categoria" class="java.lang.String" isForPrompting="true"/>
<parameter name="precioMinimo" class="java.lang.Double" isForPrompting="true"/>
<parameter name="precioMaximo" class="java.lang.Double" isForPrompting="true"/>
<parameter name="umbralUnidades" class="java.lang.Integer" isForPrompting="true">
  <defaultValueExpression><![CDATA[Integer.valueOf(5)]]></defaultValueExpression>
</parameter>
<parameter name="textoBusqueda" class="java.lang.String" isForPrompting="true"/>

```

```

<parameter name="categoriasLista" class="java.util.Collection" isForPrompting="false">
  <defaultValueExpression><![CDATA[ java.util.Arrays.asList("Novela", "Realismo mágico", "Cuento", "Poesía") ]]></
defaultValueExpression>
</parameter>
<queryString language="sql">
  <![CDATA[
    SELECT l.titulo,
           l.categoria,
           SUM(v.cantidad) AS unidades_vendidas,
           SUM(v.cantidad * v.precio_unitario) AS importe_total,
           AVG(v.precio_unitario) AS precio_medio,
           MIN(v.fecha_venta) AS primera_venta,
           MAX(v.fecha_venta) AS ultima_venta
    FROM libros l
    LEFT JOIN ventas v ON l.titulo = v.titulo_libro
    WHERE ($P{categoria}) IS NULL OR l.categoria = $P{categoria})
      AND ($P{precioMinimo}) IS NULL OR l.precio >= $P{precioMinimo})
      AND ($P{precioMaximo}) IS NULL OR l.precio <= $P{precioMaximo})
      AND ($P{textoBusqueda}) IS NULL OR $P{textoBusqueda} = '' OR l.titulo LIKE '%' || $P{textoBusqueda} || '%' )
      AND $X{IN, l.categoria, categoriasLista}
    GROUP BY l.titulo, l.categoria
    ORDER BY l.categoria, COALESCE(importe_total, 0) DESC, l.titulo
  ]]>
</queryString>
<field name="titulo" class="java.lang.String"/>
<field name="categoria" class="java.lang.String"/>
<field name="unidades_vendidas" class="java.lang.Integer"/>
<field name="importe_total" class="java.lang.Double"/>
<field name="precio_medio" class="java.lang.Double"/>
<field name="primera_venta" class="java.lang.String"/>
<field name="ultima_venta" class="java.lang.String"/>
<variable name="TotalUnidades" class="java.lang.Integer" calculation="Sum" resetType="Report">
  <variableExpression><![CDATA[$F{unidades_vendidas}]]></variableExpression>
</variable>
<variable name="TotalImporte" class="java.lang.Double" calculation="Sum" resetType="Report">
  <variableExpression><![CDATA[$F{importe_total}]]></variableExpression>
</variable>
<variable name="TotalPagina" class="java.lang.Double" calculation="Sum" resetType="Page">
  <variableExpression><![CDATA[$F{importe_total}]]></variableExpression>
</variable>
<variable name="PrecioMedio" class="java.lang.Double" calculation="Average" resetType="Report">
  <variableExpression><![CDATA[$F{precio_medio}]]></variableExpression>
</variable>
<variable name="PrecioMaximo" class="java.lang.Double" calculation="Highest" resetType="Report">
  <variableExpression><![CDATA[$F{precio_medio}]]></variableExpression>
</variable>
<variable name="NumeroLibros" class="java.lang.Integer" calculation="Count" resetType="Report">
  <variableExpression><![CDATA[$F{titulo}]]></variableExpression>
</variable>
<variable name="ImporteConIva" class="java.lang.Double" calculation="Sum" resetType="Report">
  <variableExpression><![CDATA[$F{importe_total} == null || $P{tipoIva} == null ? null :
Double.valueOf($F{importe_total}.doubleValue() * (1.0d + $P{tipoIva}.doubleValue()))]]></variableExpression>
</variable>
<variable name="GrupoUnidades" class="java.lang.Integer" calculation="Sum" resetType="Group" resetGroup="CategoriaGroup">
  <variableExpression><![CDATA[$F{unidades_vendidas}]]></variableExpression>
</variable>
<variable name="GrupoImporte" class="java.lang.Double" calculation="Sum" resetType="Group" resetGroup="CategoriaGroup">
  <variableExpression><![CDATA[$F{importe_total}]]></variableExpression>
</variable>
<variable name="GrupoLibros" class="java.lang.Integer" calculation="Count" resetType="Group" resetGroup="CategoriaGroup">
  <variableExpression><![CDATA[$F{titulo}]]></variableExpression>
</variable>
<group name="CategoriaGroup" isStartNewPage="false" isReprintHeaderOnEachPage="true" minHeightToStartNewPage="80">
  <groupExpression><![CDATA[$F{categoria}]]></groupExpression>
  <groupHeader>
    <band height="28">
      <textField><reportElement x="0" y="2" width="555" height="22" style="M5GrupoCabecera"/><textFieldExpression><![
CDATA["Categoría: " + $F{categoria}]]></textFieldExpression></textField>
    </band>
  </groupHeader>
  <groupFooter>
    <band height="34">
      <textField><reportElement x="0" y="3" width="185" height="18" style="Dato"/><textFieldExpression><![CDATA["Libros del
grupo: " + $V{GrupoLibros}]]></textFieldExpression></textField>
      <textField><reportElement x="185" y="3" width="180" height="18" style="Dato"/><textFieldExpression><![CDATA["Unidades: "
+ ($V{GrupoUnidades} == null ? 0 : $V{GrupoUnidades}]]></textFieldExpression></textField>
      <textField><reportElement x="365" y="3" width="190" height="18" style="Dato"/><textFieldExpression><![CDATA["Importe: "
+ ($V{GrupoImporte} == null ? 0 : $V{GrupoImporte}]]></textFieldExpression></textField>
    </band>
  </groupFooter>
</group>
<background><band height="0"/></background>
<title>
  <band height="124">
    <staticText>
      <reportElement x="0" y="4" width="555" height="28" uuid="40000000-0000-4000-8000-000000000001"
style="M5TituloPrincipal"/>
      <textField><reportElement x="0" y="4" width="555" height="28" style="M5TituloPrincipal"/>
      <text><![CDATA[Informe de Ventas - Agregación por Título]]></text>
    </staticText>
    <staticText><reportElement x="0" y="38" width="110" height="18" uuid="40000000-0000-4000-8000-000000000002"/><text><![

```

```

[CDATA[Generado por:]]></text></staticText>
<textField isBlankWhenNull="true"><reportElement x="110" y="38" width="160" height="18"
uuid="40000000-0000-4000-8000-000000000003"/><textFieldExpression><![CDATA[{$usuario}]]></textFieldExpression></textField>
<staticText><reportElement x="300" y="38" width="80" height="18" uuid="40000000-0000-4000-8000-000000000004"/><text><![
CDATA[Fecha:]]></text></staticText>
<textField pattern="dd/MM/yyyy"><reportElement x="380" y="38" width="175" height="18"
uuid="40000000-0000-4000-8000-000000000005"/><textFieldExpression><![CDATA[{$fechaInforme}]]></textFieldExpression></textField>
<staticText><reportElement x="0" y="62" width="100" height="18" uuid="40000000-0000-4000-8000-000000000006"/><text><![
CDATA[Departamento:]]></text></staticText>
<textField isBlankWhenNull="true"><reportElement x="100" y="62" width="170" height="18"
uuid="40000000-0000-4000-8000-000000000007"/><textFieldExpression><![CDATA[{$departamento}]]></textFieldExpression></textField>
<staticText><reportElement x="300" y="62" width="70" height="18" uuid="40000000-0000-4000-8000-000000000008"/><text><![
CDATA[Periodo:]]></text></staticText>
<textField isBlankWhenNull="true"><reportElement x="370" y="62" width="185" height="18"
uuid="40000000-0000-4000-8000-000000000009"/><textFieldExpression><![CDATA[{$periodo}]]></textFieldExpression></textField>
<staticText><reportElement x="0" y="86" width="100" height="18" uuid="40000000-0000-4000-8000-000000000010"/><text><![
CDATA[Búsqueda:]]></text></staticText>
<textField isBlankWhenNull="true"><reportElement x="100" y="86" width="170" height="18"
uuid="40000000-0000-4000-8000-000000000011"/><textFieldExpression><![CDATA[{$textoBusqueda} == null ||
{$textoBusqueda}.trim().isEmpty() ? "(todas)" : {$textoBusqueda}]]></textFieldExpression></textField>
<staticText><reportElement x="300" y="86" width="90" height="18" uuid="40000000-0000-4000-8000-000000000012"/><text><![
CDATA[ Categorías:]]></text></staticText>
<textField textAdjust="StretchHeight"><reportElement x="390" y="86" width="165" height="34"
uuid="40000000-0000-4000-8000-000000000013"/><textFieldExpression><![CDATA[String.valueOf( {$categoriasLista} )]]></
textFieldExpression></textField>
</band>
</title>
<columnHeader>
<band height="62">
<staticText><reportElement x="0" y="2" width="215" height="18" uuid="41000000-0000-4000-8000-000000000001" style="Cabecera"/
><text><![CDATA[Título]]></text></staticText>
<staticText><reportElement x="215" y="2" width="55" height="18" uuid="41000000-0000-4000-8000-000000000002"
style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[Unid.]]></text></staticText>
<staticText><reportElement x="280" y="2" width="90" height="18" uuid="41000000-0000-4000-8000-000000000003"
style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[Importe]]></text></staticText>
<staticText><reportElement x="380" y="2" width="65" height="18" uuid="41000000-0000-4000-8000-000000000004"
style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[Precio med.]]></text></staticText>
<staticText><reportElement x="455" y="2" width="100" height="18" uuid="41000000-0000-4000-8000-000000000005"
style="Cabecera"/><text><![CDATA[Categoría]]></text></staticText>
<staticText><reportElement x="0" y="24" width="130" height="18" uuid="41000000-0000-4000-8000-000000000006"
style="Cabecera"/><text><![CDATA[Primera venta]]></text></staticText>
<staticText><reportElement x="130" y="24" width="130" height="18" uuid="41000000-0000-4000-8000-000000000007"
style="Cabecera"/><text><![CDATA[Última venta]]></text></staticText>
<staticText><reportElement x="260" y="24" width="160" height="18" uuid="41000000-0000-4000-8000-000000000008"
style="Cabecera"/><text><![CDATA[Periodo de ventas]]></text></staticText>
<staticText>
<reportElement x="420" y="24" width="135" height="18" uuid="41000000-0000-4000-8000-000000000009" style="Cabecera">
<printWhenExpression><![CDATA[Boolean.TRUE.equals( {$mostrarDetalle} )]]></printWhenExpression>
</reportElement>
<textElement textAlignment="Right"/>
<text><![CDATA[Importe con IVA]]></text>
</staticText>
</band>
</columnHeader>
<detail>
<band height="82" splitType="Stretch">
<textField textAdjust="StretchHeight"><reportElement x="0" y="0" width="215" height="20"
uuid="42000000-0000-4000-8000-000000000001" style="Dato"/><textFieldExpression><![CDATA[{$titulo}]]></textFieldExpression></textField>
<textField isBlankWhenNull="true"><reportElement x="215" y="0" width="55" height="20"
uuid="42000000-0000-4000-8000-000000000002" style="UnidadesCondiciona1"/><textElement textAlignment="Right"/><textFieldExpression><![
CDATA[{$unidades_vendidas}]]></textFieldExpression></textField>
<textField pattern="#,##0.00 €" isBlankWhenNull="true"><reportElement x="280" y="0" width="90" height="20"
uuid="42000000-0000-4000-8000-000000000003" style="Dato"/><textElement textAlignment="Right"/><textFieldExpression><![
CDATA[{$importe_total}]]></textFieldExpression></textField>
<textField isBlankWhenNull="true"><reportElement x="380" y="0" width="65" height="20"
uuid="42000000-0000-4000-8000-000000000004" style="Dato"/><textElement textAlignment="Right"/><textFieldExpression><![
CDATA[{$precio_medio} == null ? "Sin datos" : new java.text.DecimalFormat("#0.00 '€").format( {$precio_medio} )]]></
textFieldExpression></textField>
<textField isBlankWhenNull="true"><reportElement x="455" y="0" width="100" height="20"
uuid="42000000-0000-4000-8000-000000000005" style="Dato"/><textFieldExpression><![CDATA[{$categoria}]]></textFieldExpression></
textField>
<textField isBlankWhenNull="true"><reportElement x="0" y="24" width="130" height="18"
uuid="42000000-0000-4000-8000-000000000006" style="Dato"/><textFieldExpression><![CDATA[{$primera_venta}]]></textFieldExpression></
textField>
<textField isBlankWhenNull="true"><reportElement x="130" y="24" width="130" height="18"
uuid="42000000-0000-4000-8000-000000000007" style="Dato"/><textFieldExpression><![CDATA[{$ultima_venta}]]></textFieldExpression></
textField>
<textField><reportElement x="260" y="24" width="160" height="18" uuid="42000000-0000-4000-8000-000000000008" style="Dato"/
><textElement textAlignment="Center"/><textFieldExpression><![CDATA[{$primera_venta} == null ? "Sin ventas" : {$primera_venta} + " - "
+ {$ultima_venta}]]></textFieldExpression></textField>
<textField pattern="#,##0.00 €" isBlankWhenNull="true">
<reportElement x="420" y="24" width="135" height="18" uuid="42000000-0000-4000-8000-000000000009" style="Dato">
<printWhenExpression><![CDATA[Boolean.TRUE.equals( {$mostrarDetalle} )]]></printWhenExpression>
</reportElement>
<textElement textAlignment="Right"/>
<textFieldExpression><![CDATA[{$importe_total} == null || {$tipoIva} == null ? null :
Double.valueOf( {$importe_total}.doubleValue() * (1.0d + {$tipoIva}.doubleValue() ) )]]></textFieldExpression>
</textField>
<textField><reportElement x="0" y="48" width="105" height="18" uuid="42000000-0000-4000-8000-000000000010" style="Dato"/
><textFieldExpression><![CDATA[{$unidades_vendidas} == null ? "Sin ventas" : ( {$unidades_vendidas}.intValue() >= 6 ? "Premium" :
( {$unidades_vendidas}.intValue() >= 3 ? "Estándar" : "Económico" )]]></textFieldExpression></textField>
<textField><reportElement x="105" y="48" width="185" height="18" uuid="42000000-0000-4000-8000-000000000011" style="Dato"/

```



```

</pageFooter>
<summary>
  <band height="700">
    <staticText><reportElement x="0" y="5" width="205" height="18" uuid="44000000-0000-4000-8000-000000000001"/><text><![
[CDATA[Total de unidades vendidas:]]></text></staticText>
    <textField><reportElement x="205" y="5" width="80" height="18" uuid="44000000-0000-4000-8000-000000000002"/><textElement
textAlignment="Right"/><textFieldExpression><![CDATA[${V{TotalUnidades}}]></textFieldExpression></textField>
    <staticText><reportElement x="300" y="5" width="120" height="18" uuid="44000000-0000-4000-8000-000000000003"/><text><![
[CDATA[Importe total:]]></text></staticText>
    <textField pattern="#,##0.00 €"><reportElement x="420" y="5" width="135" height="18"
uuid="44000000-0000-4000-8000-000000000004"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[${V{TotalImporte}}]></
textFieldExpression></textField>
    <staticText><reportElement x="0" y="30" width="205" height="18" uuid="44000000-0000-4000-8000-000000000005"/><text><![
[CDATA[Precio medio agregado:]]></text></staticText>
    <textField pattern="#,##0.00 €"><reportElement x="205" y="30" width="80" height="18"
uuid="44000000-0000-4000-8000-000000000006"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[${V{PrecioMedio}}]></
textFieldExpression></textField>
    <staticText><reportElement x="300" y="30" width="120" height="18" uuid="44000000-0000-4000-8000-000000000007"/><text><![
[CDATA[Precio máximo:]]></text></staticText>
    <textField pattern="#,##0.00 €"><reportElement x="420" y="30" width="135" height="18"
uuid="44000000-0000-4000-8000-000000000008"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[${V{PrecioMaximo}}]></
textFieldExpression></textField>
    <staticText><reportElement x="0" y="55" width="205" height="18" uuid="44000000-0000-4000-8000-000000000009"/><text><![
[CDATA[Número de libros:]]></text></staticText>
    <textField><reportElement x="205" y="55" width="80" height="18" uuid="44000000-0000-4000-8000-000000000010"/><textElement
textAlignment="Right"/><textFieldExpression><![CDATA[${V{NumeroLibros}}]></textFieldExpression></textField>
    <staticText><reportElement x="300" y="55" width="120" height="18" uuid="44000000-0000-4000-8000-000000000011"/><text><![
[CDATA[Importe con IVA:]]></text></staticText>
    <textField pattern="#,##0.00 €"><reportElement x="420" y="55" width="135" height="18"
uuid="44000000-0000-4000-8000-000000000012"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[${V{ImporteConIva}}]></
textFieldExpression></textField>
    <textField><reportElement x="0" y="80" width="555" height="18" uuid="44000000-0000-4000-8000-000000000013"/><textElement
textAlignment="Center"/><textFieldExpression><![CDATA[String.format(java.util.Locale.ROOT, "Resumen: %d títulos · %d unidades · %.2f €",
${V{NumeroLibros}}, ${V{TotalUnidades}}, ${V{TotalImporte}})]></textFieldExpression></textField>
    <textField><reportElement x="0" y="103" width="350" height="18" uuid="44000000-0000-4000-8000-000000000014"/><textElement
textAlignment="Center"/><font fontName="DejaVu Sans" size="10" isBold="true"/></textElement><textFieldExpression><![
[CDATA[${V{TotalUnidades}} != null && ${V{umbralUnidades}} != null && ${V{TotalUnidades}}.intValue() >= ${V{umbralUnidades}}.intValue() ?
"Objetivo de ventas alcanzado" : "Objetivo de ventas pendiente"]></textFieldExpression></textField>
    <textField><reportElement x="360" y="103" width="195" height="18" uuid="44000000-0000-4000-8000-000000000015"/>
<textFieldExpression><![CDATA["Resultados encontrados: " + ${V{REPORT_COUNT}}]]></textFieldExpression></textField>
    <staticText><reportElement x="0" y="140" width="555" height="20" style="Cabecera"/><text><![CDATA[Ventas por categoría –
importe]]></text></staticText>
    <barChart>
      <chart>
        <reportElement x="0" y="165" width="555" height="250"/>
        <chartTitle><titleExpression><![CDATA["Ventas por categoría"]></titleExpression></chartTitle>
        <chartSubtitle/>
        <chartLegend position="Bottom"/>
      </chart>
      <categoryDataset>
        <dataset>
          <datasetRun subDataset="DatasetVentasPorCategoría">
            <connectionExpression><![CDATA[${P{REPORT_CONNECTION}}]></connectionExpression>
          </datasetRun>
        </dataset>
        <categorySeries>
          <seriesExpression><![CDATA["Importe"]></seriesExpression>
          <categoryExpression><![CDATA[${F{categoria_grafico}}]></categoryExpression>
          <valueExpression><![CDATA[${F{importe_categoria}}]></valueExpression>
        </categorySeries>
      </categoryDataset>
      <barPlot>
        <plot/>
        <itemLabel/>
        <categoryAxisFormat><axisFormat/></categoryAxisFormat>
        <valueAxisFormat><axisFormat/></valueAxisFormat>
      </barPlot>
    </barChart>
    <staticText><reportElement x="0" y="430" width="555" height="20" style="Cabecera"/><text><![CDATA[Ventas por categoría y
año]]></text></staticText>
    <crosstab>
      <reportElement x="0" y="455" width="555" height="225"/>
      <crosstabDataset>
        <dataset>
          <datasetRun subDataset="DatasetCrosstabVentas">
            <connectionExpression><![CDATA[${P{REPORT_CONNECTION}}]></connectionExpression>
          </datasetRun>
        </dataset>
      </crosstabDataset>
      <rowGroup name="CategoríaCross" width="150" totalPosition="End">
        <bucket class="java.lang.String"><bucketExpression><![CDATA[${F{categoria_cross}}]></bucketExpression></bucket>
        <crosstabRowHeader><cellContents style="M5CrosstabCabecera"><textField><reportElement x="0" y="0" width="150"
height="34"/><textElement verticalAlignment="Middle"/><textFieldExpression><![CDATA[${V{CategoríaCross}}]></textFieldExpression></
textField></cellContents></crosstabRowHeader>
        <crosstabTotalRowHeader><cellContents style="M5CrosstabTotal"><staticText><reportElement x="0" y="0" width="150"
height="34"/><textElement verticalAlignment="Middle"/><text><![CDATA[TOTAL]]></text></staticText></cellContents></
crosstabTotalRowHeader>
      </rowGroup>
      <columnGroup name="AñoCross" height="28" totalPosition="End">
        <bucket class="java.lang.String"><bucketExpression><![CDATA[${F{anio_cross}}]></bucketExpression></bucket>
        <crosstabColumnHeader><cellContents style="M5CrosstabCabecera"><textField><reportElement x="0" y="0" width="100"
height="28"/><textElement textAlignment="Center" verticalAlignment="Middle"/><textFieldExpression><![CDATA[${V{AñoCross}}]></

```

```

textFieldExpression></textField></cellContents></crosstabColumnHeader>
    <crosstabTotalColumnHeader><cellContents style="M5CrosstabTotal"><staticText><reportElement x="0" y="0" width="100"
height="28"/><textElement textAlignment="Center" verticalAlignment="Middle"/><text><![CDATA[TOTAL]]></text></staticText></
cellContents></crosstabTotalColumnHeader>
    </columnGroup>
    <measure name="ImporteCross" class="java.lang.Double" calculation="Sum"><measureExpression><![
CDATA[$F{importe_cross}]]></measureExpression></measure>
    <measure name="VentasCross" class="java.lang.Integer" calculation="Sum"><measureExpression><![
CDATA[$F{ventas_cross}]]></measureExpression></measure>
    <crosstabCell width="100" height="34"><cellContents style="M5CrosstabDetalle"><textField pattern="#,##0.00
€"><reportElement x="0" y="0" width="100" height="18"/><textElement textAlignment="Right"/><textFieldExpression><![
CDATA[$V{ImporteCross}]]></textFieldExpression></textField><textField><reportElement x="0" y="18" width="100" height="14"/><textElement
textAlignment="Right"/><textFieldExpression><![CDATA[$V{VentasCross} + " ventas"]></textFieldExpression></textField></cellContents></
crosstabCell>
    <crosstabCell width="100" height="34" rowTotalGroup="CategoriaCross"><cellContents style="M5CrosstabTotal"><textField
pattern="#,##0.00 €"><reportElement x="0" y="0" width="100" height="18"/><textElement textAlignment="Right"/><textFieldExpression><![
CDATA[$V{ImporteCross}]]></textFieldExpression></textField><textField><reportElement x="0" y="18" width="100" height="14"/><textElement
textAlignment="Right"/><textFieldExpression><![CDATA[$V{VentasCross} + " ventas"]></textFieldExpression></textField></cellContents></
crosstabCell>
    <crosstabCell width="100" height="34" columnTotalGroup="AnioCross"><cellContents style="M5CrosstabTotal"><textField
pattern="#,##0.00 €"><reportElement x="0" y="0" width="100" height="18"/><textElement textAlignment="Right"/><textFieldExpression><![
CDATA[$V{ImporteCross}]]></textFieldExpression></textField><textField><reportElement x="0" y="18" width="100" height="14"/><textElement
textAlignment="Right"/><textFieldExpression><![CDATA[$V{VentasCross} + " ventas"]></textFieldExpression></textField></cellContents></
crosstabCell>
    <crosstabCell width="100" height="34" rowTotalGroup="CategoriaCross" columnTotalGroup="AnioCross"><cellContents
style="M5CrosstabTotal"><textField pattern="#,##0.00 €"><reportElement x="0" y="0" width="100" height="18"/><textElement
textAlignment="Right"/><textFieldExpression><![CDATA[$V{ImporteCross}]]></textFieldExpression></textField><textField><reportElement
x="0" y="18" width="100" height="14"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[$V{VentasCross} + " ventas"]></
textFieldExpression></textField></cellContents></crosstabCell>
    </crosstab>
  </band>
</summary>
</jasperReport>

```

Explicación línea por línea

Línea 1	<?xml version="1.0" encoding="UTF-8"?> → Declara XML 1.0 y codificación UTF-8 para que nombres, textos y símbolos del informe se interpreten correctamente.
Línea 2	<jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports" → Abre el documento raíz jasperReport del informe y fija el namespace principal de JasperReports.
Línea 3	xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" → Declara el namespace XML Schema Instance usado por xsi:schemaLocation para validar el documento.
Línea 4	xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/jasperreport.xsd" → Relaciona el namespace de JasperReports con su XSD para que Studio y el compilador validen la estructura.
Línea 5	name="informe_ventas" → Asigna al documento JasperReports el nombre interno informe_ventas.
Línea 6	language="java" → Configura language=java para evaluar expresiones con el lenguaje Java.
Línea 7	pageWidth="595" → Fija el ancho físico de página en 595 puntos.
Línea 8	pageHeight="842" → Fija la altura física de página en 842 puntos.
Línea 9	columnWidth="555" → Fija el ancho útil de la columna de contenido en 555 puntos.
Línea 10	leftMargin="20" → Fija el margen izquierdo del informe en 20 puntos.
Línea 11	rightMargin="20" → Fija el margen derecho del informe en 20 puntos.
Línea 12	topMargin="20" → Fija el margen superior del informe en 20 puntos.
Línea 13	bottomMargin="20" → Fija el margen inferior del informe en 20 puntos y completa la apertura del elemento raíz.
Línea 14	uuid="3d2c2bd7-3b93-4da9-8b60-6b3c45674c91"> → Asigna el UUID de diseño 3d2c2bd7-3b93-4da9-8b60-6b3c45674c91 para identificar de forma estable el informe en Studio.
Línea 15	<property name="com.jaspersoft.studio.data.defaultdataadapter" value="SQLiteEditorial"/> → Indica a Jaspersoft Studio que use el Data Adapter SQLiteEditorial como conexión de diseño por defecto.
Línea 16	<template><![CDATA["resources/styles/EditorialStyles.jrtx"]]></template> → Importa la plantilla externa cuya expresión CDATA devuelve resources/styles/EditorialStyles.jrtx.

Línea 17	<code><style name="Sans_Normal" isDefault="true" fontName="DejaVu Sans" fontSize="10"/></code> → Declara el estilo Sans_Normal; es el estilo por defecto, fuente DejaVu Sans, tamaño 10.
Línea 18	<code><style name="TituloPrincipal" style="Sans_Normal" fontSize="18" isBold="true" forecolor="#173F6B"/></code> → Declara el estilo TituloPrincipal; hereda de Sans_Normal, tamaño 18.
Línea 19	<code><style name="Cabecera" style="Sans_Normal" fontSize="9" isBold="true" forecolor="#173F6B"/></code> → Declara el estilo Cabecera; hereda de Sans_Normal, tamaño 9.
Línea 20	<code><style name="Dato" style="Sans_Normal" fontSize="9"/></code> → Declara el estilo Dato; hereda de Sans_Normal, tamaño 9.
Línea 21	<code><style name="UnidadesCondicional" style="Dato" isBold="true"></code> → Declara el estilo UnidadesCondicional; hereda de Dato.
Línea 22	<code><conditionalStyle></code> → Abre una variante condicional del estilo; sólo se aplicará cuando su conditionExpression sea verdadera.
Línea 23	<code><conditionExpression><![CDATA[{\$F{unidades_vendidas} != null && \$P{umbralUnidades} != null && \$F{unidades_vendidas}.intValue() >= \$P{umbralUnidades}.intValue()}]]></conditionExpre...</code> → Define la condición booleana que activa el estilo condicional usando field unidades_vendidas, field unidades_vendidas, parámetro umbralUnidades, parámetro umbralUnidades.
Línea 24	<code><style forecolor="#1B5E20"/></code> → Declara el estilo None.
Línea 25	<code></conditionalStyle></code> → Cierra conditionalStyle y vuelve al elemento padre de la jerarquía JRXML.
Línea 26	<code><conditionalStyle></code> → Abre una variante condicional del estilo; sólo se aplicará cuando su conditionExpression sea verdadera.
Línea 27	<code><conditionExpression><![CDATA[{\$F{unidades_vendidas} != null && \$P{umbralUnidades} != null && \$F{unidades_vendidas}.intValue() >= 3 && \$F{unidades_vendidas}.intValue() < \$P{umbra...</code> → Define la condición booleana que activa el estilo condicional usando field unidades_vendidas, field unidades_vendidas, field unidades_vendidas, parámetro umbralUnidades, parámetro umbralUnidades.
Línea 28	<code><style forecolor="#1D5D88"/></code> → Declara el estilo None.
Línea 29	<code></conditionalStyle></code> → Cierra conditionalStyle y vuelve al elemento padre de la jerarquía JRXML.
Línea 30	<code><conditionalStyle></code> → Abre una variante condicional del estilo; sólo se aplicará cuando su conditionExpression sea verdadera.
Línea 31	<code><conditionExpression><![CDATA[{\$F{unidades_vendidas} == null \$F{unidades_vendidas}.intValue() < 3}]]></conditionExpression></code> → Define la condición booleana que activa el estilo condicional usando field unidades_vendidas, field unidades_vendidas.
Línea 32	<code><style forecolor="#9D3429"/></code> → Declara el estilo None.
Línea 33	<code></conditionalStyle></code> → Cierra conditionalStyle y vuelve al elemento padre de la jerarquía JRXML.
Línea 34	<code></style></code> → Cierra style y vuelve al elemento padre de la jerarquía JRXML.
Línea 35	<code><style name="M5TableHeader" style="Dato" mode="Opaque" backcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/></code> → Declara el estilo M5TableHeader; hereda de Dato, fondo #EAF2F8.
Línea 36	<code><style name="M5TableDetail" style="Dato"/></code> → Declara el estilo M5TableDetail; hereda de Dato.
Línea 37	<code><style name="M5CrossHeader" style="Dato" mode="Opaque" backcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/></code> → Declara el estilo M5CrossHeader; hereda de Dato, fondo #EAF2F8.
Línea 38	<code><style name="M5CrossDetail" style="Dato" mode="Opaque" backcolor="FFFFFF"/></code> → Declara el estilo M5CrossDetail; hereda de Dato, fondo #FFFFFF.
Línea 39	<code><style name="M5CrossTotal" style="Dato" mode="Opaque" backcolor="#D6EAF8" forecolor="#173F6B" isBold="true"/></code> → Declara el estilo M5CrossTotal; hereda de Dato, fondo #D6EAF8.
Línea 40	<code><subDataset name="DatasetTopVentas"></code> → Declara el subdataset DatasetTopVentas, con consulta y fields propios independientes del dataset principal.

Línea 41	<code><parameter name="tituloLibro" class="java.lang.String"/></code> → Declara el parámetro <code>tituloLibro</code> con tipo <code>java.lang.String</code> y lo expone al diálogo de parámetros de Studio.
Línea 42	<code><queryString language="sql"></code> → Abre la consulta SQL que JasperReports ejecutará para el dataset actual.
Línea 43	<code><![CDATA[</code> → Abre CDATA para escribir SQL o una expresión Java sin que sus caracteres especiales se interpreten como XML.
Línea 44	<code>SELECT fecha_venta, cantidad, precio_unitario</code> → Cláusula SQL SELECT: selecciona y calcula las columnas que devolverá la consulta.
Línea 45	<code>FROM ventas</code> → Cláusula SQL FROM: define la tabla base de la consulta.
Línea 46	<code>WHERE titulo_libro = \${tituloLibro}</code> → Cláusula SQL WHERE: aplica el filtro de filas.
Línea 47	<code>ORDER BY cantidad DESC, fecha_venta</code> → Cláusula SQL ORDER BY: ordena el resultado que recibirá JasperReports.
Línea 48	<code>LIMIT 3</code> → Cláusula SQL LIMIT: limita el número de filas devueltas.
Línea 49	<code>]]></code> → Cierra el bloque CDATA y devuelve el control al parser XML.
Línea 50	<code></queryString></code> → Cierra la consulta SQL del dataset actual.
Línea 51	<code><field name="fecha_venta" class="java.lang.String"/></code> → Declara el field <code>fecha_venta</code> con tipo Java <code>java.lang.String</code> para mapear una columna del dataset.
Línea 52	<code><field name="cantidad" class="java.lang.Integer"/></code> → Declara el field <code>cantidad</code> con tipo Java <code>java.lang.Integer</code> para mapear una columna del dataset.
Línea 53	<code><field name="precio_unitario" class="java.lang.Double"/></code> → Declara el field <code>precio_unitario</code> con tipo Java <code>java.lang.Double</code> para mapear una columna del dataset.
Línea 54	<code></subDataset></code> → Finaliza el subdataset auxiliar y vuelve al nivel del informe.
Línea 55	<code><subDataset name="DatasetVentasPorCategoria"></code> → Declara el subdataset <code>DatasetVentasPorCategoria</code> , con consulta y fields propios independientes del dataset principal.
Línea 56	<code><queryString language="sql"></code> → Abre la consulta SQL que JasperReports ejecutará para el dataset actual.
Línea 57	<code><![CDATA[</code> → Abre CDATA para escribir SQL o una expresión Java sin que sus caracteres especiales se interpreten como XML.
Línea 58	<code>SELECT l.categoria AS categoria_grafico,</code> → Cláusula SQL SELECT: selecciona y calcula las columnas que devolverá la consulta.
Línea 59	<code>COALESCE(SUM(v.cantidad * v.precio_unitario), 0.0) AS importe_categoria</code> → Calcula o selecciona un valor SQL y lo expone con el alias <code>importe_categoria</code> , que después coincide con un field del subdataset.
Línea 60	<code>FROM libros l</code> → Cláusula SQL FROM: define la tabla base de la consulta.
Línea 61	<code>LEFT JOIN ventas v ON l.titulo = v.titulo_libro</code> → Cláusula SQL LEFT JOIN: une datos conservando las filas del lado izquierdo aunque no tengan ventas.
Línea 62	<code>GROUP BY l.categoria</code> → Cláusula SQL GROUP BY: agrupa las filas antes de evaluar las funciones agregadas.
Línea 63	<code>ORDER BY l.categoria</code> → Cláusula SQL ORDER BY: ordena el resultado que recibirá JasperReports.
Línea 64	<code>]]></code> → Cierra el bloque CDATA y devuelve el control al parser XML.
Línea 65	<code></queryString></code> → Cierra la consulta SQL del dataset actual.
Línea 66	<code><field name="categoria_grafico" class="java.lang.String"/></code> → Declara el field <code>categoria_grafico</code> con tipo Java <code>java.lang.String</code> para mapear una columna del dataset.
Línea 67	<code><field name="importe_categoria" class="java.lang.Double"/></code> → Declara el field <code>importe_categoria</code> con tipo Java <code>java.lang.Double</code> para mapear una columna del dataset.
Línea 68	<code></subDataset></code> → Finaliza el subdataset auxiliar y vuelve al nivel del informe.

Línea 69	<code><subDataset name="DatasetCrosstabVentas"></code> → Declara el subdataset DatasetCrosstabVentas, con consulta y fields propios independientes del dataset principal.
Línea 70	<code><queryString language="sql"></code> → Abre la consulta SQL que JasperReports ejecutará para el dataset actual.
Línea 71	<code><![CDATA[</code> → Abre CDATA para escribir SQL o una expresión Java sin que sus caracteres especiales se interpreten como XML.
Línea 72	<code>SELECT l.categoria AS categoria_cross,</code> → Cláusula SQL SELECT: selecciona y calcula las columnas que devolverá la consulta.
Línea 73	<code>SUBSTR(v.fecha_venta, 1, 4) AS anio_cross,</code> → Calcula o selecciona un valor SQL y lo expone con el alias anio_cross, que después coincide con un field del subdataset.
Línea 74	<code>(v.cantidad * v.precio_unitario) AS importe_cross,</code> → Calcula o selecciona un valor SQL y lo expone con el alias importe_cross, que después coincide con un field del subdataset.
Línea 75	<code>1 AS ventas_cross</code> → Calcula o selecciona un valor SQL y lo expone con el alias ventas_cross, que después coincide con un field del subdataset.
Línea 76	<code>FROM ventas v</code> → Cláusula SQL FROM: define la tabla base de la consulta.
Línea 77	<code>JOIN libros l ON l.titulo = v.titulo_libro</code> → Cláusula SQL JOIN: une las filas que cumplen la relación indicada.
Línea 78	<code>ORDER BY l.categoria, anio_cross, v.fecha_venta</code> → Cláusula SQL ORDER BY: ordena el resultado que recibirá JasperReports.
Línea 79	<code>]]></code> → Cierra el bloque CDATA y devuelve el control al parser XML.
Línea 80	<code></queryString></code> → Cierra la consulta SQL del dataset actual.
Línea 81	<code><field name="categoria_cross" class="java.lang.String"/></code> → Declara el field categoria_cross con tipo Java java.lang.String para mapear una columna del dataset.
Línea 82	<code><field name="anio_cross" class="java.lang.String"/></code> → Declara el field anio_cross con tipo Java java.lang.String para mapear una columna del dataset.
Línea 83	<code><field name="importe_cross" class="java.lang.Double"/></code> → Declara el field importe_cross con tipo Java java.lang.Double para mapear una columna del dataset.
Línea 84	<code><field name="ventas_cross" class="java.lang.Integer"/></code> → Declara el field ventas_cross con tipo Java java.lang.Integer para mapear una columna del dataset.
Línea 85	<code></subDataset></code> → Finaliza el subdataset auxiliar y vuelve al nivel del informe.
Línea 86	<code><parameter name="usuario" class="java.lang.String" isForPrompting="true"/></code> → Declara el parámetro usuario con tipo java.lang.String y lo expone al diálogo de parámetros de Studio.
Línea 87	<code><parameter name="fechaInforme" class="java.util.Date" isForPrompting="true"></code> → Declara el parámetro fechaInforme con tipo java.util.Date y lo expone al diálogo de parámetros de Studio.
Línea 88	<code><defaultValueExpression><![CDATA[new java.util.Date()]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 89	<code></parameter></code> → Cierra parameter y vuelve al elemento padre de la jerarquía JRXML.
Línea 90	<code><parameter name="departamento" class="java.lang.String" isForPrompting="true"></code> → Declara el parámetro departamento con tipo java.lang.String y lo expone al diálogo de parámetros de Studio.
Línea 91	<code><defaultValueExpression><![CDATA["General"]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 92	<code></parameter></code> → Cierra parameter y vuelve al elemento padre de la jerarquía JRXML.
Línea 93	<code><parameter name="periodo" class="java.lang.String" isForPrompting="true"></code> → Declara el parámetro periodo con tipo java.lang.String y lo expone al diálogo de parámetros de Studio.
Línea 94	<code><defaultValueExpression><![CDATA["Mensual"]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.

Línea 95	<code></parameter></code> → Cierra <code>parameter</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 96	<code><parameter name="tipoIva" class="java.lang.Double" isForPrompting="true"></code> → Declara el parámetro <code>tipoIva</code> con tipo <code>java.lang.Double</code> y lo expone al diálogo de parámetros de Studio.
Línea 97	<code><defaultValueExpression><![CDATA[Double.valueOf(0.21d)]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 98	<code></parameter></code> → Cierra <code>parameter</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 99	<code><parameter name="mostrarDetalle" class="java.lang.Boolean" isForPrompting="true"></code> → Declara el parámetro <code>mostrarDetalle</code> con tipo <code>java.lang.Boolean</code> y lo expone al diálogo de parámetros de Studio.
Línea 100	<code><defaultValueExpression><![CDATA[Boolean.TRUE]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 101	<code></parameter></code> → Cierra <code>parameter</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 102	<code><parameter name="categoria" class="java.lang.String" isForPrompting="true"/></code> → Declara el parámetro <code>categoria</code> con tipo <code>java.lang.String</code> y lo expone al diálogo de parámetros de Studio.
Línea 103	<code><parameter name="precioMinimo" class="java.lang.Double" isForPrompting="true"/></code> → Declara el parámetro <code>precioMinimo</code> con tipo <code>java.lang.Double</code> y lo expone al diálogo de parámetros de Studio.
Línea 104	<code><parameter name="precioMaximo" class="java.lang.Double" isForPrompting="true"/></code> → Declara el parámetro <code>precioMaximo</code> con tipo <code>java.lang.Double</code> y lo expone al diálogo de parámetros de Studio.
Línea 105	<code><parameter name="umbralUnidades" class="java.lang.Integer" isForPrompting="true"></code> → Declara el parámetro <code>umbralUnidades</code> con tipo <code>java.lang.Integer</code> y lo expone al diálogo de parámetros de Studio.
Línea 106	<code><defaultValueExpression><![CDATA[Integer.valueOf(5)]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 107	<code></parameter></code> → Cierra <code>parameter</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 108	<code><parameter name="textoBusqueda" class="java.lang.String" isForPrompting="true"/></code> → Declara el parámetro <code>textoBusqueda</code> con tipo <code>java.lang.String</code> y lo expone al diálogo de parámetros de Studio.
Línea 109	<code><parameter name="categoriasLista" class="java.util.Collection" isForPrompting="false"></code> → Declara el parámetro <code>categoriasLista</code> con tipo <code>java.util.Collection</code> como parámetro interno no solicitado al usuario.
Línea 110	<code><defaultValueExpression><![CDATA[java.util.Arrays.asList("Novela", "Realismo mágico", "Cuento", "Poesía")]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 111	<code></parameter></code> → Cierra <code>parameter</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 112	<code><queryString language="sql"></code> → Abre la consulta SQL que JasperReports ejecutará para el dataset actual.
Línea 113	<code><![CDATA[</code> → Abre CDATA para escribir SQL o una expresión Java sin que sus caracteres especiales se interpreten como XML.
Línea 114	<code>SELECT l.titulo,</code> → Cláusula SQL <code>SELECT</code> : selecciona y calcula las columnas que devolverá la consulta.
Línea 115	<code>l.categoria,</code> → Aporta el valor literal <code>l.categoria</code> , al elemento XML/POM actualmente abierto.
Línea 116	<code>SUM(v.cantidad) AS unidades_vendidas,</code> → Calcula o selecciona un valor SQL y lo expone con el alias <code>unidades_vendidas</code> , que después coincide con un field del subdataset.
Línea 117	<code>SUM(v.cantidad * v.precio_unitario) AS importe_total,</code> → Calcula o selecciona un valor SQL y lo expone con el alias <code>importe_total</code> , que después coincide con un field del subdataset.
Línea 118	<code>AVG(v.precio_unitario) AS precio_medio,</code> → Calcula o selecciona un valor SQL y lo expone con el alias <code>precio_medio</code> , que después coincide con un field del subdataset.
Línea 119	<code>MIN(v.fecha_venta) AS primera_venta,</code> → Calcula o selecciona un valor SQL y lo expone con el alias <code>primera_venta</code> , que después coincide con un field del subdataset.

Línea 120	<code>MAX(v.fecha_venta) AS ultima_venta</code> → Calcula o selecciona un valor SQL y lo expone con el alias <code>ultima_venta</code> , que después coincide con un field del subdataset.
Línea 121	<code>FROM libros l</code> → Cláusula SQL <code>FROM</code> : define la tabla base de la consulta.
Línea 122	<code>LEFT JOIN ventas v ON l.titulo = v.titulo_libro</code> → Cláusula SQL <code>LEFT JOIN</code> : une datos conservando las filas del lado izquierdo aunque no tengan ventas.
Línea 123	<code>WHERE (\$P{categoria} IS NULL OR l.categoria = \$P{categoria})</code> → Cláusula SQL <code>WHERE</code> : aplica el filtro de filas.
Línea 124	<code>AND (\$P{precioMinimo} IS NULL OR l.precio >= \$P{precioMinimo})</code> → Añade otra condición lógica al filtro SQL, combinándola con la condición anterior.
Línea 125	<code>AND (\$P{precioMaximo} IS NULL OR l.precio <= \$P{precioMaximo})</code> → Añade otra condición lógica al filtro SQL, combinándola con la condición anterior.
Línea 126	<code>AND (\$P{textoBusqueda} IS NULL OR \$P{textoBusqueda} = '' OR l.titulo LIKE '%' \$P{textoBusqueda} '%')</code> → Añade otra condición lógica al filtro SQL, combinándola con la condición anterior.
Línea 127	<code>AND \$X{IN, l.categoria, categoriasLista}</code> → Añade otra condición lógica al filtro SQL, combinándola con la condición anterior.
Línea 128	<code>GROUP BY l.titulo, l.categoria</code> → Cláusula SQL <code>GROUP BY</code> : agrupa las filas antes de evaluar las funciones agregadas.
Línea 129	<code>ORDER BY l.categoria, COALESCE(importe_total, 0) DESC, l.titulo</code> → Cláusula SQL <code>ORDER BY</code> : ordena el resultado que recibirá JasperReports.
Línea 130	<code>]]></code> → Cierra el bloque <code>CDATA</code> y devuelve el control al parser XML.
Línea 131	<code></queryString></code> → Cierra la consulta SQL del dataset actual.
Línea 132	<code><field name="titulo" class="java.lang.String"/></code> → Declara el field <code>titulo</code> con tipo Java <code>java.lang.String</code> para mapear una columna del dataset.
Línea 133	<code><field name="categoria" class="java.lang.String"/></code> → Declara el field <code>categoria</code> con tipo Java <code>java.lang.String</code> para mapear una columna del dataset.
Línea 134	<code><field name="unidades_vendidas" class="java.lang.Integer"/></code> → Declara el field <code>unidades_vendidas</code> con tipo Java <code>java.lang.Integer</code> para mapear una columna del dataset.
Línea 135	<code><field name="importe_total" class="java.lang.Double"/></code> → Declara el field <code>importe_total</code> con tipo Java <code>java.lang.Double</code> para mapear una columna del dataset.
Línea 136	<code><field name="precio_medio" class="java.lang.Double"/></code> → Declara el field <code>precio_medio</code> con tipo Java <code>java.lang.Double</code> para mapear una columna del dataset.
Línea 137	<code><field name="primera_venta" class="java.lang.String"/></code> → Declara el field <code>primera_venta</code> con tipo Java <code>java.lang.String</code> para mapear una columna del dataset.
Línea 138	<code><field name="ultima_venta" class="java.lang.String"/></code> → Declara el field <code>ultima_venta</code> con tipo Java <code>java.lang.String</code> para mapear una columna del dataset.
Línea 139	<code><variable name="TotalUnidades" class="java.lang.Integer" calculation="Sum" resetType="Report"></code> → Declara la variable <code>TotalUnidades</code> con cálculo <code>Sum</code> y reinicio <code>Report</code> .
Línea 140	<code><variableExpression><![CDATA[\$F{unidades_vendidas}]]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field <code>unidades_vendidas</code> .
Línea 141	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 142	<code><variable name="TotalImporte" class="java.lang.Double" calculation="Sum" resetType="Report"></code> → Declara la variable <code>TotalImporte</code> con cálculo <code>Sum</code> y reinicio <code>Report</code> .
Línea 143	<code><variableExpression><![CDATA[\$F{importe_total}]]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field <code>importe_total</code> .
Línea 144	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.

Línea 145	<code><variable name="TotalPagina" class="java.lang.Double" calculation="Sum" resetType="Page"> → Declara la variable TotalPagina con cálculo Sum y reinicio Page.</code>
Línea 146	<code><variableExpression><![CDATA[$\{importe_total\}$]]></variableExpression> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field importe_total.</code>
Línea 147	<code></variable> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.</code>
Línea 148	<code><variable name="PrecioMedio" class="java.lang.Double" calculation="Average" resetType="Report"> → Declara la variable PrecioMedio con cálculo Average y reinicio Report.</code>
Línea 149	<code><variableExpression><![CDATA[$\{precio_medio\}$]]></variableExpression> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field precio_medio.</code>
Línea 150	<code></variable> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.</code>
Línea 151	<code><variable name="PrecioMaximo" class="java.lang.Double" calculation="Highest" resetType="Report"> → Declara la variable PrecioMaximo con cálculo Highest y reinicio Report.</code>
Línea 152	<code><variableExpression><![CDATA[$\{precio_medio\}$]]></variableExpression> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field precio_medio.</code>
Línea 153	<code></variable> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.</code>
Línea 154	<code><variable name="NumeroLibros" class="java.lang.Integer" calculation="Count" resetType="Report"> → Declara la variable NumeroLibros con cálculo Count y reinicio Report.</code>
Línea 155	<code><variableExpression><![CDATA[$\{titulo\}$]]></variableExpression> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field titulo.</code>
Línea 156	<code></variable> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.</code>
Línea 157	<code><variable name="ImporteConIva" class="java.lang.Double" calculation="Sum" resetType="Report"> → Declara la variable ImporteConIva con cálculo Sum y reinicio Report.</code>
Línea 158	<code><variableExpression><![CDATA[$\{importe_total\}$ == null $\{tipoIva\}$ == null ? null : Double.valueOf($\{importe_total\}$.doubleValue() * (1.0d + $\{tipoIva\}$.doubleValue()))]]></v...> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field importe_total, field importe_total, parámetro tipoIva, parámetro tipoIva.</code>
Línea 159	<code></variable> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.</code>
Línea 160	<code><variable name="GrupoUnidades" class="java.lang.Integer" calculation="Sum" resetType="Group" resetGroup="CategoriaGroup"> → Declara la variable GrupoUnidades con cálculo Sum y reinicio Group asociado a CategoriaGroup.</code>
Línea 161	<code><variableExpression><![CDATA[$\{unidades_vendidas\}$]]></variableExpression> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field unidades_vendidas.</code>
Línea 162	<code></variable> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.</code>
Línea 163	<code><variable name="GrupoImporte" class="java.lang.Double" calculation="Sum" resetType="Group" resetGroup="CategoriaGroup"> → Declara la variable GrupoImporte con cálculo Sum y reinicio Group asociado a CategoriaGroup.</code>
Línea 164	<code><variableExpression><![CDATA[$\{importe_total\}$]]></variableExpression> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field importe_total.</code>
Línea 165	<code></variable> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.</code>
Línea 166	<code><variable name="GrupoLibros" class="java.lang.Integer" calculation="Count" resetType="Group" resetGroup="CategoriaGroup"> → Declara la variable GrupoLibros con cálculo Count y reinicio Group asociado a CategoriaGroup.</code>
Línea 167	<code><variableExpression><![CDATA[$\{titulo\}$]]></variableExpression> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field titulo.</code>
Línea 168	<code></variable> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.</code>

Línea 169	<code><group name="CategoriaGroup" isStartNewPage="false" isReprintHeaderOnEachPage="true" minHeightToStartNewPage="80"></code> → Declara el grupo CategoriaGroup y sus propiedades de paginación/reimpresión.
Línea 170	<code><groupExpression><![CDATA[\${categoria}]]></groupExpression></code> → Define la clave que decide cuándo cambia el grupo mediante field categoria.
Línea 171	<code><groupHeader></code> → Abre la cabecera del grupo, que se emite cuando comienza cada nuevo valor de agrupación.
Línea 172	<code><band height="28"></code> → Define una banda de 28 puntos, reservando ese espacio para sus elementos.
Línea 173	<code><textField><reportElement x="0" y="2" width="555" height="22" style="M5GrupoCabecera" /><textFieldExpression><![CDATA["Categoría: " + \${categoria}]]></textFieldExpression></text...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=2, ancho=555, alto=22 y aplica el estilo M5GrupoCabecera; evalúa la expresión usando field categoria.
Línea 174	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 175	<code></groupHeader></code> → Cierra groupHeader y vuelve al elemento padre de la jerarquía JRXML.
Línea 176	<code><groupFooter></code> → Abre el pie del grupo, donde se muestran los acumulados justo antes de cambiar de grupo.
Línea 177	<code><band height="34"></code> → Define una banda de 34 puntos, reservando ese espacio para sus elementos.
Línea 178	<code><textField><reportElement x="0" y="3" width="185" height="18" style="Dato" /><textFieldExpression><![CDATA["Libros del grupo: " + \${GrupoLibros}]]></textFieldExpression></textFi...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=3, ancho=185, alto=18 y aplica el estilo Dato; evalúa la expresión usando variable GrupoLibros.
Línea 179	<code><textField><reportElement x="185" y="3" width="180" height="18" style="Dato" /><textFieldExpression><![CDATA["Unidades: " + (\${GrupoUnidades} == null ? 0 : \${GrupoUnidades})]]></text...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=185, y=3, ancho=180, alto=18 y aplica el estilo Dato; evalúa la expresión usando variable GrupoUnidades, variable GrupoUnidades.
Línea 180	<code><textField><reportElement x="365" y="3" width="190" height="18" style="Dato" /><textElement textAlignment="Right" /><textFieldExpression><![CDATA["Importe: " + new java.text.Decim...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=365, y=3, ancho=190, alto=18 y aplica el estilo Dato; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable GrupoImporte, variable GrupoImporte.
Línea 181	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 182	<code></groupFooter></code> → Cierra groupFooter y vuelve al elemento padre de la jerarquía JRXML.
Línea 183	<code></group></code> → Finaliza la definición del grupo y sus bandas asociadas.
Línea 184	<code><background><band height="0" /></background></code> → Composición de la línea: encadena además <background>, <band> dentro de la misma jerarquía.
Línea 185	<code><title></code> → Abre la banda Title, emitida una sola vez al inicio del informe.
Línea 186	<code><band height="124"></code> → Define una banda de 124 puntos, reservando ese espacio para sus elementos.
Línea 187	<code><staticText></code> → Abre un elemento de texto literal; su contenido no depende de fields, parámetros ni variables.
Línea 188	<code><reportElement x="0" y="4" width="555" height="28" uuid="40000000-0000-4000-8000-000000000001" style="M5TituloPrincipal" /></code> → Posiciona el elemento en x=0, y=4, con ancho 555 y alto 28, aplicando el estilo M5TituloPrincipal.
Línea 189	<code><textElement textAlignment="Center" verticalAlignment="Middle" /></code> → Configura el formato interno del texto: alineación horizontal Center, alineación vertical Middle.
Línea 190	<code><text><![CDATA[Informe de Ventas - Agregación por Título]]></text></code> → Define el texto literal visible: Informe de Ventas - Agregación por Título.
Línea 191	<code></staticText></code> → Cierra staticText y vuelve al elemento padre de la jerarquía JRXML.

Línea 192	<pre><staticText><reportElement x="0" y="38" width="110" height="18" uuid="40000000-0000-4000-8000-000000000002"/><text><![CDATA[Generado por:]]></text></staticText></pre> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=38, ancho=110, alto=18; muestra el literal 'Generado por:'.
Línea 193	<pre><textField isBlankWhenNull="true"><reportElement x="110" y="38" width="160" height="18" uuid="40000000-0000-4000-8000-000000000003"/><textFieldExpression><![CDATA[{P{usuario}}]>... </textFieldExpression></textField></pre> → Composición de la línea: crea un textField dinámico; lo posiciona en x=110, y=38, ancho=160, alto=18; evalúa la expresión usando parámetro usuario.
Línea 194	<pre><staticText><reportElement x="300" y="38" width="80" height="18" uuid="40000000-0000-4000-8000-000000000004"/><text><![CDATA[Fecha:]]></text></staticText></pre> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=38, ancho=80, alto=18; muestra el literal 'Fecha:'.
Línea 195	<pre><textField pattern="dd/MM/yyyy"><reportElement x="380" y="38" width="175" height="18" uuid="40000000-0000-4000-8000-000000000005"/><textFieldExpression><![CDATA[{P{fechaInforme}}]>... </textFieldExpression></textField></pre> → Composición de la línea: crea un textField dinámico con patrón dd/MM/yyyy; lo posiciona en x=380, y=38, ancho=175, alto=18; evalúa la expresión usando parámetro fechaInforme.
Línea 196	<pre><staticText><reportElement x="0" y="62" width="100" height="18" uuid="40000000-0000-4000-8000-000000000006"/><text><![CDATA[Departamento:]]></text></staticText></pre> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=62, ancho=100, alto=18; muestra el literal 'Departamento:'.
Línea 197	<pre><textField isBlankWhenNull="true"><reportElement x="100" y="62" width="170" height="18" uuid="40000000-0000-4000-8000-000000000007"/><textFieldExpression><![CDATA[{P{departament...}}]>... </textFieldExpression></textField></pre> → Composición de la línea: crea un textField dinámico; lo posiciona en x=100, y=62, ancho=170, alto=18; evalúa la expresión usando parámetro departamento.
Línea 198	<pre><staticText><reportElement x="300" y="62" width="70" height="18" uuid="40000000-0000-4000-8000-000000000008"/><text><![CDATA[Periodo:]]></text></staticText></pre> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=62, ancho=70, alto=18; muestra el literal 'Periodo:'.
Línea 199	<pre><textField isBlankWhenNull="true"><reportElement x="370" y="62" width="185" height="18" uuid="40000000-0000-4000-8000-000000000009"/><textFieldExpression><![CDATA[{P{periodo}}]>... </textFieldExpression></textField></pre> → Composición de la línea: crea un textField dinámico; lo posiciona en x=370, y=62, ancho=185, alto=18; evalúa la expresión usando parámetro periodo.
Línea 200	<pre><staticText><reportElement x="0" y="86" width="100" height="18" uuid="40000000-0000-4000-8000-000000000010"/><text><![CDATA[Búsqueda:]]></text></staticText></pre> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=86, ancho=100, alto=18; muestra el literal 'Búsqueda:'.
Línea 201	<pre><textField isBlankWhenNull="true"><reportElement x="100" y="86" width="170" height="18" uuid="40000000-0000-4000-8000-000000000011"/><textFieldExpression><![CDATA[{P{textoBusque...}}]>... </textFieldExpression></textField></pre> → Composición de la línea: crea un textField dinámico; lo posiciona en x=100, y=86, ancho=170, alto=18; evalúa la expresión usando parámetro textoBusqueda, parámetro textoBusqueda, parámetro textoBusqueda.
Línea 202	<pre><staticText><reportElement x="300" y="86" width="90" height="18" uuid="40000000-0000-4000-8000-000000000012"/><text><![CDATA[Categorías:]]></text></staticText></pre> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=86, ancho=90, alto=18; muestra el literal 'Categorías:'.
Línea 203	<pre><textField textAdjust="StretchHeight"><reportElement x="390" y="86" width="165" height="34" uuid="40000000-0000-4000-8000-000000000013"/><textFieldExpression><![CDATA[{String.val...}}]>... </textFieldExpression></textField></pre> → Composición de la línea: crea un textField dinámico; lo posiciona en x=390, y=86, ancho=165, alto=34; evalúa la expresión usando parámetro categoriasLista.
Línea 204	</band> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 205	</title> → Cierra title y vuelve al elemento padre de la jerarquía JRXML.
Línea 206	<columnHeader> → Abre Column Header, repetida al comienzo de cada columna/página según la paginación.
Línea 207	<band height="62"> → Define una banda de 62 puntos, reservando ese espacio para sus elementos.

Línea 208	<code><staticText><reportElement x="0" y="2" width="215" height="18" uuid="41000000-0000-4000-8000-000000000001" style="Cabecera"/><text><![CDATA[Título]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=2, ancho=215, alto=18 y aplica el estilo Cabecera; muestra el literal 'Título'.
Línea 209	<code><staticText><reportElement x="215" y="2" width="55" height="18" uuid="41000000-0000-4000-8000-000000000002" style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[...]</code> → Composición de la línea: crea un texto literal; lo posiciona en x=215, y=2, ancho=55, alto=18 y aplica el estilo Cabecera; configura la alineación del texto (horizontal Right); muestra el literal 'Unid.'.
Línea 210	<code><staticText><reportElement x="280" y="2" width="90" height="18" uuid="41000000-0000-4000-8000-000000000003" style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[...]</code> → Composición de la línea: crea un texto literal; lo posiciona en x=280, y=2, ancho=90, alto=18 y aplica el estilo Cabecera; configura la alineación del texto (horizontal Right); muestra el literal 'Importe'.
Línea 211	<code><staticText><reportElement x="380" y="2" width="65" height="18" uuid="41000000-0000-4000-8000-000000000004" style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[...]</code> → Composición de la línea: crea un texto literal; lo posiciona en x=380, y=2, ancho=65, alto=18 y aplica el estilo Cabecera; configura la alineación del texto (horizontal Right); muestra el literal 'Precio med.'.
Línea 212	<code><staticText><reportElement x="455" y="2" width="100" height="18" uuid="41000000-0000-4000-8000-000000000005" style="Cabecera"/><text><![CDATA[Categoría]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=455, y=2, ancho=100, alto=18 y aplica el estilo Cabecera; muestra el literal 'Categoría'.
Línea 213	<code><staticText><reportElement x="0" y="24" width="130" height="18" uuid="41000000-0000-4000-8000-000000000006" style="Cabecera"/><text><![CDATA[Primera venta]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=24, ancho=130, alto=18 y aplica el estilo Cabecera; muestra el literal 'Primera venta'.
Línea 214	<code><staticText><reportElement x="130" y="24" width="130" height="18" uuid="41000000-0000-4000-8000-000000000007" style="Cabecera"/><text><![CDATA[Última venta]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=130, y=24, ancho=130, alto=18 y aplica el estilo Cabecera; muestra el literal 'Última venta'.
Línea 215	<code><staticText><reportElement x="260" y="24" width="160" height="18" uuid="41000000-0000-4000-8000-000000000008" style="Cabecera"/><text><![CDATA[Periodo de ventas]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=260, y=24, ancho=160, alto=18 y aplica el estilo Cabecera; muestra el literal 'Periodo de ventas'.
Línea 216	<code><staticText></code> → Abre un elemento de texto literal; su contenido no depende de fields, parámetros ni variables.
Línea 217	<code><reportElement x="420" y="24" width="135" height="18" uuid="41000000-0000-4000-8000-000000000009" style="Cabecera"></code> → Posiciona el elemento en x=420, y=24, con ancho 135 y alto 18, aplicando el estilo Cabecera.
Línea 218	<code><printWhenExpression><![CDATA[Boolean.TRUE.equals(\$P{mostrarDetalle})]]></printWhenExpression></code> → Evalúa una condición booleana para decidir si la banda o elemento se imprime usando parámetro mostrarDetalle.
Línea 219	<code></reportElement></code> → Cierra reportElement y vuelve al elemento padre de la jerarquía JRXML.
Línea 220	<code><textElement textAlignment="Right"/></code> → Configura el formato interno del texto: alineación horizontal Right.
Línea 221	<code><text><![CDATA[Importe con IVA]]></text></code> → Define el texto literal visible: Importe con IVA.
Línea 222	<code></staticText></code> → Cierra staticText y vuelve al elemento padre de la jerarquía JRXML.
Línea 223	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 224	<code></columnHeader></code> → Cierra columnHeader y vuelve al elemento padre de la jerarquía JRXML.
Línea 225	<code><detail></code> → Abre Detail, la sección que se repite para cada registro del dataset principal.

Línea 226	<code><band height="82" splitType="Stretch"></code> → Define una banda de 82 puntos con splitType Stretch, reservando ese espacio para sus elementos.
Línea 227	<code><textField textAdjust="StretchHeight"><reportElement x="0" y="0" width="215" height="20" uuid="42000000-0000-4000-8000-000000000001" style="Dato"/><textFieldExpression><![CDATA[...]</code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=0, ancho=215, alto=20 y aplica el estilo Dato; evalúa la expresión usando field titulo.
Línea 228	<code><textField isBlankWhenNull="true"><reportElement x="215" y="0" width="55" height="20" uuid="42000000-0000-4000-8000-000000000002" style="UnidadesCondional"/><textElement textA...</code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=215, y=0, ancho=55, alto=20 y aplica el estilo UnidadesCondional; configura la alineación del texto (horizontal Right); evalúa la expresión usando field unidades_vendidas.
Línea 229	<code><textField pattern="#.##0.00 €" isBlankWhenNull="true"><reportElement x="280" y="0" width="90" height="20" uuid="42000000-0000-4000-8000-000000000003" style="Dato"/><textElement...</code> → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=280, y=0, ancho=90, alto=20 y aplica el estilo Dato; configura la alineación del texto (horizontal Right); evalúa la expresión usando field importe_total.
Línea 230	<code><textField isBlankWhenNull="true"><reportElement x="380" y="0" width="65" height="20" uuid="42000000-0000-4000-8000-000000000004" style="Dato"/><textElement textAlignment="Right..."</code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=380, y=0, ancho=65, alto=20 y aplica el estilo Dato; configura la alineación del texto (horizontal Right); evalúa la expresión usando field precio_medio, field precio_medio.
Línea 231	<code><textField isBlankWhenNull="true"><reportElement x="455" y="0" width="100" height="20" uuid="42000000-0000-4000-8000-000000000005" style="Dato"/><textFieldExpression><![CDATA[{\$F...</code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=455, y=0, ancho=100, alto=20 y aplica el estilo Dato; evalúa la expresión usando field categoria.
Línea 232	<code><textField isBlankWhenNull="true"><reportElement x="0" y="24" width="130" height="18" uuid="42000000-0000-4000-8000-000000000006" style="Dato"/><textFieldExpression><![CDATA[{\$F{...</code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=24, ancho=130, alto=18 y aplica el estilo Dato; evalúa la expresión usando field primera_venta.
Línea 233	<code><textField isBlankWhenNull="true"><reportElement x="130" y="24" width="130" height="18" uuid="42000000-0000-4000-8000-000000000007" style="Dato"/><textFieldExpression><![CDATA[{\$...</code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=130, y=24, ancho=130, alto=18 y aplica el estilo Dato; evalúa la expresión usando field ultima_venta.
Línea 234	<code><textField><reportElement x="260" y="24" width="160" height="18" uuid="42000000-0000-4000-8000-000000000008" style="Dato"/><textElement textAlignment="Center..."</code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=260, y=24, ancho=160, alto=18 y aplica el estilo Dato; configura la alineación del texto (horizontal Center); evalúa la expresión usando field primera_venta, field primera_venta, field ultima_venta.
Línea 235	<code><textField pattern="#.##0.00 €" isBlankWhenNull="true"></code> → Abre un textField dinámico con formato #.##0.00 €, cuyo valor se obtiene de su textFieldExpression.
Línea 236	<code><reportElement x="420" y="24" width="135" height="18" uuid="42000000-0000-4000-8000-000000000009" style="Dato"></code> → Posiciona el elemento en x=420, y=24, con ancho 135 y alto 18, aplicando el estilo Dato.
Línea 237	<code><printWhenExpression><![CDATA[Boolean.TRUE.equals(\$P{mostrarDetalle})]]></printWhenExpression></code> → Evalúa una condición booleana para decidir si la banda o elemento se imprime usando parámetro mostrarDetalle.
Línea 238	<code></reportElement></code> → Cierra reportElement y vuelve al elemento padre de la jerarquía JRXML.
Línea 239	<code><textElement textAlignment="Right"/></code> → Configura el formato interno del texto: alineación horizontal Right.
Línea 240	<code><textFieldExpression><![CDATA[{\$F{importe_total} == null \$P{tipoIva} == null ? null : Double.valueOf(\$F{importe_total}).doubleValue() * (1.0d + \$P{tipoIva}.doubleValue())}]]></...]</code> → Calcula el valor mostrado por el textField mediante una expresión Java que usa field importe_total, field importe_total, parámetro tipoIva, parámetro tipoIva.

Línea 241	<code></textField></code> → Cierra <code>textField</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 242	<code><textField><reportElement x="0" y="48" width="105" height="18" uuid="42000000-0000-4000-8000-000000000010" style="Dato"/><textFieldExpression><![CDATA[{\$F{unidades_vendidas} == n...}</code> → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=0</code> , <code>y=48</code> , <code>ancho=105</code> , <code>alto=18</code> y aplica el estilo <code>Dato</code> ; evalúa la expresión usando <code>field unidades_vendidas</code> , <code>field unidades_vendidas</code> , <code>field unidades_vendidas</code> .
Línea 243	<code><textField><reportElement x="105" y="48" width="185" height="18" uuid="42000000-0000-4000-8000-000000000011" style="Dato"/><textFieldExpression><![CDATA[{\$F{titulo} == null ? "" ...}</code> → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=105</code> , <code>y=48</code> , <code>ancho=185</code> , <code>alto=18</code> y aplica el estilo <code>Dato</code> ; evalúa la expresión usando <code>field titulo</code> , <code>field titulo</code> .
Línea 244	<code><textField><reportElement x="290" y="48" width="80" height="18" uuid="42000000-0000-4000-8000-000000000012" style="Dato"/><textElement textAlignment="Right" /><textFieldExpressio...}</code> → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=290</code> , <code>y=48</code> , <code>ancho=80</code> , <code>alto=18</code> y aplica el estilo <code>Dato</code> ; configura la alineación del texto (horizontal <code>Right</code>); evalúa la expresión usando <code>field precio_medio</code> , <code>field precio_medio</code> .
Línea 245	<code><textField><reportElement x="370" y="48" width="90" height="18" uuid="42000000-0000-4000-8000-000000000013" style="Dato"/><textElement textAlignment="Center" /><textFieldExpressio...}</code> → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=370</code> , <code>y=48</code> , <code>ancho=90</code> , <code>alto=18</code> y aplica el estilo <code>Dato</code> ; configura la alineación del texto (horizontal <code>Center</code>); evalúa la expresión usando <code>field primera_venta</code> , <code>field ultima_venta</code> , <code>field primera_venta</code> , <code>field ultima_venta</code> .
Línea 246	<code><textField><reportElement x="460" y="48" width="95" height="18" uuid="42000000-0000-4000-8000-000000000014" style="Dato"/><textElement textAlignment="Right" /><textFieldExpressio...}</code> → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=460</code> , <code>y=48</code> , <code>ancho=95</code> , <code>alto=18</code> y aplica el estilo <code>Dato</code> ; configura la alineación del texto (horizontal <code>Right</code>); evalúa la expresión usando <code>field unidades_vendidas</code> , <code>field unidades_vendidas</code> , <code>parámetro umbralUnidades</code> , <code>parámetro umbralUnidades</code> .
Línea 247	<code></band></code> → Cierra <code>band</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 248	<code><band height="14"></code> → Define una banda de 14 puntos, reservando ese espacio para sus elementos.
Línea 249	<code><printWhenExpression><![CDATA[{\$F{unidades_vendidas} != null && \$P{umbralUnidades} != null && \$F{unidades_vendidas}.intValue() >= \$P{umbralUnidades}.intValue()}]></printWhenExpre...}</code> → Evalúa una condición booleana para decidir si la banda o elemento se imprime usando <code>field unidades_vendidas</code> , <code>field unidades_vendidas</code> , <code>parámetro umbralUnidades</code> , <code>parámetro umbralUnidades</code> .
Línea 250	<code><textField><reportElement x="0" y="0" width="555" height="12" uuid="42000000-0000-4000-8000-000000000015"/><textElement textAlignment="Center"><font fontName="DejaVu Sans" size=...}</code> → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=0</code> , <code>y=0</code> , <code>ancho=555</code> , <code>alto=12</code> ; configura la alineación del texto (horizontal <code>Center</code>); configura la fuente (familia <code>DejaVu Sans</code> , tamaño 8, negrita); evalúa la expresión usando <code>field titulo</code> , <code>parámetro umbralUnidades</code> .
Línea 251	<code></band></code> → Cierra <code>band</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 252	<code><band height="88" splitType="Stretch"></code> → Define una banda de 88 puntos con <code>splitType Stretch</code> , reservando ese espacio para sus elementos.
Línea 253	<code><printWhenExpression><![CDATA[{\$F{unidades_vendidas} != null}]></printWhenExpression></code> → Evalúa una condición booleana para decidir si la banda o elemento se imprime usando <code>field unidades_vendidas</code> .
Línea 254	<code><staticText></code> → Abre un elemento de texto literal; su contenido no depende de fields, parámetros ni variables.
Línea 255	<code><reportElement x="0" y="2" width="555" height="16" style="Cabecera"/></code> → Posiciona el elemento en <code>x=0</code> , <code>y=2</code> , con <code>ancho 555</code> y <code>alto 16</code> , aplicando el estilo <code>Cabecera</code> .
Línea 256	<code><text><![CDATA[Detalle de ventas]]></text></code> → Define el texto literal visible: <code>Detalle de ventas</code> .
Línea 257	<code></staticText></code> → Cierra <code>staticText</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 258	<code><subreport></code> → Abre el componente <code>subreport</code> que ejecuta un informe hijo dentro de la banda del maestro.

Línea 259	<code><reportElement x="0" y="22" width="555" height="60" isRemoveLineWhenBlank="true" /></code> → Posiciona el elemento en x=0, y=22, con ancho 555 y alto 60, y elimina su línea cuando queda vacío.
Línea 260	<code><subreportParameter name="tituloLibro"></code> → Declara el parámetro del subreporte tituloLibro que recibirá un valor del informe maestro.
Línea 261	<code><subreportParameterExpression><![CDATA[{\$F{titulo}}]></subreportParameterExpression></code> → Calcula el valor enviado al parámetro del subreporte a partir de field titulo.
Línea 262	<code></subreportParameter></code> → Cierra subreportParameter y vuelve al elemento padre de la jerarquía JRXML.
Línea 263	<code><connectionExpression><![CDATA[{\$P{REPORT_CONNECTION}}]></connectionExpression></code> → Entrega al subreporte/subdataset la misma REPORT_CONNECTION usada por el informe maestro, evitando abrir otra conexión.
Línea 264	<code><subreportExpression><![CDATA["reports/subinforme_ventas_detalle.jasper"]]></subreportExpression></code> → Devuelve la ruta del archivo subinforme_ventas_detalle.jasper que JasperReports cargará como informe hijo.
Línea 265	<code></subreport></code> → Finaliza el componente de subreporte.
Línea 266	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 267	<code><band height="104" splitType="Stretch"></code> → Define una banda de 104 puntos con splitType Stretch, reservando ese espacio para sus elementos.
Línea 268	<code><printWhenExpression><![CDATA[{\$F{unidades_vendidas} != null}]></printWhenExpression></code> → Evalúa una condición booleana para decidir si la banda o elemento se imprime usando field unidades_vendidas.
Línea 269	<code><staticText></code> → Abre un elemento de texto literal; su contenido no depende de fields, parámetros ni variables.
Línea 270	<code><reportElement x="0" y="2" width="555" height="16" style="Cabecera" /></code> → Posiciona el elemento en x=0, y=2, con ancho 555 y alto 16, aplicando el estilo Cabecera.
Línea 271	<code><text><![CDATA[Top 3 ventas por cantidad]]></text></code> → Define el texto literal visible: Top 3 ventas por cantidad.
Línea 272	<code></staticText></code> → Cierra staticText y vuelve al elemento padre de la jerarquía JRXML.
Línea 273	<code><componentElement></code> → Abre un contenedor de componentes extendidos; en este checkpoint contiene la tabla c:table.
Línea 274	<code><reportElement x="0" y="22" width="555" height="76" /></code> → Posiciona el elemento en x=0, y=22, con ancho 555 y alto 76.
Línea 275	<code><c:table xmlns:c="http://jasperreports.sourceforge.net/jasperreports/components"></code> → Abre la tabla del namespace de componentes JasperReports; sus columnas usan un datasetRun independiente.
Línea 276	<code>xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports/components http://jasperreports.sourceforge.net/xsd/components.xsd"></code> → Relaciona el namespace de JasperReports con su XSD para que Studio y el compilador validen la estructura.
Línea 277	<code><datasetRun subDataset="DatasetTopVentas"></code> → Asocia el componente con el subdataset DatasetTopVentas para ejecutar su consulta.
Línea 278	<code><datasetParameter name="tituloLibro"></code> → Declara el parámetro tituloLibro que se enviará al subdataset de la tabla.
Línea 279	<code><datasetParameterExpression><![CDATA[{\$F{titulo}}]></datasetParameterExpression></code> → Calcula el valor enviado al parámetro del subdataset desde field titulo.
Línea 280	<code></datasetParameter></code> → Cierra datasetParameter y vuelve al elemento padre de la jerarquía JRXML.
Línea 281	<code><connectionExpression><![CDATA[{\$P{REPORT_CONNECTION}}]></connectionExpression></code> → Entrega al subreporte/subdataset la misma REPORT_CONNECTION usada por el informe maestro, evitando abrir otra conexión.
Línea 282	<code></datasetRun></code> → Cierra datasetRun y vuelve al elemento padre de la jerarquía JRXML.

Línea 283	<code><c:column width="255"></code> → Declara una columna de tabla de 255 puntos de ancho.
Línea 284	<code><c:columnHeader style="M5TablaCabecera" height="20"><staticText><reportElement x="0" y="0" width="255" height="20"/><text><![CDATA[Fecha]]></text></staticText></c:columnHeader></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=0, ancho=255, alto=20 y aplica el estilo M5TablaCabecera; muestra el literal 'Fecha'; define una celda de cabecera de la tabla.
Línea 285	<code><c:detailCell style="M5TablaDetalle" height="18"><textField><reportElement x="0" y="0" width="255" height="18"/><textFieldExpression><![CDATA[\${fecha_venta}]]></textFieldExpres...</code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=0, ancho=255, alto=18 y aplica el estilo M5TablaDetalle; evalúa la expresión usando field fecha_venta; define una celda repetida de detalle de la tabla.
Línea 286	<code></c:column></code> → Cierra c:column y vuelve al elemento padre de la jerarquía JRXML.
Línea 287	<code><c:column width="100"></code> → Declara una columna de tabla de 100 puntos de ancho.
Línea 288	<code><c:columnHeader style="M5TablaCabecera" height="20"><staticText><reportElement x="0" y="0" width="100" height="20"/><textElement textAlignment="Right"/><text><![CDATA[Cantidad]]...</code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=0, ancho=100, alto=20 y aplica el estilo M5TablaCabecera; configura la alineación del texto (horizontal Right); muestra el literal 'Cantidad'; define una celda de cabecera de la tabla.
Línea 289	<code><c:detailCell style="M5TablaDetalle" height="18"><textField><reportElement x="0" y="0" width="100" height="18"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[...</code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=0, ancho=100, alto=18 y aplica el estilo M5TablaDetalle; configura la alineación del texto (horizontal Right); evalúa la expresión usando field cantidad; define una celda repetida de detalle de la tabla.
Línea 290	<code></c:column></code> → Cierra c:column y vuelve al elemento padre de la jerarquía JRXML.
Línea 291	<code><c:column width="200"></code> → Declara una columna de tabla de 200 puntos de ancho.
Línea 292	<code><c:columnHeader style="M5TablaCabecera" height="20"><staticText><reportElement x="0" y="0" width="200" height="20"/><textElement textAlignment="Right"/><text><![CDATA[Precio uni...</code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=0, ancho=200, alto=20 y aplica el estilo M5TablaCabecera; configura la alineación del texto (horizontal Right); muestra el literal 'Precio unitario'; define una celda de cabecera de la tabla.
Línea 293	<code><c:detailCell style="M5TablaDetalle" height="18"><textField pattern="#,##0.00 €"><reportElement x="0" y="0" width="200" height="18"/><textElement textAlignment="Right"/><textFie...</code> → Composición de la línea: crea un textField dinámico con patrón #,##0.00 €; lo posiciona en x=0, y=0, ancho=200, alto=18 y aplica el estilo M5TablaDetalle; configura la alineación del texto (horizontal Right); evalúa la expresión usando field precio_unitario; define una celda repetida de detalle de la tabla.
Línea 294	<code></c:column></code> → Cierra c:column y vuelve al elemento padre de la jerarquía JRXML.
Línea 295	<code></c:table></code> → Finaliza la tabla integrada.
Línea 296	<code></componentElement></code> → Cierra componentElement y vuelve al elemento padre de la jerarquía JRXML.
Línea 297	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 298	<code></detail></code> → Finaliza la sección Detail del informe.
Línea 299	<code><pageFooter></code> → Abre Page Footer, emitido al pie de cada página.
Línea 300	<code><band height="62"></code> → Define una banda de 62 puntos, reservando ese espacio para sus elementos.
Línea 301	<code><staticText><reportElement x="0" y="4" width="120" height="15" uuid="43000000-0000-4000-8000-000000000001"/><text><![CDATA[Total de títulos:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=4, ancho=120, alto=15; muestra el literal 'Total de títulos:'.
Línea 302	<code><textField evaluationTime="Report"><reportElement x="120" y="4" width="60" height="15" uuid="43000000-0000-4000-8000-000000000002"/><textFieldExpression><![CDATA[\${REPORT_COUNT}]]...</code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=120, y=4, ancho=60, alto=15; evalúa la expresión usando variable REPORT_COUNT.

Línea 303	<code><textField><reportElement x="190" y="28" width="180" height="15" uuid="43000000-0000-4000-8000-000000000003"/><textElement textAlignment="Right" /><textFieldExpression><![CDATA["... → Composición de la línea: crea un textField dinámico; lo posiciona en x=190, y=28, ancho=180, alto=15; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable PAGE_NUMBER.</code>
Línea 304	<code><textField evaluationTime="Report"><reportElement x="375" y="28" width="35" height="15" uuid="43000000-0000-4000-8000-000000000004"/><textFieldExpression><![CDATA[\$V{PAGE_NUMBER... → Composición de la línea: crea un textField dinámico; lo posiciona en x=375, y=28, ancho=35, alto=15; evalúa la expresión usando variable PAGE_NUMBER.</code>
Línea 305	<code><staticText><reportElement x="300" y="4" width="120" height="15" uuid="43000000-0000-4000-8000-000000000005"/><text><![CDATA[Subtotal página:]]></text></staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=4, ancho=120, alto=15; muestra el literal 'Subtotal página:'.</code>
Línea 306	<code><textField pattern="#.##0.00 €"><reportElement x="420" y="4" width="135" height="15" uuid="43000000-0000-4000-8000-000000000006"/><textElement textAlignment="Right" /><textFieldE... → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=420, y=4, ancho=135, alto=15; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable TotalPagina.</code>
Línea 307	<code></band> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.</code>
Línea 308	<code></pageFooter> → Cierra pageFooter y vuelve al elemento padre de la jerarquía JRXML.</code>
Línea 309	<code><summary> → Abre Summary, emitido una sola vez después del último registro.</code>
Línea 310	<code><band height="700"> → Define una banda de 700 puntos, reservando ese espacio para sus elementos.</code>
Línea 311	<code><staticText><reportElement x="0" y="5" width="205" height="18" uuid="44000000-0000-4000-8000-000000000001"/><text><![CDATA[Total de unidades vendidas:]]></text></staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=5, ancho=205, alto=18; muestra el literal 'Total de unidades vendidas:'.</code>
Línea 312	<code><textField><reportElement x="205" y="5" width="80" height="18" uuid="44000000-0000-4000-8000-000000000002"/><textElement textAlignment="Right" /><textFieldExpression><![CDATA[\$V{... → Composición de la línea: crea un textField dinámico; lo posiciona en x=205, y=5, ancho=80, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable TotalUnidades.</code>
Línea 313	<code><staticText><reportElement x="300" y="5" width="120" height="18" uuid="44000000-0000-4000-8000-000000000003"/><text><![CDATA[Importe total:]]></text></staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=5, ancho=120, alto=18; muestra el literal 'Importe total:'.</code>
Línea 314	<code><textField pattern="#.##0.00 €"><reportElement x="420" y="5" width="135" height="18" uuid="44000000-0000-4000-8000-000000000004"/><textElement textAlignment="Right" /><textFieldE... → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=420, y=5, ancho=135, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable TotalImporte.</code>
Línea 315	<code><staticText><reportElement x="0" y="30" width="205" height="18" uuid="44000000-0000-4000-8000-000000000005"/><text><![CDATA[Precio medio agregado:]]></text></staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=30, ancho=205, alto=18; muestra el literal 'Precio medio agregado:'.</code>
Línea 316	<code><textField pattern="#.##0.00 €"><reportElement x="205" y="30" width="80" height="18" uuid="44000000-0000-4000-8000-000000000006"/><textElement textAlignment="Right" /><textFieldE... → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=205, y=30, ancho=80, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable PrecioMedio.</code>
Línea 317	<code><staticText><reportElement x="300" y="30" width="120" height="18" uuid="44000000-0000-4000-8000-000000000007"/><text><![CDATA[Precio máximo:]]></text></staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=30, ancho=120, alto=18; muestra el literal 'Precio máximo:'.</code>

Línea 318	<code><textField pattern="#.##0.00 €"><reportElement x="420" y="30" width="135" height="18" uuid="44000000-0000-4000-8000-000000000008"/><textElement textAlignment="Right" /><textField... → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=420, y=30, ancho=135, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable PrecioMaximo.</code>
Línea 319	<code><staticText><reportElement x="0" y="55" width="205" height="18" uuid="44000000-0000-4000-8000-000000000009"/><text><![CDATA[Número de libros:]]></text></staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=55, ancho=205, alto=18; muestra el literal 'Número de libros:'.</code>
Línea 320	<code><textField><reportElement x="205" y="55" width="80" height="18" uuid="44000000-0000-4000-8000-000000000010"/><textElement textAlignment="Right" /><textFieldExpression><![CDATA[\$V... → Composición de la línea: crea un textField dinámico; lo posiciona en x=205, y=55, ancho=80, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable NumeroLibros.</code>
Línea 321	<code><staticText><reportElement x="300" y="55" width="120" height="18" uuid="44000000-0000-4000-8000-000000000011"/><text><![CDATA[Importe con IVA:]]></text></staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=55, ancho=120, alto=18; muestra el literal 'Importe con IVA:'.</code>
Línea 322	<code><textField pattern="#.##0.00 €"><reportElement x="420" y="55" width="135" height="18" uuid="44000000-0000-4000-8000-000000000012"/><textElement textAlignment="Right" /><textField... → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=420, y=55, ancho=135, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable ImporteConIva.</code>
Línea 323	<code><textField><reportElement x="0" y="80" width="555" height="18" uuid="44000000-0000-4000-8000-000000000013"/><textElement textAlignment="Center" /><textFieldExpression><![CDATA[St... → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=80, ancho=555, alto=18; configura la alineación del texto (horizontal Center); evalúa la expresión usando variable NumeroLibros, variable TotalUnidades, variable TotalImporte.</code>
Línea 324	<code><textField><reportElement x="0" y="103" width="350" height="18" uuid="44000000-0000-4000-8000-000000000014"/><textElement textAlignment="Center"><font fontName="DejaVu Sans" siz... → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=103, ancho=350, alto=18; configura la alineación del texto (horizontal Center); configura la fuente (familia DejaVu Sans, tamaño 10, negrita); evalúa la expresión usando parámetro umbralUnidades, parámetro umbralUnidades, variable TotalUnidades, variable TotalUnidades.</code>
Línea 325	<code><textField><reportElement x="360" y="103" width="195" height="18" uuid="44000000-0000-4000-8000-000000000015"/><textFieldExpression><![CDATA["Resultados encontrados: " + \$V{REPO... → Composición de la línea: crea un textField dinámico; lo posiciona en x=360, y=103, ancho=195, alto=18; evalúa la expresión usando variable REPORT_COUNT.</code>
Línea 326	<code><staticText><reportElement x="0" y="140" width="555" height="20" style="Cabecera"/><text><![CDATA[Ventas por categoría – importe]]></text></staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=140, ancho=555, alto=20 y aplica el estilo Cabecera; muestra el literal 'Ventas por categoría — importe'.</code>
Línea 327	<code><barChart> → Abre el gráfico de barras nativo de JasperReports que se integrará en el informe maestro.</code>
Línea 328	<code><chart> → Abre la configuración común del gráfico: geometría, título, subtítulo y leyenda.</code>
Línea 329	<code><reportElement x="0" y="165" width="555" height="250" /> → Posiciona el elemento en x=0, y=165, con ancho 555 y alto 250.</code>
Línea 330	<code><chartTitle><titleExpression><![CDATA["Ventas por categoría"]]></titleExpression></chartTitle> → Composición de la línea: encadena además <chartTitle>, <titleExpression> dentro de la misma jerarquía.</code>
Línea 331	<code><chartSubtitle/> → Declara el subtítulo del gráfico; en este checkpoint queda vacío.</code>
Línea 332	<code><chartLegend position="Bottom" /> → Configura la leyenda del gráfico en la posición Bottom.</code>
Línea 333	<code></chart> → Cierra chart y vuelve al elemento padre de la jerarquía JRXML.</code>
Línea 334	<code><categoryDataset> → Abre el dataset categórico que alimenta al gráfico con serie, categoría y valor.</code>

Línea 335	<code><dataset></code> → Abre el contenedor de ejecución de datos del componente actual.
Línea 336	<code><datasetRun subDataset="DatasetVentasPorCategoria"></code> → Asocia el componente con el subdataset <code>DatasetVentasPorCategoria</code> para ejecutar su consulta.
Línea 337	<code><connectionExpression><![CDATA[\${REPORT_CONNECTION}]]></connectionExpression></code> → Entrega al subreporte/subdataset la misma <code>REPORT_CONNECTION</code> usada por el informe maestro, evitando abrir otra conexión.
Línea 338	<code></datasetRun></code> → Cierra <code>datasetRun</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 339	<code></dataset></code> → Cierra <code>dataset</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 340	<code><categorySeries></code> → Abre una serie del dataset categórico; cada fila del subdataset aportará categoría y valor.
Línea 341	<code><seriesExpression><![CDATA["Importe"]]></seriesExpression></code> → Define el nombre lógico de la serie que aparecerá en la leyenda.
Línea 342	<code><categoryExpression><![CDATA[\${categoria_grafico}]]></categoryExpression></code> → Define la categoría del eje X a partir de field <code>categoria_grafico</code> .
Línea 343	<code><valueExpression><![CDATA[\${importe_categoria}]]></valueExpression></code> → Define el valor numérico representado por cada barra a partir de field <code>importe_categoria</code> .
Línea 344	<code></categorySeries></code> → Cierra <code>categorySeries</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 345	<code></categoryDataset></code> → Cierra <code>categoryDataset</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 346	<code><barPlot></code> → Abre el plot específico del gráfico de barras, donde se configuran etiquetas y ejes.
Línea 347	<code><plot/></code> → Declara el bloque base del plot; mantiene la configuración visual por defecto del checkpoint.
Línea 348	<code><itemLabel/></code> → Habilita el bloque de configuración de etiquetas de los ítems/barras.
Línea 349	<code><categoryAxisFormat><axisFormat/></categoryAxisFormat></code> → Composición de la línea: encadena además <code><categoryAxisFormat></code> , <code><axisFormat></code> dentro de la misma jerarquía.
Línea 350	<code><valueAxisFormat><axisFormat/></valueAxisFormat></code> → Composición de la línea: encadena además <code><valueAxisFormat></code> , <code><axisFormat></code> dentro de la misma jerarquía.
Línea 351	<code></barPlot></code> → Cierra <code>barPlot</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 352	<code></barChart></code> → Finaliza el gráfico de barras.
Línea 353	<code><staticText><reportElement x="0" y="430" width="555" height="20" style="Cabecera"/><text><![CDATA[Ventas por categoría y año]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=430, ancho=555, alto=20 y aplica el estilo <code>Cabecera</code> ; muestra el literal 'Ventas por categoría y año'.
Línea 354	<code><crosstab></code> → Abre la tabla cruzada nativa que genera dinámicamente la matriz de filas, columnas, medidas y totales.
Línea 355	<code><reportElement x="0" y="455" width="555" height="225"/></code> → Posiciona el elemento en x=0, y=455, con ancho 555 y alto 225.
Línea 356	<code><crosstabDataset></code> → Abre la fuente de datos específica del crosstab.
Línea 357	<code><dataset></code> → Abre el contenedor de ejecución de datos del componente actual.
Línea 358	<code><datasetRun subDataset="DatasetCrosstabVentas"></code> → Asocia el componente con el subdataset <code>DatasetCrosstabVentas</code> para ejecutar su consulta.
Línea 359	<code><connectionExpression><![CDATA[\${REPORT_CONNECTION}]]></connectionExpression></code> → Entrega al subreporte/subdataset la misma <code>REPORT_CONNECTION</code> usada por el informe maestro, evitando abrir otra conexión.
Línea 360	<code></datasetRun></code> → Cierra <code>datasetRun</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 361	<code></dataset></code> → Cierra <code>dataset</code> y vuelve al elemento padre de la jerarquía JRXML.

Línea 362	<code></crosstabDataset></code> → Cierra <code>crosstabDataset</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 363	<code><rowGroup name="CategoriaCross" width="150" totalPosition="End"></code> → Declara el grupo de filas <code>CategoriaCross</code> , ancho 150 y total en End.
Línea 364	<code><bucket class="java.lang.String"><bucketExpression><![CDATA[\${F{categoria_cross}}]></bucketExpression></bucket></code> → Composición de la línea: define la clave de agrupación del bucket desde field <code>categoria_cross</code> ; encadena además <code><bucket></code> dentro de la misma jerarquía.
Línea 365	<code><crosstabRowHeader><cellContents style="M5CrosstabCabecera"><textField><reportElement x="0" y="0" width="150" height="34"/><textElement verticalAlignment="Middle"/><textFieldExp...></code> → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=0, y=0</code> , ancho=150, alto=34 y aplica el estilo <code>M5CrosstabCabecera</code> ; configura la alineación del texto (vertical Middle); evalúa la expresión usando variable <code>CategoriaCross</code> ; abre el contenido de celda con estilo <code>M5CrosstabCabecera</code> ; define la cabecera de fila del crosstab.
Línea 366	<code><crosstabTotalRowHeader><cellContents style="M5CrosstabTotal"><staticText><reportElement x="0" y="0" width="150" height="34"/><textElement verticalAlignment="Middle"/><text><![C...></code> → Composición de la línea: crea un texto literal; lo posiciona en <code>x=0, y=0</code> , ancho=150, alto=34 y aplica el estilo <code>M5CrosstabTotal</code> ; configura la alineación del texto (vertical Middle); muestra el literal 'TOTAL'; abre el contenido de celda con estilo <code>M5CrosstabTotal</code> ; define la cabecera de total de fila.
Línea 367	<code></rowGroup></code> → Cierra <code>rowGroup</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 368	<code><columnGroup name="AnioCross" height="28" totalPosition="End"></code> → Declara el grupo de columnas <code>AnioCross</code> , altura 28 y total en End.
Línea 369	<code><bucket class="java.lang.String"><bucketExpression><![CDATA[\${F{anio_cross}}]></bucketExpression></bucket></code> → Composición de la línea: define la clave de agrupación del bucket desde field <code>anio_cross</code> ; encadena además <code><bucket></code> dentro de la misma jerarquía.
Línea 370	<code><crosstabColumnHeader><cellContents style="M5CrosstabCabecera"><textField><reportElement x="0" y="0" width="100" height="28"/><textElement textAlignment="Center" verticalAlignme...></code> → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=0, y=0</code> , ancho=100, alto=28 y aplica el estilo <code>M5CrosstabCabecera</code> ; configura la alineación del texto (horizontal Center, vertical Middle); evalúa la expresión usando variable <code>AnioCross</code> ; abre el contenido de celda con estilo <code>M5CrosstabCabecera</code> ; define la cabecera de columna del crosstab.
Línea 371	<code><crosstabTotalColumnHeader><cellContents style="M5CrosstabTotal"><staticText><reportElement x="0" y="0" width="100" height="28"/><textElement textAlignment="Center" verticalAlig...></code> → Composición de la línea: crea un texto literal; lo posiciona en <code>x=0, y=0</code> , ancho=100, alto=28 y aplica el estilo <code>M5CrosstabTotal</code> ; configura la alineación del texto (horizontal Center, vertical Middle); muestra el literal 'TOTAL'; abre el contenido de celda con estilo <code>M5CrosstabTotal</code> ; define la cabecera de total de columna.
Línea 372	<code></columnGroup></code> → Cierra <code>columnGroup</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 373	<code><measure name="ImporteCross" class="java.lang.Double" calculation="Sum"><measureExpression><![CDATA[\${F{importe_cross}}]></measureExpression></measure></code> → Composición de la línea: aporta a la medida el valor de field <code>importe_cross</code> ; encadena además <code><measure></code> dentro de la misma jerarquía.
Línea 374	<code><measure name="VentasCross" class="java.lang.Integer" calculation="Sum"><measureExpression><![CDATA[\${F{ventas_cross}}]></measureExpression></measure></code> → Composición de la línea: aporta a la medida el valor de field <code>ventas_cross</code> ; encadena además <code><measure></code> dentro de la misma jerarquía.
Línea 375	<code><crosstabCell width="100" height="34"><cellContents style="M5CrosstabDetalle"><textField pattern="# ,##0.00 €"><reportElement x="0" y="0" width="100" height="18"/><textElement te...></code> → Composición de la línea: crea un <code>textField</code> dinámico con patrón <code># ,##0.00 €</code> ; lo posiciona en <code>x=0, y=0</code> , ancho=100, alto=34 y aplica el estilo <code>M5CrosstabDetalle</code> ; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable <code>ImporteCross</code> , variable <code>VentasCross</code> ; abre el contenido de celda con estilo <code>M5CrosstabDetalle</code> ; define la celda de detalle del crosstab.
Línea 376	<code><crosstabCell width="100" height="34" rowTotalGroup="CategoriaCross"><cellContents style="M5CrosstabTotal"><textField pattern="# ,##0.00 €"><reportElement x="0" y="0" width="100" ...></code> → Composición de la línea: crea un <code>textField</code> dinámico con patrón <code># ,##0.00 €</code> ; lo posiciona en <code>x=0, y=0</code> , ancho=100, alto=34 y aplica el estilo <code>M5CrosstabTotal</code> ; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable <code>ImporteCross</code> , variable <code>VentasCross</code> ; abre el contenido de celda con estilo <code>M5CrosstabTotal</code> ; define una celda de total de fila para <code>CategoriaCross</code> .

Línea 377	<code><crosstabCell width="100" height="34" columnTotalGroup="AnioCross"><cellContents style="M5CrosstabTotal"><textField pattern="#,##0.00 €"><reportElement x="0" y="0" width="100" h... → Composición de la línea: crea un textField dinámico con patrón #,##0.00 €; lo posiciona en x=0, y=0, ancho=100, alto=34 y aplica el estilo M5CrosstabTotal; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable ImporteCross, variable VentasCross; abre el contenido de celda con estilo M5CrosstabTotal; define una celda de total de columna para AnioCross.</code>
Línea 378	<code><crosstabCell width="100" height="34" rowTotalGroup="CategoriaCross" columnTotalGroup="AnioCross"><cellContents style="M5CrosstabTotal"><textField pattern="#,##0.00 €"><reportEl... → Composición de la línea: crea un textField dinámico con patrón #,##0.00 €; lo posiciona en x=0, y=0, ancho=100, alto=34 y aplica el estilo M5CrosstabTotal; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable ImporteCross, variable VentasCross; abre el contenido de celda con estilo M5CrosstabTotal; define la celda de total general.</code>
Línea 379	<code></crosstab> → Finaliza la tabla cruzada.</code>
Línea 380	<code></band> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.</code>
Línea 381	<code></summary> → Finaliza la sección Summary.</code>
Línea 382	<code></jasperReport> → Finaliza la definición completa del informe JasperReports.</code>

Subinforme heredado y ejecutable

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/
jasperreport.xsd"
  name="subinforme_ventas_detalle"
  language="java"
  pageWidth="555"
  pageHeight="842"
  columnWidth="555"
  leftMargin="0"
  rightMargin="0"
  topMargin="0"
  bottomMargin="0">
  <property name="com.jaspersoft.studio.data.defaultdataadapter" value="SQLiteEditorial"/>
  <style name="SubBase" isDefault="true" fontName="DejaVu Sans" fontSize="8"/>
  <style name="SubHeader" style="SubBase" mode="Opaque" backgroundColor="#EAF2F8" forecolor="#173F6B" isBold="true"/>
  <parameter name="tituloLibro" class="java.lang.String"/>
  <queryString language="sql">
    <![CDATA[
      SELECT fecha_venta, cantidad, precio_unitario
      FROM ventas
      WHERE titulo_libro = $P{tituloLibro}
      ORDER BY fecha_venta
    ]]>
  </queryString>
  <field name="fecha_venta" class="java.lang.String"/>
  <field name="cantidad" class="java.lang.Integer"/>
  <field name="precio_unitario" class="java.lang.Double"/>
  <columnHeader>
    <band height="18">
      <staticText><reportElement x="0" y="0" width="245" height="18" style="SubHeader"/><text><![CDATA[Fecha]]></text></staticText>
      <staticText><reportElement x="245" y="0" width="100" height="18" style="SubHeader"/><textElement textAlignment="Right"/>
    </staticText><![CDATA[Cantidad]]></text></staticText>
      <staticText><reportElement x="345" y="0" width="210" height="18" style="SubHeader"/><textElement textAlignment="Right"/>
    </staticText><![CDATA[Precio unitario]]></text></staticText>
    </band>
  </columnHeader>
  <detail>
    <band height="18">
      <textField><reportElement x="0" y="0" width="245" height="18"/><textFieldExpression><![CDATA[$F{fecha_venta}]]></textFieldExpression></textField>
      <textField><reportElement x="245" y="0" width="100" height="18"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[$F{cantidad}]]></textFieldExpression></textField>
      <textField pattern="#,##0.00 €"><reportElement x="345" y="0" width="210" height="18"/><textElement textAlignment="Right"/>
    </textFieldExpression><![CDATA[$F{precio_unitario}]]></textFieldExpression></textField>
    </band>
  </detail>
</jasperReport>
```

Explicación línea por línea

Línea 1	<code><?xml version="1.0" encoding="UTF-8"?></code> → Declara XML 1.0 y codificación UTF-8 para que nombres, textos y símbolos del informe se interpreten correctamente.
Línea 2	<code><jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports"</code> → Abre el documento raíz <code>jasperReport</code> del informe y fija el namespace principal de JasperReports.
Línea 3	<code>xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"</code> → Declara el namespace XML Schema Instance usado por <code>xsi:schemaLocation</code> para validar el documento.
Línea 4	<code>xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/jasperreport.xsd"</code> → Relaciona el namespace de JasperReports con su XSD para que Studio y el compilador validen la estructura.
Línea 5	<code>name="subinforme_ventas_detalle"</code> → Asigna al documento JasperReports el nombre interno <code>subinforme_ventas_detalle</code> .
Línea 6	<code>language="java"</code> → Configura <code>language=java</code> para evaluar expresiones con el lenguaje Java.
Línea 7	<code>pageWidth="555"</code> → Fija el ancho físico de página en 555 puntos.
Línea 8	<code>pageHeight="842"</code> → Fija la altura física de página en 842 puntos.
Línea 9	<code>columnWidth="555"</code> → Fija el ancho útil de la columna de contenido en 555 puntos.
Línea 10	<code>leftMargin="0"</code> → Fija el margen izquierdo del informe en 0 puntos.
Línea 11	<code>rightMargin="0"</code> → Fija el margen derecho del informe en 0 puntos.
Línea 12	<code>topMargin="0"</code> → Fija el margen superior del informe en 0 puntos.
Línea 13	<code>bottomMargin="0"></code> → Fija el margen inferior del informe en 0 puntos y completa la apertura del elemento raíz.
Línea 14	<code><property name="com.jaspersoft.studio.data.defaultdataadapter" value="SQLiteEditorial"/></code> → Indica a Jaspersoft Studio que use el Data Adapter <code>SQLiteEditorial</code> como conexión de diseño por defecto.
Línea 15	<code><style name="SubBase" isDefault="true" fontName="DejaVu Sans" fontSize="8"/></code> → Declara el estilo <code>SubBase</code> ; es el estilo por defecto, fuente <code>DejaVu Sans</code> , tamaño 8.
Línea 16	<code><style name="SubHeader" style="SubBase" mode="Opaque" backcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/></code> → Declara el estilo <code>SubHeader</code> ; hereda de <code>SubBase</code> , fondo <code>#EAF2F8</code> .
Línea 17	<code><parameter name="tituloLibro" class="java.lang.String"/></code> → Declara el parámetro <code>tituloLibro</code> con tipo <code>java.lang.String</code> y lo expone al diálogo de parámetros de Studio.
Línea 18	<code><queryString language="sql"></code> → Abre la consulta SQL que JasperReports ejecutará para el dataset actual.
Línea 19	<code><![CDATA[</code> → Abre CDATA para escribir SQL o una expresión Java sin que sus caracteres especiales se interpreten como XML.
Línea 20	<code>SELECT fecha_venta, cantidad, precio_unitario</code> → Cláusula SQL <code>SELECT</code> : selecciona y calcula las columnas que devolverá la consulta.
Línea 21	<code>FROM ventas</code> → Cláusula SQL <code>FROM</code> : define la tabla base de la consulta.
Línea 22	<code>WHERE titulo_libro = \${tituloLibro}</code> → Cláusula SQL <code>WHERE</code> : aplica el filtro de filas.
Línea 23	<code>ORDER BY fecha_venta</code> → Cláusula SQL <code>ORDER BY</code> : ordena el resultado que recibirá JasperReports.
Línea 24	<code>]]></code> → Cierra el bloque CDATA y devuelve el control al parser XML.
Línea 25	<code></queryString></code> → Cierra la consulta SQL del dataset actual.
Línea 26	<code><field name="fecha_venta" class="java.lang.String"/></code> → Declara el field <code>fecha_venta</code> con tipo Java <code>java.lang.String</code> para mapear una columna del dataset.
Línea 27	<code><field name="cantidad" class="java.lang.Integer"/></code> → Declara el field <code>cantidad</code> con tipo Java <code>java.lang.Integer</code> para mapear una columna del dataset.

Línea 28	<code><field name="precio_unitario" class="java.lang.Double"/></code> → Declara el field <code>precio_unitario</code> con tipo <code>Java java.lang.Double</code> para mapear una columna del dataset.
Línea 29	<code><columnHeader></code> → Abre Column Header, repetida al comienzo de cada columna/página según la paginación.
Línea 30	<code><band height="18"></code> → Define una banda de 18 puntos, reservando ese espacio para sus elementos.
Línea 31	<code><staticText><reportElement x="0" y="0" width="245" height="18" style="SubHeader"/><text><![CDATA[Fecha]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en <code>x=0, y=0, ancho=245, alto=18</code> y aplica el estilo <code>SubHeader</code> ; muestra el literal 'Fecha'.
Línea 32	<code><staticText><reportElement x="245" y="0" width="100" height="18" style="SubHeader"/><textElement textAlignment="Right"/><text><![CDATA[Cantidad]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en <code>x=245, y=0, ancho=100, alto=18</code> y aplica el estilo <code>SubHeader</code> ; configura la alineación del texto (horizontal <code>Right</code>); muestra el literal 'Cantidad'.
Línea 33	<code><staticText><reportElement x="345" y="0" width="210" height="18" style="SubHeader"/><textElement textAlignment="Right"/><text><![CDATA[Precio unitario]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en <code>x=345, y=0, ancho=210, alto=18</code> y aplica el estilo <code>SubHeader</code> ; configura la alineación del texto (horizontal <code>Right</code>); muestra el literal 'Precio unitario'.
Línea 34	<code></band></code> → Cierra <code>band</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 35	<code></columnHeader></code> → Cierra <code>columnHeader</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 36	<code><detail></code> → Abre Detail, la sección que se repite para cada registro del dataset principal.
Línea 37	<code><band height="18"></code> → Define una banda de 18 puntos, reservando ese espacio para sus elementos.
Línea 38	<code><textField><reportElement x="0" y="0" width="245" height="18"/><textFieldExpression><![CDATA[{F{fecha_venta}}]></textFieldExpression></textField></code> → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=0, y=0, ancho=245, alto=18</code> ; evalúa la expresión usando field <code>fecha_venta</code> .
Línea 39	<code><textField><reportElement x="245" y="0" width="100" height="18"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[{F{cantidad}}]></textFieldExpression></textField></code> → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=245, y=0, ancho=100, alto=18</code> ; configura la alineación del texto (horizontal <code>Right</code>); evalúa la expresión usando field <code>cantidad</code> .
Línea 40	<code><textField pattern="#.##0.00 €"><reportElement x="345" y="0" width="210" height="18"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[{F{precio_unitario}}]></t...></code> → Composición de la línea: crea un <code>textField</code> dinámico con patrón <code>#.##0.00 €</code> ; lo posiciona en <code>x=345, y=0, ancho=210, alto=18</code> ; configura la alineación del texto (horizontal <code>Right</code>); evalúa la expresión usando field <code>precio_unitario</code> .
Línea 41	<code></band></code> → Cierra <code>band</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 42	<code></detail></code> → Finaliza la sección Detail del informe.
Línea 43	<code></jasperReport></code> → Finaliza la definición completa del informe JasperReports.

Plantilla JRTX heredada y ejecutable

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<jasperTemplate xmlns="http://jasperreports.sourceforge.net/jasperreports/template"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports/template http://jasperreports.sourceforge.net/
xsd/jasptemplate.xsd">
  <style name="M5TituloPrincipal" fontName="DejaVu Sans" fontSize="18" isBold="true" forecolor="#173F6B"/>
  <style name="M5GrupoCabecera" fontName="DejaVu Sans" fontSize="10" isBold="true" forecolor="#173F6B" mode="Opaque"
backcolor="#D6EAF8"/>
  <style name="M5TablaCabecera" fontName="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque"
backcolor="#EAF2F8"/>
  <style name="M5TablaDetalle" fontName="DejaVu Sans" fontSize="9"/>
  <style name="M5CrosstabCabecera" fontName="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque"
backcolor="#EAF2F8"/>
  <style name="M5CrosstabDetalle" fontName="DejaVu Sans" fontSize="9" mode="Opaque" backcolor="#FFFFFF"/>
  <style name="M5CrosstabTotal" fontName="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque"
backcolor="#D6EAF8"/>
</jasperTemplate>
```

Explicación línea por línea

Línea 1	<?xml version="1.0" encoding="UTF-8"?> → Declara XML 1.0 y codificación UTF-8 para que nombres, textos y símbolos del informe se interpreten correctamente.
Línea 2	<jasperTemplate xmlns="http://jasperreports.sourceforge.net/jasperreports/template" → Abre el documento raíz jasperTemplate de la plantilla JRTX que contiene estilos reutilizables.
Línea 3	xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" → Declara el namespace XML Schema Instance usado por xsi:schemaLocation para validar el documento.
Línea 4	xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports/template http://jasperreports.sourceforge.net/xsd/jaspertemplate.xsd" → Relaciona el namespace de JasperReports con su XSD para que Studio y el compilador validen la estructura.
Línea 5	<style name="M5TituloPrincipal" fontFamily="DejaVu Sans" fontSize="18" isBold="true" forecolor="#173F6B" /> → Declara el estilo M5TituloPrincipal; fuente DejaVu Sans, tamaño 18.
Línea 6	<style name="M5GrupoCabecera" fontFamily="DejaVu Sans" fontSize="10" isBold="true" forecolor="#173F6B" mode="Opaque" backcolor="#D6EAF8" /> → Declara el estilo M5GrupoCabecera; fuente DejaVu Sans, tamaño 10, fondo #D6EAF8.
Línea 7	<style name="M5TablaCabecera" fontFamily="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque" backcolor="#EAF2F8" /> → Declara el estilo M5TablaCabecera; fuente DejaVu Sans, tamaño 9, fondo #EAF2F8.
Línea 8	<style name="M5TablaDetalle" fontFamily="DejaVu Sans" fontSize="9" /> → Declara el estilo M5TablaDetalle; fuente DejaVu Sans, tamaño 9.
Línea 9	<style name="M5CrosstabCabecera" fontFamily="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque" backcolor="#EAF2F8" /> → Declara el estilo M5CrosstabCabecera; fuente DejaVu Sans, tamaño 9, fondo #EAF2F8.
Línea 10	<style name="M5CrosstabDetalle" fontFamily="DejaVu Sans" fontSize="9" mode="Opaque" backcolor="FFFFFF" /> → Declara el estilo M5CrosstabDetalle; fuente DejaVu Sans, tamaño 9, fondo #FFFFFF.
Línea 11	<style name="M5CrosstabTotal" fontFamily="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque" backcolor="#D6EAF8" /> → Declara el estilo M5CrosstabTotal; fuente DejaVu Sans, tamaño 9, fondo #D6EAF8.
Línea 12	</jasperTemplate> → Finaliza la plantilla externa JRTX.

Parte C — Código y configuración ejecutable explicados línea por línea

GeneradorInformeVentas.java

JAVA

```
import java.io.File;
import java.sql.Connection;
import java.sql.DriverManager;
import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;
import net.sf.jasperreports.engine.JasperCompileManager;
import net.sf.jasperreports.engine.JasperFillManager;
import net.sf.jasperreports.engine.JasperPrint;
import net.sf.jasperreports.engine.export.JRPdfExporter;
import net.sf.jasperreports.export.SimpleExporterInput;
import net.sf.jasperreports.export.SimpleOutputStreamExporterOutput;
import net.sf.jasperreports.export.SimplePdfExporterConfiguration;
import net.sf.jasperreports.engine.data.JRCsvDataSource;
import net.sf.jasperreports.engine.export.ooxml.JRXlsxExporter;
import net.sf.jasperreports.export.SimpleXlsxReportConfiguration;
import net.sf.jasperreports.export.SimpleXlsxExporterConfiguration;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.nio.file.StandardCopyOption;
import net.sf.jasperreports.engine.export.HtmlExporter;
import net.sf.jasperreports.engine.export.FileHtmlResourceHandler;
import net.sf.jasperreports.export.SimpleHtmlExporterConfiguration;
import net.sf.jasperreports.export.SimpleHtmlExporterOutput;

public class GeneradorInformeVentas {
    public static void main(String[] args) {
        try {
            String rutaJrxml = "reports/informe_ventas.jrxml";
```

```

String rutaJasper = "reports/informe_ventas.jasper";
String rutaPdf = "output/informe_ventas.pdf";
String rutaPdfProtegido = "output/informe_ventas_protegido.pdf";
String rutaXlsx = "output/informe_ventas.xlsx";
String rutaCatalogoJrxml = "reports/informe_catalogo_csv.jrxml";
String rutaCatalogoJasper = "reports/informe_catalogo_csv.jasper";
String rutaXlsxCatalogo = "output/informe_catalogo.xlsx";
String rutaHtml = "output/informe_ventas.html";
String urlBD = "jdbc:sqlite:../EditorialReportsJava/data/editorial.db";
new File("output").mkdirs();

String rutaSubJrxml = "reports/subinforme_ventas_detalle.jrxml";
String rutaSubJasper = "reports/subinforme_ventas_detalle.jasper";
JasperCompileManager.compileReportToFile(rutaSubJrxml, rutaSubJasper);
JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);
JasperCompileManager.compileReportToFile(rutaCatalogoJrxml, rutaCatalogoJasper);

Map<String, Object> parametros = new HashMap<String, Object>();
parametros.put("usuario", "Ana Martínez");
parametros.put("departamento", "Comercial");
parametros.put("periodo", "Septiembre 2026");
parametros.put("tipoIva", Double.valueOf(0.21d));
parametros.put("mostrarDetalle", Boolean.TRUE);
parametros.put("categoria", null);
parametros.put("precioMinimo", null);
parametros.put("precioMaximo", null);
parametros.put("umbralUnidades", Integer.valueOf(5));
parametros.put("textoBusqueda", null);
parametros.put("categoriasLista", Arrays.asList("Novela", "Realismo mágico", "Cuento", "Poesía"));

try (Connection conexion = DriverManager.getConnection(urlBD)) {
    JasperPrint documento = JasperFillManager.fillReport(rutaJasper, parametros, conexion);
    exportarPdf(documento, rutaPdf);
    exportarPdfProtegido(documento, rutaPdfProtegido);
    exportarXlsx(documento, rutaXlsx, "Ventas");
    JRCSVDataSource catalogoDataSource = new JRCSVDataSource(new File("data/catalogo.csv"), "UTF-8");
    try {
        catalogoDataSource.setFieldDelimiter(',');
        catalogoDataSource.setUseFirstRowAsHeader(true);
        JasperPrint documentoCatalogo = JasperFillManager.fillReport(
            rutaCatalogoJasper, new HashMap<String, Object>(), catalogoDataSource);
        exportarXlsx(documentoCatalogo, rutaXlsxCatalogo, "Catálogo");
    } finally {
        catalogoDataSource.close();
    }
    new File("output/images").mkdirs();
    new File("output/styles").mkdirs();
    Files.copy(Paths.get("resources/styles/editorial.css"), Paths.get("output/styles/editorial.css"),
        StandardCopyOption.REPLACE_EXISTING);
    exportarHtml(documento, rutaHtml);
    System.out.println("Informe PDF generado en: " + new File(rutaPdf).getAbsolutePath());
    System.out.println("Informe PDF protegido generado en: " + new File(rutaPdfProtegido).getAbsolutePath());
    System.out.println("Informe Excel generado en: " + new File(rutaXlsx).getAbsolutePath());
    System.out.println("Informe catálogo Excel generado en: " + new File(rutaXlsxCatalogo).getAbsolutePath());
    System.out.println("Informe HTML generado en: " + new File(rutaHtml).getAbsolutePath());
    System.out.println("Paginas del documento: " + documento.getPages().size());
    System.out.println("Parametro usuario: " + parametros.get("usuario"));
    System.out.println("M6 checkpoint generado correctamente");
}
} catch (Exception e) {
    e.printStackTrace();
    System.exit(1);
}

private static void exportarPdf(JasperPrint documento, String ruta) throws Exception {
    JRPdfExporter exportador = new JRPdfExporter();
    SimplePdfExporterConfiguration configuracion = new SimplePdfExporterConfiguration();
    configuracion.setMetadataTitle("Informe de Ventas - EditorialReports");
    configuracion.setMetadataAuthor("Departamento Comercial");
    configuracion.setMetadataSubject("Resumen de ventas del catálogo");
    configuracion.setMetadataKeywords("ventas, catálogo, libros, editorial");
    configuracion.setMetadataCreator("JasperReports 6.20.0");
    configuracion.setDisplayMetadataTitle(Boolean.TRUE);
    configuracion.setCompressed(Boolean.TRUE);
    exportador.setConfiguration(configuracion);
    exportador.setExporterInput(new SimpleExporterInput(documento));
    exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta));
    exportador.exportReport();
}

private static void exportarPdfProtegido(JasperPrint documento, String ruta) throws Exception {
    JRPdfExporter exportador = new JRPdfExporter();
    SimplePdfExporterConfiguration configuracion = new SimplePdfExporterConfiguration();
    configuracion.setMetadataTitle("Informe de Ventas Protegido - EditorialReports");
    configuracion.setEncrypted(Boolean.TRUE);
    configuracion.setUserPassword("editorial2026");
    configuracion.setOwnerPassword("editorial-admin");
    configuracion.setAllowedPermissionsHint("PRINTING|COPY|SCREENREADERS");
    exportador.setConfiguration(configuracion);
    exportador.setExporterInput(new SimpleExporterInput(documento));

```

```

        exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta));
        exportador.exportReport();
    }

    private static void exportarXlsx(JasperPrint documento, String ruta, String nombreHoja) throws Exception {
        JRXlsxExporter exportador = new JRXlsxExporter();
        SimpleXlsxReportConfiguration informe = new SimpleXlsxReportConfiguration();
        informe.setSheetNames(new String[]{nombreHoja});
        informe.setShowGridLines(Boolean.FALSE);
        informe.setCellLocked(Boolean.FALSE);
        informe.setCellHidden(Boolean.FALSE);
        informe.setDetectCellType(Boolean.TRUE);
        informe.setOnePagePerSheet(Boolean.FALSE);
        SimpleXlsxExporterConfiguration libro = new SimpleXlsxExporterConfiguration();
        libro.setCreateCustomPalette(Boolean.TRUE);
        exportador.setConfiguration(informe);
        exportador.setConfiguration(libro);
        exportador.setExporterInput(new SimpleExporterInput(documento));
        exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta));
        exportador.exportReport();
    }

    private static void exportarHtml(JasperPrint documento, String ruta) throws Exception {
        HtmlExporter exportador = new HtmlExporter();
        SimpleHtmlExporterConfiguration configuracion = new SimpleHtmlExporterConfiguration();
        configuracion.setHtmlHeader("<html><head><meta charset='UTF-8'>"
            + "<title>Informe de Ventas - EditorialReports</title>"
            + "<link rel='stylesheet' href='styles/editorial.css'>"
            + "</head><body><a class='enlace-pdf' href='informe_ventas.pdf'>Descargar PDF</a>");
        configuracion.setHtmlFooter("</body></html>");
        configuracion.setBetweenPagesHtml("<hr class='salto-pagina' />");
        SimpleHtmlExporterOutput salida = new SimpleHtmlExporterOutput(ruta, "UTF-8");
        salida.setImageHandler(new FileHtmlResourceHandler(new File("output/images"), "images/{0}"));
        exportador.setConfiguration(configuracion);
        exportador.setExporterInput(new SimpleExporterInput(documento));
        exportador.setExporterOutput(salida);
        exportador.exportReport();
    }
}

```

Explicación línea por línea

Línea 1	import java.io.File; → Importa java.io.File para gestionar rutas y crear la carpeta de salida.
Línea 2	import java.sql.Connection; → Importa java.sql.Connection para representar la conexión JDBC abierta contra SQLite.
Línea 3	import java.sql.DriverManager; → Importa java.sql.DriverManager para abrir la conexión JDBC a partir de la URL SQLite.
Línea 4	import java.util.Arrays; → Importa java.util.Arrays para construir la colección de categorías usada por el parámetro de lista.
Línea 5	import java.util.HashMap; → Importa java.util.HashMap para crear la implementación mutable del mapa de parámetros.
Línea 6	import java.util.Map; → Importa java.util.Map para tipar el mapa de parámetros que recibe JasperReports.
Línea 7	import net.sf.jasperreports.engine.JasperCompileManager; → Importa net.sf.jasperreports.engine.JasperCompileManager para compilar los JRXML a artefactos .jasper.
Línea 8	import net.sf.jasperreports.engine.JasperFillManager; → Importa net.sf.jasperreports.engine.JasperFillManager para llenar el informe compilado con parámetros y conexión.
Línea 9	import net.sf.jasperreports.engine.JasperPrint; → Importa net.sf.jasperreports.engine.JasperPrint para representar en memoria el documento ya paginado por JasperReports.
Línea 10	import net.sf.jasperreports.engine.export.JRPdfExporter; → Importa net.sf.jasperreports.engine.export.JRPdfExporter para usar la clase JRPdfExporter en el generador.
Línea 11	import net.sf.jasperreports.export.SimpleExporterInput; → Importa net.sf.jasperreports.export.SimpleExporterInput para usar la clase SimpleExporterInput en el generador.

Línea 12	<code>import net.sf.jasperreports.export.SimpleOutputStreamExporterOutput; → Importa net.sf.jasperreports.export.SimpleOutputStreamExporterOutput para usar la clase SimpleOutputStreamExporterOutput en el generador.</code>
Línea 13	<code>import net.sf.jasperreports.export.SimplePdfExporterConfiguration; → Importa net.sf.jasperreports.export.SimplePdfExporterConfiguration para usar la clase SimplePdfExporterConfiguration en el generador.</code>
Línea 14	<code>import net.sf.jasperreports.engine.data.JRCsvDataSource; → Importa net.sf.jasperreports.engine.data.JRCsvDataSource para usar la clase JRCsvDataSource en el generador.</code>
Línea 15	<code>import net.sf.jasperreports.engine.export.ooxml.JRXlsxExporter; → Importa net.sf.jasperreports.engine.export.ooxml.JRXlsxExporter para usar la clase JRXlsxExporter en el generador.</code>
Línea 16	<code>import net.sf.jasperreports.export.SimpleXlsxReportConfiguration; → Importa net.sf.jasperreports.export.SimpleXlsxReportConfiguration para usar la clase SimpleXlsxReportConfiguration en el generador.</code>
Línea 17	<code>import net.sf.jasperreports.export.SimpleXlsxExporterConfiguration; → Importa net.sf.jasperreports.export.SimpleXlsxExporterConfiguration para usar la clase SimpleXlsxExporterConfiguration en el generador.</code>
Línea 18	<code>import java.nio.file.Files; → Importa java.nio.file.Files para usar la clase Files en el generador.</code>
Línea 19	<code>import java.nio.file.Paths; → Importa java.nio.file.Paths para usar la clase Paths en el generador.</code>
Línea 20	<code>import java.nio.file.StandardCopyOption; → Importa java.nio.file.StandardCopyOption para usar la clase StandardCopyOption en el generador.</code>
Línea 21	<code>import net.sf.jasperreports.engine.export.HtmlExporter; → Importa net.sf.jasperreports.engine.export.HtmlExporter para usar la clase HtmlExporter en el generador.</code>
Línea 22	<code>import net.sf.jasperreports.engine.export.FileHtmlResourceHandler; → Importa net.sf.jasperreports.engine.export.FileHtmlResourceHandler para usar la clase FileHtmlResourceHandler en el generador.</code>
Línea 23	<code>import net.sf.jasperreports.export.SimpleHtmlExporterConfiguration; → Importa net.sf.jasperreports.export.SimpleHtmlExporterConfiguration para usar la clase SimpleHtmlExporterConfiguration en el generador.</code>
Línea 24	<code>import net.sf.jasperreports.export.SimpleHtmlExporterOutput; → Importa net.sf.jasperreports.export.SimpleHtmlExporterOutput para usar la clase SimpleHtmlExporterOutput en el generador.</code>
Línea 25	<code>`` → Separa visualmente dos bloques lógicos sin modificar la ejecución.</code>
Línea 26	<code>public class GeneradorInformeVentas { → Declara la clase ejecutable GeneradorInformeVentas que encapsula el generador del informe.</code>
Línea 27	<code>public static void main(String[] args) { → Declara main como punto de entrada de la aplicación Java; recibe los argumentos de línea de comandos aunque este ejemplo no los utiliza.</code>
Línea 28	<code>try { → Abre el bloque principal protegido: cualquier error de compilación, conexión, llenado o exportación será capturado por el catch final.</code>
Línea 29	<code>String rutaJrxml = "reports/informe_ventas.jrxml"; → Declara rutaJrxml con ruta del JRXML maestro que se compilará; el valor configurado es "reports/informe_ventas.jrxml".</code>
Línea 30	<code>String rutaJasper = "reports/informe_ventas.jasper"; → Declara rutaJasper con ruta del .jasper maestro que producirá la compilación; el valor configurado es "reports/informe_ventas.jasper".</code>
Línea 31	<code>String rutaPdf = "output/informe_ventas.pdf"; → Declara rutaPdf con ruta del PDF final exportado; el valor configurado es "output/informe_ventas.pdf".</code>
Línea 32	<code>String rutaPdfProtegido = "output/informe_ventas_protegido.pdf"; → Declara rutaPdfProtegido con valor de configuración usado por el generador; el valor configurado es "output/informe_ventas_protegido.pdf".</code>

Línea 33	<code>String rutaXlsx = "output/informe_ventas.xlsx";</code> → Declara <code>rutaXlsx</code> con valor de configuración usado por el generador; el valor configurado es <code>"output/informe_ventas.xlsx"</code> .
Línea 34	<code>String rutaCatalogoJrxml = "reports/informe_catalogo_csv.jrxml";</code> → Declara <code>rutaCatalogoJrxml</code> con valor de configuración usado por el generador; el valor configurado es <code>"reports/informe_catalogo_csv.jrxml"</code> .
Línea 35	<code>String rutaCatalogoJasper = "reports/informe_catalogo_csv.jasper";</code> → Declara <code>rutaCatalogoJasper</code> con valor de configuración usado por el generador; el valor configurado es <code>"reports/informe_catalogo_csv.jasper"</code> .
Línea 36	<code>String rutaXlsxCatalogo = "output/informe_catalogo.xlsx";</code> → Declara <code>rutaXlsxCatalogo</code> con valor de configuración usado por el generador; el valor configurado es <code>"output/informe_catalogo.xlsx"</code> .
Línea 37	<code>String rutaHtml = "output/informe_ventas.html";</code> → Declara <code>rutaHtml</code> con valor de configuración usado por el generador; el valor configurado es <code>"output/informe_ventas.html"</code> .
Línea 38	<code>String urlBD = "jdbc:sqlite:../EditorialReportsJava/data/editorial.db";</code> → Declara <code>urlBD</code> con URL JDBC de la base SQLite; el valor configurado es <code>"jdbc:sqlite:../EditorialReportsJava/data/editorial.db"</code> .
Línea 39	<code>new File("output").mkdirs();</code> → Crea la carpeta <code>output</code> si todavía no existe para evitar que la exportación falle por una ruta inexistente.
Línea 40	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 41	<code>String rutaSubJrxml = "reports/subinforme_ventas_detalle.jrxml";</code> → Declara <code>rutaSubJrxml</code> con ruta del JRXML del subinforme de detalle; el valor configurado es <code>"reports/subinforme_ventas_detalle.jrxml"</code> .
Línea 42	<code>String rutaSubJasper = "reports/subinforme_ventas_detalle.jasper";</code> → Declara <code>rutaSubJasper</code> con ruta del <code>.jasper</code> del subinforme compilado; el valor configurado es <code>"reports/subinforme_ventas_detalle.jasper"</code> .
Línea 43	<code>JasperCompileManager.compileReportToFile(rutaSubJrxml, rutaSubJasper);</code> → Compila el JRXML indicado en <code>rutaSubJrxml</code> y escribe el artefacto compilado en <code>rutaSubJasper</code> .
Línea 44	<code>JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);</code> → Compila el JRXML indicado en <code>rutaJrxml</code> y escribe el artefacto compilado en <code>rutaJasper</code> .
Línea 45	<code>JasperCompileManager.compileReportToFile(rutaCatalogoJrxml, rutaCatalogoJasper);</code> → Compila el JRXML indicado en <code>rutaCatalogoJrxml</code> y escribe el artefacto compilado en <code>rutaCatalogoJasper</code> .
Línea 46	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 47	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 48	<code>Map<String, Object> parametros = new HashMap<String, Object>();</code> → Crea el mapa tipado de parámetros que se entregará a <code>JasperFillManager.fillReport</code> .
Línea 49	<code>parametros.put("usuario", "Ana Martínez");</code> → Asigna al parámetro JasperReports <code>usuario</code> el valor Java <code>"Ana Martínez"</code> antes del llenado.
Línea 50	<code>parametros.put("departamento", "Comercial");</code> → Asigna al parámetro JasperReports <code>departamento</code> el valor Java <code>"Comercial"</code> antes del llenado.
Línea 51	<code>parametros.put("periodo", "Septiembre 2026");</code> → Asigna al parámetro JasperReports <code>periodo</code> el valor Java <code>"Septiembre 2026"</code> antes del llenado.
Línea 52	<code>parametros.put("tipoIva", Double.valueOf(0.21d));</code> → Asigna al parámetro JasperReports <code>tipoIva</code> el valor Java <code>Double.valueOf(0.21d)</code> antes del llenado.
Línea 53	<code>parametros.put("mostrarDetalle", Boolean.TRUE);</code> → Asigna al parámetro JasperReports <code>mostrarDetalle</code> el valor Java <code>Boolean.TRUE</code> antes del llenado.
Línea 54	<code>parametros.put("categoria", null);</code> → Asigna al parámetro JasperReports <code>categoria</code> el valor Java <code>null</code> antes del llenado.
Línea 55	<code>parametros.put("precioMinimo", null);</code> → Asigna al parámetro JasperReports <code>precioMinimo</code> el valor Java <code>null</code> antes del llenado.

Línea 56	<code>parametros.put("precioMaximo", null);</code> → Asigna al parámetro JasperReports <code>precioMaximo</code> el valor Java <code>null</code> antes del llenado.
Línea 57	<code>parametros.put("umbralUnidades", Integer.valueOf(5));</code> → Asigna al parámetro JasperReports <code>umbralUnidades</code> el valor Java <code>Integer.valueOf(5)</code> antes del llenado.
Línea 58	<code>parametros.put("textoBusqueda", null);</code> → Asigna al parámetro JasperReports <code>textoBusqueda</code> el valor Java <code>null</code> antes del llenado.
Línea 59	<code>parametros.put("categoriasLista", Arrays.asList("Novela", "Realismo mágico", "Cuento", "Poesía"));</code> → Asigna al parámetro JasperReports <code>categoriasLista</code> el valor Java <code>Arrays.asList("Novela", "Realismo mágico", "Cuento", "Poesía")</code> antes del llenado.
Línea 60	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 61	<code>try (Connection conexion = DriverManager.getConnection(urlBD)) {</code> → Abre la conexión SQLite mediante <code>DriverManager</code> dentro de un <code>try-with-resources</code> , por lo que <code>conexion</code> se cierra automáticamente al terminar el bloque.
Línea 62	<code>JasperPrint documento = JasperFillManager.fillReport(rutaJasper, parametros, conexion);</code> → Inicia el llenado del informe y guarda en <code>documento</code> el <code>JasperPrint</code> paginado que devolverá JasperReports.
Línea 63	<code>exportarPdf(documento, rutaPdf);</code> → Invoca la exportación PDF normal usando el <code>JasperPrint</code> ya llenado y la ruta principal de salida.
Línea 64	<code>exportarPdfProtegido(documento, rutaPdfProtegido);</code> → Genera una segunda salida PDF cifrada para validar contraseñas y permisos sin alterar el PDF normal.
Línea 65	<code>exportarXlsx(documento, rutaXlsx, "Ventas");</code> → Exporta el <code>JasperPrint</code> de ventas a <code>informe_ventas.xlsx</code> usando el nombre de hoja <code>Ventas</code> .
Línea 66	<code>JRCsvDataSource catalogoDataSource = new JRCsvDataSource(new File("data/catalogo.csv"), "UTF-8");</code> → Abre <code>data/catalogo.csv</code> como <code>datasource</code> JasperReports en UTF-8 para llenar el informe de catálogo sin JDBC.
Línea 67	<code>try {</code> → Abre el bloque principal protegido: cualquier error de compilación, conexión, llenado o exportación será capturado por el <code>catch</code> final.
Línea 68	<code>catalogoDataSource.setFieldDelimiter(',');</code> → Configura punto y coma como delimitador de campos CSV.
Línea 69	<code>catalogoDataSource.setUseFirstRowAsHeader(true);</code> → Indica que la primera fila del CSV contiene los nombres de los fields del informe de catálogo.
Línea 70	<code>JasperPrint documentoCatalogo = JasperFillManager.fillReport(</code> → Inicia el llenado del informe de catálogo y guarda el resultado paginado en <code>documentoCatalogo</code> .
Línea 71	<code>rutaCatalogoJasper, new HashMap<String, Object>(), catalogoDataSource);</code> → Completa <code>fillReport</code> pasando el catálogo compilado, un mapa de parámetros vacío y el <code>JRCsvDataSource</code> .
Línea 72	<code>exportarXlsx(documentoCatalogo, rutaXlsxCatalogo, "Catálogo");</code> → Exporta el <code>JasperPrint</code> del catálogo a <code>informe_catalogo.xlsx</code> usando la hoja <code>Catálogo</code> .
Línea 73	<code>} finally {</code> → Abre el bloque <code>finally</code> que se ejecutará siempre para liberar el <code>datasource</code> CSV aunque falle el llenado o la exportación.
Línea 74	<code>catalogoDataSource.close();</code> → Cierra explícitamente el <code>JRCsvDataSource</code> para liberar el lector del archivo de catálogo.
Línea 75	<code>}</code> → Cierra el bloque <code>try-with-resources</code> de la conexión JDBC.
Línea 76	<code>new File("output/images").mkdirs();</code> → Crea la carpeta física donde el handler HTML podrá escribir recursos de imagen.
Línea 77	<code>new File("output/styles").mkdirs();</code> → Crea la carpeta publicada de estilos que debe acompañar a <code>informe_ventas.html</code> .

Línea 78	<code>Files.copy(Paths.get("resources/styles/editorial.css"), Paths.get("output/styles/editorial.css"))</code> ; → Copia la hoja CSS fuente a la carpeta publicada junto al HTML, sustituyéndola si ya existe.
Línea 79	<code>StandardCopyOption.REPLACE_EXISTING</code> ; → Finaliza la copia del CSS indicando que una versión anterior debe reemplazarse para mantener la salida sincronizada.
Línea 80	<code>exportarHtml(documento, rutaHtml)</code> ; → Reutiliza el mismo <code>JasperPrint</code> para generar la salida HTML y sus recursos.
Línea 81	<code>System.out.println("Informe PDF generado en: " + new File(rutaPdf).getAbsolutePath());</code> ; → Escribe en la consola la evidencia "Informe PDF generado en: " + <code>new File(rutaPdf).getAbsolutePath()</code> , que queda registrada por el workflow E2E.
Línea 82	<code>System.out.println("Informe PDF protegido generado en: " + new File(rutaPdfProtegido).getAbsolutePath());</code> ; → Escribe en la consola la evidencia "Informe PDF protegido generado en: " + <code>new File(rutaPdfProtegido).getAbsolutePath()</code> , que queda registrada por el workflow E2E.
Línea 83	<code>System.out.println("Informe Excel generado en: " + new File(rutaXlsx).getAbsolutePath());</code> ; → Escribe en la consola la evidencia "Informe Excel generado en: " + <code>new File(rutaXlsx).getAbsolutePath()</code> , que queda registrada por el workflow E2E.
Línea 84	<code>System.out.println("Informe catálogo Excel generado en: " + new File(rutaXlsxCatalogo).getAbsolutePath());</code> ; → Escribe en la consola la evidencia "Informe catálogo Excel generado en: " + <code>new File(rutaXlsxCatalogo).getAbsolutePath()</code> , que queda registrada por el workflow E2E.
Línea 85	<code>System.out.println("Informe HTML generado en: " + new File(rutaHtml).getAbsolutePath());</code> ; → Escribe en la consola la evidencia "Informe HTML generado en: " + <code>new File(rutaHtml).getAbsolutePath()</code> , que queda registrada por el workflow E2E.
Línea 86	<code>System.out.println("Paginas del documento: " + documento.getPages().size());</code> ; → Escribe en la consola la evidencia "Paginas del documento: " + <code>documento.getPages().size()</code> , que queda registrada por el workflow E2E.
Línea 87	<code>System.out.println("Parametro usuario: " + parametros.get("usuario"))</code> ; → Escribe en la consola la evidencia "Parametro usuario: " + <code>parametros.get("usuario")</code> , que queda registrada por el workflow E2E.
Línea 88	<code>System.out.println("M6 checkpoint generado correctamente")</code> ; → Escribe en la consola la evidencia "M6 checkpoint generado correctamente", que queda registrada por el workflow E2E.
Línea 89	<code>}</code> → Cierra el bloque <code>try-with-resources</code> de la conexión JDBC.
Línea 90	<code>} catch (Exception e) {</code> → Cierra el bloque protegido y abre el manejador que captura cualquier excepción del proceso completo.
Línea 91	<code>e.printStackTrace()</code> ; → Imprime la traza completa de la excepción para que el fallo sea diagnosticable en local y en GitHub Actions.
Línea 92	<code>System.exit(1)</code> ; → Finaliza el proceso con código 1 para que CI marque la ejecución como fallida y no oculte el error.
Línea 93	<code>}</code> → Cierra el bloque <code>catch</code> o el bloque principal de control asociado a <code>main</code> .
Línea 94	<code>}</code> → Cierra el método <code>main</code> .
Línea 95	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 96	<code>private static void exportarPdf(JasperPrint documento, String ruta) throws Exception {</code> → Declara el helper privado <code>exportarPdf</code> , que encapsula una exportación concreta y propaga cualquier error al <code>catch</code> principal.
Línea 97	<code>JRPdfExporter exportador = new JRPdfExporter();</code> → Crea el exportador PDF avanzado que admite configuración documental y de seguridad.
Línea 98	<code>SimplePdfExporterConfiguration configuracion = new SimplePdfExporterConfiguration();</code> → Crea el objeto <code>SimplePdfExporterConfiguration</code> que recibirá las opciones específicas de esta exportación.

Línea 99	<code>configuracion.setMetadataTitle("Informe de Ventas - EditorialReports");</code> → Fija el título de los metadatos PDF con la API específica de <code>SimplePdfExporterConfiguration</code> .
Línea 100	<code>configuracion.setMetadataAuthor("Departamento Comercial");</code> → Fija el autor en los metadatos del PDF.
Línea 101	<code>configuracion.setMetadataSubject("Resumen de ventas del catálogo");</code> → Fija el asunto documental del PDF.
Línea 102	<code>configuracion.setMetadataKeywords("ventas, catálogo, libros, editorial");</code> → Fija las palabras clave que quedarán registradas en las propiedades del PDF.
Línea 103	<code>configuracion.setMetadataCreator("JasperReports 6.20.0");</code> → Registra JasperReports 6.20.0 como creador del PDF.
Línea 104	<code>configuracion.setDisplayMetadataTitle(Boolean.TRUE);</code> → Solicita a los lectores PDF que utilicen el título de metadatos cuando soporten esa preferencia.
Línea 105	<code>configuracion.setCompressed(Boolean.TRUE);</code> → Activa la compresión del PDF mediante la configuración del exportador.
Línea 106	<code>exportador.setConfiguration(configuracion);</code> → Asocia al exportador la configuración específica preparada en las líneas anteriores.
Línea 107	<code>exportador.setExporterInput(new SimpleExporterInput(documento));</code> → Entrega al exportador el <code>JasperPrint</code> documento ya llenado; no se vuelve a consultar la base de datos.
Línea 108	<code>exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta));</code> → Dirige la salida binaria del exportador al archivo indicado por el parámetro <code>ruta</code> .
Línea 109	<code>exportador.exportReport();</code> → Ejecuta la exportación con la entrada, salida y configuración ya asignadas; aquí se materializa el archivo.
Línea 110	<code>}</code> → Cierra el método <code>main</code> .
Línea 111	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 112	<code>private static void exportarPdfProtegido(JasperPrint documento, String ruta) throws Exception {</code> → Declara el helper privado <code>exportarPdfProtegido</code> , que encapsula una exportación concreta y propaga cualquier error al <code>catch</code> principal.
Línea 113	<code>JRPdfExporter exportador = new JRPdfExporter();</code> → Crea el exportador PDF avanzado que admite configuración documental y de seguridad.
Línea 114	<code>SimplePdfExporterConfiguration configuracion = new SimplePdfExporterConfiguration();</code> → Crea el objeto <code>SimplePdfExporterConfiguration</code> que recibirá las opciones específicas de esta exportación.
Línea 115	<code>configuracion.setMetadataTitle("Informe de Ventas Protegido - EditorialReports");</code> → Fija el título de los metadatos PDF con la API específica de <code>SimplePdfExporterConfiguration</code> .
Línea 116	<code>configuracion.setEncrypted(Boolean.TRUE);</code> → Activa el cifrado de la salida PDF protegida.
Línea 117	<code>configuracion.setUserPassword("editorial2026");</code> → Configura la contraseña de apertura que el E2E verifica con <code>pdftinfo -upw</code> .
Línea 118	<code>configuracion.setOwnerPassword("editorial-admin");</code> → Configura la contraseña de propietario del PDF protegido.
Línea 119	<code>configuracion.setAllowedPermissionsHint("PRINTING COPY SCREENREADERS");</code> → Declara los permisos PDF autorizados mediante la cadena de hints admitida por JasperReports.
Línea 120	<code>exportador.setConfiguration(configuracion);</code> → Asocia al exportador la configuración específica preparada en las líneas anteriores.
Línea 121	<code>exportador.setExporterInput(new SimpleExporterInput(documento));</code> → Entrega al exportador el <code>JasperPrint</code> documento ya llenado; no se vuelve a consultar la base de datos.
Línea 122	<code>exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta));</code> → Dirige la salida binaria del exportador al archivo indicado por el parámetro <code>ruta</code> .

Línea 123	<code>exportador.exportReport();</code> → Ejecuta la exportación con la entrada, salida y configuración ya asignadas; aquí se materializa el archivo.
Línea 124	<code>}</code> → Cierra el método <code>main</code> .
Línea 125	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 126	<code>private static void exportarXlsx(JasperPrint documento, String ruta, String nombreHoja) throws Exception {</code> → Declara el helper privado <code>exportarXlsx</code> , que encapsula una exportación concreta y propaga cualquier error al <code>catch</code> principal.
Línea 127	<code>JRXLsxExporter exportador = new JRXLsxExporter();</code> → Crea el exportador OOXML que escribirá el libro XLSX.
Línea 128	<code>SimpleXlsxReportConfiguration informe = new SimpleXlsxReportConfiguration();</code> → Crea la configuración de cómo el <code>JasperPrint</code> se distribuye en hojas y celdas XLSX.
Línea 129	<code>informe.setSheetNames(new String[]{nombreHoja});</code> → Asigna el nombre Ventas a la hoja; el E2E lo comprueba dentro de <code>xl/workbook.xml</code> .
Línea 130	<code>informe.setShowGridLines(Boolean.FALSE);</code> → Desactiva la cuadrícula predeterminada de la hoja Excel.
Línea 131	<code>informe.setCellLocked(Boolean.FALSE);</code> → Configura las celdas exportadas sin bloqueo adicional.
Línea 132	<code>informe.setCellHidden(Boolean.FALSE);</code> → Evita marcar como ocultas las celdas exportadas.
Línea 133	<code>informe.setDetectCellType(Boolean.TRUE);</code> → Pide al exportador detectar tipos numéricos/fecha en lugar de convertir indiscriminadamente a texto.
Línea 134	<code>informe.setOnePagePerSheet(Boolean.FALSE);</code> → Mantiene el informe en una misma hoja lógica en lugar de crear una hoja por página.
Línea 135	<code>SimpleXlsxExporterConfiguration libro = new SimpleXlsxExporterConfiguration();</code> → Crea la configuración propia del libro/exportador XLSX.
Línea 136	<code>libro.setCreateCustomPalette(Boolean.TRUE);</code> → Activa la paleta personalizada del exportador XLSX para reproducir mejor los colores.
Línea 137	<code>exportador.setConfiguration(informe);</code> → Aplica la configuración de informe XLSX: hoja, cuadrícula, bloqueo, tipos y paginación.
Línea 138	<code>exportador.setConfiguration(libro);</code> → Aplica además la configuración global del libro XLSX, incluida la paleta personalizada.
Línea 139	<code>exportador.setExporterInput(new SimpleExporterInput(documento));</code> → Entrega al exportador el <code>JasperPrint</code> <code>documento</code> ya llenado; no se vuelve a consultar la base de datos.
Línea 140	<code>exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta));</code> → Dirige la salida binaria del exportador al archivo indicado por el parámetro <code>ruta</code> .
Línea 141	<code>exportador.exportReport();</code> → Ejecuta la exportación con la entrada, salida y configuración ya asignadas; aquí se materializa el archivo.
Línea 142	<code>}</code> → Cierra el método <code>main</code> .
Línea 143	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 144	<code>private static void exportarHtml(JasperPrint documento, String ruta) throws Exception {</code> → Declara el helper privado <code>exportarHtml</code> , que encapsula una exportación concreta y propaga cualquier error al <code>catch</code> principal.
Línea 145	<code>HtmlExporter exportador = new HtmlExporter();</code> → Crea el exportador HTML vigente en JasperReports 6.20.0.
Línea 146	<code>SimpleHtmlExporterConfiguration configuracion = new SimpleHtmlExporterConfiguration();</code> → Crea el objeto <code>SimpleHtmlExporterConfiguration</code> que recibirá las opciones específicas de esta exportación.
Línea 147	<code>configuracion.setHtmlHeader("<html><head><meta charset='UTF-8'>")</code> → Define la cabecera HTML, incluyendo UTF-8, título y enlace a la hoja CSS externa.

Línea 148	+ "<title>Informe de Ventas - EditorialReports</title>" → Añade a la cabecera HTML el título visible en la pestaña/metadato del navegador.
Línea 149	+ "<link rel='stylesheet' href='styles/editorial.css'>" → Añade a la cabecera HTML el enlace relativo a styles/editorial.css.
Línea 150	+ "</head><body>Descargar PDF"; → Añade el enlace del reto Descargar PDF, apuntando al PDF generado en la misma carpeta output.
Línea 151	configuracion.setHtmlFooter("</body></html>"); → Define el cierre de body y html del documento exportado.
Línea 152	configuracion.setBetweenPagesHtml("<hr class='salto-pagina' />"); → Inserta el separador HTML que representa el cambio entre páginas del JasperPrint.
Línea 153	SimpleHtmlExporterOutput salida = new SimpleHtmlExporterOutput(ruta, "UTF-8"); → Crea la salida HTML con codificación UTF-8.
Línea 154	salida.setImageHandler(new FileHtmlResourceHandler(new File("output/images"), "images/{0}")); → Asocia un gestor de recursos para escribir imágenes en disco y generar sus URI relativas.
Línea 155	exportador.setConfiguration(configuracion); → Asocia al exportador la configuración específica preparada en las líneas anteriores.
Línea 156	exportador.setExporterInput(new SimpleExporterInput(documento)); → Entrega al exportador el JasperPrint documento ya llenado; no se vuelve a consultar la base de datos.
Línea 157	exportador.setExporterOutput(salida); → Conecta al exportador el objeto salida previamente configurado con codificación y/o recursos.
Línea 158	exportador.exportReport(); → Ejecuta la exportación con la entrada, salida y configuración ya asignadas; aquí se materializa el archivo.
Línea 159	} → Cierra el método main.
Línea 160	`` → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 161	} → Cierra la clase GeneradorInformeVentas.

editorial.css

CSS

```
body {
    margin: 24px;
    background: #ffffff;
    color: #173f6b;
    font-family: "DejaVu Sans", Arial, sans-serif;
}
.jrPage {
    margin: 0 auto 24px auto;
    box-shadow: 0 1px 8px rgba(0,0,0,.12);
}
.salto-pagina {
    border: 0;
    border-top: 1px solid #d6eaf8;
    margin: 24px 0;
}
.enlace-pdf {
    display: block;
    padding: 8px;
    background: #173f6b;
    color: #ffffff;
    text-align: center;
    text-decoration: none;
    font-family: "DejaVu Sans", Arial, sans-serif;
}
```

Explicación línea por línea

Línea 1	body { → Abre la regla CSS del selector body.
Línea 2	margin: 24px; → Asigna la propiedad CSS margin al valor 24px.

Línea 3	background: #ffffff; → Asigna la propiedad CSS background al valor #ffffff.
Línea 4	color: #173f6b; → Asigna la propiedad CSS color al valor #173f6b.
Línea 5	font-family: "DejaVu Sans", Arial, sans-serif; → Asigna la propiedad CSS font-family al valor "DejaVu Sans", Arial, sans-serif.
Línea 6	} → Cierra la regla CSS abierta.
Línea 7	.jrPage { → Abre la regla CSS del selector .jrPage.
Línea 8	margin: 0 auto 24px auto; → Asigna la propiedad CSS margin al valor 0 auto 24px auto.
Línea 9	box-shadow: 0 1px 8px rgba(0,0,0,.12); → Asigna la propiedad CSS box-shadow al valor 0 1px 8px rgba(0,0,0,.12).
Línea 10	} → Cierra la regla CSS abierta.
Línea 11	.salto-pagina { → Abre la regla CSS del selector .salto-pagina.
Línea 12	border: 0; → Asigna la propiedad CSS border al valor 0.
Línea 13	border-top: 1px solid #d6eaf8; → Asigna la propiedad CSS border-top al valor 1px solid #d6eaf8.
Línea 14	margin: 24px 0; → Asigna la propiedad CSS margin al valor 24px 0.
Línea 15	} → Cierra la regla CSS abierta.
Línea 16	.enlace-pdf { → Abre la regla CSS del selector .enlace-pdf.
Línea 17	display: block; → Asigna la propiedad CSS display al valor block.
Línea 18	padding: 8px; → Asigna la propiedad CSS padding al valor 8px.
Línea 19	background: #173f6b; → Asigna la propiedad CSS background al valor #173f6b.
Línea 20	color: #ffffff; → Asigna la propiedad CSS color al valor #ffffff.
Línea 21	text-align: center; → Asigna la propiedad CSS text-align al valor center.
Línea 22	text-decoration: none; → Asigna la propiedad CSS text-decoration al valor none.
Línea 23	font-family: "DejaVu Sans", Arial, sans-serif; → Asigna la propiedad CSS font-family al valor "DejaVu Sans", Arial, sans-serif.
Línea 24	} → Cierra la regla CSS abierta.

Parte D — Simulación del resultado y de la estructura del proyecto

D.1 — Flujo de exportación / diseño

TEXT

```
JasperPrint ventas
├── HtmlExporter
│   ├── header: charset + title + CSS + Descargar PDF
│   ├── footer
│   ├── separador entre páginas
│   └── FileHtmlResourceHandler -> images/{0}
```

Qué representa: la transformación funcional que debe existir al terminar 6.3.

Cómo verificarlo: comparar el flujo con la Parte C y ejecutar el generador; el JRXML/JRTX debe seguir siendo el heredado de M5/5.6.

D.2 — Estructura lógica en código y recursos

TEXT

```
resources/styles/editorial.css
├── body
└── .jrPage
```

```
└─ .salto-pagina
└─ .enlace-pdf
```

Qué representa: las clases, métodos y recursos que sustituyen en M6 al trabajo visual sobre bandas y componentes.

Cómo verificarlo: abrir Java/POM/CSS/properties según corresponda y contrastar nombres y tipos con la Parte C ejecutable.

D.3 — Archivos de salida

```
output/
└─ informe_ventas.html
└─ informe_ventas.pdf
└─ images/
└─ styles/editorial.css
```

TEXT

Qué representa: los artefactos acumulativos esperados en output.

Cómo verificarlo: ejecutar `GeneradorInformeVentas`, abrir cada formato con una herramienta compatible y contrastar los contratos automatizados del E2E.

D.4 — Árbol acumulativo del checkpoint

```
M6/6.3/
└─ EditorialReports/EXPORTACION_HTML.md
└─ EditorialReports/resources/styles/editorial.css
└─ EditorialReportsJava/src/GeneradorInformeVentas.java
```

TEXT

Evidencia E2E: run **36249131955**, commit **12a0eba90859a92b12578d59ae592ad17dac5fb6**, artifact runtime **10907968651**.

Invariantes: 14 libros, 9 ventas, 31 unidades, 633,40 € y 6 páginas en `informe_ventas`.

Errores comunes del ejercicio completo

Error	Causa	Solución
JRHtmlExporter no existe	Clase antigua/incorrecta	Usar <code>HtmlExporter</code>
CSS no carga	Ruta relativa incorrecta	Copiar a <code>output/styles/editorial.css</code>
Imágenes rotas	Handler mal configurado	Usar <code>FileHtmlResourceHandler</code> en el output
Falta el enlace al PDF	La cabecera no lo contiene	Incluir <code>href='informe_ventas.pdf'</code>
Acentos incorrectos	Salida sin UTF-8	Crear <code>SimpleHtmlExporterOutput</code> con UTF-8

Reto resuelto paso a paso

Enunciado original: añadir un enlace `Descargar PDF` en la cabecera HTML.

1. La cabecera importa `styles/editorial.css`.
2. Se añade `<a class='enlace-pdf' href='informe_ventas.pdf'>Descargar PDF</a>`.
3. El CSS define `.enlace-pdf`.
4. PDF y HTML se escriben en output, por lo que la URI relativa es válida.
5. El E2E verifica `href='informe_ventas.pdf'` y el texto `Descargar PDF`.

Resultado del reto: el HTML ofrece acceso directo al PDF de la misma ejecución.

Analogía final con el contexto de la editorial

HTML es la edición navegable del catálogo y el enlace al PDF es la puerta hacia su versión imprimible.

Resultado esperado

- `informe_ventas.html` válido.
- `output/styles/editorial.css` y `output/images/`.
- enlace Descargar PDF presente.
- PDF/XLSX anteriores conservados.
- `EXPORTACION_HTML.md` coherente con `HtmlExporter` y el handler.

Conclusión y enlace al siguiente punto

6.3 añade una salida web con recursos externos y preserva el acceso a la versión PDF del mismo informe.

Punto 6.4 — Exportación a CSV y otros formatos

Objetivos de aprendizaje

- Comprender las características del formato CSV y sus limitaciones.
- Utilizar el exportador JRCSVExporter con SimpleCsvExporterConfiguration.
- Configurar el separador de campos y la codificación del archivo CSV.
- Exportar a formato XML con JRXMLExporter y a RTF con JRRtfExporter.
- Combinar varios exportadores en una misma ejecución del programa.
- Documentar la exportación a CSV, XML y RTF del proyecto EditorialReports.

Parte A — Práctica visual/IDE verificada

Paso 1: Abrir 6.4 y comprobar las salidas acumuladas

Acciones:

1. Abrir `GeneradorInformeVentas.java` de 6.4.
2. Confirmar PDF, XLSX y HTML heredados.
3. No modificar JRXML/JRTX.

Verificación visual: el checkpoint parte físicamente de 6.3.

Qué hace: completa la operación «Abrir 6.4 y comprobar las salidas acumuladas» en el checkpoint 6.4.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.4.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.4.

Analogía: es como entregar el mismo catálogo a distintos departamentos en el formato de intercambio que cada uno utiliza.

Paso 2: Añadir imports CSV, XML, RTF y ODT

Acciones:

1. Importar JRCSVExporter, JRXMLExporter y JRRtfExporter.
2. Importar `net.sf.jasperreports.engine.export.oasis.JR0dtExporter`.
3. Importar SimpleCsvExporterConfiguration, SimpleWriterExporterOutput y SimpleXMLExporterOutput.

Verificación visual: Problems resuelve los cuatro exportadores.

Qué hace: completa la operación «Añadir imports CSV, XML, RTF y ODT» en el checkpoint 6.4.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.4.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.4.

Analogía: es como entregar el mismo catálogo a distintos departamentos en el formato de intercambio que cada uno utiliza.

Paso 3: Declarar las cuatro nuevas rutas

Acciones:

1. Añadir `output/informe_ventas.csv`.

2. Añadir `output/informe_ventas.xml`.
3. Añadir `output/informe_ventas.rtf`.
4. Añadir `output/informe_ventas.odt`.

Verificación visual: Source muestra las cuatro rutas después de HTML.

Qué hace: completa la operación «Declarar las cuatro nuevas rutas» en el checkpoint 6.4.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.4.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.4.

Analogía: es como entregar el mismo catálogo a distintos departamentos en el formato de intercambio que cada uno utiliza.

Paso 4: Crear `exportarCsv`

Acciones:

1. Crear `JRCsvExporter` y `SimpleCsvExporterConfiguration`.
2. Configurar `setFieldDelimiter(";")`, `setRecordDelimiter("\n")` y `setWriteBOM(Boolean.TRUE)`.
3. Asignar input y new `SimpleWriterExporterOutput(ruta, "UTF-8")`.

Verificación visual: la codificación se fija en el output y no mediante `setEncoding` en la configuración CSV.

Qué hace: completa la operación «Crear `exportarCsv`» en el checkpoint 6.4.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.4.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.4.

Analogía: es como entregar el mismo catálogo a distintos departamentos en el formato de intercambio que cada uno utiliza.

Paso 5: Crear `exportarXml`

Acciones:

1. Crear `JRXmlExporter`.
2. Crear `SimpleXmlExporterOutput(ruta, "UTF-8")`.
3. Activar `setEmbeddingImages(Boolean.TRUE)`.
4. Asignar input/output y ejecutar.

Verificación visual: el método XML utiliza un output específico y conserva UTF-8.

Qué hace: completa la operación «Crear `exportarXml`» en el checkpoint 6.4.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.4.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.4.

Analogía: es como entregar el mismo catálogo a distintos departamentos en el formato de intercambio que cada uno utiliza.

Paso 6: Crear `exportarRtf`

Acciones:

1. Crear `JRRtfExporter`.
2. Asignar `SimpleExporterInput(documento)`.
3. Usar `SimpleWriterExporterOutput(ruta, "UTF-8")`.
4. Ejecutar `exportReport()`.

Verificación visual: la codificación RTF queda en el writer output.

Qué hace: completa la operación «Crear exportarRtf» en el checkpoint 6.4.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.4.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.4.

Analogía: es como entregar el mismo catálogo a distintos departamentos en el formato de intercambio que cada uno utiliza.

Paso 7: Resolver el reto ODT con la clase correcta**Acciones:**

1. Crear `exportarOdt(JasperPrint documento, String ruta)`.
2. Instanciar `JR0dtExporter` del paquete `engine.export.oasis`.
3. Asignar input y `SimpleOutputStreamExporterOutput(ruta)`.
4. Ejecutar `exportReport()`.

Verificación visual: Source no usa el paquete incorrecto `engine.export.JR0dtExporter`.

Qué hace: completa la operación «Resolver el reto ODT con la clase correcta» en el checkpoint 6.4.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.4.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.4.

Analogía: es como entregar el mismo catálogo a distintos departamentos en el formato de intercambio que cada uno utiliza.

Paso 8: Invocar las cuatro exportaciones**Acciones:**

1. Después de HTML, invocar CSV, XML, RTF y ODT sobre documento.
2. No repetir el llenado del informe.
3. Añadir mensajes de consola para las cuatro rutas.

Verificación visual: todas las salidas parten del mismo JasperPrint de seis páginas.

Qué hace: completa la operación «Invocar las cuatro exportaciones» en el checkpoint 6.4.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.4.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.4.

Analogía: es como entregar el mismo catálogo a distintos departamentos en el formato de intercambio que cada uno utiliza.

Paso 9: Compilar el proyecto

Acciones:

1. Guardar el Java.
2. Ejecutar `mvn clean package`.
3. Revisar que `JR0dtExporter` y los demás exportadores compilan.

Verificación visual: Maven termina con BUILD SUCCESS.

Qué hace: completa la operación «Compilar el proyecto» en el checkpoint 6.4.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.4.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.4.

Analogía: es como entregar el mismo catálogo a distintos departamentos en el formato de intercambio que cada uno utiliza.

Paso 10: Ejecutar el generador multiformato

Acciones:

1. Ejecutar `GeneradorInformeVentas`.
2. Confirmar en Console PDF/PDF protegido/XLSX/HTML/CSV/XML/RTF/ODT.
3. Confirmar Paginas del documento: 6.

Verificación visual: la ejecución termina con M6 checkpoint generado correctamente.

Qué hace: completa la operación «Ejecutar el generador multiformato» en el checkpoint 6.4.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.4.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.4.

Analogía: es como entregar el mismo catálogo a distintos departamentos en el formato de intercambio que cada uno utiliza.

Paso 11: Validar CSV y XML

Acciones:

1. Abrir el CSV con un editor capaz de mostrar UTF-8 y verificar punto y coma.
2. Comprobar que el CSV contiene más de una línea.
3. Abrir el XML y confirmar que comienza con declaración XML.

Verificación visual: CSV y XML contienen datos y estructura reconocible.

Qué hace: completa la operación «Validar CSV y XML» en el checkpoint 6.4.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.4.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.4.

Analogía: es como entregar el mismo catálogo a distintos departamentos en el formato de intercambio que cada uno utiliza.

Paso 12: Validar RTF y ODT

Acciones:

1. Abrir RTF con un procesador de texto y comprobar el contenido.
2. Abrir ODT con LibreOffice Writer.
3. Confirmar que ambos archivos se generan sin reparación.

Verificación visual: RTF y ODT son artefactos reales y no simples archivos con extensión cambiada.

Qué hace: completa la operación «Validar RTF y ODT» en el checkpoint 6.4.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.4.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.4.

Analogía: es como entregar el mismo catálogo a distintos departamentos en el formato de intercambio que cada uno utiliza.

Paso 13: Documentar formatos adicionales

Acciones:

1. Crear/abrir EXPORTACION_OTROS.md.
2. Registrar CSV, XML, RTF y ODT con sus clases y rutas.
3. Explicar que las codificaciones CSV/RTF pertenecen al output.

Verificación visual: la documentación coincide con el checkpoint ejecutable y el reto ODT.

Qué hace: completa la operación «Documentar formatos adicionales» en el checkpoint 6.4.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.4.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.4.

Analogía: es como entregar el mismo catálogo a distintos departamentos en el formato de intercambio que cada uno utiliza.

Parte B — JRXML/JRTX completo explicado línea por línea

En M6 el diseño no cambia: estos tres archivos deben permanecer byte a byte iguales a M5/5.6.

Informe maestro heredado y ejecutable

```
<?xml version="1.0" encoding="UTF-8"?>
<jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/
jasperreport.xsd"
  name="informe_ventas"
  language="java"
  pageWidth="595"
  pageHeight="842"
  columnWidth="555"
  leftMargin="20"
  rightMargin="20"
  topMargin="20"
  bottomMargin="20"
  uuid="3d2c2bd7-3b93-4da9-8b60-6b3c45674c91">
  <property name="com.jaspersoft.studio.data.defaultdataadapter" value="SQLiteEditorial"/>
  <template><![CDATA["resources/styles/EditorialStyles.jrtx"]]></template>
```

XML

```

<style name="Sans_Normal" isDefault="true" fontName="DejaVu Sans" fontSize="10"/>
<style name="TituloPrincipal" style="Sans_Normal" fontSize="18" isBold="true" forecolor="#173F6B"/>
<style name="Cabecera" style="Sans_Normal" fontSize="9" isBold="true" forecolor="#173F6B"/>
<style name="Dato" style="Sans_Normal" fontSize="9"/>
<style name="UnidadesCondicional" style="Dato" isBold="true">
  <conditionalStyle>
    <conditionExpression><![CDATA[{$f{unidades_vendidas} != null && $P{umbralUnidades} != null &&
$F{unidades_vendidas}.intValue() >= $P{umbralUnidades}.intValue()}]></conditionExpression>
    <style forecolor="#1B5E20"/>
  </conditionalStyle>
  <conditionalStyle>
    <conditionExpression><![CDATA[{$f{unidades_vendidas} != null && $P{umbralUnidades} != null &&
$F{unidades_vendidas}.intValue() >= 3 && $F{unidades_vendidas}.intValue() < $P{umbralUnidades}.intValue()}]></conditionExpression>
    <style forecolor="#1D5D88"/>
  </conditionalStyle>
  <conditionalStyle>
    <conditionExpression><![CDATA[{$f{unidades_vendidas} == null || $F{unidades_vendidas}.intValue() < 3}]></conditionExpression>
    <style forecolor="#9D3429"/>
  </conditionalStyle>
</style>
<style name="M5TableHeader" style="Dato" mode="Opaque" bgcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/>
<style name="M5TableDetail" style="Dato"/>
<style name="M5CrossHeader" style="Dato" mode="Opaque" bgcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/>
<style name="M5CrossDetail" style="Dato" mode="Opaque" bgcolor="#FFFFFF"/>
<style name="M5CrossTotal" style="Dato" mode="Opaque" bgcolor="#D6EAF8" forecolor="#173F6B" isBold="true"/>
<subDataset name="DatasetTopVentas">
  <parameter name="tituloLibro" class="java.lang.String"/>
  <queryString language="sql">
    <![CDATA[
      SELECT fecha_venta, cantidad, precio_unitario
      FROM ventas
      WHERE titulo_libro = $P{tituloLibro}
      ORDER BY cantidad DESC, fecha_venta
      LIMIT 3
    ]]>
  </queryString>
  <field name="fecha_venta" class="java.lang.String"/>
  <field name="cantidad" class="java.lang.Integer"/>
  <field name="precio_unitario" class="java.lang.Double"/>
</subDataset>
<subDataset name="DatasetVentasPorCategoria">
  <queryString language="sql">
    <![CDATA[
      SELECT l.categoria AS categoria_grafico,
      COALESCE(SUM(v.cantidad * v.precio_unitario), 0.0) AS importe_categoria
      FROM libros l
      LEFT JOIN ventas v ON l.titulo = v.titulo_libro
      GROUP BY l.categoria
      ORDER BY l.categoria
    ]]>
  </queryString>
  <field name="categoria_grafico" class="java.lang.String"/>
  <field name="importe_categoria" class="java.lang.Double"/>
</subDataset>
<subDataset name="DatasetCrosstabVentas">
  <queryString language="sql">
    <![CDATA[
      SELECT l.categoria AS categoria_cross,
      SUBSTR(v.fecha_venta, 1, 4) AS anio_cross,
      (v.cantidad * v.precio_unitario) AS importe_cross,
      1 AS ventas_cross
      FROM ventas v
      JOIN libros l ON l.titulo = v.titulo_libro
      ORDER BY l.categoria, anio_cross, v.fecha_venta
    ]]>
  </queryString>
  <field name="categoria_cross" class="java.lang.String"/>
  <field name="anio_cross" class="java.lang.String"/>
  <field name="importe_cross" class="java.lang.Double"/>
  <field name="ventas_cross" class="java.lang.Integer"/>
</subDataset>
<parameter name="usuario" class="java.lang.String" isForPrompting="true"/>
<parameter name="fechaInforme" class="java.util.Date" isForPrompting="true">
  <defaultValueExpression><![CDATA[new java.util.Date()]]></defaultValueExpression>
</parameter>
<parameter name="departamento" class="java.lang.String" isForPrompting="true">
  <defaultValueExpression><![CDATA["General"]]></defaultValueExpression>
</parameter>
<parameter name="periodo" class="java.lang.String" isForPrompting="true">
  <defaultValueExpression><![CDATA["Mensual"]]></defaultValueExpression>
</parameter>
<parameter name="tipoIva" class="java.lang.Double" isForPrompting="true">
  <defaultValueExpression><![CDATA[Double.valueOf(0.21d)]]></defaultValueExpression>
</parameter>
<parameter name="mostrarDetalle" class="java.lang.Boolean" isForPrompting="true">
  <defaultValueExpression><![CDATA[Boolean.TRUE]]></defaultValueExpression>
</parameter>
<parameter name="categoria" class="java.lang.String" isForPrompting="true"/>
<parameter name="precioMinimo" class="java.lang.Double" isForPrompting="true"/>
<parameter name="precioMaximo" class="java.lang.Double" isForPrompting="true"/>
<parameter name="umbralUnidades" class="java.lang.Integer" isForPrompting="true">
  <defaultValueExpression><![CDATA[Integer.valueOf(5)]]></defaultValueExpression>

```

Página 135 de 225

```

</staticText>
<staticText><reportElement x="0" y="38" width="110" height="18" uuid="40000000-0000-4000-8000-000000000002"/><text><![
[CDATA[Generado por:]]></text></staticText>
<textField isBlankWhenNull="true"><reportElement x="110" y="38" width="160" height="18"
uuid="40000000-0000-4000-8000-000000000003"/><textFieldExpression><![CDATA[{P{usuario}}]></textFieldExpression></textField>
<staticText><reportElement x="300" y="38" width="80" height="18" uuid="40000000-0000-4000-8000-000000000004"/><text><![
[CDATA[Fecha:]]></text></staticText>
<textField pattern="dd/MM/yyyy"><reportElement x="380" y="38" width="175" height="18"
uuid="40000000-0000-4000-8000-000000000005"/><textFieldExpression><![CDATA[{P{fechaInforme}}]></textFieldExpression></textField>
<staticText><reportElement x="0" y="62" width="100" height="18" uuid="40000000-0000-4000-8000-000000000006"/><text><![
[CDATA[Departamento:]]></text></staticText>
<textField isBlankWhenNull="true"><reportElement x="100" y="62" width="170" height="18"
uuid="40000000-0000-4000-8000-000000000007"/><textFieldExpression><![CDATA[{P{departamento}}]></textFieldExpression></textField>
<staticText><reportElement x="300" y="62" width="70" height="18" uuid="40000000-0000-4000-8000-000000000008"/><text><![
[CDATA[Periodo:]]></text></staticText>
<textField isBlankWhenNull="true"><reportElement x="370" y="62" width="185" height="18"
uuid="40000000-0000-4000-8000-000000000009"/><textFieldExpression><![CDATA[{P{periodo}}]></textFieldExpression></textField>
<staticText><reportElement x="0" y="86" width="100" height="18" uuid="40000000-0000-4000-8000-000000000010"/><text><![
[CDATA[Búsqueda:]]></text></staticText>
<textField isBlankWhenNull="true"><reportElement x="100" y="86" width="170" height="18"
uuid="40000000-0000-4000-8000-000000000011"/><textFieldExpression><![CDATA[{P{textoBusqueda}} == null ||
{P{textoBusqueda}}.trim().isEmpty() ? "(todas)" : {P{textoBusqueda}}]></textFieldExpression></textField>
<staticText><reportElement x="300" y="86" width="90" height="18" uuid="40000000-0000-4000-8000-000000000012"/><text><![
[CDATA[Categorías:]]></text></staticText>
<textField textAdjust="StretchHeight"><reportElement x="390" y="86" width="165" height="34"
uuid="40000000-0000-4000-8000-000000000013"/><textFieldExpression><![CDATA[String.valueOf({P{categoriasLista}})]></
textFieldExpression></textField>
</band>
</title>
<columnHeader>
<band height="62">
<staticText><reportElement x="0" y="2" width="215" height="18" uuid="41000000-0000-4000-8000-000000000001" style="Cabecera"/
><text><![CDATA[{Titulo}]]></text></staticText>
<staticText><reportElement x="215" y="2" width="55" height="18" uuid="41000000-0000-4000-8000-000000000002"
style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[{Unid.}]]></text></staticText>
<staticText><reportElement x="280" y="2" width="90" height="18" uuid="41000000-0000-4000-8000-000000000003"
style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[{Importe}]]></text></staticText>
<staticText><reportElement x="380" y="2" width="65" height="18" uuid="41000000-0000-4000-8000-000000000004"
style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[{Precio med.}]]></text></staticText>
<staticText><reportElement x="455" y="2" width="100" height="18" uuid="41000000-0000-4000-8000-000000000005"
style="Cabecera"/><text><![CDATA[{Categoría}]]></text></staticText>
<staticText><reportElement x="0" y="24" width="130" height="18" uuid="41000000-0000-4000-8000-000000000006"
style="Cabecera"/><text><![CDATA[{Primera venta}]]></text></staticText>
<staticText><reportElement x="130" y="24" width="130" height="18" uuid="41000000-0000-4000-8000-000000000007"
style="Cabecera"/><text><![CDATA[{Última venta}]]></text></staticText>
<staticText><reportElement x="260" y="24" width="160" height="18" uuid="41000000-0000-4000-8000-000000000008"
style="Cabecera"/><text><![CDATA[{Periodo de ventas}]]></text></staticText>
<staticText>
<reportElement x="420" y="24" width="135" height="18" uuid="41000000-0000-4000-8000-000000000009" style="Cabecera">
<printWhenExpression><![CDATA[{Boolean.TRUE.equals({P{mostrarDetalle}})}]]></printWhenExpression>
</reportElement>
<textElement textAlignment="Right"/>
<text><![CDATA[{Importe con IVA}]]></text>
</staticText>
</band>
</columnHeader>
<detail>
<band height="82" splitType="Stretch">
<textField textAdjust="StretchHeight"><reportElement x="0" y="0" width="215" height="20"
uuid="42000000-0000-4000-8000-000000000001" style="Dato"/><textFieldExpression><![CDATA[{F{titulo}}]]></textFieldExpression></textField>
<textField isBlankWhenNull="true"><reportElement x="215" y="0" width="55" height="20"
uuid="42000000-0000-4000-8000-000000000002" style="UnidadesCondicionales"/><textElement textAlignment="Right"/><textFieldExpression><![
[CDATA[{F{unidades_vendidas}}]]></textFieldExpression></textField>
<textField pattern="#,##0.00 €" isBlankWhenNull="true"><reportElement x="280" y="0" width="90" height="20"
uuid="42000000-0000-4000-8000-000000000003" style="Dato"/><textElement textAlignment="Right"/><textFieldExpression><![
[CDATA[{F{importe_total}}]]></textFieldExpression></textField>
<textField isBlankWhenNull="true"><reportElement x="380" y="0" width="65" height="20"
uuid="42000000-0000-4000-8000-000000000004" style="Dato"/><textElement textAlignment="Right"/><textFieldExpression><![
[CDATA[{F{precio_medio}} == null ? "Sin datos" : new java.text.DecimalFormat("#0.00 '€').format({F{precio_medio}})]></
textFieldExpression></textField>
<textField isBlankWhenNull="true"><reportElement x="455" y="0" width="100" height="20"
uuid="42000000-0000-4000-8000-000000000005" style="Dato"/><textFieldExpression><![CDATA[{F{categoria}}]]></textFieldExpression></
textField>
<textField isBlankWhenNull="true"><reportElement x="0" y="24" width="130" height="18"
uuid="42000000-0000-4000-8000-000000000006" style="Dato"/><textFieldExpression><![CDATA[{F{primera_venta}}]]></textFieldExpression></
textField>
<textField isBlankWhenNull="true"><reportElement x="130" y="24" width="130" height="18"
uuid="42000000-0000-4000-8000-000000000007" style="Dato"/><textFieldExpression><![CDATA[{F{ultima_venta}}]]></textFieldExpression></
textField>
<textField><reportElement x="260" y="24" width="160" height="18" uuid="42000000-0000-4000-8000-000000000008" style="Dato"/
><textElement textAlignment="Center"/><textFieldExpression><![CDATA[{F{primera_venta}} == null ? "Sin ventas" : {F{primera_venta}} + " → "
+ {F{ultima_venta}}]]></textFieldExpression></textField>
<textField pattern="#,##0.00 €" isBlankWhenNull="true">
<reportElement x="420" y="24" width="135" height="18" uuid="42000000-0000-4000-8000-000000000009" style="Dato">
<printWhenExpression><![CDATA[{Boolean.TRUE.equals({P{mostrarDetalle}})}]]></printWhenExpression>
</reportElement>
<textElement textAlignment="Right"/>
<textFieldExpression><![CDATA[{F{importe_total}} == null || {P{tipoIva}} == null ? null :
Double.valueOf({F{importe_total}}.doubleValue()) * (1.0d + {P{tipoIva}}.doubleValue())]]></textFieldExpression>
</textField>
<textField><reportElement x="0" y="48" width="105" height="18" uuid="42000000-0000-4000-8000-000000000010" style="Dato"/
><textFieldExpression><![CDATA[{F{unidades_vendidas}} == null ? "Sin ventas" : ({F{unidades_vendidas}}.intValue()) >= 6 ? "Premium" :

```

```

($F{unidades_vendidas}.intValue() >= 3 ? "Estándar" : "Económico"))]]></textFieldExpression></textField>
<textField><reportElement x="105" y="48" width="185" height="18" uuid="42000000-0000-4000-8000-000000000011" style="Dato"/>
<textFieldExpression><![CDATA[$F{titulo} == null ? "" : $F{titulo}.trim().toUpperCase(java.util.Locale.ROOT)]]></textFieldExpression></textField>
<textField><reportElement x="290" y="48" width="80" height="18" uuid="42000000-0000-4000-8000-000000000012" style="Dato"/>
<textFieldExpression><![CDATA[$F{precio_medio} == null ? "-" : String.format(java.util.Locale.ROOT, "%.2f", Double.valueOf(Math.round($F{precio_medio}.doubleValue() * 100.0d) / 100.0d)]]></textFieldExpression></textField>
<textField><reportElement x="370" y="48" width="90" height="18" uuid="42000000-0000-4000-8000-000000000013" style="Dato"/>
<textFieldExpression><![CDATA[$F{primera_venta} == null || $F{ultima_venta} == null ? "-" : java.lang.Long.toString(java.time.temporal.ChronoUnit.DAYS.between(java.time.LocalDate.parse($F{primera_venta}), java.time.LocalDate.parse($F{ultima_venta}))) + " días"]></textFieldExpression></textField>
<textField><reportElement x="460" y="48" width="95" height="18" uuid="42000000-0000-4000-8000-000000000014" style="Dato"/>
<textFieldExpression><![CDATA[$F{unidades_vendidas} == null ? "0.0%" : String.format(java.util.Locale.ROOT, "%.1f%", Double.valueOf($F{unidades_vendidas}.doubleValue() / Math.max(1.0d, $P{umbralUnidades} == null ? 1.0d : $P{umbralUnidades}.doubleValue()) * 100.0d)]]></textFieldExpression></textField>
</band>
<band height="14">
<printWhenExpression><![CDATA[$F{unidades_vendidas} != null && $P{umbralUnidades} != null && $F{unidades_vendidas}.intValue() >= $P{umbralUnidades}.intValue()]]></printWhenExpression>
<textField><reportElement x="0" y="0" width="555" height="12" uuid="42000000-0000-4000-8000-000000000015"/>
<textFieldExpression><![CDATA["Fila destacada: " + $F{titulo} + " supera el umbral de " + $P{umbralUnidades} + " unidades"]></textFieldExpression></textField>
</band>
<band height="88" splitType="Stretch">
<printWhenExpression><![CDATA[$F{unidades_vendidas} != null]]></printWhenExpression>
<staticText>
<reportElement x="0" y="2" width="555" height="16" style="Cabecera"/>
<text><![CDATA[Detalle de ventas]]></text>
</staticText>
<subreport>
<reportElement x="0" y="22" width="555" height="60" isRemoveLineWhenBlank="true"/>
<subreportParameter name="tituloLibro">
<subreportParameterExpression><![CDATA[$F{titulo}]]></subreportParameterExpression>
</subreportParameter>
<connectionExpression><![CDATA[$P{REPORT_CONNECTION}]]></connectionExpression>
<subreportExpression><![CDATA["reports/subinforme_ventas_detalle.jasper"]></subreportExpression>
</subreport>
</band>
<band height="104" splitType="Stretch">
<printWhenExpression><![CDATA[$F{unidades_vendidas} != null]]></printWhenExpression>
<staticText>
<reportElement x="0" y="2" width="555" height="16" style="Cabecera"/>
<text><![CDATA[Top 3 ventas por cantidad]]></text>
</staticText>
<componentElement>
<reportElement x="0" y="22" width="555" height="76"/>
<c:table xmlns:c="http://jasperreports.sourceforge.net/jasperreports/components" xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports/components http://jasperreports.sourceforge.net/xsd/components.xsd">
<datasetRun subDataset="DatasetTopVentas">
<datasetParameter name="tituloLibro">
<datasetParameterExpression><![CDATA[$F{titulo}]]></datasetParameterExpression>
</datasetParameter>
<connectionExpression><![CDATA[$P{REPORT_CONNECTION}]]></connectionExpression>
</datasetRun>
<c:column width="255">
<c:columnHeader style="M5TablaCabecera" height="20"><staticText><reportElement x="0" y="0" width="255" height="20"/><text><![CDATA[Fecha]]></text></staticText></c:columnHeader>
<c:detailCell style="M5TablaDetalle" height="18"><textField><reportElement x="0" y="0" width="255" height="18"/>
<textFieldExpression><![CDATA[$F{fecha_venta}]]></textFieldExpression></textField></c:detailCell>
</c:column>
<c:column width="100">
<c:columnHeader style="M5TablaCabecera" height="20"><staticText><reportElement x="0" y="0" width="100" height="20"/><text><![CDATA[Cantidad]]></text></staticText></c:columnHeader>
<c:detailCell style="M5TablaDetalle" height="18"><textField><reportElement x="0" y="0" width="100" height="18"/>
<textFieldExpression><![CDATA[$F{cantidad}]]></textFieldExpression></textField></c:detailCell>
</c:column>
<c:column width="200">
<c:columnHeader style="M5TablaCabecera" height="20"><staticText><reportElement x="0" y="0" width="200" height="20"/><text><![CDATA[Precio unitario]]></text></staticText></c:columnHeader>
<c:detailCell style="M5TablaDetalle" height="18"><textField pattern="#.##0.00 €"><reportElement x="0" y="0" width="200" height="18"/><text><![CDATA[$F{precio_unitario}]]></text></textField></c:detailCell>
</c:column>
</c:table>
</componentElement>
</band>
</detail>
<pageFooter>
<band height="62">
<staticText><reportElement x="0" y="4" width="120" height="15" uuid="43000000-0000-4000-8000-000000000001"/><text><![CDATA[Total de títulos:]]></text></staticText>
<textField evaluationTime="Report"><reportElement x="120" y="4" width="60" height="15" uuid="43000000-0000-4000-8000-000000000002"/>
<textFieldExpression><![CDATA[$P{REPORT_COUNT}]]></textFieldExpression></textField>
<textField><reportElement x="190" y="28" width="180" height="15" uuid="43000000-0000-4000-8000-000000000003"/>
<textFieldExpression><![CDATA["Página " + $V{PAGE_NUMBER} + " de"]></textFieldExpression></textField>
<textField evaluationTime="Report"><reportElement x="375" y="28" width="35" height="15" uuid="43000000-0000-4000-8000-000000000004"/>
<textFieldExpression><![CDATA[$V{PAGE_NUMBER}]]></textFieldExpression></textField>
<staticText><reportElement x="300" y="4" width="120" height="15" uuid="43000000-0000-4000-8000-000000000005"/><text><![CDATA[Subtotal página:]]></text></staticText>
<textField pattern="#.##0.00 €"><reportElement x="420" y="4" width="135" height="15" uuid="43000000-0000-4000-8000-000000000006"/>
<textFieldExpression><![CDATA[$V{TotalPagina}]]></textFieldExpression></textField>

```

```

textFieldExpression></textField>
</band>
</pageFooter>
<summary>
<band height="700">
<staticText><reportElement x="0" y="5" width="205" height="18" uuid="44000000-0000-4000-8000-000000000001"/><text><![
[CDATA[Total de unidades vendidas:]]></text></staticText>
<textField><reportElement x="205" y="5" width="80" height="18" uuid="44000000-0000-4000-8000-000000000002"/><textElement
textAlignment="Right"/><textFieldExpression><![CDATA[${V{TotalUnidades}}]></textFieldExpression></textField>
<staticText><reportElement x="300" y="5" width="120" height="18" uuid="44000000-0000-4000-8000-000000000003"/><text><![
[CDATA[Importe total:]]></text></staticText>
<textField pattern="#,#0.00 €"><reportElement x="420" y="5" width="135" height="18"
uuid="44000000-0000-4000-8000-000000000004"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[${V{TotalImporte}}]></
textFieldExpression></textField>
<staticText><reportElement x="0" y="30" width="205" height="18" uuid="44000000-0000-4000-8000-000000000005"/><text><![
[CDATA[Precio medio agregado:]]></text></staticText>
<textField pattern="#,#0.00 €"><reportElement x="205" y="30" width="80" height="18"
uuid="44000000-0000-4000-8000-000000000006"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[${V{PrecioMedio}}]></
textFieldExpression></textField>
<staticText><reportElement x="300" y="30" width="120" height="18" uuid="44000000-0000-4000-8000-000000000007"/><text><![
[CDATA[Precio máximo:]]></text></staticText>
<textField pattern="#,#0.00 €"><reportElement x="420" y="30" width="135" height="18"
uuid="44000000-0000-4000-8000-000000000008"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[${V{PrecioMaximo}}]></
textFieldExpression></textField>
<staticText><reportElement x="0" y="55" width="205" height="18" uuid="44000000-0000-4000-8000-000000000009"/><text><![
[CDATA[Número de libros:]]></text></staticText>
<textField><reportElement x="205" y="55" width="80" height="18" uuid="44000000-0000-4000-8000-000000000010"/><textElement
textAlignment="Right"/><textFieldExpression><![CDATA[${V{NumeroLibros}}]></textFieldExpression></textField>
<staticText><reportElement x="300" y="55" width="120" height="18" uuid="44000000-0000-4000-8000-000000000011"/><text><![
[CDATA[Importe con IVA:]]></text></staticText>
<textField pattern="#,#0.00 €"><reportElement x="420" y="55" width="135" height="18"
uuid="44000000-0000-4000-8000-000000000012"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[${V{ImporteConIva}}]></
textFieldExpression></textField>
<textField><reportElement x="0" y="80" width="555" height="18" uuid="44000000-0000-4000-8000-000000000013"/><textElement
textAlignment="Center"/><textFieldExpression><![CDATA[String.format(java.util.Locale.ROOT, "Resumen: %d títulos · %d unidades · %.2f €",
${V{NumeroLibros}}, ${V{TotalUnidades}}, ${V{TotalImporte}})]></textFieldExpression></textField>
<textField><reportElement x="0" y="103" width="350" height="18" uuid="44000000-0000-4000-8000-000000000014"/><textElement
textAlignment="Center"><font fontName="DejaVu Sans" size="10" isBold="true"/></textElement><textFieldExpression><![
[CDATA[${V{TotalUnidades}} != null && ${P{umbralUnidades}} != null && ${V{TotalUnidades}.intValue() >= ${P{umbralUnidades}.intValue() ?
"Objetivo de ventas alcanzado" : "Objetivo de ventas pendiente"}]]></textFieldExpression></textField>
<textField><reportElement x="360" y="103" width="195" height="18" uuid="44000000-0000-4000-8000-000000000015"/>
><textFieldExpression><![CDATA["Resultados encontrados: " + ${V{REPORT_COUNT}}]]></textFieldExpression></textField>
<staticText><reportElement x="0" y="140" width="555" height="20" style="Cabecera"/><text><![CDATA[Ventas por categoría –
importe]]></text></staticText>
<barChart>
<chart>
<reportElement x="0" y="165" width="555" height="250"/>
<chartTitle><titleExpression><![CDATA["Ventas por categoría"]]></titleExpression></chartTitle>
<chartSubtitle/>
<chartLegend position="Bottom"/>
</chart>
<categoryDataset>
<dataset>
<datasetRun subDataset="DatasetVentasPorCategoria">
<connectionExpression><![CDATA[${P{REPORT_CONNECTION}}]></connectionExpression>
</datasetRun>
</dataset>
<categorySeries>
<seriesExpression><![CDATA["Importe"]]></seriesExpression>
<categoryExpression><![CDATA[${F{categoria_grafico}}]></categoryExpression>
<valueExpression><![CDATA[${F{importe_categoria}}]></valueExpression>
</categorySeries>
</categoryDataset>
<barPlot>
<plot/>
<itemLabel/>
<categoryAxisFormat><axisFormat/></categoryAxisFormat>
<valueAxisFormat><axisFormat/></valueAxisFormat>
</barPlot>
</barChart>
<staticText><reportElement x="0" y="430" width="555" height="20" style="Cabecera"/><text><![CDATA[Ventas por categoría y
año]]></text></staticText>
<crosstab>
<reportElement x="0" y="455" width="555" height="225"/>
<crosstabDataset>
<dataset>
<datasetRun subDataset="DatasetCrosstabVentas">
<connectionExpression><![CDATA[${P{REPORT_CONNECTION}}]></connectionExpression>
</datasetRun>
</dataset>
</crosstabDataset>
<rowGroup name="CategoriaCross" width="150" totalPosition="End">
<bucket class="java.lang.String"><bucketExpression><![CDATA[${F{categoria_cross}}]></bucketExpression></bucket>
<crosstabRowHeader><cellContents style="M5CrosstabCabecera"><textField><reportElement x="0" y="0" width="150"
height="34"/><textElement verticalAlignment="Middle"/><textFieldExpression><![CDATA[${V{CategoriaCross}}]></textFieldExpression></
textField></cellContents></crosstabRowHeader>
<crosstabTotalRowHeader><cellContents style="M5CrosstabTotal"><staticText><reportElement x="0" y="0" width="150"
height="34"/><textElement verticalAlignment="Middle"/><text><![CDATA[TOTAL]]></text></staticText></cellContents></
crosstabTotalRowHeader>
</rowGroup>
<columnGroup name="AnioCross" height="28" totalPosition="End">
<bucket class="java.lang.String"><bucketExpression><![CDATA[${F{anio_cross}}]></bucketExpression></bucket>

```

```

<crossstabColumnHeader><cellContents style="M5CrosstabCabecera"><textField><reportElement x="0" y="0" width="100"
height="28"/><textFieldExpression><![CDATA[$V{AnioCross}]]></textFieldExpression></cellContents></crossstabColumnHeader>
<crossstabTotalColumnHeader><cellContents style="M5CrosstabTotal"><staticText><reportElement x="0" y="0" width="100"
height="28"/><textFieldExpression><![CDATA[TOTAL]]></textFieldExpression></staticText></cellContents></crossstabTotalColumnHeader>
</columnGroup>
<measure name="ImporteCross" class="java.lang.Double" calculation="Sum"><measureExpression><![CDATA[$F{importe_cross}]]></measureExpression></measure>
<measure name="VentasCross" class="java.lang.Integer" calculation="Sum"><measureExpression><![CDATA[$F{ventas_cross}]]></measureExpression></measure>
<crossstabCell width="100" height="34"><cellContents style="M5CrosstabDetalle"><textField pattern="#,##0.00
€"><reportElement x="0" y="0" width="100" height="18"/><textFieldExpression><![CDATA[$V{ImporteCross}]]></textFieldExpression></textField><textField><reportElement x="0" y="18" width="100" height="14"/><textFieldExpression><![CDATA[$V{VentasCross} + " ventas"]></textFieldExpression></textField></cellContents></crossstabCell>
<crossstabCell width="100" height="34" rowTotalGroup="CategoriaCross"><cellContents style="M5CrosstabTotal"><textField pattern="#,##0.00 €"><reportElement x="0" y="0" width="100" height="18"/><textFieldExpression><![CDATA[$V{ImporteCross}]]></textFieldExpression></textField><textField><reportElement x="0" y="18" width="100" height="14"/><textFieldExpression><![CDATA[$V{VentasCross} + " ventas"]></textFieldExpression></textField></cellContents></crossstabCell>
<crossstabCell width="100" height="34" columnTotalGroup="AnioCross"><cellContents style="M5CrosstabTotal"><textField pattern="#,##0.00 €"><reportElement x="0" y="0" width="100" height="18"/><textFieldExpression><![CDATA[$V{ImporteCross}]]></textFieldExpression></textField><textField><reportElement x="0" y="18" width="100" height="14"/><textFieldExpression><![CDATA[$V{VentasCross} + " ventas"]></textFieldExpression></textField></cellContents></crossstabCell>
<crossstabCell width="100" height="34" rowTotalGroup="CategoriaCross" columnTotalGroup="AnioCross"><cellContents style="M5CrosstabTotal"><textField pattern="#,##0.00 €"><reportElement x="0" y="0" width="100" height="18"/><textFieldExpression><![CDATA[$V{ImporteCross}]]></textFieldExpression></textField><textField><reportElement x="0" y="18" width="100" height="14"/><textFieldExpression><![CDATA[$V{VentasCross} + " ventas"]></textFieldExpression></textField></cellContents></crossstabCell>
</band>
</summary>
</jasperReport>

```

Explicación línea por línea

Línea 1	<?xml version="1.0" encoding="UTF-8"?> → Declara XML 1.0 y codificación UTF-8 para que nombres, textos y símbolos del informe se interpreten correctamente.
Línea 2	<jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports" → Abre el documento raíz jasperReport del informe y fija el namespace principal de JasperReports.
Línea 3	xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" → Declara el namespace XML Schema Instance usado por xsi:schemaLocation para validar el documento.
Línea 4	xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/jasperreport.xsd" → Relaciona el namespace de JasperReports con su XSD para que Studio y el compilador validen la estructura.
Línea 5	name="informe_ventas" → Asigna al documento JasperReports el nombre interno informe_ventas.
Línea 6	language="java" → Configura language=java para evaluar expresiones con el lenguaje Java.
Línea 7	pageWidth="595" → Fija el ancho físico de página en 595 puntos.
Línea 8	pageHeight="842" → Fija la altura física de página en 842 puntos.
Línea 9	columnWidth="555" → Fija el ancho útil de la columna de contenido en 555 puntos.
Línea 10	leftMargin="20" → Fija el margen izquierdo del informe en 20 puntos.
Línea 11	rightMargin="20" → Fija el margen derecho del informe en 20 puntos.
Línea 12	topMargin="20" → Fija el margen superior del informe en 20 puntos.
Línea 13	bottomMargin="20" → Fija el margen inferior del informe en 20 puntos y completa la apertura del elemento raíz.
Línea 14	uuid="3d2c2bd7-3b93-4da9-8b60-6b3c45674c91"> → Asigna el UUID de diseño 3d2c2bd7-3b93-4da9-8b60-6b3c45674c91 para identificar de forma estable el informe en Studio.
Línea 15	<property name="com.jaspersoft.studio.data.defaultdataadapter" value="SQLiteEditorial"/> → Indica a Jaspersoft Studio que use el Data Adapter SQLiteEditorial como conexión de diseño por defecto.

Línea 16	<code><template><![CDATA["resources/styles/EditorialStyles.jrtx"]]></template></code> → Importa la plantilla externa cuya expresión CDATA devuelve <code>resources/styles/EditorialStyles.jrtx</code> .
Línea 17	<code><style name="Sans_Normal" isDefault="true" fontName="DejaVu Sans" fontSize="10"/></code> → Declara el estilo <code>Sans_Normal</code> ; es el estilo por defecto, fuente <code>DejaVu Sans</code> , tamaño 10.
Línea 18	<code><style name="TituloPrincipal" style="Sans_Normal" fontSize="18" isBold="true" forecolor="#173F6B"/></code> → Declara el estilo <code>TituloPrincipal</code> ; hereda de <code>Sans_Normal</code> , tamaño 18.
Línea 19	<code><style name="Cabecera" style="Sans_Normal" fontSize="9" isBold="true" forecolor="#173F6B"/></code> → Declara el estilo <code>Cabecera</code> ; hereda de <code>Sans_Normal</code> , tamaño 9.
Línea 20	<code><style name="Dato" style="Sans_Normal" fontSize="9"/></code> → Declara el estilo <code>Dato</code> ; hereda de <code>Sans_Normal</code> , tamaño 9.
Línea 21	<code><style name="UnidadesCondicional" style="Dato" isBold="true"></code> → Declara el estilo <code>UnidadesCondicional</code> ; hereda de <code>Dato</code> .
Línea 22	<code><conditionalStyle></code> → Abre una variante condicional del estilo; sólo se aplicará cuando su <code>conditionExpression</code> sea verdadera.
Línea 23	<code><conditionExpression><![CDATA[\$F{unidades_vendidas} != null && \$P{umbralUnidades} != null && \$F{unidades_vendidas}.intValue() >= \$P{umbralUnidades}.intValue()]]></conditionExpres...></code> → Define la condición booleana que activa el estilo condicional usando field <code>unidades_vendidas</code> , field <code>unidades_vendidas</code> , parámetro <code>umbralUnidades</code> , parámetro <code>umbralUnidades</code> .
Línea 24	<code><style forecolor="#1B5E20"/></code> → Declara el estilo <code>None</code> .
Línea 25	<code></conditionalStyle></code> → Cierra <code>conditionalStyle</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 26	<code><conditionalStyle></code> → Abre una variante condicional del estilo; sólo se aplicará cuando su <code>conditionExpression</code> sea verdadera.
Línea 27	<code><conditionExpression><![CDATA[\$F{unidades_vendidas} != null && \$P{umbralUnidades} != null && \$F{unidades_vendidas}.intValue() >= 3 && \$F{unidades_vendidas}.intValue() < \$P{umbralUnidades}.intValue()]]></conditionExpression></code> → Define la condición booleana que activa el estilo condicional usando field <code>unidades_vendidas</code> , field <code>unidades_vendidas</code> , parámetro <code>umbralUnidades</code> , parámetro <code>umbralUnidades</code> .
Línea 28	<code><style forecolor="#1D5D88"/></code> → Declara el estilo <code>None</code> .
Línea 29	<code></conditionalStyle></code> → Cierra <code>conditionalStyle</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 30	<code><conditionalStyle></code> → Abre una variante condicional del estilo; sólo se aplicará cuando su <code>conditionExpression</code> sea verdadera.
Línea 31	<code><conditionExpression><![CDATA[\$F{unidades_vendidas} == null \$F{unidades_vendidas}.intValue() < 3]]></conditionExpression></code> → Define la condición booleana que activa el estilo condicional usando field <code>unidades_vendidas</code> , field <code>unidades_vendidas</code> .
Línea 32	<code><style forecolor="#9D3429"/></code> → Declara el estilo <code>None</code> .
Línea 33	<code></conditionalStyle></code> → Cierra <code>conditionalStyle</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 34	<code></style></code> → Cierra <code>style</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 35	<code><style name="M5TableHeader" style="Dato" mode="Opaque" backcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/></code> → Declara el estilo <code>M5TableHeader</code> ; hereda de <code>Dato</code> , fondo <code>#EAF2F8</code> .
Línea 36	<code><style name="M5TableDetail" style="Dato"/></code> → Declara el estilo <code>M5TableDetail</code> ; hereda de <code>Dato</code> .
Línea 37	<code><style name="M5CrossHeader" style="Dato" mode="Opaque" backcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/></code> → Declara el estilo <code>M5CrossHeader</code> ; hereda de <code>Dato</code> , fondo <code>#EAF2F8</code> .
Línea 38	<code><style name="M5CrossDetail" style="Dato" mode="Opaque" backcolor="FFFFFF"/></code> → Declara el estilo <code>M5CrossDetail</code> ; hereda de <code>Dato</code> , fondo <code>FFFFFF</code> .
Línea 39	<code><style name="M5CrossTotal" style="Dato" mode="Opaque" backcolor="#D6EAF8" forecolor="#173F6B" isBold="true"/></code> → Declara el estilo <code>M5CrossTotal</code> ; hereda de <code>Dato</code> , fondo <code>#D6EAF8</code> .

Línea 40	<code><subDataset name="DatasetTopVentas"></code> → Declara el subdataset DatasetTopVentas, con consulta y fields propios independientes del dataset principal.
Línea 41	<code><parameter name="tituloLibro" class="java.lang.String"/></code> → Declara el parámetro tituloLibro con tipo java.lang.String y lo expone al diálogo de parámetros de Studio.
Línea 42	<code><queryString language="sql"></code> → Abre la consulta SQL que JasperReports ejecutará para el dataset actual.
Línea 43	<code><![CDATA[</code> → Abre CDATA para escribir SQL o una expresión Java sin que sus caracteres especiales se interpreten como XML.
Línea 44	<code>SELECT fecha_venta, cantidad, precio_unitario</code> → Cláusula SQL SELECT: selecciona y calcula las columnas que devolverá la consulta.
Línea 45	<code>FROM ventas</code> → Cláusula SQL FROM: define la tabla base de la consulta.
Línea 46	<code>WHERE titulo_libro = \$P{tituloLibro}</code> → Cláusula SQL WHERE: aplica el filtro de filas.
Línea 47	<code>ORDER BY cantidad DESC, fecha_venta</code> → Cláusula SQL ORDER BY: ordena el resultado que recibirá JasperReports.
Línea 48	<code>LIMIT 3</code> → Cláusula SQL LIMIT: limita el número de filas devueltas.
Línea 49	<code>]]></code> → Cierra el bloque CDATA y devuelve el control al parser XML.
Línea 50	<code></queryString></code> → Cierra la consulta SQL del dataset actual.
Línea 51	<code><field name="fecha_venta" class="java.lang.String"/></code> → Declara el field fecha_venta con tipo Java java.lang.String para mapear una columna del dataset.
Línea 52	<code><field name="cantidad" class="java.lang.Integer"/></code> → Declara el field cantidad con tipo Java java.lang.Integer para mapear una columna del dataset.
Línea 53	<code><field name="precio_unitario" class="java.lang.Double"/></code> → Declara el field precio_unitario con tipo Java java.lang.Double para mapear una columna del dataset.
Línea 54	<code></subDataset></code> → Finaliza el subdataset auxiliar y vuelve al nivel del informe.
Línea 55	<code><subDataset name="DatasetVentasPorCategoria"></code> → Declara el subdataset DatasetVentasPorCategoria, con consulta y fields propios independientes del dataset principal.
Línea 56	<code><queryString language="sql"></code> → Abre la consulta SQL que JasperReports ejecutará para el dataset actual.
Línea 57	<code><![CDATA[</code> → Abre CDATA para escribir SQL o una expresión Java sin que sus caracteres especiales se interpreten como XML.
Línea 58	<code>SELECT l.categoria AS categoria_grafico,</code> → Cláusula SQL SELECT: selecciona y calcula las columnas que devolverá la consulta.
Línea 59	<code>COALESCE(SUM(v.cantidad * v.precio_unitario), 0.0) AS importe_categoria</code> → Calcula o selecciona un valor SQL y lo expone con el alias importe_categoria, que después coincide con un field del subdataset.
Línea 60	<code>FROM libros l</code> → Cláusula SQL FROM: define la tabla base de la consulta.
Línea 61	<code>LEFT JOIN ventas v ON l.titulo = v.titulo_libro</code> → Cláusula SQL LEFT JOIN: une datos conservando las filas del lado izquierdo aunque no tengan ventas.
Línea 62	<code>GROUP BY l.categoria</code> → Cláusula SQL GROUP BY: agrupa las filas antes de evaluar las funciones agregadas.
Línea 63	<code>ORDER BY l.categoria</code> → Cláusula SQL ORDER BY: ordena el resultado que recibirá JasperReports.
Línea 64	<code>]]></code> → Cierra el bloque CDATA y devuelve el control al parser XML.
Línea 65	<code></queryString></code> → Cierra la consulta SQL del dataset actual.
Línea 66	<code><field name="categoria_grafico" class="java.lang.String"/></code> → Declara el field categoria_grafico con tipo Java java.lang.String para mapear una columna del dataset.
Línea 67	<code><field name="importe_categoria" class="java.lang.Double"/></code> → Declara el field importe_categoria con tipo Java java.lang.Double para mapear una columna del dataset.

Línea 68	<code></subDataset></code> → Finaliza el subdataset auxiliar y vuelve al nivel del informe.
Línea 69	<code><subDataset name="DatasetCrosstabVentas"></code> → Declara el subdataset DatasetCrosstabVentas, con consulta y fields propios independientes del dataset principal.
Línea 70	<code><queryString language="sql"></code> → Abre la consulta SQL que JasperReports ejecutará para el dataset actual.
Línea 71	<code><![CDATA[</code> → Abre CDATA para escribir SQL o una expresión Java sin que sus caracteres especiales se interpreten como XML.
Línea 72	<code>SELECT l.categoria AS categoria_cross,</code> → Cláusula SQL SELECT: selecciona y calcula las columnas que devolverá la consulta.
Línea 73	<code>SUBSTR(v.fecha_venta, 1, 4) AS anio_cross,</code> → Calcula o selecciona un valor SQL y lo expone con el alias anio_cross, que después coincide con un field del subdataset.
Línea 74	<code>(v.cantidad * v.precio_unitario) AS importe_cross,</code> → Calcula o selecciona un valor SQL y lo expone con el alias importe_cross, que después coincide con un field del subdataset.
Línea 75	<code>1 AS ventas_cross</code> → Calcula o selecciona un valor SQL y lo expone con el alias ventas_cross, que después coincide con un field del subdataset.
Línea 76	<code>FROM ventas v</code> → Cláusula SQL FROM: define la tabla base de la consulta.
Línea 77	<code>JOIN libros l ON l.titulo = v.titulo_libro</code> → Cláusula SQL JOIN: une las filas que cumplen la relación indicada.
Línea 78	<code>ORDER BY l.categoria, anio_cross, v.fecha_venta</code> → Cláusula SQL ORDER BY: ordena el resultado que recibirá JasperReports.
Línea 79	<code>]]></code> → Cierra el bloque CDATA y devuelve el control al parser XML.
Línea 80	<code></queryString></code> → Cierra la consulta SQL del dataset actual.
Línea 81	<code><field name="categoria_cross" class="java.lang.String"/></code> → Declara el field categoria_cross con tipo Java java.lang.String para mapear una columna del dataset.
Línea 82	<code><field name="anio_cross" class="java.lang.String"/></code> → Declara el field anio_cross con tipo Java java.lang.String para mapear una columna del dataset.
Línea 83	<code><field name="importe_cross" class="java.lang.Double"/></code> → Declara el field importe_cross con tipo Java java.lang.Double para mapear una columna del dataset.
Línea 84	<code><field name="ventas_cross" class="java.lang.Integer"/></code> → Declara el field ventas_cross con tipo Java java.lang.Integer para mapear una columna del dataset.
Línea 85	<code></subDataset></code> → Finaliza el subdataset auxiliar y vuelve al nivel del informe.
Línea 86	<code><parameter name="usuario" class="java.lang.String" isForPrompting="true"/></code> → Declara el parámetro usuario con tipo java.lang.String y lo expone al diálogo de parámetros de Studio.
Línea 87	<code><parameter name="fechaInforme" class="java.util.Date" isForPrompting="true"></code> → Declara el parámetro fechaInforme con tipo java.util.Date y lo expone al diálogo de parámetros de Studio.
Línea 88	<code><defaultValueExpression><![CDATA[new java.util.Date()]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 89	<code></parameter></code> → Cierra parameter y vuelve al elemento padre de la jerarquía JRXML.
Línea 90	<code><parameter name="departamento" class="java.lang.String" isForPrompting="true"></code> → Declara el parámetro departamento con tipo java.lang.String y lo expone al diálogo de parámetros de Studio.
Línea 91	<code><defaultValueExpression><![CDATA["General"]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 92	<code></parameter></code> → Cierra parameter y vuelve al elemento padre de la jerarquía JRXML.
Línea 93	<code><parameter name="periodo" class="java.lang.String" isForPrompting="true"></code> → Declara el parámetro periodo con tipo java.lang.String y lo expone al diálogo de parámetros de Studio.

Línea 94	<code><defaultValueExpression><![CDATA["Mensual"]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 95	<code></parameter></code> → Cierra parameter y vuelve al elemento padre de la jerarquía JRXML.
Línea 96	<code><parameter name="tipoIva" class="java.lang.Double" isForPrompting="true"></code> → Declara el parámetro tipoIva con tipo <code>java.lang.Double</code> y lo expone al diálogo de parámetros de Studio.
Línea 97	<code><defaultValueExpression><![CDATA[Double.valueOf(0.21d)]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 98	<code></parameter></code> → Cierra parameter y vuelve al elemento padre de la jerarquía JRXML.
Línea 99	<code><parameter name="mostrarDetalle" class="java.lang.Boolean" isForPrompting="true"></code> → Declara el parámetro mostrarDetalle con tipo <code>java.lang.Boolean</code> y lo expone al diálogo de parámetros de Studio.
Línea 100	<code><defaultValueExpression><![CDATA[Boolean.TRUE]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 101	<code></parameter></code> → Cierra parameter y vuelve al elemento padre de la jerarquía JRXML.
Línea 102	<code><parameter name="categoria" class="java.lang.String" isForPrompting="true"/></code> → Declara el parámetro categoria con tipo <code>java.lang.String</code> y lo expone al diálogo de parámetros de Studio.
Línea 103	<code><parameter name="precioMinimo" class="java.lang.Double" isForPrompting="true"/></code> → Declara el parámetro precioMinimo con tipo <code>java.lang.Double</code> y lo expone al diálogo de parámetros de Studio.
Línea 104	<code><parameter name="precioMaximo" class="java.lang.Double" isForPrompting="true"/></code> → Declara el parámetro precioMaximo con tipo <code>java.lang.Double</code> y lo expone al diálogo de parámetros de Studio.
Línea 105	<code><parameter name="umbralUnidades" class="java.lang.Integer" isForPrompting="true"></code> → Declara el parámetro umbralUnidades con tipo <code>java.lang.Integer</code> y lo expone al diálogo de parámetros de Studio.
Línea 106	<code><defaultValueExpression><![CDATA[Integer.valueOf(5)]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 107	<code></parameter></code> → Cierra parameter y vuelve al elemento padre de la jerarquía JRXML.
Línea 108	<code><parameter name="textoBusqueda" class="java.lang.String" isForPrompting="true"/></code> → Declara el parámetro textoBusqueda con tipo <code>java.lang.String</code> y lo expone al diálogo de parámetros de Studio.
Línea 109	<code><parameter name="categoriasLista" class="java.util.Collection" isForPrompting="false"></code> → Declara el parámetro categoriasLista con tipo <code>java.util.Collection</code> como parámetro interno no solicitado al usuario.
Línea 110	<code><defaultValueExpression><![CDATA[java.util.Arrays.asList("Novela", "Realismo mágico", "Cuento", "Poesía")]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 111	<code></parameter></code> → Cierra parameter y vuelve al elemento padre de la jerarquía JRXML.
Línea 112	<code><queryString language="sql"></code> → Abre la consulta SQL que JasperReports ejecutará para el dataset actual.
Línea 113	<code><![CDATA[</code> → Abre CDATA para escribir SQL o una expresión Java sin que sus caracteres especiales se interpreten como XML.
Línea 114	<code>SELECT l.titulo,</code> → Cláusula SQL SELECT: selecciona y calcula las columnas que devolverá la consulta.
Línea 115	<code>l.categoria,</code> → Aporta el valor literal <code>l.categoria</code> , al elemento XML/POM actualmente abierto.
Línea 116	<code>SUM(v.cantidad) AS unidades_vendidas,</code> → Calcula o selecciona un valor SQL y lo expone con el alias <code>unidades_vendidas</code> , que después coincide con un field del subdataset.
Línea 117	<code>SUM(v.cantidad * v.precio_unitario) AS importe_total,</code> → Calcula o selecciona un valor SQL y lo expone con el alias <code>importe_total</code> , que después coincide con un field del subdataset.
Línea 118	<code>AVG(v.precio_unitario) AS precio_medio,</code> → Calcula o selecciona un valor SQL y lo expone con el alias <code>precio_medio</code> , que después coincide con un field del subdataset.

Línea 119	<code>MIN(v.fecha_venta) AS primera_venta</code> , → Calcula o selecciona un valor SQL y lo expone con el alias <code>primera_venta</code> , que después coincide con un field del subdataset.
Línea 120	<code>MAX(v.fecha_venta) AS ultima_venta</code> → Calcula o selecciona un valor SQL y lo expone con el alias <code>ultima_venta</code> , que después coincide con un field del subdataset.
Línea 121	<code>FROM libros l</code> → Cláusula SQL <code>FROM</code> : define la tabla base de la consulta.
Línea 122	<code>LEFT JOIN ventas v ON l.titulo = v.titulo_libro</code> → Cláusula SQL <code>LEFT JOIN</code> : une datos conservando las filas del lado izquierdo aunque no tengan ventas.
Línea 123	<code>WHERE (\$P{categoria} IS NULL OR l.categoria = \$P{categoria})</code> → Cláusula SQL <code>WHERE</code> : aplica el filtro de filas.
Línea 124	<code>AND (\$P{precioMinimo} IS NULL OR l.precio >= \$P{precioMinimo})</code> → Añade otra condición lógica al filtro SQL, combinándola con la condición anterior.
Línea 125	<code>AND (\$P{precioMaximo} IS NULL OR l.precio <= \$P{precioMaximo})</code> → Añade otra condición lógica al filtro SQL, combinándola con la condición anterior.
Línea 126	<code>AND (\$P{textoBusqueda} IS NULL OR \$P{textoBusqueda} = '' OR l.titulo LIKE '%' \$P{textoBusqueda} '%')</code> → Añade otra condición lógica al filtro SQL, combinándola con la condición anterior.
Línea 127	<code>AND \$X{IN, l.categoria, categoriasLista}</code> → Añade otra condición lógica al filtro SQL, combinándola con la condición anterior.
Línea 128	<code>GROUP BY l.titulo, l.categoria</code> → Cláusula SQL <code>GROUP BY</code> : agrupa las filas antes de evaluar las funciones agregadas.
Línea 129	<code>ORDER BY l.categoria, COALESCE(importe_total, 0) DESC, l.titulo</code> → Cláusula SQL <code>ORDER BY</code> : ordena el resultado que recibirá JasperReports.
Línea 130	<code>]]></code> → Cierra el bloque <code>CDATA</code> y devuelve el control al parser XML.
Línea 131	<code></queryString></code> → Cierra la consulta SQL del dataset actual.
Línea 132	<code><field name="titulo" class="java.lang.String"/></code> → Declara el field <code>titulo</code> con tipo Java <code>java.lang.String</code> para mapear una columna del dataset.
Línea 133	<code><field name="categoria" class="java.lang.String"/></code> → Declara el field <code>categoria</code> con tipo Java <code>java.lang.String</code> para mapear una columna del dataset.
Línea 134	<code><field name="unidades_vendidas" class="java.lang.Integer"/></code> → Declara el field <code>unidades_vendidas</code> con tipo Java <code>java.lang.Integer</code> para mapear una columna del dataset.
Línea 135	<code><field name="importe_total" class="java.lang.Double"/></code> → Declara el field <code>importe_total</code> con tipo Java <code>java.lang.Double</code> para mapear una columna del dataset.
Línea 136	<code><field name="precio_medio" class="java.lang.Double"/></code> → Declara el field <code>precio_medio</code> con tipo Java <code>java.lang.Double</code> para mapear una columna del dataset.
Línea 137	<code><field name="primera_venta" class="java.lang.String"/></code> → Declara el field <code>primera_venta</code> con tipo Java <code>java.lang.String</code> para mapear una columna del dataset.
Línea 138	<code><field name="ultima_venta" class="java.lang.String"/></code> → Declara el field <code>ultima_venta</code> con tipo Java <code>java.lang.String</code> para mapear una columna del dataset.
Línea 139	<code><variable name="TotalUnidades" class="java.lang.Integer" calculation="Sum" resetType="Report"></code> → Declara la variable <code>TotalUnidades</code> con cálculo <code>Sum</code> y reinicio <code>Report</code> .
Línea 140	<code><variableExpression><![CDATA[\$F{unidades_vendidas}]]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field <code>unidades_vendidas</code> .
Línea 141	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 142	<code><variable name="TotalImporte" class="java.lang.Double" calculation="Sum" resetType="Report"></code> → Declara la variable <code>TotalImporte</code> con cálculo <code>Sum</code> y reinicio <code>Report</code> .

Línea 143	<code><variableExpression><![CDATA[{\$importe_total}]]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field importe_total.
Línea 144	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 145	<code><variable name="TotalPagina" class="java.lang.Double" calculation="Sum" resetType="Page"></code> → Declara la variable TotalPagina con cálculo Sum y reinicio Page.
Línea 146	<code><variableExpression><![CDATA[{\$importe_total}]]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field importe_total.
Línea 147	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 148	<code><variable name="PrecioMedio" class="java.lang.Double" calculation="Average" resetType="Report"></code> → Declara la variable PrecioMedio con cálculo Average y reinicio Report.
Línea 149	<code><variableExpression><![CDATA[{\$precio_medio}]]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field precio_medio.
Línea 150	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 151	<code><variable name="PrecioMaximo" class="java.lang.Double" calculation="Highest" resetType="Report"></code> → Declara la variable PrecioMaximo con cálculo Highest y reinicio Report.
Línea 152	<code><variableExpression><![CDATA[{\$precio_medio}]]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field precio_medio.
Línea 153	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 154	<code><variable name="NumeroLibros" class="java.lang.Integer" calculation="Count" resetType="Report"></code> → Declara la variable NumeroLibros con cálculo Count y reinicio Report.
Línea 155	<code><variableExpression><![CDATA[{\$titulo}]]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field titulo.
Línea 156	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 157	<code><variable name="ImporteConIva" class="java.lang.Double" calculation="Sum" resetType="Report"></code> → Declara la variable ImporteConIva con cálculo Sum y reinicio Report.
Línea 158	<code><variableExpression><![CDATA[{\$importe_total} == null \$P{tipoIva} == null ? null : Double.valueOf(\$F{importe_total}.doubleValue() * (1.0d + \$P{tipoIva}.doubleValue()))]]></v...></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field importe_total, field importe_total, parámetro tipoIva, parámetro tipoIva.
Línea 159	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 160	<code><variable name="GrupoUnidades" class="java.lang.Integer" calculation="Sum" resetType="Group" resetGroup="CategoriaGroup"></code> → Declara la variable GrupoUnidades con cálculo Sum y reinicio Group asociado a CategoriaGroup.
Línea 161	<code><variableExpression><![CDATA[{\$unidades_vendidas}]]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field unidades_vendidas.
Línea 162	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 163	<code><variable name="GrupoImporte" class="java.lang.Double" calculation="Sum" resetType="Group" resetGroup="CategoriaGroup"></code> → Declara la variable GrupoImporte con cálculo Sum y reinicio Group asociado a CategoriaGroup.
Línea 164	<code><variableExpression><![CDATA[{\$importe_total}]]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field importe_total.
Línea 165	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 166	<code><variable name="GrupoLibros" class="java.lang.Integer" calculation="Count" resetType="Group" resetGroup="CategoriaGroup"></code> → Declara la variable GrupoLibros con cálculo Count y reinicio Group asociado a CategoriaGroup.

Línea 167	<code><variableExpression><![CDATA[{\$F{titulo}}]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field titulo.
Línea 168	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 169	<code><group name="CategoriaGroup" isStartNewPage="false" isReprintHeaderOnEachPage="true" minHeightToStartNewPage="80"></code> → Declara el grupo CategoriaGroup y sus propiedades de paginación/reimpresión.
Línea 170	<code><groupExpression><![CDATA[{\$F{categoria}}]></groupExpression></code> → Define la clave que decide cuándo cambia el grupo mediante field categoria.
Línea 171	<code><groupHeader></code> → Abre la cabecera del grupo, que se emite cuando comienza cada nuevo valor de agrupación.
Línea 172	<code><band height="28"></code> → Define una banda de 28 puntos, reservando ese espacio para sus elementos.
Línea 173	<code><textField><reportElement x="0" y="2" width="555" height="22" style="M5GrupoCabecera"/><textFieldExpression><![CDATA["Categoría: " + {\$F{categoria}}]></textFieldExpression></textField></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=2, ancho=555, alto=22 y aplica el estilo M5GrupoCabecera; evalúa la expresión usando field categoria.
Línea 174	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 175	<code></groupHeader></code> → Cierra groupHeader y vuelve al elemento padre de la jerarquía JRXML.
Línea 176	<code><groupFooter></code> → Abre el pie del grupo, donde se muestran los acumulados justo antes de cambiar de grupo.
Línea 177	<code><band height="34"></code> → Define una banda de 34 puntos, reservando ese espacio para sus elementos.
Línea 178	<code><textField><reportElement x="0" y="3" width="185" height="18" style="Dato"/><textFieldExpression><![CDATA["Libros del grupo: " + {\$V{GrupoLibros}}]></textFieldExpression></textField></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=3, ancho=185, alto=18 y aplica el estilo Dato; evalúa la expresión usando variable GrupoLibros.
Línea 179	<code><textField><reportElement x="185" y="3" width="180" height="18" style="Dato"/><textFieldExpression><![CDATA["Unidades: " + (\$V{GrupoUnidades} == null ? 0 : {\$V{GrupoUnidades}})]></textFieldExpression></textField></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=185, y=3, ancho=180, alto=18 y aplica el estilo Dato; evalúa la expresión usando variable GrupoUnidades, variable GrupoUnidades.
Línea 180	<code><textField><reportElement x="365" y="3" width="190" height="18" style="Dato"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA["Importe: " + new java.text.DecimalFormat("#,###.00").format(\$F{GrupoImporte})]></textFieldExpression></textField></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=365, y=3, ancho=190, alto=18 y aplica el estilo Dato; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable GrupoImporte, variable GrupoImporte.
Línea 181	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 182	<code></groupFooter></code> → Cierra groupFooter y vuelve al elemento padre de la jerarquía JRXML.
Línea 183	<code></group></code> → Finaliza la definición del grupo y sus bandas asociadas.
Línea 184	<code><background><band height="0"/></background></code> → Composición de la línea: encadena además <background>, <band> dentro de la misma jerarquía.
Línea 185	<code><title></code> → Abre la banda Title, emitida una sola vez al inicio del informe.
Línea 186	<code><band height="124"></code> → Define una banda de 124 puntos, reservando ese espacio para sus elementos.
Línea 187	<code><staticText></code> → Abre un elemento de texto literal; su contenido no depende de fields, parámetros ni variables.
Línea 188	<code><reportElement x="0" y="4" width="555" height="28" uuid="40000000-0000-4000-8000-000000000001" style="M5TituloPrincipal"/></code> → Posiciona el elemento en x=0, y=4, con ancho 555 y alto 28, aplicando el estilo M5TituloPrincipal.
Línea 189	<code><textElement textAlignment="Center" verticalAlignment="Middle"/></code> → Configura el formato interno del texto: alineación horizontal Center, alineación vertical Middle.

Línea 190	<code><text><![CDATA[Informe de Ventas - Agregación por Título]]></text></code> → Define el texto literal visible: Informe de Ventas - Agregación por Título.
Línea 191	<code></staticText></code> → Cierra <code>staticText</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 192	<code><staticText><reportElement x="0" y="38" width="110" height="18" uuid="40000000-0000-4000-8000-000000000002"/><text><![CDATA[Generado por:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=38, ancho=110, alto=18; muestra el literal 'Generado por:'.
Línea 193	<code><textField isBlankWhenNull="true"><reportElement x="110" y="38" width="160" height="18" uuid="40000000-0000-4000-8000-000000000003"/><textFieldExpression><![CDATA[\$P{usuario}]]>...</code> → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en x=110, y=38, ancho=160, alto=18; evalúa la expresión usando parámetro usuario.
Línea 194	<code><staticText><reportElement x="300" y="38" width="80" height="18" uuid="40000000-0000-4000-8000-000000000004"/><text><![CDATA[Fecha:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=38, ancho=80, alto=18; muestra el literal 'Fecha:'.
Línea 195	<code><textField pattern="dd/MM/yyyy"><reportElement x="380" y="38" width="175" height="18" uuid="40000000-0000-4000-8000-000000000005"/><textFieldExpression><![CDATA[\$P{fechaInforme}]]>...</code> → Composición de la línea: crea un <code>textField</code> dinámico con patrón dd/MM/yyyy; lo posiciona en x=380, y=38, ancho=175, alto=18; evalúa la expresión usando parámetro fechaInforme.
Línea 196	<code><staticText><reportElement x="0" y="62" width="100" height="18" uuid="40000000-0000-4000-8000-000000000006"/><text><![CDATA[Departamento:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=62, ancho=100, alto=18; muestra el literal 'Departamento:'.
Línea 197	<code><textField isBlankWhenNull="true"><reportElement x="100" y="62" width="170" height="18" uuid="40000000-0000-4000-8000-000000000007"/><textFieldExpression><![CDATA[\$P{departament}]]>...</code> → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en x=100, y=62, ancho=170, alto=18; evalúa la expresión usando parámetro departamento.
Línea 198	<code><staticText><reportElement x="300" y="62" width="70" height="18" uuid="40000000-0000-4000-8000-000000000008"/><text><![CDATA[Periodo:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=62, ancho=70, alto=18; muestra el literal 'Periodo:'.
Línea 199	<code><textField isBlankWhenNull="true"><reportElement x="370" y="62" width="185" height="18" uuid="40000000-0000-4000-8000-000000000009"/><textFieldExpression><![CDATA[\$P{periodo}]]>...</code> → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en x=370, y=62, ancho=185, alto=18; evalúa la expresión usando parámetro periodo.
Línea 200	<code><staticText><reportElement x="0" y="86" width="100" height="18" uuid="40000000-0000-4000-8000-000000000010"/><text><![CDATA[Búsqueda:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=86, ancho=100, alto=18; muestra el literal 'Búsqueda:'.
Línea 201	<code><textField isBlankWhenNull="true"><reportElement x="100" y="86" width="170" height="18" uuid="40000000-0000-4000-8000-000000000011"/><textFieldExpression><![CDATA[\$P{textoBusque}]]>...</code> → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en x=100, y=86, ancho=170, alto=18; evalúa la expresión usando parámetro textoBusqueda, parámetro textoBusqueda, parámetro textoBusqueda.
Línea 202	<code><staticText><reportElement x="300" y="86" width="90" height="18" uuid="40000000-0000-4000-8000-000000000012"/><text><![CDATA[Categorías:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=86, ancho=90, alto=18; muestra el literal 'Categorías:'.
Línea 203	<code><textField textAdjust="StretchHeight"><reportElement x="390" y="86" width="165" height="34" uuid="40000000-0000-4000-8000-000000000013"/><textFieldExpression><![CDATA[String.val]...></code> → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en x=390, y=86, ancho=165, alto=34; evalúa la expresión usando parámetro categoriasLista.
Línea 204	<code></band></code> → Cierra <code>band</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 205	<code></title></code> → Cierra <code>title</code> y vuelve al elemento padre de la jerarquía JRXML.

Línea 206	<code><columnHeader></code> → Abre Column Header, repetida al comienzo de cada columna/página según la paginación.
Línea 207	<code><band height="62"></code> → Define una banda de 62 puntos, reservando ese espacio para sus elementos.
Línea 208	<code><staticText><reportElement x="0" y="2" width="215" height="18" uuid="41000000-0000-4000-8000-000000000001" style="Cabecera"/><text><![CDATA[Título]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=2, ancho=215, alto=18 y aplica el estilo Cabecera; muestra el literal 'Título'.
Línea 209	<code><staticText><reportElement x="215" y="2" width="55" height="18" uuid="41000000-0000-4000-8000-000000000002" style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[...]</code> → Composición de la línea: crea un texto literal; lo posiciona en x=215, y=2, ancho=55, alto=18 y aplica el estilo Cabecera; configura la alineación del texto (horizontal Right); muestra el literal 'Unid.'.
Línea 210	<code><staticText><reportElement x="280" y="2" width="90" height="18" uuid="41000000-0000-4000-8000-000000000003" style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[...]</code> → Composición de la línea: crea un texto literal; lo posiciona en x=280, y=2, ancho=90, alto=18 y aplica el estilo Cabecera; configura la alineación del texto (horizontal Right); muestra el literal 'Importe'.
Línea 211	<code><staticText><reportElement x="380" y="2" width="65" height="18" uuid="41000000-0000-4000-8000-000000000004" style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[...]</code> → Composición de la línea: crea un texto literal; lo posiciona en x=380, y=2, ancho=65, alto=18 y aplica el estilo Cabecera; configura la alineación del texto (horizontal Right); muestra el literal 'Precio med.'.
Línea 212	<code><staticText><reportElement x="455" y="2" width="100" height="18" uuid="41000000-0000-4000-8000-000000000005" style="Cabecera"/><text><![CDATA[Categoría]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=455, y=2, ancho=100, alto=18 y aplica el estilo Cabecera; muestra el literal 'Categoría'.
Línea 213	<code><staticText><reportElement x="0" y="24" width="130" height="18" uuid="41000000-0000-4000-8000-000000000006" style="Cabecera"/><text><![CDATA[Primera venta]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=24, ancho=130, alto=18 y aplica el estilo Cabecera; muestra el literal 'Primera venta'.
Línea 214	<code><staticText><reportElement x="130" y="24" width="130" height="18" uuid="41000000-0000-4000-8000-000000000007" style="Cabecera"/><text><![CDATA[Última venta]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=130, y=24, ancho=130, alto=18 y aplica el estilo Cabecera; muestra el literal 'Última venta'.
Línea 215	<code><staticText><reportElement x="260" y="24" width="160" height="18" uuid="41000000-0000-4000-8000-000000000008" style="Cabecera"/><text><![CDATA[Periodo de ventas]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=260, y=24, ancho=160, alto=18 y aplica el estilo Cabecera; muestra el literal 'Periodo de ventas'.
Línea 216	<code><staticText></code> → Abre un elemento de texto literal; su contenido no depende de fields, parámetros ni variables.
Línea 217	<code><reportElement x="420" y="24" width="135" height="18" uuid="41000000-0000-4000-8000-000000000009" style="Cabecera"></code> → Posiciona el elemento en x=420, y=24, con ancho 135 y alto 18, aplicando el estilo Cabecera.
Línea 218	<code><printWhenExpression><![CDATA[Boolean.TRUE.equals(\$P{mostrarDetalle})]]></printWhenExpression></code> → Evalúa una condición booleana para decidir si la banda o elemento se imprime usando parámetro mostrarDetalle.
Línea 219	<code></reportElement></code> → Cierra reportElement y vuelve al elemento padre de la jerarquía JRXML.
Línea 220	<code><textElement textAlignment="Right"/></code> → Configura el formato interno del texto: alineación horizontal Right.
Línea 221	<code><text><![CDATA[Importe con IVA]]></text></code> → Define el texto literal visible: Importe con IVA.
Línea 222	<code></staticText></code> → Cierra staticText y vuelve al elemento padre de la jerarquía JRXML.
Línea 223	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.

Línea 224	<code></columnHeader></code> → Cierra <code>columnHeader</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 225	<code><detail></code> → Abre <code>Detail</code> , la sección que se repite para cada registro del dataset principal.
Línea 226	<code><band height="82" splitType="Stretch"></code> → Define una banda de 82 puntos con <code>splitType Stretch</code> , reservando ese espacio para sus elementos.
Línea 227	<code><textField textAdjust="StretchHeight"><reportElement x="0" y="0" width="215" height="20" uuid="42000000-0000-4000-8000-000000000001" style="Dato"/><textFieldExpression><![CDATA[... → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=0, y=0</code>, ancho=215, alto=20 y aplica el estilo <code>Dato</code>; evalúa la expresión usando <code>field titulo</code>.</code>
Línea 228	<code><textField isBlankWhenNull="true"><reportElement x="215" y="0" width="55" height="20" uuid="42000000-0000-4000-8000-000000000002" style="UnidadesCondicional"/><textElement textA... → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=215, y=0</code>, ancho=55, alto=20 y aplica el estilo <code>UnidadesCondicional</code>; configura la alineación del texto (<code>horizontal Right</code>); evalúa la expresión usando <code>field unidades_vendidas</code>.</code>
Línea 229	<code><textField pattern="#,#0.00 €" isBlankWhenNull="true"><reportElement x="280" y="0" width="90" height="20" uuid="42000000-0000-4000-8000-000000000003" style="Dato"/><textElement... → Composición de la línea: crea un <code>textField</code> dinámico con patrón <code>#,#0.00 €</code>; lo posiciona en <code>x=280, y=0</code>, ancho=90, alto=20 y aplica el estilo <code>Dato</code>; configura la alineación del texto (<code>horizontal Right</code>); evalúa la expresión usando <code>field importe_total</code>.</code>
Línea 230	<code><textField isBlankWhenNull="true"><reportElement x="380" y="0" width="65" height="20" uuid="42000000-0000-4000-8000-000000000004" style="Dato"/><textElement textAlignment="Right"... → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=380, y=0</code>, ancho=65, alto=20 y aplica el estilo <code>Dato</code>; configura la alineación del texto (<code>horizontal Right</code>); evalúa la expresión usando <code>field precio_medio</code>, <code>field precio_medio</code>.</code>
Línea 231	<code><textField isBlankWhenNull="true"><reportElement x="455" y="0" width="100" height="20" uuid="42000000-0000-4000-8000-000000000005" style="Dato"/><textFieldExpression><![CDATA[\$F... → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=455, y=0</code>, ancho=100, alto=20 y aplica el estilo <code>Dato</code>; evalúa la expresión usando <code>field categoria</code>.</code>
Línea 232	<code><textField isBlankWhenNull="true"><reportElement x="0" y="24" width="130" height="18" uuid="42000000-0000-4000-8000-000000000006" style="Dato"/><textFieldExpression><![CDATA[\$F{... → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=0, y=24</code>, ancho=130, alto=18 y aplica el estilo <code>Dato</code>; evalúa la expresión usando <code>field primera_venta</code>.</code>
Línea 233	<code><textField isBlankWhenNull="true"><reportElement x="130" y="24" width="130" height="18" uuid="42000000-0000-4000-8000-000000000007" style="Dato"/><textFieldExpression><![CDATA[\$... → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=130, y=24</code>, ancho=130, alto=18 y aplica el estilo <code>Dato</code>; evalúa la expresión usando <code>field ultima_venta</code>.</code>
Línea 234	<code><textField><reportElement x="260" y="24" width="160" height="18" uuid="42000000-0000-4000-8000-000000000008" style="Dato"/><textElement textAlignment="Center"/><textFieldExpress... → Composición de la línea: crea un <code>textField</code> dinámico; lo posiciona en <code>x=260, y=24</code>, ancho=160, alto=18 y aplica el estilo <code>Dato</code>; configura la alineación del texto (<code>horizontal Center</code>); evalúa la expresión usando <code>field primera_venta</code>, <code>field primera_venta</code>, <code>field ultima_venta</code>.</code>
Línea 235	<code><textField pattern="#,#0.00 €" isBlankWhenNull="true"></code> → Abre un <code>textField</code> dinámico con formato <code>#,#0.00 €</code> , cuyo valor se obtiene de su <code>textFieldExpression</code> .
Línea 236	<code><reportElement x="420" y="24" width="135" height="18" uuid="42000000-0000-4000-8000-000000000009" style="Dato"></code> → Posiciona el elemento en <code>x=420, y=24</code> , con ancho 135 y alto 18, aplicando el estilo <code>Dato</code> .
Línea 237	<code><printWhenExpression><![CDATA[Boolean.TRUE.equals(\$P{mostrarDetalle})]]></printWhenExpression></code> → Evalúa una condición booleana para decidir si la banda o elemento se imprime usando parámetro <code>mostrarDetalle</code> .
Línea 238	<code></reportElement></code> → Cierra <code>reportElement</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 239	<code><textElement textAlignment="Right"/></code> → Configura el formato interno del texto: alineación horizontal <code>Right</code> .

Línea 240	<code><textFieldExpression><![CDATA[\$F{importe_total} == null \$P{tipoIva} == null ? null : Double.valueOf(\$F{importe_total}.doubleValue()) * (1.0d + \$P{tipoIva}.doubleValue())]]></...></code> → Calcula el valor mostrado por el textField mediante una expresión Java que usa field importe_total, field importe_total, parámetro tipoIva, parámetro tipoIva.
Línea 241	<code></textField></code> → Cierra textField y vuelve al elemento padre de la jerarquía JRXML.
Línea 242	<code><textField><reportElement x="0" y="48" width="105" height="18" uuid="42000000-0000-4000-8000-000000000010" style="Dato"/><textFieldExpression><![CDATA[\$F{unidades_vendidas} == n...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=48, ancho=105, alto=18 y aplica el estilo Dato; evalúa la expresión usando field unidades_vendidas, field unidades_vendidas, field unidades_vendidas.
Línea 243	<code><textField><reportElement x="105" y="48" width="185" height="18" uuid="42000000-0000-4000-8000-000000000011" style="Dato"/><textFieldExpression><![CDATA[\$F{titulo} == null ? "" ...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=105, y=48, ancho=185, alto=18 y aplica el estilo Dato; evalúa la expresión usando field titulo, field titulo.
Línea 244	<code><textField><reportElement x="290" y="48" width="80" height="18" uuid="42000000-0000-4000-8000-000000000012" style="Dato"/><textElement textAlignment="Right"/><textFieldExpressio...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=290, y=48, ancho=80, alto=18 y aplica el estilo Dato; configura la alineación del texto (horizontal Right); evalúa la expresión usando field precio_medio, field precio_medio.
Línea 245	<code><textField><reportElement x="370" y="48" width="90" height="18" uuid="42000000-0000-4000-8000-000000000013" style="Dato"/><textElement textAlignment="Center"/><textFieldExpressi...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=370, y=48, ancho=90, alto=18 y aplica el estilo Dato; configura la alineación del texto (horizontal Center); evalúa la expresión usando field primera_venta, field ultima_venta, field primera_venta, field ultima_venta.
Línea 246	<code><textField><reportElement x="460" y="48" width="95" height="18" uuid="42000000-0000-4000-8000-000000000014" style="Dato"/><textElement textAlignment="Right"/><textFieldExpressio...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=460, y=48, ancho=95, alto=18 y aplica el estilo Dato; configura la alineación del texto (horizontal Right); evalúa la expresión usando field unidades_vendidas, field unidades_vendidas, parámetro umbralUnidades, parámetro umbralUnidades.
Línea 247	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 248	<code><band height="14"></code> → Define una banda de 14 puntos, reservando ese espacio para sus elementos.
Línea 249	<code><printWhenExpression><![CDATA[\$F{unidades_vendidas} != null && \$P{umbralUnidades} != null && \$F{unidades_vendidas}.intValue() >= \$P{umbralUnidades}.intValue()]]></printWhenExpre...></code> → Evalúa una condición booleana para decidir si la banda o elemento se imprime usando field unidades_vendidas, field unidades_vendidas, parámetro umbralUnidades, parámetro umbralUnidades.
Línea 250	<code><textField><reportElement x="0" y="0" width="555" height="12" uuid="42000000-0000-4000-8000-000000000015"/><textElement textAlignment="Center"></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=0, ancho=555, alto=12; configura la alineación del texto (horizontal Center); configura la fuente (familia DejaVu Sans, tamaño 8, negrita); evalúa la expresión usando field titulo, parámetro umbralUnidades.
Línea 251	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 252	<code><band height="88" splitType="Stretch"></code> → Define una banda de 88 puntos con splitType Stretch, reservando ese espacio para sus elementos.
Línea 253	<code><printWhenExpression><![CDATA[\$F{unidades_vendidas} != null]]></printWhenExpression></code> → Evalúa una condición booleana para decidir si la banda o elemento se imprime usando field unidades_vendidas.
Línea 254	<code><staticText></code> → Abre un elemento de texto literal; su contenido no depende de fields, parámetros ni variables.
Línea 255	<code><reportElement x="0" y="2" width="555" height="16" style="Cabecera"/></code> → Posiciona el elemento en x=0, y=2, con ancho 555 y alto 16, aplicando el estilo Cabecera.
Línea 256	<code><text><![CDATA[Detalle de ventas]]></text></code> → Define el texto literal visible: Detalle de ventas.

Línea 257	<code></staticText></code> → Cierra <code>staticText</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 258	<code><subreport></code> → Abre el componente subreport que ejecuta un informe hijo dentro de la banda del maestro.
Línea 259	<code><reportElement x="0" y="22" width="555" height="60" isRemoveLineWhenBlank="true"/></code> → Posiciona el elemento en x=0, y=22, con ancho 555 y alto 60, y elimina su línea cuando queda vacío.
Línea 260	<code><subreportParameter name="tituloLibro"></code> → Declara el parámetro del subreporte <code>tituloLibro</code> que recibirá un valor del informe maestro.
Línea 261	<code><subreportParameterExpression><![CDATA[{\$F{titulo}}]></subreportParameterExpression></code> → Calcula el valor enviado al parámetro del subreporte a partir de field <code>titulo</code> .
Línea 262	<code></subreportParameter></code> → Cierra <code>subreportParameter</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 263	<code><connectionExpression><![CDATA[{\$P{REPORT_CONNECTION}}]></connectionExpression></code> → Entrega al subreporte/subdataset la misma <code>REPORT_CONNECTION</code> usada por el informe maestro, evitando abrir otra conexión.
Línea 264	<code><subreportExpression><![CDATA["reports/subinforme_ventas_detalle.jasper"]]></subreportExpression></code> → Devuelve la ruta del archivo <code>subinforme_ventas_detalle.jasper</code> que JasperReports cargará como informe hijo.
Línea 265	<code></subreport></code> → Finaliza el componente de subreporte.
Línea 266	<code></band></code> → Cierra <code>band</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 267	<code><band height="104" splitType="Stretch"></code> → Define una banda de 104 puntos con <code>splitType Stretch</code> , reservando ese espacio para sus elementos.
Línea 268	<code><printWhenExpression><![CDATA[{\$F{unidades_vendidas} != null}]></printWhenExpression></code> → Evalúa una condición booleana para decidir si la banda o elemento se imprime usando field <code>unidades_vendidas</code> .
Línea 269	<code><staticText></code> → Abre un elemento de texto literal; su contenido no depende de fields, parámetros ni variables.
Línea 270	<code><reportElement x="0" y="2" width="555" height="16" style="Cabecera"/></code> → Posiciona el elemento en x=0, y=2, con ancho 555 y alto 16, aplicando el estilo <code>Cabecera</code> .
Línea 271	<code><text><![CDATA[Top 3 ventas por cantidad]]></text></code> → Define el texto literal visible: <code>Top 3 ventas por cantidad</code> .
Línea 272	<code></staticText></code> → Cierra <code>staticText</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 273	<code><componentElement></code> → Abre un contenedor de componentes extendidos; en este checkpoint contiene la tabla <code>c:table</code> .
Línea 274	<code><reportElement x="0" y="22" width="555" height="76"/></code> → Posiciona el elemento en x=0, y=22, con ancho 555 y alto 76.
Línea 275	<code><c:table xmlns:c="http://jasperreports.sourceforge.net/jasperreports/components"></code> → Abre la tabla del namespace de componentes JasperReports; sus columnas usan un <code>datasetRun</code> independiente.
Línea 276	<code>xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports/components http://jasperreports.sourceforge.net/xsd/components.xsd"></code> → Relaciona el namespace de JasperReports con su XSD para que Studio y el compilador validen la estructura.
Línea 277	<code><datasetRun subDataset="DatasetTopVentas"></code> → Asocia el componente con el subdataset <code>DatasetTopVentas</code> para ejecutar su consulta.
Línea 278	<code><datasetParameter name="tituloLibro"></code> → Declara el parámetro <code>tituloLibro</code> que se enviará al subdataset de la tabla.
Línea 279	<code><datasetParameterExpression><![CDATA[{\$F{titulo}}]></datasetParameterExpression></code> → Calcula el valor enviado al parámetro del subdataset desde field <code>titulo</code> .
Línea 280	<code></datasetParameter></code> → Cierra <code>datasetParameter</code> y vuelve al elemento padre de la jerarquía JRXML.

Línea 281	<code><connectionExpression><![CDATA[\$P{REPORT_CONNECTION}]]></connectionExpression></code> → Entrega al subreporte/subdataset la misma REPORT_CONNECTION usada por el informe maestro, evitando abrir otra conexión.
Línea 282	<code></datasetRun></code> → Cierra datasetRun y vuelve al elemento padre de la jerarquía JRXML.
Línea 283	<code><c:column width="255"></code> → Declara una columna de tabla de 255 puntos de ancho.
Línea 284	<code><c:columnHeader style="M5TablaCabecera" height="20"><staticText><reportElement x="0" y="0" width="255" height="20"/><text><![CDATA[Fecha]]></text></staticText></c:columnHeader></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=0, ancho=255, alto=20 y aplica el estilo M5TablaCabecera; muestra el literal 'Fecha'; define una celda de cabecera de la tabla.
Línea 285	<code><c:detailCell style="M5TablaDetalle" height="18"><textField><reportElement x="0" y="0" width="255" height="18"/><textFieldExpression><![CDATA[\$F{fecha_venta}]]></textFieldExpres...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=0, ancho=255, alto=18 y aplica el estilo M5TablaDetalle; evalúa la expresión usando field fecha_venta; define una celda repetida de detalle de la tabla.
Línea 286	<code></c:column></code> → Cierra c:column y vuelve al elemento padre de la jerarquía JRXML.
Línea 287	<code><c:column width="100"></code> → Declara una columna de tabla de 100 puntos de ancho.
Línea 288	<code><c:columnHeader style="M5TablaCabecera" height="20"><staticText><reportElement x="0" y="0" width="100" height="20"/><textElement textAlignment="Right"/><text><![CDATA[Cantidad]]...></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=0, ancho=100, alto=20 y aplica el estilo M5TablaCabecera; configura la alineación del texto (horizontal Right); muestra el literal 'Cantidad'; define una celda de cabecera de la tabla.
Línea 289	<code><c:detailCell style="M5TablaDetalle" height="18"><textField><reportElement x="0" y="0" width="100" height="18"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=0, ancho=100, alto=18 y aplica el estilo M5TablaDetalle; configura la alineación del texto (horizontal Right); evalúa la expresión usando field cantidad; define una celda repetida de detalle de la tabla.
Línea 290	<code></c:column></code> → Cierra c:column y vuelve al elemento padre de la jerarquía JRXML.
Línea 291	<code><c:column width="200"></code> → Declara una columna de tabla de 200 puntos de ancho.
Línea 292	<code><c:columnHeader style="M5TablaCabecera" height="20"><staticText><reportElement x="0" y="0" width="200" height="20"/><textElement textAlignment="Right"/><text><![CDATA[Precio uni...></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=0, ancho=200, alto=20 y aplica el estilo M5TablaCabecera; configura la alineación del texto (horizontal Right); muestra el literal 'Precio unitario'; define una celda de cabecera de la tabla.
Línea 293	<code><c:detailCell style="M5TablaDetalle" height="18"><textField pattern="#.##0.00 €"><reportElement x="0" y="0" width="200" height="18"/><textElement textAlignment="Right"/><textFie...></code> → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=0, y=0, ancho=200, alto=18 y aplica el estilo M5TablaDetalle; configura la alineación del texto (horizontal Right); evalúa la expresión usando field precio_unitario; define una celda repetida de detalle de la tabla.
Línea 294	<code></c:column></code> → Cierra c:column y vuelve al elemento padre de la jerarquía JRXML.
Línea 295	<code></c:table></code> → Finaliza la tabla integrada.
Línea 296	<code></componentElement></code> → Cierra componentElement y vuelve al elemento padre de la jerarquía JRXML.
Línea 297	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 298	<code></detail></code> → Finaliza la sección Detail del informe.
Línea 299	<code><pageFooter></code> → Abre Page Footer, emitido al pie de cada página.
Línea 300	<code><band height="62"></code> → Define una banda de 62 puntos, reservando ese espacio para sus elementos.
Línea 301	<code><staticText><reportElement x="0" y="4" width="120" height="15" uuid="43000000-0000-4000-8000-000000000001"/><text><![CDATA[Total de títulos:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=4, ancho=120, alto=15; muestra el literal 'Total de títulos:'.

Línea 302	<pre><textField evaluationTime="Report"><reportElement x="120" y="4" width="60" height="15" uuid="43000000-0000-4000-8000-000000000002"/><textFieldExpression><![CDATA[{\$V{REPORT_COUNT... → Composición de la línea: crea un textField dinámico; lo posiciona en x=120, y=4, ancho=60, alto=15; evalúa la expresión usando variable REPORT_COUNT.</pre>
Línea 303	<pre><textField><reportElement x="190" y="28" width="180" height="15" uuid="43000000-0000-4000-8000-000000000003"/><textElement textAlignment="Right"/ ><textFieldExpression><![CDATA["... → Composición de la línea: crea un textField dinámico; lo posiciona en x=190, y=28, ancho=180, alto=15; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable PAGE_NUMBER.</pre>
Línea 304	<pre><textField evaluationTime="Report"><reportElement x="375" y="28" width="35" height="15" uuid="43000000-0000-4000-8000-000000000004"/><textFieldExpression><![CDATA[{\$V{PAGE_NUMBER... → Composición de la línea: crea un textField dinámico; lo posiciona en x=375, y=28, ancho=35, alto=15; evalúa la expresión usando variable PAGE_NUMBER.</pre>
Línea 305	<pre><staticText><reportElement x="300" y="4" width="120" height="15" uuid="43000000-0000-4000-8000-000000000005"/><text><![CDATA[Subtotal página:]]></text></ staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=4, ancho=120, alto=15; muestra el literal 'Subtotal página:'.</pre>
Línea 306	<pre><textField pattern="#.##0.00 €"><reportElement x="420" y="4" width="135" height="15" uuid="43000000-0000-4000-8000-000000000006"/><textElement textAlignment="Right"/ ><textFieldE... → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=420, y=4, ancho=135, alto=15; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable TotalPagina.</pre>
Línea 307	<pre></band></pre> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 308	<pre></pageFooter></pre> → Cierra pageFooter y vuelve al elemento padre de la jerarquía JRXML.
Línea 309	<pre><summary></pre> → Abre Summary, emitido una sola vez después del último registro.
Línea 310	<pre><band height="700"></pre> → Define una banda de 700 puntos, reservando ese espacio para sus elementos.
Línea 311	<pre><staticText><reportElement x="0" y="5" width="205" height="18" uuid="44000000-0000-4000-8000-000000000001"/><text><![CDATA[Total de unidades vendidas:]]></ text></staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=5, ancho=205, alto=18; muestra el literal 'Total de unidades vendidas:'.</pre>
Línea 312	<pre><textField><reportElement x="205" y="5" width="80" height="18" uuid="44000000-0000-4000-8000-000000000002"/><textElement textAlignment="Right"/ ><textFieldExpression><![CDATA[{\$V{... → Composición de la línea: crea un textField dinámico; lo posiciona en x=205, y=5, ancho=80, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable TotalUnidades.</pre>
Línea 313	<pre><staticText><reportElement x="300" y="5" width="120" height="18" uuid="44000000-0000-4000-8000-000000000003"/><text><![CDATA[Importe total:]]></text></ staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=5, ancho=120, alto=18; muestra el literal 'Importe total:'.</pre>
Línea 314	<pre><textField pattern="#.##0.00 €"><reportElement x="420" y="5" width="135" height="18" uuid="44000000-0000-4000-8000-000000000004"/><textElement textAlignment="Right"/ ><textFieldE... → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=420, y=5, ancho=135, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable TotalImporte.</pre>
Línea 315	<pre><staticText><reportElement x="0" y="30" width="205" height="18" uuid="44000000-0000-4000-8000-000000000005"/><text><![CDATA[Precio medio agregado:]]></ text></staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=30, ancho=205, alto=18; muestra el literal 'Precio medio agregado:'.</pre>
Línea 316	<pre><textField pattern="#.##0.00 €"><reportElement x="205" y="30" width="80" height="18" uuid="44000000-0000-4000-8000-000000000006"/><textElement textAlignment="Right"/ ><textFieldE... → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=205, y=30, ancho=80, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable PrecioMedio.</pre>

Línea 317	<code><staticText><reportElement x="300" y="30" width="120" height="18" uuid="44000000-0000-4000-8000-000000000007"/><text><![CDATA[Precio máximo:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=30, ancho=120, alto=18; muestra el literal 'Precio máximo:'.
Línea 318	<code><textField pattern="#.##0.00 €"><reportElement x="420" y="30" width="135" height="18" uuid="44000000-0000-4000-8000-000000000008"/><textElement textAlignment="Right"/><textField...></code> → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=420, y=30, ancho=135, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable PrecioMaximo.
Línea 319	<code><staticText><reportElement x="0" y="55" width="205" height="18" uuid="44000000-0000-4000-8000-000000000009"/><text><![CDATA[Número de libros:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=55, ancho=205, alto=18; muestra el literal 'Número de libros:'.
Línea 320	<code><textField><reportElement x="205" y="55" width="80" height="18" uuid="44000000-0000-4000-8000-000000000010"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[\$V...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=205, y=55, ancho=80, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable NumeroLibros.
Línea 321	<code><staticText><reportElement x="300" y="55" width="120" height="18" uuid="44000000-0000-4000-8000-000000000011"/><text><![CDATA[Importe con IVA:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=55, ancho=120, alto=18; muestra el literal 'Importe con IVA:'.
Línea 322	<code><textField pattern="#.##0.00 €"><reportElement x="420" y="55" width="135" height="18" uuid="44000000-0000-4000-8000-000000000012"/><textElement textAlignment="Right"/><textField...></code> → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=420, y=55, ancho=135, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable ImporteConIva.
Línea 323	<code><textField><reportElement x="0" y="80" width="555" height="18" uuid="44000000-0000-4000-8000-000000000013"/><textElement textAlignment="Center"/><textFieldExpression><![CDATA[St...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=80, ancho=555, alto=18; configura la alineación del texto (horizontal Center); evalúa la expresión usando variable NumeroLibros, variable TotalUnidades, variable TotalImporte.
Línea 324	<code><textField><reportElement x="0" y="103" width="350" height="18" uuid="44000000-0000-4000-8000-000000000014"/><textElement textAlignment="Center"></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=103, ancho=350, alto=18; configura la alineación del texto (horizontal Center); configura la fuente (familia DejaVu Sans, tamaño 10, negrita); evalúa la expresión usando parámetro umbralUnidades, parámetro umbralUnidades, variable TotalUnidades, variable TotalUnidades.
Línea 325	<code><textField><reportElement x="360" y="103" width="195" height="18" uuid="44000000-0000-4000-8000-000000000015"/><textFieldExpression><![CDATA["Resultados encontrados: " + \$V{REPO...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=360, y=103, ancho=195, alto=18; evalúa la expresión usando variable REPORT_COUNT.
Línea 326	<code><staticText><reportElement x="0" y="140" width="555" height="20" style="Cabecera"/><text><![CDATA[Ventas por categoría – importe]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=140, ancho=555, alto=20 y aplica el estilo Cabecera; muestra el literal 'Ventas por categoría — importe'.
Línea 327	<code><barChart></code> → Abre el gráfico de barras nativo de JasperReports que se integrará en el informe maestro.
Línea 328	<code><chart></code> → Abre la configuración común del gráfico: geometría, título, subtítulo y leyenda.
Línea 329	<code><reportElement x="0" y="165" width="555" height="250"/></code> → Posiciona el elemento en x=0, y=165, con ancho 555 y alto 250.
Línea 330	<code><chartTitle><titleExpression><![CDATA["Ventas por categoría"]]></titleExpression></chartTitle></code> → Composición de la línea: encadena además <chartTitle>, <titleExpression> dentro de la misma jerarquía.
Línea 331	<code><chartSubtitle/></code> → Declara el subtítulo del gráfico; en este checkpoint queda vacío.

Línea 332	<code><chartLegend position="Bottom"/></code> → Configura la leyenda del gráfico en la posición Bottom.
Línea 333	<code></chart></code> → Cierra chart y vuelve al elemento padre de la jerarquía JRXML.
Línea 334	<code><categoryDataset></code> → Abre el dataset categórico que alimenta al gráfico con serie, categoría y valor.
Línea 335	<code><dataset></code> → Abre el contenedor de ejecución de datos del componente actual.
Línea 336	<code><datasetRun subDataset="DatasetVentasPorCategoría"></code> → Asocia el componente con el subdataset DatasetVentasPorCategoría para ejecutar su consulta.
Línea 337	<code><connectionExpression><![CDATA[\${REPORT_CONNECTION}]]></connectionExpression></code> → Entrega al subreporte/subdataset la misma REPORT_CONNECTION usada por el informe maestro, evitando abrir otra conexión.
Línea 338	<code></datasetRun></code> → Cierra datasetRun y vuelve al elemento padre de la jerarquía JRXML.
Línea 339	<code></dataset></code> → Cierra dataset y vuelve al elemento padre de la jerarquía JRXML.
Línea 340	<code><categorySeries></code> → Abre una serie del dataset categórico; cada fila del subdataset aportará categoría y valor.
Línea 341	<code><seriesExpression><![CDATA["Importe"]]></seriesExpression></code> → Define el nombre lógico de la serie que aparecerá en la leyenda.
Línea 342	<code><categoryExpression><![CDATA[\${categoría_grafico}]]></categoryExpression></code> → Define la categoría del eje X a partir de field categoría_grafico.
Línea 343	<code><valueExpression><![CDATA[\${importe_categoria}]]></valueExpression></code> → Define el valor numérico representado por cada barra a partir de field importe_categoria.
Línea 344	<code></categorySeries></code> → Cierra categorySeries y vuelve al elemento padre de la jerarquía JRXML.
Línea 345	<code></categoryDataset></code> → Cierra categoryDataset y vuelve al elemento padre de la jerarquía JRXML.
Línea 346	<code><barPlot></code> → Abre el plot específico del gráfico de barras, donde se configuran etiquetas y ejes.
Línea 347	<code><plot/></code> → Declara el bloque base del plot; mantiene la configuración visual por defecto del checkpoint.
Línea 348	<code><itemLabel/></code> → Habilita el bloque de configuración de etiquetas de los ítems/barras.
Línea 349	<code><categoryAxisFormat><axisFormat/></categoryAxisFormat></code> → Composición de la línea: encadena además <code><categoryAxisFormat></code> , <code><axisFormat></code> dentro de la misma jerarquía.
Línea 350	<code><valueAxisFormat><axisFormat/></valueAxisFormat></code> → Composición de la línea: encadena además <code><valueAxisFormat></code> , <code><axisFormat></code> dentro de la misma jerarquía.
Línea 351	<code></barPlot></code> → Cierra barPlot y vuelve al elemento padre de la jerarquía JRXML.
Línea 352	<code></barChart></code> → Finaliza el gráfico de barras.
Línea 353	<code><staticText><reportElement x="0" y="430" width="555" height="20" style="Cabecera"/><text><![CDATA[Ventas por categoría y año]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=430, ancho=555, alto=20 y aplica el estilo Cabecera; muestra el literal 'Ventas por categoría y año'.
Línea 354	<code><crosstab></code> → Abre la tabla cruzada nativa que genera dinámicamente la matriz de filas, columnas, medidas y totales.
Línea 355	<code><reportElement x="0" y="455" width="555" height="225"/></code> → Posiciona el elemento en x=0, y=455, con ancho 555 y alto 225.
Línea 356	<code><crosstabDataset></code> → Abre la fuente de datos específica del crosstab.
Línea 357	<code><dataset></code> → Abre el contenedor de ejecución de datos del componente actual.
Línea 358	<code><datasetRun subDataset="DatasetCrosstabVentas"></code> → Asocia el componente con el subdataset DatasetCrosstabVentas para ejecutar su consulta.

Línea 359	<code><connectionExpression><![CDATA[\$P{REPORT_CONNECTION}]]></connectionExpression></code> → Entrega al subreporte/subdataset la misma REPORT_CONNECTION usada por el informe maestro, evitando abrir otra conexión.
Línea 360	<code></datasetRun></code> → Cierra datasetRun y vuelve al elemento padre de la jerarquía JRXML.
Línea 361	<code></dataset></code> → Cierra dataset y vuelve al elemento padre de la jerarquía JRXML.
Línea 362	<code></crosstabDataset></code> → Cierra crosstabDataset y vuelve al elemento padre de la jerarquía JRXML.
Línea 363	<code><rowGroup name="CategoriaCross" width="150" totalPosition="End"></code> → Declara el grupo de filas CategoriaCross, ancho 150 y total en End.
Línea 364	<code><bucket class="java.lang.String"><bucketExpression><![CDATA[\$F{categoria_cross}]]></bucketExpression></bucket></code> → Composición de la línea: define la clave de agrupación del bucket desde field categoria_cross; encadena además <bucket> dentro de la misma jerarquía.
Línea 365	<code><crosstabRowHeader><cellContents style="M5CrosstabCabecera"><textField><reportElement x="0" y="0" width="150" height="34"><textElement verticalAlignment="Middle"><textFieldExp...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=0, ancho=150, alto=34 y aplica el estilo M5CrosstabCabecera; configura la alineación del texto (vertical Middle); evalúa la expresión usando variable CategoriaCross; abre el contenido de celda con estilo M5CrosstabCabecera; define la cabecera de fila del crosstab.
Línea 366	<code><crosstabTotalRowHeader><cellContents style="M5CrosstabTotal"><staticText><reportElement x="0" y="0" width="150" height="34"><textElement verticalAlignment="Middle"><text><![CDATA[...></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=0, ancho=150, alto=34 y aplica el estilo M5CrosstabTotal; configura la alineación del texto (vertical Middle); muestra el literal 'TOTAL'; abre el contenido de celda con estilo M5CrosstabTotal; define la cabecera de total de fila.
Línea 367	<code></rowGroup></code> → Cierra rowGroup y vuelve al elemento padre de la jerarquía JRXML.
Línea 368	<code><columnGroup name="AnioCross" height="28" totalPosition="End"></code> → Declara el grupo de columnas AnioCross, altura 28 y total en End.
Línea 369	<code><bucket class="java.lang.String"><bucketExpression><![CDATA[\$F{anio_cross}]]></bucketExpression></bucket></code> → Composición de la línea: define la clave de agrupación del bucket desde field anio_cross; encadena además <bucket> dentro de la misma jerarquía.
Línea 370	<code><crosstabColumnHeader><cellContents style="M5CrosstabCabecera"><textField><reportElement x="0" y="0" width="100" height="28"><textElement textAlignment="Center" verticalAlignme...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=0, ancho=100, alto=28 y aplica el estilo M5CrosstabCabecera; configura la alineación del texto (horizontal Center, vertical Middle); evalúa la expresión usando variable AnioCross; abre el contenido de celda con estilo M5CrosstabCabecera; define la cabecera de columna del crosstab.
Línea 371	<code><crosstabTotalColumnHeader><cellContents style="M5CrosstabTotal"><staticText><reportElement x="0" y="0" width="100" height="28"><textElement textAlignment="Center" verticalAlig...></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=0, ancho=100, alto=28 y aplica el estilo M5CrosstabTotal; configura la alineación del texto (horizontal Center, vertical Middle); muestra el literal 'TOTAL'; abre el contenido de celda con estilo M5CrosstabTotal; define la cabecera de total de columna.
Línea 372	<code></columnGroup></code> → Cierra columnGroup y vuelve al elemento padre de la jerarquía JRXML.
Línea 373	<code><measure name="ImporteCross" class="java.lang.Double" calculation="Sum"><measureExpression><![CDATA[\$F{importe_cross}]]></measureExpression></measure></code> → Composición de la línea: aporta a la medida el valor de field importe_cross; encadena además <measure> dentro de la misma jerarquía.
Línea 374	<code><measure name="VentasCross" class="java.lang.Integer" calculation="Sum"><measureExpression><![CDATA[\$F{ventas_cross}]]></measureExpression></measure></code> → Composición de la línea: aporta a la medida el valor de field ventas_cross; encadena además <measure> dentro de la misma jerarquía.
Línea 375	<code><crosstabCell width="100" height="34"><cellContents style="M5CrosstabDetalle"><textField pattern="#.##0.00 €"><reportElement x="0" y="0" width="100" height="18"><textElement te...></code> → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=0, y=0, ancho=100, alto=34 y aplica el estilo M5CrosstabDetalle; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable ImporteCross, variable VentasCross; abre el contenido de celda con estilo M5CrosstabDetalle; define la celda de detalle del crosstab.

Línea 376	<code><crosstabCell width="100" height="34" rowTotalGroup="CategoriaCross"><cellContents style="M5CrosstabTotal"><textField pattern="#,##0.00 €"><reportElement x="0" y="0" width="100" . . . → Composición de la línea: crea un textField dinámico con patrón #,##0.00 €; lo posiciona en x=0, y=0, ancho=100, alto=34 y aplica el estilo M5CrosstabTotal; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable ImporteCross, variable VentasCross; abre el contenido de celda con estilo M5CrosstabTotal; define una celda de total de fila para CategoriaCross.</code>
Línea 377	<code><crosstabCell width="100" height="34" columnTotalGroup="AnioCross"><cellContents style="M5CrosstabTotal"><textField pattern="#,##0.00 €"><reportElement x="0" y="0" width="100" h. . . → Composición de la línea: crea un textField dinámico con patrón #,##0.00 €; lo posiciona en x=0, y=0, ancho=100, alto=34 y aplica el estilo M5CrosstabTotal; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable ImporteCross, variable VentasCross; abre el contenido de celda con estilo M5CrosstabTotal; define una celda de total de columna para AnioCross.</code>
Línea 378	<code><crosstabCell width="100" height="34" rowTotalGroup="CategoriaCross" columnTotalGroup="AnioCross"><cellContents style="M5CrosstabTotal"><textField pattern="#,##0.00 €"><reportEl. . . → Composición de la línea: crea un textField dinámico con patrón #,##0.00 €; lo posiciona en x=0, y=0, ancho=100, alto=34 y aplica el estilo M5CrosstabTotal; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable ImporteCross, variable VentasCross; abre el contenido de celda con estilo M5CrosstabTotal; define la celda de total general.</code>
Línea 379	<code></crosstab> → Finaliza la tabla cruzada.</code>
Línea 380	<code></band> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.</code>
Línea 381	<code></summary> → Finaliza la sección Summary.</code>
Línea 382	<code></jasperReport> → Finaliza la definición completa del informe JasperReports.</code>

Subinforme heredado y ejecutable

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/
jasperreport.xsd"
  name="subinforme_ventas_detalle"
  language="java"
  pageWidth="555"
  pageHeight="842"
  columnWidth="555"
  leftMargin="0"
  rightMargin="0"
  topMargin="0"
  bottomMargin="0">
  <property name="com.jaspersoft.studio.data.defaultdataadapter" value="SQLiteEditorial"/>
  <style name="SubBase" isDefault="true" fontName="DejaVu Sans" fontSize="8"/>
  <style name="SubHeader" style="SubBase" mode="Opaque" backcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/>
  <parameter name="tituloLibro" class="java.lang.String"/>
  <queryString language="sql">
    <![CDATA[
      SELECT fecha_venta, cantidad, precio_unitario
      FROM ventas
      WHERE titulo_libro = $P{tituloLibro}
      ORDER BY fecha_venta
    ]]>
  </queryString>
  <field name="fecha_venta" class="java.lang.String"/>
  <field name="cantidad" class="java.lang.Integer"/>
  <field name="precio_unitario" class="java.lang.Double"/>
  <columnHeader>
    <band height="18">
      <staticText><reportElement x="0" y="0" width="245" height="18" style="SubHeader"/><text><![CDATA[Fecha]]></text></staticText>
      <staticText><reportElement x="245" y="0" width="100" height="18" style="SubHeader"/><textElement textAlignment="Right"/>
    </band>
  </columnHeader>
  <detail>
    <band height="18">
      <textField><reportElement x="0" y="0" width="245" height="18"/><textFieldExpression><![CDATA[$F{fecha_venta}]]></textFieldExpression></textField>
      <textField><reportElement x="245" y="0" width="100" height="18"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[$F{cantidad}]]></textFieldExpression></textField>
      <textField><reportElement x="345" y="0" width="210" height="18"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[$F{precio_unitario}]]></textFieldExpression></textField>
    </band>
  </detail>
</jasperReport>
```

```

</band>
</detail>
</jasperReport>

```

Explicación línea por línea

Línea 1	<code><?xml version="1.0" encoding="UTF-8"?></code> → Declara XML 1.0 y codificación UTF-8 para que nombres, textos y símbolos del informe se interpreten correctamente.
Línea 2	<code><jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports"</code> → Abre el documento raíz <code>jasperReport</code> del informe y fija el namespace principal de JasperReports.
Línea 3	<code>xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"</code> → Declara el namespace XML Schema Instance usado por <code>xsi:schemaLocation</code> para validar el documento.
Línea 4	<code>xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/jasperreport.xsd"</code> → Relaciona el namespace de JasperReports con su XSD para que Studio y el compilador validen la estructura.
Línea 5	<code>name="subinforme_ventas_detalle"</code> → Asigna al documento JasperReports el nombre interno <code>subinforme_ventas_detalle</code> .
Línea 6	<code>language="java"</code> → Configura <code>language=java</code> para evaluar expresiones con el lenguaje Java.
Línea 7	<code>pageWidth="555"</code> → Fija el ancho físico de página en 555 puntos.
Línea 8	<code>pageHeight="842"</code> → Fija la altura física de página en 842 puntos.
Línea 9	<code>columnWidth="555"</code> → Fija el ancho útil de la columna de contenido en 555 puntos.
Línea 10	<code>leftMargin="0"</code> → Fija el margen izquierdo del informe en 0 puntos.
Línea 11	<code>rightMargin="0"</code> → Fija el margen derecho del informe en 0 puntos.
Línea 12	<code>topMargin="0"</code> → Fija el margen superior del informe en 0 puntos.
Línea 13	<code>bottomMargin="0"></code> → Fija el margen inferior del informe en 0 puntos y completa la apertura del elemento raíz.
Línea 14	<code><property name="com.jaspersoft.studio.data.defaultdataadapter" value="SQLiteEditorial"/></code> → Indica a Jaspersoft Studio que use el Data Adapter <code>SQLiteEditorial</code> como conexión de diseño por defecto.
Línea 15	<code><style name="SubBase" isDefault="true" fontName="DejaVu Sans" fontSize="8"/></code> → Declara el estilo <code>SubBase</code> ; es el estilo por defecto, fuente <code>DejaVu Sans</code> , tamaño 8.
Línea 16	<code><style name="SubHeader" style="SubBase" mode="Opaque" backcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/></code> → Declara el estilo <code>SubHeader</code> ; hereda de <code>SubBase</code> , fondo <code>#EAF2F8</code> .
Línea 17	<code><parameter name="tituloLibro" class="java.lang.String"/></code> → Declara el parámetro <code>tituloLibro</code> con tipo <code>java.lang.String</code> y lo expone al diálogo de parámetros de Studio.
Línea 18	<code><queryString language="sql"></code> → Abre la consulta SQL que JasperReports ejecutará para el dataset actual.
Línea 19	<code><![CDATA[</code> → Abre CDATA para escribir SQL o una expresión Java sin que sus caracteres especiales se interpreten como XML.
Línea 20	<code>SELECT fecha_venta, cantidad, precio_unitario</code> → Cláusula SQL <code>SELECT</code> : selecciona y calcula las columnas que devolverá la consulta.
Línea 21	<code>FROM ventas</code> → Cláusula SQL <code>FROM</code> : define la tabla base de la consulta.
Línea 22	<code>WHERE titulo_libro = \${tituloLibro}</code> → Cláusula SQL <code>WHERE</code> : aplica el filtro de filas.
Línea 23	<code>ORDER BY fecha_venta</code> → Cláusula SQL <code>ORDER BY</code> : ordena el resultado que recibirá JasperReports.
Línea 24	<code>]]></code> → Cierra el bloque CDATA y devuelve el control al parser XML.
Línea 25	<code></queryString></code> → Cierra la consulta SQL del dataset actual.

Línea 26	<code><field name="fecha_venta" class="java.lang.String"/></code> → Declara el field fecha_venta con tipo Java <code>java.lang.String</code> para mapear una columna del dataset.
Línea 27	<code><field name="cantidad" class="java.lang.Integer"/></code> → Declara el field cantidad con tipo Java <code>java.lang.Integer</code> para mapear una columna del dataset.
Línea 28	<code><field name="precio_unitario" class="java.lang.Double"/></code> → Declara el field precio_unitario con tipo Java <code>java.lang.Double</code> para mapear una columna del dataset.
Línea 29	<code><columnHeader></code> → Abre Column Header, repetida al comienzo de cada columna/página según la paginación.
Línea 30	<code><band height="18"></code> → Define una banda de 18 puntos, reservando ese espacio para sus elementos.
Línea 31	<code><staticText><reportElement x="0" y="0" width="245" height="18" style="SubHeader"/><text><![CDATA[Fecha]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=0, ancho=245, alto=18 y aplica el estilo SubHeader; muestra el literal 'Fecha'.
Línea 32	<code><staticText><reportElement x="245" y="0" width="100" height="18" style="SubHeader"/><textElement textAlignment="Right"><text><![CDATA[Cantidad]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=245, y=0, ancho=100, alto=18 y aplica el estilo SubHeader; configura la alineación del texto (horizontal Right); muestra el literal 'Cantidad'.
Línea 33	<code><staticText><reportElement x="345" y="0" width="210" height="18" style="SubHeader"/><textElement textAlignment="Right"><text><![CDATA[Precio unitario]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=345, y=0, ancho=210, alto=18 y aplica el estilo SubHeader; configura la alineación del texto (horizontal Right); muestra el literal 'Precio unitario'.
Línea 34	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 35	<code></columnHeader></code> → Cierra columnHeader y vuelve al elemento padre de la jerarquía JRXML.
Línea 36	<code><detail></code> → Abre Detail, la sección que se repite para cada registro del dataset principal.
Línea 37	<code><band height="18"></code> → Define una banda de 18 puntos, reservando ese espacio para sus elementos.
Línea 38	<code><textField><reportElement x="0" y="0" width="245" height="18"/><textFieldExpression><![CDATA[\${fecha_venta}]]></textFieldExpression></textField></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=0, ancho=245, alto=18; evalúa la expresión usando field fecha_venta.
Línea 39	<code><textField><reportElement x="245" y="0" width="100" height="18"/><textElement textAlignment="Right"><textFieldExpression><![CDATA[\${cantidad}]]></textFieldExpression></textElement></textField></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=245, y=0, ancho=100, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando field cantidad.
Línea 40	<code><textField pattern="#.##0.00 €"><reportElement x="345" y="0" width="210" height="18"/><textElement textAlignment="Right"><textFieldExpression><![CDATA[\${precio_unitario}]]></textElement></textField></code> → Composición de la línea: crea un textField dinámico con patrón <code>#.##0.00 €</code> ; lo posiciona en x=345, y=0, ancho=210, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando field precio_unitario.
Línea 41	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 42	<code></detail></code> → Finaliza la sección Detail del informe.
Línea 43	<code></jasperReport></code> → Finaliza la definición completa del informe JasperReports.

Plantilla JRTX heredada y ejecutable

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<jasperTemplate xmlns="http://jasperreports.sourceforge.net/jasperreports/template"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports/template http://jasperreports.sourceforge.net/
  xsd/jasptemplate.xsd">
  <style name="MSTituloPrincipal" fontName="DejaVu Sans" fontSize="18" isBold="true" forecolor="#173F6B"/>
  <style name="M5GrupoCabecera" fontName="DejaVu Sans" fontSize="10" isBold="true" forecolor="#173F6B" mode="Opaque"
  backcolor="#D6EAF8"/>
  <style name="M5TablaCabecera" fontName="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque"
  backcolor="#EAF2F8"/>
  <style name="M5TablaDetalle" fontName="DejaVu Sans" fontSize="9"/>
  <style name="M5CrosstabCabecera" fontName="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque">
```

```

backcolor="#EAF2F8"/>
<style name="M5CrosstabDetalle" fontName="DejaVu Sans" fontSize="9" mode="Opaque" backcolor="#FFFFFF"/>
<style name="M5CrosstabTotal" fontName="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque"
backcolor="#D6EAF8"/>
</jasperTemplate>

```

Explicación línea por línea

Línea 1	<?xml version="1.0" encoding="UTF-8"?> → Declara XML 1.0 y codificación UTF-8 para que nombres, textos y símbolos del informe se interpreten correctamente.
Línea 2	<jasperTemplate xmlns="http://jasperreports.sourceforge.net/jasperreports/template" → Abre el documento raíz jasperTemplate de la plantilla JRTX que contiene estilos reutilizables.
Línea 3	xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" → Declara el namespace XML Schema Instance usado por xsi:schemaLocation para validar el documento.
Línea 4	xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports/template http://jasperreports.sourceforge.net/xsd/jasptemplate.xsd" → Relaciona el namespace de JasperReports con su XSD para que Studio y el compilador validen la estructura.
Línea 5	<style name="M5TituloPrincipal" fontName="DejaVu Sans" fontSize="18" isBold="true" forecolor="#173F6B"/> → Declara el estilo M5TituloPrincipal; fuente DejaVu Sans, tamaño 18.
Línea 6	<style name="M5GrupoCabecera" fontName="DejaVu Sans" fontSize="10" isBold="true" forecolor="#173F6B" mode="Opaque" backcolor="#D6EAF8"/> → Declara el estilo M5GrupoCabecera; fuente DejaVu Sans, tamaño 10, fondo #D6EAF8.
Línea 7	<style name="M5TablaCabecera" fontName="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque" backcolor="#EAF2F8"/> → Declara el estilo M5TablaCabecera; fuente DejaVu Sans, tamaño 9, fondo #EAF2F8.
Línea 8	<style name="M5TablaDetalle" fontName="DejaVu Sans" fontSize="9"/> → Declara el estilo M5TablaDetalle; fuente DejaVu Sans, tamaño 9.
Línea 9	<style name="M5CrosstabCabecera" fontName="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque" backcolor="#EAF2F8"/> → Declara el estilo M5CrosstabCabecera; fuente DejaVu Sans, tamaño 9, fondo #EAF2F8.
Línea 10	<style name="M5CrosstabDetalle" fontName="DejaVu Sans" fontSize="9" mode="Opaque" backcolor="#FFFFFF"/> → Declara el estilo M5CrosstabDetalle; fuente DejaVu Sans, tamaño 9, fondo #FFFFFF.
Línea 11	<style name="M5CrosstabTotal" fontName="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque" backcolor="#D6EAF8"/> → Declara el estilo M5CrosstabTotal; fuente DejaVu Sans, tamaño 9, fondo #D6EAF8.
Línea 12	</jasperTemplate> → Finaliza la plantilla externa JRTX.

Parte C — Código y configuración ejecutable explicados línea por línea

GeneradorInformeVentas.java

JAVA

```

import java.io.File;
import java.sql.Connection;
import java.sql.DriverManager;
import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;
import net.sf.jasperreports.engine.JasperCompileManager;
import net.sf.jasperreports.engine.JasperFillManager;
import net.sf.jasperreports.engine.JasperPrint;
import net.sf.jasperreports.engine.export.JRPdfExporter;
import net.sf.jasperreports.export.SimpleExporterInput;
import net.sf.jasperreports.export.SimpleOutputStreamExporterOutput;
import net.sf.jasperreports.export.SimplePdfExporterConfiguration;
import net.sf.jasperreports.engine.data.JRCsvDataSource;
import net.sf.jasperreports.engine.export.ooxml.JRXlsxExporter;
import net.sf.jasperreports.export.SimpleXlsxReportConfiguration;
import net.sf.jasperreports.export.SimpleXlsxExporterConfiguration;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.nio.file.StandardCopyOption;

```

```

import net.sf.jasperreports.engine.export.HtmlExporter;
import net.sf.jasperreports.engine.export.FileHtmlResourceHandler;
import net.sf.jasperreports.export.SimpleHtmlExporterConfiguration;
import net.sf.jasperreports.export.SimpleHtmlExporterOutput;
import net.sf.jasperreports.engine.export.JRCsvExporter;
import net.sf.jasperreports.engine.export.JRXmlExporter;
import net.sf.jasperreports.engine.export.JRRtfExporter;
import net.sf.jasperreports.engine.export.oasis.JROdtExporter;
import net.sf.jasperreports.export.SimpleCsvExporterConfiguration;
import net.sf.jasperreports.export.SimpleWriterExporterOutput;
import net.sf.jasperreports.export.SimpleXmlExporterOutput;

public class GeneradorInformeVentas {
    public static void main(String[] args) {
        try {
            String rutaJrxml = "reports/informe_ventas.jrxml";
            String rutaJasper = "reports/informe_ventas.jasper";
            String rutaPdf = "output/informe_ventas.pdf";
            String rutaPdfProtegido = "output/informe_ventas_protegido.pdf";
            String rutaXlsx = "output/informe_ventas.xlsx";
            String rutaCatalogoJrxml = "reports/informe_catalogo_csv.jrxml";
            String rutaCatalogoJasper = "reports/informe_catalogo_csv.jasper";
            String rutaXlsxCatalogo = "output/informe_catalogo.xlsx";
            String rutaHtml = "output/informe_ventas.html";
            String rutaCsv = "output/informe_ventas.csv";
            String rutaXml = "output/informe_ventas.xml";
            String rutaRtf = "output/informe_ventas.rtf";
            String rutaOdt = "output/informe_ventas.odt";
            String urlBD = "jdbc:sqlite:../EditorialReportsJava/data/editorial.db";
            new File("output").mkdirs();

            String rutaSubJrxml = "reports/subinforme_ventas_detalle.jrxml";
            String rutaSubJasper = "reports/subinforme_ventas_detalle.jasper";
            JasperCompileManager.compileReportToFile(rutaSubJrxml, rutaSubJasper);
            JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);
            JasperCompileManager.compileReportToFile(rutaCatalogoJrxml, rutaCatalogoJasper);

            Map<String, Object> parametros = new HashMap<String, Object>();
            parametros.put("usuario", "Ana Martínez");
            parametros.put("departamento", "Comercial");
            parametros.put("periodo", "Septiembre 2026");
            parametros.put("tipoIva", Double.valueOf(0.21d));
            parametros.put("mostrarDetalle", Boolean.TRUE);
            parametros.put("categoria", null);
            parametros.put("precioMinimo", null);
            parametros.put("precioMaximo", null);
            parametros.put("umbralUnidades", Integer.valueOf(5));
            parametros.put("textoBusqueda", null);
            parametros.put("categoriasLista", Arrays.asList("Novela", "Realismo mágico", "Cuento", "Poesía"));

            try (Connection conexion = DriverManager.getConnection(urlBD)) {
                JasperPrint documento = JasperFillManager.fillReport(rutaJasper, parametros, conexion);
                exportarPdf(documento, rutaPdf);
                exportarPdfProtegido(documento, rutaPdfProtegido);
                exportarXlsx(documento, rutaXlsx, "Ventas");
                JRCsvDataSource catalogoDataSource = new JRCsvDataSource(new File("data/catalogo.csv"), "UTF-8");
                try {
                    catalogoDataSource.setFieldDelimiter(',');
                    catalogoDataSource.setUseFirstRowAsHeader(true);
                    JasperPrint documentoCatalogo = JasperFillManager.fillReport(
                        rutaCatalogoJasper, new HashMap<String, Object>(), catalogoDataSource);
                    exportarXlsx(documentoCatalogo, rutaXlsxCatalogo, "Catálogo");
                } finally {
                    catalogoDataSource.close();
                }
                new File("output/images").mkdirs();
                new File("output/styles").mkdirs();
                Files.copy(Paths.get("resources/styles/editorial.css"), Paths.get("output/styles/editorial.css"),
                    StandardCopyOption.REPLACE_EXISTING);
                exportarHtml(documento, rutaHtml);
                exportarCsv(documento, rutaCsv);
                exportarXml(documento, rutaXml);
                exportarRtf(documento, rutaRtf);
                exportarOdt(documento, rutaOdt);
                System.out.println("Informe PDF generado en: " + new File(rutaPdf).getAbsolutePath());
                System.out.println("Informe PDF protegido generado en: " + new File(rutaPdfProtegido).getAbsolutePath());
                System.out.println("Informe Excel generado en: " + new File(rutaXlsx).getAbsolutePath());
                System.out.println("Informe catálogo Excel generado en: " + new File(rutaXlsxCatalogo).getAbsolutePath());
                System.out.println("Informe HTML generado en: " + new File(rutaHtml).getAbsolutePath());
                System.out.println("Informe CSV generado en: " + new File(rutaCsv).getAbsolutePath());
                System.out.println("Informe XML generado en: " + new File(rutaXml).getAbsolutePath());
                System.out.println("Informe RTF generado en: " + new File(rutaRtf).getAbsolutePath());
                System.out.println("Informe ODT generado en: " + new File(rutaOdt).getAbsolutePath());
                System.out.println("Paginas del documento: " + documento.getPages().size());
                System.out.println("Parametro usuario: " + parametros.get("usuario"));
                System.out.println("M6 checkpoint generado correctamente");
            }
        } catch (Exception e) {
            e.printStackTrace();
            System.exit(1);
        }
    }
}

```

```

}

private static void exportarPdf(JasperPrint documento, String ruta) throws Exception {
    JRPdfExporter exportador = new JRPdfExporter();
    SimplePdfExporterConfiguration configuracion = new SimplePdfExporterConfiguration();
    configuracion.setMetadataTitle("Informe de Ventas - EditorialReports");
    configuracion.setMetadataAuthor("Departamento Comercial");
    configuracion.setMetadataSubject("Resumen de ventas del catálogo");
    configuracion.setMetadataKeywords("ventas, catálogo, libros, editorial");
    configuracion.setMetadataCreator("JasperReports 6.20.0");
    configuracion.setDisplayMetadataTitle(Boolean.TRUE);
    configuracion.setCompressed(Boolean.TRUE);
    exportador.setConfiguration(configuracion);
    exportador.setExporterInput(new SimpleExporterInput(documento));
    exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta));
    exportador.exportReport();
}

private static void exportarPdfProtegido(JasperPrint documento, String ruta) throws Exception {
    JRPdfExporter exportador = new JRPdfExporter();
    SimplePdfExporterConfiguration configuracion = new SimplePdfExporterConfiguration();
    configuracion.setMetadataTitle("Informe de Ventas Protegido - EditorialReports");
    configuracion.setEncrypted(Boolean.TRUE);
    configuracion.setUserPassword("editorial2026");
    configuracion.setOwnerPassword("editorial-admin");
    configuracion.setAllowedPermissionsHint("PRINTING|COPY|SCREENREADERS");
    exportador.setConfiguration(configuracion);
    exportador.setExporterInput(new SimpleExporterInput(documento));
    exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta));
    exportador.exportReport();
}

private static void exportarXlsx(JasperPrint documento, String ruta, String nombreHoja) throws Exception {
    JRXlsxExporter exportador = new JRXlsxExporter();
    SimpleXlsxReportConfiguration informe = new SimpleXlsxReportConfiguration();
    informe.setSheetNames(new String[]{nombreHoja});
    informe.setShowGridLines(Boolean.FALSE);
    informe.setCellLocked(Boolean.FALSE);
    informe.setCellHidden(Boolean.FALSE);
    informe.setDetectCellType(Boolean.TRUE);
    informe.setOnePagePerSheet(Boolean.FALSE);
    SimpleXlsxExporterConfiguration libro = new SimpleXlsxExporterConfiguration();
    libro.setCreateCustomPalette(Boolean.TRUE);
    exportador.setConfiguration(informe);
    exportador.setConfiguration(libro);
    exportador.setExporterInput(new SimpleExporterInput(documento));
    exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta));
    exportador.exportReport();
}

private static void exportarHtml(JasperPrint documento, String ruta) throws Exception {
    HtmlExporter exportador = new HtmlExporter();
    SimpleHtmlExporterConfiguration configuracion = new SimpleHtmlExporterConfiguration();
    configuracion.setHtmlHeader("<html><head><meta charset='UTF-8'>"
        + "<title>Informe de Ventas - EditorialReports</title>"
        + "<link rel='stylesheet' href='styles/editorial.css'>"
        + "</head><body><a class='enlace-pdf' href='informe_ventas.pdf'>Descargar PDF</a>");
    configuracion.setHtmlFooter("</body></html>");
    configuracion.setBetweenPagesHtml("<hr class='salto-pagina' />");
    SimpleHtmlExporterOutput salida = new SimpleHtmlExporterOutput(ruta, "UTF-8");
    salida.setImageHandler(new FileHtmlResourceHandler(new File("output/images"), "images/{0}"));
    exportador.setConfiguration(configuracion);
    exportador.setExporterInput(new SimpleExporterInput(documento));
    exportador.setExporterOutput(salida);
    exportador.exportReport();
}

private static void exportarCsv(JasperPrint documento, String ruta) throws Exception {
    JRCsvExporter exportador = new JRCsvExporter();
    SimpleCsvExporterConfiguration configuracion = new SimpleCsvExporterConfiguration();
    configuracion.setFieldDelimiter(";");
    configuracion.setRecordDelimiter("\n");
    configuracion.setWriteBOM(Boolean.TRUE);
    exportador.setConfiguration(configuracion);
    exportador.setExporterInput(new SimpleExporterInput(documento));
    exportador.setExporterOutput(new SimpleWriterExporterOutput(ruta, "UTF-8"));
    exportador.exportReport();
}

private static void exportarXml(JasperPrint documento, String ruta) throws Exception {
    JRXmlExporter exportador = new JRXmlExporter();
    SimpleXmlExporterOutput salida = new SimpleXmlExporterOutput(ruta, "UTF-8");
    salida.setEmbeddingImages(Boolean.TRUE);
    exportador.setExporterInput(new SimpleExporterInput(documento));
    exportador.setExporterOutput(salida);
    exportador.exportReport();
}

private static void exportarRtf(JasperPrint documento, String ruta) throws Exception {
    JRRtfExporter exportador = new JRRtfExporter();
    exportador.setExporterInput(new SimpleExporterInput(documento));
    exportador.setExporterOutput(new SimpleWriterExporterOutput(ruta, "UTF-8"));
}

```

```

        exportador.exportReport();
    }

    private static void exportarOdt(JasperPrint documento, String ruta) throws Exception {
        JRodtExporter exportador = new JRodtExporter();
        exportador.setExporterInput(new SimpleExporterInput(documento));
        exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta));
        exportador.exportReport();
    }
}

```

Explicación línea por línea

Línea 1	<code>import java.io.File;</code> → Importa <code>java.io.File</code> para gestionar rutas y crear la carpeta de salida.
Línea 2	<code>import java.sql.Connection;</code> → Importa <code>java.sql.Connection</code> para representar la conexión JDBC abierta contra SQLite.
Línea 3	<code>import java.sql.DriverManager;</code> → Importa <code>java.sql.DriverManager</code> para abrir la conexión JDBC a partir de la URL SQLite.
Línea 4	<code>import java.util.Arrays;</code> → Importa <code>java.util.Arrays</code> para construir la colección de categorías usada por el parámetro de lista.
Línea 5	<code>import java.util.HashMap;</code> → Importa <code>java.util.HashMap</code> para crear la implementación mutable del mapa de parámetros.
Línea 6	<code>import java.util.Map;</code> → Importa <code>java.util.Map</code> para tipar el mapa de parámetros que recibe JasperReports.
Línea 7	<code>import net.sf.jasperreports.engine.JasperCompileManager;</code> → Importa <code>net.sf.jasperreports.engine.JasperCompileManager</code> para compilar los JRXML a artefactos .jasper.
Línea 8	<code>import net.sf.jasperreports.engine.JasperFillManager;</code> → Importa <code>net.sf.jasperreports.engine.JasperFillManager</code> para llenar el informe compilado con parámetros y conexión.
Línea 9	<code>import net.sf.jasperreports.engine.JasperPrint;</code> → Importa <code>net.sf.jasperreports.engine.JasperPrint</code> para representar en memoria el documento ya paginado por JasperReports.
Línea 10	<code>import net.sf.jasperreports.engine.export.JRPdfExporter;</code> → Importa <code>net.sf.jasperreports.engine.export.JRPdfExporter</code> para usar la clase <code>JRPdfExporter</code> en el generador.
Línea 11	<code>import net.sf.jasperreports.export.SimpleExporterInput;</code> → Importa <code>net.sf.jasperreports.export.SimpleExporterInput</code> para usar la clase <code>SimpleExporterInput</code> en el generador.
Línea 12	<code>import net.sf.jasperreports.export.SimpleOutputStreamExporterOutput;</code> → Importa <code>net.sf.jasperreports.export.SimpleOutputStreamExporterOutput</code> para usar la clase <code>SimpleOutputStreamExporterOutput</code> en el generador.
Línea 13	<code>import net.sf.jasperreports.export.SimplePdfExporterConfiguration;</code> → Importa <code>net.sf.jasperreports.export.SimplePdfExporterConfiguration</code> para usar la clase <code>SimplePdfExporterConfiguration</code> en el generador.
Línea 14	<code>import net.sf.jasperreports.engine.data.JRCsvDataSource;</code> → Importa <code>net.sf.jasperreports.engine.data.JRCsvDataSource</code> para usar la clase <code>JRCsvDataSource</code> en el generador.
Línea 15	<code>import net.sf.jasperreports.engine.export.ooxml.JRXlsxExporter;</code> → Importa <code>net.sf.jasperreports.engine.export.ooxml.JRXlsxExporter</code> para usar la clase <code>JRXlsxExporter</code> en el generador.
Línea 16	<code>import net.sf.jasperreports.export.SimpleXlsxReportConfiguration;</code> → Importa <code>net.sf.jasperreports.export.SimpleXlsxReportConfiguration</code> para usar la clase <code>SimpleXlsxReportConfiguration</code> en el generador.
Línea 17	<code>import net.sf.jasperreports.export.SimpleXlsxExporterConfiguration;</code> → Importa <code>net.sf.jasperreports.export.SimpleXlsxExporterConfiguration</code> para usar la clase <code>SimpleXlsxExporterConfiguration</code> en el generador.

Línea 18	<code>import java.nio.file.Files;</code> → Importa <code>java.nio.file.Files</code> para usar la clase <code>Files</code> en el generador.
Línea 19	<code>import java.nio.file.Paths;</code> → Importa <code>java.nio.file.Paths</code> para usar la clase <code>Paths</code> en el generador.
Línea 20	<code>import java.nio.file.StandardCopyOption;</code> → Importa <code>java.nio.file.StandardCopyOption</code> para usar la clase <code>StandardCopyOption</code> en el generador.
Línea 21	<code>import net.sf.jasperreports.engine.export.HtmlExporter;</code> → Importa <code>net.sf.jasperreports.engine.export.HtmlExporter</code> para usar la clase <code>HtmlExporter</code> en el generador.
Línea 22	<code>import net.sf.jasperreports.engine.export.FileHtmlResourceHandler;</code> → Importa <code>net.sf.jasperreports.engine.export.FileHtmlResourceHandler</code> para usar la clase <code>FileHtmlResourceHandler</code> en el generador.
Línea 23	<code>import net.sf.jasperreports.export.SimpleHtmlExporterConfiguration;</code> → Importa <code>net.sf.jasperreports.export.SimpleHtmlExporterConfiguration</code> para usar la clase <code>SimpleHtmlExporterConfiguration</code> en el generador.
Línea 24	<code>import net.sf.jasperreports.export.SimpleHtmlExporterOutput;</code> → Importa <code>net.sf.jasperreports.export.SimpleHtmlExporterOutput</code> para usar la clase <code>SimpleHtmlExporterOutput</code> en el generador.
Línea 25	<code>import net.sf.jasperreports.engine.export.JRCsvExporter;</code> → Importa <code>net.sf.jasperreports.engine.export.JRCsvExporter</code> para usar la clase <code>JRCsvExporter</code> en el generador.
Línea 26	<code>import net.sf.jasperreports.engine.export.JRXmlExporter;</code> → Importa <code>net.sf.jasperreports.engine.export.JRXmlExporter</code> para usar la clase <code>JRXmlExporter</code> en el generador.
Línea 27	<code>import net.sf.jasperreports.engine.export.JRRtfExporter;</code> → Importa <code>net.sf.jasperreports.engine.export.JRRtfExporter</code> para usar la clase <code>JRRtfExporter</code> en el generador.
Línea 28	<code>import net.sf.jasperreports.engine.export.oasis.JROdtExporter;</code> → Importa <code>net.sf.jasperreports.engine.export.oasis.JROdtExporter</code> para usar la clase <code>JROdtExporter</code> en el generador.
Línea 29	<code>import net.sf.jasperreports.export.SimpleCsvExporterConfiguration;</code> → Importa <code>net.sf.jasperreports.export.SimpleCsvExporterConfiguration</code> para usar la clase <code>SimpleCsvExporterConfiguration</code> en el generador.
Línea 30	<code>import net.sf.jasperreports.export.SimpleWriterExporterOutput;</code> → Importa <code>net.sf.jasperreports.export.SimpleWriterExporterOutput</code> para usar la clase <code>SimpleWriterExporterOutput</code> en el generador.
Línea 31	<code>import net.sf.jasperreports.export.SimpleXmlExporterOutput;</code> → Importa <code>net.sf.jasperreports.export.SimpleXmlExporterOutput</code> para usar la clase <code>SimpleXmlExporterOutput</code> en el generador.
Línea 32	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 33	<code>public class GeneradorInformeVentas {</code> → Declara la clase ejecutable <code>GeneradorInformeVentas</code> que encapsula el generador del informe.
Línea 34	<code>public static void main(String[] args) {</code> → Declara <code>main</code> como punto de entrada de la aplicación Java; recibe los argumentos de línea de comandos aunque este ejemplo no los utiliza.
Línea 35	<code>try {</code> → Abre el bloque principal protegido: cualquier error de compilación, conexión, llenado o exportación será capturado por el <code>catch</code> final.
Línea 36	<code>String rutaJrxml = "reports/informe_ventas.jrxml";</code> → Declara <code>rutaJrxml</code> con ruta del JRXML maestro que se compilará; el valor configurado es <code>"reports/informe_ventas.jrxml"</code> .
Línea 37	<code>String rutaJasper = "reports/informe_ventas.jasper";</code> → Declara <code>rutaJasper</code> con ruta del <code>.jasper</code> maestro que producirá la compilación; el valor configurado es <code>"reports/informe_ventas.jasper"</code> .
Línea 38	<code>String rutaPdf = "output/informe_ventas.pdf";</code> → Declara <code>rutaPdf</code> con ruta del PDF final exportado; el valor configurado es <code>"output/informe_ventas.pdf"</code> .

Línea 39	<code>String rutaPdfProtegido = "output/informe_ventas_protegido.pdf";</code> → Declara <code>rutaPdfProtegido</code> con valor de configuración usado por el generador; el valor configurado es <code>"output/informe_ventas_protegido.pdf"</code> .
Línea 40	<code>String rutaXlsx = "output/informe_ventas.xlsx";</code> → Declara <code>rutaXlsx</code> con valor de configuración usado por el generador; el valor configurado es <code>"output/informe_ventas.xlsx"</code> .
Línea 41	<code>String rutaCatalogoJrxml = "reports/informe_catalogo_csv.jrxml";</code> → Declara <code>rutaCatalogoJrxml</code> con valor de configuración usado por el generador; el valor configurado es <code>"reports/informe_catalogo_csv.jrxml"</code> .
Línea 42	<code>String rutaCatalogoJasper = "reports/informe_catalogo_csv.jasper";</code> → Declara <code>rutaCatalogoJasper</code> con valor de configuración usado por el generador; el valor configurado es <code>"reports/informe_catalogo_csv.jasper"</code> .
Línea 43	<code>String rutaXlsxCatalogo = "output/informe_catalogo.xlsx";</code> → Declara <code>rutaXlsxCatalogo</code> con valor de configuración usado por el generador; el valor configurado es <code>"output/informe_catalogo.xlsx"</code> .
Línea 44	<code>String rutaHtml = "output/informe_ventas.html";</code> → Declara <code>rutaHtml</code> con valor de configuración usado por el generador; el valor configurado es <code>"output/informe_ventas.html"</code> .
Línea 45	<code>String rutaCsv = "output/informe_ventas.csv";</code> → Declara <code>rutaCsv</code> con valor de configuración usado por el generador; el valor configurado es <code>"output/informe_ventas.csv"</code> .
Línea 46	<code>String rutaXml = "output/informe_ventas.xml";</code> → Declara <code>rutaXml</code> con valor de configuración usado por el generador; el valor configurado es <code>"output/informe_ventas.xml"</code> .
Línea 47	<code>String rutaRtf = "output/informe_ventas.rtf";</code> → Declara <code>rutaRtf</code> con valor de configuración usado por el generador; el valor configurado es <code>"output/informe_ventas.rtf"</code> .
Línea 48	<code>String rutaOdt = "output/informe_ventas.odt";</code> → Declara <code>rutaOdt</code> con valor de configuración usado por el generador; el valor configurado es <code>"output/informe_ventas.odt"</code> .
Línea 49	<code>String urlBD = "jdbc:sqlite:../EditorialReportsJava/data/editorial.db";</code> → Declara <code>urlBD</code> con URL JDBC de la base SQLite; el valor configurado es <code>"jdbc:sqlite:../EditorialReportsJava/data/editorial.db"</code> .
Línea 50	<code>new File("output").mkdirs();</code> → Crea la carpeta <code>output</code> si todavía no existe para evitar que la exportación falle por una ruta inexistente.
Línea 51	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 52	<code>String rutaSubJrxml = "reports/subinforme_ventas_detalle.jrxml";</code> → Declara <code>rutaSubJrxml</code> con ruta del JRXML del subinforme de detalle; el valor configurado es <code>"reports/subinforme_ventas_detalle.jrxml"</code> .
Línea 53	<code>String rutaSubJasper = "reports/subinforme_ventas_detalle.jasper";</code> → Declara <code>rutaSubJasper</code> con ruta del <code>.jasper</code> del subinforme compilado; el valor configurado es <code>"reports/subinforme_ventas_detalle.jasper"</code> .
Línea 54	<code>JasperCompileManager.compileReportToFile(rutaSubJrxml, rutaSubJasper);</code> → Compila el JRXML indicado en <code>rutaSubJrxml</code> y escribe el artefacto compilado en <code>rutaSubJasper</code> .
Línea 55	<code>JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);</code> → Compila el JRXML indicado en <code>rutaJrxml</code> y escribe el artefacto compilado en <code>rutaJasper</code> .
Línea 56	<code>JasperCompileManager.compileReportToFile(rutaCatalogoJrxml, rutaCatalogoJasper);</code> → Compila el JRXML indicado en <code>rutaCatalogoJrxml</code> y escribe el artefacto compilado en <code>rutaCatalogoJasper</code> .
Línea 57	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 58	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 59	<code>Map<String, Object> parametros = new HashMap<String, Object>();</code> → Crea el mapa tipado de parámetros que se entregará a <code>JasperFillManager.fillReport</code> .
Línea 60	<code>parametros.put("usuario", "Ana Martínez");</code> → Asigna al parámetro <code>JasperReports usuario</code> el valor Java <code>"Ana Martínez"</code> antes del llenado.

Línea 61	<code>parametros.put("departamento", "Comercial");</code> → Asigna al parámetro JasperReports departamento el valor Java "Comercial" antes del llenado.
Línea 62	<code>parametros.put("periodo", "Septiembre 2026");</code> → Asigna al parámetro JasperReports periodo el valor Java "Septiembre 2026" antes del llenado.
Línea 63	<code>parametros.put("tipoIva", Double.valueOf(0.21d));</code> → Asigna al parámetro JasperReports tipoIva el valor Java Double.valueOf(0.21d) antes del llenado.
Línea 64	<code>parametros.put("mostrarDetalle", Boolean.TRUE);</code> → Asigna al parámetro JasperReports mostrarDetalle el valor Java Boolean.TRUE antes del llenado.
Línea 65	<code>parametros.put("categoria", null);</code> → Asigna al parámetro JasperReports categoria el valor Java null antes del llenado.
Línea 66	<code>parametros.put("precioMinimo", null);</code> → Asigna al parámetro JasperReports precioMinimo el valor Java null antes del llenado.
Línea 67	<code>parametros.put("precioMaximo", null);</code> → Asigna al parámetro JasperReports precioMaximo el valor Java null antes del llenado.
Línea 68	<code>parametros.put("umbralUnidades", Integer.valueOf(5));</code> → Asigna al parámetro JasperReports umbralUnidades el valor Java Integer.valueOf(5) antes del llenado.
Línea 69	<code>parametros.put("textoBusqueda", null);</code> → Asigna al parámetro JasperReports textoBusqueda el valor Java null antes del llenado.
Línea 70	<code>parametros.put("categoriasLista", Arrays.asList("Novela", "Realismo mágico", "Cuento", "Poesía"));</code> → Asigna al parámetro JasperReports categoriasLista el valor Java Arrays.asList("Novela", "Realismo mágico", "Cuento", "Poesía") antes del llenado.
Línea 71	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 72	<code>try (Connection conexion = DriverManager.getConnection(urlBD)) {</code> → Abre la conexión SQLite mediante DriverManager dentro de un try-with-resources, por lo que conexion se cierra automáticamente al terminar el bloque.
Línea 73	<code>JasperPrint documento = JasperFillManager.fillReport(rutaJasper, parametros, conexion);</code> → Inicia el llenado del informe y guarda en documento el JasperPrint paginado que devolverá JasperReports.
Línea 74	<code>exportarPdf(documento, rutaPdf);</code> → Invoca la exportación PDF normal usando el JasperPrint ya llenado y la ruta principal de salida.
Línea 75	<code>exportarPdfProtegido(documento, rutaPdfProtegido);</code> → Genera una segunda salida PDF cifrada para validar contraseñas y permisos sin alterar el PDF normal.
Línea 76	<code>exportarXlsx(documento, rutaXlsx, "Ventas");</code> → Exporta el JasperPrint de ventas a informe_ventas.xlsx usando el nombre de hoja Ventas.
Línea 77	<code>JRCsvDataSource catalogoDataSource = new JRCsvDataSource(new File("data/catalogo.csv"), "UTF-8");</code> → Abre data/catalogo.csv como datasource JasperReports en UTF-8 para llenar el informe de catálogo sin JDBC.
Línea 78	<code>try {</code> → Abre el bloque principal protegido: cualquier error de compilación, conexión, llenado o exportación será capturado por el catch final.
Línea 79	<code>catalogoDataSource.setFieldDelimiter(',');</code> → Configura punto y coma como delimitador de campos CSV.
Línea 80	<code>catalogoDataSource.setUseFirstRowAsHeader(true);</code> → Indica que la primera fila del CSV contiene los nombres de los fields del informe de catálogo.
Línea 81	<code>JasperPrint documentoCatalogo = JasperFillManager.fillReport(</code> → Inicia el llenado del informe de catálogo y guarda el resultado paginado en documentoCatalogo.
Línea 82	<code>rutaCatalogoJasper, new HashMap<String, Object>(), catalogoDataSource);</code> → Completa fillReport pasando el catálogo compilado, un mapa de parámetros vacío y el JRCsvDataSource.

Línea 83	<code>exportarXlsx(documentoCatalogo, rutaXlsxCatalogo, "Catálogo");</code> → Exporta el JasperPrint del catálogo a <code>informe_catalogo.xlsx</code> usando la hoja Catálogo.
Línea 84	<code>}</code> <code>finally {</code> → Abre el bloque <code>finally</code> que se ejecutará siempre para liberar el datasource CSV aunque falle el llenado o la exportación.
Línea 85	<code>catalogoDataSource.close();</code> → Cierra explícitamente el <code>JRCsvDataSource</code> para liberar el lector del archivo de catálogo.
Línea 86	<code>}</code> → Cierra el bloque <code>try-with-resources</code> de la conexión JDBC.
Línea 87	<code>new File("output/images").mkdirs();</code> → Crea la carpeta física donde el handler HTML podrá escribir recursos de imagen.
Línea 88	<code>new File("output/styles").mkdirs();</code> → Crea la carpeta publicada de estilos que debe acompañar a <code>informe_ventas.html</code> .
Línea 89	<code>Files.copy(Paths.get("resources/styles/editorial.css"), Paths.get("output/styles/editorial.css"));</code> → Copia la hoja CSS fuente a la carpeta publicada junto al HTML, sustituyéndola si ya existe.
Línea 90	<code>StandardCopyOption.REPLACE_EXISTING);</code> → Finaliza la copia del CSS indicando que una versión anterior debe reemplazarse para mantener la salida sincronizada.
Línea 91	<code>exportarHtml(documento, rutaHtml);</code> → Reutiliza el mismo JasperPrint para generar la salida HTML y sus recursos.
Línea 92	<code>exportarCsv(documento, rutaCsv);</code> → Exporta el documento a CSV usando la configuración de delimitadores del checkpoint.
Línea 93	<code>exportarXml(documento, rutaXml);</code> → Serializa el JasperPrint a XML en la ruta documentada.
Línea 94	<code>exportarRtf(documento, rutaRtf);</code> → Exporta el documento a RTF para procesadores de texto.
Línea 95	<code>exportarOdt(documento, rutaOdt);</code> → Reutiliza el JasperPrint de ventas para generar el reto ODT en <code>output/informe_ventas.odt</code> .
Línea 96	<code>System.out.println("Informe PDF generado en: " + new File(rutaPdf).getAbsolutePath());</code> → Escribe en la consola la evidencia "Informe PDF generado en: " + <code>new File(rutaPdf).getAbsolutePath()</code> , que queda registrada por el workflow E2E.
Línea 97	<code>System.out.println("Informe PDF protegido generado en: " + new File(rutaPdfProtegido).getAbsolutePath());</code> → Escribe en la consola la evidencia "Informe PDF protegido generado en: " + <code>new File(rutaPdfProtegido).getAbsolutePath()</code> , que queda registrada por el workflow E2E.
Línea 98	<code>System.out.println("Informe Excel generado en: " + new File(rutaXlsx).getAbsolutePath());</code> → Escribe en la consola la evidencia "Informe Excel generado en: " + <code>new File(rutaXlsx).getAbsolutePath()</code> , que queda registrada por el workflow E2E.
Línea 99	<code>System.out.println("Informe catálogo Excel generado en: " + new File(rutaXlsxCatalogo).getAbsolutePath());</code> → Escribe en la consola la evidencia "Informe catálogo Excel generado en: " + <code>new File(rutaXlsxCatalogo).getAbsolutePath()</code> , que queda registrada por el workflow E2E.
Línea 100	<code>System.out.println("Informe HTML generado en: " + new File(rutaHtml).getAbsolutePath());</code> → Escribe en la consola la evidencia "Informe HTML generado en: " + <code>new File(rutaHtml).getAbsolutePath()</code> , que queda registrada por el workflow E2E.
Línea 101	<code>System.out.println("Informe CSV generado en: " + new File(rutaCsv).getAbsolutePath());</code> → Escribe en la consola la evidencia "Informe CSV generado en: " + <code>new File(rutaCsv).getAbsolutePath()</code> , que queda registrada por el workflow E2E.
Línea 102	<code>System.out.println("Informe XML generado en: " + new File(rutaXml).getAbsolutePath());</code> → Escribe en la consola la evidencia "Informe XML generado en: " + <code>new File(rutaXml).getAbsolutePath()</code> , que queda registrada por el workflow E2E.

Línea 103	<code>System.out.println("Informe RTF generado en: " + new File(rutaRtf).getAbsolutePath());</code> → Escribe en la consola la evidencia "Informe RTF generado en: " + <code>new File(rutaRtf).getAbsolutePath()</code> , que queda registrada por el workflow E2E.
Línea 104	<code>System.out.println("Informe ODT generado en: " + new File(rutaOdt).getAbsolutePath());</code> → Escribe en la consola la evidencia "Informe ODT generado en: " + <code>new File(rutaOdt).getAbsolutePath()</code> , que queda registrada por el workflow E2E.
Línea 105	<code>System.out.println("Paginas del documento: " + documento.getPages().size());</code> → Escribe en la consola la evidencia "Paginas del documento: " + <code>documento.getPages().size()</code> , que queda registrada por el workflow E2E.
Línea 106	<code>System.out.println("Parametro usuario: " + parametros.get("usuario"));</code> → Escribe en la consola la evidencia "Parametro usuario: " + <code>parametros.get("usuario")</code> , que queda registrada por el workflow E2E.
Línea 107	<code>System.out.println("M6 checkpoint generado correctamente");</code> → Escribe en la consola la evidencia "M6 checkpoint generado correctamente", que queda registrada por el workflow E2E.
Línea 108	<code>}</code> → Cierra el bloque try-with-resources de la conexión JDBC.
Línea 109	<code>} catch (Exception e) {</code> → Cierra el bloque protegido y abre el manejador que captura cualquier excepción del proceso completo.
Línea 110	<code>e.printStackTrace();</code> → Imprime la traza completa de la excepción para que el fallo sea diagnosticable en local y en GitHub Actions.
Línea 111	<code>System.exit(1);</code> → Finaliza el proceso con código 1 para que CI marque la ejecución como fallida y no oculte el error.
Línea 112	<code>}</code> → Cierra el bloque catch o el bloque principal de control asociado a main.
Línea 113	<code>}</code> → Cierra el método main.
Línea 114	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 115	<code>private static void exportarPdf(JasperPrint documento, String ruta) throws Exception {</code> → Declara el helper privado <code>exportarPdf</code> , que encapsula una exportación concreta y propaga cualquier error al catch principal.
Línea 116	<code>JRPdfExporter exportador = new JRPdfExporter();</code> → Crea el exportador PDF avanzado que admite configuración documental y de seguridad.
Línea 117	<code>SimplePdfExporterConfiguration configuracion = new SimplePdfExporterConfiguration();</code> → Crea el objeto <code>SimplePdfExporterConfiguration</code> que recibirá las opciones específicas de esta exportación.
Línea 118	<code>configuracion.setMetadataTitle("Informe de Ventas - EditorialReports");</code> → Fija el título de los metadatos PDF con la API específica de <code>SimplePdfExporterConfiguration</code> .
Línea 119	<code>configuracion.setMetadataAuthor("Departamento Comercial");</code> → Fija el autor en los metadatos del PDF.
Línea 120	<code>configuracion.setMetadataSubject("Resumen de ventas del catálogo");</code> → Fija el asunto documental del PDF.
Línea 121	<code>configuracion.setMetadataKeywords("ventas, catálogo, libros, editorial");</code> → Fija las palabras clave que quedarán registradas en las propiedades del PDF.
Línea 122	<code>configuracion.setMetadataCreator("JasperReports 6.20.0");</code> → Registra JasperReports 6.20.0 como creador del PDF.
Línea 123	<code>configuracion.setDisplayMetadataTitle(Boolean.TRUE);</code> → Solicita a los lectores PDF que utilicen el título de metadatos cuando soporten esa preferencia.
Línea 124	<code>configuracion.setCompressed(Boolean.TRUE);</code> → Activa la compresión del PDF mediante la configuración del exportador.
Línea 125	<code>exportador.setConfiguration(configuracion);</code> → Asocia al exportador la configuración específica preparada en las líneas anteriores.

Línea 126	<code>exportador.setExporterInput(new SimpleExporterInput(documento));</code> → Entrega al exportador el JasperPrint documento ya llenado; no se vuelve a consultar la base de datos.
Línea 127	<code>exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta));</code> → Dirige la salida binaria del exportador al archivo indicado por el parámetro ruta.
Línea 128	<code>exportador.exportReport();</code> → Ejecuta la exportación con la entrada, salida y configuración ya asignadas; aquí se materializa el archivo.
Línea 129	<code>}</code> → Cierra el método main.
Línea 130	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 131	<code>private static void exportarPdfProtegido(JasperPrint documento, String ruta) throws Exception {</code> → Declara el helper privado exportarPdfProtegido, que encapsula una exportación concreta y propaga cualquier error al catch principal.
Línea 132	<code>JRPdfExporter exportador = new JRPdfExporter();</code> → Crea el exportador PDF avanzado que admite configuración documental y de seguridad.
Línea 133	<code>SimplePdfExporterConfiguration configuracion = new SimplePdfExporterConfiguration();</code> → Crea el objeto SimplePdfExporterConfiguration que recibirá las opciones específicas de esta exportación.
Línea 134	<code>configuracion.setMetadataTitle("Informe de Ventas Protegido - EditorialReports");</code> → Fija el título de los metadatos PDF con la API específica de SimplePdfExporterConfiguration.
Línea 135	<code>configuracion.setEncrypted(Boolean.TRUE);</code> → Activa el cifrado de la salida PDF protegida.
Línea 136	<code>configuracion.setUserPassword("editorial2026");</code> → Configura la contraseña de apertura que el E2E verifica con <code>pdftinfo -upw</code> .
Línea 137	<code>configuracion.setOwnerPassword("editorial-admin");</code> → Configura la contraseña de propietario del PDF protegido.
Línea 138	<code>configuracion.setAllowedPermissionsHint("PRINTING COPY SCREENREADERS");</code> → Declara los permisos PDF autorizados mediante la cadena de hints admitida por JasperReports.
Línea 139	<code>exportador.setConfiguration(configuracion);</code> → Asocia al exportador la configuración específica preparada en las líneas anteriores.
Línea 140	<code>exportador.setExporterInput(new SimpleExporterInput(documento));</code> → Entrega al exportador el JasperPrint documento ya llenado; no se vuelve a consultar la base de datos.
Línea 141	<code>exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta));</code> → Dirige la salida binaria del exportador al archivo indicado por el parámetro ruta.
Línea 142	<code>exportador.exportReport();</code> → Ejecuta la exportación con la entrada, salida y configuración ya asignadas; aquí se materializa el archivo.
Línea 143	<code>}</code> → Cierra el método main.
Línea 144	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 145	<code>private static void exportarXlsx(JasperPrint documento, String ruta, String nombreHoja) throws Exception {</code> → Declara el helper privado exportarXlsx, que encapsula una exportación concreta y propaga cualquier error al catch principal.
Línea 146	<code>JRXlsxExporter exportador = new JRXlsxExporter();</code> → Crea el exportador OOXML que escribirá el libro XLSX.
Línea 147	<code>SimpleXlsxReportConfiguration informe = new SimpleXlsxReportConfiguration();</code> → Crea la configuración de cómo el JasperPrint se distribuye en hojas y celdas XLSX.
Línea 148	<code>informe.setSheetNames(new String[]{nombreHoja});</code> → Asigna el nombre Ventas a la hoja; el E2E lo comprueba dentro de <code>xl/workbook.xml</code> .
Línea 149	<code>informe.setShowGridLines(Boolean.FALSE);</code> → Desactiva la cuadrícula predeterminada de la hoja Excel.
Línea 150	<code>informe.setCellLocked(Boolean.FALSE);</code> → Configura las celdas exportadas sin bloqueo adicional.

Línea 151	<code>informe.setCellHidden(Boolean.FALSE);</code> → Evita marcar como ocultas las celdas exportadas.
Línea 152	<code>informe.setDetectCellType(Boolean.TRUE);</code> → Pide al exportador detectar tipos numéricos/fecha en lugar de convertir indiscriminadamente a texto.
Línea 153	<code>informe.setOnePagePerSheet(Boolean.FALSE);</code> → Mantiene el informe en una misma hoja lógica en lugar de crear una hoja por página.
Línea 154	<code>SimpleXlsxExporterConfiguration libro = new SimpleXlsxExporterConfiguration();</code> → Crea la configuración propia del libro/exportador XLSX.
Línea 155	<code>libro.setCreateCustomPalette(Boolean.TRUE);</code> → Activa la paleta personalizada del exportador XLSX para reproducir mejor los colores.
Línea 156	<code>exportador.setConfiguration(informe);</code> → Aplica la configuración de informe XLSX: hoja, cuadrícula, bloqueo, tipos y paginación.
Línea 157	<code>exportador.setConfiguration(libro);</code> → Aplica además la configuración global del libro XLSX, incluida la paleta personalizada.
Línea 158	<code>exportador.setExporterInput(new SimpleExporterInput(documento));</code> → Entrega al exportador el JasperPrint documento ya llenado; no se vuelve a consultar la base de datos.
Línea 159	<code>exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta));</code> → Dirige la salida binaria del exportador al archivo indicado por el parámetro ruta.
Línea 160	<code>exportador.exportReport();</code> → Ejecuta la exportación con la entrada, salida y configuración ya asignadas; aquí se materializa el archivo.
Línea 161	<code>}</code> → Cierra el método main.
Línea 162	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 163	<code>private static void exportarHtml(JasperPrint documento, String ruta) throws Exception {</code> → Declara el helper privado <code>exportarHtml</code> , que encapsula una exportación concreta y propaga cualquier error al <code>catch</code> principal.
Línea 164	<code>HtmlExporter exportador = new HtmlExporter();</code> → Crea el exportador HTML vigente en JasperReports 6.20.0.
Línea 165	<code>SimpleHtmlExporterConfiguration configuracion = new SimpleHtmlExporterConfiguration();</code> → Crea el objeto <code>SimpleHtmlExporterConfiguration</code> que recibirá las opciones específicas de esta exportación.
Línea 166	<code>configuracion.setHtmlHeader("<html><head><meta charset='UTF-8'>")</code> → Define la cabecera HTML, incluyendo UTF-8, título y enlace a la hoja CSS externa.
Línea 167	<code>+ "<title>Informe de Ventas - EditorialReports</title>"</code> → Añade a la cabecera HTML el título visible en la pestaña/metadata del navegador.
Línea 168	<code>+ "<link rel='stylesheet' href='styles/editorial.css'>"</code> → Añade a la cabecera HTML el enlace relativo a <code>styles/editorial.css</code> .
Línea 169	<code>+ "</head><body>Descargar PDF"</code> → Añade el enlace del reto <code>Descargar PDF</code> , apuntando al PDF generado en la misma carpeta <code>output</code> .
Línea 170	<code>configuracion.setHtmlFooter("</body></html>")</code> → Define el cierre de <code>body</code> y <code>html</code> del documento exportado.
Línea 171	<code>configuracion.setBetweenPagesHtml("<hr class='salto-pagina' />")</code> → Inserta el separador HTML que representa el cambio entre páginas del JasperPrint.
Línea 172	<code>SimpleHtmlExporterOutput salida = new SimpleHtmlExporterOutput(ruta, "UTF-8");</code> → Crea la salida HTML con codificación UTF-8.
Línea 173	<code>salida.setImageHandler(new FileHtmlResourceHandler(new File("output/images"), "images/{0}"));</code> → Asocia un gestor de recursos para escribir imágenes en disco y generar sus URI relativas.
Línea 174	<code>exportador.setConfiguration(configuracion);</code> → Asocia al exportador la configuración específica preparada en las líneas anteriores.

Línea 175	<code>exportador.setExporterInput(new SimpleExporterInput(documento));</code> → Entrega al exportador el JasperPrint documento ya llenado; no se vuelve a consultar la base de datos.
Línea 176	<code>exportador.setExporterOutput(salida);</code> → Conecta al exportador el objeto salida previamente configurado con codificación y/o recursos.
Línea 177	<code>exportador.exportReport();</code> → Ejecuta la exportación con la entrada, salida y configuración ya asignadas; aquí se materializa el archivo.
Línea 178	<code>}</code> → Cierra el método main.
Línea 179	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 180	<code>private static void exportarCsv(JasperPrint documento, String ruta) throws Exception {</code> → Declara el helper privado <code>exportarCsv</code> , que encapsula una exportación concreta y propaga cualquier error al catch principal.
Línea 181	<code>JRCsvExporter exportador = new JRCsvExporter();</code> → Crea el exportador CSV orientado a texto delimitado.
Línea 182	<code>SimpleCsvExporterConfiguration configuracion = new SimpleCsvExporterConfiguration();</code> → Crea el objeto <code>SimpleCsvExporterConfiguration</code> que recibirá las opciones específicas de esta exportación.
Línea 183	<code>configuracion.setFieldDelimiter(";");</code> → Configura punto y coma como delimitador de campos CSV.
Línea 184	<code>configuracion.setRecordDelimiter("\n");</code> → Configura el salto de línea como delimitador de registros CSV.
Línea 185	<code>configuracion.setWriteBOM(Boolean.TRUE);</code> → Activa el BOM UTF-8 para facilitar la detección de codificación en aplicaciones de escritorio.
Línea 186	<code>exportador.setConfiguration(configuracion);</code> → Asocia al exportador la configuración específica preparada en las líneas anteriores.
Línea 187	<code>exportador.setExporterInput(new SimpleExporterInput(documento));</code> → Entrega al exportador el JasperPrint documento ya llenado; no se vuelve a consultar la base de datos.
Línea 188	<code>exportador.setExporterOutput(new SimpleWriterExporterOutput(ruta, "UTF-8"));</code> → Crea una salida textual con UTF-8 para el formato correspondiente.
Línea 189	<code>exportador.exportReport();</code> → Ejecuta la exportación con la entrada, salida y configuración ya asignadas; aquí se materializa el archivo.
Línea 190	<code>}</code> → Cierra el método main.
Línea 191	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 192	<code>private static void exportarXml(JasperPrint documento, String ruta) throws Exception {</code> → Declara el helper privado <code>exportarXml</code> , que encapsula una exportación concreta y propaga cualquier error al catch principal.
Línea 193	<code>JRXmlExporter exportador = new JRXmlExporter();</code> → Crea el exportador que serializa el JasperPrint a XML.
Línea 194	<code>SimpleXmlExporterOutput salida = new SimpleXmlExporterOutput(ruta, "UTF-8");</code> → Crea la salida XML con codificación UTF-8.
Línea 195	<code>salida.setEmbeddingImages(Boolean.TRUE);</code> → Solicita que los recursos gráficos de la salida XML queden embebidos.
Línea 196	<code>exportador.setExporterInput(new SimpleExporterInput(documento));</code> → Entrega al exportador el JasperPrint documento ya llenado; no se vuelve a consultar la base de datos.
Línea 197	<code>exportador.setExporterOutput(salida);</code> → Conecta al exportador el objeto salida previamente configurado con codificación y/o recursos.
Línea 198	<code>exportador.exportReport();</code> → Ejecuta la exportación con la entrada, salida y configuración ya asignadas; aquí se materializa el archivo.

Línea 199	} → Cierra el método main.
Línea 200	`` → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 201	private static void exportarRtf(JasperPrint documento, String ruta) throws Exception { → Declara el helper privado exportarRtf, que encapsula una exportación concreta y propaga cualquier error al catch principal.
Línea 202	JRRtfExporter exportador = new JRRtfExporter(); → Crea el exportador RTF.
Línea 203	exportador.setExporterInput(new SimpleExporterInput(documento)); → Entrega al exportador el JasperPrint documento ya llenado; no se vuelve a consultar la base de datos.
Línea 204	exportador.setExporterOutput(new SimpleWriterExporterOutput(ruta, "UTF-8")); → Crea una salida textual con UTF-8 para el formato correspondiente.
Línea 205	exportador.exportReport(); → Ejecuta la exportación con la entrada, salida y configuración ya asignadas; aquí se materializa el archivo.
Línea 206	} → Cierra el método main.
Línea 207	`` → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 208	private static void exportarOdt(JasperPrint documento, String ruta) throws Exception { → Declara el helper privado exportarOdt, que encapsula una exportación concreta y propaga cualquier error al catch principal.
Línea 209	JROdtExporter exportador = new JROdtExporter(); → Crea JROdtExporter del paquete Oasis para producir un OpenDocument Text real.
Línea 210	exportador.setExporterInput(new SimpleExporterInput(documento)); → Entrega al exportador el JasperPrint documento ya llenado; no se vuelve a consultar la base de datos.
Línea 211	exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta)); → Dirige la salida binaria del exportador al archivo indicado por el parámetro ruta.
Línea 212	exportador.exportReport(); → Ejecuta la exportación con la entrada, salida y configuración ya asignadas; aquí se materializa el archivo.
Línea 213	} → Cierra el método main.
Línea 214	`` → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 215	} → Cierra la clase GeneradorInformeVentas.

Parte D — Simulación del resultado y de la estructura del proyecto

D.1 — Flujo de exportación / diseño

TEXT

```
JasperPrint ventas
├── JRCsvExporter -> CSV
├── JRXmlExporter -> XML
├── JRRtfExporter -> RTF
└── oasis.JROdtExporter -> ODT
```

Qué representa: la transformación funcional que debe existir al terminar 6.4.

Cómo verificarlo: comparar el flujo con la Parte C y ejecutar el generador; el JRXML/JRTX debe seguir siendo el heredado de M5/5.6.

D.2 — Estructura lógica en código y recursos

TEXT

```
Java
├── exportarCsv
├── exportarXml
├── exportarRtf
└── exportarOdt
```

Todos reutilizan el mismo JasperPrint.

Qué representa: las clases, métodos y recursos que sustituyen en M6 al trabajo visual sobre bandas y componentes.

Cómo verificarlo: abrir Java/POM/CSS/properties según corresponda y contrastar nombres y tipos con la Parte C ejecutable.

D.3 — Archivos de salida

```
output/
├── informe_ventas.csv
├── informe_ventas.xml
├── informe_ventas.rtf
└── informe_ventas.odt
```

TEXT

Qué representa: los artefactos acumulativos esperados en output.

Cómo verificarlo: ejecutar `GeneradorInformeVentas`, abrir cada formato con una herramienta compatible y contrastar los contratos automatizados del E2E.

D.4 — Árbol acumulativo del checkpoint

```
M6/6.4/
├── EditorialReports/EXPORTACION_OTROS.md
└── EditorialReportsJava/src/GeneradorInformeVentas.java
```

TEXT

Evidencia E2E: run **36249131955**, commit `12a0eba90859a92b12578d59ae592ad17dac5fb6`, artifact runtime **10908577199**.

Invariantes: 14 libros, 9 ventas, 31 unidades, 633,40 € y 6 páginas en `informe_ventas`.

Errores comunes del ejercicio completo

Error	Causa	Solución
CSV sin UTF-8	Codificación puesta en la configuración equivocada	Usar <code>SimpleWriterExporterOutput(ruta, "UTF-8")</code>
CSV sin delimitadores esperados	Falta configuración	Usar <code>setFieldDelimiter(";")</code> y <code>setRecordDelimiter("\n")</code>
XML sin imágenes embebidas	No se configuró el output	Activar <code>setEmbeddingImages(Boolean.TRUE)</code>
RTF con caracteres dañados	Writer sin UTF-8	Usar <code>SimpleWriterExporterOutput</code> con UTF-8
JROdtExporter no se resuelve	Import incorrecto	Importar <code>net.sf.jasperreports.engine.export.oasis.JROdtExporter</code>

Reto resuelto paso a paso

Enunciado original: añadir exportación ODT a las salidas CSV/XML/RTF.

1. Se importa `oasis.JROdtExporter`.
2. Se declara `output/informe_ventas.odt`.
3. Se crea `exportarOdt(JasperPrint, String)`.
4. El exportador recibe `SimpleExporterInput(documento)`.
5. La salida usa `SimpleOutputStreamExporterOutput`.
6. Se ejecuta después de CSV/XML/RTF sobre el mismo `JasperPrint`.
7. El E2E comprueba que el ODT es ZIP íntegro y que su entrada `mimetype` vale `application/vnd.oasis.opendocument.text`.

Resultado del reto: el ODT es un documento OpenDocument real, no un archivo renombrado.

Analogía final con el contexto de la editorial

CSV, XML, RTF y ODT son distintas rutas de distribución del mismo catálogo: datos tabulares, integración estructurada y documentos editables.

Resultado esperado

- CSV con BOM UTF-8 y ;.
- XML estructural de JasperPrint.
- RTF válido en UTF-8.
- ODT real y abrible en Writer.
- PDF/XLSX/HTML heredados conservados.
- EXPORTACION_OTROS.md trazado al Java.

Conclusión y enlace al siguiente punto

6.4 completa la salida multiformato. 6.5 no añade otro diseño: centraliza las políticas de configuración y demuestra que todas las salidas sobreviven a la refactorización.

Punto 6.5 — Configuración de exportación

Objetivos de aprendizaje

- Comprender el papel del objeto JasperReportsContext en la configuración global.
- Utilizar las propiedades del sistema para configurar el comportamiento de los exportadores.
- Crear un archivo jasperreports.properties con propiedades por defecto.
- Definir una clase de configuración personalizada para centralizar las opciones del proyecto.
- Combinar la configuración global con la configuración por exportador.
- Documentar la configuración de exportación del proyecto EditorialReports.

Parte A — Práctica visual/IDE verificada

Paso 1: Abrir el cierre 6.4 como baseline

Acciones:

1. Abrir `GeneradorInformeVentas.java` de 6.5.
2. Confirmar que ya genera PDF, XLSX, HTML, CSV, XML, RTF y ODT.
3. Mantener intactos JRXML y JRTX.

Verificación visual: el contenido de 6.4 está presente antes de refactorizar.

Qué hace: completa la operación «Abrir el cierre 6.4 como baseline» en el checkpoint 6.5.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.5.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.5.

Analogía: es como reunir en un manual único las reglas de producción para que todas las salidas se configuren de forma coherente.

Paso 2: Crear `ConfiguracionExportacion.java`

Acciones:

1. En `EditorialReportsJava/src`, crear `ConfiguracionExportacion.java`.
2. Importar las configuraciones PDF, XLSX report/exporter, HTML, CSV y RTF.
3. Declarar la clase pública sin estado de instancia.

Verificación visual: Project Explorer muestra la nueva clase junto al generador.

Qué hace: completa la operación «Crear `ConfiguracionExportacion.java`» en el checkpoint 6.5.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.5.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.5.

Analogía: es como reunir en un manual único las reglas de producción para que todas las salidas se configuren de forma coherente.

Paso 3: Centralizar la configuración PDF

Acciones:

1. Crear `getConfiguracionPdf(String titulo, String autor)`.
2. Usar `setMetadataTitle`, `setMetadataAuthor`, `setMetadataCreator`, `setDisplayMetadataTitle` y `setCompressed`.
3. Devolver el objeto configurado.

Verificación visual: el método usa la API específica PDF y compila.

Qué hace: completa la operación «Centralizar la configuración PDF» en el checkpoint 6.5.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.5.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.5.

Analogía: es como reunir en un manual único las reglas de producción para que todas las salidas se configuren de forma coherente.

Paso 4: Centralizar las dos configuraciones XLSX**Acciones:**

1. Crear `getConfiguracionXlsxReport(String nombreHoja)` para hoja, cuadrícula, bloqueo, tipos y paginación.
2. Crear `getConfiguracionXlsxExportador()` para `setCreateCustomPalette(Boolean.TRUE)`.
3. No mezclar ambos niveles en un único tipo.

Verificación visual: la clase devuelve `SimpleXlsxReportConfiguration` y `SimpleXlsxExporterConfiguration` por separado.

Qué hace: completa la operación «Centralizar las dos configuraciones XLSX» en el checkpoint 6.5.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.5.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.5.

Analogía: es como reunir en un manual único las reglas de producción para que todas las salidas se configuren de forma coherente.

Paso 5: Centralizar HTML**Acciones:**

1. Crear `getConfiguracionHtml(String titulo)`.
2. Incluir charset, título, CSS y enlace Descargar PDF en la cabecera.
3. Configurar footer y separador entre páginas.

Verificación visual: el reto HTML sigue presente después de la refactorización.

Qué hace: completa la operación «Centralizar HTML» en el checkpoint 6.5.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.5.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.5.

Analogía: es como reunir en un manual único las reglas de producción para que todas las salidas se configuren de forma coherente.

Paso 6: Centralizar CSV y RTF

Acciones:

1. Crear `getConfiguracionCsv()` con delimitadores y BOM.
2. Crear `getConfiguracionRtf()` devolviendo `SimpleRtfExporterConfiguration`.
3. Mantener UTF-8 del RTF en `SimpleWriterExporterOutput`.

Verificación visual: no aparece una llamada inexistente `setEncoding` en la configuración RTF.

Qué hace: completa la operación «Centralizar CSV y RTF» en el checkpoint 6.5.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.5.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.5.

Analogía: es como reunir en un manual único las reglas de producción para que todas las salidas se configuren de forma coherente.

Paso 7: Crear `jasperreports.properties`

Acciones:

1. Crear `EditorialReportsJava/src/jasperreports.properties`.
2. Añadir `net.sf.jasperreports.export.pdf.compressed=true`.
3. Añadir `net.sf.jasperreports.export.csv.field.delimiter=;`.

Verificación visual: el archivo contiene las dos propiedades globales exactas.

Qué hace: completa la operación «Crear `jasperreports.properties`» en el checkpoint 6.5.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.5.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.5.

Analogía: es como reunir en un manual único las reglas de producción para que todas las salidas se configuren de forma coherente.

Paso 8: Configurar Maven para copiar recursos de `src`

Acciones:

1. Abrir `pom.xml`.
2. Dentro de `build`, añadir un recurso con `directory = src`.
3. Excluir `**/*.java` para copiar únicamente recursos no Java.

Verificación visual: tras `package`, `target/classes/jasperreports.properties` existe.

Qué hace: completa la operación «Configurar Maven para copiar recursos de `src`» en el checkpoint 6.5.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.5.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.5.

Analogía: es como reunir en un manual único las reglas de producción para que todas las salidas se configuren de forma coherente.

Paso 9: Refactorizar PDF y XLSX en GeneradorInformeVentas

Acciones:

1. Hacer que PDF reciba `ConfiguracionExportacion.getConfiguracionPdf(...)`.
2. Usar `getConfiguracionXlsxReport(nombreHoja)` y `getConfiguracionXlsxExportador()` en el helper XLSX.
3. Mantener las hojas Ventas y Catálogo.

Verificación visual: el generador ya no recrea manualmente esas configuraciones.

Qué hace: completa la operación «Refactorizar PDF y XLSX en GeneradorInformeVentas» en el checkpoint 6.5.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.5.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.5.

Analogía: es como reunir en un manual único las reglas de producción para que todas las salidas se configuren de forma coherente.

Paso 10: Refactorizar HTML, CSV y RTF

Acciones:

1. Usar `getConfiguracionHtml(...)` en `HtmlExporter`.
2. Usar `getConfiguracionCsv()` en `JRCsvExporter`.
3. Usar `getConfiguracionRtf()` en `JRRtfExporter`.
4. Mantener ODT sin configuración adicional porque el reto sólo exige la exportación.

Verificación visual: Source contiene llamadas a los métodos centrales y conserva todas las salidas.

Qué hace: completa la operación «Refactorizar HTML, CSV y RTF» en el checkpoint 6.5.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.5.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.5.

Analogía: es como reunir en un manual único las reglas de producción para que todas las salidas se configuren de forma coherente.

Paso 11: Compilar y verificar el classpath

Acciones:

1. Ejecutar `mvn clean package`.
2. Comprobar `target/classes/jasperreports.properties`.
3. Resolver cualquier error antes de ejecutar.

Verificación visual: Maven termina con éxito y el properties está en el classpath.

Qué hace: completa la operación «Compilar y verificar el classpath» en el checkpoint 6.5.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.5.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.5.

Analogía: es como reunir en un manual único las reglas de producción para que todas las salidas se configuren de forma coherente.

Paso 12: Ejecutar todo el cierre M6

Acciones:

1. Ejecutar `GeneradorInformeVentas`.
2. Confirmar que se regeneran PDF normal/protegido, dos XLSX, HTML, CSV, XML, RTF y ODT.
3. Confirmar seis páginas y ausencia de excepciones.

Verificación visual: todas las salidas acumuladas siguen funcionando después de centralizar configuración.

Qué hace: completa la operación «Ejecutar todo el cierre M6» en el checkpoint 6.5.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.5.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.5.

Analogía: es como reunir en un manual único las reglas de producción para que todas las salidas se configuren de forma coherente.

Paso 13: Verificar que los retos siguen resueltos

Acciones:

1. Abrir PDF protegido con `editorial2026`.
2. Confirmar hojas Ventas y Catálogo.
3. Comprobar enlace HTML al PDF y abrir ODT.
4. Comprobar acentos del RTF.

Verificación visual: la refactorización no rompe ningún reto de 6.1–6.4.

Qué hace: completa la operación «Verificar que los retos siguen resueltos» en el checkpoint 6.5.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.5.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.5.

Analogía: es como reunir en un manual único las reglas de producción para que todas las salidas se configuren de forma coherente.

Paso 14: Documentar la configuración centralizada

Acciones:

1. Crear/abrir CONFIGURACION_EXPORTACION.md.
2. Listar los métodos de ConfiguracionExportacion y el papel de jasperreports.properties.
3. Explicar la separación ReportConfiguration/ExporterConfiguration y el classpath Maven.

Verificación visual: la documentación refleja exactamente la estructura del checkpoint 6.5.

Qué hace: completa la operación «Documentar la configuración centralizada» en el checkpoint 6.5.

Por qué: la Parte A debe conducir al mismo estado que el código ejecutable de las Partes B/C y el checkpoint 6.5.

Error común: usar un nombre, ruta, clase o método distinto del documentado.

Solución: contrastar Source con la Parte C ejecutable de 6.5.

Analogía: es como reunir en un manual único las reglas de producción para que todas las salidas se configuren de forma coherente.

Parte B — JRXML/JRTX completo explicado línea por línea

En M6 el diseño no cambia: estos tres archivos deben permanecer byte a byte iguales a M5/5.6.

Informe maestro heredado y ejecutable

```
<?xml version="1.0" encoding="UTF-8"?>
<jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/
jasperreport.xsd"
  name="informe_ventas"
  language="java"
  pageWidth="595"
  pageHeight="842"
  columnWidth="555"
  leftMargin="20"
  rightMargin="20"
  topMargin="20"
  bottomMargin="20"
  uid="3d2c2bd7-3b93-4da9-8b60-6b3c45674c91">
  <property name="com.jaspersoft.studio.data.defaultdataadapter" value="SQLiteEditorial"/>
  <template><![CDATA[resources/styles/EditorialStyles.jrtx]]></template>
  <style name="Sans_Normal" isDefault="true" fontName="DejaVu Sans" fontSize="10"/>
  <style name="TituloPrincipal" style="Sans_Normal" fontSize="18" isBold="true" forecolor="#173F6B"/>
  <style name="Cabecera" style="Sans_Normal" fontSize="9" isBold="true" forecolor="#173F6B"/>
  <style name="Dato" style="Sans_Normal" fontSize="9"/>
  <style name="UnidadesCondicional" style="Dato" isBold="true">
    <conditionalStyle>
      <conditionExpression><![CDATA[{F{unidades_vendidas} != null && {P{umbralUnidades} != null &&
$F{unidades_vendidas}.intValue() >= {P{umbralUnidades}.intValue()}}]></conditionExpression>
      <style forecolor="#1B5E20"/>
    </conditionalStyle>
    <conditionalStyle>
      <conditionExpression><![CDATA[{F{unidades_vendidas} != null && {P{umbralUnidades} != null &&
$F{unidades_vendidas}.intValue() >= 3 && {F{unidades_vendidas}.intValue() < {P{umbralUnidades}.intValue()}}]></conditionExpression>
      <style forecolor="#1D5D88"/>
    </conditionalStyle>
    <conditionalStyle>
      <conditionExpression><![CDATA[{F{unidades_vendidas} == null || {F{unidades_vendidas}.intValue() < 3}]]></conditionExpression>
      <style forecolor="#9D3429"/>
    </conditionalStyle>
  </style>
  <style name="M5TableHeader" style="Dato" mode="Opaque" backcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/>
  <style name="M5TableDetail" style="Dato"/>
  <style name="M5CrossHeader" style="Dato" mode="Opaque" backcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/>
  <style name="M5CrossDetail" style="Dato" mode="Opaque" backcolor="FFFFFF"/>
  <style name="M5CrossTotal" style="Dato" mode="Opaque" backcolor="#D6EAF8" forecolor="#173F6B" isBold="true"/>
  <subDataset name="DatasetTopVentas">
    <parameter name="tituloLibro" class="java.lang.String"/>
    <queryString language="sql">
      <![CDATA[
        SELECT fecha_venta, cantidad, precio_unitario
        FROM ventas
        WHERE titulo_libro = {P{tituloLibro}}
        ORDER BY cantidad DESC, fecha_venta
      ]]>
    </queryString>
  </subDataset>
</jasperReport>
```

```

        LIMIT 3
    ]]>
</queryString>
<field name="fecha_venta" class="java.lang.String"/>
<field name="cantidad" class="java.lang.Integer"/>
<field name="precio_unitario" class="java.lang.Double"/>
</subDataset>
<subDataset name="DatasetVentasPorCategoria">
    <queryString language="sql">
        <![CDATA[
            SELECT l.categoria AS categoria_grafico,
                   COALESCE(SUM(v.cantidad * v.precio_unitario), 0.0) AS importe_categoria
            FROM libros l
            LEFT JOIN ventas v ON l.titulo = v.titulo_libro
            GROUP BY l.categoria
            ORDER BY l.categoria
        ]]>
    </queryString>
    <field name="categoria_grafico" class="java.lang.String"/>
    <field name="importe_categoria" class="java.lang.Double"/>
</subDataset>
<subDataset name="DatasetCrosstabVentas">
    <queryString language="sql">
        <![CDATA[
            SELECT l.categoria AS categoria_cross,
                   SUBSTR(v.fecha_venta, 1, 4) AS anio_cross,
                   (v.cantidad * v.precio_unitario) AS importe_cross,
                   1 AS ventas_cross
            FROM ventas v
            JOIN libros l ON l.titulo = v.titulo_libro
            ORDER BY l.categoria, anio_cross, v.fecha_venta
        ]]>
    </queryString>
    <field name="categoria_cross" class="java.lang.String"/>
    <field name="anio_cross" class="java.lang.String"/>
    <field name="importe_cross" class="java.lang.Double"/>
    <field name="ventas_cross" class="java.lang.Integer"/>
</subDataset>
<parameter name="usuario" class="java.lang.String" isForPrompting="true"/>
<parameter name="fechaInforme" class="java.util.Date" isForPrompting="true">
    <defaultValueExpression><![CDATA[new java.util.Date()]]></defaultValueExpression>
</parameter>
<parameter name="departamento" class="java.lang.String" isForPrompting="true">
    <defaultValueExpression><![CDATA["General"]]></defaultValueExpression>
</parameter>
<parameter name="periodo" class="java.lang.String" isForPrompting="true">
    <defaultValueExpression><![CDATA["Mensual"]]></defaultValueExpression>
</parameter>
<parameter name="tipoIva" class="java.lang.Double" isForPrompting="true">
    <defaultValueExpression><![CDATA[Double.valueOf(0.21d)]]></defaultValueExpression>
</parameter>
<parameter name="mostrarDetalle" class="java.lang.Boolean" isForPrompting="true">
    <defaultValueExpression><![CDATA[Boolean.TRUE]]></defaultValueExpression>
</parameter>
<parameter name="categoria" class="java.lang.String" isForPrompting="true"/>
<parameter name="precioMinimo" class="java.lang.Double" isForPrompting="true"/>
<parameter name="precioMaximo" class="java.lang.Double" isForPrompting="true"/>
<parameter name="umbralUnidades" class="java.lang.Integer" isForPrompting="true">
    <defaultValueExpression><![CDATA[Integer.valueOf(5)]]></defaultValueExpression>
</parameter>
<parameter name="textoBusqueda" class="java.lang.String" isForPrompting="true"/>
<parameter name="categoriasLista" class="java.util.Collection" isForPrompting="false">
    <defaultValueExpression><![CDATA[java.util.Arrays.asList("Novela", "Realismo mágico", "Cuento", "Poesía")]]></
defaultValueExpression>
</parameter>
<queryString language="sql">
    <![CDATA[
        SELECT l.titulo,
               l.categoria,
               SUM(v.cantidad) AS unidades_vendidas,
               SUM(v.cantidad * v.precio_unitario) AS importe_total,
               AVG(v.precio_unitario) AS precio_medio,
               MIN(v.fecha_venta) AS primera_venta,
               MAX(v.fecha_venta) AS ultima_venta
        FROM libros l
        LEFT JOIN ventas v ON l.titulo = v.titulo_libro
        WHERE ($P{categoria} IS NULL OR l.categoria = $P{categoria})
        AND ($P{precioMinimo} IS NULL OR l.precio >= $P{precioMinimo})
        AND ($P{precioMaximo} IS NULL OR l.precio <= $P{precioMaximo})
        AND ($P{textoBusqueda} IS NULL OR $P{textoBusqueda} = '' OR l.titulo LIKE '%' || $P{textoBusqueda} || '%')
        AND $X{IN, l.categoria, categoriasLista}
        GROUP BY l.titulo, l.categoria
        ORDER BY l.categoria, COALESCE(importe_total, 0) DESC, l.titulo
    ]]>
</queryString>
<field name="titulo" class="java.lang.String"/>
<field name="categoria" class="java.lang.String"/>
<field name="unidades_vendidas" class="java.lang.Integer"/>
<field name="importe_total" class="java.lang.Double"/>
<field name="precio_medio" class="java.lang.Double"/>
<field name="primera_venta" class="java.lang.String"/>
<field name="ultima_venta" class="java.lang.String"/>

```

```

<variable name="TotalUnidades" class="java.lang.Integer" calculation="Sum" resetType="Report">
  <variableExpression><![CDATA[{${unidades_vendidas}]]></variableExpression>
</variable>
<variable name="TotalImporte" class="java.lang.Double" calculation="Sum" resetType="Report">
  <variableExpression><![CDATA[{${importe_total}]]></variableExpression>
</variable>
<variable name="TotalPagina" class="java.lang.Double" calculation="Sum" resetType="Page">
  <variableExpression><![CDATA[{${importe_total}]]></variableExpression>
</variable>
<variable name="PrecioMedio" class="java.lang.Double" calculation="Average" resetType="Report">
  <variableExpression><![CDATA[{${precio_medio}]]></variableExpression>
</variable>
<variable name="PrecioMaximo" class="java.lang.Double" calculation="Highest" resetType="Report">
  <variableExpression><![CDATA[{${precio_medio}]]></variableExpression>
</variable>
<variable name="NumeroLibros" class="java.lang.Integer" calculation="Count" resetType="Report">
  <variableExpression><![CDATA[{${titulo}]]></variableExpression>
</variable>
<variable name="ImporteConIva" class="java.lang.Double" calculation="Sum" resetType="Report">
  <variableExpression><![CDATA[{${importe_total} == null || $P{tipoIva} == null ? null :
Double.valueOf($F{importe_total}.doubleValue() * (1.0d + $P{tipoIva}.doubleValue()))]]></variableExpression>
</variable>
<variable name="GrupoUnidades" class="java.lang.Integer" calculation="Sum" resetType="Group" resetGroup="CategoriaGroup">
  <variableExpression><![CDATA[{$F{unidades_vendidas}]]></variableExpression>
</variable>
<variable name="GrupoImporte" class="java.lang.Double" calculation="Sum" resetType="Group" resetGroup="CategoriaGroup">
  <variableExpression><![CDATA[{$F{importe_total}]]></variableExpression>
</variable>
<variable name="GrupoLibros" class="java.lang.Integer" calculation="Count" resetType="Group" resetGroup="CategoriaGroup">
  <variableExpression><![CDATA[{$F{titulo}]]></variableExpression>
</variable>
<group name="CategoriaGroup" isStartNewPage="false" isReprintHeaderOnEachPage="true" minHeightToStartNewPage="80">
  <groupExpression><![CDATA[{$F{categoria}]]></groupExpression>
  <groupHeader>
    <band height="28">
      <textField><reportElement x="0" y="2" width="555" height="22" style="M5GrupoCabecera"/><textFieldExpression><![
CDATA["Categoria: " + $F{categoria}]]></textFieldExpression></textField>
    </band>
  </groupHeader>
  <groupFooter>
    <band height="34">
      <textField><reportElement x="0" y="3" width="185" height="18" style="Dato"/><textFieldExpression><![CDATA["Libros del
grupo: " + $V{GrupoLibros}]]></textFieldExpression></textField>
      <textField><reportElement x="185" y="3" width="180" height="18" style="Dato"/><textFieldExpression><![CDATA["Unidades: "
+ ($V{GrupoUnidades} == null ? 0 : $V{GrupoUnidades})]]></textFieldExpression></textField>
      <textField><reportElement x="365" y="3" width="190" height="18" style="Dato"/><textFieldExpression><![CDATA["Importe: " + new java.text.DecimalFormat("#,##0.00 '€').format($V{GrupoImporte} == null ?
Double.valueOf(0d) : $V{GrupoImporte})]]></textFieldExpression></textField>
    </band>
  </groupFooter>
</group>
<background><band height="0"/></background>
<title>
  <band height="124">
    <staticText>
      <reportElement x="0" y="4" width="555" height="28" uuid="40000000-0000-4000-8000-000000000001"
style="M5TituloPrincipal"/>
      <textElement textAlignment="Center" verticalAlignment="Middle"/>
      <text><![CDATA[Informe de Ventas - Agregación por Título]]></text>
    </staticText>
    <staticText><reportElement x="0" y="38" width="110" height="18" uuid="40000000-0000-4000-8000-000000000002"/><text><![
CDATA[Generado por:]]></text></staticText>
    <textField isBlankWhenNull="true"><reportElement x="110" y="38" width="160" height="18"
uuid="40000000-0000-4000-8000-000000000003"/><textFieldExpression><![CDATA[{$P{usuario}]]></textFieldExpression></textField>
    <staticText><reportElement x="300" y="38" width="80" height="18" uuid="40000000-0000-4000-8000-000000000004"/><text><![
CDATA[Fecha:]]></text></staticText>
    <textField pattern="dd/MM/yyyy"><reportElement x="380" y="38" width="175" height="18"
uuid="40000000-0000-4000-8000-000000000005"/><textFieldExpression><![CDATA[{$P{fechaInforme}]]></textFieldExpression></textField>
    <staticText><reportElement x="0" y="62" width="100" height="18" uuid="40000000-0000-4000-8000-000000000006"/><text><![
CDATA[Departamento:]]></text></staticText>
    <textField isBlankWhenNull="true"><reportElement x="100" y="62" width="170" height="18"
uuid="40000000-0000-4000-8000-000000000007"/><textFieldExpression><![CDATA[{$P{departamento}]]></textFieldExpression></textField>
    <staticText><reportElement x="300" y="62" width="70" height="18" uuid="40000000-0000-4000-8000-000000000008"/><text><![
CDATA[Periodo:]]></text></staticText>
    <textField isBlankWhenNull="true"><reportElement x="370" y="62" width="185" height="18"
uuid="40000000-0000-4000-8000-000000000009"/><textFieldExpression><![CDATA[{$P{periodo}]]></textFieldExpression></textField>
    <staticText><reportElement x="0" y="86" width="100" height="18" uuid="40000000-0000-4000-8000-000000000010"/><text><![
CDATA[Búsqueda:]]></text></staticText>
    <textField isBlankWhenNull="true"><reportElement x="100" y="86" width="170" height="18"
uuid="40000000-0000-4000-8000-000000000011"/><textFieldExpression><![CDATA[{$P{textoBusqueda} == null ||
$P{textoBusqueda}.trim().isEmpty() ? "(todas)" : $P{textoBusqueda}]]></textFieldExpression></textField>
    <staticText><reportElement x="300" y="86" width="90" height="18" uuid="40000000-0000-4000-8000-000000000012"/><text><![
CDATA[Categorías:]]></text></staticText>
    <textField textAdjust="StretchHeight"><reportElement x="390" y="86" width="165" height="34"
uuid="40000000-0000-4000-8000-000000000013"/><textFieldExpression><![CDATA[String.valueOf($P{categoriasLista}]]></
textFieldExpression></textField>
  </band>
</title>
<columnHeader>
  <band height="62">
    <staticText><reportElement x="0" y="2" width="215" height="18" uuid="41000000-0000-4000-8000-000000000001" style="Cabecera"/
><text><![CDATA[Índice]]></text></staticText>

```

```

<staticText><reportElement x="215" y="2" width="55" height="18" uuid="41000000-0000-4000-8000-000000000002"
style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[Unid.]]></text></staticText>
<staticText><reportElement x="280" y="2" width="90" height="18" uuid="41000000-0000-4000-8000-000000000003"
style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[Importe]]></text></staticText>
<staticText><reportElement x="380" y="2" width="65" height="18" uuid="41000000-0000-4000-8000-000000000004"
style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[Precio med.]]></text></staticText>
<staticText><reportElement x="455" y="2" width="100" height="18" uuid="41000000-0000-4000-8000-000000000005"
style="Cabecera"/><text><![CDATA[Categoría]]></text></staticText>
<staticText><reportElement x="0" y="24" width="130" height="18" uuid="41000000-0000-4000-8000-000000000006"
style="Cabecera"/><text><![CDATA[Primera venta]]></text></staticText>
<staticText><reportElement x="130" y="24" width="130" height="18" uuid="41000000-0000-4000-8000-000000000007"
style="Cabecera"/><text><![CDATA[Última venta]]></text></staticText>
<staticText><reportElement x="260" y="24" width="160" height="18" uuid="41000000-0000-4000-8000-000000000008"
style="Cabecera"/><text><![CDATA[Periodo de ventas]]></text></staticText>
<staticText>
<reportElement x="420" y="24" width="135" height="18" uuid="41000000-0000-4000-8000-000000000009" style="Cabecera">
<printWhenExpression><![CDATA[Boolean.TRUE.equals($P{mostrarDetalle})]]></printWhenExpression>
</reportElement>
<textElement textAlignment="Right"/>
<text><![CDATA[Importe con IVA]]></text>
</staticText>
</band>
</columnHeader>
<detail>
<band height="82" splitType="Stretch">
<textField textAdjust="StretchHeight"><reportElement x="0" y="0" width="215" height="20"
uuid="42000000-0000-4000-8000-000000000001" style="Dato"/><textFieldExpression><![CDATA[${titulo}]]></textFieldExpression></textField>
<textField isBlankWhenNull="true"><reportElement x="215" y="0" width="55" height="20"
uuid="42000000-0000-4000-8000-000000000002" style="UnidadesCondicional"/><textElement textAlignment="Right"/><textFieldExpression><![
CDATA[${unidades_vendidas}]]></textFieldExpression></textField>
<textField pattern="#,##0.00 €" isBlankWhenNull="true"><reportElement x="280" y="0" width="90" height="20"
uuid="42000000-0000-4000-8000-000000000003" style="Dato"/><textElement textAlignment="Right"/><textFieldExpression><![
CDATA[${importe_total}]]></textFieldExpression></textField>
<textField isBlankWhenNull="true"><reportElement x="380" y="0" width="65" height="20"
uuid="42000000-0000-4000-8000-000000000004" style="Dato"/><textElement textAlignment="Right"/><textFieldExpression><![
CDATA[${precio_medio} == null ? "Sin datos" : new java.text.DecimalFormat("#0.00 '€').format($F{precio_medio})]]></
textFieldExpression></textField>
<textField isBlankWhenNull="true"><reportElement x="455" y="0" width="100" height="20"
uuid="42000000-0000-4000-8000-000000000005" style="Dato"/><textFieldExpression><![CDATA[${categoria}]]></textFieldExpression></
textField>
<textField isBlankWhenNull="true"><reportElement x="0" y="24" width="130" height="18"
uuid="42000000-0000-4000-8000-000000000006" style="Dato"/><textFieldExpression><![CDATA[${primera_venta}]]></textFieldExpression></
textField>
<textField isBlankWhenNull="true"><reportElement x="130" y="24" width="130" height="18"
uuid="42000000-0000-4000-8000-000000000007" style="Dato"/><textFieldExpression><![CDATA[${ultima_venta}]]></textFieldExpression></
textField>
<textField><reportElement x="260" y="24" width="160" height="18" uuid="42000000-0000-4000-8000-000000000008" style="Dato"/
><textElement textAlignment="Center"/><textFieldExpression><![CDATA[${primera_venta} == null ? "Sin ventas" : $F{primera_venta} + "
+ $F{ultima_venta}]]></textFieldExpression></textField>
<textField pattern="#,##0.00 €" isBlankWhenNull="true">
<reportElement x="420" y="24" width="135" height="18" uuid="42000000-0000-4000-8000-000000000009" style="Dato">
<printWhenExpression><![CDATA[Boolean.TRUE.equals($P{mostrarDetalle})]]></printWhenExpression>
</reportElement>
<textElement textAlignment="Right"/>
<textFieldExpression><![CDATA[${importe_total} == null || $P{tipoIva} == null ? null :
Double.valueOf($F{importe_total}.doubleValue()) * (1.0d + $P{tipoIva}.doubleValue())]]></textFieldExpression>
</textField>
<textField><reportElement x="0" y="48" width="105" height="18" uuid="42000000-0000-4000-8000-000000000010" style="Dato"/
><textFieldExpression><![CDATA[${unidades_vendidas} == null ? "Sin ventas" : ($F{unidades_vendidas}.intValue() >= 6 ? "Premium" :
($F{unidades_vendidas}.intValue() >= 3 ? "Estándar" : "Económico"))]]></textFieldExpression></textField>
<textField><reportElement x="105" y="48" width="185" height="18" uuid="42000000-0000-4000-8000-000000000011" style="Dato"/
><textFieldExpression><![CDATA[${titulo} == null ? "" : $F{titulo}.trim().toUpperCase(java.util.Locale.ROOT)]]></textFieldExpression></
textField>
<textField><reportElement x="290" y="48" width="80" height="18" uuid="42000000-0000-4000-8000-000000000012" style="Dato"/
><textElement textAlignment="Right"/><textFieldExpression><![CDATA[${precio_medio} == null ? "-" : String.format(java.util.Locale.ROOT,
"%0.2f", Double.valueOf(Math.round($F{precio_medio}.doubleValue() * 100.0d) / 100.0d)]]></textFieldExpression></textField>
<textField><reportElement x="370" y="48" width="90" height="18" uuid="42000000-0000-4000-8000-000000000013" style="Dato"/
><textElement textAlignment="Center"/><textFieldExpression><![CDATA[${primera_venta} == null || $F{ultima_venta} == null ? "-" :
java.lang.Long.toString(java.time.temporal.ChronoUnit.DAYS.between(java.time.LocalDate.parse($F{primera_venta}),
java.time.LocalDate.parse($F{ultima_venta}))) + " días]]></textFieldExpression></textField>
<textField><reportElement x="460" y="48" width="95" height="18" uuid="42000000-0000-4000-8000-000000000014" style="Dato"/
><textElement textAlignment="Right"/><textFieldExpression><![CDATA[${unidades_vendidas} == null ? "0.0%" :
String.format(java.util.Locale.ROOT, "%0.1f%%", Double.valueOf($F{unidades_vendidas}.doubleValue() / Math.max(1.0d, $P{umbralUnidades} ==
null ? 1.0d : $P{umbralUnidades}.doubleValue()) * 100.0d)]]></textFieldExpression></textField>
</band>
<band height="14">
<printWhenExpression><![CDATA[${unidades_vendidas} != null && $P{umbralUnidades} != null &&
$F{unidades_vendidas}.intValue() >= $P{umbralUnidades}.intValue()]]></printWhenExpression>
<textField><reportElement x="0" y="0" width="555" height="12" uuid="42000000-0000-4000-8000-000000000015"/><textElement
textAlignment="Center"><font fontName="DejaVu Sans" size="8" isBold="true"/><textFieldExpression><![CDATA["Fila destacada:
" + $F{titulo} + " supera el umbral de " + $P{umbralUnidades} + " unidades"]></textFieldExpression></textField>
</band>
<band height="88" splitType="Stretch">
<printWhenExpression><![CDATA[${unidades_vendidas} != null]]></printWhenExpression>
<staticText>
<reportElement x="0" y="2" width="555" height="16" style="Cabecera"/>
<text><![CDATA[Detalle de ventas]]></text>
</staticText>
<subreport>
<reportElement x="0" y="22" width="555" height="60" isRemoveLineWhenBlank="true"/>
<subreportParameter name="tituloLibro">
<subreportParameterExpression><![CDATA[${titulo}]]></subreportParameterExpression>

```

```

        </subreportParameter>
        <connectionExpression><![CDATA[{$REPORT_CONNECTION}]]></connectionExpression>
        <subreportExpression><![CDATA["reports/subinforme_ventas_detalle.jasper"]]></subreportExpression>
    </subreport>
</band>
<band height="104" splitType="Stretch">
    <printWhenExpression><![CDATA[{$unidades_vendidas} != null]]></printWhenExpression>
    <staticText>
        <reportElement x="0" y="2" width="555" height="16" style="Cabecera"/>
        <text><![CDATA[Top 3 ventas por cantidad]]></text>
    </staticText>
    <componentElement>
        <reportElement x="0" y="22" width="555" height="76"/>
        <c:table xmlns:c="http://jasperreports.sourceforge.net/jasperreports/components"
            xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports/components http://jasperreports.sourceforge.net/xsd/components.xsd">
            <datasetRun subDataset="DatasetTopVentas">
                <datasetParameter name="tituloLibro">
                    <datasetParameterExpression><![CDATA[{$F{titulo}]]></datasetParameterExpression>
                </datasetParameter>
                <connectionExpression><![CDATA[{$REPORT_CONNECTION}]]></connectionExpression>
            </datasetRun>
            <c:column width="255">
                <c:columnHeader style="M5TablaCabecera" height="20"><staticText><reportElement x="0" y="0" width="255"
height="20"/><text><![CDATA[Fecha]]></text></staticText></c:columnHeader>
                <c:detailCell style="M5TablaDetalle" height="18"><textField><reportElement x="0" y="0" width="255" height="18"/
><textFieldExpression><![CDATA[{$F{fecha_venta}]]></textFieldExpression></textField></c:detailCell>
            </c:column>
            <c:column width="100">
                <c:columnHeader style="M5TablaCabecera" height="20"><staticText><reportElement x="0" y="0" width="100"
height="20"/><textElement textAlignment="Right"/><text><![CDATA[Cantidad]]></text></staticText></c:columnHeader>
                <c:detailCell style="M5TablaDetalle" height="18"><textField><reportElement x="0" y="0" width="100" height="18"/
><textElement textAlignment="Right"/><textFieldExpression><![CDATA[{$F{cantidad}]]></textFieldExpression></textField></c:detailCell>
            </c:column>
            <c:column width="200">
                <c:columnHeader style="M5TablaCabecera" height="20"><staticText><reportElement x="0" y="0" width="200"
height="20"/><textElement textAlignment="Right"/><text><![CDATA[Precio unitario]]></text></staticText></c:columnHeader>
                <c:detailCell style="M5TablaDetalle" height="18"><textField pattern="#,##0.00 €"><reportElement x="0" y="0"
width="200" height="18"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[{$F{precio_unitario}]]></
textFieldExpression></textField></c:detailCell>
            </c:column>
        </c:table>
    </componentElement>
</band>
</detail>
<pageFooter>
    <band height="62">
        <staticText><reportElement x="0" y="4" width="120" height="15" uuid="43000000-0000-4000-8000-000000000001"/><text><![
CDATA[Total de títulos:]]></text></staticText>
        <textField evaluationTime="Report"><reportElement x="120" y="4" width="60" height="15"
uuid="43000000-0000-4000-8000-000000000002"/><textFieldExpression><![CDATA[{$REPORT_COUNT}]]></textFieldExpression></textField>
        <textField><reportElement x="190" y="28" width="180" height="15" uuid="43000000-0000-4000-8000-000000000003"/><textElement
textAlignment="Right"/><textFieldExpression><![CDATA["Página " + {$PAGE_NUMBER} + " de"]]></textFieldExpression></textField>
        <textField evaluationTime="Report"><reportElement x="375" y="28" width="35" height="15"
uuid="43000000-0000-4000-8000-000000000004"/><textFieldExpression><![CDATA[{$PAGE_NUMBER}]]></textFieldExpression></textField>
        <staticText><reportElement x="300" y="4" width="120" height="15" uuid="43000000-0000-4000-8000-000000000005"/><text><![
CDATA[Subtotal página:]]></text></staticText>
        <textField pattern="#,##0.00 €"><reportElement x="420" y="4" width="135" height="15"
uuid="43000000-0000-4000-8000-000000000006"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[{$TotalPagina}]]></
textFieldExpression></textField>
    </band>
</pageFooter>
<summary>
    <band height="700">
        <staticText><reportElement x="0" y="5" width="205" height="18" uuid="44000000-0000-4000-8000-000000000001"/><text><![
CDATA[Total de unidades vendidas:]]></text></staticText>
        <textField><reportElement x="205" y="5" width="80" height="18" uuid="44000000-0000-4000-8000-000000000002"/><textElement
textAlignment="Right"/><textFieldExpression><![CDATA[{$TotalUnidades}]]></textFieldExpression></textField>
        <staticText><reportElement x="300" y="5" width="120" height="18" uuid="44000000-0000-4000-8000-000000000003"/><text><![
CDATA[Importe total:]]></text></staticText>
        <textField pattern="#,##0.00 €"><reportElement x="420" y="5" width="135" height="18"
uuid="44000000-0000-4000-8000-000000000004"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[{$TotalImporte}]]></
textFieldExpression></textField>
        <staticText><reportElement x="0" y="30" width="205" height="18" uuid="44000000-0000-4000-8000-000000000005"/><text><![
CDATA[Precio medio agregado:]]></text></staticText>
        <textField pattern="#,##0.00 €"><reportElement x="205" y="30" width="80" height="18"
uuid="44000000-0000-4000-8000-000000000006"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[{$PrecioMedio}]]></
textFieldExpression></textField>
        <staticText><reportElement x="300" y="30" width="120" height="18" uuid="44000000-0000-4000-8000-000000000007"/><text><![
CDATA[Precio máximo:]]></text></staticText>
        <textField pattern="#,##0.00 €"><reportElement x="420" y="30" width="135" height="18"
uuid="44000000-0000-4000-8000-000000000008"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[{$PrecioMaximo}]]></
textFieldExpression></textField>
        <staticText><reportElement x="0" y="55" width="205" height="18" uuid="44000000-0000-4000-8000-000000000009"/><text><![
CDATA[Número de libros:]]></text></staticText>
        <textField><reportElement x="205" y="55" width="80" height="18" uuid="44000000-0000-4000-8000-000000000010"/><textElement
textAlignment="Right"/><textFieldExpression><![CDATA[{$NumeroLibros}]]></textFieldExpression></textField>
        <staticText><reportElement x="300" y="55" width="120" height="18" uuid="44000000-0000-4000-8000-000000000011"/><text><![
CDATA[Importe con IVA:]]></text></staticText>
        <textField pattern="#,##0.00 €"><reportElement x="420" y="55" width="135" height="18"
uuid="44000000-0000-4000-8000-000000000012"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[{$ImporteConIva}]]></
textFieldExpression></textField>
    </band>
</summary>

```

```

        <textField><reportElement x="0" y="80" width="555" height="18" uuid="44000000-0000-4000-8000-000000000013"/><textField
textAlignment="Center"/><textFieldExpression><![CDATA[String.format(java.util.Locale.ROOT, "Resumen: %d títulos · %d unidades · %.2f €",
${NumeroLibros}, ${TotalUnidades}, ${TotalImporte})]]></textFieldExpression></textField>
        <textField><reportElement x="0" y="103" width="350" height="18" uuid="44000000-0000-4000-8000-000000000014"/><textField
textAlignment="Center"><font fontName="DejaVu Sans" size="10" isBold="true"/></textField><textFieldExpression><![
CDATA[${V{TotalUnidades}} != null && ${P{umbralUnidades}} != null && ${V{TotalUnidades}}.intValue() >= ${P{umbralUnidades}}.intValue() ?
"Objetivo de ventas alcanzado" : "Objetivo de ventas pendiente"]]></textFieldExpression></textField>
        <textField><reportElement x="360" y="103" width="195" height="18" uuid="44000000-0000-4000-8000-000000000015"/
><textFieldExpression><![CDATA["Resultados encontrados: " + ${V{REPORT_COUNT}}]]></textFieldExpression></textField>
        <staticText><reportElement x="0" y="140" width="555" height="20" style="Cabecera"/><text><![CDATA[Ventas por categoría –
importe]]></text></staticText>
        <barChart>
            <chart>
                <reportElement x="0" y="165" width="555" height="250"/>
                <chartTitle><titleExpression><![CDATA["Ventas por categoría"]]></titleExpression></chartTitle>
                <chartSubtitle/>
                <chartLegend position="Bottom"/>
            </chart>
            <categoryDataset>
                <dataset>
                    <datasetRun subDataset="DatasetVentasPorCategoría">
                        <connectionExpression><![CDATA[${P{REPORT_CONNECTION}}]></connectionExpression>
                    </datasetRun>
                </dataset>
                <categorySeries>
                    <seriesExpression><![CDATA["Importe"]]></seriesExpression>
                    <categoryExpression><![CDATA[${F{categoria_grafico}}]></categoryExpression>
                    <valueExpression><![CDATA[${F{importe_categoria}}]></valueExpression>
                </categorySeries>
            </categoryDataset>
            <barPlot>
                <plot/>
                <itemLabel/>
                <categoryAxisFormat><axisFormat/></categoryAxisFormat>
                <valueAxisFormat><axisFormat/></valueAxisFormat>
            </barPlot>
        </barChart>
        <staticText><reportElement x="0" y="430" width="555" height="20" style="Cabecera"/><text><![CDATA[Ventas por categoría y
año]]></text></staticText>
        <crossstab>
            <reportElement x="0" y="455" width="555" height="225"/>
            <crossstabDataset>
                <dataset>
                    <datasetRun subDataset="DatasetCrossstabVentas">
                        <connectionExpression><![CDATA[${P{REPORT_CONNECTION}}]></connectionExpression>
                    </datasetRun>
                </dataset>
            </crossstabDataset>
            <rowGroup name="CategoriaCross" width="150" totalPosition="End">
                <bucket class="java.lang.String"><bucketExpression><![CDATA[${F{categoria_cross}}]></bucketExpression></bucket>
                <crossstabRowHeader><cellContents style="M5CrossstabCabecera"><textField><reportElement x="0" y="0" width="150"
height="34"/><textField verticalAlignment="Middle"/><textFieldExpression><![CDATA[${V{CategoriaCross}}]></textFieldExpression></
textField></cellContents></crossstabRowHeader>
                <crossstabTotalRowHeader><cellContents style="M5CrossstabTotal"><staticText><reportElement x="0" y="0" width="150"
height="34"/><textField verticalAlignment="Middle"/><text><![CDATA[TOTAL]]></text></staticText></cellContents></
crossstabTotalRowHeader>
            </rowGroup>
            <columnGroup name="AnioCross" height="28" totalPosition="End">
                <bucket class="java.lang.String"><bucketExpression><![CDATA[${F{anio_cross}}]></bucketExpression></bucket>
                <crossstabColumnHeader><cellContents style="M5CrossstabCabecera"><textField><reportElement x="0" y="0" width="100"
height="28"/><textField textAlignment="Center" verticalAlignment="Middle"/><textFieldExpression><![CDATA[${V{AnioCross}}]></
textFieldExpression></textField></cellContents></crossstabColumnHeader>
                <crossstabTotalColumnHeader><cellContents style="M5CrossstabTotal"><staticText><reportElement x="0" y="0" width="100"
height="28"/><textField textAlignment="Center" verticalAlignment="Middle"/><text><![CDATA[TOTAL]]></text></staticText></
cellContents></crossstabTotalColumnHeader>
            </columnGroup>
            <measure name="ImporteCross" class="java.lang.Double" calculation="Sum"><measureExpression><![
CDATA[${F{importe_cross}}]></measureExpression></measure>
            <measure name="VentasCross" class="java.lang.Integer" calculation="Sum"><measureExpression><![
CDATA[${F{ventas_cross}}]></measureExpression></measure>
            <crossstabCell width="100" height="34"><cellContents style="M5CrossstabDetalle"><textField pattern="#,##0.00
€"><reportElement x="0" y="0" width="100" height="18"/><textField textAlignment="Right"/><textFieldExpression><![
CDATA[${V{importe_cross}}]></textFieldExpression></textField><textField><reportElement x="0" y="18" width="100" height="14"/><textField
textAlignment="Right"/><textFieldExpression><![CDATA[${V{VentasCross}} + " ventas"]></textFieldExpression></textField></cellContents></
crossstabCell>
            <crossstabCell width="100" height="34" rowTotalGroup="CategoriaCross"><cellContents style="M5CrossstabTotal"><textField
pattern="#,##0.00 €"><reportElement x="0" y="0" width="100" height="18"/><textField textAlignment="Right"/><textFieldExpression><![
CDATA[${V{importe_cross}}]></textFieldExpression></textField><textField><reportElement x="0" y="18" width="100" height="14"/><textField
textAlignment="Right"/><textFieldExpression><![CDATA[${V{VentasCross}} + " ventas"]></textFieldExpression></textField></cellContents></
crossstabCell>
            <crossstabCell width="100" height="34" columnTotalGroup="AnioCross"><cellContents style="M5CrossstabTotal"><textField
pattern="#,##0.00 €"><reportElement x="0" y="0" width="100" height="18"/><textField textAlignment="Right"/><textFieldExpression><![
CDATA[${V{importe_cross}}]></textFieldExpression></textField><textField><reportElement x="0" y="18" width="100" height="14"/><textField
textAlignment="Right"/><textFieldExpression><![CDATA[${V{VentasCross}} + " ventas"]></textFieldExpression></textField></cellContents></
crossstabCell>
            <crossstabCell width="100" height="34" rowTotalGroup="CategoriaCross" columnTotalGroup="AnioCross"><cellContents
style="M5CrossstabTotal"><textField pattern="#,##0.00 €"><reportElement x="0" y="0" width="100" height="18"/><textField
textAlignment="Right"/><textFieldExpression><![CDATA[${V{importe_cross}}]></textFieldExpression></textField><textField><reportElement
x="0" y="18" width="100" height="14"/><textField textAlignment="Right"/><textFieldExpression><![CDATA[${V{VentasCross}} + " ventas"]></
textFieldExpression></textField></cellContents></crossstabCell>
        </crossstab>
    </band>

```

```
</summary>
</jasperReport>
```

Explicación línea por línea

Línea 1	<code><?xml version="1.0" encoding="UTF-8"?></code> → Declara XML 1.0 y codificación UTF-8 para que nombres, textos y símbolos del informe se interpreten correctamente.
Línea 2	<code><jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports"</code> → Abre el documento raíz <code>jasperReport</code> del informe y fija el namespace principal de JasperReports.
Línea 3	<code>xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"</code> → Declara el namespace XML Schema Instance usado por <code>xsi:schemaLocation</code> para validar el documento.
Línea 4	<code>xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/jasperreport.xsd"</code> → Relaciona el namespace de JasperReports con su XSD para que Studio y el compilador validen la estructura.
Línea 5	<code>name="informe_ventas"</code> → Asigna al documento JasperReports el nombre interno <code>informe_ventas</code> .
Línea 6	<code>language="java"</code> → Configura <code>language=java</code> para evaluar expresiones con el lenguaje Java.
Línea 7	<code>pageWidth="595"</code> → Fija el ancho físico de página en 595 puntos.
Línea 8	<code>pageHeight="842"</code> → Fija la altura física de página en 842 puntos.
Línea 9	<code>columnWidth="555"</code> → Fija el ancho útil de la columna de contenido en 555 puntos.
Línea 10	<code>leftMargin="20"</code> → Fija el margen izquierdo del informe en 20 puntos.
Línea 11	<code>rightMargin="20"</code> → Fija el margen derecho del informe en 20 puntos.
Línea 12	<code>topMargin="20"</code> → Fija el margen superior del informe en 20 puntos.
Línea 13	<code>bottomMargin="20"</code> → Fija el margen inferior del informe en 20 puntos y completa la apertura del elemento raíz.
Línea 14	<code>uuid="3d2c2bd7-3b93-4da9-8b60-6b3c45674c91"></code> → Asigna el UUID de diseño <code>3d2c2bd7-3b93-4da9-8b60-6b3c45674c91</code> para identificar de forma estable el informe en Studio.
Línea 15	<code><property name="com.jaspersoft.studio.data.defaultdataadapter" value="SQLiteEditorial"/></code> → Indica a Jaspersoft Studio que use el Data Adapter <code>SQLiteEditorial</code> como conexión de diseño por defecto.
Línea 16	<code><template><![CDATA["resources/styles/EditorialStyles.jrtx"]]></template></code> → Importa la plantilla externa cuya expresión CDATA devuelve <code>resources/styles/EditorialStyles.jrtx</code> .
Línea 17	<code><style name="Sans_Normal" isDefault="true" fontName="DejaVu Sans" fontSize="10"/></code> → Declara el estilo <code>Sans_Normal</code> ; es el estilo por defecto, fuente <code>DejaVu Sans</code> , tamaño 10.
Línea 18	<code><style name="TituloPrincipal" style="Sans_Normal" fontSize="18" isBold="true" forecolor="#173F6B"/></code> → Declara el estilo <code>TituloPrincipal</code> ; hereda de <code>Sans_Normal</code> , tamaño 18.
Línea 19	<code><style name="Cabecera" style="Sans_Normal" fontSize="9" isBold="true" forecolor="#173F6B"/></code> → Declara el estilo <code>Cabecera</code> ; hereda de <code>Sans_Normal</code> , tamaño 9.
Línea 20	<code><style name="Dato" style="Sans_Normal" fontSize="9"/></code> → Declara el estilo <code>Dato</code> ; hereda de <code>Sans_Normal</code> , tamaño 9.
Línea 21	<code><style name="UnidadesCondicional" style="Dato" isBold="true"></code> → Declara el estilo <code>UnidadesCondicional</code> ; hereda de <code>Dato</code> .
Línea 22	<code><conditionalStyle></code> → Abre una variante condicional del estilo; sólo se aplicará cuando su <code>conditionExpression</code> sea verdadera.
Línea 23	<code><conditionExpression><![CDATA[\${unidades_vendidas} != null && \${umbralUnidades} != null && \${unidades_vendidas}.intValue() >= \${umbralUnidades}.intValue()]]></conditionExpre...></code> → Define la condición booleana que activa el estilo condicional usando field <code>unidades_vendidas</code> , field <code>unidades_vendidas</code> , parámetro <code>umbralUnidades</code> , parámetro <code>umbralUnidades</code> .
Línea 24	<code><style forecolor="#1B5E20"/></code> → Declara el estilo <code>None</code> .

Línea 25	<code></conditionalStyle></code> → Cierra <code>conditionalStyle</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 26	<code><conditionalStyle></code> → Abre una variante condicional del estilo; sólo se aplicará cuando su <code>conditionExpression</code> sea verdadera.
Línea 27	<code><conditionExpression><![CDATA[{\$F{unidades_vendidas} != null && \$P{umbralUnidades} != null && \$F{unidades_vendidas}.intValue() >= 3 && \$F{unidades_vendidas}.intValue() < \$P{umbralUnidades}}]</code> → Define la condición booleana que activa el estilo condicional usando field <code>unidades_vendidas</code> , field <code>unidades_vendidas</code> , field <code>unidades_vendidas</code> , parámetro <code>umbralUnidades</code> , parámetro <code>umbralUnidades</code> .
Línea 28	<code><style forecolor="#1D5D88"/></code> → Declara el estilo <code>None</code> .
Línea 29	<code></conditionalStyle></code> → Cierra <code>conditionalStyle</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 30	<code><conditionalStyle></code> → Abre una variante condicional del estilo; sólo se aplicará cuando su <code>conditionExpression</code> sea verdadera.
Línea 31	<code><conditionExpression><![CDATA[{\$F{unidades_vendidas} == null \$F{unidades_vendidas}.intValue() < 3}]]></conditionExpression></code> → Define la condición booleana que activa el estilo condicional usando field <code>unidades_vendidas</code> , field <code>unidades_vendidas</code> .
Línea 32	<code><style forecolor="#9D3429"/></code> → Declara el estilo <code>None</code> .
Línea 33	<code></conditionalStyle></code> → Cierra <code>conditionalStyle</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 34	<code></style></code> → Cierra <code>style</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 35	<code><style name="M5TableHeader" style="Dato" mode="Opaque" backcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/></code> → Declara el estilo <code>M5TableHeader</code> ; hereda de <code>Dato</code> , fondo <code>#EAF2F8</code> .
Línea 36	<code><style name="M5TableDetail" style="Dato"/></code> → Declara el estilo <code>M5TableDetail</code> ; hereda de <code>Dato</code> .
Línea 37	<code><style name="M5CrossHeader" style="Dato" mode="Opaque" backcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/></code> → Declara el estilo <code>M5CrossHeader</code> ; hereda de <code>Dato</code> , fondo <code>#EAF2F8</code> .
Línea 38	<code><style name="M5CrossDetail" style="Dato" mode="Opaque" backcolor="#FFFFFF"/></code> → Declara el estilo <code>M5CrossDetail</code> ; hereda de <code>Dato</code> , fondo <code>#FFFFFF</code> .
Línea 39	<code><style name="M5CrossTotal" style="Dato" mode="Opaque" backcolor="#D6EAF8" forecolor="#173F6B" isBold="true"/></code> → Declara el estilo <code>M5CrossTotal</code> ; hereda de <code>Dato</code> , fondo <code>#D6EAF8</code> .
Línea 40	<code><subDataset name="DatasetTopVentas"></code> → Declara el subdataset <code>DatasetTopVentas</code> , con consulta y fields propios independientes del dataset principal.
Línea 41	<code><parameter name="tituloLibro" class="java.lang.String"/></code> → Declara el parámetro <code>tituloLibro</code> con tipo <code>java.lang.String</code> y lo expone al diálogo de parámetros de Studio.
Línea 42	<code><queryString language="sql"></code> → Abre la consulta SQL que JasperReports ejecutará para el dataset actual.
Línea 43	<code><![CDATA[</code> → Abre <code>CDATA</code> para escribir SQL o una expresión Java sin que sus caracteres especiales se interpreten como XML.
Línea 44	<code>SELECT fecha_venta, cantidad, precio_unitario</code> → Cláusula SQL <code>SELECT</code> : selecciona y calcula las columnas que devolverá la consulta.
Línea 45	<code>FROM ventas</code> → Cláusula SQL <code>FROM</code> : define la tabla base de la consulta.
Línea 46	<code>WHERE titulo_libro = \$P{tituloLibro}</code> → Cláusula SQL <code>WHERE</code> : aplica el filtro de filas.
Línea 47	<code>ORDER BY cantidad DESC, fecha_venta</code> → Cláusula SQL <code>ORDER BY</code> : ordena el resultado que recibirá JasperReports.
Línea 48	<code>LIMIT 3</code> → Cláusula SQL <code>LIMIT</code> : limita el número de filas devueltas.
Línea 49	<code>]]></code> → Cierra el bloque <code>CDATA</code> y devuelve el control al parser XML.
Línea 50	<code></queryString></code> → Cierra la consulta SQL del dataset actual.
Línea 51	<code><field name="fecha_venta" class="java.lang.String"/></code> → Declara el field <code>fecha_venta</code> con tipo <code>java.lang.String</code> para mapear una columna del dataset.

Línea 52	<code><field name="cantidad" class="java.lang.Integer"/></code> → Declara el field cantidad con tipo Java <code>java.lang.Integer</code> para mapear una columna del dataset.
Línea 53	<code><field name="precio_unitario" class="java.lang.Double"/></code> → Declara el field precio_unitario con tipo Java <code>java.lang.Double</code> para mapear una columna del dataset.
Línea 54	<code></subDataset></code> → Finaliza el subdataset auxiliar y vuelve al nivel del informe.
Línea 55	<code><subDataset name="DatasetVentasPorCategoria"></code> → Declara el subdataset <code>DatasetVentasPorCategoria</code> , con consulta y fields propios independientes del dataset principal.
Línea 56	<code><queryString language="sql"></code> → Abre la consulta SQL que JasperReports ejecutará para el dataset actual.
Línea 57	<code><![CDATA[</code> → Abre CDATA para escribir SQL o una expresión Java sin que sus caracteres especiales se interpreten como XML.
Línea 58	<code>SELECT l.categoria AS categoria_grafico,</code> → Cláusula SQL <code>SELECT</code> : selecciona y calcula las columnas que devolverá la consulta.
Línea 59	<code>COALESCE(SUM(v.cantidad * v.precio_unitario), 0.0) AS importe_categoria</code> → Calcula o selecciona un valor SQL y lo expone con el alias <code>importe_categoria</code> , que después coincide con un field del subdataset.
Línea 60	<code>FROM libros l</code> → Cláusula SQL <code>FROM</code> : define la tabla base de la consulta.
Línea 61	<code>LEFT JOIN ventas v ON l.titulo = v.titulo_libro</code> → Cláusula SQL <code>LEFT JOIN</code> : une datos conservando las filas del lado izquierdo aunque no tengan ventas.
Línea 62	<code>GROUP BY l.categoria</code> → Cláusula SQL <code>GROUP BY</code> : agrupa las filas antes de evaluar las funciones agregadas.
Línea 63	<code>ORDER BY l.categoria</code> → Cláusula SQL <code>ORDER BY</code> : ordena el resultado que recibirá JasperReports.
Línea 64	<code>]]></code> → Cierra el bloque CDATA y devuelve el control al parser XML.
Línea 65	<code></queryString></code> → Cierra la consulta SQL del dataset actual.
Línea 66	<code><field name="categoria_grafico" class="java.lang.String"/></code> → Declara el field <code>categoria_grafico</code> con tipo Java <code>java.lang.String</code> para mapear una columna del dataset.
Línea 67	<code><field name="importe_categoria" class="java.lang.Double"/></code> → Declara el field <code>importe_categoria</code> con tipo Java <code>java.lang.Double</code> para mapear una columna del dataset.
Línea 68	<code></subDataset></code> → Finaliza el subdataset auxiliar y vuelve al nivel del informe.
Línea 69	<code><subDataset name="DatasetCrosstabVentas"></code> → Declara el subdataset <code>DatasetCrosstabVentas</code> , con consulta y fields propios independientes del dataset principal.
Línea 70	<code><queryString language="sql"></code> → Abre la consulta SQL que JasperReports ejecutará para el dataset actual.
Línea 71	<code><![CDATA[</code> → Abre CDATA para escribir SQL o una expresión Java sin que sus caracteres especiales se interpreten como XML.
Línea 72	<code>SELECT l.categoria AS categoria_cross,</code> → Cláusula SQL <code>SELECT</code> : selecciona y calcula las columnas que devolverá la consulta.
Línea 73	<code>SUBSTR(v.fecha_venta, 1, 4) AS anio_cross,</code> → Calcula o selecciona un valor SQL y lo expone con el alias <code>anio_cross</code> , que después coincide con un field del subdataset.
Línea 74	<code>(v.cantidad * v.precio_unitario) AS importe_cross,</code> → Calcula o selecciona un valor SQL y lo expone con el alias <code>importe_cross</code> , que después coincide con un field del subdataset.
Línea 75	<code>1 AS ventas_cross</code> → Calcula o selecciona un valor SQL y lo expone con el alias <code>ventas_cross</code> , que después coincide con un field del subdataset.
Línea 76	<code>FROM ventas v</code> → Cláusula SQL <code>FROM</code> : define la tabla base de la consulta.
Línea 77	<code>JOIN libros l ON l.titulo = v.titulo_libro</code> → Cláusula SQL <code>JOIN</code> : une las filas que cumplen la relación indicada.
Línea 78	<code>ORDER BY l.categoria, anio_cross, v.fecha_venta</code> → Cláusula SQL <code>ORDER BY</code> : ordena el resultado que recibirá JasperReports.

Línea 79	<code>]]></code> → Cierra el bloque CDATA y devuelve el control al parser XML.
Línea 80	<code></queryString></code> → Cierra la consulta SQL del dataset actual.
Línea 81	<code><field name="categoria_cross" class="java.lang.String"/></code> → Declara el field <code>categoria_cross</code> con tipo Java <code>java.lang.String</code> para mapear una columna del dataset.
Línea 82	<code><field name="anio_cross" class="java.lang.String"/></code> → Declara el field <code>anio_cross</code> con tipo Java <code>java.lang.String</code> para mapear una columna del dataset.
Línea 83	<code><field name="importe_cross" class="java.lang.Double"/></code> → Declara el field <code>importe_cross</code> con tipo Java <code>java.lang.Double</code> para mapear una columna del dataset.
Línea 84	<code><field name="ventas_cross" class="java.lang.Integer"/></code> → Declara el field <code>ventas_cross</code> con tipo Java <code>java.lang.Integer</code> para mapear una columna del dataset.
Línea 85	<code></subDataset></code> → Finaliza el subdataset auxiliar y vuelve al nivel del informe.
Línea 86	<code><parameter name="usuario" class="java.lang.String" isForPrompting="true"/></code> → Declara el parámetro <code>usuario</code> con tipo <code>java.lang.String</code> y lo expone al diálogo de parámetros de Studio.
Línea 87	<code><parameter name="fechaInforme" class="java.util.Date" isForPrompting="true"></code> → Declara el parámetro <code>fechaInforme</code> con tipo <code>java.util.Date</code> y lo expone al diálogo de parámetros de Studio.
Línea 88	<code><defaultValueExpression><![CDATA[new java.util.Date()]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 89	<code></parameter></code> → Cierra <code>parameter</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 90	<code><parameter name="departamento" class="java.lang.String" isForPrompting="true"></code> → Declara el parámetro <code>departamento</code> con tipo <code>java.lang.String</code> y lo expone al diálogo de parámetros de Studio.
Línea 91	<code><defaultValueExpression><![CDATA["General"]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 92	<code></parameter></code> → Cierra <code>parameter</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 93	<code><parameter name="periodo" class="java.lang.String" isForPrompting="true"></code> → Declara el parámetro <code>periodo</code> con tipo <code>java.lang.String</code> y lo expone al diálogo de parámetros de Studio.
Línea 94	<code><defaultValueExpression><![CDATA["Mensual"]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 95	<code></parameter></code> → Cierra <code>parameter</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 96	<code><parameter name="tipoIva" class="java.lang.Double" isForPrompting="true"></code> → Declara el parámetro <code>tipoIva</code> con tipo <code>java.lang.Double</code> y lo expone al diálogo de parámetros de Studio.
Línea 97	<code><defaultValueExpression><![CDATA[Double.valueOf(0.21d)]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 98	<code></parameter></code> → Cierra <code>parameter</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 99	<code><parameter name="mostrarDetalle" class="java.lang.Boolean" isForPrompting="true"></code> → Declara el parámetro <code>mostrarDetalle</code> con tipo <code>java.lang.Boolean</code> y lo expone al diálogo de parámetros de Studio.
Línea 100	<code><defaultValueExpression><![CDATA[Boolean.TRUE]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 101	<code></parameter></code> → Cierra <code>parameter</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 102	<code><parameter name="categoria" class="java.lang.String" isForPrompting="true"/></code> → Declara el parámetro <code>categoria</code> con tipo <code>java.lang.String</code> y lo expone al diálogo de parámetros de Studio.
Línea 103	<code><parameter name="precioMinimo" class="java.lang.Double" isForPrompting="true"/></code> → Declara el parámetro <code>precioMinimo</code> con tipo <code>java.lang.Double</code> y lo expone al diálogo de parámetros de Studio.
Línea 104	<code><parameter name="precioMaximo" class="java.lang.Double" isForPrompting="true"/></code> → Declara el parámetro <code>precioMaximo</code> con tipo <code>java.lang.Double</code> y lo expone al diálogo de parámetros de Studio.

Línea 105	<code><parameter name="umbralUnidades" class="java.lang.Integer" isForPrompting="true"></code> → Declara el parámetro <code>umbralUnidades</code> con tipo <code>java.lang.Integer</code> y lo expone al diálogo de parámetros de Studio.
Línea 106	<code><defaultValueExpression><![CDATA[Integer.valueOf(5)]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 107	<code></parameter></code> → Cierra <code>parameter</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 108	<code><parameter name="textoBusqueda" class="java.lang.String" isForPrompting="true"/></code> → Declara el parámetro <code>textoBusqueda</code> con tipo <code>java.lang.String</code> y lo expone al diálogo de parámetros de Studio.
Línea 109	<code><parameter name="categoriasLista" class="java.util.Collection" isForPrompting="false"></code> → Declara el parámetro <code>categoriasLista</code> con tipo <code>java.util.Collection</code> como parámetro interno no solicitado al usuario.
Línea 110	<code><defaultValueExpression><![CDATA[java.util.Arrays.asList("Novela", "Realismo mágico", "Cuento", "Poesía")]]></defaultValueExpression></code> → Define el valor que tomará el parámetro cuando el llamador no suministre uno explícitamente.
Línea 111	<code></parameter></code> → Cierra <code>parameter</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 112	<code><queryString language="sql"></code> → Abre la consulta SQL que JasperReports ejecutará para el dataset actual.
Línea 113	<code><![CDATA[</code> → Abre CDATA para escribir SQL o una expresión Java sin que sus caracteres especiales se interpreten como XML.
Línea 114	<code>SELECT l.titulo,</code> → Cláusula SQL <code>SELECT</code> : selecciona y calcula las columnas que devolverá la consulta.
Línea 115	<code>l.categoria,</code> → Aporta el valor literal <code>l.categoria</code> , al elemento XML/POM actualmente abierto.
Línea 116	<code>SUM(v.cantidad) AS unidades_vendidas,</code> → Calcula o selecciona un valor SQL y lo expone con el alias <code>unidades_vendidas</code> , que después coincide con un field del subdataset.
Línea 117	<code>SUM(v.cantidad * v.precio_unitario) AS importe_total,</code> → Calcula o selecciona un valor SQL y lo expone con el alias <code>importe_total</code> , que después coincide con un field del subdataset.
Línea 118	<code>AVG(v.precio_unitario) AS precio_medio,</code> → Calcula o selecciona un valor SQL y lo expone con el alias <code>precio_medio</code> , que después coincide con un field del subdataset.
Línea 119	<code>MIN(v.fecha_venta) AS primera_venta,</code> → Calcula o selecciona un valor SQL y lo expone con el alias <code>primera_venta</code> , que después coincide con un field del subdataset.
Línea 120	<code>MAX(v.fecha_venta) AS ultima_venta</code> → Calcula o selecciona un valor SQL y lo expone con el alias <code>ultima_venta</code> , que después coincide con un field del subdataset.
Línea 121	<code>FROM libros l</code> → Cláusula SQL <code>FROM</code> : define la tabla base de la consulta.
Línea 122	<code>LEFT JOIN ventas v ON l.titulo = v.titulo_libro</code> → Cláusula SQL <code>LEFT JOIN</code> : une datos conservando las filas del lado izquierdo aunque no tengan ventas.
Línea 123	<code>WHERE (\$P{categoria} IS NULL OR l.categoria = \$P{categoria})</code> → Cláusula SQL <code>WHERE</code> : aplica el filtro de filas.
Línea 124	<code>AND (\$P{precioMinimo} IS NULL OR l.precio >= \$P{precioMinimo})</code> → Añade otra condición lógica al filtro SQL, combinándola con la condición anterior.
Línea 125	<code>AND (\$P{precioMaximo} IS NULL OR l.precio <= \$P{precioMaximo})</code> → Añade otra condición lógica al filtro SQL, combinándola con la condición anterior.
Línea 126	<code>AND (\$P{textoBusqueda} IS NULL OR \$P{textoBusqueda} = '' OR l.titulo LIKE '%' \$P{textoBusqueda} '%')</code> → Añade otra condición lógica al filtro SQL, combinándola con la condición anterior.
Línea 127	<code>AND \$X{IN, l.categoria, categoriasLista}</code> → Añade otra condición lógica al filtro SQL, combinándola con la condición anterior.
Línea 128	<code>GROUP BY l.titulo, l.categoria</code> → Cláusula SQL <code>GROUP BY</code> : agrupa las filas antes de evaluar las funciones agregadas.

Línea 129	ORDER BY l.categoria, COALESCE(importe_total, 0) DESC, l.titulo → Cláusula SQL ORDER BY: ordena el resultado que recibirá JasperReports.
Línea 130	]]> → Cierra el bloque CDATA y devuelve el control al parser XML.
Línea 131	</queryString> → Cierra la consulta SQL del dataset actual.
Línea 132	<field name="titulo" class="java.lang.String"/> → Declara el field titulo con tipo Java java.lang.String para mapear una columna del dataset.
Línea 133	<field name="categoria" class="java.lang.String"/> → Declara el field categoria con tipo Java java.lang.String para mapear una columna del dataset.
Línea 134	<field name="unidades_vendidas" class="java.lang.Integer"/> → Declara el field unidades_vendidas con tipo Java java.lang.Integer para mapear una columna del dataset.
Línea 135	<field name="importe_total" class="java.lang.Double"/> → Declara el field importe_total con tipo Java java.lang.Double para mapear una columna del dataset.
Línea 136	<field name="precio_medio" class="java.lang.Double"/> → Declara el field precio_medio con tipo Java java.lang.Double para mapear una columna del dataset.
Línea 137	<field name="primera_venta" class="java.lang.String"/> → Declara el field primera_venta con tipo Java java.lang.String para mapear una columna del dataset.
Línea 138	<field name="ultima_venta" class="java.lang.String"/> → Declara el field ultima_venta con tipo Java java.lang.String para mapear una columna del dataset.
Línea 139	<variable name="TotalUnidades" class="java.lang.Integer" calculation="Sum" resetType="Report"> → Declara la variable TotalUnidades con cálculo Sum y reinicio Report.
Línea 140	<variableExpression><![CDATA[\$F{unidades_vendidas}]]></variableExpression> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field unidades_vendidas.
Línea 141	</variable> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 142	<variable name="TotalImporte" class="java.lang.Double" calculation="Sum" resetType="Report"> → Declara la variable TotalImporte con cálculo Sum y reinicio Report.
Línea 143	<variableExpression><![CDATA[\$F{importe_total}]]></variableExpression> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field importe_total.
Línea 144	</variable> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 145	<variable name="TotalPagina" class="java.lang.Double" calculation="Sum" resetType="Page"> → Declara la variable TotalPagina con cálculo Sum y reinicio Page.
Línea 146	<variableExpression><![CDATA[\$F{importe_total}]]></variableExpression> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field importe_total.
Línea 147	</variable> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 148	<variable name="PrecioMedio" class="java.lang.Double" calculation="Average" resetType="Report"> → Declara la variable PrecioMedio con cálculo Average y reinicio Report.
Línea 149	<variableExpression><![CDATA[\$F{precio_medio}]]></variableExpression> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field precio_medio.
Línea 150	</variable> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 151	<variable name="PrecioMaximo" class="java.lang.Double" calculation="Highest" resetType="Report"> → Declara la variable PrecioMaximo con cálculo Highest y reinicio Report.
Línea 152	<variableExpression><![CDATA[\$F{precio_medio}]]></variableExpression> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field precio_medio.
Línea 153	</variable> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 154	<variable name="NumeroLibros" class="java.lang.Integer" calculation="Count" resetType="Report"> → Declara la variable NumeroLibros con cálculo Count y reinicio Report.

Línea 155	<code><variableExpression><![CDATA[{\$\{titulo}]]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field titulo.
Línea 156	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 157	<code><variable name="ImporteConIva" class="java.lang.Double" calculation="Sum" resetType="Report"></code> → Declara la variable ImporteConIva con cálculo Sum y reinicio Report.
Línea 158	<code><variableExpression><![CDATA[{\$\{importe_total} == null \$\{tipoIva} == null ? null : Double.valueOf(\$\{importe_total}.doubleValue()) * (1.0d + \$\{tipoIva}.doubleValue())]]></v...></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field importe_total, field importe_total, parámetro tipoIva, parámetro tipoIva.
Línea 159	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 160	<code><variable name="GrupoUnidades" class="java.lang.Integer" calculation="Sum" resetType="Group" resetGroup="CategoriaGroup"></code> → Declara la variable GrupoUnidades con cálculo Sum y reinicio Group asociado a CategoriaGroup.
Línea 161	<code><variableExpression><![CDATA[{\$\{unidades_vendidas}]]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field unidades_vendidas.
Línea 162	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 163	<code><variable name="GrupoImporte" class="java.lang.Double" calculation="Sum" resetType="Group" resetGroup="CategoriaGroup"></code> → Declara la variable GrupoImporte con cálculo Sum y reinicio Group asociado a CategoriaGroup.
Línea 164	<code><variableExpression><![CDATA[{\$\{importe_total}]]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field importe_total.
Línea 165	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 166	<code><variable name="GrupoLibros" class="java.lang.Integer" calculation="Count" resetType="Group" resetGroup="CategoriaGroup"></code> → Declara la variable GrupoLibros con cálculo Count y reinicio Group asociado a CategoriaGroup.
Línea 167	<code><variableExpression><![CDATA[{\$\{titulo}]]></variableExpression></code> → Define el valor de entrada que JasperReports evaluará/acumulará para la variable a partir de field titulo.
Línea 168	<code></variable></code> → Cierra variable y vuelve al elemento padre de la jerarquía JRXML.
Línea 169	<code><group name="CategoriaGroup" isStartNewPage="false" isReprintHeaderOnEachPage="true" minHeightToStartNewPage="80"></code> → Declara el grupo CategoriaGroup y sus propiedades de paginación/reimpresión.
Línea 170	<code><groupExpression><![CDATA[{\$\{categoria}]]></groupExpression></code> → Define la clave que decide cuándo cambia el grupo mediante field categoria.
Línea 171	<code><groupHeader></code> → Abre la cabecera del grupo, que se emite cuando comienza cada nuevo valor de agrupación.
Línea 172	<code><band height="28"></code> → Define una banda de 28 puntos, reservando ese espacio para sus elementos.
Línea 173	<code><textField><reportElement x="0" y="2" width="555" height="22" style="M5GrupoCabecera"/><textFieldExpression><![CDATA["Categoría: " + \$\{categoria}]]></textFieldExpression></text...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=2, ancho=555, alto=22 y aplica el estilo M5GrupoCabecera; evalúa la expresión usando field categoria.
Línea 174	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 175	<code></groupHeader></code> → Cierra groupHeader y vuelve al elemento padre de la jerarquía JRXML.
Línea 176	<code><groupFooter></code> → Abre el pie del grupo, donde se muestran los acumulados justo antes de cambiar de grupo.
Línea 177	<code><band height="34"></code> → Define una banda de 34 puntos, reservando ese espacio para sus elementos.

Línea 178	<code><textField><reportElement x="0" y="3" width="185" height="18" style="Dato"/><textFieldExpression><![CDATA["Libros del grupo: " + \${GrupoLibros}]]></textFieldExpression></textField></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=3, ancho=185, alto=18 y aplica el estilo Dato; evalúa la expresión usando variable GrupoLibros.
Línea 179	<code><textField><reportElement x="185" y="3" width="180" height="18" style="Dato"/><textFieldExpression><![CDATA["Unidades: " + (\${GrupoUnidades} == null ? 0 : \${GrupoUnidades})]]></textFieldExpression></textField></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=185, y=3, ancho=180, alto=18 y aplica el estilo Dato; evalúa la expresión usando variable GrupoUnidades, variable GrupoUnidades.
Línea 180	<code><textField><reportElement x="365" y="3" width="190" height="18" style="Dato"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA["Importe: " + new java.text.DecimalFormat("#,###.##").format(\${GrupoImporte})]]></textFieldExpression></textField></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=365, y=3, ancho=190, alto=18 y aplica el estilo Dato; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable GrupoImporte, variable GrupoImporte.
Línea 181	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 182	<code></groupFooter></code> → Cierra groupFooter y vuelve al elemento padre de la jerarquía JRXML.
Línea 183	<code></group></code> → Finaliza la definición del grupo y sus bandas asociadas.
Línea 184	<code><background><band height="0"/></background></code> → Composición de la línea: encadena además <code><background></code> , <code><band></code> dentro de la misma jerarquía.
Línea 185	<code><title></code> → Abre la banda Title, emitida una sola vez al inicio del informe.
Línea 186	<code><band height="124"></code> → Define una banda de 124 puntos, reservando ese espacio para sus elementos.
Línea 187	<code><staticText></code> → Abre un elemento de texto literal; su contenido no depende de fields, parámetros ni variables.
Línea 188	<code><reportElement x="0" y="4" width="555" height="28" uuid="40000000-0000-4000-8000-000000000001" style="M5TituloPrincipal"/></code> → Posiciona el elemento en x=0, y=4, con ancho 555 y alto 28, aplicando el estilo M5TituloPrincipal.
Línea 189	<code><textElement textAlignment="Center" verticalAlignment="Middle"/></code> → Configura el formato interno del texto: alineación horizontal Center, alineación vertical Middle.
Línea 190	<code><text><![CDATA[Informe de Ventas - Agregación por Título]]></text></code> → Define el texto literal visible: Informe de Ventas - Agregación por Título.
Línea 191	<code></staticText></code> → Cierra staticText y vuelve al elemento padre de la jerarquía JRXML.
Línea 192	<code><staticText><reportElement x="0" y="38" width="110" height="18" uuid="40000000-0000-4000-8000-000000000002"/><text><![CDATA[Generado por:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=38, ancho=110, alto=18; muestra el literal 'Generado por:'.
Línea 193	<code><textField isBlankWhenNull="true"><reportElement x="110" y="38" width="160" height="18" uuid="40000000-0000-4000-8000-000000000003"/><textFieldExpression><![CDATA[\${P{usuario}}]]></textFieldExpression></textField></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=110, y=38, ancho=160, alto=18; evalúa la expresión usando parámetro usuario.
Línea 194	<code><staticText><reportElement x="300" y="38" width="80" height="18" uuid="40000000-0000-4000-8000-000000000004"/><text><![CDATA[Fecha:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=38, ancho=80, alto=18; muestra el literal 'Fecha:'.
Línea 195	<code><textField pattern="dd/MM/yyyy"><reportElement x="380" y="38" width="175" height="18" uuid="40000000-0000-4000-8000-000000000005"/><textFieldExpression><![CDATA[\${P{fechaInforme}}]]></textFieldExpression></textField></code> → Composición de la línea: crea un textField dinámico con patrón dd/MM/yyyy; lo posiciona en x=380, y=38, ancho=175, alto=18; evalúa la expresión usando parámetro fechaInforme.
Línea 196	<code><staticText><reportElement x="0" y="62" width="100" height="18" uuid="40000000-0000-4000-8000-000000000006"/><text><![CDATA[Departamento:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=62, ancho=100, alto=18; muestra el literal 'Departamento:'.

Línea 197	<pre><textField isBlankWhenNull="true"><reportElement x="100" y="62" width="170" height="18" uuid="40000000-0000-4000-8000-000000000007"/><textFieldExpression><![CDATA[{\$P{departament... → Composición de la línea: crea un textField dinámico; lo posiciona en x=100, y=62, ancho=170, alto=18; evalúa la expresión usando parámetro departamento.</pre>
Línea 198	<pre><staticText><reportElement x="300" y="62" width="70" height="18" uuid="40000000-0000-4000-8000-000000000008"/><text><![CDATA[Periodo:]]></text></staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=62, ancho=70, alto=18; muestra el literal 'Periodo:'.</pre>
Línea 199	<pre><textField isBlankWhenNull="true"><reportElement x="370" y="62" width="185" height="18" uuid="40000000-0000-4000-8000-000000000009"/><textFieldExpression><![CDATA[{\$P{periodo}]]>... → Composición de la línea: crea un textField dinámico; lo posiciona en x=370, y=62, ancho=185, alto=18; evalúa la expresión usando parámetro periodo.</pre>
Línea 200	<pre><staticText><reportElement x="0" y="86" width="100" height="18" uuid="40000000-0000-4000-8000-000000000010"/><text><![CDATA[Búsqueda:]]></text></staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=86, ancho=100, alto=18; muestra el literal 'Búsqueda:'.</pre>
Línea 201	<pre><textField isBlankWhenNull="true"><reportElement x="100" y="86" width="170" height="18" uuid="40000000-0000-4000-8000-000000000011"/><textFieldExpression><![CDATA[{\$P{textoBusque... → Composición de la línea: crea un textField dinámico; lo posiciona en x=100, y=86, ancho=170, alto=18; evalúa la expresión usando parámetro textoBusqueda, parámetro textoBusqueda, parámetro textoBusqueda.</pre>
Línea 202	<pre><staticText><reportElement x="300" y="86" width="90" height="18" uuid="40000000-0000-4000-8000-000000000012"/><text><![CDATA[Categorías:]]></text></staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=86, ancho=90, alto=18; muestra el literal 'Categorías:'.</pre>
Línea 203	<pre><textField textAdjust="StretchHeight"><reportElement x="390" y="86" width="165" height="34" uuid="40000000-0000-4000-8000-000000000013"/><textFieldExpression><![CDATA[String.val... → Composición de la línea: crea un textField dinámico; lo posiciona en x=390, y=86, ancho=165, alto=34; evalúa la expresión usando parámetro categoriasLista.</pre>
Línea 204	<pre></band></pre> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 205	<pre></title></pre> → Cierra title y vuelve al elemento padre de la jerarquía JRXML.
Línea 206	<pre><columnHeader></pre> → Abre Column Header, repetida al comienzo de cada columna/página según la paginación.
Línea 207	<pre><band height="62"></pre> → Define una banda de 62 puntos, reservando ese espacio para sus elementos.
Línea 208	<pre><staticText><reportElement x="0" y="2" width="215" height="18" uuid="41000000-0000-4000-8000-000000000001" style="Cabecera"/><text><![CDATA[Título]]></text></staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=2, ancho=215, alto=18 y aplica el estilo Cabecera; muestra el literal 'Título'.</pre>
Línea 209	<pre><staticText><reportElement x="215" y="2" width="55" height="18" uuid="41000000-0000-4000-8000-000000000002" style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[... → Composición de la línea: crea un texto literal; lo posiciona en x=215, y=2, ancho=55, alto=18 y aplica el estilo Cabecera; configura la alineación del texto (horizontal Right); muestra el literal 'Unid:'.</pre>
Línea 210	<pre><staticText><reportElement x="280" y="2" width="90" height="18" uuid="41000000-0000-4000-8000-000000000003" style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[... → Composición de la línea: crea un texto literal; lo posiciona en x=280, y=2, ancho=90, alto=18 y aplica el estilo Cabecera; configura la alineación del texto (horizontal Right); muestra el literal 'Importe'.</pre>
Línea 211	<pre><staticText><reportElement x="380" y="2" width="65" height="18" uuid="41000000-0000-4000-8000-000000000004" style="Cabecera"/><textElement textAlignment="Right"/><text><![CDATA[... → Composición de la línea: crea un texto literal; lo posiciona en x=380, y=2, ancho=65, alto=18 y aplica el estilo Cabecera; configura la alineación del texto (horizontal Right); muestra el literal 'Precio med:'.</pre>

Línea 212	<code><staticText><reportElement x="455" y="2" width="100" height="18" uuid="41000000-0000-4000-8000-000000000005" style="Cabecera"/><text><![CDATA[Categoría]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=455, y=2, ancho=100, alto=18 y aplica el estilo Cabecera; muestra el literal 'Categoría'.
Línea 213	<code><staticText><reportElement x="0" y="24" width="130" height="18" uuid="41000000-0000-4000-8000-000000000006" style="Cabecera"/><text><![CDATA[Primera venta]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=24, ancho=130, alto=18 y aplica el estilo Cabecera; muestra el literal 'Primera venta'.
Línea 214	<code><staticText><reportElement x="130" y="24" width="130" height="18" uuid="41000000-0000-4000-8000-000000000007" style="Cabecera"/><text><![CDATA[Última venta]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=130, y=24, ancho=130, alto=18 y aplica el estilo Cabecera; muestra el literal 'Última venta'.
Línea 215	<code><staticText><reportElement x="260" y="24" width="160" height="18" uuid="41000000-0000-4000-8000-000000000008" style="Cabecera"/><text><![CDATA[Periodo de ventas]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=260, y=24, ancho=160, alto=18 y aplica el estilo Cabecera; muestra el literal 'Periodo de ventas'.
Línea 216	<code><staticText></code> → Abre un elemento de texto literal; su contenido no depende de fields, parámetros ni variables.
Línea 217	<code><reportElement x="420" y="24" width="135" height="18" uuid="41000000-0000-4000-8000-000000000009" style="Cabecera"></code> → Posiciona el elemento en x=420, y=24, con ancho 135 y alto 18, aplicando el estilo Cabecera.
Línea 218	<code><printWhenExpression><![CDATA[Boolean.TRUE.equals(\$P{mostrarDetalle})]]></printWhenExpression></code> → Evalúa una condición booleana para decidir si la banda o elemento se imprime usando parámetro mostrarDetalle.
Línea 219	<code></reportElement></code> → Cierra reportElement y vuelve al elemento padre de la jerarquía JRXML.
Línea 220	<code><textElement textAlignment="Right"/></code> → Configura el formato interno del texto: alineación horizontal Right.
Línea 221	<code><text><![CDATA[Importe con IVA]]></text></code> → Define el texto literal visible: Importe con IVA.
Línea 222	<code></staticText></code> → Cierra staticText y vuelve al elemento padre de la jerarquía JRXML.
Línea 223	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 224	<code></columnHeader></code> → Cierra columnHeader y vuelve al elemento padre de la jerarquía JRXML.
Línea 225	<code><detail></code> → Abre Detail, la sección que se repite para cada registro del dataset principal.
Línea 226	<code><band height="82" splitType="Stretch"></code> → Define una banda de 82 puntos con splitType Stretch, reservando ese espacio para sus elementos.
Línea 227	<code><textField textAdjust="StretchHeight"><reportElement x="0" y="0" width="215" height="20" uuid="42000000-0000-4000-8000-000000000001" style="Dato"/><textFieldExpression><![CDATA[...]]></textFieldExpression></reportElement></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=0, ancho=215, alto=20 y aplica el estilo Dato; evalúa la expresión usando field titulo.
Línea 228	<code><textField isBlankWhenNull="true"><reportElement x="215" y="0" width="55" height="20" uuid="42000000-0000-4000-8000-000000000002" style="UnidadesCondicional"/><textElement textA...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=215, y=0, ancho=55, alto=20 y aplica el estilo UnidadesCondicional; configura la alineación del texto (horizontal Right); evalúa la expresión usando field unidades_vendidas.
Línea 229	<code><textField pattern="#.##0.00 €" isBlankWhenNull="true"><reportElement x="280" y="0" width="90" height="20" uuid="42000000-0000-4000-8000-000000000003" style="Dato"/><textElement...></code> → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=280, y=0, ancho=90, alto=20 y aplica el estilo Dato; configura la alineación del texto (horizontal Right); evalúa la expresión usando field importe_total.

Línea 230	<code><textField isBlankWhenNull="true"><reportElement x="380" y="0" width="65" height="20" uuid="42000000-0000-4000-8000-000000000004" style="Dato"/><textElement textAlignment="Right"...</code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=380, y=0, ancho=65, alto=20 y aplica el estilo Dato; configura la alineación del texto (horizontal Right); evalúa la expresión usando field precio_medio, field precio_medio.
Línea 231	<code><textField isBlankWhenNull="true"><reportElement x="455" y="0" width="100" height="20" uuid="42000000-0000-4000-8000-000000000005" style="Dato"/><textFieldExpression><![CDATA[\$F...]</code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=455, y=0, ancho=100, alto=20 y aplica el estilo Dato; evalúa la expresión usando field categoria.
Línea 232	<code><textField isBlankWhenNull="true"><reportElement x="0" y="24" width="130" height="18" uuid="42000000-0000-4000-8000-000000000006" style="Dato"/><textFieldExpression><![CDATA[\$F{...}]</code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=24, ancho=130, alto=18 y aplica el estilo Dato; evalúa la expresión usando field primera_venta.
Línea 233	<code><textField isBlankWhenNull="true"><reportElement x="130" y="24" width="130" height="18" uuid="42000000-0000-4000-8000-000000000007" style="Dato"/><textFieldExpression><![CDATA[\$...]</code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=130, y=24, ancho=130, alto=18 y aplica el estilo Dato; evalúa la expresión usando field ultima_venta.
Línea 234	<code><textField><reportElement x="260" y="24" width="160" height="18" uuid="42000000-0000-4000-8000-000000000008" style="Dato"/><textElement textAlignment="Center"/><textFieldExpress...]</code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=260, y=24, ancho=160, alto=18 y aplica el estilo Dato; configura la alineación del texto (horizontal Center); evalúa la expresión usando field primera_venta, field primera_venta, field ultima_venta.
Línea 235	<code><textField pattern="#.##0.00 €" isBlankWhenNull="true"></code> → Abre un textField dinámico con formato #,##0.00 €, cuyo valor se obtiene de su textFieldExpression.
Línea 236	<code><reportElement x="420" y="24" width="135" height="18" uuid="42000000-0000-4000-8000-000000000009" style="Dato"></code> → Posiciona el elemento en x=420, y=24, con ancho 135 y alto 18, aplicando el estilo Dato.
Línea 237	<code><printWhenExpression><![CDATA[Boolean.TRUE.equals(\$P{mostrarDetalle})]]></printWhenExpression></code> → Evalúa una condición booleana para decidir si la banda o elemento se imprime usando parámetro mostrarDetalle.
Línea 238	<code></reportElement></code> → Cierra reportElement y vuelve al elemento padre de la jerarquía JRXML.
Línea 239	<code><textElement textAlignment="Right"/></code> → Configura el formato interno del texto: alineación horizontal Right.
Línea 240	<code><textFieldExpression><![CDATA[\$F{importe_total} == null \$P{tipoIva} == null ? null : Double.valueOf(\$F{importe_total}.doubleValue() * (1.0d + \$P{tipoIva}.doubleValue()))]]></...]</code> → Calcula el valor mostrado por el textField mediante una expresión Java que usa field importe_total, field importe_total, parámetro tipoIva, parámetro tipoIva.
Línea 241	<code></textField></code> → Cierra textField y vuelve al elemento padre de la jerarquía JRXML.
Línea 242	<code><textField><reportElement x="0" y="48" width="105" height="18" uuid="42000000-0000-4000-8000-000000000010" style="Dato"/><textFieldExpression><![CDATA[\$F{unidades_vendidas} == n...]</code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=48, ancho=105, alto=18 y aplica el estilo Dato; evalúa la expresión usando field unidades_vendidas, field unidades_vendidas, field unidades_vendidas.
Línea 243	<code><textField><reportElement x="105" y="48" width="185" height="18" uuid="42000000-0000-4000-8000-000000000011" style="Dato"/><textFieldExpression><![CDATA[\$F{titulo} == null ? "" ...]</code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=105, y=48, ancho=185, alto=18 y aplica el estilo Dato; evalúa la expresión usando field titulo, field titulo.
Línea 244	<code><textField><reportElement x="290" y="48" width="80" height="18" uuid="42000000-0000-4000-8000-000000000012" style="Dato"/><textElement textAlignment="Right"/><textFieldExpressio...]</code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=290, y=48, ancho=80, alto=18 y aplica el estilo Dato; configura la alineación del texto (horizontal Right); evalúa la expresión usando field precio_medio, field precio_medio.

Línea 245	<code><textField><reportElement x="370" y="48" width="90" height="18" uuid="42000000-0000-4000-8000-000000000013" style="Dato"/><textElement textAlignment="Center"/><textFieldExpressi... → Composición de la línea: crea un textField dinámico; lo posiciona en x=370, y=48, ancho=90, alto=18 y aplica el estilo Dato; configura la alineación del texto (horizontal Center); evalúa la expresión usando field primera_venta, field ultima_venta, field primera_venta, field ultima_venta.</code>
Línea 246	<code><textField><reportElement x="460" y="48" width="95" height="18" uuid="42000000-0000-4000-8000-000000000014" style="Dato"/><textElement textAlignment="Right"/><textFieldExpressio... → Composición de la línea: crea un textField dinámico; lo posiciona en x=460, y=48, ancho=95, alto=18 y aplica el estilo Dato; configura la alineación del texto (horizontal Right); evalúa la expresión usando field unidades_vendidas, field unidades_vendidas, parámetro umbralUnidades, parámetro umbralUnidades.</code>
Línea 247	<code></band> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.</code>
Línea 248	<code><band height="14"> → Define una banda de 14 puntos, reservando ese espacio para sus elementos.</code>
Línea 249	<code><printWhenExpression><![CDATA[\$F{unidades_vendidas} != null && \$P{umbralUnidades} != null && \$F{unidades_vendidas}.intValue() >= \$P{umbralUnidades}.intValue()]]></printWhenExpre... → Evalúa una condición booleana para decidir si la banda o elemento se imprime usando field unidades_vendidas, field unidades_vendidas, parámetro umbralUnidades, parámetro umbralUnidades.</code>
Línea 250	<code><textField><reportElement x="0" y="0" width="555" height="12" uuid="42000000-0000-4000-8000-000000000015"/><textElement textAlignment="Center"><font fontName="DejaVu Sans" size=... → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=0, ancho=555, alto=12; configura la alineación del texto (horizontal Center); configura la fuente (familia DejaVu Sans, tamaño 8, negrita); evalúa la expresión usando field titulo, parámetro umbralUnidades.</code>
Línea 251	<code></band> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.</code>
Línea 252	<code><band height="88" splitType="Stretch"> → Define una banda de 88 puntos con splitType Stretch, reservando ese espacio para sus elementos.</code>
Línea 253	<code><printWhenExpression><![CDATA[\$F{unidades_vendidas} != null]]></printWhenExpression> → Evalúa una condición booleana para decidir si la banda o elemento se imprime usando field unidades_vendidas.</code>
Línea 254	<code><staticText> → Abre un elemento de texto literal; su contenido no depende de fields, parámetros ni variables.</code>
Línea 255	<code><reportElement x="0" y="2" width="555" height="16" style="Cabecera"/> → Posiciona el elemento en x=0, y=2, con ancho 555 y alto 16, aplicando el estilo Cabecera.</code>
Línea 256	<code><text><![CDATA[Detalle de ventas]]></text> → Define el texto literal visible: Detalle de ventas.</code>
Línea 257	<code></staticText> → Cierra staticText y vuelve al elemento padre de la jerarquía JRXML.</code>
Línea 258	<code><subreport> → Abre el componente subreport que ejecuta un informe hijo dentro de la banda del maestro.</code>
Línea 259	<code><reportElement x="0" y="22" width="555" height="60" isRemoveLineWhenBlank="true"/> → Posiciona el elemento en x=0, y=22, con ancho 555 y alto 60, y elimina su línea cuando queda vacío.</code>
Línea 260	<code><subreportParameter name="tituloLibro"> → Declara el parámetro del subreporte tituloLibro que recibirá un valor del informe maestro.</code>
Línea 261	<code><subreportParameterExpression><![CDATA[\$F{titulo}]]></subreportParameterExpression> → Calcula el valor enviado al parámetro del subreporte a partir de field titulo.</code>
Línea 262	<code></subreportParameter> → Cierra subreportParameter y vuelve al elemento padre de la jerarquía JRXML.</code>
Línea 263	<code><connectionExpression><![CDATA[\$P{REPORT_CONNECTION}]]></connectionExpression> → Entrega al subreporte/subdataset la misma REPORT_CONNECTION usada por el informe maestro, evitando abrir otra conexión.</code>
Línea 264	<code><subreportExpression><![CDATA["reports/subinforme_ventas_detalle.jasper"]]></subreportExpression> → Devuelve la ruta del archivo subinforme_ventas_detalle.jasper que JasperReports cargará como informe hijo.</code>
Línea 265	<code></subreport> → Finaliza el componente de subreporte.</code>

Línea 266	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 267	<code><band height="104" splitType="Stretch"></code> → Define una banda de 104 puntos con splitType Stretch, reservando ese espacio para sus elementos.
Línea 268	<code><printWhenExpression><![CDATA[{\$F{unidades_vendidas} != null}]></printWhenExpression></code> → Evalúa una condición booleana para decidir si la banda o elemento se imprime usando field unidades_vendidas.
Línea 269	<code><staticText></code> → Abre un elemento de texto literal; su contenido no depende de fields, parámetros ni variables.
Línea 270	<code><reportElement x="0" y="2" width="555" height="16" style="Cabecera"/></code> → Posiciona el elemento en x=0, y=2, con ancho 555 y alto 16, aplicando el estilo Cabecera.
Línea 271	<code><text><![CDATA[Top 3 ventas por cantidad]]></text></code> → Define el texto literal visible: Top 3 ventas por cantidad.
Línea 272	<code></staticText></code> → Cierra staticText y vuelve al elemento padre de la jerarquía JRXML.
Línea 273	<code><componentElement></code> → Abre un contenedor de componentes extendidos; en este checkpoint contiene la tabla c:table.
Línea 274	<code><reportElement x="0" y="22" width="555" height="76"/></code> → Posiciona el elemento en x=0, y=22, con ancho 555 y alto 76.
Línea 275	<code><c:table xmlns:c="http://jasperreports.sourceforge.net/jasperreports/components"></code> → Abre la tabla del namespace de componentes JasperReports; sus columnas usan un datasetRun independiente.
Línea 276	<code>xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports/components http://jasperreports.sourceforge.net/xsd/components.xsd"></code> → Relaciona el namespace de JasperReports con su XSD para que Studio y el compilador validen la estructura.
Línea 277	<code><datasetRun subDataset="DatasetTopVentas"></code> → Asocia el componente con el subdataset DatasetTopVentas para ejecutar su consulta.
Línea 278	<code><datasetParameter name="tituloLibro"></code> → Declara el parámetro tituloLibro que se enviará al subdataset de la tabla.
Línea 279	<code><datasetParameterExpression><![CDATA[{\$F{titulo}}]></datasetParameterExpression></code> → Calcula el valor enviado al parámetro del subdataset desde field titulo.
Línea 280	<code></datasetParameter></code> → Cierra datasetParameter y vuelve al elemento padre de la jerarquía JRXML.
Línea 281	<code><connectionExpression><![CDATA[{\$P{REPORT_CONNECTION}}]></connectionExpression></code> → Entrega al subreporte/subdataset la misma REPORT_CONNECTION usada por el informe maestro, evitando abrir otra conexión.
Línea 282	<code></datasetRun></code> → Cierra datasetRun y vuelve al elemento padre de la jerarquía JRXML.
Línea 283	<code><c:column width="255"></code> → Declara una columna de tabla de 255 puntos de ancho.
Línea 284	<code><c:columnHeader style="M5TablaCabecera" height="20"><staticText><reportElement x="0" y="0" width="255" height="20"/><text><![CDATA[Fecha]]></text></staticText></c:columnHeader></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=0, ancho=255, alto=20 y aplica el estilo M5TablaCabecera; muestra el literal 'Fecha'; define una celda de cabecera de la tabla.
Línea 285	<code><c:detailCell style="M5TablaDetalle" height="18"><textField><reportElement x="0" y="0" width="255" height="18"/><textFieldExpression><![CDATA[{\$F{fecha_venta}}]></textFieldExpres...</code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=0, ancho=255, alto=18 y aplica el estilo M5TablaDetalle; evalúa la expresión usando field fecha_venta; define una celda repetida de detalle de la tabla.
Línea 286	<code></c:column></code> → Cierra c:column y vuelve al elemento padre de la jerarquía JRXML.
Línea 287	<code><c:column width="100"></code> → Declara una columna de tabla de 100 puntos de ancho.

Línea 288	<code><c:columnHeader style="M5TablaCabecera" height="20"><staticText><reportElement x="0" y="0" width="100" height="20"/><textElement textAlignment="Right"/><text><![CDATA[Cantidad]]... → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=0, ancho=100, alto=20 y aplica el estilo M5TablaCabecera; configura la alineación del texto (horizontal Right); muestra el literal 'Cantidad'; define una celda de cabecera de la tabla.</code>
Línea 289	<code><c:detailCell style="M5TablaDetalle" height="18"><textField><reportElement x="0" y="0" width="100" height="18"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[... → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=0, ancho=100, alto=18 y aplica el estilo M5TablaDetalle; configura la alineación del texto (horizontal Right); evalúa la expresión usando field cantidad; define una celda repetida de detalle de la tabla.</code>
Línea 290	<code></c:column> → Cierra c:column y vuelve al elemento padre de la jerarquía JRXML.</code>
Línea 291	<code><c:column width="200"> → Declara una columna de tabla de 200 puntos de ancho.</code>
Línea 292	<code><c:columnHeader style="M5TablaCabecera" height="20"><staticText><reportElement x="0" y="0" width="200" height="20"/><textElement textAlignment="Right"/><text><![CDATA[Precio uni... → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=0, ancho=200, alto=20 y aplica el estilo M5TablaCabecera; configura la alineación del texto (horizontal Right); muestra el literal 'Precio unitario'; define una celda de cabecera de la tabla.</code>
Línea 293	<code><c:detailCell style="M5TablaDetalle" height="18"><textField pattern="#,##0.00 €"><reportElement x="0" y="0" width="200" height="18"/><textElement textAlignment="Right"/><textFie... → Composición de la línea: crea un textField dinámico con patrón #,##0.00 €; lo posiciona en x=0, y=0, ancho=200, alto=18 y aplica el estilo M5TablaDetalle; configura la alineación del texto (horizontal Right); evalúa la expresión usando field precio_unitario; define una celda repetida de detalle de la tabla.</code>
Línea 294	<code></c:column> → Cierra c:column y vuelve al elemento padre de la jerarquía JRXML.</code>
Línea 295	<code></c:table> → Finaliza la tabla integrada.</code>
Línea 296	<code></componentElement> → Cierra componentElement y vuelve al elemento padre de la jerarquía JRXML.</code>
Línea 297	<code></band> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.</code>
Línea 298	<code></detail> → Finaliza la sección Detail del informe.</code>
Línea 299	<code><pageFooter> → Abre Page Footer, emitido al pie de cada página.</code>
Línea 300	<code><band height="62"> → Define una banda de 62 puntos, reservando ese espacio para sus elementos.</code>
Línea 301	<code><staticText><reportElement x="0" y="4" width="120" height="15" uuid="43000000-0000-4000-8000-000000000001"/><text><![CDATA[Total de títulos:]]></text></staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=4, ancho=120, alto=15; muestra el literal 'Total de títulos:'.</code>
Línea 302	<code><textField evaluationTime="Report"><reportElement x="120" y="4" width="60" height="15" uuid="43000000-0000-4000-8000-000000000002"/><textFieldExpression><![CDATA[\$V{REPORT_COUNT... → Composición de la línea: crea un textField dinámico; lo posiciona en x=120, y=4, ancho=60, alto=15; evalúa la expresión usando variable REPORT_COUNT.</code>
Línea 303	<code><textField><reportElement x="190" y="28" width="180" height="15" uuid="43000000-0000-4000-8000-000000000003"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA["... → Composición de la línea: crea un textField dinámico; lo posiciona en x=190, y=28, ancho=180, alto=15; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable PAGE_NUMBER.</code>
Línea 304	<code><textField evaluationTime="Report"><reportElement x="375" y="28" width="35" height="15" uuid="43000000-0000-4000-8000-000000000004"/><textFieldExpression><![CDATA[\$V{PAGE_NUMBER... → Composición de la línea: crea un textField dinámico; lo posiciona en x=375, y=28, ancho=35, alto=15; evalúa la expresión usando variable PAGE_NUMBER.</code>
Línea 305	<code><staticText><reportElement x="300" y="4" width="120" height="15" uuid="43000000-0000-4000-8000-000000000005"/><text><![CDATA[Subtotal página:]]></text></staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=4, ancho=120, alto=15; muestra el literal 'Subtotal página:'.</code>

Línea 306	<code><textField pattern="#.##0.00 €"><reportElement x="420" y="4" width="135" height="15" uuid="43000000-0000-4000-8000-000000000006"/><textElement textAlignment="Right" /><textFieldE... → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=420, y=4, ancho=135, alto=15; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable TotalPagina.</code>
Línea 307	<code></band> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.</code>
Línea 308	<code></pageFooter> → Cierra pageFooter y vuelve al elemento padre de la jerarquía JRXML.</code>
Línea 309	<code><summary> → Abre Summary, emitido una sola vez después del último registro.</code>
Línea 310	<code><band height="700"> → Define una banda de 700 puntos, reservando ese espacio para sus elementos.</code>
Línea 311	<code><staticText><reportElement x="0" y="5" width="205" height="18" uuid="44000000-0000-4000-8000-000000000001"/><text><![CDATA[Total de unidades vendidas:]]></text></staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=5, ancho=205, alto=18; muestra el literal 'Total de unidades vendidas:'.</code>
Línea 312	<code><textField><reportElement x="205" y="5" width="80" height="18" uuid="44000000-0000-4000-8000-000000000002"/><textElement textAlignment="Right" /><textFieldExpression><![CDATA[\$V{... → Composición de la línea: crea un textField dinámico; lo posiciona en x=205, y=5, ancho=80, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable TotalUnidades.</code>
Línea 313	<code><staticText><reportElement x="300" y="5" width="120" height="18" uuid="44000000-0000-4000-8000-000000000003"/><text><![CDATA[Importe total:]]></text></staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=5, ancho=120, alto=18; muestra el literal 'Importe total:'.</code>
Línea 314	<code><textField pattern="#.##0.00 €"><reportElement x="420" y="5" width="135" height="18" uuid="44000000-0000-4000-8000-000000000004"/><textElement textAlignment="Right" /><textFieldE... → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=420, y=5, ancho=135, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable TotalImporte.</code>
Línea 315	<code><staticText><reportElement x="0" y="30" width="205" height="18" uuid="44000000-0000-4000-8000-000000000005"/><text><![CDATA[Precio medio agregado:]]></text></staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=30, ancho=205, alto=18; muestra el literal 'Precio medio agregado:'.</code>
Línea 316	<code><textField pattern="#.##0.00 €"><reportElement x="205" y="30" width="80" height="18" uuid="44000000-0000-4000-8000-000000000006"/><textElement textAlignment="Right" /><textFieldE... → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=205, y=30, ancho=80, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable PrecioMedio.</code>
Línea 317	<code><staticText><reportElement x="300" y="30" width="120" height="18" uuid="44000000-0000-4000-8000-000000000007"/><text><![CDATA[Precio máximo:]]></text></staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=30, ancho=120, alto=18; muestra el literal 'Precio máximo:'.</code>
Línea 318	<code><textField pattern="#.##0.00 €"><reportElement x="420" y="30" width="135" height="18" uuid="44000000-0000-4000-8000-000000000008"/><textElement textAlignment="Right" /><textFieldE... → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=420, y=30, ancho=135, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable PrecioMaximo.</code>
Línea 319	<code><staticText><reportElement x="0" y="55" width="205" height="18" uuid="44000000-0000-4000-8000-000000000009"/><text><![CDATA[Número de libros:]]></text></staticText> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=55, ancho=205, alto=18; muestra el literal 'Número de libros:'.</code>
Línea 320	<code><textField><reportElement x="205" y="55" width="80" height="18" uuid="44000000-0000-4000-8000-000000000010"/><textElement textAlignment="Right" /><textFieldExpression><![CDATA[\$V{... → Composición de la línea: crea un textField dinámico; lo posiciona en x=205, y=55, ancho=80, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable NumeroLibros.</code>

Línea 321	<code><staticText><reportElement x="300" y="55" width="120" height="18" uuid="44000000-0000-4000-8000-000000000011"/><text><![CDATA[Importe con IVA:]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=300, y=55, ancho=120, alto=18; muestra el literal 'Importe con IVA:'.
Línea 322	<code><textField pattern="#.##0.00 €"><reportElement x="420" y="55" width="135" height="18" uuid="44000000-0000-4000-8000-000000000012"/><textElement textAlignment="Right" /><textField...></code> → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=420, y=55, ancho=135, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable ImporteConIva.
Línea 323	<code><textField><reportElement x="0" y="80" width="555" height="18" uuid="44000000-0000-4000-8000-000000000013"/><textElement textAlignment="Center" /><textFieldExpression><![CDATA[St...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=80, ancho=555, alto=18; configura la alineación del texto (horizontal Center); evalúa la expresión usando variable NumeroLibros, variable TotalUnidades, variable TotalImporte.
Línea 324	<code><textField><reportElement x="0" y="103" width="350" height="18" uuid="44000000-0000-4000-8000-000000000014"/><textElement textAlignment="Center"></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=103, ancho=350, alto=18; configura la alineación del texto (horizontal Center); configura la fuente (familia DejaVu Sans, tamaño 10, negrita); evalúa la expresión usando parámetro umbralUnidades, parámetro umbralUnidades, variable TotalUnidades, variable TotalUnidades.
Línea 325	<code><textField><reportElement x="360" y="103" width="195" height="18" uuid="44000000-0000-4000-8000-000000000015"/><textFieldExpression><![CDATA["Resultados encontrados: " + \$V{REPO...></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=360, y=103, ancho=195, alto=18; evalúa la expresión usando variable REPORT_COUNT.
Línea 326	<code><staticText><reportElement x="0" y="140" width="555" height="20" style="Cabecera"/><text><![CDATA[Ventas por categoría — importe]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=140, ancho=555, alto=20 y aplica el estilo Cabecera; muestra el literal 'Ventas por categoría — importe'.
Línea 327	<code><barChart></code> → Abre el gráfico de barras nativo de JasperReports que se integrará en el informe maestro.
Línea 328	<code><chart></code> → Abre la configuración común del gráfico: geometría, título, subtítulo y leyenda.
Línea 329	<code><reportElement x="0" y="165" width="555" height="250"/></code> → Posiciona el elemento en x=0, y=165, con ancho 555 y alto 250.
Línea 330	<code><chartTitle><titleExpression><![CDATA["Ventas por categoría"]]></titleExpression></chartTitle></code> → Composición de la línea: encadena además <code><chartTitle></code> , <code><titleExpression></code> dentro de la misma jerarquía.
Línea 331	<code><chartSubtitle/></code> → Declara el subtítulo del gráfico; en este checkpoint queda vacío.
Línea 332	<code><chartLegend position="Bottom"/></code> → Configura la leyenda del gráfico en la posición Bottom.
Línea 333	<code></chart></code> → Cierra chart y vuelve al elemento padre de la jerarquía JRXML.
Línea 334	<code><categoryDataset></code> → Abre el dataset categórico que alimenta al gráfico con serie, categoría y valor.
Línea 335	<code><dataset></code> → Abre el contenedor de ejecución de datos del componente actual.
Línea 336	<code><datasetRun subDataset="DatasetVentasPorCategoría"></code> → Asocia el componente con el subdataset DatasetVentasPorCategoría para ejecutar su consulta.
Línea 337	<code><connectionExpression><![CDATA[\$P{REPORT_CONNECTION}]]></connectionExpression></code> → Entrega al subreporte/subdataset la misma REPORT_CONNECTION usada por el informe maestro, evitando abrir otra conexión.
Línea 338	<code></datasetRun></code> → Cierra datasetRun y vuelve al elemento padre de la jerarquía JRXML.
Línea 339	<code></dataset></code> → Cierra dataset y vuelve al elemento padre de la jerarquía JRXML.
Línea 340	<code><categorySeries></code> → Abre una serie del dataset categórico; cada fila del subdataset aportará categoría y valor.

Línea 341	<code><seriesExpression><![CDATA["Importe"]]></seriesExpression></code> → Define el nombre lógico de la serie que aparecerá en la leyenda.
Línea 342	<code><categoryExpression><![CDATA[\${F{categoria_grafico}}]></categoryExpression></code> → Define la categoría del eje X a partir de field categoria_grafico.
Línea 343	<code><valueExpression><![CDATA[\${F{importe_categoria}}]></valueExpression></code> → Define el valor numérico representado por cada barra a partir de field importe_categoria.
Línea 344	<code></categorySeries></code> → Cierra categorySeries y vuelve al elemento padre de la jerarquía JRXML.
Línea 345	<code></categoryDataset></code> → Cierra categoryDataset y vuelve al elemento padre de la jerarquía JRXML.
Línea 346	<code><barPlot></code> → Abre el plot específico del gráfico de barras, donde se configuran etiquetas y ejes.
Línea 347	<code><plot/></code> → Declara el bloque base del plot; mantiene la configuración visual por defecto del checkpoint.
Línea 348	<code><itemLabel/></code> → Habilita el bloque de configuración de etiquetas de los ítems/barras.
Línea 349	<code><categoryAxisFormat><axisFormat/></categoryAxisFormat></code> → Composición de la línea: encadena además <code><categoryAxisFormat></code> , <code><axisFormat></code> dentro de la misma jerarquía.
Línea 350	<code><valueAxisFormat><axisFormat/></valueAxisFormat></code> → Composición de la línea: encadena además <code><valueAxisFormat></code> , <code><axisFormat></code> dentro de la misma jerarquía.
Línea 351	<code></barPlot></code> → Cierra barPlot y vuelve al elemento padre de la jerarquía JRXML.
Línea 352	<code></barChart></code> → Finaliza el gráfico de barras.
Línea 353	<code><staticText><reportElement x="0" y="430" width="555" height="20" style="Cabecera"/><text><![CDATA[Ventas por categoría y año]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=430, ancho=555, alto=20 y aplica el estilo Cabecera; muestra el literal 'Ventas por categoría y año'.
Línea 354	<code><crosstab></code> → Abre la tabla cruzada nativa que genera dinámicamente la matriz de filas, columnas, medidas y totales.
Línea 355	<code><reportElement x="0" y="455" width="555" height="225"/></code> → Posiciona el elemento en x=0, y=455, con ancho 555 y alto 225.
Línea 356	<code><crosstabDataset></code> → Abre la fuente de datos específica del crosstab.
Línea 357	<code><dataset></code> → Abre el contenedor de ejecución de datos del componente actual.
Línea 358	<code><datasetRun subDataset="DatasetCrosstabVentas"></code> → Asocia el componente con el subdataset DatasetCrosstabVentas para ejecutar su consulta.
Línea 359	<code><connectionExpression><![CDATA[\${REPORT_CONNECTION}]]></connectionExpression></code> → Entrega al subreporte/subdataset la misma REPORT_CONNECTION usada por el informe maestro, evitando abrir otra conexión.
Línea 360	<code></datasetRun></code> → Cierra datasetRun y vuelve al elemento padre de la jerarquía JRXML.
Línea 361	<code></dataset></code> → Cierra dataset y vuelve al elemento padre de la jerarquía JRXML.
Línea 362	<code></crosstabDataset></code> → Cierra crosstabDataset y vuelve al elemento padre de la jerarquía JRXML.
Línea 363	<code><rowGroup name="CategoriaCross" width="150" totalPosition="End"></code> → Declara el grupo de filas CategoriaCross, ancho 150 y total en End.
Línea 364	<code><bucket class="java.lang.String"><bucketExpression><![CDATA[\${F{categoria_cross}}]></bucketExpression></bucket></code> → Composición de la línea: define la clave de agrupación del bucket desde field categoria_cross; encadena además <code><bucket></code> dentro de la misma jerarquía.

Línea 365	<code><crosstabRowHeader><cellContents style="M5CrosstabCabecera"><textField><reportElement x="0" y="0" width="150" height="34"/><textElement verticalAlignment="Middle"/><textFieldExp... → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=0, ancho=150, alto=34 y aplica el estilo M5CrosstabCabecera; configura la alineación del texto (vertical Middle); evalúa la expresión usando variable CategoriaCross; abre el contenido de celda con estilo M5CrosstabCabecera; define la cabecera de fila del crosstab.</code>
Línea 366	<code><crosstabTotalRowHeader><cellContents style="M5CrosstabTotal"><staticText><reportElement x="0" y="0" width="150" height="34"/><textElement verticalAlignment="Middle"/><text><![C... → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=0, ancho=150, alto=34 y aplica el estilo M5CrosstabTotal; configura la alineación del texto (vertical Middle); muestra el literal 'TOTAL'; abre el contenido de celda con estilo M5CrosstabTotal; define la cabecera de total de fila.</code>
Línea 367	<code></rowGroup> → Cierra rowGroup y vuelve al elemento padre de la jerarquía JRXML.</code>
Línea 368	<code><columnGroup name="AnioCross" height="28" totalPosition="End"> → Declara el grupo de columnas AnioCross, altura 28 y total en End.</code>
Línea 369	<code><bucket class="java.lang.String"><bucketExpression><![CDATA[\$F{anio_cross}]]></bucketExpression></bucket> → Composición de la línea: define la clave de agrupación del bucket desde field anio_cross; encadena además <bucket> dentro de la misma jerarquía.</code>
Línea 370	<code><crosstabColumnHeader><cellContents style="M5CrosstabCabecera"><textField><reportElement x="0" y="0" width="100" height="28"/><textElement textAlignment="Center" verticalAlignme... → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=0, ancho=100, alto=28 y aplica el estilo M5CrosstabCabecera; configura la alineación del texto (horizontal Center, vertical Middle); evalúa la expresión usando variable AnioCross; abre el contenido de celda con estilo M5CrosstabCabecera; define la cabecera de columna del crosstab.</code>
Línea 371	<code><crosstabTotalColumnHeader><cellContents style="M5CrosstabTotal"><staticText><reportElement x="0" y="0" width="100" height="28"/><textElement textAlignment="Center" verticalAlig... → Composición de la línea: crea un texto literal; lo posiciona en x=0, y=0, ancho=100, alto=28 y aplica el estilo M5CrosstabTotal; configura la alineación del texto (horizontal Center, vertical Middle); muestra el literal 'TOTAL'; abre el contenido de celda con estilo M5CrosstabTotal; define la cabecera de total de columna.</code>
Línea 372	<code></columnGroup> → Cierra columnGroup y vuelve al elemento padre de la jerarquía JRXML.</code>
Línea 373	<code><measure name="ImporteCross" class="java.lang.Double" calculation="Sum"><measureExpression><![CDATA[\$F{importe_cross}]]></measureExpression></measure> → Composición de la línea: aporta a la medida el valor de field importe_cross; encadena además <measure> dentro de la misma jerarquía.</code>
Línea 374	<code><measure name="VentasCross" class="java.lang.Integer" calculation="Sum"><measureExpression><![CDATA[\$F{ventas_cross}]]></measureExpression></measure> → Composición de la línea: aporta a la medida el valor de field ventas_cross; encadena además <measure> dentro de la misma jerarquía.</code>
Línea 375	<code><crosstabCell width="100" height="34"><cellContents style="M5CrosstabDetalle"><textField pattern="#.##0.00 €"><reportElement x="0" y="0" width="100" height="18"/><textElement te... → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=0, y=0, ancho=100, alto=34 y aplica el estilo M5CrosstabDetalle; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable ImporteCross, variable VentasCross; abre el contenido de celda con estilo M5CrosstabDetalle; define la celda de detalle del crosstab.</code>
Línea 376	<code><crosstabCell width="100" height="34" rowTotalGroup="CategoriaCross"><cellContents style="M5CrosstabTotal"><textField pattern="#.##0.00 €"><reportElement x="0" y="0" width="100" h... → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=0, y=0, ancho=100, alto=34 y aplica el estilo M5CrosstabTotal; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable ImporteCross, variable VentasCross; abre el contenido de celda con estilo M5CrosstabTotal; define una celda de total de fila para CategoriaCross.</code>
Línea 377	<code><crosstabCell width="100" height="34" columnTotalGroup="AnioCross"><cellContents style="M5CrosstabTotal"><textField pattern="#.##0.00 €"><reportElement x="0" y="0" width="100" h... → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=0, y=0, ancho=100, alto=34 y aplica el estilo M5CrosstabTotal; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable ImporteCross, variable VentasCross; abre el contenido de celda con estilo M5CrosstabTotal; define una celda de total de columna para AnioCross.</code>

Línea 378	<code><crosstabCell width="100" height="34" rowTotalGroup="CategoriaCross" columnTotalGroup="AnioCross"><cellContents style="M5CrosstabTotal"><textField pattern="#,#0.00 €"><reportEl... → Composición de la línea: crea un textField dinámico con patrón #,#0.00 €; lo posiciona en x=0, y=0, ancho=100, alto=34 y aplica el estilo M5CrosstabTotal; configura la alineación del texto (horizontal Right); evalúa la expresión usando variable ImporteCross, variable VentasCross; abre el contenido de celda con estilo M5CrosstabTotal; define la celda de total general.</code>
Línea 379	<code></crosstab> → Finaliza la tabla cruzada.</code>
Línea 380	<code></band> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.</code>
Línea 381	<code></summary> → Finaliza la sección Summary.</code>
Línea 382	<code></jasperReport> → Finaliza la definición completa del informe JasperReports.</code>

Subinforme heredado y ejecutable

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/
jasperreport.xsd"
  name="subinforme_ventas_detalle"
  language="java"
  pageWidth="555"
  pageHeight="842"
  columnWidth="555"
  leftMargin="0"
  rightMargin="0"
  topMargin="0"
  bottomMargin="0">
  <property name="com.jaspersoft.studio.data.defaultdataadapter" value="SQLiteEditorial"/>
  <style name="SubBase" isDefault="true" fontName="DejaVu Sans" fontSize="8"/>
  <style name="SubHeader" style="SubBase" mode="Opaque" backcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/>
  <parameter name="tituloLibro" class="java.lang.String"/>
  <queryString language="sql">
    <![CDATA[
      SELECT fecha_venta, cantidad, precio_unitario
      FROM ventas
      WHERE titulo_libro = ${tituloLibro}
      ORDER BY fecha_venta
    ]]>
  </queryString>
  <field name="fecha_venta" class="java.lang.String"/>
  <field name="cantidad" class="java.lang.Integer"/>
  <field name="precio_unitario" class="java.lang.Double"/>
  <columnHeader>
    <band height="18">
      <staticText><reportElement x="0" y="0" width="245" height="18" style="SubHeader"/><text><![CDATA[Fecha]]></text></
staticText>
      <staticText><reportElement x="245" y="0" width="100" height="18" style="SubHeader"/><textElement textAlignment="Right"/
><text><![CDATA[Cantidad]]></text></staticText>
      <staticText><reportElement x="345" y="0" width="210" height="18" style="SubHeader"/><textElement textAlignment="Right"/
><text><![CDATA[Precio unitario]]></text></staticText>
    </band>
  </columnHeader>
  <detail>
    <band height="18">
      <textField><reportElement x="0" y="0" width="245" height="18"/><textFieldExpression><![CDATA[${fecha_venta}]]></
textFieldExpression></textField>
      <textField><reportElement x="245" y="0" width="100" height="18"/><textElement textAlignment="Right"/><textFieldExpression><![
CDATA[${cantidad}]]></textFieldExpression></textField>
      <textField pattern="#,#0.00 €"><reportElement x="345" y="0" width="210" height="18"/><textElement textAlignment="Right"/
><textFieldExpression><![CDATA[${precio_unitario}]]></textFieldExpression></textField>
    </band>
  </detail>
</jasperReport>
```

Explicación línea por línea

Línea 1	<code><?xml version="1.0" encoding="UTF-8"?> → Declara XML 1.0 y codificación UTF-8 para que nombres, textos y símbolos del informe se interpreten correctamente.</code>
Línea 2	<code><jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports" → Abre el documento raíz jasperReport del informe y fija el namespace principal de JasperReports.</code>
Línea 3	<code>xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" → Declara el namespace XML Schema Instance usado por xsi:schemaLocation para validar el documento.</code>

Línea 4	<code>xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/jasperreport.xsd"</code> → Relaciona el namespace de JasperReports con su XSD para que Studio y el compilador validen la estructura.
Línea 5	<code>name="subinforme_ventas_detalle"</code> → Asigna al documento JasperReports el nombre interno <code>subinforme_ventas_detalle</code> .
Línea 6	<code>language="java"</code> → Configura <code>language=java</code> para evaluar expresiones con el lenguaje Java.
Línea 7	<code>pageWidth="555"</code> → Fija el ancho físico de página en 555 puntos.
Línea 8	<code>pageHeight="842"</code> → Fija la altura física de página en 842 puntos.
Línea 9	<code>columnWidth="555"</code> → Fija el ancho útil de la columna de contenido en 555 puntos.
Línea 10	<code>leftMargin="0"</code> → Fija el margen izquierdo del informe en 0 puntos.
Línea 11	<code>rightMargin="0"</code> → Fija el margen derecho del informe en 0 puntos.
Línea 12	<code>topMargin="0"</code> → Fija el margen superior del informe en 0 puntos.
Línea 13	<code>bottomMargin="0"></code> → Fija el margen inferior del informe en 0 puntos y completa la apertura del elemento raíz.
Línea 14	<code><property name="com.jaspersoft.studio.data.defaultdataadapter" value="SQLiteEditorial"/></code> → Indica a Jaspersoft Studio que use el Data Adapter <code>SQLiteEditorial</code> como conexión de diseño por defecto.
Línea 15	<code><style name="SubBase" isDefault="true" fontFamily="DejaVu Sans" fontSize="8"/></code> → Declara el estilo <code>SubBase</code> ; es el estilo por defecto, fuente <code>DejaVu Sans</code> , tamaño 8.
Línea 16	<code><style name="SubHeader" style="SubBase" mode="Opaque" backcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/></code> → Declara el estilo <code>SubHeader</code> ; hereda de <code>SubBase</code> , fondo <code>#EAF2F8</code> .
Línea 17	<code><parameter name="tituloLibro" class="java.lang.String"/></code> → Declara el parámetro <code>tituloLibro</code> con tipo <code>java.lang.String</code> y lo expone al diálogo de parámetros de Studio.
Línea 18	<code><queryString language="sql"></code> → Abre la consulta SQL que JasperReports ejecutará para el dataset actual.
Línea 19	<code><![CDATA[</code> → Abre <code>CDATA</code> para escribir SQL o una expresión Java sin que sus caracteres especiales se interpreten como XML.
Línea 20	<code>SELECT fecha_venta, cantidad, precio_unitario</code> → Cláusula SQL <code>SELECT</code> : selecciona y calcula las columnas que devolverá la consulta.
Línea 21	<code>FROM ventas</code> → Cláusula SQL <code>FROM</code> : define la tabla base de la consulta.
Línea 22	<code>WHERE titulo_libro = \${tituloLibro}</code> → Cláusula SQL <code>WHERE</code> : aplica el filtro de filas.
Línea 23	<code>ORDER BY fecha_venta</code> → Cláusula SQL <code>ORDER BY</code> : ordena el resultado que recibirá JasperReports.
Línea 24	<code>]]></code> → Cierra el bloque <code>CDATA</code> y devuelve el control al parser XML.
Línea 25	<code></queryString></code> → Cierra la consulta SQL del dataset actual.
Línea 26	<code><field name="fecha_venta" class="java.lang.String"/></code> → Declara el field <code>fecha_venta</code> con tipo Java <code>java.lang.String</code> para mapear una columna del dataset.
Línea 27	<code><field name="cantidad" class="java.lang.Integer"/></code> → Declara el field <code>cantidad</code> con tipo Java <code>java.lang.Integer</code> para mapear una columna del dataset.
Línea 28	<code><field name="precio_unitario" class="java.lang.Double"/></code> → Declara el field <code>precio_unitario</code> con tipo Java <code>java.lang.Double</code> para mapear una columna del dataset.
Línea 29	<code><columnHeader></code> → Abre <code>Column Header</code> , repetida al comienzo de cada columna/página según la paginación.
Línea 30	<code><band height="18"></code> → Define una banda de 18 puntos, reservando ese espacio para sus elementos.
Línea 31	<code><staticText><reportElement x="0" y="0" width="245" height="18" style="SubHeader"/><text><![CDATA[Fecha]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en <code>x=0</code> , <code>y=0</code> , <code>ancho=245</code> , <code>alto=18</code> y aplica el estilo <code>SubHeader</code> ; muestra el literal 'Fecha'.

Línea 32	<code><staticText><reportElement x="245" y="0" width="100" height="18" style="SubHeader" /><textElement textAlignment="Right" /><text><![CDATA[Cantidad]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=245, y=0, ancho=100, alto=18 y aplica el estilo SubHeader; configura la alineación del texto (horizontal Right); muestra el literal 'Cantidad'.
Línea 33	<code><staticText><reportElement x="345" y="0" width="210" height="18" style="SubHeader" /><textElement textAlignment="Right" /><text><![CDATA[Precio unitario]]></text></staticText></code> → Composición de la línea: crea un texto literal; lo posiciona en x=345, y=0, ancho=210, alto=18 y aplica el estilo SubHeader; configura la alineación del texto (horizontal Right); muestra el literal 'Precio unitario'.
Línea 34	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 35	<code></columnHeader></code> → Cierra columnHeader y vuelve al elemento padre de la jerarquía JRXML.
Línea 36	<code><detail></code> → Abre Detail, la sección que se repite para cada registro del dataset principal.
Línea 37	<code><band height="18"></code> → Define una banda de 18 puntos, reservando ese espacio para sus elementos.
Línea 38	<code><textField><reportElement x="0" y="0" width="245" height="18" /><textFieldExpression><![CDATA[\${fecha_venta}]]></textFieldExpression></textField></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=0, y=0, ancho=245, alto=18; evalúa la expresión usando field fecha_venta.
Línea 39	<code><textField><reportElement x="245" y="0" width="100" height="18" /><textElement textAlignment="Right" /><textFieldExpression><![CDATA[\${cantidad}]]></textFieldExpression></textField></code> → Composición de la línea: crea un textField dinámico; lo posiciona en x=245, y=0, ancho=100, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando field cantidad.
Línea 40	<code><textField pattern="#.##0.00 €"><reportElement x="345" y="0" width="210" height="18" /><textElement textAlignment="Right" /><textFieldExpression><![CDATA[\${precio_unitario}]]></textFieldExpression></textField></code> → Composición de la línea: crea un textField dinámico con patrón #.##0.00 €; lo posiciona en x=345, y=0, ancho=210, alto=18; configura la alineación del texto (horizontal Right); evalúa la expresión usando field precio_unitario.
Línea 41	<code></band></code> → Cierra band y vuelve al elemento padre de la jerarquía JRXML.
Línea 42	<code></detail></code> → Finaliza la sección Detail del informe.
Línea 43	<code></jasperReport></code> → Finaliza la definición completa del informe JasperReports.

Plantilla JRTX heredada y ejecutable

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<jasperTemplate xmlns="http://jasperreports.sourceforge.net/jasperreports/template"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports/template http://jasperreports.sourceforge.net/
xsd/jaspertemplate.xsd">
  <style name="M5TituloPrincipal" fontName="DejaVu Sans" fontSize="18" isBold="true" forecolor="#173F6B"/>
  <style name="M5GrupoCabecera" fontName="DejaVu Sans" fontSize="10" isBold="true" forecolor="#173F6B" mode="Opaque"
backcolor="#D6EAF8"/>
  <style name="M5TablaCabecera" fontName="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque"
backcolor="#EAF2F8"/>
  <style name="M5TablaDetalle" fontName="DejaVu Sans" fontSize="9"/>
  <style name="M5CrosstabCabecera" fontName="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque"
backcolor="#EAF2F8"/>
  <style name="M5CrosstabDetalle" fontName="DejaVu Sans" fontSize="9" mode="Opaque" backcolor="#FFFFFF"/>
  <style name="M5CrosstabTotal" fontName="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque"
backcolor="#D6EAF8"/>
</jasperTemplate>
```

Explicación línea por línea

Línea 1	<code><?xml version="1.0" encoding="UTF-8"?></code> → Declara XML 1.0 y codificación UTF-8 para que nombres, textos y símbolos del informe se interpreten correctamente.
Línea 2	<code><jasperTemplate xmlns="http://jasperreports.sourceforge.net/jasperreports/template"></code> → Abre el documento raíz jasperTemplate de la plantilla JRTX que contiene estilos reutilizables.
Línea 3	<code>xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"</code> → Declara el namespace XML Schema Instance usado por xsi:schemaLocation para validar el documento.

Línea 4	<code>xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports/template http://jasperreports.sourceforge.net/xsd/jasptemplate.xsd"></code> → Relaciona el namespace de JasperReports con su XSD para que Studio y el compilador validen la estructura.
Línea 5	<code><style name="M5TituloPrincipal" fontFamily="DejaVu Sans" fontSize="18" isBold="true" forecolor="#173F6B" /></code> → Declara el estilo M5TituloPrincipal; fuente DejaVu Sans, tamaño 18.
Línea 6	<code><style name="M5GrupoCabecera" fontFamily="DejaVu Sans" fontSize="10" isBold="true" forecolor="#173F6B" mode="Opaque" backcolor="#D6EAF8" /></code> → Declara el estilo M5GrupoCabecera; fuente DejaVu Sans, tamaño 10, fondo #D6EAF8.
Línea 7	<code><style name="M5TablaCabecera" fontFamily="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque" backcolor="#EAF2F8" /></code> → Declara el estilo M5TablaCabecera; fuente DejaVu Sans, tamaño 9, fondo #EAF2F8.
Línea 8	<code><style name="M5TablaDetalle" fontFamily="DejaVu Sans" fontSize="9" /></code> → Declara el estilo M5TablaDetalle; fuente DejaVu Sans, tamaño 9.
Línea 9	<code><style name="M5CrosstabCabecera" fontFamily="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque" backcolor="#EAF2F8" /></code> → Declara el estilo M5CrosstabCabecera; fuente DejaVu Sans, tamaño 9, fondo #EAF2F8.
Línea 10	<code><style name="M5CrosstabDetalle" fontFamily="DejaVu Sans" fontSize="9" mode="Opaque" backcolor="#FFFFFF" /></code> → Declara el estilo M5CrosstabDetalle; fuente DejaVu Sans, tamaño 9, fondo #FFFFFF.
Línea 11	<code><style name="M5CrosstabTotal" fontFamily="DejaVu Sans" fontSize="9" isBold="true" forecolor="#173F6B" mode="Opaque" backcolor="#D6EAF8" /></code> → Declara el estilo M5CrosstabTotal; fuente DejaVu Sans, tamaño 9, fondo #D6EAF8.
Línea 12	<code></jasperTemplate></code> → Finaliza la plantilla externa JRTX.

Parte C — Código y configuración ejecutable explicados línea por línea

GeneradorInformeVentas.java

JAVA

```
import java.io.File;
import java.sql.Connection;
import java.sql.DriverManager;
import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;
import net.sf.jasperreports.engine.JasperCompileManager;
import net.sf.jasperreports.engine.JasperFillManager;
import net.sf.jasperreports.engine.JasperPrint;
import net.sf.jasperreports.engine.export.JRPdfExporter;
import net.sf.jasperreports.export.SimpleExporterInput;
import net.sf.jasperreports.export.SimpleOutputStreamExporterOutput;
import net.sf.jasperreports.export.SimplePdfExporterConfiguration;
import net.sf.jasperreports.engine.data.JRCsvDataSource;
import net.sf.jasperreports.engine.export.ooxml.JRXlsxExporter;
import net.sf.jasperreports.export.SimpleXlsxReportConfiguration;
import net.sf.jasperreports.export.SimpleXlsxExporterConfiguration;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.nio.file.StandardCopyOption;
import net.sf.jasperreports.engine.export.HtmlExporter;
import net.sf.jasperreports.engine.export.FileHtmlResourceHandler;
import net.sf.jasperreports.export.SimpleHtmlExporterConfiguration;
import net.sf.jasperreports.export.SimpleHtmlExporterOutput;
import net.sf.jasperreports.engine.export.JRCsvExporter;
import net.sf.jasperreports.engine.export.JRXmlExporter;
import net.sf.jasperreports.engine.export.JRRtfExporter;
import net.sf.jasperreports.engine.export.oasis.JROdtExporter;
import net.sf.jasperreports.export.SimpleCsvExporterConfiguration;
import net.sf.jasperreports.export.SimpleWriterExporterOutput;
import net.sf.jasperreports.export.SimpleXmlExporterOutput;
import net.sf.jasperreports.export.SimpleRtfExporterConfiguration;

public class GeneradorInformeVentas {
    public static void main(String[] args) {
        try {
            String rutaJrxml = "reports/informe_ventas.jrxml";
            String rutaJasper = "reports/informe_ventas.jasper";
            String rutaPdf = "output/informe_ventas.pdf";
            String rutaPdfProtegido = "output/informe_ventas_protegido.pdf";
            String rutaXlsx = "output/informe_ventas.xlsx";
```

```

String rutaCatalogoJrxml = "reports/informe_catalogo_csv.jrxml";
String rutaCatalogoJasper = "reports/informe_catalogo_csv.jasper";
String rutaXlsxCatalogo = "output/informe_catalogo.xlsx";
String rutaHtml = "output/informe_ventas.html";
String rutaCsv = "output/informe_ventas.csv";
String rutaXml = "output/informe_ventas.xml";
String rutaRtf = "output/informe_ventas.rtf";
String rutaOdt = "output/informe_ventas.odt";
String urlBD = "jdbc:sqlite:../EditorialReportsJava/data/editorial.db";
new File("output").mkdirs();

String rutaSubJrxml = "reports/subinforme_ventas_detalle.jrxml";
String rutaSubJasper = "reports/subinforme_ventas_detalle.jasper";
JasperCompileManager.compileReportToFile(rutaSubJrxml, rutaSubJasper);
JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);
JasperCompileManager.compileReportToFile(rutaCatalogoJrxml, rutaCatalogoJasper);

Map<String, Object> parametros = new HashMap<String, Object>();
parametros.put("usuario", "Ana Martínez");
parametros.put("departamento", "Comercial");
parametros.put("periodo", "Septiembre 2026");
parametros.put("tipoIva", Double.valueOf(0.21d));
parametros.put("mostrarDetalle", Boolean.TRUE);
parametros.put("categoria", null);
parametros.put("precioMinimo", null);
parametros.put("precioMaximo", null);
parametros.put("umbralUnidades", Integer.valueOf(5));
parametros.put("textoBusqueda", null);
parametros.put("categoriasLista", Arrays.asList("Novela", "Realismo mágico", "Cuento", "Poesía"));

try (Connection conexion = DriverManager.getConnection(urlBD)) {
    JasperPrint documento = JasperFillManager.fillReport(rutaJasper, parametros, conexion);
    exportarPdf(documento, rutaPdf, ConfiguracionExportacion.getConfiguracionPdf(
        "Informe de Ventas - EditorialReports", "Departamento Comercial"));
    exportarPdfProtegido(documento, rutaPdfProtegido);
    exportarXlsx(documento, rutaXlsx, "Ventas");
    JRCSVDataSource catalogoDataSource = new JRCSVDataSource(new File("data/catalogo.csv"), "UTF-8");
    try {
        catalogoDataSource.setFieldDelimiter(',');
        catalogoDataSource.setUseFirstRowAsHeader(true);
        JasperPrint documentoCatalogo = JasperFillManager.fillReport(
            rutaCatalogoJasper, new HashMap<String, Object>(), catalogoDataSource);
        exportarXlsx(documentoCatalogo, rutaXlsxCatalogo, "Catálogo");
    } finally {
        catalogoDataSource.close();
    }
    new File("output/images").mkdirs();
    new File("output/styles").mkdirs();
    Files.copy(Paths.get("resources/styles/editorial.css"), Paths.get("output/styles/editorial.css"),
        StandardCopyOption.REPLACE_EXISTING);
    exportarHtml(documento, rutaHtml);
    exportarCsv(documento, rutaCsv);
    exportarXml(documento, rutaXml);
    exportarRtf(documento, rutaRtf);
    exportarOdt(documento, rutaOdt);
    System.out.println("Informe PDF generado en: " + new File(rutaPdf).getAbsolutePath());
    System.out.println("Informe PDF protegido generado en: " + new File(rutaPdfProtegido).getAbsolutePath());
    System.out.println("Informe Excel generado en: " + new File(rutaXlsx).getAbsolutePath());
    System.out.println("Informe catálogo Excel generado en: " + new File(rutaXlsxCatalogo).getAbsolutePath());
    System.out.println("Informe HTML generado en: " + new File(rutaHtml).getAbsolutePath());
    System.out.println("Informe CSV generado en: " + new File(rutaCsv).getAbsolutePath());
    System.out.println("Informe XML generado en: " + new File(rutaXml).getAbsolutePath());
    System.out.println("Informe RTF generado en: " + new File(rutaRtf).getAbsolutePath());
    System.out.println("Informe ODT generado en: " + new File(rutaOdt).getAbsolutePath());
    System.out.println("Paginas del documento: " + documento.getPages().size());
    System.out.println("Parametro usuario: " + parametros.get("usuario"));
    System.out.println("M6 checkpoint generado correctamente");
}
} catch (Exception e) {
    e.printStackTrace();
    System.exit(1);
}

private static void exportarPdf(JasperPrint documento, String ruta, SimplePdfExporterConfiguration configuracion) throws Exception {
    JRPDFExporter exportador = new JRPDFExporter();
    exportador.setConfiguration(configuracion);
    exportador.setExporterInput(new SimpleExporterInput(documento));
    exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta));
    exportador.exportReport();
}

private static void exportarPdfProtegido(JasperPrint documento, String ruta) throws Exception {
    JRPDFExporter exportador = new JRPDFExporter();
    SimplePdfExporterConfiguration configuracion = new SimplePdfExporterConfiguration();
    configuracion.setMetadataTitle("Informe de Ventas Protegido - EditorialReports");
    configuracion.setEncrypted(Boolean.TRUE);
    configuracion.setUserPassword("editorial2026");
    configuracion.setOwnerPassword("editorial-admin");
    configuracion.setAllowedPermissionsHint("PRINTING|COPY|SCREENREADERS");
    exportador.setConfiguration(configuracion);

```

```

        exportador.setExporterInput(new SimpleExporterInput(documento));
        exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta));
        exportador.exportReport();
    }

    private static void exportarXlsx(JasperPrint documento, String ruta, String nombreHoja) throws Exception {
        JRXlsxExporter exportador = new JRXlsxExporter();
        exportador.setConfiguration(ConfiguracionExportacion.getConfiguracionXlsxReport(nombreHoja));
        exportador.setConfiguration(ConfiguracionExportacion.getConfiguracionXlsxExportador());
        exportador.setExporterInput(new SimpleExporterInput(documento));
        exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta));
        exportador.exportReport();
    }

    private static void exportarHtml(JasperPrint documento, String ruta) throws Exception {
        HtmlExporter exportador = new HtmlExporter();
        SimpleHtmlExporterOutput salida = new SimpleHtmlExporterOutput(ruta, "UTF-8");
        salida.setImageHandler(new FileHtmlResourceHandler(new File("output/images"), "images/{0}"));
        exportador.setConfiguration(ConfiguracionExportacion.getConfiguracionHtml("Informe de Ventas - EditorialReports"));
        exportador.setExporterInput(new SimpleExporterInput(documento));
        exportador.setExporterOutput(salida);
        exportador.exportReport();
    }

    private static void exportarCsv(JasperPrint documento, String ruta) throws Exception {
        JRCsvExporter exportador = new JRCsvExporter();
        exportador.setConfiguration(ConfiguracionExportacion.getConfiguracionCsv());
        exportador.setExporterInput(new SimpleExporterInput(documento));
        exportador.setExporterOutput(new SimpleWriterExporterOutput(ruta, "UTF-8"));
        exportador.exportReport();
    }

    private static void exportarXml(JasperPrint documento, String ruta) throws Exception {
        JRXmlExporter exportador = new JRXmlExporter();
        SimpleXmlExporterOutput salida = new SimpleXmlExporterOutput(ruta, "UTF-8");
        salida.setEmbeddingImages(Boolean.TRUE);
        exportador.setExporterInput(new SimpleExporterInput(documento));
        exportador.setExporterOutput(salida);
        exportador.exportReport();
    }

    private static void exportarRtf(JasperPrint documento, String ruta) throws Exception {
        JRRtfExporter exportador = new JRRtfExporter();
        exportador.setConfiguration(ConfiguracionExportacion.getConfiguracionRtf());
        exportador.setExporterInput(new SimpleExporterInput(documento));
        exportador.setExporterOutput(new SimpleWriterExporterOutput(ruta, "UTF-8"));
        exportador.exportReport();
    }

    private static void exportarOdt(JasperPrint documento, String ruta) throws Exception {
        JROdtExporter exportador = new JROdtExporter();
        exportador.setExporterInput(new SimpleExporterInput(documento));
        exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta));
        exportador.exportReport();
    }
}

```

Explicación línea por línea

Línea 1	import java.io.File; → Importa java.io.File para gestionar rutas y crear la carpeta de salida.
Línea 2	import java.sql.Connection; → Importa java.sql.Connection para representar la conexión JDBC abierta contra SQLite.
Línea 3	import java.sql.DriverManager; → Importa java.sql.DriverManager para abrir la conexión JDBC a partir de la URL SQLite.
Línea 4	import java.util.Arrays; → Importa java.util.Arrays para construir la colección de categorías usada por el parámetro de lista.
Línea 5	import java.util.HashMap; → Importa java.util.HashMap para crear la implementación mutable del mapa de parámetros.
Línea 6	import java.util.Map; → Importa java.util.Map para tipar el mapa de parámetros que recibe JasperReports.
Línea 7	import net.sf.jasperreports.engine.JasperCompileManager; → Importa net.sf.jasperreports.engine.JasperCompileManager para compilar los JRXML a artefactos .jasper.

Línea 8	<code>import net.sf.jasperreports.engine.JasperFillManager;</code> → Importa <code>net.sf.jasperreports.engine.JasperFillManager</code> para llenar el informe compilado con parámetros y conexión.
Línea 9	<code>import net.sf.jasperreports.engine.JasperPrint;</code> → Importa <code>net.sf.jasperreports.engine.JasperPrint</code> para representar en memoria el documento ya paginado por JasperReports.
Línea 10	<code>import net.sf.jasperreports.engine.export.JRPdfExporter;</code> → Importa <code>net.sf.jasperreports.engine.export.JRPdfExporter</code> para usar la clase <code>JRPdfExporter</code> en el generador.
Línea 11	<code>import net.sf.jasperreports.export.SimpleExporterInput;</code> → Importa <code>net.sf.jasperreports.export.SimpleExporterInput</code> para usar la clase <code>SimpleExporterInput</code> en el generador.
Línea 12	<code>import net.sf.jasperreports.export.SimpleOutputStreamExporterOutput;</code> → Importa <code>net.sf.jasperreports.export.SimpleOutputStreamExporterOutput</code> para usar la clase <code>SimpleOutputStreamExporterOutput</code> en el generador.
Línea 13	<code>import net.sf.jasperreports.export.SimplePdfExporterConfiguration;</code> → Importa <code>net.sf.jasperreports.export.SimplePdfExporterConfiguration</code> para usar la clase <code>SimplePdfExporterConfiguration</code> en el generador.
Línea 14	<code>import net.sf.jasperreports.engine.data.JRCsvDataSource;</code> → Importa <code>net.sf.jasperreports.engine.data.JRCsvDataSource</code> para usar la clase <code>JRCsvDataSource</code> en el generador.
Línea 15	<code>import net.sf.jasperreports.engine.export.ooxml.JRXlsxExporter;</code> → Importa <code>net.sf.jasperreports.engine.export.ooxml.JRXlsxExporter</code> para usar la clase <code>JRXlsxExporter</code> en el generador.
Línea 16	<code>import net.sf.jasperreports.export.SimpleXlsxReportConfiguration;</code> → Importa <code>net.sf.jasperreports.export.SimpleXlsxReportConfiguration</code> para usar la clase <code>SimpleXlsxReportConfiguration</code> en el generador.
Línea 17	<code>import net.sf.jasperreports.export.SimpleXlsxExporterConfiguration;</code> → Importa <code>net.sf.jasperreports.export.SimpleXlsxExporterConfiguration</code> para usar la clase <code>SimpleXlsxExporterConfiguration</code> en el generador.
Línea 18	<code>import java.nio.file.Files;</code> → Importa <code>java.nio.file.Files</code> para usar la clase <code>Files</code> en el generador.
Línea 19	<code>import java.nio.file.Paths;</code> → Importa <code>java.nio.file.Paths</code> para usar la clase <code>Paths</code> en el generador.
Línea 20	<code>import java.nio.file.StandardCopyOption;</code> → Importa <code>java.nio.file.StandardCopyOption</code> para usar la clase <code>StandardCopyOption</code> en el generador.
Línea 21	<code>import net.sf.jasperreports.engine.export.HtmlExporter;</code> → Importa <code>net.sf.jasperreports.engine.export.HtmlExporter</code> para usar la clase <code>HtmlExporter</code> en el generador.
Línea 22	<code>import net.sf.jasperreports.engine.export.FileHtmlResourceHandler;</code> → Importa <code>net.sf.jasperreports.engine.export.FileHtmlResourceHandler</code> para usar la clase <code>FileHtmlResourceHandler</code> en el generador.
Línea 23	<code>import net.sf.jasperreports.export.SimpleHtmlExporterConfiguration;</code> → Importa <code>net.sf.jasperreports.export.SimpleHtmlExporterConfiguration</code> para usar la clase <code>SimpleHtmlExporterConfiguration</code> en el generador.
Línea 24	<code>import net.sf.jasperreports.export.SimpleHtmlExporterOutput;</code> → Importa <code>net.sf.jasperreports.export.SimpleHtmlExporterOutput</code> para usar la clase <code>SimpleHtmlExporterOutput</code> en el generador.
Línea 25	<code>import net.sf.jasperreports.engine.export.JRCsvExporter;</code> → Importa <code>net.sf.jasperreports.engine.export.JRCsvExporter</code> para usar la clase <code>JRCsvExporter</code> en el generador.
Línea 26	<code>import net.sf.jasperreports.engine.export.JRXmlExporter;</code> → Importa <code>net.sf.jasperreports.engine.export.JRXmlExporter</code> para usar la clase <code>JRXmlExporter</code> en el generador.
Línea 27	<code>import net.sf.jasperreports.engine.export.JRRtfExporter;</code> → Importa <code>net.sf.jasperreports.engine.export.JRRtfExporter</code> para usar la clase <code>JRRtfExporter</code> en el generador.

Línea 28	<code>import net.sf.jasperreports.engine.export.oasis.JROdtExporter;</code> → Importa <code>net.sf.jasperreports.engine.export.oasis.JROdtExporter</code> para usar la clase <code>JROdtExporter</code> en el generador.
Línea 29	<code>import net.sf.jasperreports.export.SimpleCsvExporterConfiguration;</code> → Importa <code>net.sf.jasperreports.export.SimpleCsvExporterConfiguration</code> para usar la clase <code>SimpleCsvExporterConfiguration</code> en el generador.
Línea 30	<code>import net.sf.jasperreports.export.SimpleWriterExporterOutput;</code> → Importa <code>net.sf.jasperreports.export.SimpleWriterExporterOutput</code> para usar la clase <code>SimpleWriterExporterOutput</code> en el generador.
Línea 31	<code>import net.sf.jasperreports.export.SimpleXmlExporterOutput;</code> → Importa <code>net.sf.jasperreports.export.SimpleXmlExporterOutput</code> para usar la clase <code>SimpleXmlExporterOutput</code> en el generador.
Línea 32	<code>import net.sf.jasperreports.export.SimpleRtfExporterConfiguration;</code> → Importa <code>net.sf.jasperreports.export.SimpleRtfExporterConfiguration</code> para usar la clase <code>SimpleRtfExporterConfiguration</code> en el generador.
Línea 33	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 34	<code>public class GeneradorInformeVentas {</code> → Declara la clase ejecutable <code>GeneradorInformeVentas</code> que encapsula el generador del informe.
Línea 35	<code>public static void main(String[] args) {</code> → Declara <code>main</code> como punto de entrada de la aplicación Java; recibe los argumentos de línea de comandos aunque este ejemplo no los utiliza.
Línea 36	<code>try {</code> → Abre el bloque principal protegido: cualquier error de compilación, conexión, llenado o exportación será capturado por el <code>catch</code> final.
Línea 37	<code>String rutaJrxml = "reports/informe_ventas.jrxml";</code> → Declara <code>rutaJrxml</code> con ruta del JRXML maestro que se compilará; el valor configurado es <code>"reports/informe_ventas.jrxml"</code> .
Línea 38	<code>String rutaJasper = "reports/informe_ventas.jasper";</code> → Declara <code>rutaJasper</code> con ruta del <code>.jasper</code> maestro que producirá la compilación; el valor configurado es <code>"reports/informe_ventas.jasper"</code> .
Línea 39	<code>String rutaPdf = "output/informe_ventas.pdf";</code> → Declara <code>rutaPdf</code> con ruta del PDF final exportado; el valor configurado es <code>"output/informe_ventas.pdf"</code> .
Línea 40	<code>String rutaPdfProtegido = "output/informe_ventas_protegido.pdf";</code> → Declara <code>rutaPdfProtegido</code> con valor de configuración usado por el generador; el valor configurado es <code>"output/informe_ventas_protegido.pdf"</code> .
Línea 41	<code>String rutaXlsx = "output/informe_ventas.xlsx";</code> → Declara <code>rutaXlsx</code> con valor de configuración usado por el generador; el valor configurado es <code>"output/informe_ventas.xlsx"</code> .
Línea 42	<code>String rutaCatalogoJrxml = "reports/informe_catalogo_csv.jrxml";</code> → Declara <code>rutaCatalogoJrxml</code> con valor de configuración usado por el generador; el valor configurado es <code>"reports/informe_catalogo_csv.jrxml"</code> .
Línea 43	<code>String rutaCatalogoJasper = "reports/informe_catalogo_csv.jasper";</code> → Declara <code>rutaCatalogoJasper</code> con valor de configuración usado por el generador; el valor configurado es <code>"reports/informe_catalogo_csv.jasper"</code> .
Línea 44	<code>String rutaXlsxCatalogo = "output/informe_catalogo.xlsx";</code> → Declara <code>rutaXlsxCatalogo</code> con valor de configuración usado por el generador; el valor configurado es <code>"output/informe_catalogo.xlsx"</code> .
Línea 45	<code>String rutaHtml = "output/informe_ventas.html";</code> → Declara <code>rutaHtml</code> con valor de configuración usado por el generador; el valor configurado es <code>"output/informe_ventas.html"</code> .
Línea 46	<code>String rutaCsv = "output/informe_ventas.csv";</code> → Declara <code>rutaCsv</code> con valor de configuración usado por el generador; el valor configurado es <code>"output/informe_ventas.csv"</code> .
Línea 47	<code>String rutaXml = "output/informe_ventas.xml";</code> → Declara <code>rutaXml</code> con valor de configuración usado por el generador; el valor configurado es <code>"output/informe_ventas.xml"</code> .
Línea 48	<code>String rutaRtf = "output/informe_ventas.rtf";</code> → Declara <code>rutaRtf</code> con valor de configuración usado por el generador; el valor configurado es <code>"output/informe_ventas.rtf"</code> .

Línea 49	<code>String rutaOdt = "output/informe_ventas.odt";</code> → Declara <code>rutaOdt</code> con valor de configuración usado por el generador; el valor configurado es <code>"output/informe_ventas.odt"</code> .
Línea 50	<code>String urlBD = "jdbc:sqlite:../EditorialReportsJava/data/editorial.db";</code> → Declara <code>urlBD</code> con URL JDBC de la base SQLite; el valor configurado es <code>"jdbc:sqlite:../EditorialReportsJava/data/editorial.db"</code> .
Línea 51	<code>new File("output").mkdirs();</code> → Crea la carpeta <code>output</code> si todavía no existe para evitar que la exportación falle por una ruta inexistente.
Línea 52	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 53	<code>String rutaSubJrxml = "reports/subinforme_ventas_detalle.jrxml";</code> → Declara <code>rutaSubJrxml</code> con ruta del JRXML del subinforme de detalle; el valor configurado es <code>"reports/subinforme_ventas_detalle.jrxml"</code> .
Línea 54	<code>String rutaSubJasper = "reports/subinforme_ventas_detalle.jasper";</code> → Declara <code>rutaSubJasper</code> con ruta del <code>.jasper</code> del subinforme compilado; el valor configurado es <code>"reports/subinforme_ventas_detalle.jasper"</code> .
Línea 55	<code>JasperCompileManager.compileReportToFile(rutaSubJrxml, rutaSubJasper);</code> → Compila el JRXML indicado en <code>rutaSubJrxml</code> y escribe el artefacto compilado en <code>rutaSubJasper</code> .
Línea 56	<code>JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);</code> → Compila el JRXML indicado en <code>rutaJrxml</code> y escribe el artefacto compilado en <code>rutaJasper</code> .
Línea 57	<code>JasperCompileManager.compileReportToFile(rutaCatalogoJrxml, rutaCatalogoJasper);</code> → Compila el JRXML indicado en <code>rutaCatalogoJrxml</code> y escribe el artefacto compilado en <code>rutaCatalogoJasper</code> .
Línea 58	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 59	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 60	<code>Map<String, Object> parametros = new HashMap<String, Object>();</code> → Crea el mapa tipado de parámetros que se entregará a <code>JasperFillManager.fillReport</code> .
Línea 61	<code>parametros.put("usuario", "Ana Martínez");</code> → Asigna al parámetro JasperReports <code>usuario</code> el valor Java <code>"Ana Martínez"</code> antes del llenado.
Línea 62	<code>parametros.put("departamento", "Comercial");</code> → Asigna al parámetro JasperReports <code>departamento</code> el valor Java <code>"Comercial"</code> antes del llenado.
Línea 63	<code>parametros.put("periodo", "Septiembre 2026");</code> → Asigna al parámetro JasperReports <code>periodo</code> el valor Java <code>"Septiembre 2026"</code> antes del llenado.
Línea 64	<code>parametros.put("tipoIva", Double.valueOf(0.21d));</code> → Asigna al parámetro JasperReports <code>tipoIva</code> el valor Java <code>Double.valueOf(0.21d)</code> antes del llenado.
Línea 65	<code>parametros.put("mostrarDetalle", Boolean.TRUE);</code> → Asigna al parámetro JasperReports <code>mostrarDetalle</code> el valor Java <code>Boolean.TRUE</code> antes del llenado.
Línea 66	<code>parametros.put("categoria", null);</code> → Asigna al parámetro JasperReports <code>categoria</code> el valor Java <code>null</code> antes del llenado.
Línea 67	<code>parametros.put("precioMinimo", null);</code> → Asigna al parámetro JasperReports <code>precioMinimo</code> el valor Java <code>null</code> antes del llenado.
Línea 68	<code>parametros.put("precioMaximo", null);</code> → Asigna al parámetro JasperReports <code>precioMaximo</code> el valor Java <code>null</code> antes del llenado.
Línea 69	<code>parametros.put("umbralUnidades", Integer.valueOf(5));</code> → Asigna al parámetro JasperReports <code>umbralUnidades</code> el valor Java <code>Integer.valueOf(5)</code> antes del llenado.
Línea 70	<code>parametros.put("textoBusqueda", null);</code> → Asigna al parámetro JasperReports <code>textoBusqueda</code> el valor Java <code>null</code> antes del llenado.
Línea 71	<code>parametros.put("categoriasLista", Arrays.asList("Novela", "Realismo mágico", "Cuento", "Poesía"));</code> → Asigna al parámetro JasperReports <code>categoriasLista</code> el valor Java <code>Arrays.asList("Novela", "Realismo mágico", "Cuento", "Poesía")</code> antes del llenado.

Línea 72	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 73	<code>try (Connection conexion = DriverManager.getConnection(urlBD)) {</code> → Abre la conexión SQLite mediante <code>DriverManager</code> dentro de un <code>try-with-resources</code> , por lo que <code>conexion</code> se cierra automáticamente al terminar el bloque.
Línea 74	<code>JasperPrint documento = JasperFillManager.fillReport(rutaJasper, parametros, conexion);</code> → Inicia el llenado del informe y guarda en <code>documento</code> el <code>JasperPrint</code> paginado que devolverá <code>JasperReports</code> .
Línea 75	<code>exportarPdf(documento, rutaPdf, ConfiguracionExportacion.getConfiguracionPdf(</code> → Invoca la exportación PDF normal usando el <code>JasperPrint</code> ya llenado y la ruta principal de salida.
Línea 76	<code>"Informe de Ventas - EditorialReports", "Departamento Comercial"));</code> → Completa la llamada a la fábrica PDF pasando el título y el autor que deben aparecer en los metadatos.
Línea 77	<code>exportarPdfProtegido(documento, rutaPdfProtegido);</code> → Genera una segunda salida PDF cifrada para validar contraseñas y permisos sin alterar el PDF normal.
Línea 78	<code>exportarXlsx(documento, rutaXlsx, "Ventas");</code> → Exporta el <code>JasperPrint</code> de ventas a <code>informe_ventas.xlsx</code> usando el nombre de hoja <code>Ventas</code> .
Línea 79	<code>JRCsvDataSource catalogoDataSource = new JRCsvDataSource(new File("data/catalogo.csv"), "UTF-8");</code> → Abre <code>data/catalogo.csv</code> como <code>datasource</code> <code>JasperReports</code> en UTF-8 para llenar el informe de catálogo sin JDBC.
Línea 80	<code>try {</code> → Abre el bloque principal protegido: cualquier error de compilación, conexión, llenado o exportación será capturado por el <code>catch</code> final.
Línea 81	<code>catalogoDataSource.setFieldDelimiter(' ');</code> → Configura punto y coma como delimitador de campos CSV.
Línea 82	<code>catalogoDataSource.setUseFirstRowAsHeader(true);</code> → Indica que la primera fila del CSV contiene los nombres de los fields del informe de catálogo.
Línea 83	<code>JasperPrint documentoCatalogo = JasperFillManager.fillReport(</code> → Inicia el llenado del informe de catálogo y guarda el resultado paginado en <code>documentoCatalogo</code> .
Línea 84	<code>rutaCatalogoJasper, new HashMap<String, Object>(), catalogoDataSource);</code> → Completa <code>fillReport</code> pasando el catálogo compilado, un mapa de parámetros vacío y el <code>JRCsvDataSource</code> .
Línea 85	<code>exportarXlsx(documentoCatalogo, rutaXlsxCatalogo, "Catálogo");</code> → Exporta el <code>JasperPrint</code> del catálogo a <code>informe_catalogo.xlsx</code> usando la hoja <code>Catálogo</code> .
Línea 86	<code>} finally {</code> → Abre el bloque <code>finally</code> que se ejecutará siempre para liberar el <code>datasource</code> CSV aunque falle el llenado o la exportación.
Línea 87	<code>catalogoDataSource.close();</code> → Cierra explícitamente el <code>JRCsvDataSource</code> para liberar el lector del archivo de catálogo.
Línea 88	<code>}</code> → Cierra el bloque <code>try-with-resources</code> de la conexión JDBC.
Línea 89	<code>new File("output/images").mkdirs();</code> → Crea la carpeta física donde el handler HTML podrá escribir recursos de imagen.
Línea 90	<code>new File("output/styles").mkdirs();</code> → Crea la carpeta publicada de estilos que debe acompañar a <code>informe_ventas.html</code> .
Línea 91	<code>Files.copy(Paths.get("resources/styles/editorial.css"), Paths.get("output/styles/editorial.css"));</code> → Copia la hoja CSS fuente a la carpeta publicada junto al HTML, sustituyéndola si ya existe.
Línea 92	<code>StandardCopyOption.REPLACE_EXISTING);</code> → Finaliza la copia del CSS indicando que una versión anterior debe reemplazarse para mantener la salida sincronizada.
Línea 93	<code>exportarHtml(documento, rutaHtml);</code> → Reutiliza el mismo <code>JasperPrint</code> para generar la salida HTML y sus recursos.
Línea 94	<code>exportarCsv(documento, rutaCsv);</code> → Exporta el documento a CSV usando la configuración de delimitadores del checkpoint.

Línea 95	<code>exportarXml(documento, rutaXml);</code> → Serializa el JasperPrint a XML en la ruta documentada.
Línea 96	<code>exportarRtf(documento, rutaRtf);</code> → Exporta el documento a RTF para procesadores de texto.
Línea 97	<code>exportarOdt(documento, rutaOdt);</code> → Reutiliza el JasperPrint de ventas para generar el reto ODT en <code>output/informe_ventas.odt</code> .
Línea 98	<code>System.out.println("Informe PDF generado en: " + new File(rutaPdf).getAbsolutePath());</code> → Escribe en la consola la evidencia "Informe PDF generado en: " + <code>new File(rutaPdf).getAbsolutePath()</code> , que queda registrada por el workflow E2E.
Línea 99	<code>System.out.println("Informe PDF protegido generado en: " + new File(rutaPdfProtegido).getAbsolutePath());</code> → Escribe en la consola la evidencia "Informe PDF protegido generado en: " + <code>new File(rutaPdfProtegido).getAbsolutePath()</code> , que queda registrada por el workflow E2E.
Línea 100	<code>System.out.println("Informe Excel generado en: " + new File(rutaXlsx).getAbsolutePath());</code> → Escribe en la consola la evidencia "Informe Excel generado en: " + <code>new File(rutaXlsx).getAbsolutePath()</code> , que queda registrada por el workflow E2E.
Línea 101	<code>System.out.println("Informe catálogo Excel generado en: " + new File(rutaXlsxCatalogo).getAbsolutePath());</code> → Escribe en la consola la evidencia "Informe catálogo Excel generado en: " + <code>new File(rutaXlsxCatalogo).getAbsolutePath()</code> , que queda registrada por el workflow E2E.
Línea 102	<code>System.out.println("Informe HTML generado en: " + new File(rutaHtml).getAbsolutePath());</code> → Escribe en la consola la evidencia "Informe HTML generado en: " + <code>new File(rutaHtml).getAbsolutePath()</code> , que queda registrada por el workflow E2E.
Línea 103	<code>System.out.println("Informe CSV generado en: " + new File(rutaCsv).getAbsolutePath());</code> → Escribe en la consola la evidencia "Informe CSV generado en: " + <code>new File(rutaCsv).getAbsolutePath()</code> , que queda registrada por el workflow E2E.
Línea 104	<code>System.out.println("Informe XML generado en: " + new File(rutaXml).getAbsolutePath());</code> → Escribe en la consola la evidencia "Informe XML generado en: " + <code>new File(rutaXml).getAbsolutePath()</code> , que queda registrada por el workflow E2E.
Línea 105	<code>System.out.println("Informe RTF generado en: " + new File(rutaRtf).getAbsolutePath());</code> → Escribe en la consola la evidencia "Informe RTF generado en: " + <code>new File(rutaRtf).getAbsolutePath()</code> , que queda registrada por el workflow E2E.
Línea 106	<code>System.out.println("Informe ODT generado en: " + new File(rutaOdt).getAbsolutePath());</code> → Escribe en la consola la evidencia "Informe ODT generado en: " + <code>new File(rutaOdt).getAbsolutePath()</code> , que queda registrada por el workflow E2E.
Línea 107	<code>System.out.println("Paginas del documento: " + documento.getPages().size());</code> → Escribe en la consola la evidencia "Paginas del documento: " + <code>documento.getPages().size()</code> , que queda registrada por el workflow E2E.
Línea 108	<code>System.out.println("Parametro usuario: " + parametros.get("usuario"));</code> → Escribe en la consola la evidencia "Parametro usuario: " + <code>parametros.get("usuario")</code> , que queda registrada por el workflow E2E.
Línea 109	<code>System.out.println("M6 checkpoint generado correctamente");</code> → Escribe en la consola la evidencia "M6 checkpoint generado correctamente", que queda registrada por el workflow E2E.
Línea 110	<code>}</code> → Cierra el bloque try-with-resources de la conexión JDBC.
Línea 111	<code>} catch (Exception e) {</code> → Cierra el bloque protegido y abre el manejador que captura cualquier excepción del proceso completo.
Línea 112	<code>e.printStackTrace();</code> → Imprime la traza completa de la excepción para que el fallo sea diagnosticable en local y en GitHub Actions.
Línea 113	<code>System.exit(1);</code> → Finaliza el proceso con código 1 para que CI marque la ejecución como fallida y no oculte el error.
Línea 114	<code>}</code> → Cierra el bloque catch o el bloque principal de control asociado a main.

Línea 115	} → Cierra el método main.
Línea 116	`` → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 117	private static void exportarPdf(JasperPrint documento, String ruta, SimplePdfExporterConfiguration configuracion) throws Exception { → Declara el helper privado exportarPdf, que encapsula una exportación concreta y propaga cualquier error al catch principal.
Línea 118	JRPdfExporter exportador = new JRPdfExporter(); → Crea el exportador PDF avanzado que admite configuración documental y de seguridad.
Línea 119	exportador.setConfiguration(configuracion); → Asocia al exportador la configuración específica preparada en las líneas anteriores.
Línea 120	exportador.setExporterInput(new SimpleExporterInput(documento)); → Entrega al exportador el JasperPrint documento ya llenado; no se vuelve a consultar la base de datos.
Línea 121	exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta)); → Dirige la salida binaria del exportador al archivo indicado por el parámetro ruta.
Línea 122	exportador.exportReport(); → Ejecuta la exportación con la entrada, salida y configuración ya asignadas; aquí se materializa el archivo.
Línea 123	} → Cierra el método main.
Línea 124	`` → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 125	private static void exportarPdfProtegido(JasperPrint documento, String ruta) throws Exception { → Declara el helper privado exportarPdfProtegido, que encapsula una exportación concreta y propaga cualquier error al catch principal.
Línea 126	JRPdfExporter exportador = new JRPdfExporter(); → Crea el exportador PDF avanzado que admite configuración documental y de seguridad.
Línea 127	SimplePdfExporterConfiguration configuracion = new SimplePdfExporterConfiguration(); → Crea el objeto SimplePdfExporterConfiguration que recibirá las opciones específicas de esta exportación.
Línea 128	configuracion.setMetadataTitle("Informe de Ventas Protegido - EditorialReports"); → Fija el título de los metadatos PDF con la API específica de SimplePdfExporterConfiguration.
Línea 129	configuracion.setEncrypted(Boolean.TRUE); → Activa el cifrado de la salida PDF protegida.
Línea 130	configuracion.setUserPassword("editorial2026"); → Configura la contraseña de apertura que el E2E verifica con pdfinfo -upw.
Línea 131	configuracion.setOwnerPassword("editorial-admin"); → Configura la contraseña de propietario del PDF protegido.
Línea 132	configuracion.setAllowedPermissionsHint("PRINTING COPY SCREENREADERS"); → Declara los permisos PDF autorizados mediante la cadena de hints admitida por JasperReports.
Línea 133	exportador.setConfiguration(configuracion); → Asocia al exportador la configuración específica preparada en las líneas anteriores.
Línea 134	exportador.setExporterInput(new SimpleExporterInput(documento)); → Entrega al exportador el JasperPrint documento ya llenado; no se vuelve a consultar la base de datos.
Línea 135	exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta)); → Dirige la salida binaria del exportador al archivo indicado por el parámetro ruta.
Línea 136	exportador.exportReport(); → Ejecuta la exportación con la entrada, salida y configuración ya asignadas; aquí se materializa el archivo.
Línea 137	} → Cierra el método main.
Línea 138	`` → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 139	private static void exportarXlsx(JasperPrint documento, String ruta, String nombreHoja) throws Exception { → Declara el helper privado exportarXlsx, que encapsula una exportación concreta y propaga cualquier error al catch principal.

Línea 140	<code>JRXLsxExporter exportador = new JRXLsxExporter();</code> → Crea el exportador OOXML que escribirá el libro XLSX.
Línea 141	<code>exportador.setConfiguration(ConfiguracionExportacion.getConfiguracionXlsxReport(nombreHoja));</code> → Obtiene la configuración XLSX dependiente del informe y del nombre de hoja.
Línea 142	<code>exportador.setConfiguration(ConfiguracionExportacion.getConfiguracionXlsxExportador());</code> → Obtiene la configuración global del exportador XLSX.
Línea 143	<code>exportador.setExporterInput(new SimpleExporterInput(documento));</code> → Entrega al exportador el JasperPrint documento ya llenado; no se vuelve a consultar la base de datos.
Línea 144	<code>exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta));</code> → Dirige la salida binaria del exportador al archivo indicado por el parámetro ruta.
Línea 145	<code>exportador.exportReport();</code> → Ejecuta la exportación con la entrada, salida y configuración ya asignadas; aquí se materializa el archivo.
Línea 146	<code>}</code> → Cierra el método main.
Línea 147	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 148	<code>private static void exportarHtml(JasperPrint documento, String ruta) throws Exception {</code> → Declara el helper privado <code>exportarHtml</code> , que encapsula una exportación concreta y propaga cualquier error al catch principal.
Línea 149	<code>HtmlExporter exportador = new HtmlExporter();</code> → Crea el exportador HTML vigente en JasperReports 6.20.0.
Línea 150	<code>SimpleHtmlExporterOutput salida = new SimpleHtmlExporterOutput(ruta, "UTF-8");</code> → Crea la salida HTML con codificación UTF-8.
Línea 151	<code>salida.setImageHandler(new FileHtmlResourceHandler(new File("output/images"), "images/{0}"));</code> → Asocia un gestor de recursos para escribir imágenes en disco y generar sus URI relativas.
Línea 152	<code>exportador.setConfiguration(ConfiguracionExportacion.getConfiguracionHtml("Informe de Ventas - EditorialReports"));</code> → Obtiene la cabecera, pie y separador HTML centralizados.
Línea 153	<code>exportador.setExporterInput(new SimpleExporterInput(documento));</code> → Entrega al exportador el JasperPrint documento ya llenado; no se vuelve a consultar la base de datos.
Línea 154	<code>exportador.setExporterOutput(salida);</code> → Conecta al exportador el objeto salida previamente configurado con codificación y/o recursos.
Línea 155	<code>exportador.exportReport();</code> → Ejecuta la exportación con la entrada, salida y configuración ya asignadas; aquí se materializa el archivo.
Línea 156	<code>}</code> → Cierra el método main.
Línea 157	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 158	<code>private static void exportarCsv(JasperPrint documento, String ruta) throws Exception {</code> → Declara el helper privado <code>exportarCsv</code> , que encapsula una exportación concreta y propaga cualquier error al catch principal.
Línea 159	<code>JRCsvExporter exportador = new JRCsvExporter();</code> → Crea el exportador CSV orientado a texto delimitado.
Línea 160	<code>exportador.setConfiguration(ConfiguracionExportacion.getConfiguracionCsv());</code> → Obtiene delimitadores y BOM CSV desde la clase de configuración central.
Línea 161	<code>exportador.setExporterInput(new SimpleExporterInput(documento));</code> → Entrega al exportador el JasperPrint documento ya llenado; no se vuelve a consultar la base de datos.
Línea 162	<code>exportador.setExporterOutput(new SimpleWriterExporterOutput(ruta, "UTF-8"));</code> → Crea una salida textual con UTF-8 para el formato correspondiente.
Línea 163	<code>exportador.exportReport();</code> → Ejecuta la exportación con la entrada, salida y configuración ya asignadas; aquí se materializa el archivo.

Línea 164	} → Cierra el método main.
Línea 165	`` → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 166	private static void exportarXml(JasperPrint documento, String ruta) throws Exception { → Declara el helper privado exportarXml, que encapsula una exportación concreta y propaga cualquier error al catch principal.
Línea 167	JRXmlExporter exportador = new JRXmlExporter(); → Crea el exportador que serializa el JasperPrint a XML.
Línea 168	SimpleXmlExporterOutput salida = new SimpleXmlExporterOutput(ruta, "UTF-8"); → Crea la salida XML con codificación UTF-8.
Línea 169	salida.setEmbeddingImages(Boolean.TRUE); → Solicita que los recursos gráficos de la salida XML queden embebidos.
Línea 170	exportador.setExporterInput(new SimpleExporterInput(documento)); → Entrega al exportador el JasperPrint documento ya llenado; no se vuelve a consultar la base de datos.
Línea 171	exportador.setExporterOutput(salida); → Conecta al exportador el objeto salida previamente configurado con codificación y/o recursos.
Línea 172	exportador.exportReport(); → Ejecuta la exportación con la entrada, salida y configuración ya asignadas; aquí se materializa el archivo.
Línea 173	} → Cierra el método main.
Línea 174	`` → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 175	private static void exportarRtf(JasperPrint documento, String ruta) throws Exception { → Declara el helper privado exportarRtf, que encapsula una exportación concreta y propaga cualquier error al catch principal.
Línea 176	JRRtfExporter exportador = new JRRtfExporter(); → Crea el exportador RTF.
Línea 177	exportador.setConfiguration(ConfiguracionExportacion.getConfiguracionRtf()); → Obtiene la configuración RTF centralizada antes de escribir la salida textual.
Línea 178	exportador.setExporterInput(new SimpleExporterInput(documento)); → Entrega al exportador el JasperPrint documento ya llenado; no se vuelve a consultar la base de datos.
Línea 179	exportador.setExporterOutput(new SimpleWriterExporterOutput(ruta, "UTF-8")); → Crea una salida textual con UTF-8 para el formato correspondiente.
Línea 180	exportador.exportReport(); → Ejecuta la exportación con la entrada, salida y configuración ya asignadas; aquí se materializa el archivo.
Línea 181	} → Cierra el método main.
Línea 182	`` → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 183	private static void exportarOdt(JasperPrint documento, String ruta) throws Exception { → Declara el helper privado exportarOdt, que encapsula una exportación concreta y propaga cualquier error al catch principal.
Línea 184	JROdtExporter exportador = new JROdtExporter(); → Crea JROdtExporter del paquete Oasis para producir un OpenDocument Text real.
Línea 185	exportador.setExporterInput(new SimpleExporterInput(documento)); → Entrega al exportador el JasperPrint documento ya llenado; no se vuelve a consultar la base de datos.
Línea 186	exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(ruta)); → Dirige la salida binaria del exportador al archivo indicado por el parámetro ruta.
Línea 187	exportador.exportReport(); → Ejecuta la exportación con la entrada, salida y configuración ya asignadas; aquí se materializa el archivo.
Línea 188	} → Cierra el método main.

Línea 189 `` → Separa visualmente dos bloques lógicos sin modificar la ejecución.

Línea 190 } → Cierra la clase GeneradorInformeVentas.

pom.xml

XML

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>es.jaimegallo.editorialreports</groupId>
    <artifactId>editorial-reports-m3</artifactId>
    <version>1.0-SNAPSHOT</version>
    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <maven.compiler.source>8</maven.compiler.source>
        <maven.compiler.target>8</maven.compiler.target>
    </properties>
    <repositories>
        <repository><id>jaspersoft-third-party</id><url>https://jaspersoft.jfrog.io/jaspersoft/third-party-ce-artifacts</url></repository>
        <repository><id>jr-ce-releases</id><url>https://jaspersoft.jfrog.io/jaspersoft/jr-ce-releases</url></repository>
    </repositories>
    <dependencies>
        <dependency><groupId>net.sf.jasperreports</groupId><artifactId>jasperreports</artifactId><version>6.20.0</version></dependency>
        <dependency><groupId>net.sf.jasperreports</groupId><artifactId>jasperreports-fonts</artifactId><version>6.20.0</version></dependency>
        <dependency><groupId>xalan</groupId><artifactId>xalan</artifactId><version>2.7.2</version></dependency>
        <dependency><groupId>org.xerial</groupId><artifactId>sqlite-jdbc</artifactId><version>3.44.0.0</version></dependency>
        <dependency><groupId>org.apache.poi</groupId><artifactId>poi</artifactId><version>5.1.0</version></dependency>
        <dependency><groupId>org.apache.poi</groupId><artifactId>poi-ooxml</artifactId><version>5.1.0</version></dependency>
        <dependency><groupId>org.slf4j</groupId><artifactId>slf4j-simple</artifactId><version>1.7.36</version></dependency>
    </dependencies>
    <build>
        <sourceDirectory>src</sourceDirectory>
        <resources>
            <resource>
                <directory>src</directory>
                <excludes><exclude>/**/*.java</exclude></excludes>
            </resource>
        </resources>
        <plugins>
            <plugin><groupId>org.apache.maven.plugins</groupId><artifactId>maven-compiler-plugin</artifactId><version>3.11.0</version></plugin>
            <plugin><groupId>org.apache.maven.plugins</groupId><artifactId>maven-dependency-plugin</artifactId><version>3.6.1</version></plugin>
        </plugins>
    </build>
</project>
```

Explicación línea por línea

Línea 1	<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" → Abre el documento Maven project y declara los namespaces del modelo POM.
Línea 2	xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd" → Relaciona el namespace de JasperReports con su XSD para que Studio y el compilador validen la estructura.
Línea 3	<modelVersion>4.0.0</modelVersion> → Declara la versión 4.0.0 del modelo de proyecto Maven.
Línea 4	<groupId>es.jaimegallo.editorialreports</groupId> → Identifica el grupo Maven del proyecto EditorialReports.
Línea 5	<artifactId>editorial-reports-m3</artifactId> → Define el identificador del artefacto Maven que se compila y empaqueta.
Línea 6	<version>1.0-SNAPSHOT</version> → Fija la versión del artefacto Maven.
Línea 7	<properties> → Abre el bloque de propiedades Maven usado para codificación y nivel del compilador Java.
Línea 8	<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding> → Fija UTF-8 como codificación fuente del proyecto Maven.
Línea 9	<maven.compiler.source>8</maven.compiler.source> → Fija Java 8 como nivel de lenguaje de compilación.
Línea 10	<maven.compiler.target>8</maven.compiler.target> → Fija Java 8 como bytecode objetivo.

Línea 11	<code></properties></code> → Cierra <code>properties</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 12	<code><repositories></code> → Abre la lista de repositorios adicionales desde los que Maven puede resolver dependencias.
Línea 13	<code><repository><id>jaspersoft-third-party</id><url>https://jaspersoft.jfrog.io/jaspersoft/third-party-ce-artifacts/</url></repository></code> → Composición de la línea: encadena además <code><repository></code> , <code><id></code> , <code><url></code> dentro de la misma jerarquía.
Línea 14	<code><repository><id>jr-ce-releases</id><url>https://jaspersoft.jfrog.io/jaspersoft/jr-ce-releases/</url></repository></code> → Composición de la línea: encadena además <code><repository></code> , <code><id></code> , <code><url></code> dentro de la misma jerarquía.
Línea 15	<code></repositories></code> → Cierra <code>repositories</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 16	<code><dependencies></code> → Abre la colección de dependencias runtime/compilación, incluida JasperReports y POI.
Línea 17	<code><dependency><groupId>net.sf.jasperreports</groupId><artifactId>jasperreports</artifactId><version>6.20.0</version></dependency></code> → Composición de la línea: encadena además <code><dependency></code> , <code><groupId></code> , <code><artifactId></code> , <code><version></code> dentro de la misma jerarquía.
Línea 18	<code><dependency><groupId>net.sf.jasperreports</groupId><artifactId>jasperreports-fonts</artifactId><version>6.20.0</version></dependency></code> → Composición de la línea: encadena además <code><dependency></code> , <code><groupId></code> , <code><artifactId></code> , <code><version></code> dentro de la misma jerarquía.
Línea 19	<code><dependency><groupId>xalan</groupId><artifactId>xalan</artifactId><version>2.7.2</version></dependency></code> → Composición de la línea: encadena además <code><dependency></code> , <code><groupId></code> , <code><artifactId></code> , <code><version></code> dentro de la misma jerarquía.
Línea 20	<code><dependency><groupId>org.xerial</groupId><artifactId>sqlite-jdbc</artifactId><version>3.44.0.0</version></dependency></code> → Composición de la línea: encadena además <code><dependency></code> , <code><groupId></code> , <code><artifactId></code> , <code><version></code> dentro de la misma jerarquía.
Línea 21	<code><dependency><groupId>org.apache.poi</groupId><artifactId>poi</artifactId><version>5.1.0</version></dependency></code> → Composición de la línea: encadena además <code><dependency></code> , <code><groupId></code> , <code><artifactId></code> , <code><version></code> dentro de la misma jerarquía.
Línea 22	<code><dependency><groupId>org.apache.poi</groupId><artifactId>poi-ooxml</artifactId><version>5.1.0</version></dependency></code> → Composición de la línea: encadena además <code><dependency></code> , <code><groupId></code> , <code><artifactId></code> , <code><version></code> dentro de la misma jerarquía.
Línea 23	<code><dependency><groupId>org.slf4j</groupId><artifactId>slf4j-simple</artifactId><version>1.7.36</version></dependency></code> → Composición de la línea: encadena además <code><dependency></code> , <code><groupId></code> , <code><artifactId></code> , <code><version></code> dentro de la misma jerarquía.
Línea 24	<code></dependencies></code> → Cierra <code>dependencies</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 25	<code><build></code> → Abre la configuración de construcción Maven.
Línea 26	<code><sourceDirectory>src</sourceDirectory></code> → Indica que las clases Java fuente del proyecto están directamente en la carpeta <code>src</code> .
Línea 27	<code><resources></code> → Abre la configuración de recursos que Maven copiará al classpath.
Línea 28	<code><resource></code> → Declara una fuente de recursos adicional para el empaquetado.
Línea 29	<code><directory>src</directory></code> → Usa <code>src</code> como origen de recursos para incluir <code>jasperreports.properties</code> en <code>target/classes</code> .
Línea 30	<code><excludes><exclude>/**/*.java</exclude></excludes></code> → Composición de la línea: encadena además <code><excludes></code> , <code><exclude></code> dentro de la misma jerarquía.
Línea 31	<code></resource></code> → Cierra <code>resource</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 32	<code></resources></code> → Cierra <code>resources</code> y vuelve al elemento padre de la jerarquía JRXML.
Línea 33	<code><plugins></code> → Abre la lista de plugins Maven usados durante la construcción.

Línea 34	<code><plugin><groupId>org.apache.maven.plugins</groupId><artifactId>maven-compiler-plugin</artifactId><version>3.11.0</version></plugin></code> → Composición de la línea: encadena además <code><plugin></code> , <code><groupId></code> , <code><artifactId></code> , <code><version></code> dentro de la misma jerarquía.
Línea 35	<code><plugin><groupId>org.apache.maven.plugins</groupId><artifactId>maven-dependency-plugin</artifactId><version>3.6.1</version></plugin></code> → Composición de la línea: encadena además <code><plugin></code> , <code><groupId></code> , <code><artifactId></code> , <code><version></code> dentro de la misma jerarquía.
Línea 36	<code></plugins></code> → Cierra plugins y vuelve al elemento padre de la jerarquía JRXML.
Línea 37	<code></build></code> → Cierra build y vuelve al elemento padre de la jerarquía JRXML.
Línea 38	<code></project></code> → Cierra project y vuelve al elemento padre de la jerarquía JRXML.

ConfiguracionExportacion.java

JAVA

```
import net.sf.jasperreports.export.SimpleCsvExporterConfiguration;
import net.sf.jasperreports.export.SimpleHtmlExporterConfiguration;
import net.sf.jasperreports.export.SimplePdfExporterConfiguration;
import net.sf.jasperreports.export.SimpleRtfExporterConfiguration;
import net.sf.jasperreports.export.SimpleXlsxExporterConfiguration;
import net.sf.jasperreports.export.SimpleXlsxReportConfiguration;

public class ConfiguracionExportacion {
    public static SimplePdfExporterConfiguration getConfiguracionPdf(String titulo, String autor) {
        SimplePdfExporterConfiguration c = new SimplePdfExporterConfiguration();
        c.setMetadataTitle(titulo);
        c.setMetadataAuthor(autor);
        c.setMetadataCreator("JasperReports 6.20.0");
        c.setDisplayMetadataTitle(Boolean.TRUE);
        c.setCompressed(Boolean.TRUE);
        return c;
    }

    public static SimpleXlsxReportConfiguration getConfiguracionXlsxReport(String nombreHoja) {
        SimpleXlsxReportConfiguration c = new SimpleXlsxReportConfiguration();
        c.setSheetNames(new String[]{nombreHoja});
        c.setShowGridLines(Boolean.FALSE);
        c.setCellLocked(Boolean.FALSE);
        c.setCellHidden(Boolean.FALSE);
        c.setDetectCellType(Boolean.TRUE);
        c.setOnePagePerSheet(Boolean.FALSE);
        return c;
    }

    public static SimpleXlsxExporterConfiguration getConfiguracionXlsxExportador() {
        SimpleXlsxExporterConfiguration c = new SimpleXlsxExporterConfiguration();
        c.setCreateCustomPalette(Boolean.TRUE);
        return c;
    }

    public static SimpleHtmlExporterConfiguration getConfiguracionHtml(String titulo) {
        SimpleHtmlExporterConfiguration c = new SimpleHtmlExporterConfiguration();
        c.setHtmlHeader("<html><head><meta charset='UTF-8'><title>" + titulo
            + "</title><link rel='stylesheet' href='styles/editorial.css'></head><body>"
            + "<a class='enlace-pdf' href='informe_ventas.pdf'>Descargar PDF</a>");
        c.setHtmlFooter("</body></html>");
        c.setBetweenPagesHtml("<hr class='salto-pagina' />");
        return c;
    }

    public static SimpleCsvExporterConfiguration getConfiguracionCsv() {
        SimpleCsvExporterConfiguration c = new SimpleCsvExporterConfiguration();
        c.setFieldDelimiter(";");
        c.setRecordDelimiter("\n");
        c.setWriteBOM(Boolean.TRUE);
        return c;
    }

    public static SimpleRtfExporterConfiguration getConfiguracionRtf() {
        return new SimpleRtfExporterConfiguration();
    }
}
```

Explicación línea por línea

Línea 1	<code>import net.sf.jasperreports.export.SimpleCsvExporterConfiguration;</code> → Importa <code>net.sf.jasperreports.export.SimpleCsvExporterConfiguration</code> para usar la clase <code>SimpleCsvExporterConfiguration</code> en el generador.
----------------	---

Línea 2	<code>import net.sf.jasperreports.export.SimpleHtmlExporterConfiguration;</code> → Importa <code>net.sf.jasperreports.export.SimpleHtmlExporterConfiguration</code> para usar la clase <code>SimpleHtmlExporterConfiguration</code> en el generador.
Línea 3	<code>import net.sf.jasperreports.export.SimplePdfExporterConfiguration;</code> → Importa <code>net.sf.jasperreports.export.SimplePdfExporterConfiguration</code> para usar la clase <code>SimplePdfExporterConfiguration</code> en el generador.
Línea 4	<code>import net.sf.jasperreports.export.SimpleRtfExporterConfiguration;</code> → Importa <code>net.sf.jasperreports.export.SimpleRtfExporterConfiguration</code> para usar la clase <code>SimpleRtfExporterConfiguration</code> en el generador.
Línea 5	<code>import net.sf.jasperreports.export.SimpleXlsxExporterConfiguration;</code> → Importa <code>net.sf.jasperreports.export.SimpleXlsxExporterConfiguration</code> para usar la clase <code>SimpleXlsxExporterConfiguration</code> en el generador.
Línea 6	<code>import net.sf.jasperreports.export.SimpleXlsxReportConfiguration;</code> → Importa <code>net.sf.jasperreports.export.SimpleXlsxReportConfiguration</code> para usar la clase <code>SimpleXlsxReportConfiguration</code> en el generador.
Línea 7	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 8	<code>public class ConfiguracionExportacion {</code> → Declara la clase ejecutable <code>ConfiguracionExportacion</code> que encapsula el generador del informe.
Línea 9	<code>public static SimplePdfExporterConfiguration getConfiguracionPdf(String titulo, String autor)</code> <code>{</code> → Declara el método fábrica <code>getConfiguracionPdf</code> que construye y devuelve una configuración reutilizable para el formato correspondiente.
Línea 10	<code>SimplePdfExporterConfiguration c = new SimplePdfExporterConfiguration();</code> → Crea el objeto <code>SimplePdfExporterConfiguration</code> que recibirá las opciones específicas de esta exportación.
Línea 11	<code>c.setMetadataTitle(titulo);</code> → Fija el título de los metadatos PDF con la API específica de <code>SimplePdfExporterConfiguration</code> .
Línea 12	<code>c.setMetadataAuthor(autor);</code> → Fija el autor en los metadatos del PDF.
Línea 13	<code>c.setMetadataCreator("JasperReports 6.20.0");</code> → Registra JasperReports 6.20.0 como creador del PDF.
Línea 14	<code>c.setDisplayMetadataTitle(Boolean.TRUE);</code> → Solicita a los lectores PDF que utilicen el título de metadatos cuando soporten esa preferencia.
Línea 15	<code>c.setCompressed(Boolean.TRUE);</code> → Activa la compresión del PDF mediante la configuración del exportador.
Línea 16	<code>return c;</code> → Devuelve al llamador la configuración ya preparada por el método fábrica.
Línea 17	<code>}</code> → Cierra el método <code>main</code> .
Línea 18	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 19	<code>public static SimpleXlsxReportConfiguration getConfiguracionXlsxReport(String nombreHoja) {</code> → Declara el método fábrica <code>getConfiguracionXlsxReport</code> que construye y devuelve una configuración reutilizable para el formato correspondiente.
Línea 20	<code>SimpleXlsxReportConfiguration c = new SimpleXlsxReportConfiguration();</code> → Crea la configuración de cómo el JasperPrint se distribuye en hojas y celdas XLSX.
Línea 21	<code>c.setSheetNames(new String[] {nombreHoja});</code> → Asigna el nombre Ventas a la hoja; el E2E lo comprueba dentro de <code>xl/workbook.xml</code> .
Línea 22	<code>c.setShowGridLines(Boolean.FALSE);</code> → Desactiva la cuadrícula predeterminada de la hoja Excel.
Línea 23	<code>c.setCellLocked(Boolean.FALSE);</code> → Configura las celdas exportadas sin bloqueo adicional.
Línea 24	<code>c.setCellHidden(Boolean.FALSE);</code> → Evita marcar como ocultas las celdas exportadas.
Línea 25	<code>c.setDetectCellType(Boolean.TRUE);</code> → Pide al exportador detectar tipos numéricos/fecha en lugar de convertir indiscriminadamente a texto.

Línea 26	<code>c.setOnePagePerSheet(Boolean.FALSE);</code> → Mantiene el informe en una misma hoja lógica en lugar de crear una hoja por página.
Línea 27	<code>return c;</code> → Devuelve al llamador la configuración ya preparada por el método fábrica.
Línea 28	<code>}</code> → Cierra el método main.
Línea 29	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 30	<code>public static SimpleXlsxExporterConfiguration getConfiguracionXlsxExportador() {</code> → Declara el método fábrica <code>getConfiguracionXlsxExportador</code> que construye y devuelve una configuración reutilizable para el formato correspondiente.
Línea 31	<code>SimpleXlsxExporterConfiguration c = new SimpleXlsxExporterConfiguration();</code> → Crea la configuración propia del libro/exportador XLSX.
Línea 32	<code>c.setCreateCustomPalette(Boolean.TRUE);</code> → Activa la paleta personalizada del exportador XLSX para reproducir mejor los colores.
Línea 33	<code>return c;</code> → Devuelve al llamador la configuración ya preparada por el método fábrica.
Línea 34	<code>}</code> → Cierra el método main.
Línea 35	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 36	<code>public static SimpleHtmlExporterConfiguration getConfiguracionHtml(String titulo) {</code> → Declara el método fábrica <code>getConfiguracionHtml</code> que construye y devuelve una configuración reutilizable para el formato correspondiente.
Línea 37	<code>SimpleHtmlExporterConfiguration c = new SimpleHtmlExporterConfiguration();</code> → Crea el objeto <code>SimpleHtmlExporterConfiguration</code> que recibirá las opciones específicas de esta exportación.
Línea 38	<code>c.setHtmlHeader("<html><head><meta charset='UTF-8'><title>" + titulo</code> → Define la cabecera HTML, incluyendo UTF-8, título y enlace a la hoja CSS externa.
Línea 39	<code>+ "</title><link rel='stylesheet' href='styles/editorial.css'></head><body>"</code> → Continúa la expresión Java anterior concatenando o añadiendo el fragmento <code>"</title><link rel='stylesheet' href='styles/editorial.css'></head><body>"</code> .
Línea 40	<code>+ "Descargar PDF");</code> → Añade el enlace del reto Descargar PDF, apuntando al PDF generado en la misma carpeta output.
Línea 41	<code>c.setHtmlFooter("</body></html>");</code> → Define el cierre de body y html del documento exportado.
Línea 42	<code>c.setBetweenPagesHtml("<hr class='salto-pagina' />");</code> → Inserta el separador HTML que representa el cambio entre páginas del JasperPrint.
Línea 43	<code>return c;</code> → Devuelve al llamador la configuración ya preparada por el método fábrica.
Línea 44	<code>}</code> → Cierra el método main.
Línea 45	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 46	<code>public static SimpleCsvExporterConfiguration getConfiguracionCsv() {</code> → Declara el método fábrica <code>getConfiguracionCsv</code> que construye y devuelve una configuración reutilizable para el formato correspondiente.
Línea 47	<code>SimpleCsvExporterConfiguration c = new SimpleCsvExporterConfiguration();</code> → Crea el objeto <code>SimpleCsvExporterConfiguration</code> que recibirá las opciones específicas de esta exportación.
Línea 48	<code>c.setFieldDelimiter(";");</code> → Configura punto y coma como delimitador de campos CSV.
Línea 49	<code>c.setRecordDelimiter("\n");</code> → Configura el salto de línea como delimitador de registros CSV.
Línea 50	<code>c.setWriteBOM(Boolean.TRUE);</code> → Activa el BOM UTF-8 para facilitar la detección de codificación en aplicaciones de escritorio.
Línea 51	<code>return c;</code> → Devuelve al llamador la configuración ya preparada por el método fábrica.
Línea 52	<code>}</code> → Cierra el método main.

Línea 53	<code>``</code> → Separa visualmente dos bloques lógicos sin modificar la ejecución.
Línea 54	<code>public static SimpleRtfExporterConfiguration getConfiguracionRtf() {</code> → Declara el método fábrica <code>getConfiguracionRtf</code> que construye y devuelve una configuración reutilizable para el formato correspondiente.
Línea 55	<code>return new SimpleRtfExporterConfiguration();</code> → Devuelve una configuración RTF nueva; la codificación seguirá definiéndose correctamente en el writer de salida.
Línea 56	<code>}</code> → Cierra el método <code>main</code> .
Línea 57	<code>}</code> → Cierra la clase <code>GeneradorInformeVentas</code> .

jasperreports.properties

PROPERTIES

```
net.sf.jasperreports.export.pdf.compressed=true
net.sf.jasperreports.export.csv.field.delimiter=;
```

Explicación línea por línea

Línea 1	<code>net.sf.jasperreports.export.pdf.compressed=true</code> → Define la propiedad global <code>net.sf.jasperreports.export.pdf.compressed</code> con valor <code>true</code> .
Línea 2	<code>net.sf.jasperreports.export.csv.field.delimiter=;</code> → Define la propiedad global <code>net.sf.jasperreports.export.csv.field.delimiter</code> con valor <code>;</code> .

Parte D — Simulación del resultado y de la estructura del proyecto

D.1 — Flujo de exportación / diseño

TEXT

```
GeneradorInformeVentas
├── PDF -> ConfiguracionExportacion.getConfiguracionPdf
├── XLSX -> getConfiguracionXlsxReport + getConfiguracionXlsxExportador
├── HTML -> getConfiguracionHtml
├── CSV -> getConfiguracionCsv
├── RTF -> getConfiguracionRtf
└── ODT -> exportador directo
```

Qué representa: la transformación funcional que debe existir al terminar 6.5.

Cómo verificarlo: comparar el flujo con la Parte C y ejecutar el generador; el JRXML/JRTX debe seguir siendo el heredado de M5/5.6.

D.2 — Estructura lógica en código y recursos

TEXT

```
EditorialReportsJava/src/
├── GeneradorInformeVentas.java
├── ConfiguracionExportacion.java
└── jasperreports.properties

target/classes/
└── jasperreports.properties
```

Qué representa: las clases, métodos y recursos que sustituyen en M6 al trabajo visual sobre bandas y componentes.

Cómo verificarlo: abrir Java/POM/CSS/properties según corresponda y contrastar nombres y tipos con la Parte C ejecutable.

D.3 — Archivos de salida

TEXT

```
output/
├── informe_ventas.pdf
├── informe_ventas_protegido.pdf
├── informe_ventas.xlsx
├── informe_catalogo.xlsx
├── informe_ventas.html
├── informe_ventas.csv
└── informe_ventas.xml
```

```
└─ informe_ventas.rtf
└─ informe_ventas.odt
```

Qué representa: los artefactos acumulativos esperados en output.

Cómo verificarlo: ejecutar `GeneradorInformeVentas`, abrir cada formato con una herramienta compatible y contrastar los contratos automatizados del E2E.

D.4 — Árbol acumulativo del checkpoint

TEXT

```
M6/6.5/
└─ EditorialReports/CONFIGURACION_EXPORTACION.md
└─ EditorialReportsJava/pom.xml
└─ EditorialReportsJava/src/
    └─ GeneradorInformeVentas.java
    └─ ConfiguracionExportacion.java
    └─ jasperreports.properties
```

Evidencia E2E: run **36249131955**, commit **12a0eba90859a92b12578d59ae592ad17dac5fb6**, artifact runtime **10908427540**.

Invariantes: 14 libros, 9 ventas, 31 unidades, 633,40 € y 6 páginas en `informe_ventas`.

Errores comunes del ejercicio completo

Error	Causa	Solución
ConfiguracionExportacion no se encuentra	Clase fuera de src o compilación incompleta	Mantenerla en EditorialReportsJava/src y ejecutar Maven
El properties no se lee	No llega al classpath	Configurar <code><resources></code> para copiar no-Java desde src
XLSX deja de usar la hoja correcta	Se mezclan tipos de configuración	Mantener métodos Report/Exporter separados
HTML pierde Descargar PDF	La fábrica no conserva la cabecera de 6.3	Incluir el enlace en <code>getConfiguracionHtml</code>
RTF intenta usar <code>setEncoding</code> en configuración	API incorrecta	Mantener UTF-8 en <code>SimpleWriterExporterOutput</code>
Una salida desaparece tras refactorizar	Se eliminó una llamada acumulada	Ejecutar E2E completo 6.5

Reto resuelto paso a paso

Enunciado original: añadir `getConfiguracionRtf()` y utilizarlo desde el generador.

1. `ConfiguracionExportacion` importa `SimpleRtfExporterConfiguration`.
2. Se declara `getConfiguracionRtf()`.
3. El método devuelve una nueva configuración RTF.
4. `GeneradorInformeVentas.exportarRtf` la aplica con `setConfiguration`.
5. UTF-8 se mantiene en `SimpleWriterExporterOutput`, no en la configuración.
6. Maven compila la clase y el E2E vuelve a validar la cabecera RTF.

Resultado del reto: RTF queda integrado en el patrón de configuración central sin usar un método inexistente de codificación.

Analogía final con el contexto de la editorial

`jasperreports.properties` es el manual general de imprenta; `ConfiguracionExportacion` son las fichas técnicas por formato y `GeneradorInformeVentas` es el operario que aplica esas fichas a cada tirada.

Resultado esperado

- ConfiguracionExportacion.java con PDF, XLSX Report, XLSX Exporter, HTML, CSV y RTF.
- jasperreports.properties disponible en target/classes.
- GeneradorInformeVentas usando la configuración central.
- todos los retos de 6.1–6.4 conservados.
- PDF normal/protegido, dos XLSX, HTML, CSV, XML, RTF y ODT generados.
- JRXML/JRTX idénticos a M5/5.6.

Conclusión del Módulo 6

El Módulo 6 cierra la capa de distribución de EditorialReports. Un único flujo de llenado produce múltiples formatos reales y validados, y las configuraciones comunes quedan centralizadas sin romper el diseño ni los invariantes heredados.